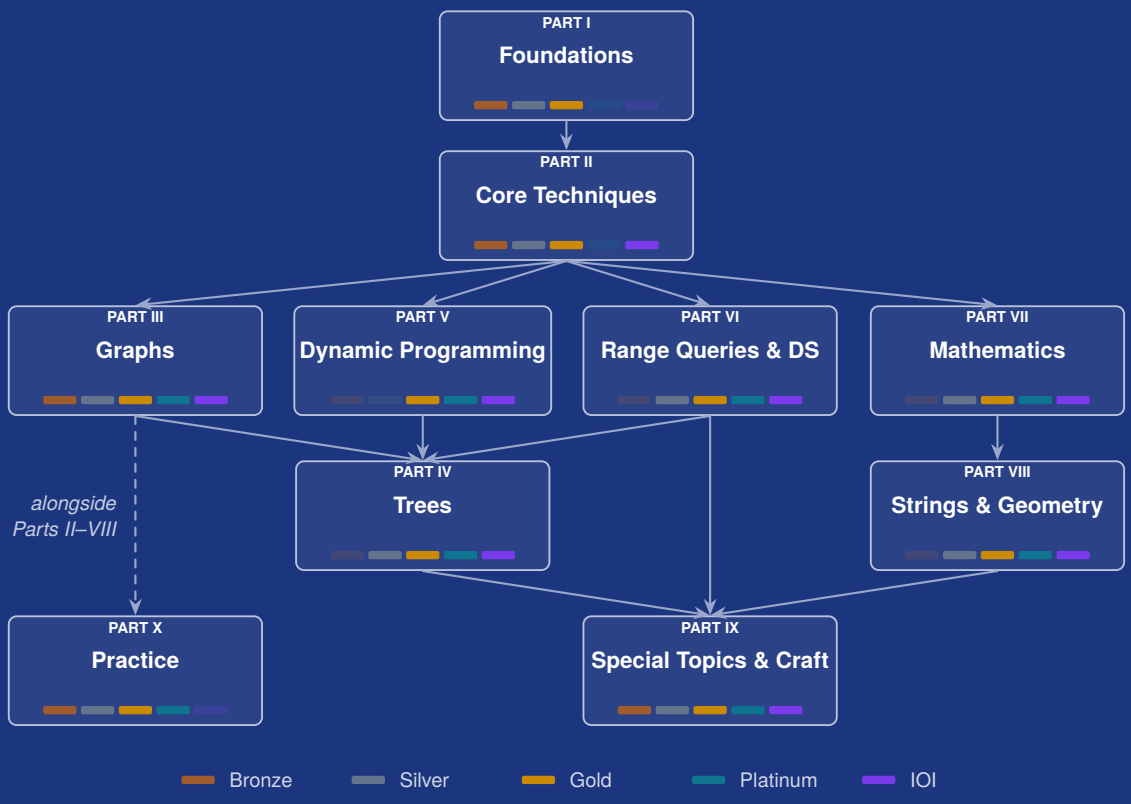
The Competitive Programming Handbook

Data structures, algorithms, patterns and tricks,
from your first loop to IOI gold

SECOND EDITION · 2026



The map of this book: each Part, what it builds on, and the levels it covers.

Kshitish Madbhavi

Companion to the template library in `Desktop/competitive`

The Competitive Programming Handbook

Second edition

About this book

The Competitive Programming Handbook, second edition, by **Kshitish Madbhavi**.

This handbook is the companion to the template library in `Desktop/competitive`. Every technique that has a template shows its keyboard shortcut and snippet prefix next to the explanation, so you can read an idea and have its code at your cursor a second later. The tools around it (CPH integration, the test runner, stress testing, the compiler setup) are described in the companion *User Guide*, `docs/GUIDE.pdf`.

Rebuilding. The book is generated from LaTeX sources in `docs/algorithms/`. Run the VS Code task *CP: Build Docs* or `python tools/cp.py docs`; `python tools/cp.py docs -chapter 13` rebuilds a single chapter for a quick look.

Code. Every program is C++17 and assumes the template (`#define int long long`, `rep`, `all`, `print`, ...). Library templates were checked against brute-force solutions on thousands of random inputs; the worked problems were compiled with the exact template and compiler flags and stress tested.

Typography. Set with pdfLaTeX in TeX Gyre Pagella, TeX Gyre Heros and Source Code Pro. Figures are drawn with TikZ and pgfplots in the Okabe–Ito palette, which stays distinguishable for colour-blind readers; every highlight also differs in line weight, so the figures survive grayscale printing.

Build: October 1, 2026.

Contents

How to Read This Book xiv

I Foundations 1

1 How to Approach a Problem 2

1.1 Anatomy of a problem . . . 2
1.2 A method that works . . . 3
1.3 Thinking tools . . . 7
1.4 When you are stuck . . . 7
1.5 Proving greedy choices . . . 9
1.6 Testing an idea before trusting it . . . 11
1.7 Kinds of problems . . . 11
1.8 How to practise . . . 12

2 Problem-Solving Patterns 14

2.1 Contribution: count each piece once . . . 14
2.2 Fix one thing . . . 16
2.3 Complementary counting . . . 17
2.4 Reverse time and go offline . . . 18
2.5 Events and sweeps . . . 19
2.6 Invariants and colourings . . . 20
2.7 Monovariants and termination . . . 21
2.8 Pigeonhole . . . 22
2.9 The extremal principle . . . 23
2.10 Symmetry and reflection . . . 23
2.11 When the answer is small . . . 24
2.12 Heavy and light . . . 25
2.13 Differences instead of values . . . 26
2.14 Reductions to known problems . . . 27

3 Complexity and Constraints 29

3.1 Counting operations . . . 29

3.2 Big-O notation . . . 30
3.3 What the limits tell you . . . 31
3.4 Analysing loops . . . 32
3.5 Amortized analysis . . . 33
3.6 Recursion . . . 34
3.7 Memory . . . 35
3.8 How fast is an operation, really? . . . 36
3.9 Constant factors that matter . . . 36
3.10 Estimating before coding . . . 37

4 C++ and the STL for Contests **39**
4.1 Input and output . . . 39
4.2 Integer types and overflow . . . 40
4.3 Choosing a container . . . 41
4.4 Sorting and comparators . . . 45
4.5 The algorithm toolbox . . . 46
4.6 Modern C++ that saves time . . . 47
4.7 Policy-based data structures . . . 47
4.8 Randomness . . . 48
4.9 Undefined behaviour and common bugs . . . 48

5 Choosing a Data Structure **50**
5.1 Operations decide . . . 50
5.2 Stacks . . . 51
5.3 Queues and deques . . . 53
5.4 Heaps . . . 54
5.5 Ordered sets and interval sets . . . 56
5.6 Hash maps for counting . . . 57
5.7 Augmenting simple structures . . . 57
5.8 Linked lists in arrays . . . 58

II Core Techniques **61**

6 Simulation and Complete Search **62**
6.1 Simulation . . . 62
6.2 Complete search with loops . . . 64

6.3 Generating subsets . . . 66
6.4 Generating permutations . . . 68
6.5 Backtracking and pruning . . . 69
6.6 Meet in the middle . . . 73
6.7 Fracturing search . . . 75
6.8 Casework and ad hoc problems . . . 76

7 Sorting and Greedy Algorithms 79
7.1 Sorting as a tool . . . 79
7.2 Counting sort and radix sort . . . 81
7.3 Comparators and exchange arguments . . . 82
7.4 Coordinate compression . . . 84
7.5 Greedy algorithms and their proofs . . . 85
7.6 Classic greedy problems . . . 86
7.7 Greedy with heaps and ordered sets . . . 89

8 Binary Search and Unimodal Optimisation 94
8.1 Searching a sorted array . . . 94
8.2 Binary search on a monotone predicate . . . 96
8.3 Binary search on the answer . . . 98
8.4 Real numbers and fractional programming . . . 100
8.5 Ternary search . . . 102
8.6 Parallel binary search . . . 104

9 Prefix Sums, Two Pointers and Sliding Windows 106
9.1 Prefix sums . . . 106
9.2 Difference arrays . . . 108
9.3 Counting subarrays with a hash map . . . 110
9.4 Two pointers . . . 112
9.5 Sliding windows . . . 116
9.6 Monotonic stacks in action . . . 117

10 Bits and Rectangles 120
10.1 Binary numbers and two's complement . . . 120
10.2 Bit tricks . . . 121
10.3 XOR . . . 123

10.4 Bitsets . . . 126
10.5 Gray codes and de Bruijn sequences . . . 127
10.6 Rectangle geometry . . . 130

III Graphs **132**

11 Graph Fundamentals **133**
11.1 Vocabulary . . . 133
11.2 Representing a graph . . . 134
11.3 Depth-first search . . . 135
11.4 Breadth-first search . . . 137
11.5 Grids and flood fill . . . 140
11.6 Bipartite graphs . . . 142
11.7 Finding cycles . . . 143
11.8 Functional graphs . . . 146
11.9 Bidirectional search and A* . . . 147

12 Shortest Paths **151**
12.1 Choosing the algorithm . . . 151
12.2 Relaxation . . . 152
12.3 Dijkstra's algorithm . . . 152
12.4 0–1 BFS . . . 156
12.5 Negative weights: Bellman–Ford . . . 157
12.6 Difference constraints . . . 159
12.7 All pairs: Floyd–Warshall . . . 161
12.8 Shortest and longest paths in DAGs . . . 162

13 Union–Find, Spanning Trees and Topological Order **165**
13.1 Disjoint set union . . . 165
13.2 Minimum spanning trees . . . 168
13.3 Prüfer codes . . . 172
13.4 Topological order . . . 174

14 Connectivity **177**
14.1 Strongly connected components . . . 177
14.2 2-SAT . . . 181

14.3 Bridges and articulation points . . . 183
14.4 Eulerian paths and circuits . . . 186
14.5 Offline dynamic connectivity . . . 188
14.6 Euler's formula and planar graphs . . . 190
14.7 Critical vertices and dominators . . . 191

15 Flows and Matchings **193**
15.1 Maximum flow . . . 193
15.2 Minimum cut . . . 195
15.3 Disjoint paths . . . 197
15.4 Bipartite matching . . . 198
15.5 König, Hall and path covers . . . 199
15.6 The assignment problem . . . 201
15.7 Minimum-cost flow . . . 202
15.8 Flows with lower bounds . . . 202
15.9 Matching in general graphs . . . 203
15.10 Global and all-pairs minimum cuts . . . 203

IV Trees **206**

16 Tree Techniques **207**
16.1 Rooted trees . . . 207
16.2 Diameter and farthest vertices . . . 209
16.3 The Euler tour technique . . . 210
16.4 Binary lifting and the LCA . . . 213
16.5 Offline LCA . . . 216
16.6 DP on trees . . . 218
16.7 Rerooting . . . 220
16.8 Merging child tables . . . 222
16.9 Small-to-large merging . . . 224
16.10 Tree isomorphism . . . 225

17 Tree Decompositions **230**
17.1 Heavy-light decomposition . . . 230
17.2 Centroid decomposition . . . 233
17.3 DSU on tree . . . 238

17.4 Long-path decomposition . . . 241
17.5 Virtual trees . . . 243
17.6 Euler tour trees . . . 245
17.7 Link-cut trees and top trees . . . 246

V Dynamic Programming 249

18 Dynamic Programming Fundamentals 250
18.1 What DP is . . . 250
18.2 The recipe . . . 251
18.3 Coins and sums . . . 253
18.4 Paths in grids . . . 255
18.5 Knapsack . . . 256
18.6 Increasing subsequences . . . 258
18.7 Two sequences: LCS and edit distance . . . 259
18.8 Choosing the state . . . 261
18.9 Reconstruction, memory and debugging . . . 262

19 DP Patterns 265
19.1 Bitmask DP . . . 265
19.2 Interval DP . . . 268
19.3 Digit DP . . . 269
19.4 Boundaries and profiles . . . 270
19.5 Sums over subsets . . . 272
19.6 Scheduling with sorted events . . . 274
19.7 Probability and expectation . . . 275
19.8 Games . . . 278
19.9 Matrix exponentiation . . . 279

20 DP Optimizations 281
20.1 Windows and monotone queues . . . 282
20.2 Divide and conquer optimization . . . 283
20.3 Monotone minima and SMAWK . . . 284
20.4 Knuth's optimization . . . 285
20.5 Convex hull trick . . . 286
20.6 Slope trick . . . 288

20.7 Aliens trick . . . 290
20.8 Convex (min, +) convolution . . . 291
20.9 Bitsets and swapped dimensions . . . 292

VI Range Queries and Data Structures **294**

21 Static Range Queries **295**
21.1 Sparse table . . . 295
21.2 Divide and conquer for static queries . . . 295
21.3 Square root decomposition . . . 296
21.4 Mo's algorithm . . . 296

22 Fenwick Trees and Segment Trees **298**
22.1 Fenwick tree . . . 298
22.2 Segment tree . . . 298
22.3 Lazy propagation . . . 299
22.4 Counting minimums . . . 300
22.5 Sparse (dynamic) segment trees . . . 300
22.6 Persistent segment trees . . . 300
22.7 Segment tree beats . . . 301
22.8 Range queries with sweep lines . . . 301
22.9 2D range queries . . . 301
22.10 Max suffix query with insertions only . . . 301

23 Ordered Sets, Treaps and Other Structures **302**
23.1 More operations on sorted sets . . . 302
23.2 Sorted sets with custom comparators . . . 302
23.3 Order statistics . . . 303
23.4 Treaps . . . 303
23.5 Wavelet trees . . . 303
23.6 Hash maps . . . 304

VII Mathematics **305**

24 Number Theory **306**
24.1 Divisibility . . . 306

24.2 Primes and sieves . . . 306
24.3 Large numbers: Miller–Rabin and Pollard rho . . . 307
24.4 Modular arithmetic . . . 307
24.5 Extended Euclid and linear equations . . . 308
24.6 Chinese remainder theorem . . . 308
24.7 Euler's totient . . . 308
24.8 Möbius function and inversion . . . 309
24.9 Floor-division blocks . . . 309

25 Combinatorics, Probability and Games 310
25.1 Counting principles . . . 310
25.2 Binomial coefficients . . . 310
25.3 Stars and bars . . . 311
25.4 Inclusion–exclusion . . . 311
25.5 Catalan numbers . . . 311
25.6 Burnside's lemma . . . 312
25.7 Probability . . . 312
25.8 Game theory . . . 312

26 Linear Algebra and Polynomials 314
26.1 Matrices . . . 314
26.2 Gaussian elimination . . . 314
26.3 XOR basis . . . 314
26.4 Polynomial multiplication and FFT . . . 315
26.5 Applications of convolution . . . 316

VIII Strings and Geometry 317

27 Strings 318
27.1 String hashing . . . 318
27.2 Prefix function and KMP . . . 318
27.3 Z-function . . . 319
27.4 Tries . . . 319
27.5 Aho–Corasick . . . 320
27.6 Manacher's algorithm . . . 320
27.7 Suffix arrays . . . 320

27.8 Suffix automaton . . . 321
27.9 Choosing a string tool . . . 322

28 Computational Geometry **323**
28.1 Vectors, dot and cross products . . . 323
28.2 Segment intersection . . . 323
28.3 Polygons . . . 324
28.4 Convex hull . . . 324
28.5 Closest pair of points . . . 324
28.6 Sweep lines . . . 325

IX Special Topics and Contest Craft **326**

29 Special Topics **327**
29.1 Randomized algorithms . . . 327
29.2 Interactive problems . . . 327
29.3 Communication problems . . . 328
29.4 Bitsets . . . 328
29.5 Matroids . . . 328
29.6 Vectorization and pragmas . . . 329
29.7 Constructive problems . . . 329

30 Contest Craft **330**
30.1 During a contest . . . 330
30.2 Debugging . . . 330
30.3 Recognising the technique . . . 331
30.4 Numbers worth remembering . . . 333
30.5 How to practise . . . 333

X Practice **334**

31 Worked Problems **335**
31.1 The method . . . 335
31.2 Maximum subarray sum . . . 336
31.3 Apartments . . . 338
31.4 Movie festival . . . 339

31.5 Factory machines . . . 340
31.6 Subarray sums . . . 342
31.7 Counting rooms . . . 343
31.8 Message route . . . 345
31.9 Shortest routes . . . 346
31.10 Road construction . . . 348
31.11 Coin combinations . . . 349
31.12 Book shop . . . 350
31.13 Longest increasing subsequence . . . 351
31.14 Subordinates . . . 353
31.15 Tree diameter . . . 354
31.16 Range sum queries with updates . . . 356
31.17 String matching . . . 357
31.18 Teleporters . . . 358
31.19 Where to go next . . . 359

A Reference Tables **361**
A.1 Complexity of the standard algorithms . . . 361
A.2 Formula sheet . . . 362
A.3 Where the templates are . . . 362

B Template Index **363**

Index **369**

How to Read This Book

This handbook teaches competitive programming from the first loop to the techniques that decide olympiad medals. It follows the USACO Guide syllabus (Bronze, Silver, Gold, Platinum, Advanced) and goes beyond it wherever contests do. Every topic is presented the same way:

- **What it is**, explained from scratch with a picture;
- **how to recognize** a problem that needs it (the part most books skip);
- the **key ideas, facts and complexity**, with proofs sketched where they build intuition;
- the **pitfalls** that cost people contests;
- **pro tricks**: less-known ideas that experienced competitors rely on;
- and the **template** in your library that implements it, with its keyboard shortcut.

Difficulty badges

Each section carries a badge showing roughly where it sits:

BRONZE	first steps: loops, simulation, brute force, simple math
SILVER	sorting, binary search, prefix sums, basic graphs, greedy
GOLD	DP, shortest paths, DSU, segment trees, number theory
PLATINUM	advanced data structures, tree decompositions, DP optimizations
IOI / ADVANCED	beyond Platinum: flows, suffix structures, FFT, exotic tricks

If you are starting out, read Parts I and II in order and skim the rest to know what exists. If you are experienced, jump to any chapter: they are self-contained, and the *Pro trick* boxes are where most of the non-obvious material lives. Part X, *Practice*, solves seventeen classic problems step by step, from reading the limits to tested code; try each one yourself before reading its solution.

Box legend

Key idea The central insight that makes the technique work.

How to recognize it Signals in a problem statement that point to this technique.

Pitfall A mistake that produces wrong answers, TLE or runtime errors.

Tip Practical advice for writing or debugging the code.

Pro trick An advanced idea, often the difference between a good and a great solution.

Fact A useful number, identity or theorem to remember.

Template: Name of the template Ctrl+Alt+T, key prefix `cpprefix`

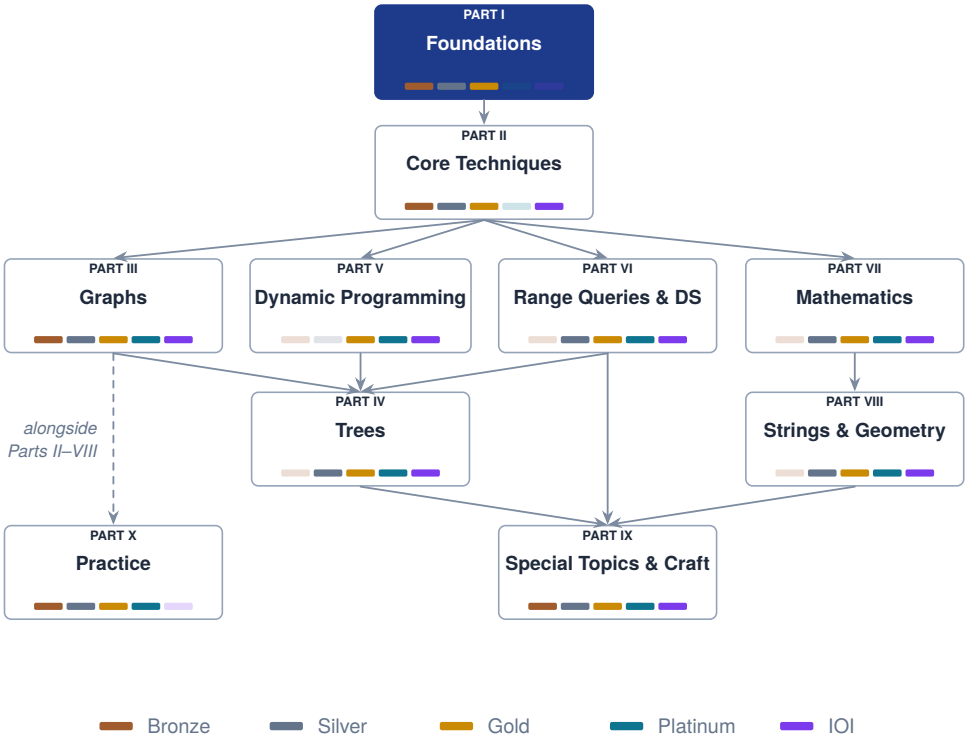
Conventions

Arrays and vertices are 0-indexed; ranges are half-open, $[l, r)$ meaning $l \le i < r$, unless stated otherwise. n is the input size, m the number of edges, q the number of queries, log is base 2. Code is C++17 and assumes your template (`#define int long long`, `ll`, `vi`, `all`, ...). The companion *User Guide* (GUIDE.pdf) explains the template, the library shortcuts and the tools.

PART I

Foundations

How to attack a problem, how to know whether an idea is fast enough, the C++ you need, and the data structures every solution is built from.



CHAPTER 1

How to Approach a Problem

Knowing algorithms is half of competitive programming. The other half is getting from a problem statement to the right algorithm, under time pressure, without fooling yourself. That second half is a skill like any other: it has a method, the method can be practised, and people who are good at it are good because they practised it deliberately, not because they were born knowing which problem wants a segment tree.

This chapter describes that method. It follows one problem from the first reading of its statement to accepted code, then collects the questions that experienced competitors ask themselves when they are stuck, and ends with the most useful proof technique for greedy algorithms: the exchange argument.

IN THIS CHAPTER BRONZE SILVER

- how to read a statement so that it tells you what the solution looks like;
- an eight-step loop that takes you from the statement to tested code;
- a checklist of questions for when you have no idea;
- how to prove a greedy algorithm correct, and how to find the counterexample when it is wrong;
- how to tell what kind of problem you are facing, and how to practise.

Before you start: nothing; this is the place to start.

1.1 Anatomy of a problem BRONZE

A problem statement is not only a question; it is also a collection of hints about the answer. Every part carries information, and learning to extract that information is the first skill to build. Figure 1.1 dissects a typical statement.

The story. Almost every statement wraps a mathematical question in a story: cows, cities, robots, games. The first job is to strip the story away and restate the question in the vocabulary of this book. "Barns connected by paths" is a graph. "A schedule of meetings" is a set of intervals. "Pouring water between buckets" is a state space you can search. "Each day the price changes" is an array, and "answer q questions about days l to r" is a range query. Once you can name the objects, you can ask what you know about them.

The input format. The shape of the input narrows things further. An input with exactly $n - 1$ edges on n vertices is a tree (if it is connected), and trees have their own toolbox (Part IV). A grid of characters is a graph whose edges you never store. A list of queries after the data means you must answer many questions fast, which suggests precomputation or a data structure. Read the input format with the question "what objects am I given?" in mind.

The limits. This is the line that experienced competitors read first. A judge performs about 10^8 simple operations per second, so the limits tell you which complexities are possible (Table 3.1 in

Barn Paths

Farmer John's n barns are joined by m two-way paths. Bessie is in barn 1 and wants to reach barn n walking along as few paths as possible. Print that number, or -1 if she cannot.

Input. The first line has n and m. Each of the next m lines has two barns a_i and b_i joined by a path.

Limits. $2 \le n \le 10^5$, $1 \le m \le 2 \cdot 10^5$.

Sample. `4 4 / 1 2 / 2 4 / 1 3 / 3 4` → `2`

Subtask 1 (30 points). $m = n - 1$ and path i joins barns i and $i + 1$.

The story hides the mathematics: barns are vertices, paths are edges, "as few paths as possible" is an unweighted shortest path.

The input format names the objects: an edge list, so a general graph, not a tree and not a grid.

The limits set the budget: about $3 \cdot 10^5$ items, so $O(n + m)$ time. BFS fits.

The sample checks your reading: $1 \to 2 \to 4$ uses two paths.

The subtask is a hint: on a line the answer is $n - 1$.

Figure 1.1: The parts of a statement and what each one tells you. The limits are the most informative line of all: they tell you how fast the solution must be before you know what it is.

the next chapter). $n \le 20$ invites trying all 2^n subsets; $n \le 5000$ allows $O(n^2)$; $n \le 2 \cdot 10^5$ demands $O(n \log n)$; $n \le 10^{18}$ means the answer is a formula or a logarithmic algorithm. The same story with different limits is a different problem.

The samples. Samples exist to check that you understood the question, not to test your code. Work at least one sample by hand before designing anything. Many wrong answers come from solving a slightly different problem than the one asked: 0-indexed versus 1-indexed, "at most" versus "exactly", "subsequence" versus "substring".

Subtasks. Olympiads split the points into subtasks with extra restrictions, and those restrictions are hints. "All values are distinct", "the graph is a line", "$k = 1$": each one removes a difficulty, and the solution to a subtask is often a stepping stone to the full solution. Even when you cannot solve the whole problem, a subtask gives you points; in the IOI, most medals are decided by partial scores.

> **Tip** Before writing code, write the restated problem in one sentence of mathematics at the top of your file as a comment: `// shortest path from 0 to n-1 in an unweighted graph, n <= 1e5`. If you cannot write that sentence, you have not understood the problem yet, and no amount of coding will fix that.

1.2 A method that works

BRONZE

Figure 1.2 shows the loop that takes a problem from statement to accepted code. The steps are not new; what matters is doing all of them, in order, every time. Most people who get stuck on a problem skipped steps 3 and 4 and are trying to jump straight to a clever idea.

We follow one problem through all eight steps. It is a classic from the CSES Problem Set, and it is a good example because the right idea is not visible in the statement.

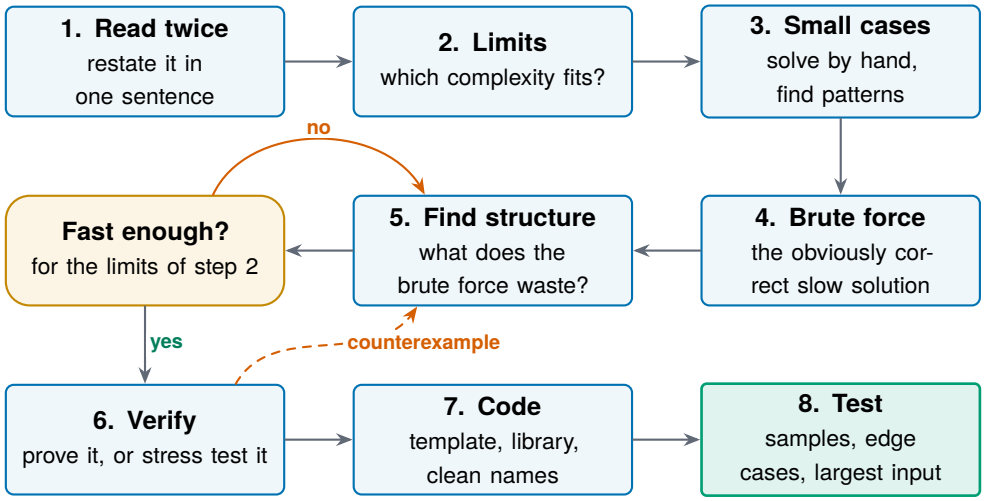


Figure 1.2: The problem-solving loop. Steps 3 to 5 are where the thinking happens; step 6 is what separates people who get accepted from people who get *wrong answer on test 17*.

Example: Array Division (CSES)

You are given n positive integers and a number k. Divide the array into k contiguous, non-empty parts so that the largest sum of a part is as small as possible, and print that largest sum. Limits: $1 \le k \le n \le 2 \cdot 10^5$, values up to 10^9.

1.2.1 Step 1: read twice, restate once

Read the whole statement, then restate it in one sentence of mathematics: *choose $k-1$ cut positions in the array to minimize the maximum part sum.* Check the restatement against the words: the parts are contiguous (so we cut, we do not choose subsets), every part is non-empty (so the cuts are distinct), and all values are positive (remember this: positivity often matters). The sample in CSES is $n = 5$, $k = 3$, array $2, 4, 7, 3, 5$, answer 8.

1.2.2 Step 2: let the limits talk

$n \le 2 \cdot 10^5$ rules out anything quadratic: $n^2 = 4 \cdot 10^{10}$ operations would take minutes. We need roughly $O(n \log n)$ or $O(n \log(\text{something}))$. That already hints at sorting, a binary search, a heap or a segment tree, and it rules out "try every way to cut", which is $\binom{n-1}{k-1}$, astronomically many. The sums can reach $2 \cdot 10^5 \cdot 10^9 = 2 \cdot 10^{14}$, so the answer needs 64-bit integers.

1.2.3 Step 3: solve small cases by hand

Take the sample and try splits by hand (Figure 1.3). A few observations fall out immediately. The answer is at least the largest element (7 here), because that element sits in some part. It is at most the total (21), the answer for $k = 1$. And the good splits are "balanced": moving a cut towards the heavier side makes things better.

1.2.4 Step 4: write down the brute force

The obviously correct solution tries every way to cut: choose $k-1$ of the $n-1$ gaps and take the best. A slightly smarter brute force is a DP, best$[i][j]$ = the smallest possible maximum when the

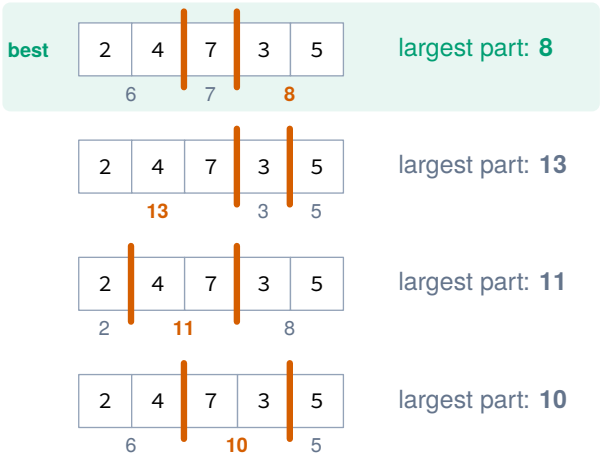


Figure 1.3: Four of the six ways to cut $2, 4, 7, 3, 5$ into three parts. The best one (green) has maximum 8; no split can do better than 7, the largest element, and here even 7 is impossible.

first i elements are cut into j parts, which takes $O(n^2 k)$ time. Both are far too slow for the real limits, and that is fine: the brute force is not meant to be submitted. It fixes *what* we compute, gives us a reference to test against, and for small subtasks it is often worth points on its own.

1.2.5 Step 5: find structure

Now ask what the brute force wastes. It builds every split and *then* measures its maximum. What if we turn the question around and fix the maximum first? "Can the array be cut into k parts, each with sum at most C?" is a much easier question, because a greedy answers it: walk through the array, extend the current part while it still fits under C, and start a new part when it does not. Extending a part as far as possible never hurts, since it leaves the fewest elements for the remaining parts (this is an exchange argument, Section 1.5). If the greedy needs at most k parts, the answer is yes; with fewer than k parts we can always split some part further, because every value is positive and every part has at least one element.

The question also has the crucial property of *monotonicity*: if cap C works, every larger cap works too. So the answers form a block of "no" followed by a block of "yes" (Figure 1.4), and we can binary search for the boundary. This is binary search on the answer, the subject of Chapter 8.

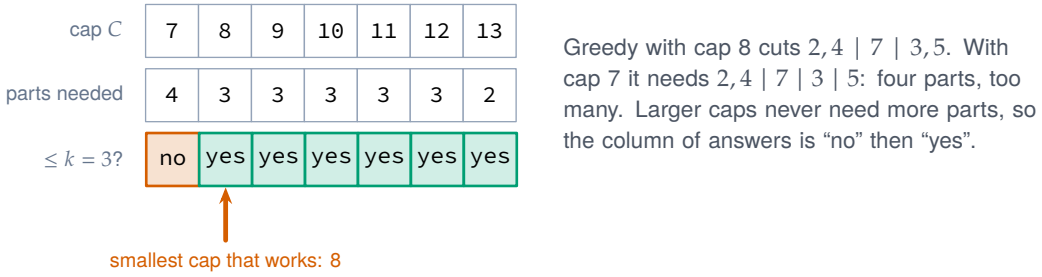


Figure 1.4: For each cap C, the number of parts the greedy needs. The yes/no row is monotone, so binary search finds the first "yes" in about $\log_2(2 \cdot 10^{14}) \approx 48$ steps.

1.2.6 Step 6: verify before you code

Two claims carry the solution: the greedy check is correct, and the answers are monotone. The monotonicity is immediate (a split that respects cap C also respects $C + 1$). For the greedy, a short

exchange argument works: take any valid split into at most k parts; its first part ends no later than the greedy's first part, because the greedy extends as far as the cap allows; moving that first cut to where the greedy put it only shrinks the second part; repeat. So if any valid split exists, the greedy finds one with no more parts. When you cannot find a proof in a few minutes, the fallback is to stress test the idea against the brute force on thousands of small random inputs (Section 1.6).

1.2.7 Step 7: code

Only now do we type. The search range is from the largest element (the answer can never be smaller) to the total (the answer for one part).

```
void solve() {
    int n, k;
    cin >> n >> k;
    vi a(n);
    cin >> a;
    auto parts_needed = [&](ll cap) {  // greedy: extend a part while it fits
        ll parts = 1, cur = 0;
        for (ll x : a) {
            if (cur + x > cap) parts++, cur = 0;
            cur += x;
        }
        return parts;
    };
    ll lo = *max_element(all(a)), hi = accumulate(all(a), 0LL);
    while (lo < hi) {  // smallest cap that needs at most k parts
        ll mid = lo + (hi - lo) / 2;
        if (parts_needed(mid) <= k) hi = mid;
        else lo = mid + 1;
    }
    print(lo);
}
```

The complexity is $O(n \log S)$ where S is the total: about $2 \cdot 10^5 \cdot 48 \approx 10^7$ operations, which runs in a few milliseconds. This code, like every complete program in this book, was compiled with your template and checked against a brute force on random inputs.

1.2.8 Step 8: test

Run the CPH samples first. Then think about the edges of the input space: $n = 1$; $k = 1$ (the answer is the total); $k = n$ (the answer is the maximum); all elements equal; one huge element among small ones; the largest possible input (to check speed and 64-bit overflow). Each of these catches a different class of bug. If anything is uncertain, a stress test against the brute force settles it.

> **The loop in real contests** Experienced competitors run through this loop in their heads in a minute or two for easy problems, and spend most of their contest time in steps 3 to 6 for hard ones. The two most common causes of a lost problem are skipping step 2 (an $O(n^2)$ solution for $n = 2 \cdot 10^5$) and skipping step 6 (a greedy that "obviously" works and does not).

1.3 Thinking tools

BRONZE

Step 5 of the loop, "find structure", is where problems are solved, and it is the step that feels least like a method. It becomes much more mechanical once you have a set of moves to try. These are the ones that work most often, adapted from George Pólya's classic *How to Solve It*.

Draw a picture. Arrays become bars, intervals become segments on a line, pairs (x, y) become points in the plane, states become vertices. A picture makes monotonicity, overlaps and symmetry visible in a way that indices never do. Most figures in this book exist because the author needed them to see the idea.

Specialise. Solve an easier version first: all values equal, $k = 1$, the graph is a path, the grid has one row. The special case either solves the problem outright (surprisingly often) or shows which part is hard. Subtasks are special cases chosen by the problem setter, and they are chosen because they are stepping stones.

Generalise. Sometimes the more general problem is easier. "Is there a path from 1 to n?" is best answered by computing *all* reachable vertices. "Answer this for $k = 5$" becomes easier as "answer this for every k", which may have a recurrence.

Work backwards. Start from the goal. What must the last move of a game look like? What does the final state of a process look like, and what could have come just before it? Many DP and game problems are clearest when read from the end.

Look at the answer. What does an optimal solution look like? Can you assume it has a normal form (sorted, with no crossing pairs, with the largest element first)? Proving that "some optimal solution has form X" often cuts the search space from exponential to polynomial.

Change one thing at a time. If the answer for an input is known, how does it change when one element is added, removed or modified? The answer to that question is often an incremental algorithm or a data structure.

Find the bottleneck. In the brute force, which part is slow? If it is "for every pair, compute something", can the something be precomputed? If it is "for every query, scan the array", can the array be preprocessed? Improving the bottleneck by one factor of n is usually the whole solution.

1.4 When you are stuck: a checklist

SILVER

When nothing comes, go through these questions one by one. Each has solved thousands of problems, and together they cover the large majority of what appears in contests up to the Gold level. Chapter 2 develops the most important of them into full patterns with worked examples.

Is the answer monotone in some parameter?
If "x works" implies "$x + 1$ works", binary search on the answer turns an optimisation problem into a sequence of yes/no checks, as in Array Division above (Chapter 8).

Would sorting change nothing?
If the answer does not depend on the input order, sort it. Sorted data enables greedy choices, two pointers, sweeps and binary search (Chapter 7).

Can each element's contribution be counted separately?
"Sum over all subarrays of their minimum" is hard as stated; "for each element, in how many subarrays is it the minimum?" is easy with a monotonic stack. This is the contribution technique (Section 2.1).

Is the complement easier?
Counting "at least one" is often done as total minus "none". Counting pairs that are *not* coprime is total minus coprime pairs. Inclusion–exclusion generalises this (Chapter 25).

Can I fix one thing and solve the rest fast?
Fix the left endpoint, the maximum element, the middle of a palindrome, the first move. Then the remaining problem is often one-dimensional and fast with a data structure.

Can I process things in a different order?
Offline (read all queries first, answer them in a convenient order), backwards in time (deletions become insertions, which DSU handles), or sorted by some key (sweep line).

Is there an invariant?
A quantity that never changes proves that something is impossible; a quantity that always decreases proves that a process terminates. Parity and colouring arguments are the classic ones (Section 2.6).

Does the answer depend only on a small state?
If the future depends only on "where I am" and a few numbers, not on the whole history, dynamic programming applies (Part V).

Are things connected to other things?
Model it as a graph: vertices for the objects, edges for the relation, and reach for Part III.

Is some parameter small?
Values up to 100, $k \leq 20$, at most 60 distinct colours: iterate over the small parameter, or use it as a bitmask.

Do large and small cases behave differently?
Handle "heavy" items (few, each large) and "light" items (many, each small) with different methods; the threshold $\sqrt{n}$ balances them (Chapter 21).

What does an optimal solution look like?
Prove it can be assumed sorted, non-crossing or canonical, then search only among such solutions.

Is it a known problem in disguise?
Shortest path, matching, interval scheduling, LIS, knapsack and flow appear in costume constantly. The recognition table in Chapter 30 maps statement signals to techniques.

> **Change the question** Many hard problems become easy when you ask a different but equivalent question. "Minimise the maximum" becomes "can the maximum be at most X?". "Count pairs with property P" becomes "for each element, count the partners seen so far". "Is there a path?" becomes "are they in the same component?". "What is the k-th smallest?" becomes "how many are at most x?". Train yourself to rephrase every problem at least twice before choosing an algorithm; the second phrasing is usually the one with a known solution.

1.5 Proving greedy choices: the exchange argument

SILVER

A greedy algorithm builds a solution one piece at a time, always taking the choice that looks best right now, and never reconsidering. Greedy algorithms are short, fast and very often wrong. The exchange argument is the standard way to tell the difference before you spend half an hour coding one.

1.5.1 The argument

To show that a greedy algorithm is optimal, show that *some* optimal solution agrees with the greedy's first choice. Then the rest of the problem is a smaller instance of the same kind, and induction finishes the proof:

1. Take any optimal solution O. If its first choice equals the greedy choice g, there is nothing to do.
2. Otherwise, *exchange*: modify O so that it makes the choice g, and show that the result O' is still valid and is no worse than O. Then O' is optimal too.
3. O' agrees with the greedy on the first choice; repeat the argument on what remains.

Example: Interval scheduling

Given n intervals, choose as many pairwise disjoint ones as possible.

Solution. Sort by end time and repeatedly take the first interval that does not overlap the ones already taken. Exchange: let O be an optimal solution whose first interval o_1 is not the greedy's first interval g, which is the interval that ends first overall. Replace o_1 by g (Figure 1.5). Since g ends no later than o_1, it cannot overlap o_2, which starts after o_1 ends. The new solution is valid and has the same size, so it is optimal and starts with g.

Template: Intervals: merge + max disjoint Ctrl+Alt+T, Shift+N prefix `cpintervals`

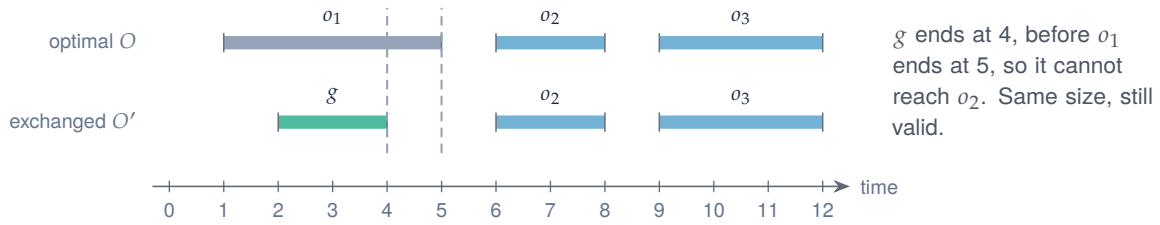


Figure 1.5: The exchange for interval scheduling: swapping the first interval of an optimal solution for the interval that ends earliest keeps the solution valid and just as large.

1.5.2 A second example: adjacent swaps

When the greedy is an *ordering* ("process items sorted by X"), the cleanest exchange is between two neighbours. Take any order that is not sorted by the greedy key; somewhere two adjacent items are in the wrong order; swapping them changes only their own contribution, so you just compare two numbers.

Example: Tasks and Deadlines (CSES)

There are n tasks; task i takes a_i time units and has deadline d_i. You do them one after another, in an order you choose, starting at time 0. Finishing task i at time f_i earns $d_i - f_i$ points (possibly negative). Maximise the total. Limits: $n \le 2 \cdot 10^5$.

Solution. The total is $\sum d_i - \sum f_i$, and $\sum d_i$ does not depend on the order, so we must minimise the sum of finishing times. Do the shortest task first. Proof by adjacent swap (Figure 1.6): if task i comes immediately before task j and both start after time T, their finishing times are $T + a_i$ and $T + a_i + a_j$; swapped, they are $T + a_j$ and $T + a_j + a_i$. Every other task is unaffected, so the swap changes the total by $a_j - a_i$. If $a_i > a_j$, swapping strictly helps; hence an optimal order has no "longer before shorter" neighbours, which means it is sorted by duration.

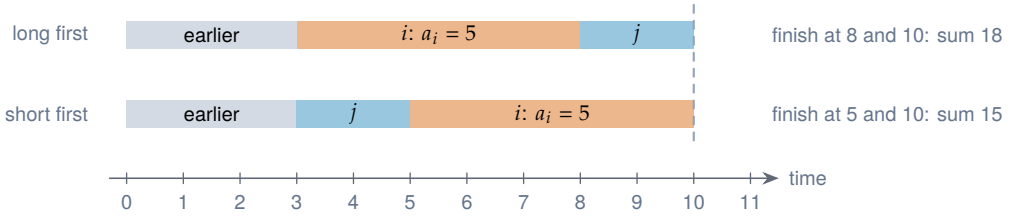


Figure 1.6: Swapping two adjacent tasks leaves everything outside them untouched (both orders end at time 10), and puts the shorter task's finishing time first. The sum drops by $a_i - a_j = 3$.

```
void solve() {
    int n;
    cin >> n;
    vector<pll> task(n);  // (duration, deadline)
    for (auto& [a, d] : task) cin >> a >> d;
    sort(all(task));  // shortest first
    ll now = 0, reward = 0;
    for (auto [a, d] : task) now += a, reward += d - now;
    print(reward);
}
```

Notice what the deadlines did: nothing. The statement made them look important, the mathematics showed they are irrelevant. This happens more often than you would think, and it is exactly what the restatement in step 1 is for.

1.5.3 When greedy is wrong

A greedy that "obviously works" fails more often than it succeeds. The classic example is making change: with coins $\{1, 3, 4\}$ and amount 6, "always take the largest coin that fits" pays $4 + 1 + 1$ (three coins), but $3 + 3$ uses two. The exchange argument breaks exactly where the greedy breaks: you cannot swap the 4 into an optimal solution without paying extra elsewhere. The cure is dynamic programming (Chapter 18).

Hunting counterexamples If you cannot prove a greedy in a few minutes, look for a counterexample in the smallest cases: two or three items, values from 1 to 5. Wrong greedy ideas almost always fail on inputs of size 3 to 6. Even better, let the computer look: a brute force plus a random generator plus a loop finds the counterexample in seconds (next section).

1.6 Testing an idea before trusting it

SILVER

A **stress test** runs your fast solution and a brute force on thousands of small random inputs and stops at the first input where they disagree (Figure 1.7). It is the single most effective debugging technique in competitive programming, and it is just as useful *before* coding the real solution: implement the brute force and a quick, throwaway version of the idea, and let the stress test tell you whether the idea is right.

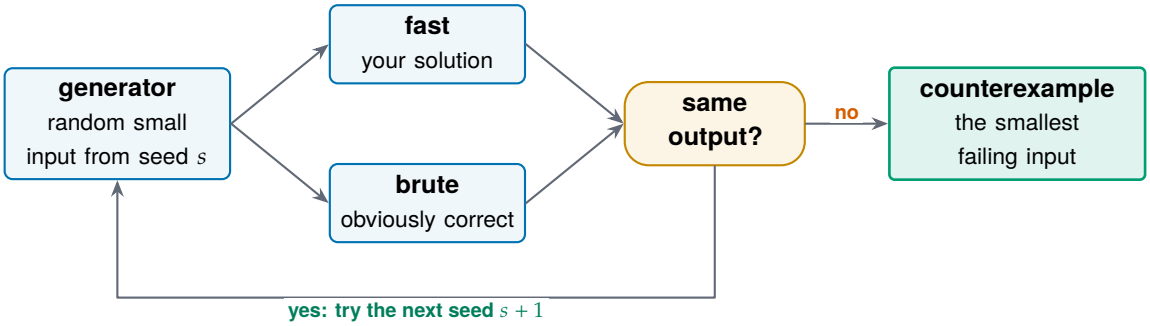


Figure 1.7: A stress test. Because the inputs are small, the counterexample it finds is small enough to debug by hand. Your setup runs this loop with Ctrl+Alt+X (GUIDE.pdf, Section 4.4).

Three details make stress tests effective. First, keep the inputs *tiny*: n from 1 to 6, values from 1 to 10. Bugs show up on small inputs just as reliably as on large ones, and a failing input of size 4 can be debugged by hand. Second, make the brute force as dumb as possible, so that it is obviously correct; its speed does not matter. Third, when the test finds a counterexample, add it to CPH as a test case before fixing anything, so that it stays fixed.

1.7 Kinds of problems

BRONZE

Problems fall into a handful of kinds, and each kind has its own habits. Recognising the kind is often the first real step towards the solution.

Kind	Asks for	Typical tools
Decision	yes or no: is it possible?	invariants, greedy, graph search, 2-SAT
Optimisation	the best value, or a best object	greedy, DP, binary search on the answer, flows
Counting	how many (usually modulo $10^9 + 7$)	combinatorics, DP, inclusion–exclusion
Construction	any valid object, or proof there is none	patterns from small cases, induction, greedy
Queries	many questions on one structure	prefix sums, segment trees, offline processing
Interactive	ask questions, deduce a hidden object	binary search, adaptive strategies
Games	who wins with perfect play	game DP, Sprague–Grundy, symmetry

Decision problems often have a clean characterisation: "possible if and only if the sum is even and no element exceeds half of it". Look for the invariant that makes impossible cases impossible, then check that everything else is possible.

Optimisation problems ask for a minimum or maximum. The first question is always whether a greedy works (try to prove it, or break it on small cases); the second is whether the answer is monotone in some parameter (binary search); the third is what state a DP would need.

Counting problems print huge numbers modulo a prime. They almost never ask you to enumerate: count by structure. Decompose the objects you count, count each part independently, multiply; when parts overlap, use inclusion–exclusion; when the structure is sequential, use DP.

Construction problems accept any valid answer. Solve small cases by hand and look for a pattern that generalises, often by induction: a construction for n from one for $n-1$ or $n-2$. Checking your output with a small validator program is worth the minute it takes.

Query problems give data and many questions. Decide first whether the data changes (static or dynamic), whether the queries can be answered offline (read them all first), and what operation combines two ranges (sum, min, gcd). Part VI is about exactly this.

Interactive problems hide an object and let you ask questions. The number of allowed questions is the key constraint: $\log_2$ of the number of possibilities means binary search; more questions allow more naive strategies (Chapter 29).

Games ask who wins with perfect play. Small games are solved by DP over positions (winning if some move leads to a losing position); sums of independent games are solved by Sprague–Grundy theory (Chapter 25).

1.8 How to practise

BRONZE

Solving problems is the only way to get better at solving problems, but not all practice is equal.

- **Work slightly above your level.** Problems you solve in five minutes teach little; problems you cannot even start teach little too. The sweet spot is problems you solve in 30 to 60 minutes, or solve after reading one hint.
- **Upsolve.** After every contest, solve the problems you could not solve during it, reading the editorial if needed. Then write down, in one sentence, the idea you were missing. This habit is worth more than any single technique in this book.
- **Implement, do not just read.** Reading an editorial gives the feeling of understanding; implementing it shows whether you really do. Most of the details that matter only appear when you write the code.
- **Build a library and know it.** This book comes with one (GUIDE.pdf). Knowing what is in it, and pasting it with one shortcut, saves minutes in every contest and removes a whole class of bugs.
- **Review your mistakes.** Keep a list of the bugs you made (overflow, off-by-one, uninitialised variables, wrong reading of the statement). The same five bugs cause most of everyone's wrong answers.

Coding before thinking Five minutes of thinking saves thirty minutes of debugging. If you cannot explain your algorithm, and why it is correct, in three sentences, do not start typing. The pressure of a contest makes typing feel like progress; it usually is not.

Chapter summary

- Read the statement for information: the story names the objects, the input format their shape, the limits the complexity, the samples your understanding, the subtasks the path to the solution.
- Follow the loop: read, budget, small cases, brute force, structure, verify, code, test. Do not skip steps 3, 4 and 6.
- When stuck, run through the checklist; the most powerful move is to change the question.
- Prove greedy algorithms with an exchange argument, or break them with a small counterexample, ideally found by a stress test.

Practice problems (CSES Problem Set)

WARM-UP: READING AND RESTATING

- Weird Algorithm
- Missing Number
- Repetitions
- Increasing Array
- Permutations
- Number Spiral
- Two Knights
- Two Sets
- Coin Piles

SMALL CASES, PATTERNS AND CONSTRUCTIONS

- Bit Strings
- Trailing Zeros
- Palindrome Reorder
- Gray Code
- Tower of Hanoi
- Digit Queries

GREEDY AND EXCHANGE ARGUMENTS

- Stick Lengths
- Movie Festival
- Tasks and Deadlines
- Array Division
- Ferris Wheel
- Missing Coin Sum

CHAPTER 2

Problem-Solving Patterns

Algorithms are tools; patterns are the habits of thought that tell you which tool to pick up. Experienced competitors do not solve a new problem from scratch: they recognise that it "is a contribution problem", "wants to be run backwards", or "has a small answer, so enumerate it". None of these patterns is an algorithm you could put in a library, yet together they solve a large fraction of all contest problems, from Bronze to the IOI.

Each section below describes one pattern, shows it on a concrete problem with a figure, and ends with the signals that suggest it. Where a problem needs code, the code is complete and was tested against a brute force.

IN THIS CHAPTER SILVER GOLD PLATINUM

- count by *contribution*, and count the complement;
- fix one part of the answer and solve the rest fast;
- reverse time, go offline, and sweep over events;
- prove impossibility with invariants and termination with monovariants;
- use pigeonhole, extremal and reflection arguments;
- exploit answers that are small, split heavy from light, and work with differences instead of values.

Before you start: Chapter 1; some examples use sorting, prefix sums and a Fenwick tree, which later chapters explain in full.

2.1 Contribution: count each piece once GOLD

Many counting problems ask for a sum over a huge family of objects: over all subarrays, all pairs, all paths in a tree. Enumerating the objects is hopeless (there are $\Theta(n^2)$ subarrays), but the sum can be regrouped: instead of asking "what does each object contribute?", ask "for each *element*, in how many objects does it play a role, and what does it contribute there?". The second question usually has a closed-form answer. This regrouping is the **contribution technique**, and it is the single most useful idea in counting problems.

Example: Sum of subarray minimums

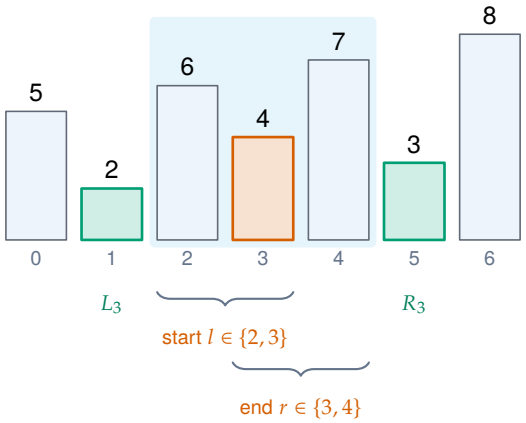
Given $a_0, \ldots, a_{n-1}$ ($n \le 2 \cdot 10^5$, values up to 10^6), compute the sum of $\min(a_l, \ldots, a_r)$ over all $\frac{n(n+1)}{2}$ subarrays.

Fix an element a_i and ask in which subarrays it is the minimum. Going left from i, the subarray may extend until it would include an element smaller than a_i; going right, likewise. If L_i is the index of the previous smaller element and R_i the index of the next smaller one, then a_i is the minimum of exactly the subarrays that start in $(L_i, i]$ and end in $[i, R_i)$, and there are $(i - L_i)(R_i - i)$

of them (Figure 2.1). So

$$\sum_{l \le r} \min(a_l, \ldots, a_r) \;=\; \sum_i a_i \cdot (i - L_i) \cdot (R_i - i).$$

Both L_i and R_i come from a monotonic stack in $O(n)$ (Chapter 9). One detail needs care: when equal values occur, a subarray with two equal minima must be counted once, not twice. Breaking ties asymmetrically does it: use "strictly smaller" on the left and "smaller or equal" on the right, so the subarray is charged to the last copy of its minimum.



The element 4 at index 3 is the minimum of the four subarrays $[6,4]$, $[6,4,7]$, $[4]$ and $[4,7]$: every choice of a start in $\{2,3\}$ and an end in $\{3,4\}$. Its window stops just before the nearest smaller elements $L_3 = 1$ and $R_3 = 5$. It contributes $4 \cdot 2 \cdot 2 = 16$.

Figure 2.1: The contribution of one element. It rules the shaded window, which stops just before the nearest smaller elements on each side (green). Summing $a_i \cdot$ (choices of start) $\cdot$ (choices of end) over all i counts every subarray exactly once.

```
void solve() {
    int n;
    cin >> n;
    vi a(n);
    cin >> a;
    // a[i] is the minimum of a[l..r] iff left[i] < l <= i <= r < right[i]
    vi left(n), right(n), st;
    rep(i, n) {  // previous strictly smaller element
        while (!st.empty() && a[st.back()] >= a[i]) st.pop_back();
        left[i] = st.empty() ? -1 : st.back();
        st.push_back(i);
    }
    st.clear();
    per(i, n) {  // next smaller or equal element: a tie goes to its last copy
        while (!st.empty() && a[st.back()] > a[i]) st.pop_back();
        right[i] = st.empty() ? n : st.back();
        st.push_back(i);
    }
    ll ans = 0;
    rep(i, n) ans += a[i] * (i - left[i]) * (right[i] - i);
    print(ans);
}
```

The same regrouping works far beyond minimums. The sum of $|a_i - a_j|$ over all pairs becomes, after sorting, $\sum_i a_i\,(2i - n + 1)$: the i-th smallest value is added i times (against every smaller value) and subtracted $n - 1 - i$ times. The sum over all paths of a tree of their lengths becomes a sum over

edges of (size of one side) × (size of the other). The expected number of fixed points of a random permutation is $n \cdot \frac{1}{n} = 1$ by linearity, without ever looking at a single permutation.

> **How to recognize it** "Sum over all subarrays / pairs / paths / subsets of f" where enumerating the objects is too slow. Ask for each element (or edge, or value) how many objects it influences and how.

2.2 Fix one thing, then solve the rest fast

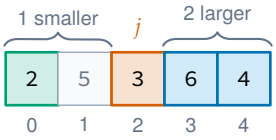
SILVER

When an answer involves several choices (a triple of indices, a subarray's two ends, a path's two endpoints), fixing one of them often makes the rest easy. The art is choosing *which* one to fix: it is usually the one in the middle, the one that plays a special role (the maximum, the root, the first move), or the one that makes the remaining choices independent.

Example: Increasing triplets

Count the triples $i < j < k$ with $a_i < a_j < a_k$ ($n \le 2 \cdot 10^5$).

Fixing i leaves a two-dimensional problem. Fixing the *middle* index j splits it into two independent one-dimensional counts: the number of smaller elements to its left, and the number of larger elements to its right. Every pair of a left partner and a right partner is a valid triple, so j contributes the product (Figure 2.2). Both counts come from a Fenwick tree over compressed values (Chapter 22): sweep left to right counting smaller elements seen so far, then right to left counting larger ones.



With $a_j = 3$ in the middle, the triples are $(2,3,6)$ and $(2,3,4)$: one choice on the left times two on the right.

Figure 2.2: Fixing the middle element makes the two sides independent: j contributes (smaller on the left) × (larger on the right).

```
void solve() {
    int n;
    cin >> n;
    vi a(n);
    cin >> a;
    vi vals = a;  // compress values to 0..m-1
    sort(all(vals)), vals.erase(unique(all(vals)), vals.end());
    for (auto& x : a) x = lower_bound(all(vals), x) - vals.begin();
    int m = sz(vals);
    vi bit(m + 1, 0);
    auto add = [&](int i) { for (i++; i <= m; i += i & -i) bit[i]++; };
    auto count_below = [&](int v) {  // how many stored values are < v
        ll s = 0;
        for (; v > 0; v -= v & -v) s += bit[v];
        return s;
    };
    vi smaller_left(n);
    rep(j, n) smaller_left[j] = count_below(a[j]), add(a[j]);
    fill(all(bit), 0);
```

```
    ll ans = 0;
    per(j, n) {  // fix the middle element j
        ll larger_right = (n - 1 - j) - count_below(a[j] + 1);
        ans += smaller_left[j] * larger_right;
        add(a[j]);
    }
    print(ans);
}
```

Other classic "fix one thing" moves: fix the right end of a subarray and maintain something about all left ends (prefix sums with a hash map, Chapter 9); fix the maximum of a subarray and count the subarrays where it is the maximum (the contribution technique again); fix the lowest common ancestor of a tree path (Part IV); fix the value of the answer and check it (binary search on the answer, Chapter 8); fix the number of items taken from one group and optimise the rest.

How to recognize it Triples, pairs or intervals with a condition that couples all parts. Look for the part whose choice makes the others independent or one-dimensional.

2.3 Complementary counting

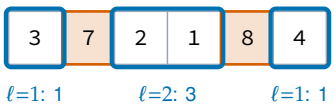
SILVER

Counting objects that satisfy "at least one", "not all" or "some pair differs" is often awkward, while their complement ("none", "all", "all equal") is simple. Then count the complement and subtract it from the total. It sounds trivial, and it is, but it is missed surprisingly often under contest pressure.

Example: Subarrays with a large element

Count the subarrays that contain at least one element $\geq K$.

A subarray *without* a large element lies entirely inside a maximal run of small elements, and a run of length ℓ contains $\frac{\ell(\ell+1)}{2}$ subarrays. So the answer is $\frac{n(n+1)}{2}$ minus the sum over runs (Figure 2.3), computed in one pass.



$K = 5$. All subarrays: $\frac{6 \cdot 7}{2} = 21$. Without a large element (inside a blue run): $1 + 3 + 1 = 5$. So $21 - 5 = 16$ subarrays contain a 7 or an 8.

Figure 2.3: Counting the complement: subarrays that avoid the large elements (orange) live inside the maximal runs of small elements (blue), and a run of length ℓ holds $\ell(\ell+1)/2$ of them.

Complementary counting generalises to **inclusion–exclusion** when there are several "bad" events (Chapter 25): the number of objects avoiding all bad events is the total, minus those with each single bad event, plus those with each pair, and so on. It is also the natural way to count pairs of *distinct* things: count all pairs, subtract the pairs that are equal.

Double subtraction When the "bad" objects overlap (a subarray can contain two different forbidden patterns), subtracting each kind separately removes some objects twice. Either make the bad cases disjoint by definition, as the maximal runs above are, or use inclusion–exclusion.

2.4 Reverse time and go offline

GOLD

Some operations are easy in one direction and hard in the other. A disjoint set union merges components in almost constant time but cannot split them; a set supports insertion and deletion, but a max-heap cannot delete an arbitrary element. When the problem gives you every operation in advance, you can process them in whatever order is convenient, including *backwards*. Deletions then become insertions.

Example: Network Breakdown (CSES)

A network has n computers and m connections. Connections are removed one at a time, k times; after each removal, print the number of connected components. Limits: $n \le 10^5$, $m, k \le 2 \cdot 10^5$.

Forwards, every removal can split a component, and nothing simple maintains that. Backwards, start from the final network (all k connections already removed), compute its components with a DSU, then add the removed connections back in reverse order. The number of components before adding connection i back is exactly the answer after removal i (Figure 2.4).

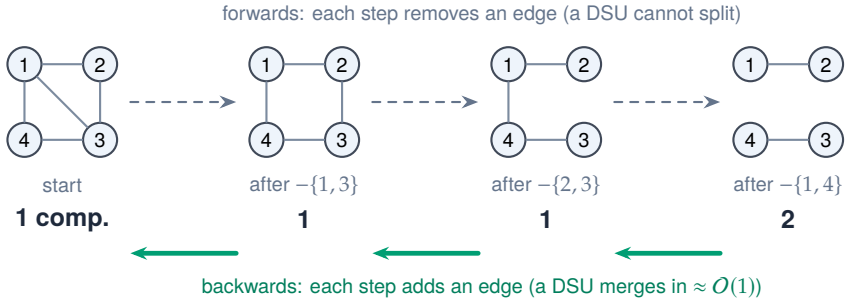


Figure 2.4: Forwards, the network loses edges; backwards, it gains them. Starting from the last state and adding the edges back in reverse order visits the same states, so the component counts can be read off the DSU. Output: `1 1 2`.

```
void solve() {
    int n, m, k;
    cin >> n >> m >> k;
    set<pii> edges;
    rep(i, m) {
        int a, b;
        cin >> a >> b;
        edges.insert({min(a, b), max(a, b)});
    }
    vector<pii> removed(k);
    for (auto& [a, b] : removed) {
        cin >> a >> b;
        if (a > b) swap(a, b);
        edges.erase({a, b});
    }
    vi par(n + 1);
    iota(all(par), 0);
    auto root = [&](int x) {
        while (x != par[x]) x = par[x] = par[par[x]];
        return x;
```

```
    };
    int comps = n;
    auto unite = [&](int a, int b) {
        a = root(a), b = root(b);
        if (a != b) par[a] = b, comps--;
    };
    for (auto [a, b] : edges) unite(a, b);  // the graph after every removal
    vi ans(k);
    per(i, k) {  // play the removals backwards: each one becomes an insertion
        ans[i] = comps;
        unite(removed[i].first, removed[i].second);
    }
    print(ans);
}
```

Offline processing is the general idea: when all queries are known before you must answer any of them, you may reorder them. Sort queries by their right endpoint and sweep the array once (distinct values in ranges, Chapter 23); sort them into blocks and move two pointers (Mo's algorithm); answer them in a divide-and-conquer over time (CDQ); or process them in reverse. Online problems, where each query depends on the previous answer (often through an XOR with the last answer), deliberately forbid this.

> **How to recognize it** Deletions that would be insertions if time ran backwards; queries that are all given before any answer is needed; an operation your data structure supports in only one direction.

2.5 Events and sweeps

SILVER

Problems about intervals in time or space become simple when you stop thinking about the intervals and start thinking about their *endpoints*. Each interval becomes two events, "something starts" and "something ends"; sort the events and walk through them in order, maintaining the current state. This is a **sweep**, and in two dimensions it becomes the sweep line of computational geometry (Chapter 28).

Example: Restaurant Customers (CSES)

Customer i is in the restaurant from time a_i to b_i. What is the largest number of customers present at the same time? Limits: $n \le 2 \cdot 10^5$, all times distinct.

Turn each customer into an arrival (+1) and a departure (−1), sort the $2n$ events by time, and keep a running sum. The running sum is the number of customers present, and its maximum is the answer (Figure 2.5). Sorting costs $O(n \log n)$; the walk is linear. If equal times were allowed, the order of simultaneous events would matter: to count someone who leaves at time t and someone who arrives at t as *not* overlapping, process departures first. Sorting pairs $(t, \pm 1)$ does exactly that, since $-1 < +1$.

```
void solve() {
    int n;
    cin >> n;
    vector<pii> ev;  // (time, +1 for an arrival, -1 for a departure)
```

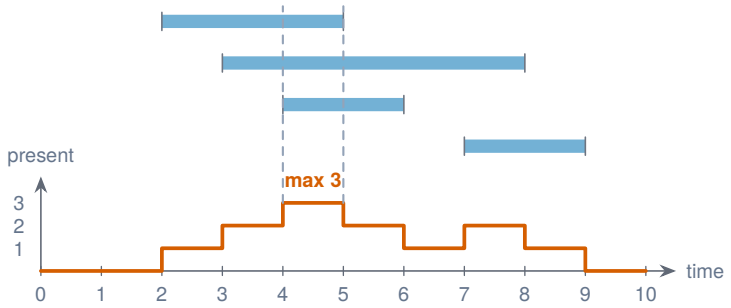


Figure 2.5: Four customers (bars) and the number present over time (the step function below). Each arrival lifts the step by one and each departure lowers it; the maximum, 3, is reached between times 4 and 5.

```
    rep(i, n) {
        int a, b;
        cin >> a >> b;
        ev.push_back({a, 1}), ev.push_back({b, -1});
    }
    sort(all(ev));
    int cur = 0, best = 0;
    for (auto [t, d] : ev) chmax(best, cur += d);
    print(best);
}
```

Coordinate compression plus a difference array When the times are small integers (say up to 10^6), skip the sort: add $+1$ at a_i and -1 at b_i in an array and take prefix sums (a difference array, Chapter 9). When they are large but few, compress them first. The sweep and the difference array are the same idea seen from two sides.

2.6 Invariants and colourings

SILVER

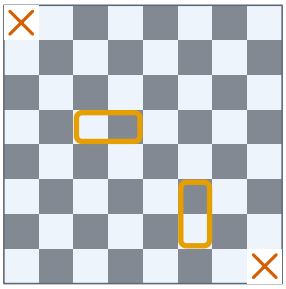
An **invariant** is a quantity that an operation never changes. If the start and the goal have different values of an invariant, the goal is unreachable, no matter how clever the sequence of operations. Invariants are how contest problems prove that something is *impossible*, and many "print `NO` or a construction" problems are really "find the invariant, then show everything else is reachable".

Example: The mutilated chessboard

Remove two opposite corner squares from an 8×8 board. Can the remaining 62 squares be tiled by 31 dominoes?

Solution. Colour the board like a chessboard (Figure 2.6). Every domino covers one dark and one light square, wherever it is placed: that is the invariant. Opposite corners have the same colour, so the mutilated board has 30 squares of one colour and 32 of the other. Thirty-one dominoes would need 31 of each. Impossible.

Parity is the most common invariant. If an operation swaps two elements of a permutation, the parity of the number of inversions flips every time; so whether a target permutation is reachable in an even number of swaps is decided before you start (this is why half the configurations of the 15-puzzle are unsolvable). If an operation replaces two numbers by their sum and difference, the



Every domino (orange) covers one dark and one light square, horizontal or vertical. Both removed corners (red crosses) are dark, leaving 30 dark and 32 light squares: no tiling can exist.

Figure 2.6: A colouring invariant. The count "dark squares covered minus light squares covered" is zero for any set of dominoes, but it would have to be -2 here.

parity of the total sum never changes. Other common invariants are a sum modulo some m, the greatest common divisor of all elements, the number of connected components, and weighted sums like $\sum_i a_i \cdot 2^i$.

> **Finding the invariant** Apply the operation to small examples and write down what stays the same: the sum, its parity, the sum mod 3, the multiset of values, the gcd, the number of odd elements. When the problem colours something, try colouring it yourself: chessboard colouring for dominoes, colouring by x mod 3 or $(x + y)$ mod k for longer tiles.

2.7 Monovariants and termination

GOLD

A **monovariant** is a quantity that changes in only one direction: every operation strictly decreases it, say, and it cannot go below zero. Then the process must stop, and the number of steps is at most the starting value. Monovariants are how you prove that a local-improvement algorithm terminates, and they bound its running time.

A classic example: "while some adjacent pair is out of order, swap it". Each swap removes exactly one inversion (a pair $i < j$ with $a_i > a_j$), and the number of inversions is at most $\binom{n}{2}$, so the process ends after exactly *inversions* swaps. That count is also the minimum number of adjacent swaps that sorts the array, because no single swap can remove more than one inversion. Counting inversions quickly (merge sort or a Fenwick tree, $O(n \log n)$) thus answers "how many adjacent swaps are needed to sort?".

Monovariants also justify local search. "While some vertex has more than half of its neighbours on its own side, move it to the other side" terminates, because each move strictly increases the number of edges between the two sides, which is at most m. When it stops, every vertex has at least half of its neighbours across, which proves that a cut of at least $m/2$ edges always exists, and finds one in at most m moves.

> **How to recognize it** "Show that the process ends", "how many operations until nothing changes", or a local-improvement algorithm whose running time you need to bound. Look for something that strictly improves: a sum, a count of bad pairs, a potential.

2.8 Pigeonhole

SILVER

The **pigeonhole principle**: if $n+1$ objects go into n boxes, some box holds two of them. It is as obvious as it sounds and as powerful as any theorem in this book, because it proves that something *exists* without constructing it, and the proof usually shows how to find it.

Example: Subarray Divisibility (CSES)

Count the subarrays of $a_1, \ldots, a_n$ whose sum is divisible by n ($n \le 2 \cdot 10^5$, values may be negative).

Solution. The subarray $a_{l+1}, \ldots, a_r$ has sum $P_r - P_l$, which is divisible by n exactly when the prefix sums P_l and P_r have the same remainder mod n. Put the $n+1$ prefix sums $P_0, \ldots, P_n$ into n boxes by remainder (Figure 2.7); every pair inside one box is a good subarray. The pigeonhole principle adds a bonus: since there are $n+1$ prefix sums and only n remainders, *some* box has two, so a divisible subarray always exists.

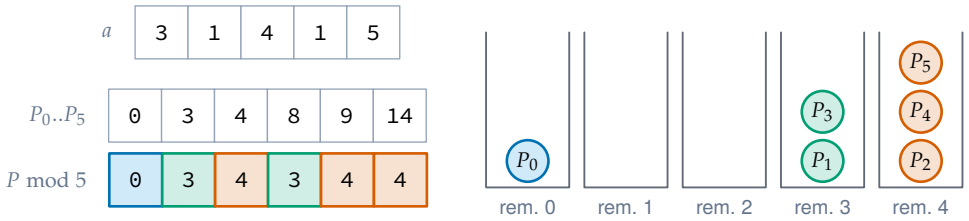


Figure 2.7: $n = 5$: six prefix sums and five possible remainders, so some box must hold two. Box 3 holds P_1, P_3 (the subarray $1, 4$); box 4 holds P_2, P_4, P_5 (the subarrays $4, 1$ and $4, 1, 5$ and 5). In total $\binom{2}{2} + \binom{3}{2} = 4$ subarrays have a sum divisible by 5.

```
void solve() {
    int n;
    cin >> n;
    vi cnt(n, 0);  // how many prefixes so far have each remainder mod n
    cnt[0] = 1;
    ll pre = 0, ans = 0;
    rep(i, n) {
        ll x;
        cin >> x;
        pre = ((pre + x) % n + n) % n;
        ans += cnt[pre]++;  // pair with earlier prefixes of the same remainder
    }
    print(ans);
}
```

Pigeonhole arguments also bound search spaces. A sequence defined by $x_{i+1} = f(x_i)$ over m possible values repeats within $m+1$ steps, so it is eventually periodic (Chapter 11, functional graphs). Among any $n+1$ integers in $[1, 2n]$, two are coprime (two are consecutive). Among 5 points in a 2×2 square, two are at distance at most $\sqrt{2}$. Whenever a problem says "prove that there always exists", look for boxes and pigeons.

2.9 The extremal principle

SILVER

Look at the largest, the smallest, the first or the last. An extreme object has properties that ordinary objects do not, because nothing can exceed it. Many existence proofs, and many greedy algorithms, start by picking an extreme object and showing it must behave nicely.

Example: Every tree has two leaves

Show that every tree with at least two vertices has at least two leaves (vertices of degree 1).

Solution. Take a *longest* path in the tree, with endpoints u and v (Figure 2.8). If u had a neighbour w other than the next vertex on the path, then either w is on the path (and the tree would contain a cycle) or it is not (and prepending w would make a longer path). Both are impossible, so u has degree 1; the same holds for v.

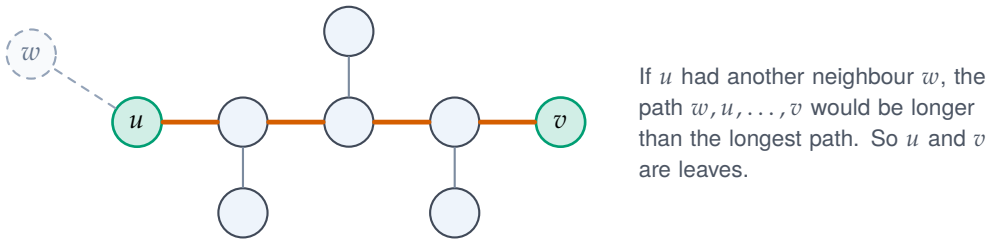


Figure 2.8: The extremal principle: the endpoints of a longest path cannot have any other neighbour.

The same move appears everywhere in this book. The leftmost point of a point set is on its convex hull. The interval that ends first is in some optimal schedule (Section 1.5). In a graph where every vertex has degree at least d, a longest path's endpoint has all its neighbours on the path, so the graph has a cycle of length at least $d + 1$. The smallest counterexample to a claim ("consider a minimal counterexample") is the extremal principle applied to proofs by contradiction, and it is exactly how induction proofs are usually discovered.

2.10 Symmetry and reflection

GOLD

Symmetry lets you count one representative and multiply, or count everything and divide. Unordered pairs are ordered pairs divided by two. "Assume without loss of generality that $a \le b$" halves the casework. Rotations of a necklace are counted by Burnside's lemma (Chapter 25). The most beautiful use of symmetry in contest counting is the **reflection principle**.

Example: Balanced bracket sequences

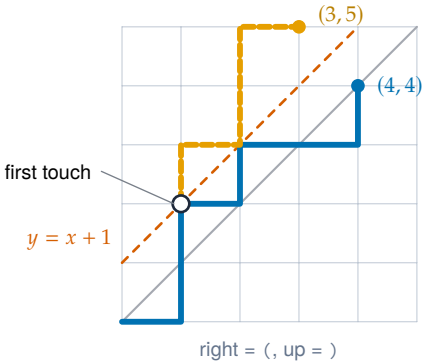
How many sequences of n opening and n closing brackets are balanced (no prefix has more closing than opening brackets)?

Solution. Draw a sequence as a lattice path from $(0, 0)$ to (n, n): an opening bracket is a step right, a closing one a step up. The sequence is balanced exactly when the path never goes above the diagonal, i.e. never touches the line $y = x + 1$. Count the bad paths: reflect the part of a bad path *after its first touch* of $y = x + 1$ across that line (Figure 2.9). The result is a path to $(n - 1, n + 1)$, and every path to $(n - 1, n + 1)$ arises from exactly one bad path this way (it must cross the line, and reflecting back undoes the map). So there are $\binom{2n}{n-1}$ bad paths, and

the balanced ones number

$$\binom{2n}{n} - \binom{2n}{n-1} = \frac{1}{n+1}\binom{2n}{n},$$

the Catalan number C_n.



Blue: the bad path ( )) ( ) (( ), which touches $y = x + 1$ (red, dashed) at the white dot. Swapping right and up steps after that point reflects the rest of the path (orange) and moves its end from $(4, 4)$ to $(3, 5)$. Every path to $(3, 5)$ arises this way exactly once.

Figure 2.9: The reflection principle. Reflecting the tail of a bad path across $y = x + 1$ moves its endpoint from (n, n) to $(n - 1, n + 1)$, which turns "count the bad paths" into a single binomial coefficient.

Fact The reflection principle counts lattice paths that avoid a line for any start and end point, and it solves ballot problems ("candidate A is always strictly ahead during the count") and bounded random walks. With several forbidden lines it becomes an alternating sum of reflections.

2.11 When the answer is small, enumerate it

PLATINUM

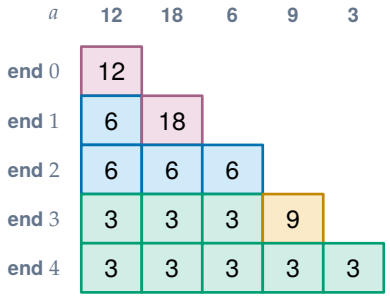
Sometimes a quantity that looks like it could be huge is provably tiny, and then you can simply list all its values. The classic source is *divisibility*: a chain of numbers where each divides the previous one can only have $\log_2 A + 1$ distinct values below A, because each strict step at least halves the value.

Example: Subarrays with gcd 1

Count the subarrays whose greatest common divisor is 1 ($n \le 2 \cdot 10^5$, values up to 10^9).

Solution. Fix the right end i and look at the gcds of $a_l, \ldots, a_i$ as l moves left: each is a divisor of the previous one, so there are at most $\log_2 10^9 + 1 \approx 31$ distinct values (Figure 2.10). Store them as (value, count) pairs; moving to $i + 1$ means taking the gcd of every stored value with a_{i+1} and merging equal neighbours. The total cost is $O(n \log A)$ gcd computations instead of $O(n^2)$ subarrays.

```
void solve() {
    int n;
    cin >> n;
    vi a(n);
    cin >> a;
    vector<pll> cur;  // (gcd, count) for subarrays ending at i: a short chain
    ll ones = 0;
    rep(i, n) {
```



Row i lists $\gcd(a_l, \ldots, a_i)$ for every start l. Reading any row from right to left, each value divides the one before it, so a row has at most $\log_2 A + 1$ *distinct* values, however long it is. Row 3 is the chain $9 \to 3$ with multiplicities $1, 3$.

Figure 2.10: Subarray gcds of $12, 18, 6, 9, 3$, one row per right end. Colours mark equal values; every row is a short chain of divisors, which is what makes the $O(n \log A)$ algorithm possible.

```
        vector<pll> nxt{{a[i], 1}};
        for (auto [g, c] : cur) {
            ll h = gcd(g, a[i]);
            if (nxt.back().first == h) nxt.back().second += c;
            else nxt.push_back({h, c});
        }
        cur = nxt;
        for (auto [g, c] : cur)
            if (g == 1) ones += c;
    }
    print(ones);
}
```

The same argument works for bitwise OR and AND over subarrays (each new element can only add or remove bits, at most 30 times), for the number of distinct values of $\lfloor n/i \rfloor$ (at most $2\sqrt{n}$, Chapter 24), and for the number of distinct subtree sizes along a root path in heavy-light decomposition ($O(\log n)$ light edges). A second family of small answers comes from sums: the harmonic sum $\frac{n}{1} + \frac{n}{2} + \cdots + \frac{n}{n} \approx n \ln n$ means that "for each d, visit every multiple of d" costs only $O(n \log n)$ in total.

> **Swap the loops** "For each x, iterate over its divisors" costs $O(n\sqrt{n})$ if done naively, but "for each d, iterate over its multiples" touches the same pairs in $O(n \log n)$ by the harmonic sum. Whenever a double loop's inner range depends on the outer variable, ask whether the other order is cheaper.

2.12 Heavy and light: the square-root split

PLATINUM

When some objects are big and others small, a single method is often bad for one of the groups: iterating over an object costs its size, and precomputing an answer for an object costs a fixed amount. Split them at a threshold B: handle *light* objects (size below B) by brute force and *heavy* ones (size at least B) by precomputation. If the total size is S, there are at most S/B heavy objects, and choosing $B = \sqrt{S}$ balances the two costs.

Example: Set intersections

There are n sets of integers with total size $S \le 2 \cdot 10^5$, and q queries "how many elements do sets x and y have in common?".

Solution. If one of the two sets is light, iterate over it and look each element up in a hash set of the other: $O(B)$ per query. If both are heavy, answer from a table: for each heavy set h, mark its elements and scan every other set once, recording $|h \cap s|$ for all s; that is $O(S)$ per heavy set and $O(S \cdot S/B)$ in total. With $B = \sqrt{S}$ everything costs $O((S+q)\sqrt{S})$ (Figure 2.11).

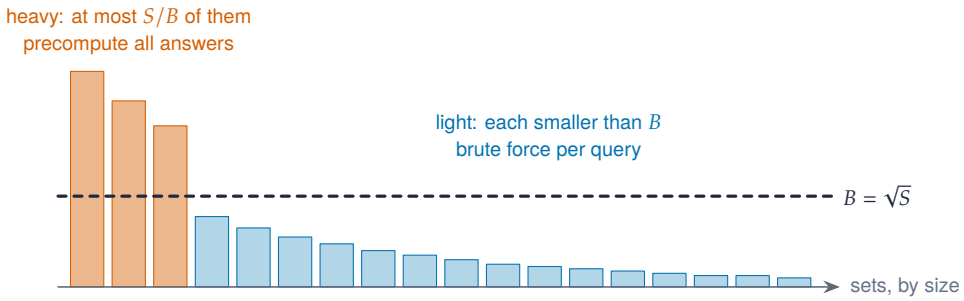


Figure 2.11: The square-root split. Few objects can be large (their sizes add up to at most S), so they can afford expensive precomputation; the many small ones are cheap to handle directly.

The split appears in graphs (vertices of degree above $\sqrt{m}$ are heavy; there are at most $2\sqrt{m}$ of them, which gives the $O(m\sqrt{m})$ triangle-counting algorithm), in strings (patterns longer than $\sqrt{L}$ are few), and in queries (rebuild a structure every $\sqrt{q}$ updates and brute force the pending ones, Chapter 23). It rarely gives the fastest possible algorithm, but it often turns a problem nobody can see how to solve into one that is merely careful work.

2.13 Differences instead of values

SILVER

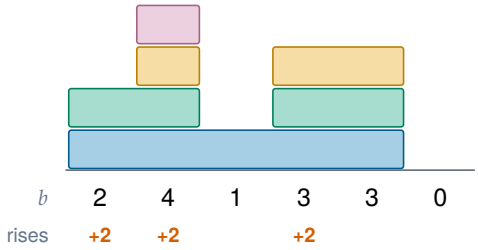
Operations on ranges of an array are awkward to reason about directly, but in the array of *differences* $d_i = b_i - b_{i-1}$ they become local: adding 1 to $b_l, \ldots, b_r$ changes only d_l (up by one) and d_{r+1} (down by one). Many "minimum number of range operations" problems collapse once rewritten this way.

Example: Range increments

Starting from an array of zeros, one operation adds 1 to every element of a subarray. What is the minimum number of operations that produce a given target b ($b_i \ge 0$)?

Solution. With $b_0 = 0$ before the array, each operation raises one difference by one and lowers another by one (or none, at the end). The positive differences sum to $\sum_i \max(0, b_i - b_{i-1})$, and every operation raises that sum by at most one, so at least that many operations are needed. It is also enough: peel the target into horizontal layers (Figure 2.12); each maximal horizontal run in a layer is one operation, and each run starts exactly at a rise.

```
void solve() {
    int n;
    cin >> n;
    ll prev = 0, ops = 0;
    rep(i, n) {
```



Each coloured bar is one operation (a maximal run inside one layer). There are $1+2+2+1=6$ of them, and every bar starts at a position where b rises: the rises $2+2+2=6$ give the answer directly.

Figure 2.12: Target $2, 4, 1, 3, 3, 0$ cut into horizontal layers. The number of runs equals the sum of the positive differences $b_i - b_{i-1}$, and no sequence of range increments can do better.

```
        ll b;
        cin >> b;
        ops += max(0LL, b - prev);  // a rise of d starts d new ranges here
        prev = b;
    }
    print(ops);
}
```

Differences also turn "add x to a range" into two point updates (the difference array of Chapter 9), turn arithmetic progressions into constants (take differences twice), and make "all elements equal" into "all differences zero". The inverse operation, prefix sums, turns a range query into two point queries. Keeping both views in mind is what makes range problems tractable.

2.14 Reductions to known problems

GOLD

The fastest way to solve a problem is to notice that it has already been solved. A large share of contest problems are classic problems wearing a costume, and part of training is building a mental catalogue of costumes.

The problem says...	...and is really
fewest non-increasing sequences covering an array	longest strictly increasing subsequence (Dilworth), Ch. 18
boxes that nest inside each other, maximise a chain	sort by one side, LIS on the other
assign workers to jobs, each to at most one	bipartite matching or min-cost flow, Ch. 15
remove fewest vertices so no conflicting pair remains	vertex cover; on bipartite graphs, a matching (Kőnig)
variables with "$x_u - x_v \le c$" constraints	shortest paths with negative edges, Ch. 12
each variable true or false, clauses of two literals	2-SAT via strongly connected components, Ch. 14
minimum cost to make a sequence non-decreasing	slope trick, Ch. 20
minimise a maximum edge on a path	minimum spanning tree (its path is optimal), Ch. 13
states and moves, fewest moves	BFS on an implicit graph, Ch. 11
choose a subset with a sum constraint, maximise value	knapsack, Ch. 18

Tip When a problem looks new, restate it with all names removed, then ask: what is the underlying object (sequence, graph, set system), what is the constraint, what is optimised? Compare that skeleton with the classics above and with the recognition table of Chapter 30.

Chapter summary

- Count by **contribution** and by **complement**; fix the part that makes the rest independent.
- Reorder time: go **offline**, run it **backwards**, or **sweep** over sorted events.
- Prove impossibility with **invariants** and termination with **monovariants**; prove existence with **pigeonhole**, **extremal** objects and **reflection**.
- Exploit **small answers** (divisor chains, harmonic sums), the **heavy/light** split, and **differences** for range operations; above all, look for a known problem in disguise.

Practice problems (CSES Problem Set)

CONTRIBUTION AND COMPLEMENT

- Sum of Two Values
- Nearest Smaller Values
- Maximum Subarray Sum
- Subarray Sums II
- Distinct Numbers

OFFLINE, REVERSED, SWEEPS

- Restaurant Customers
- Network Breakdown
- Distinct Values Queries
- Room Allocation
- Nested Ranges Check

INVARIANTS, PIGEONHOLE, DIFFERENCES

- Subarray Divisibility
- Coin Piles
- Collecting Numbers
- Two Sets
- Increasing Array
- Bracket Sequences I

CHAPTER 3

Complexity and Constraints

A correct solution that is too slow scores exactly the same as no solution at all. Complexity analysis tells you, before you write a line, whether an idea can pass; it is the tool you use in step 2 of the problem-solving loop to throw away hopeless ideas and to recognise which technique the problem setter had in mind. This chapter teaches you to estimate the running time and memory of an algorithm quickly and reliably, and it gives you real measured costs of the operations you will use every day, so that "fast enough" stops being a guess.

IN THIS CHAPTER BRONZE SILVER GOLD

- count operations and express them in O-notation;
- read a problem's limits and deduce the complexity it expects;
- analyse loops, including harmonic sums, square-root loops and amortized costs;
- solve recurrences with recursion trees and the master theorem;
- budget memory, and know what each C++ operation really costs, measured.

Before you start: Chapter 1.

3.1 Counting operations BRONZE

A modern judge executes on the order of 10^8 to 10^9 *simple operations* per second: additions, comparisons, array accesses, branches. The exact number depends on the machine and on what the operations are (Section 3.8 measures them), but for planning purposes one rule is enough:

> **The budget** Plan for about 10^8 simple operations per second of time limit. Below 10^7 you are safe by a wide margin; around 10^8 you are fine if the inner loop is simple; above 10^9 you will not pass without a very special trick.

So the question "will it pass?" becomes "how many operations does it do on the largest input?". For loops that is mostly multiplication. The code below does work once for every pair $i < j$:

```
for (int i = 0; i < n; i++)          // n iterations
    for (int j = i + 1; j < n; j++)  // n-1, n-2, ..., 1, 0 iterations
        work(i, j);                  // in total n(n-1)/2 calls
```

For $n = 5000$ that is $1.25 \cdot 10^7$ calls: instant. For $n = 2 \cdot 10^5$ it is $2 \cdot 10^{10}$: over a minute on any judge. The same idea is fine or hopeless depending on a single number in the statement, which is why you always compute the count *with the real limit plugged in*.

Counting is rarely exact, and it does not need to be: you only need the order of magnitude. Three habits make it quick. First, count the innermost operation and multiply by the number of times each enclosing loop runs. Second, round generously: $\log_2(2 \cdot 10^5) \approx 18$, $\sqrt{2 \cdot 10^5} \approx 450$, $2^{20} \approx 10^6$.

Third, remember that a "simple operation" inside a library call may itself be a loop: `s.count(x)` on a `vector` is n operations, not one.

3.2 Big-O notation

BRONZE

Counting exactly is tedious and machine-dependent, so we describe how the count *grows* with the input size and ignore the rest. We say that an algorithm runs in time $O(f(n))$ if, for all large enough n, it performs at most $c \cdot f(n)$ operations for some fixed constant c. Two companions complete the vocabulary: $\Omega(f(n))$ means "at least $c \cdot f(n)$", and $\Theta(f(n))$ means both, i.e. exactly this order of growth. In contests people say O even when they mean Θ; the meaning is usually clear.

The rules follow from the definition. Constant factors disappear: $5n$ and $n/3$ are both $O(n)$. Lower-order terms disappear: $3n^2 + 10n + 7$ is $O(n^2)$, because for large n the n^2 term dominates everything else. Logarithm bases disappear too, since $\log_2 n = \log_2 10 \cdot \log_{10} n$ differs by a constant factor. With several inputs you keep each of them: BFS on a graph is $O(n + m)$, and neither term may be dropped, because either can be the larger one.

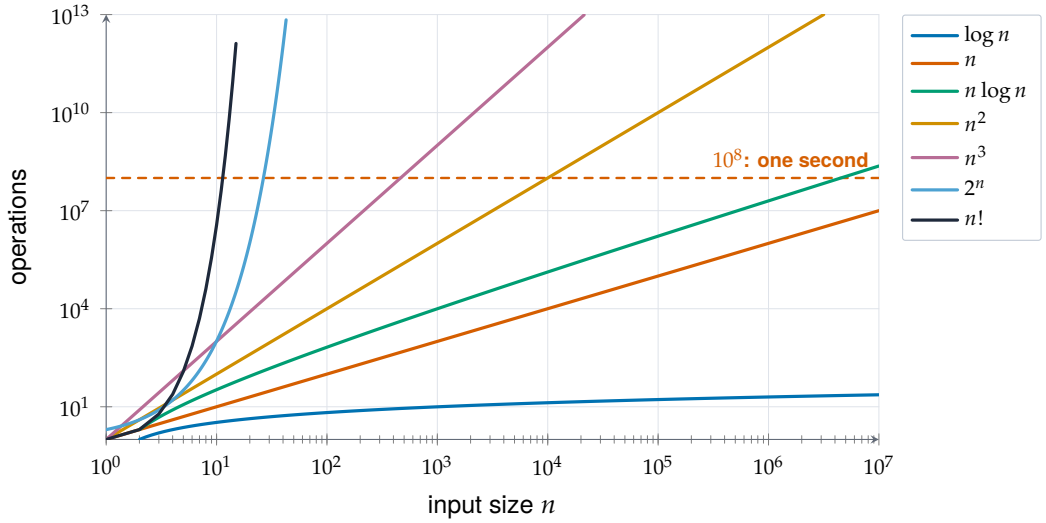


Figure 3.1: How fast common complexities grow (both axes logarithmic). Each curve crosses the one-second budget (dashed) at the largest n it can handle: $n!$ near 11, 2^n near 27, n^3 near 460, n^2 near 10^4, $n \log n$ near $4.5 \cdot 10^6$. Polynomials are straight lines in this view; exponentials bend upwards and escape almost immediately.

> **Big-O hides constants, and constants can be big** $O(n)$ with a constant of 100 is slower than $O(n \log n)$ with a constant of 1 for every n you will meet. Big-O tells you which of two ideas *scales* better, not which is faster on today's input. When two ideas have the same complexity, the constant factor (Section 3.8) decides.

Worst case, average case, amortized. Unless stated otherwise, complexity means the *worst case*: the most operations over all inputs of size n. That is what matters in contests, because the tests are chosen by someone trying to break you. *Expected* (average) complexity appears with randomised algorithms: quicksort with a random pivot, hash tables, treaps. *Amortized* complexity bounds the total cost of a sequence of operations even when single operations are occasionally expensive (Section 3.5).

3.3 What the limits tell you

BRONZE

Problem setters choose limits to allow the intended solution and to kill slower ones. Reading their intention from the limits is the single most useful skill in this chapter: find the largest input parameter in the statement, look it up in Table 3.1, and you know the target complexity and a short list of techniques before you have had a single idea. Figure 3.2 shows where each complexity runs out of budget.

Table 3.1: Limits and the complexities that fit in about one second.

Largest n	Complexity	Typical techniques
$n \le 10$	$O(n!), O(n^6)$	all permutations, brute force over everything
$n \le 20$	$O(2^n \cdot n)$	all subsets, bitmask DP (Chapter 19)
$n \le 40$	$O(2^{n/2} \cdot n)$	meet in the middle (Chapter 6)
$n \le 100$	$O(n^4)$	DP with four nested indices
$n \le 500$	$O(n^3)$	Floyd–Warshall, interval DP, Gaussian elimination
$n \le 5000$	$O(n^2)$	all pairs, two-dimensional DP, LCS
$n \le 10^5$	$O(n\sqrt{n})$	square-root decomposition, Mo's algorithm
$n \le 2 \cdot 10^5$	$O(n \log n), O(n \log^2 n)$	sorting, binary search, heaps, segment trees, Dijkstra
$n \le 10^6$	$O(n)$, or $O(n \log n)$ if simple	linear scans, two pointers, prefix sums, sieves
$n \le 10^9$	$O(\sqrt{n}), O(\log n)$	trial division, formulas, binary search
$n \le 10^{18}$	$O(\log n), O(\log^2 n), O(1)$	fast exponentiation, matrix power, digit DP, formulas

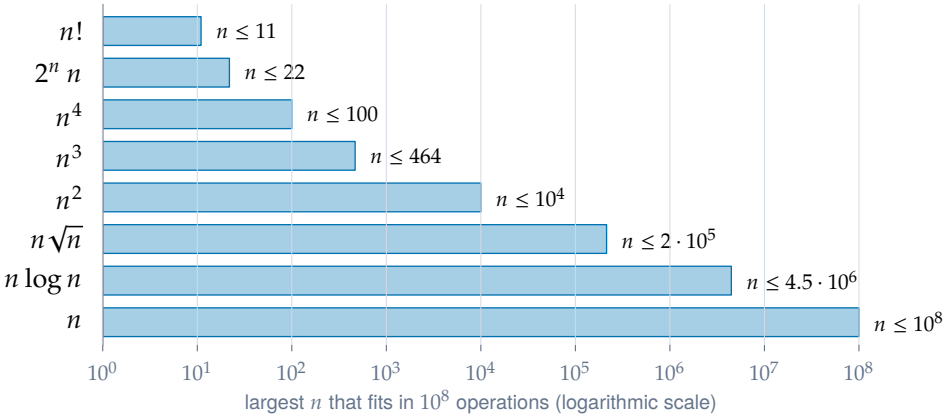


Figure 3.2: The largest input each complexity can process with about 10^8 operations. Moving one row up the table costs orders of magnitude of input size; this is why the limits reveal the intended complexity so reliably.

Several parameters. Statements often have more than one limit: n vertices and m edges, n elements and q queries, a grid of $r \times c$ cells. Multiply out the candidate complexity with all of them: $O((n+q)\log n)$ with $n = q = 2 \cdot 10^5$ is about $7 \cdot 10^6$; $O(nq)$ is $4 \cdot 10^{10}$. The two limits that look alike often play different roles: a grid with $r \cdot c \le 10^6$ but r up to 10^6 is a hint that the solution must not assume a square.

The small limit is the hint. When one limit is suspiciously small (values up to 100, at most 20 special vertices, an alphabet of size 3, $k \le 10$), the intended solution almost certainly iterates over it. "$n \le 2 \cdot 10^5$, $k \le 20$" strongly suggests $O(nk)$ or $O(n \cdot 2^k)$, never something independent of k.

Sums over test cases. "The sum of n over all test cases does not exceed $2 \cdot 10^5$" allows $O(n \log n)$ *per test*. It does not allow work that is independent of n: resetting a global array of size 10^6, or a loop to 10^6, costs 10^6 *per test case*, and there may be 10^4 tests. Reset only what you used.

Other time limits. A 2-second limit doubles the budget; half a second halves it. Limits far above one second (5 or 10 seconds) usually mean the intended solution is heavy, often $O(n\sqrt{n})$ or a large constant, and a clean implementation of it will pass comfortably. Limits far below usually mean that input and output are large, so fast I/O matters.

Memory limits. The usual limit is 256 MB, sometimes 512 MB or 1 GB, and occasionally as low as 16–64 MB to force a clever idea. Section 3.7 converts megabytes into numbers of elements.

3.4 Analysing loops

SILVER

Most running times are loops, and a small number of loop shapes cover almost everything.

Independent nested loops multiply. Two loops to n and m cost $n \cdot m$. Three nested loops to n cost n^3 even if the body is a single addition.

Dependent loops sum. If the inner loop's length depends on the outer variable, add up the lengths. The pair loop of Section 3.1 sums $0 + 1 + \cdots + (n-1) = n(n-1)/2$, still $\Theta(n^2)$; a triple of nested dependent loops sums to about $n^3/6$. Halving the constant is nice but does not change the class.

Loops that halve are logarithmic. `while (x > 0) x /= 2;` runs $\lfloor \log_2 x \rfloor + 1$ times. Binary search, fast exponentiation, Euclid's gcd algorithm and binary lifting all shrink the problem by a constant factor per step, so they take $O(\log n)$ steps. A loop that multiplies by 2 until it passes n is the same loop read backwards.

Square-root loops. `for (ll d = 1; d * d <= n; d++)` runs $\sqrt{n}$ times. It enumerates divisors in pairs $(d, n/d)$ and tests primality by trial division: 10^6 steps for $n = 10^{12}$.

The harmonic sum. Loops over multiples are cheaper than they look:

```
for (int d = 1; d <= n; d++)
    for (int x = d; x <= n; x += d)  // n / d iterations
        cnt[x]++;                     // then cnt[x] = number of divisors
```

The inner loop runs n/d times, so the total is $n\,(1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}) = n \cdot H_n \approx n \ln n$. For $n = 10^6$ that is $1.4 \cdot 10^7$ steps: this "for each d, visit every multiple" pattern is how the sieve of Eratosthenes, divisor counting and many number-theory precomputations run in $O(n \log n)$ or better (Chapter 24). The reverse, "for each x, try every d and check whether it divides", costs n^2.

Loops over submasks. Iterating over every submask of every mask of n bits looks like 4^n but is 3^n: each of the n bits is in one of three states (in the submask, in the mask only, in neither). For $n = 15$ that is $1.4 \cdot 10^7$ instead of 10^9 (Chapter 19).

> **Tip** When a loop's cost is not obvious, instrument it: count the iterations in a global variable on the largest input you can generate and print the count. Measuring beats guessing whenever the analysis gets subtle.

3.5 Amortized analysis

GOLD

Sometimes a single step is expensive but the steps *together* are cheap. Amortized analysis bounds the total cost of a whole sequence of operations instead of multiplying the worst single step by the number of steps.

3.5.1 The aggregate method

Count the total work directly. The standard example is `vector::push_back`. When the vector is full, it allocates twice the capacity and copies every element, which costs as much as the current size. Most pushes cost 1, but pushes number 2, 3, 5, 9, 17, ... copy 1, 2, 4, 8, 16, ... elements (Figure 3.3). Over n pushes the copies total at most $1 + 2 + 4 + \cdots < 2n$, so all n pushes cost less than $3n$: $O(1)$ amortized per push.

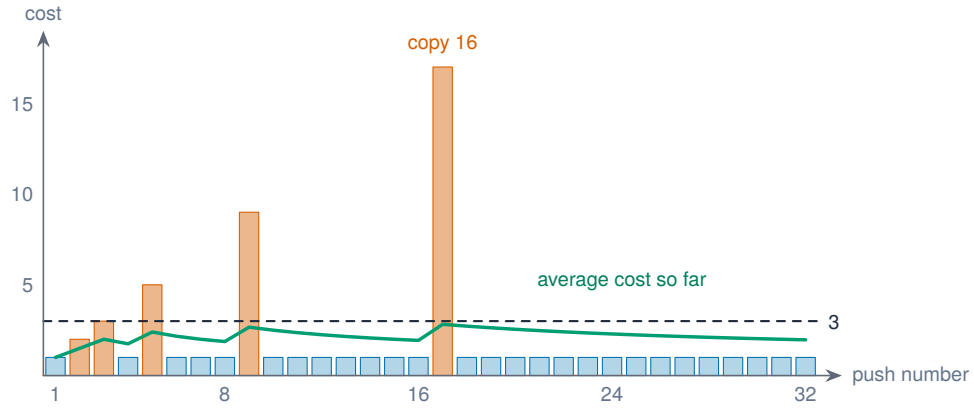


Figure 3.3: The cost of 32 `push_back` calls with capacity doubling. A few pushes are expensive (red: they copy the whole vector), but the average cost per push (green) never exceeds 3.

The same argument explains the two-pointer loop below. The inner `while` can run many times for a single `r`, but `l` only ever moves forward and never passes n, so over the whole loop the inner body runs at most n times in total. The loop is $O(n)$, not $O(n^2)$.

```
for (int l = 0, r = 0; r < n; r++) {
    add(a[r]);
    while (bad()) remove(a[l++]);  // l moves at most n times in total
}
```

3.5.2 The potential method

For more complicated structures, define a **potential** $\Phi \geq 0$ that measures "stored-up work", and charge each operation its real cost plus the change in potential: $\hat{c} = c + \Phi_{\text{after}} - \Phi_{\text{before}}$. Summed over a sequence, the potentials telescope, so the total real cost is at most the total charged cost

(as long as Φ starts at 0 and never goes negative). The art is choosing Φ so that every operation's charged cost is small.

For a monotonic stack (Chapter 9), take Φ = the number of elements on the stack. A push costs 1 and raises Φ by 1: charged cost 2. Popping k elements costs k and lowers Φ by k: charged cost 0. Every operation is charged at most 2, so n operations cost at most $2n$ in total, however the pops are distributed. The same method proves the $O(\log n)$ amortized bounds of splay trees, the linear time of the KMP prefix function, and the nearly constant time of union–find with path compression.

> **Amortized is not worst case** An amortized bound holds for the whole sequence, not for each operation. If a problem has multiple test cases and you "reset" a structure by rebuilding it, or if a data structure is persistent and an old expensive state can be revisited many times, the amortization can break. Rollback DSU (Chapter 13) avoids path compression for exactly this reason.

3.6 Recursion: trees and the master theorem

GOLD

A recursive algorithm that splits a problem of size n into a subproblems of size n/b and does $f(n)$ extra work (splitting and combining) takes time

$$T(n) = a\,T(n/b) + f(n).$$

The reliable way to solve such a recurrence is to draw its **recursion tree**: the root is the original call, each node has a children, and each node is labelled with its own extra work. Adding up the work level by level gives the total (Figure 3.4).

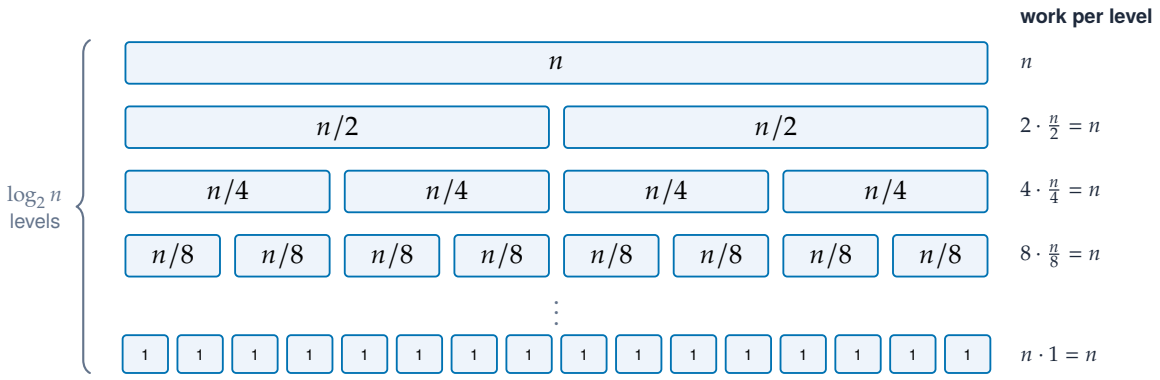


Figure 3.4: The recursion tree of merge sort, $T(n) = 2T(n/2) + \Theta(n)$. Every level does $\Theta(n)$ work in total and there are $\log_2 n$ levels, so $T(n) = \Theta(n \log n)$.

Whether the root, the leaves, or all levels together dominate depends on how the work per level changes as you go down (Figure 3.5). That gives the three cases of the **master theorem**:

> **Master theorem** Let $T(n) = a\,T(n/b) + \Theta(n^c)$ and compare c with the critical exponent $\log_b a$.
>
> - If $c > \log_b a$, the work shrinks down the tree and the root dominates: $T(n) = \Theta(n^c)$.
> - If $c = \log_b a$, every level does the same work: $T(n) = \Theta(n^c \log n)$.
> - If $c < \log_b a$, the work grows down the tree and the leaves dominate: $T(n) = \Theta(n^{\log_b a})$.

Some recurrences do not fit the theorem but are easy with a tree anyway. Linear recursion $T(n) = T(n-1) + \Theta(n)$ (a quicksort that always picks the worst pivot, or selection sort) has n levels

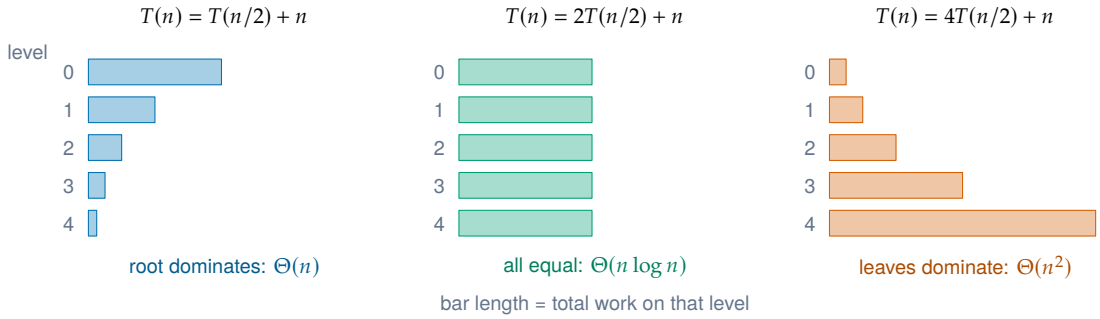


Figure 3.5: Work per level of the recursion tree in the three cases of the master theorem. Geometric decrease means the top level dominates; a constant means $\log n$ equal levels; geometric increase means the leaves dominate.

with $n, n-1, \ldots$ work: $\Theta(n^2)$. $T(n) = 2T(n-1) + 1$ doubles the number of calls per level: $\Theta(2^n)$, which is why naive recursive Fibonacci is hopeless. Memoised recursion is analysed differently: its cost is (number of distinct states) $\times$ (work per state), the same as the equivalent DP (Chapter 18).

Algorithm	Recurrence	Solution
binary search	$T(n) = T(n/2) + \Theta(1)$	$\Theta(\log n)$
merge sort, divide-and-conquer DP	$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \log n)$
building a segment tree bottom-up	$T(n) = 2T(n/2) + \Theta(1)$	$\Theta(n)$
Karatsuba multiplication	$T(n) = 3T(n/2) + \Theta(n)$	$\Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$
Strassen matrix multiplication	$T(n) = 7T(n/2) + \Theta(n^2)$	$\Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$
CDQ divide and conquer with a Fenwick tree	$T(n) = 2T(n/2) + \Theta(n \log n)$	$\Theta(n \log^2 n)$
naive recursive Fibonacci	$T(n) = T(n-1) + T(n-2) + \Theta(1)$	$\Theta(\varphi^n)$, $\varphi \approx 1.618$

3.7 Memory

BRONZE

Memory limits are checked as strictly as time limits, and running out gives *memory limit exceeded* or a runtime error. Budgeting memory is simple multiplication once you know the sizes.

Type	Bytes	How many fit in 256 MB
`char`, `bool`	1	$2.6 \cdot 10^8$ (a `vector<bool>` or `bitset` stores 8 per byte)
`int32_t`, `float`	4	$6.7 \cdot 10^7$
`long long`, `double`	8	$3.3 \cdot 10^7$
`pair<ll, ll>`, `array<ll, 2>`	16	$1.6 \cdot 10^7$
`vector` header (empty vector)	24	plus the heap block it owns
`set` / `map` node	about 40–48	about $5 \cdot 10^6$ elements
`unordered_map` entry	about 40–64	fewer than a `map`; plus the bucket array

> **⚠ Your template doubles every int** With `#define int long long`, a `vector<int>` of 10^7 elements takes 80 MB instead of 40 MB, and a DP table of 5000×5000 ints takes 200 MB. For big arrays that only hold small values, write `int32_t` or `char` explicitly; the macro does not touch them.

Two memory traps have nothing to do with element sizes. The first is *many small vectors*: a `vector<vector<int>>` with 10^6 rows spends 24 bytes per row on headers alone, plus a separate heap allocation (with its own overhead of 16 bytes or more) for every non-empty row, and its rows are scattered across memory, which also makes it slow (Figure 3.6). When the rows are fixed after reading the input, a flat array with offsets (compressed sparse rows) uses a fraction of the memory and is several times faster to scan.

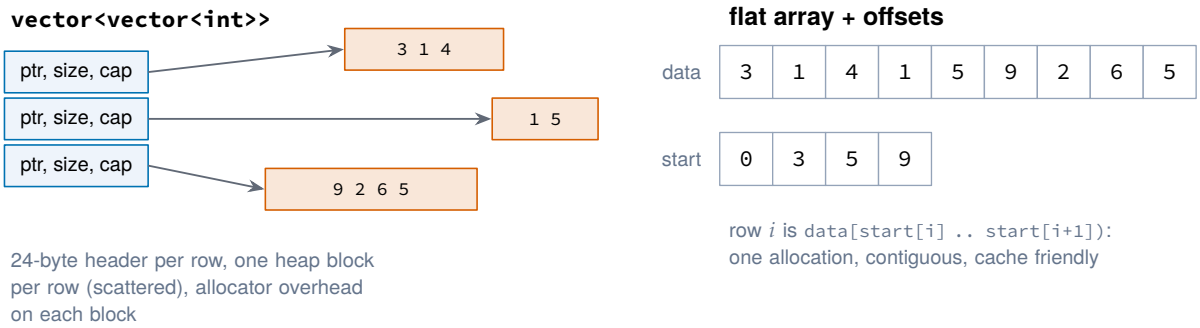


Figure 3.6: The same three rows stored two ways. A vector of vectors pays a header and an allocation per row and scatters the rows; a flat array with offsets stores only the data and one integer per row.

The second trap is the *stack*. Local arrays and recursion live on the call stack, which on many judges is limited to 8 MB or less, though Codeforces and your local setup use 256 MB. A recursive DFS on a path of 10^6 vertices needs a million stack frames; with 50–100 bytes per frame it fits in 256 MB but not in 8 MB. Big arrays belong in global variables or vectors, never in local arrays; deep recursion should become an explicit stack when the judge is unknown.

3.8 How fast is an operation, really?

GOLD

"10^8 operations per second" is an average over very different operations. Table 3.2 was measured on a 2026 laptop with GCC 15 and `-O2`, the same flags as most judges; judges differ from it by a factor of about two in either direction, but the *ratios* are what matter, and they are stable across machines.

Four lessons stand out. **Memory access patterns dominate**: a random access into a large array costs as much as a hundred additions, because it misses the cache and waits for main memory. Scanning arrays in order is nearly free. **Balanced trees are slow**: every `set` or `map` operation is a chain of dependent cache misses down a tree, so 10^6 operations take almost half a second; a sorted `vector` with `lower_bound`, or an array indexed by compressed values, is often ten times faster. **Division is expensive** relative to addition: reduce modulo only when needed. **I/O is not free**: reading 10^6 numbers takes 0.08 s even with fast settings, and many times more without them.

3.9 Constant factors that matter

GOLD

Two $O(n \log n)$ solutions can differ by a factor of twenty in running time, and near the limit that factor decides between accepted and *time limit exceeded*. The usual culprits, in order of how often they matter:

Table 3.2: Measured cost of common operations (GCC 15, `-O2`, averaged over 10^6 to 10^9 repetitions).

Operation	Time per operation	Operations per second
reading an array sequentially (the compiler vectorizes the loop)	0.1 ns	10^{10}
dependent add / xor / shift in a loop	0.6 ns	$1.6 \cdot 10^9$
`vector::push_back` (amortized)	1.5 ns	$6.6 \cdot 10^8$
`x = x * c % MOD` with 64-bit numbers	2.9 ns	$3.4 \cdot 10^8$
`std::sort` of 10^6 ints, per element	43 ns	$2.3 \cdot 10^7$
`priority_queue` push + pop, per element	60 ns	$1.7 \cdot 10^7$
reading one integer with `cin` (sync off)	80 ns	$1.2 \cdot 10^7$
random access into a 64 MB array (cache miss)	88 ns	$1.1 \cdot 10^7$
`unordered_map::operator[]`, with / without `reserve`	135 / 215 ns	$5–7 \cdot 10^6$
`set::insert`, `map::operator[]`	400–440 ns	$2.4 \cdot 10^6$
`set::count` on a set of 10^6 random keys	470–500 ns	$2 \cdot 10^6$

- **Slow I/O.** Without `ios::sync_with_stdio(false)` and `cin.tie(nullptr)`, reading 10^6 numbers can take a second; `endl` flushes the output buffer on every line. Your template sets up the fast mode; print `'\n'`, never `endl`, in loops.
- **Node-based containers.** Replace `set`/`map` by sorted vectors, prefix sums or arrays over compressed values whenever the data does not change after it is built.
- **Cache-hostile loops.** Traverse two-dimensional arrays row by row, never column by column. In matrix multiplication, the loop order i, k, j is several times faster than i, j, k.
- **Needless modulo.** Add, then subtract `MOD` if the sum reached it; accumulate several products in a 64-bit variable and reduce once.
- **Copies.** Passing a `vector` or `string` by value copies it on every call; pass `const&`.
- **Hidden allocation.** Creating a `vector` inside a hot loop allocates every time; hoist it out and `clear()` it.

Hidden linear costs These look like single operations but cost $O(n)$ each: `s = s + c` on a string (use `s += c`), `v.erase(v.begin())` and `v.insert(v.begin(), x)`, `std::distance` or `std::next` on a `set` iterator, `count` or `find` on a `vector`, `memset` of a whole array in every test case, `s.substr(...)` inside a loop, and `mp[key]`, which *inserts* a default element when the key is missing and slowly fills the map.

Bitsets divide by 64 A `bitset<N>` stores bits in 64-bit words, and `&`, `|`, `^`, shifts and `count()` process 64 bits per instruction. Any computation that is "OR of shifted boolean arrays" becomes 64 times faster: subset sum up to 10^5 over 10^4 items needs $10^9/64 \approx 1.6 \cdot 10^7$ word operations. Reachability in a DAG, triangle detection and longest common subsequence have bitset versions too (Chapter 29).

3.10 Estimating before coding: a worked example

SILVER

Suppose a problem has $n = 2 \cdot 10^5$ elements and $q = 2 \cdot 10^5$ queries, and you have three ideas.

1. **Answer each query by scanning the array**: $nq = 4 \cdot 10^{10}$. Rejected immediately, without coding.
2. **Mo's algorithm with a `map` inside**: $(n+q)\sqrt{n}$ pointer moves, each doing a `map` operation: $4 \cdot 10^5 \cdot 450 \approx 1.8 \cdot 10^8$ moves at 400 ns each is 72 seconds. Rejected: the complexity was fine but the constant is not (Section 3.8). With an array instead of the `map`, each move costs a few nanoseconds and the same idea runs in under a second.
3. **Offline sweep with a Fenwick tree**: $(n+q)\log n \approx 4 \cdot 10^5 \cdot 18 = 7 \cdot 10^6$ simple operations. Clearly fast; code this one.

Doing this arithmetic takes a minute and saves you from implementing the wrong idea. Notice that idea 2 was rejected for its constant, not its class: once you know the measured cost of your operations, "fast enough" is a calculation.

Chapter summary

- Budget about 10^8 simple operations per second; count the operations of your idea with the real limits.
- Read the intended complexity from the limits (Table 3.1); the smallest limit is often the hint.
- Harmonic sums, halving loops, submask loops and amortized arguments are cheaper than they look.
- Solve recurrences with a recursion tree; the master theorem compares c with $\log_b a$.
- Memory is multiplication: know your type sizes, beware vectors of vectors and the stack.
- Constants matter near the limit: cache misses, tree containers, modulo and I/O cost far more than an addition.

Practice problems (CSES Problem Set)

ESTIMATE, THEN CODE

- Weird Algorithm
- Distinct Numbers
- Sum of Two Values
- Josephus Problem I
- Concert Tickets
- Reading Books

HARMONIC SUMS AND SQUARE ROOTS

- Counting Divisors
- Common Divisors
- Sum of Divisors
- Prime Multiples

CHAPTER 4

C++ and the STL for Contests

C++ is the language of competitive programming for two reasons: it compiles to fast machine code, and its standard library (the **STL**) already contains most of the data structures and algorithms you need, implemented and debugged by experts. Knowing the STL well is worth as much as knowing a dozen algorithms: every minute you do not spend writing and debugging a balanced tree or a heap is a minute spent on the actual problem.

This chapter is a working reference, organised by what you need to do. It explains the parts of C++ you will use every day, what each operation costs (measured, where it matters), and the traps that cost people contests.

IN THIS CHAPTER BRONZE SILVER GOLD

- read and write data fast, and why the defaults are ten to fifty times slower;
- choose integer and floating-point types that cannot overflow;
- pick the right container for the job, and know the cost of every operation;
- use iterators, binary search, sorting and the rest of `<algorithm>` fluently;
- write recursive lambdas, use policy-based trees, random numbers, and avoid undefined behaviour.

Before you start: Chapter 3. The code assumes your template.

4.1 Input and output BRONZE

By default, C++ streams stay synchronised with C's `printf` and `scanf` so that you may mix the two, and `cin` flushes `cout` before every read so that prompts appear before input is requested. Both courtesies are expensive, and contests need neither. Your template turns them off:

```
ios::sync_with_stdio(false);  // stop synchronising with scanf / printf
cin.tie(nullptr);             // do not flush cout before every read
```

The difference is dramatic. Reading 10^6 integers takes 0.72 s with the default settings and 0.08 s without synchronisation; even `scanf` needs 0.27 s. For output, printing 10^6 lines with `endl`, which flushes the buffer every time, takes almost 2 seconds, while `'\n'` with synchronisation off takes 0.04 s: forty-seven times faster. Every one of those numbers was measured on the same machine as Table 3.2.

> **After turning sync off** Never mix `cin`/`cout` with `scanf`/`printf` once synchronisation is off: the two sides buffer separately and their output comes out interleaved in the wrong order. Pick one family per program.

4.1.1 Reading patterns

```
int n;
cin >> n;
vi a(n);
cin >> a;                      // template: reads a whole vector
string s;
cin >> s;                      // one word (stops at whitespace)
string line;
getline(cin >> ws, line);      // a whole line (ws skips the old newline)
ll x;
while (cin >> x) { /* ... */ }  // until the end of input (count not given)
vector<string> grid(r);        // a grid of characters, one row per word
cin >> grid;
```

The one trap in reading is mixing `>>` with `getline`: after `cin >> n`, the newline that ended the line is still in the input, and a plain `getline` returns an empty string. Skipping whitespace with `cin >> ws` first, as above, fixes it.

4.1.2 Writing patterns

```
print(a, b, c);                // template: "a b c\n"
print(v);                      // template: a vector, space separated
yn(ok);                        // template: YES / NO
cout << fixed << setprecision(10) << ans << '\n';  // the template sets this
cout << flush;                 // interactive problems: flush after each query
```

Floating-point answers are usually checked with an absolute or relative error of 10^{-6}, so print more digits than that: the template's 10 digits after the decimal point are safe. For interactive problems (Chapter 29), the judge only sees your query after you flush, so there `endl` or an explicit flush is required, not a mistake.

4.2 Integer types and overflow

BRONZE

Overflow is the most common cause of wrong answers that pass the samples. A signed integer that overflows is *undefined behaviour* in C++: in practice it wraps around to a negative number, but the compiler is also allowed to assume it never happens and optimise accordingly, which can make the bug appear or vanish between compilers. The defence is to know your ranges (Figure 4.1) and to estimate the largest value every expression can reach.

> **Overflow happens in the expression, not the variable** `long long x = a * b;` still overflows when `a` and `b` are 32-bit `int`s, because the product is computed in `int` before it is stored. Write `1LL * a * b` or make the operands 64-bit. Your template's `#define int long long` removes most of these bugs, but not the ones in `1 << k` (the literal `1` is still a 32-bit `int`: write `1LL << k`), in `__builtin_popcount` (use `__builtin_popcountll`), or in constants like `1e18`, which is a `double`.

When 64 bits are not enough. A product of two numbers below $9.2 \cdot 10^{18}$ can reach $8.5 \cdot 10^{37}$, which fits in GCC's `__int128` (but not on 32-bit judges, and `cin`/`cout` cannot read or print it directly).

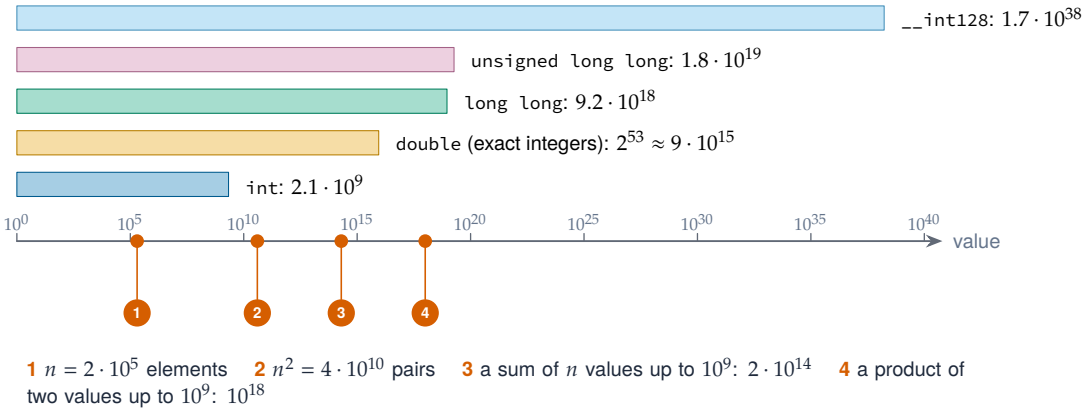


Figure 4.1: Type ranges (bars) against quantities that appear in contests (red ticks), on a logarithmic ruler. A sum of $2 \cdot 10^5$ values up to 10^9 overflows `int` by a factor of 10^5; a product of two such values fits in `long long` with a factor of 9 to spare, but a product of three does not fit anywhere except `__int128`.

For modular multiplication with a modulus up to 10^{18}, the portable alternatives are a `long double` trick or splitting into 32-bit halves, both in your library (`cpmodmul` ideas inside `cppollard` and `cphash`). To *detect* overflow instead, GCC provides `__builtin_mul_overflow(a, b, &res)`, which returns true when the product does not fit.

Unsigned arithmetic wraps by design. `unsigned long long` arithmetic is defined modulo 2^{64}, which makes it perfect for hashing (Chapter 27) and dangerous for everything else: `v.size() - 1` is a huge number when the vector is empty, because `size()` is unsigned. The template's `sz(v)` returns a signed value for exactly this reason.

Floating point. A `double` has 53 bits of mantissa: about 15–16 significant decimal digits, and every integer up to $2^{53} \approx 9 \cdot 10^{15}$ is exact. `long double` on x86 GCC has 64 bits of mantissa (18–19 digits). Floating-point answers are fine for geometry and probability; for anything that must be exact (counts, sums of large integers, comparisons of fractions), use integers and cross-multiplication.

> **Powers of two to remember** $2^{10} \approx 10^3, 2^{20} \approx 10^6, 2^{30} \approx 10^9, 2^{31}-1 = 2\,147\,483\,647, 2^{53} \approx 9{\cdot}10^{15}$, $2^{60} \approx 10^{18}$, $2^{63} - 1 \approx 9.2 \cdot 10^{18}$. A number up to 10^{18} has 60 bits, so binary search over $[0, 10^{18}]$ takes 60 steps.

> **Division and remainder with negative numbers** C++ rounds integer division towards zero: $-7/2 = -3$ and $-7 \bmod 2 = -1$. When you need the mathematical floor or a non-negative remainder, use the template's `fdiv(a, b)` and `cdiv(a, b)`, and write `((x % m) + m) % m`.

4.3 Choosing a container

SILVER

The STL has a container for almost every access pattern, and choosing the right one is most of the work. Figure 4.2 is a decision procedure; Table 4.1 lists what each operation costs.

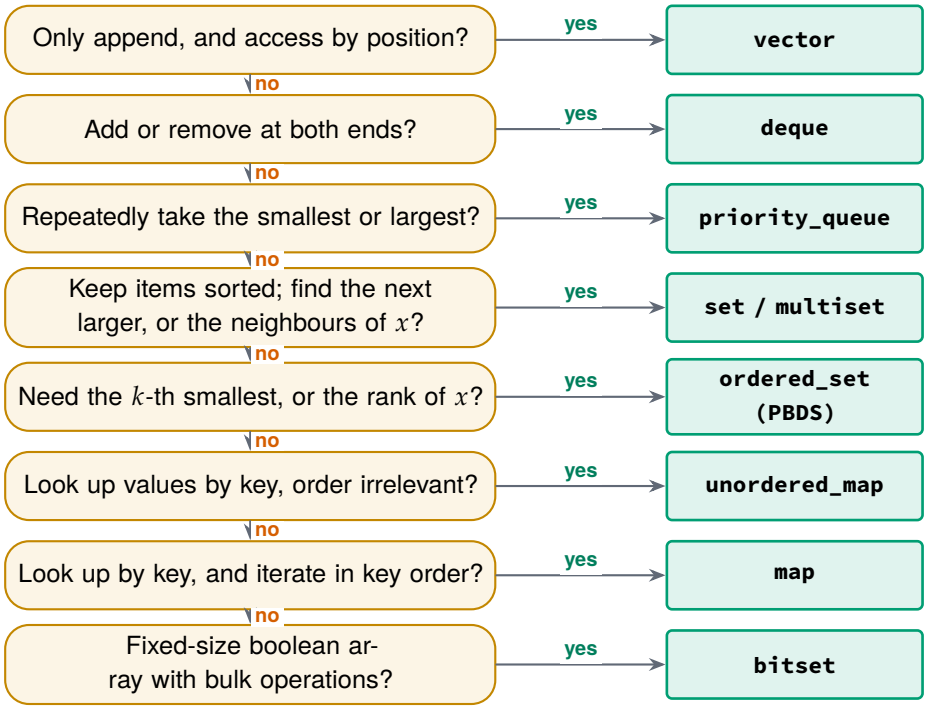


Figure 4.2: Choosing a container: ask the questions from the top and take the first "yes". Most problems stop at the first question; `vector` is the right answer far more often than any other container.

Table 4.1: STL containers and the cost of their operations (n = number of elements).

Container	Access / search	Insert / erase	Typical use
`vector`	$O(1)$ by index	$O(1)$ amortized at the end	almost everything
`deque`	$O(1)$ by index	$O(1)$ at both ends	sliding windows, 0-1 BFS
`stack`, `queue`	top / front only	$O(1)$	DFS, BFS, bracket matching
`priority_queue`	top in $O(1)$	$O(\log n)$	Dijkstra, "take the best"
`set`, `map`	$O(\log n)$	$O(\log n)$	sorted keys, neighbours
`multiset`	$O(\log n)$	$O(\log n)$	sorted with duplicates, medians
`unordered_map`	$O(1)$ expected	$O(1)$ expected	counting by key
`bitset<N>`	$O(1)$	$O(N/64)$ per bulk op	boolean DP, reachability
`string`	$O(1)$ by index	$O(1)$ at the end	text; `substr` copies

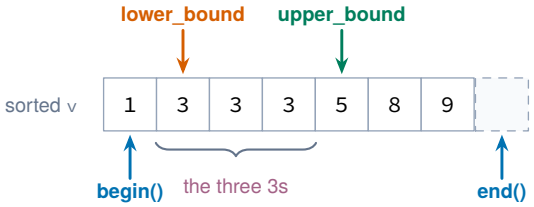
4.3.1 vector, and resetting between test cases

```
vi v(n, 0);                          // n zeros
vvi g(n, vi(m, -1));                 // an n x m grid of -1
v.push_back(x); v.pop_back(); v.back(); sz(v); v.empty();
v.assign(n, 0);                      // reset to n zeros, reusing the memory
v.clear();                           // size 0, capacity kept
sort(all(v)); reverse(all(v));
v.erase(unique(all(v)), v.end());    // remove duplicates (sort first)
v.reserve(n);                        // no reallocations if the size is known
```

`vector` stores its elements contiguously, which makes it the fastest container to scan by a wide margin (Section 3.8). In multi-test problems, declare vectors inside `solve()` so that each test starts fresh, or reset them with `assign`: a global array cleared with `memset` in every test costs its full size each time, even when the test is tiny.

4.3.2 Iterators and half-open ranges

Every STL algorithm works on a **half-open range** $[first, last)$: `begin()` points at the first element and `end()` points one past the last (Figure 4.3). Half-open ranges compose without off-by-one errors: $[a, b)$ followed by $[b, c)$ is $[a, c)$, the length is $last - first$, and an empty range has $first = last$. The whole of this book uses the same convention for array ranges.



`lower_bound(x)`: first element $\geq x$. `upper_bound(x)`: first element $> x$. Between them lie exactly the copies of x: here 3 of them, so `count = upper − lower`. `end()` is one past the last element and must never be dereferenced.

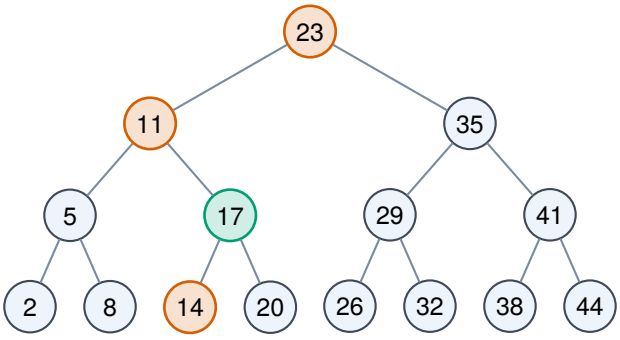
Figure 4.3: Iterators on a sorted vector. Both bounds take $O(\log n)$ and return `end()` when no element qualifies.

```
auto it = lower_bound(all(v), x);      // first element >= x (v sorted)
int idx = it - v.begin();              // its index; idx == sz(v) means "none"
auto [lo, hi] = equal_range(all(v), x); // both bounds at once
bool present = binary_search(all(v), x);
int cnt = upper_bound(all(v), x) - lower_bound(all(v), x);
```

4.3.3 set, multiset and map

`set` and `map` are balanced binary search trees (red-black trees in GCC). Every operation walks one root-to-leaf path, so it costs $O(\log n)$, and the elements are always available in sorted order. Their power lies in the neighbour queries: `lower_bound` walks down the tree remembering the smallest key that was not too small (Figure 4.4).

```
set<int> s;
s.insert(x); s.erase(x); s.count(x);   // count is 0 or 1
auto it = s.lower_bound(x);            // first element >= x, or s.end()
if (it != s.begin()) { int below = *prev(it); }  // largest element < x
*s.begin(); *s.rbegin();               // minimum, maximum
multiset<int> ms;
ms.erase(ms.find(x));                  // erase ONE copy (erase(x) erases all!)
map<string, int> cnt;
cnt[w]++;                              // a missing key starts at 0
for (auto& [key, value] : cnt) { }     // in increasing key order
```



`s.lower_bound(15)` visits:

node	23	11	17	14	
compare	15 < 23: left	15 > 11: right	15 < 17: left	15 > 14: right	
best so far	23	23	**17**	17	⇒ answer 17

Figure 4.4: `lower_bound` in a balanced search tree walks one path from the root (orange), remembering the last node where it went left. With n keys the path has about $\log_2 n$ nodes.

> **Use the member function** `lower_bound(all(s), x)` on a `set` compiles, but it is the generic algorithm, which moves an iterator step by step and costs $O(n)$. Always call `s.lower_bound(x)`. The same goes for `s.count(x)` versus `count(all(s), x)`.

4.3.4 priority_queue and the binary heap

A `priority_queue` is a **binary heap** stored in a vector: the element at index i has its children at $2i+1$ and $2i+2$, and every parent is at least as large as its children (Figure 4.5). The largest element sits at the root, `push` lets a new element rise to its place and `pop` lets the last element sink from the root, each along one path of length $\log_2 n$.

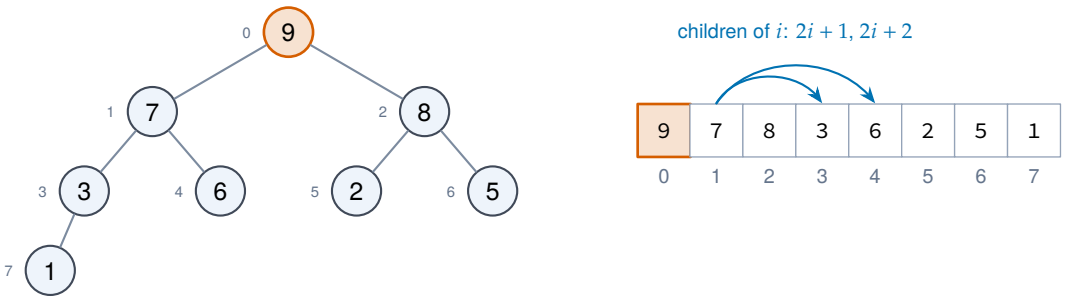


Figure 4.5: A max-heap drawn as a tree (left) and stored as an array (right). The tree is complete, so the array has no gaps, and no pointers are needed: the parent of index i is $\lfloor (i-1)/2 \rfloor$.

```
priority_queue<int> pq;                 // max-heap: top() is the largest
minpq<pll> q;                           // min-heap of (distance, vertex)
q.push({d, v});
auto [dist, u] = q.top(); q.pop();      // pairs compare by first, then by second
```

A heap cannot find or delete an arbitrary element. When you need that, either use a `set` (which can) or use *lazy deletion*: leave outdated entries in the heap and skip them when they surface, as Dijkstra's algorithm does (Chapter 12).

4.3.5 unordered_map and hash attacks

`unordered_map` is a hash table: an array of buckets, each holding the keys whose hash falls into it. With a good hash, buckets are short and every operation is expected $O(1)$. GCC's hash of an integer is the integer itself, and the bucket is the hash modulo a prime from a fixed list, so an adversary who knows that prime can send keys that all land in one bucket (Figure 4.6). Every operation then scans the whole chain: $O(n)$ each, $O(n^2)$ in total. On Codeforces, where participants hack each other's solutions, this happens routinely.

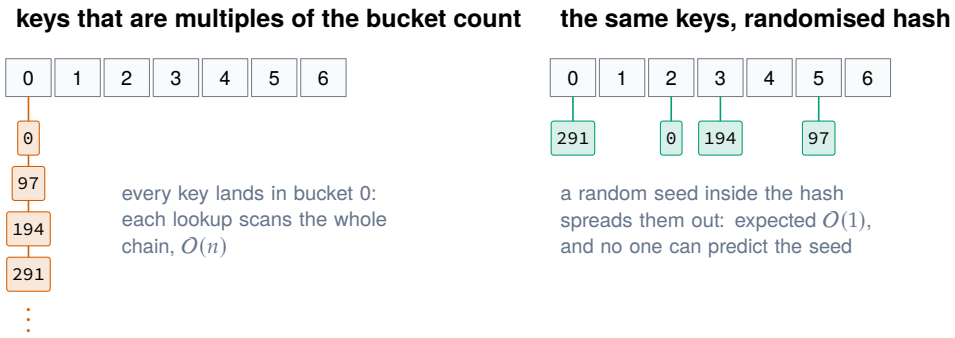


Figure 4.6: An anti-hash test against `unordered_map` (left) and the fix (right): hash with a random seed, as the library's `custom_hash` does, so that no fixed input can force collisions.

```
unordered_map<ll, int, custom_hash> cnt;  // template cphash: seeded splitmix64
cnt.reserve(1 << 20);                     // pre-size the buckets: 1.6x faster
```

4.3.6 bitset

`bitset<N>` is a fixed-size array of N bits packed into 64-bit words. Single bits cost $O(1)$; whole-set operations (`&`, `|`, `^`, `~`, shifts, `count()`, `any()`) cost $O(N/64)$ because they process 64 bits per machine instruction. GCC adds `_Find_first()` and `_Find_next(i)` to jump between set bits. The size must be a compile-time constant; when it depends on the input, round up to the largest possible value.

```
bitset<100001> can;                    // subset sums reachable so far
can[0] = 1;
for (int w : weights) can |= can << w;  // N/64 word operations per item
int count_reachable = can.count();
for (int i = can._Find_first(); i < 100001; i = can._Find_next(i)) {
    // i is a reachable sum
}
```

4.4 Sorting and comparators

SILVER

`std::sort` is an introsort: quicksort that falls back to heapsort on bad inputs, so it is $O(n \log n)$ in the worst case, and it sorts 10^6 integers in about 0.04 s. It sorts by `operator<` unless you pass a comparator.

```
sort(all(v));                                   // increasing
sort(all(v), greater<>());                      // decreasing
sort(all(p), [](const pll& a, const pll& b) {   // by second, then first desc.
    if (a.second != b.second) return a.second < b.second;
    return a.first > b.first;
});
stable_sort(all(v), cmp);                       // equal elements keep order
vi ord(n);                                      // sort indices by a key
iota(all(ord), 0);
sort(all(ord), [&](int i, int j) { return key[i] < key[j]; });
```

Strict weak ordering A comparator must behave like `<`: `cmp(x, x)` must be false, and if `cmp(a, b)` and `cmp(b, c)` then `cmp(a, c)`. Writing `<=` instead of `<` breaks the first rule, and `sort` may then read past the end of the array and crash. Your local builds use the checked STL, which detects an invalid comparator and stops with a message.

Sorting pairs and tuples. Pairs, tuples and arrays compare lexicographically: first by the first component, ties by the second, and so on. That makes "sort by a, then by b decreasing" a one-liner: sort the pairs `{a, -b}`. Sorting events as `(time, type)` pairs orders simultaneous events by type for free, which Section 2.5 used.

Coordinate compression. Values up to 10^9 that only matter through their order can be replaced by their rank in $0, \ldots, k-1$, so they can index an array or a Fenwick tree (template `cpcompress`):

```
vi vals = a;
sort(all(vals)), vals.erase(unique(all(vals)), vals.end());
for (auto& x : a) x = lower_bound(all(vals), x) - vals.begin();  // 0 <= x < sz
```

4.5 The algorithm toolbox

SILVER

Call	What it does
`accumulate(all(v), 0LL)`	sum; the `0LL` makes the sum 64-bit
`*max_element(all(v)), min_element`	largest / smallest element (an iterator)
`count(all(v), x), find(all(v), x)`	occurrences / first occurrence: $O(n)$
`iota(all(v), 0)`	fills $0, 1, 2, \ldots$
`partial_sum, adjacent_difference`	prefix sums and differences, in place
`next_permutation(all(v))`	next arrangement in lexicographic order; `false` after the last
`nth_element(v.begin() + k, ...)`	the k-th smallest in place, expected $O(n)$
`rotate, reverse, unique, fill`	rearrangements, all $O(n)$
`merge, inplace_merge`	merge two sorted ranges in $O(n)$
`gcd(a, b), lcm(a, b)`	greatest common divisor, least common multiple
`__builtin_popcountll(x)`	number of set bits
`__builtin_ctzll(x), __builtin_clzll(x)`	trailing / leading zero bits ($x > 0$)
`__lg(x)`	$\lfloor \log_2 x \rfloor$ for $x > 0$
`shuffle(all(v), rng)`	random order

nth_element for medians and top-k `nth_element(v.begin(), v.begin() + k, v.end())` puts the element that would be at index k after sorting into position k, with smaller elements before it and larger after, in expected linear time. It finds a median in one call and is several times faster than sorting when you only need the k smallest elements.

4.6 Modern C++ that saves time

SILVER

C++17 has several features that make contest code shorter and less error-prone. All of them are used throughout this book.

```
for (auto& x : v) x *= 2;              // by reference: modifies the elements
for (const auto& s : words) { }        // const& avoids copying strings
auto [a, b] = pr;                      // structured bindings unpack pairs etc.
for (auto& [key, val] : mp) { }
array<int, 3> t = {1, 2, 3};           // fixed size, usable as a map key
v.emplace_back(a, b);                  // construct a pair or struct in place
if (auto it = mp.find(k); it != mp.end()) { }  // "it" stays local to the if
```

4.6.1 Lambdas and recursion

A lambda is a function defined inside another function. `[&]` lets it read and modify every local variable by reference, which is why lambdas are the natural way to write helpers inside `solve()`. A lambda cannot call itself by name, so a recursive lambda receives itself as its first parameter:

```
auto dfs = [&](auto&& self, int u, int p) -> void {
    for (int v : adj[u])
        if (v != p) self(self, v, u);
};
dfs(dfs, 0, -1);
```

This style is used by the library's inside-`solve()` templates: no global arrays that must be reset between test cases. It is also faster than `std::function`, which adds an indirect call and may allocate.

4.7 Policy-based data structures

GOLD

GCC ships an extension library with a balanced tree that also knows the *size of every subtree*, so it can answer "what is the k-th smallest element?" and "how many elements are smaller than x?" in $O(\log n)$. `std::set` cannot do either.

```
ordered_set<int> s;                    // template cposet
s.insert(5); s.insert(1); s.insert(9);
*s.find_by_order(1);                   // 5: the element at index 1 (sorted)
s.order_of_key(6);                     // 2: the number of elements < 6
```

The tree is a set, so duplicates are dropped; for a multiset, store pairs `{value, unique_id}`. It is about twice as slow as `std::set`, which is still fast enough for $2 \cdot 10^5$ operations, and it replaces a

Fenwick tree over compressed values in many problems (Josephus Problem II, sliding medians, counting inversions online).

4.8 Randomness

GOLD

`rand()` is slow, returns only 15 random bits on some compilers, and is predictable. Use a 64-bit Mersenne Twister seeded from the clock (template `cprng`):

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
ll r = uniform_int_distribution<ll>(lo, hi)(rng);   // uniform in [lo, hi]
shuffle(all(v), rng);
```

Seeding from the clock matters on Codeforces, where other contestants can read your code: with a fixed seed, they can construct an input that makes your randomised algorithm take its worst case. Chapter 29 shows what randomness buys: hashing, randomised pivots, random restarts and probabilistic checks.

4.9 Undefined behaviour and common bugs

BRONZE

Some mistakes in C++ are not errors the compiler reports but **undefined behaviour**: the program may crash, print garbage, or work perfectly on your machine and fail on the judge. The checked STL and the flags in your setup catch many of them locally (GUIDE.pdf, Chapter 5); the rest you must avoid by habit.

- **Out-of-bounds access**: `a[n]` in an array of size n, `v[-1]`, `*s.begin()` on an empty set. The checked STL catches it for `vector` and `string` and names the line.
- **Uninitialised variables**: a local `int sum;` holds garbage, not 0. Global variables start at 0.
- **Signed overflow**: undefined, not merely wrapping (Section 4.2).
- **Invalid iterators**: an iterator or reference into a `vector` dies when the vector reallocates (`push_back`); erasing from a container while iterating over it with range-for skips or repeats elements.
- **Missing return**: a non-`void` function that ends without `return`. With optimisation, GCC may turn this into a crash or an infinite loop; `-Wall` warns.
- **Bad comparator**: see Section 4.4.

And the bugs that are not undefined but equally common: `=` instead of `==` in a condition (`-Wall` warns), integer division where a real number was meant (`1 / 2 == 0`), comparing doubles with `==` instead of `fabs(a - b) < 1e-9`, forgetting to reset state between test cases, and reading the input in a different order from the statement.

> **Debug output that cannot break the judge** Your template's `dbg(x, v, mp)` prints variables with their names to standard error locally, and compiles to nothing on the judge. Leave the calls in: they cost nothing when submitted, and they are there when the next bug appears.

Chapter summary

- Fast I/O (sync off, `'\n'`) is worth a factor of 10 to 50; the template sets it up.
- Estimate every expression's largest value; use 64-bit integers, `1LL <<`, and exact integer arithmetic.
- `vector` first; `set/map` for sorted neighbours; `priority_queue` for repeated minimums; `unordered_map` with a randomised hash for counting; `bitset` for bulk boolean work.
- Ranges are half-open; call member functions like `s.lower_bound`; comparators must be strict.
- Lambdas with `[&]`, structured bindings and `const&` make contest code short and safe.

Practice problems (CSES Problem Set)

CONTAINERS IN ACTION

- Distinct Numbers
- Apartments
- Concert Tickets
- Sum of Two Values
- Playlist
- Towers
- Traffic Lights
- Room Allocation
- Collecting Numbers II

ORDERED STATISTICS

- Josephus Problem II
- Sliding Median
- Sliding Cost

CHAPTER 5

Choosing a Data Structure

A **data structure** is a way of organising data so that the operations an algorithm needs are fast. Choosing the right one is often the entire solution: the same idea runs in $O(n^2)$ with a plain array and in $O(n \log n)$ with a heap, and the heap version is only three lines longer. The previous chapter showed how to use the STL containers; this chapter is about the thinking behind them: which operations each structure makes cheap, which it makes expensive, and how simple structures (stacks, queues, heaps, sets) combine into solutions for problems that look nothing like textbook exercises.

IN THIS CHAPTER BRONZE SILVER GOLD

- compare structures by the cost of each operation, and pick one from the operations you need;
- use stacks for nesting and "nearest" questions, including the monotonic stack;
- use queues and deques for arrival order and sliding windows;
- use heaps for repeated minimums, scheduling and running medians;
- maintain sets of values and sets of intervals, and simulate deletions with array-based linked lists;
- augment simple structures (a stack with its minimum, a queue made of two stacks).

Before you start: Chapters 3 and 4.

5.1 Operations decide

BRONZE

Before choosing a structure, list the operations your algorithm performs, with how often it performs each: "n inserts, q queries for the smallest element, occasional deletions". Then pick the structure that makes the *frequent* operations cheap. Figure 5.1 compares five ways of storing a changing set of numbers.

Structure	insert	delete x	find x	minimum	k-th smallest	sorted scan
unsorted array	1	n	n	n	n	$n \log n$
sorted array	n	n	$\log n$	1	1	n
balanced tree (`set`)	$\log n$	$\log n$	$\log n$	1	n *	n
hash table	1 †	1 †	1 †	n	n	$n \log n$
binary heap	$\log n$	—	—	1	—	—

* $\log n$ with the policy-based `ordered_set` † expected — not supported

Figure 5.1: The cost of each operation, in O-notation, for five ways to store a dynamic set (green: constant, orange: logarithmic or $n \log n$, red: linear). No structure is best at everything; the balanced tree is the only one with no red cell among the everyday operations, which is why `set` is the default for "sorted and changing".

The table explains several habits of experienced competitors. When the data does not change after it is read, a *sorted array* beats every tree: sort once, then binary search. When you only ever need the smallest element, a *heap* beats a `set` by a large constant factor. When order does not matter at all, a *hash table* is fastest. And when you need several operations that no single structure provides, you combine structures, as the rest of this chapter shows repeatedly.

5.2 Stacks

BRONZE

A **stack** is a sequence where you can only add and remove at one end, the *top*: the last element pushed is the first popped (LIFO, last in, first out). Stacks model anything with *nesting*: brackets, function calls, undo history, the path of a depth-first search.

Example: Balanced brackets

Given a string of `()[]{}`, decide whether every bracket is closed by a matching bracket of the same type in the right order.

Solution. Scan the string. Push every opening bracket; for a closing bracket, the top of the stack must be the matching opening bracket, which you pop. The string is balanced if no check fails and the stack is empty at the end (Figure 5.2). Each character is pushed and popped at most once: $O(n)$.

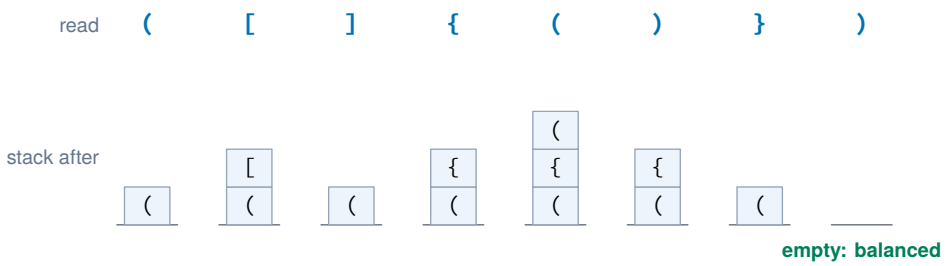


Figure 5.2: Matching `([]{()})` with a stack. Each column shows the stack after reading the character above it; a closing bracket always meets its partner on top, and the stack ends empty.

The same scan evaluates arithmetic expressions (a stack of numbers and a stack of operators), checks that tags in an HTML-like format are properly nested, and computes the depth of the deepest nesting. Recursion itself is a stack: every function call pushes a frame and every return pops one, which is why a very deep recursion can run out of stack memory (Section 3.7) and why an explicit stack is the standard fix.

5.2.1 The monotonic stack

A stack whose contents are kept sorted, by popping everything that would break the order before each push, answers "nearest" questions in linear time. It is one of the most useful patterns in this book.

Example: Nearest Smaller Values (CSES)

For every position of an array, find the nearest position to its left with a smaller value (print its 1-indexed position, or 0 if there is none). Limits: $n \le 2 \cdot 10^5$.

Scan from left to right and keep a stack of candidate positions. When a_i arrives, every candidate whose value is $\ge a_i$ can never be the answer for any later position: a_i is both closer and at least as small. So pop them all; what remains on top is the answer for i; then push i. The stack's values increase from bottom to top at all times (Figure 5.3).

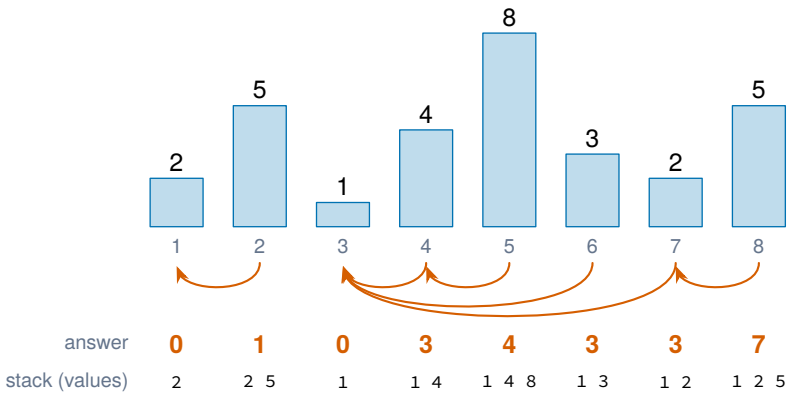


Figure 5.3: Nearest smaller values of $2, 5, 1, 4, 8, 3, 2, 5$ (positions under the bars). Each arrow points to the answer. The stack, bottom to top, only ever holds increasing values: when 3 arrives, 8 and 4 are popped for good, because 3 hides them from everything to its right.

```
void solve() {
    int n;
    cin >> n;
    vi a(n), ans(n);
    cin >> a;
    vi st;  // indices; their values increase from the bottom to the top
    rep(i, n) {
        while (!st.empty() && a[st.back()] >= a[i])
            st.pop_back();  // can never be an answer again
        ans[i] = st.empty() ? 0 : st.back() + 1;  // 1-indexed, 0 if none
        st.push_back(i);
    }
    print(ans);
}
```

The inner `while` can pop many elements at once, but every index is pushed once and popped at most once, so the whole scan is $O(n)$ (the potential argument of Section 3.5). Running the same scan from the right gives the nearest smaller element on the right; flipping the comparison gives nearest larger. Together they give each element's "reign", the maximal window where it is the minimum, which is exactly what the contribution technique of Section 2.1 needed. The largest rectangle in a histogram, the number of visible buildings, and the next warmer day are all one monotonic stack each.

</> Template: Monotonic stack (next greater / previous smaller) Ctrl+Alt+T, 7 prefix `cpmonostack`

5.3 Queues and deques

SILVER

A **queue** removes elements in the order they arrived (FIFO, first in, first out). Queues model waiting lines and, above all, *breadth-first processing*: BFS visits vertices in the order they were discovered, which is why it finds shortest paths in unweighted graphs (Chapter 11). A **deque** (double-ended queue) allows pushing and popping at both ends in $O(1)$.

5.3.1 The sliding window maximum

Example: Window maximum

For every window of k consecutive elements, print its maximum ($n \le 2 \cdot 10^5$).

Recomputing each window costs $O(nk)$. The deque version keeps the indices of the elements that could still become a window maximum, with their values decreasing from front to back. When a_i arrives, every element at the back that is $\le a_i$ can never be a maximum again (it is older and not larger), so pop it; then push i. If the front has slid out of the window, pop it from the front. The front is now the maximum of the window (Table 5.1). Like the monotonic stack, every index enters and leaves once: $O(n)$ in total.

Table 5.1: The monotonic deque on $1, 3, -1, -3, 5, 3, 6, 7$ with $k = 3$. Values are shown instead of indices.

i	a_i	what happens	deque (front to back)	window max
0	1	push	1	
1	3	pop 1 (smaller), push	3	
2	−1	push	3, −1	3
3	−3	push	3, −1, −3	3
4	5	pop −3, −1, 3, push	5	5
5	3	push	5, 3	5
6	6	pop 3, 5, push	6	6
7	7	pop 6, push	7	7

```
void solve() {
    int n, k;
    cin >> n >> k;
    vi a(n);
    cin >> a;
    deque<int> dq;  // indices in the window; values decrease front to back
    vi out;
    rep(i, n) {
        while (!dq.empty() && a[dq.back()] <= a[i])
            dq.pop_back();  // dominated by a[i]
        dq.push_back(i);
        if (dq.front() <= i - k) dq.pop_front();  // slid out of the window
        if (i >= k - 1) out.push_back(a[dq.front()]);
    }
    print(out);
}
```

How to recognize it "For every window of length k" or "for every i, the best j in $[i-k, i)$": a monotonic deque. It also speeds up dynamic programming with transitions from a sliding range, $dp_i = \min_{i-k \le j < i} dp_j + c_i$, from $O(nk)$ to $O(n)$ (Chapter 20).

Template: Sliding window min / max (monotonic deque) Ctrl+Alt+T, 6 prefix `cpwindow`

5.4 Heaps

SILVER

A **heap** always knows its smallest (or largest) element. Inserting and removing the top cost $O(\log n)$ each, and nothing else is supported. That narrow interface is exactly what many algorithms need: Dijkstra's algorithm, Prim's algorithm, event simulations and scheduling all repeat "take the best remaining item, then add a few new items".

5.4.1 How a heap works

A binary heap is a complete binary tree stored in an array (Section 4.3) where every parent is at least as large as its children (for a max-heap). To insert, put the new element in the first free slot at the bottom and let it *sift up*: while it is larger than its parent, swap them (Figure 5.4). To remove the top, move the last element to the root and let it *sift down*: while it is smaller than its larger child, swap them. Both walk one path of the tree, so both cost $O(\log n)$.

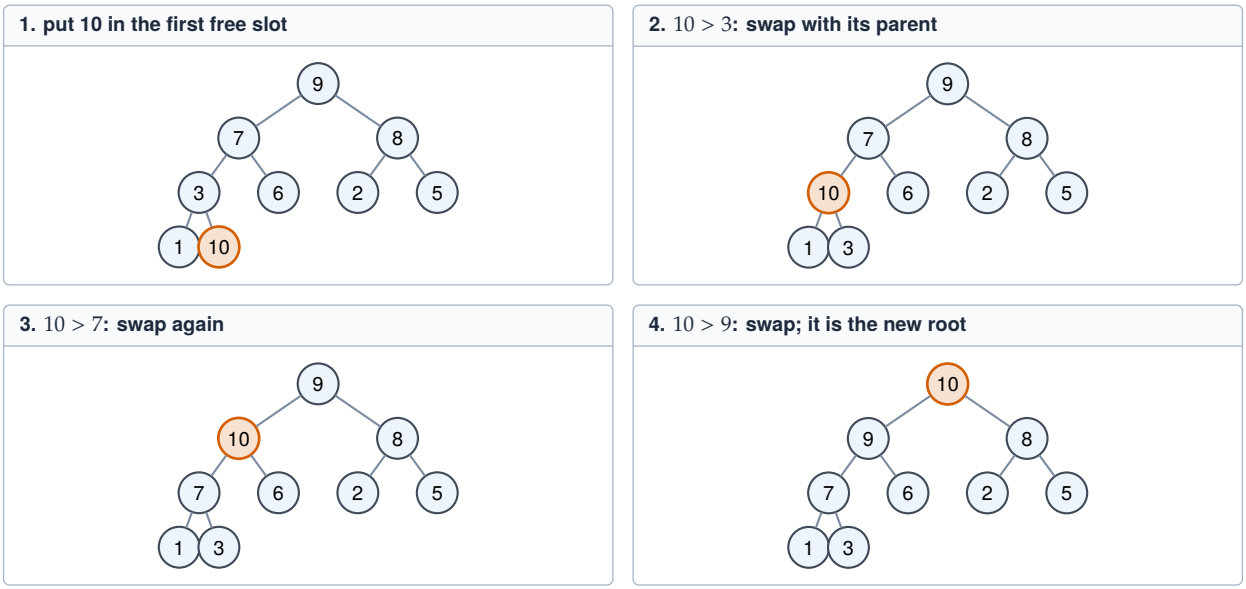


Figure 5.4: Inserting 10 into a max-heap. The new element rises along one path from the bottom to the root; everything off that path is untouched, so the cost is the height of the tree, $O(\log n)$.

Building a heap is linear Inserting n elements one by one costs $O(n \log n)$, but `make_heap` (or constructing a `priority_queue` from a whole vector) builds a heap in $O(n)$ by sifting down from the bottom level up: half the nodes are leaves and never move, a quarter move at most one level, an eighth at most two, and $\sum_h h \cdot n/2^{h+1} = O(n)$.

5.4.2 Scheduling with a heap

Example: Room Allocation (CSES)

A hotel has n guests; guest i stays from day a_i to day b_i. Two guests can share a room if one leaves strictly before the other arrives. Find the minimum number of rooms and a valid assignment. Limits: $n \le 2 \cdot 10^5$.

Solution. Process guests by arrival. Keep a min-heap of the occupied rooms keyed by their guest's departure day. For a new guest, look at the room that frees up earliest: if its guest left before today, reuse it; otherwise open a new room. Reusing the earliest-freed room is never worse (an exchange argument), and a new room is opened only when every existing room is occupied today, so the number of rooms equals the maximum number of guests present at once, which is a lower bound.

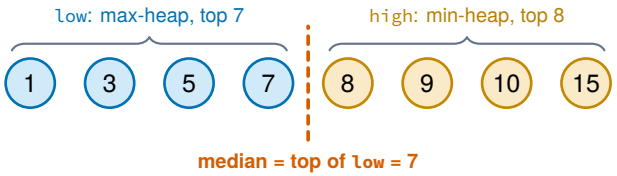
```
void solve() {
    int n;
    cin >> n;
    vector<array<ll, 3>> c(n);  // (arrival, departure, index)
    rep(i, n) cin >> c[i][0] >> c[i][1], c[i][2] = i;
    sort(all(c));
    minpq<pll> busy;  // (departure day, room) of the occupied rooms
    vi room(n);
    int rooms = 0;
    for (auto [a, b, i] : c) {
        if (!busy.empty() && busy.top().first < a) {  // earliest leaver gone
            room[i] = busy.top().second;
            busy.pop();
        } else room[i] = ++rooms;
        busy.push({b, room[i]});
    }
    print(rooms);
    print(room);
}
```

5.4.3 The running median with two heaps

Example: Running median

Numbers arrive one at a time. After each one, print the median of the numbers so far (the lower median when the count is even).

Solution. Keep the smaller half in a max-heap `low` and the larger half in a min-heap `high`, with `low` holding the same number of elements or one more. The median is then the top of `low` (Figure 5.5). Insert a new number into the half it belongs to, then move one top element across if the sizes drifted apart. Every step costs $O(\log n)$.



After the stream $5, 15, 1, 3, 8, 7, 9, 10$ the printed medians were $5, 5, 5, 3, 5, 5, 7, 7$.

Figure 5.5: Two heaps split the numbers at the median. Only the two tops are ever inspected, and they sit on either side of the split, so the median is always at hand.

```
void solve() {
    int n;
    cin >> n;
    priority_queue<ll> low;  // the smaller half, largest on top
    minpq<ll> high;          // the larger half, smallest on top
    vi med;
    rep(i, n) {
        ll x;
        cin >> x;
        if (low.empty() || x <= low.top()) low.push(x);
        else high.push(x);
        // keep the sizes equal, or low one larger
        if (sz(low) > sz(high) + 1) high.push(low.top()), low.pop();
        else if (sz(high) > sz(low)) low.push(high.top()), high.pop();
        med.push_back(low.top());  // the lower median
    }
    print(med);
}
```

Deleting from a heap anyway When elements must also leave (a sliding median, where the oldest element expires), keep a second heap of "deleted but not yet removed" values. Whenever the tops of the real heap and the deletion heap are equal, pop both. The heap then behaves as if it supported deletion, at the same asymptotic cost. For the sliding median, a `multiset` with an iterator to the median is often simpler.

5.5 Ordered sets and interval sets

SILVER

A `set` is the structure for questions about *order* in a changing collection: the smallest element, the successor of x, the element closest to x. It shines when the values are huge (coordinates up to 10^9) and the collection changes.

Example: Closest occupied seat

People take and leave seats in a row of length 10^9. When a person arrives at seat x, report the distance to the nearest occupied seat.

Solution. Keep the occupied seats in a `set`. For an arrival at x, `s.lower_bound(x)` is the nearest occupied seat at or to the right of x, and its predecessor (if any) is the nearest on the left. Each operation is $O(\log n)$, and the row's length never matters.

5.5.1 Sets of intervals

Many problems maintain a collection of disjoint intervals: painted segments of a line, free blocks of memory, booked time slots. Store them in a `map` from left end to right end. To insert $[l, r]$, find the first interval that could overlap it with `lower_bound`, merge every overlapping interval into $[l, r]$ while erasing it, then insert the union (Figure 5.6). Every interval is inserted once and erased at most once, so n insertions cost $O(n \log n)$ in total, however much merging happens.

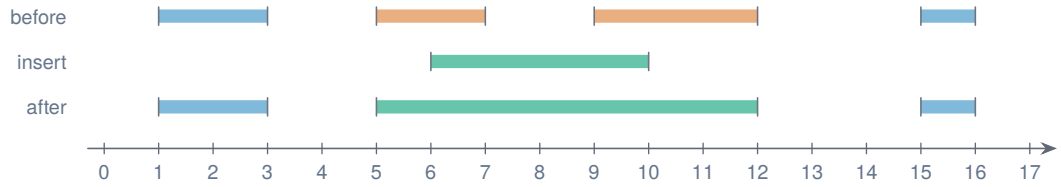


Figure 5.6: Inserting $[6, 10]$ into a set of disjoint intervals. It overlaps $[5, 7]$ and $[9, 12]$ (red), which are erased and replaced, together with it, by their union $[5, 12]$. The overlapping intervals are consecutive in the `map`, so one `lower_bound` and a short walk find them all.

The same structure, with each interval carrying a value, handles "assign value v to the range $[l, r]$" operations: split the intervals at l and r, erase everything in between, insert one new interval. When the data is random this runs fast and is known as the *Chtholly tree*; with adversarial data, prefer a segment tree with lazy assignment (Chapter 22).

5.6 Hash maps for counting

SILVER

A hash table answers "have I seen this key, and how often?" in expected $O(1)$. It is the natural structure for frequency counting, for detecting duplicates, and for the classic "two sum" pattern: to find two elements with sum x, scan the array and, for each a_i, ask whether $x - a_i$ was seen before. The prefix-sum version of the same idea counts subarrays with a given sum (Section 2.8 and Chapter 9).

Two warnings from Chapter 4 apply. Hash tables have a large constant factor, so when the keys are small integers, an array indexed by the key is faster still. And a predictable hash can be attacked; use the randomised `custom_hash` from the library whenever others can see your code.

5.7 Augmenting simple structures

GOLD

A structure can often store a little extra information per element that makes a new query cheap. This **augmentation** is one of the most powerful ideas in data structure design: segment trees, order-statistic trees and link-cut trees are all augmented versions of simpler structures.

A stack with its minimum. Store with each element the minimum of the stack from the bottom up to it. The minimum of the whole stack is then the value stored with the top, in $O(1)$, and popping never invalidates the stored minimums below, because they only depend on what lies beneath.

A queue made of two stacks. A queue can be built from two stacks: push onto an `in` stack; pop from an `out` stack, and when `out` is empty, pour all of `in` into it, which reverses the order so that the oldest element ends on top (Figure 5.7). Each element is moved once, so every operation is $O(1)$ amortized. Combined with the stack-with-minimum, this gives a *queue with minimum*: a sliding window minimum that works even when the window does not move one step at a time.

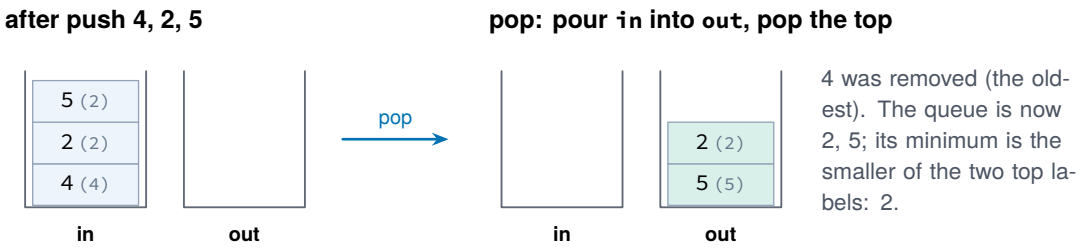


Figure 5.7: A queue with minimum built from two stacks. Each entry stores its value and, in grey, the minimum of its stack from the bottom up to it. Pouring reverses the order, so the oldest element is popped first.

```
void solve() {
    int q;
    cin >> q;
    vector<pll> in, out;  // stacks of (value, minimum of the stack up to here)
    auto push = [](vector<pll>& st, ll x) {
        st.push_back({x, st.empty() ? x : min(x, st.back().second)});
    };
    while (q--) {
        int type;
        cin >> type;
        if (type == 1) {  // push x at the back
            ll x;
            cin >> x;
            push(in, x);
        } else if (type == 2) {  // pop the front
            if (out.empty())  // pour "in" into "out": the oldest ends on top
                while (!in.empty()) push(out, in.back().first), in.pop_back();
            out.pop_back();
        } else {  // minimum of the whole queue
            ll m = LLONG_MAX;
            if (!in.empty()) chmin(m, in.back().second);
            if (!out.empty()) chmin(m, out.back().second);
            print(m);
        }
    }
}
```

The same trick works for any *associative* operation, not just the minimum: gcd, bitwise AND, even matrix products or function composition. It is the simplest way to answer "aggregate of a sliding window" when the operation has no inverse, so prefix sums cannot help.

5.8 Linked lists in arrays

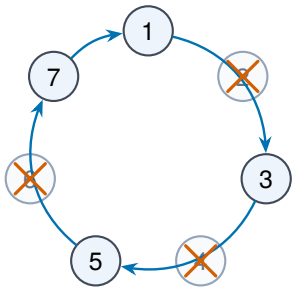
SILVER

`std::list` is rarely worth using: it is slow to traverse and gives no random access. But the *idea* of a linked list, two arrays `nxt[i]` and `prv[i]` holding each element's neighbours, is the standard way to simulate deletions from a sequence. Removing element i is two assignments, `nxt[prv[i]] = nxt[i]` and `prv[nxt[i]] = prv[i]`, in $O(1)$, and the neighbours of any remaining element are available immediately.

Example: Josephus Problem I (CSES)

Children $1, \ldots, n$ stand in a circle. Starting from child 1, every second child is removed until none remain. Print the order of removal. Limits: $n \le 2 \cdot 10^5$.

Solution. Keep a circular linked list in the array `nxt`. The child after the current one is removed by linking the current child to the next-but-one, then we skip ahead by one (Figure 5.8). Each removal is $O(1)$, so the whole simulation is $O(n)$, where erasing from a `vector` would cost $O(n^2)$.



After three removals (2, 4, 6, crossed out) the links skip them: `nxt[1] = 3`, `nxt[3] = 5`, `nxt[5] = 7`, `nxt[7] = 1`. The next child removed is `nxt[7] = 1`, and the full order for $n = 7$ is $2, 4, 6, 1, 5, 3, 7$.

Figure 5.8: The Josephus circle as a linked list in an array. Removing a child only redirects one link.

```
void solve() {
    int n;
    cin >> n;
    vi nxt(n + 1);
    rep(i, 1, n + 1) nxt[i] = i % n + 1;  // a circular linked list in an array
    vi order;
    int cur = 1;  // the child just before the next one to be removed
    rep(k, n) {
        int victim = nxt[cur];
        order.push_back(victim);
        nxt[cur] = nxt[victim];  // unlink the victim in O(1)
        cur = nxt[cur];          // skip one child
    }
    print(order);
}
```

Removing every k-th child instead of every second one needs "find the k-th remaining child", which a linked list cannot do quickly; that is Josephus Problem II, solved with an order-statistic tree or a Fenwick tree in $O(n \log n)$.

Deleting in reverse Many "delete elements one by one and answer after each deletion" problems become "add elements one by one" when processed backwards, and adding is usually easier: a DSU can merge but not split, a sorted structure can gain neighbours cheaply, a maximum can only grow (Section 2.4).

Chapter summary

- List the operations and their frequencies first; the structure follows from them.
- Stacks handle nesting; the monotonic stack answers every "nearest smaller / larger" question in $O(n)$.

- Queues give arrival order; the monotonic deque gives every sliding-window extreme in $O(n)$.
- Heaps repeatedly give the best item; two heaps give a running median; lazy deletion adds removal.
- Ordered sets answer neighbour queries on huge coordinates; a `map` of intervals keeps disjoint ranges.
- Augment structures with extra information (a minimum per stack entry), and simulate deletions with `nxt/prv` arrays.

Practice problems (CSES Problem Set)

STACKS, QUEUES AND HEAPS

- Nearest Smaller Values
- Room Allocation
- Movie Festival II
- Towers
- Traffic Lights
- Maximum Subarray Sum II

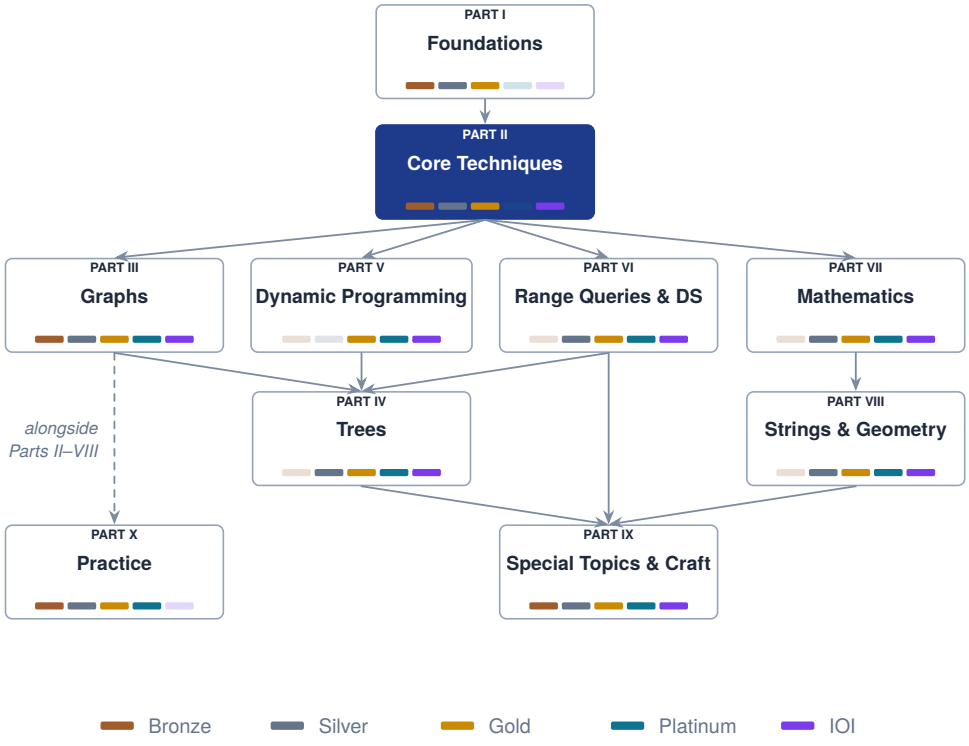
ORDERED SETS AND SIMULATION

- Concert Tickets
- Playlist
- Josephus Problem I
- Josephus Problem II
- Sliding Median
- Sliding Cost

PART II

Core Techniques

The techniques that solve most Bronze and Silver problems and appear inside nearly every harder one: exhaustive search, sorting and greedy choices, binary search, prefix sums, two pointers and bit manipulation.



CHAPTER 6

Simulation and Complete Search

The simplest correct algorithm is to try everything. When the input is small enough, that is also the intended solution; when it is not, a complete search is still where you begin: it pins down exactly what you are computing, it is the reference you stress test against, and it often earns the points of the small subtasks. This chapter is about doing exhaustive work *systematically*: simulating a process exactly as described, enumerating subsets and permutations without missing or repeating any, and organising a search so that it abandons hopeless branches early. It closes with three techniques that stretch the reach of exhaustive search far beyond its naive limits: meet in the middle, fracturing search, and the brute-force-assisted casework that solves so many ad hoc problems.

IN THIS CHAPTER BRONZE SILVER GOLD IOI / ADVANCED

- simulate processes correctly, and skip ahead when a state repeats;
- estimate the size of a search space and choose the smallest one;
- enumerate subsets (bitmasks, Gray codes, submasks, k-subsets) and permutations;
- write backtracking searches and prune them by orders of magnitude;
- halve an exponent with meet in the middle, and list the k best solutions with fracturing search;
- solve casework problems by tabulating small cases with a brute force.

Before you start: Chapters 1 to 4.

6.1 Simulation BRONZE

Some problems ask you to do exactly what the statement describes: move a robot for T steps, apply a list of swaps, play a game by fixed rules, run a queue of customers through a shop. There is no clever algorithm to discover; the whole difficulty is in getting every detail right, and a handful of habits turn that from a gamble into a routine.

- **Make the state explicit.** Write down exactly what describes the situation at one moment (positions, directions, counters, whose turn it is) and keep it in a few named variables or a struct. Everything the rules mention must be part of the state, and nothing else should be.
- **Write one step function.** A single piece of code that advances the state by one step is far easier to check than logic spread across a loop. Test it on the first steps of the sample before running it for long.
- **Use direction arrays for grids.** Store the four directions in clockwise order (up, right, down, left) as `dr = {-1, 0, 1, 0}` and `dc = {0, 1, 0, -1}`. Then moving forward is `r += dr[d], c += dc[d]`, turning right is `d = (d + 1) % 4`, turning left `d = (d + 3) % 4` and turning around `d ^= 2`. Eight directions work the same way with arrays of length eight.
- **Pad the grid.** Surrounding the grid with a border of blocked cells removes every bounds check: a step off the grid simply hits a wall. The Grid Paths solution in Section 6.5 uses this.

- **Trace the samples.** Print the state after every step with `dbg` and compare it with a hand simulation of the first sample; the first line that differs is where the bug is.

6.1.1 When T is huge: find the cycle

If the number of possible states is finite, a deterministic process must eventually revisit a state, and from then on it repeats forever, because the next state depends only on the current one. So a process that runs for $T = 10^{18}$ steps is never simulated step by step. Record the time at which each state first appears; when a state appears again at time t, having first appeared at time s, the process has a **cycle** of length $L = t - s$, and the state at any time $T \geq s$ equals the state at time $s + (T - s) \bmod L$. Equivalently, jump over $\lfloor (T - t)/L \rfloor$ whole cycles at once and simulate the fewer than L steps that remain. Drawn as a sequence of states, the process has the shape of the Greek letter ρ: a *tail* of states that are never seen again, leading into the cycle (Figure 6.1).

Example: Robot on a grid

A robot stands on a grid with walls, facing up. At each step it moves forward if the cell in front is free and inside the grid; otherwise it turns right (and does not move in that step). Where is it after T steps? Limits: grid up to 1000×1000, $T \leq 10^{18}$.

Solution. The state is the triple (row, column, direction), so there are at most $4nm = 4 \cdot 10^6$ states. Store the first time each state occurs in an array indexed by `(r * m + c) * 4 + d`. At the first repetition, skip all complete cycles that fit before T, then simulate the rest. Before the repetition at most $4nm$ steps are simulated, and after it fewer than $L \leq 4nm$, so the total work is $O(nm)$ whatever the value of T.

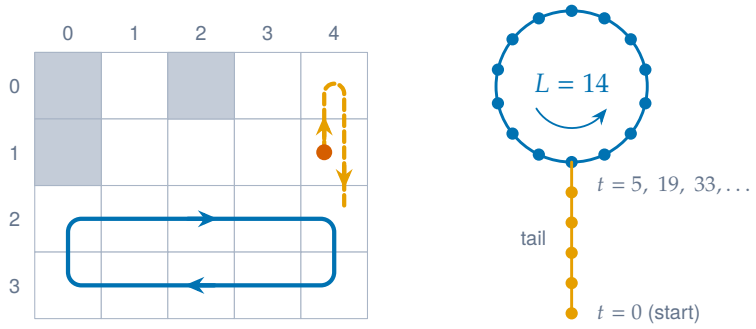


Figure 6.1: A robot run on a 4×5 grid with three walls (grey), starting at the dot facing up. It goes up, turns right twice at the edge, comes back down (orange tail) and then circles rows 2 and 3 forever (blue cycle). Right: the same run as a sequence of states. The state at time 5 recurs at time 19, so for every $T \geq 5$ the state at time T is the state at time $5 + (T - 5) \bmod 14$.

```
void solve() {
    int n, m;
    ll T;
    cin >> n >> m >> T;
    vector<string> g(n);
    cin >> g;
    const int dr[4] = {-1, 0, 1, 0}, dc[4] = {0, 1, 0, -1};  // U, R, D, L
    int r = 0, c = 0, d = 0;
    rep(i, n) rep(j, m) if (g[i][j] == 'R') r = i, c = j;
    vector<ll> seen(4 * n * m, -1);  // first time each state (r, c, d) occurred
```

```
    ll t = 0;
    bool skipped = false;
    while (t < T) {
        int id = (r * m + c) * 4 + d;
        if (!skipped && seen[id] >= 0) {  // repeated state: skip whole cycles
            ll len = t - seen[id];
            t += (T - t) / len * len;
            skipped = true;
            if (t == T) break;
        }
        seen[id] = t;
        int nr = r + dr[d], nc = c + dc[d];
        bool ok = nr >= 0 && nr < n && nc >= 0 && nc < m && g[nr][nc] != '#';
        if (ok) r = nr, c = nc;
        else d = (d + 1) % 4;  // blocked: turn right
        t++;
    }
    print(r + 1, c + 1);
}
```

After the jump, the flag `skipped` stops the program from detecting the same cycle again, and the loop simulates the remaining steps normally. On a 1000×1000 grid with $T = 10^{18}$ the program runs in 0.08 s. When the states are too many to store, or are not small integers, Floyd's and Brent's cycle-finding algorithms find the same tail and cycle lengths with $O(1)$ memory by running two copies of the process at different speeds (Chapter 11); when many queries ask for the state after T steps from *different* starting states, binary lifting answers each in $O(\log T)$ (Chapter 16).

> **⚠ The state must be complete** Cycle skipping is correct only if the state captures *everything* that influences the future. Forgetting the direction, a counter, a timer that changes the rules every k steps, or "whose turn it is" makes two different situations look identical and produces a wrong cycle, and often the samples still pass. When a step also depends on t itself (for example "every tenth step is special"), add t mod 10 to the state.

6.2 Complete search with loops

BRONZE

A **complete search** enumerates every candidate answer, checks each one, and keeps the best valid one. Its correctness is obvious, which is its great strength: the only questions are whether the candidates really cover every possible answer, and whether there are few enough of them. When a candidate is described by a few numbers, nested loops are all you need: all pairs of points, all triples of indices, all rectangles given by two opposite corners, all starting times of a schedule. The limits tell you how many nested loops you can afford: two loops over $n = 5000$ or three over $n = 400$ are about $2.5 \cdot 10^7$ and $6.4 \cdot 10^7$ steps.

The skill that makes complete search powerful is reducing the number of candidates *without losing the optimum*. Very often an infinite or huge space of candidates contains a small set of *critical values* that is guaranteed to contain an optimal answer, and only those need to be tried.

Example: The most covered point

Given $n \le 2000$ intervals $[a_i, b_i]$ with real endpoints, find a point covered by as many intervals as possible.

Solution. The point can be any real number, but some optimal point is always a left endpoint a_i: take any optimal point x and slide it to the left until it reaches the nearest left endpoint $a_i \le x$. While sliding, the point never leaves an interval (it would have to pass that interval's left endpoint first), so the number of covering intervals cannot decrease. Therefore it suffices to try the n left endpoints and count the intervals covering each, which is $O(n^2)$ (Figure 6.2). The sweep of Section 2.5 solves the same problem in $O(n \log n)$, and the argument above is exactly why the sweep only needs to look at endpoints.

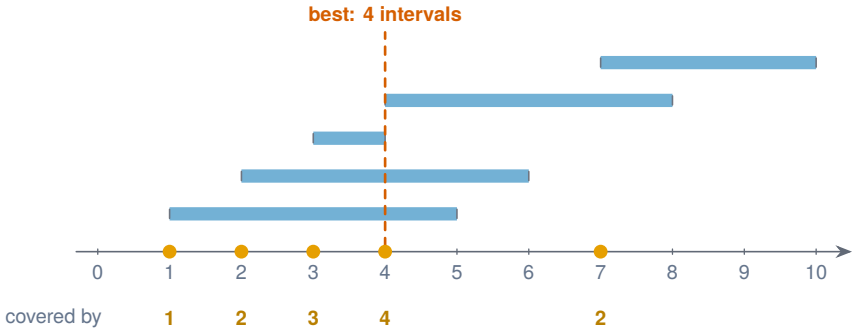


Figure 6.2: Five intervals. Only the left endpoints (orange dots) are candidate answers: sliding any point to the left until it reaches a left endpoint never loses an interval. The counts below the axis are the numbers of intervals covering each candidate; the best is 4, at $x = 4$.

How big is the search space? Before writing a search, count its candidates and compare with the time limit. Table 6.1 lists the spaces that come up again and again, with the largest n for which they hold about 10^8 candidates. Two rules of thumb follow from it: enumerating subsets is fine up to $n \approx 20$ (where each subset costs $O(n)$ to process), and enumerating permutations up to $n \approx 10$.

Search space	Size	n at $\approx 10^8$	Typical use
subsets of n items	2^n	26	choose any group (Apple Division)
subsets of size $n/2$	$\binom{n}{n/2}$	29	split into two equal halves
permutations	$n!$	11	orders, schedules (Creating Strings)
pairs (mask, submask)	3^n	17	DP over subsets of subsets
partitions into groups	Bell number B_n	13	split items into any number of teams
balanced bracket sequences	Catalan C_n	17	expressions, binary tree shapes
partitions of the integer n	$p(n)$	95	write n as a sum of positive parts
one half, meet in the middle	$2 \cdot 2^{n/2}$	54	subset sums (Section 6.6)

Table 6.1: Sizes of common search spaces. The third column is the largest n for which the space has about 10^8 elements; if each candidate costs more than a few operations, reduce it accordingly.

Search the smaller space Before searching, compare the space of *inputs* with the space of *answers*, and search whichever is smaller. If the answer is a small integer (a number of bits, a number of distinct values, a length up to 60), loop over the answer and check each value. If an optimum is always attained at an input value (a median, an endpoint, a coordinate of some point), try only those. And if a rectangle or a line is optimal, it can usually be moved until it touches input points, so only the rectangles or lines determined by one or two input points need to be tried.

6.3 Generating subsets

BRONZE

A set of n items has 2^n subsets. Represent a subset by an n-bit number, a **bitmask**, whose bit i is 1 exactly when item i belongs to the subset. Counting from 0 to $2^n - 1$ then visits every subset exactly once (Figure 6.3, left), and set operations become single machine instructions: union is `a | b`, intersection `a & b`, difference `a & ~b`, "is item i in S?" is `S >> i & 1`, and the size of S is `__builtin_popcount(S)`. Chapter 10 collects many more bit operations.

counting order

mask	c	b	a	subset
0	0	0	0	$\emptyset$
1	0	0	1	$\{a\}$
2	0	1	0	$\{b\}$
3	0	1	1	$\{a,b\}$
4	1	0	0	$\{c\}$
5	1	0	1	$\{a,c\}$
6	1	1	0	$\{b,c\}$
7	1	1	1	$\{a,b,c\}$

Gray code order

k	c	b	a	change	subset
0	0	0	0		$\emptyset$
1	0	0	1	$+a$	$\{a\}$
2	0	1	1	$+b$	$\{a,b\}$
3	0	1	0	$-a$	$\{b\}$
4	1	1	0	$+c$	$\{b,c\}$
5	1	1	1	$+a$	$\{a,b,c\}$
6	1	0	1	$-b$	$\{a,c\}$
7	1	0	0	$-a$	$\{c\}$

Figure 6.3: The 8 subsets of $\{a,b,c\}$ (bit 0 is a, bit 1 is b, bit 2 is c; blue bits are chosen items). Left: in counting order, one step can change many bits ($3 \to 4$ changes all three). Right: in Gray code order $g_k = k \oplus \lfloor k/2 \rfloor$, every step flips exactly one bit (red), and step k flips bit number $\mathrm{ctz}(k)$.

Example: Apple Division (CSES)

Divide $n \le 20$ apples with given weights into two groups so that the difference of the group weights is as small as possible.

Solution. Try all 2^n splits: the mask lists the apples that go into the first group, and the second group weighs the total minus the first. With $n = 20$ that is about 10^6 masks times 20 bits each, $2 \cdot 10^7$ simple operations.

```
void solve() {
    int n;
    cin >> n;
    vi p(n);
    cin >> p;
    ll total = accumulate(all(p), 0LL), best = LLONG_MAX;
    rep(mask, 1 << n) {  // the apples in the first group
        ll s = 0;
```

```
        rep(i, n) if (mask >> i & 1) s += p[i];
        chmin(best, abs(total - 2 * s));  // |group 1 - group 2|
    }
    print(best);
}
```

</> Template: All subsets (bitmask loop) Ctrl+Alt+T, 8 prefix `cpsubsets`

6.3.1 Faster subset loops

The loop above spends $O(n)$ on each subset to recompute its sum, $O(n2^n)$ in total. Two classic tricks remove the factor n, and both matter when n is 25 or when the work per subset is heavier than a sum.

Incremental tables. Every non-empty mask is a smaller mask plus one item, its lowest set bit. So `sum[mask] = sum[mask & (mask - 1)] + a[__builtin_ctz(mask)]` fills a table of all 2^n subset sums in $O(2^n)$, in increasing order of masks, because `mask & (mask - 1)` (the mask without its lowest bit) is always smaller. The same works for any quantity that can be extended by one item: the number of edges inside a vertex set, a bitwise OR, a product modulo a prime.

Gray code order. When the per-subset state is a whole structure that cannot be stored 2^n times (a multiset, a DSU, a board position), visit the subsets in **Gray code** order instead: consecutive subsets differ in exactly one item (Figure 6.3, right), so the structure is updated by a single insertion or deletion per step. The k-th subset is $g_k = k \oplus \lfloor k/2 \rfloor$, and going from g_{k-1} to g_k flips bit $\mathrm{ctz}(k)$, the number of trailing zeros of k. The template generates this order.

</> Template: Gray code order Ctrl+Alt+U, 0 prefix `cpgray`

6.3.2 Submasks and k-subsets

To iterate over all **submasks** of a mask S (the subsets of the set S), including S itself and the empty set, use `for (int T = S; ; T = (T - 1) & S) { ...; if (T == 0) break; }`. Subtracting one clears the lowest set bit of T and sets all the bits below it; masking with S keeps only the bits that belong to S, and the result is exactly the next smaller submask. The loop visits the $2^{|S|}$ submasks in decreasing order. Summed over all masks S of n bits it visits $\sum_S 2^{|S|} = 3^n$ pairs, because each of the n items is independently outside S, in S but not in T, or in both: this is the 3^n of Table 6.1, and the basis of the subset DPs of Chapter 19.

Gosper's hack: all subsets of size k To visit only the $\binom{n}{k}$ masks with exactly k bits (for $k \geq 1$), in increasing order, jump from each such mask directly to the next one:

```
for (int x = (1 << k) - 1; x < (1 << n);) {
    // ... use x ...
    int low = x & -x, r = x + low;  // r: the lowest block of ones carried
    x = (((r ^ x) >> 2) / low) | r;  // move the rest of the block to the bottom
}
```

Adding the lowest set bit `low` turns the lowest block of consecutive ones into a single one bit, one position higher; the other ones of that block are then shifted back down to the lowest

positions. This is **Gosper's hack**, and it beats filtering all 2^n masks by popcount whenever $\binom{n}{k}$ is much smaller than 2^n.

6.4 Generating permutations

BRONZE

An ordering of n items is a permutation; there are $n!$ of them ($8! = 40\,320$, $10! \approx 3.6 \cdot 10^6$, $12! \approx 4.8 \cdot 10^8$), so enumerating them is feasible up to about $n = 10$. The standard library does the work: `next_permutation` rearranges a range into the next permutation in lexicographic order and returns `false` after the last one (when it wraps around to the sorted order). Starting from the sorted order therefore visits every permutation once. Because it compares values, a sequence with repeated values yields each *distinct* arrangement exactly once: the four letters of `aabb` give $4!/(2!\,2!) = 6$ arrangements, not 24.

The algorithm behind `next_permutation` is short and worth knowing, because variants of it solve many problems ("the next larger number with the same digits", "the lexicographically next valid bracket sequence"). Scan from the right for the longest *non-increasing* suffix: it is already the largest arrangement of its elements, so it cannot be increased on its own. The element just before it, the *pivot*, must grow, and by as little as possible: swap it with the smallest element of the suffix that is larger than it. The suffix is still non-increasing after the swap, so reversing it sorts it into its smallest arrangement (Figure 6.4). Each call costs $O(n)$ in the worst case and $O(1)$ on average over all permutations.

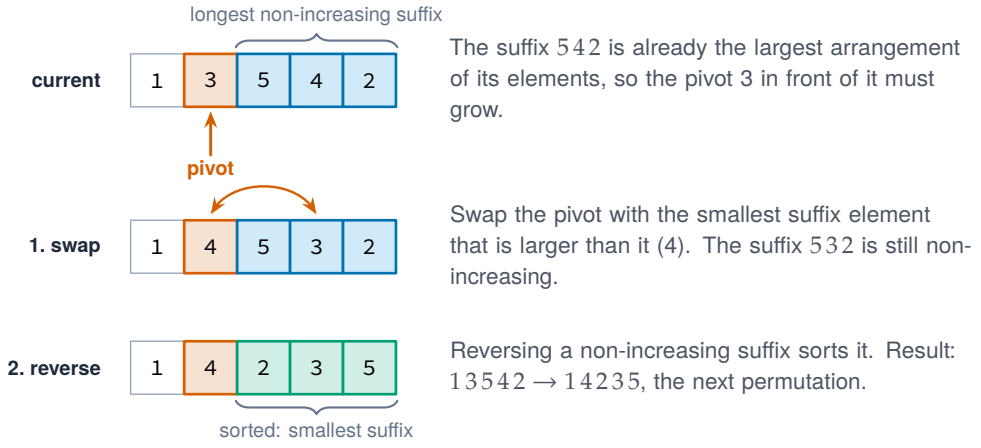


Figure 6.4: How `next_permutation` finds the next arrangement in lexicographic order: find the pivot, swap it with its successor in the suffix, reverse the suffix. Only the pivot and the suffix change.

Example: Creating Strings (CSES)

Print all distinct rearrangements of a string of at most 8 letters, in alphabetical order, preceded by their number.

```
void solve() {
    string s;
    cin >> s;
    sort(all(s));
    vector<string> out;
    do out.push_back(s);
```

```
    while (next_permutation(all(s)));  // each distinct arrangement once
    print(sz(out));
    for (auto& t : out) print(t);
}
```

</> Template: All permutations Ctrl+Alt+T, 9 prefix `cpperm`

⚠ Start sorted `next_permutation` only visits the permutations that come *after* the starting sequence. Forgetting to sort first silently skips part of the search space, and the samples, which are often already sorted, may still pass. When permuting indices rather than values, start from `iota(all(p), 0)`.

Jump straight to the k-th permutation The permutations of $0, \ldots, n-1$ in lexicographic order come in n blocks of $(n-1)!$, one block per first element; inside a block the remaining elements again form blocks of $(n-2)!$, and so on. So the k-th permutation (counting from 0) takes as its first element the $\lfloor k/(n-1)! \rfloor$-th smallest unused value, and continues with $k \bmod (n-1)!$. The successive quotients are the digits of k in the **factorial number system** (the **Lehmer code** of the permutation), and running the procedure backwards gives the rank of a permutation. Both take $O(n^2)$ as written, or $O(n \log n)$ with a Fenwick tree that finds the d-th unused value (Chapter 22).

```
vi kth_perm(int n, ll k) {  // n <= 20
    vector<ll> f(n + 1, 1);
    rep(i, 1, n + 1) f[i] = f[i - 1] * i;
    vi pool(n), out;
    iota(all(pool), 0);
    for (int i = n - 1; i >= 0; i--) {
        int d = k / f[i];  // skip d whole blocks of i! permutations
        k %= f[i];
        out.push_back(pool[d]);
        pool.erase(pool.begin() + d);
    }
    return out;
}
```

6.5 Backtracking and pruning

SILVER

Backtracking builds a solution one choice at a time and abandons a partial solution as soon as it is clear that it cannot be completed into a valid (or better) one. The partial solutions form a *search tree*: the root is the empty solution, the children of a node are its extensions by one more choice, and the leaves are complete candidates. A plain loop over all candidates visits every leaf; backtracking walks the tree depth first and *prunes*, that is, refuses to enter a subtree once it knows that the subtree contains nothing useful. A single pruned node near the root removes an exponential number of leaves, which is why the difference between a search that runs in a second and one that runs for a year is almost always **pruning**: how early, and how often, the search can say no.

Algorithm: Backtracking

```
1 procedure Search(depth)
2     if the partial solution is complete then
3         record it (count it, or compare it with the best so far)
4         return
5     for all choices x for the next position do
6         if x is consistent with the choices so far and can still win then
7             apply x to the shared state                ▷ mark a column, add a weight, ...
8             Search(depth + 1)
9             undo x                                     ▷ restore the state exactly
```

The *undo* line is the heart of backtracking. The search keeps one shared state, modifies it before descending and restores it after returning, so no copies are ever made and each step costs $O(1)$. Forgetting to undo, or undoing only part of the change, is the classic backtracking bug: the state after returning must be *identical* to the state before descending.

Example: Chessboard and Queens (CSES)

Place eight queens on an 8×8 board, avoiding reserved squares, so that no two queens attack each other. Count the ways.

Solution. Two queens in the same row attack each other, so every row holds exactly one queen: choose the column of the queen in row 0, then in row 1, and so on. A queen in row r, column c attacks its column, its diagonal (on which $r + c$ is constant) and its anti-diagonal (on which $r - c$ is constant). Three boolean arrays record which columns and diagonals are taken, so each square is checked in $O(1)$. Figure 6.5 shows the complete search tree for four queens: the search visits 16 placements, while the tree of all ways to put one queen in each row has $4 + 16 + 64 + 256 = 340$ nodes.

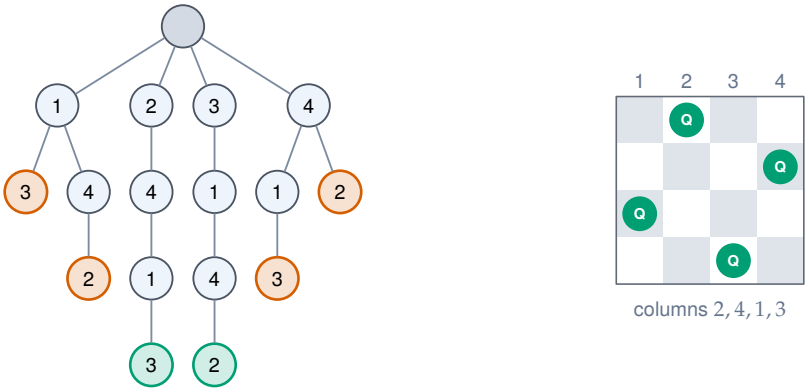


Figure 6.5: Left: the whole backtracking tree for four queens; a node is the column of the queen placed in the next row. Red nodes are dead ends (the following row has no safe square), green nodes complete a solution. Right: the solution $2, 4, 1, 3$; the other one is its mirror image $3, 1, 4, 2$.

```
void solve() {
    vector<string> b(8);
    cin >> b;
    array<bool, 8> col{};
    array<bool, 15> diag{}, anti{};  // r + c, and r - c + 7
    ll ways = 0;
```

```
    auto place = [&](auto&& self, int r) -> void {  // one queen per row
        if (r == 8) {
            ways++;
            return;
        }
        rep(c, 8) {
            bool taken = col[c] || diag[r + c] || anti[r - c + 7];
            if (b[r][c] == '*' || taken) continue;
            col[c] = diag[r + c] = anti[r - c + 7] = true;
            self(self, r + 1);
            col[c] = diag[r + c] = anti[r - c + 7] = false;  // undo
        }
    };
    place(place, 0);
    print(ways);
}
```

</> Template: Recursive complete search Ctrl+Alt+T, Shift+Y prefix `cpbacktrack`

6.5.1 A case study in pruning: Grid Paths

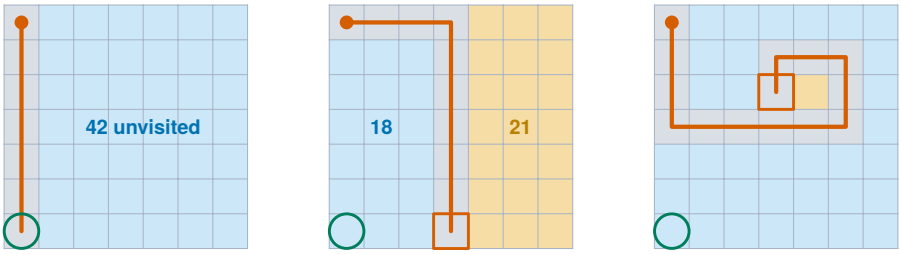
Example: Grid Path Description (CSES)

A path on a 7×7 grid starts in the upper-left corner, ends in the lower-left corner, and visits every one of the 49 squares exactly once, so it consists of 48 moves. You are given a description of 48 characters, each `U`, `D`, `L`, `R` (the move is fixed) or `?` (any move). Count the paths that match the description.

A naive search tries up to three directions at each of 48 steps, $3^{47} \approx 2.7 \cdot 10^{22}$ sequences in the worst case (a description of 48 question marks). Refusing to step onto visited squares already cuts this enormously, but the search still visits $1.9 \cdot 10^{10}$ nodes and needs two minutes (Table 6.2). Two observations about *dead* partial paths bring it down to a quarter of a second (Figure 18.6).

1. **The end is reached too early.** Every path must end at the lower-left corner, so a path that arrives there before its 48th move can never be completed: stop at once instead of wandering on.
2. **The free squares split in two.** Suppose the squares above and below the current square are blocked (visited or outside the grid) while the squares to its left and right are both free, or the same with the two directions exchanged. The blocked squares, the path and the border then form a barrier through the current square, and the free squares on its two sides are disconnected from each other. The path can continue into only one side and can never come back, so the squares on the other side will never be visited: the partial path is dead.

The second rule fires far more often than one would guess: every time the path runs straight into a wall or into its own earlier trail with space on both sides. Because it cuts subtrees close to the root, it removes 99.6% of the remaining search.



(a) the end corner is reached after 6 moves: stop

(b) blocked below, free left and right: one side is lost

(c) blocked above and below by its own trail: the pocket is lost

Figure 6.6: Dead partial paths in Grid Paths. The path (red) starts at the dot; the green circle marks the end corner and the red square the current position. Blue squares are free; orange squares are free but can no longer be reached once the path chooses a side. In (b) and (c) the current square is blocked on two opposite sides and free on the other two, which is exactly the test in the code.

Pruning rules	Search nodes	Time
never step onto a visited square	$1.9 \cdot 10^{10}$	120 s
+ stop when the end corner is reached	$1.1 \cdot 10^{10}$	54 s
+ stop when the free squares split in two	$4.7 \cdot 10^{7}$	0.24 s

Table 6.2: Counting all 88 418 paths (a description of 48 question marks) with more and more pruning, measured with `g++ -O2`. The split rule alone makes the search 230 times faster.

```
void solve() {
    string s;
    cin >> s;
    // the 7 x 7 grid is rows and columns 1..7, surrounded by blocked cells
    array<array<bool, 9>, 9> vis{};
    rep(i, 9) vis[i][0] = vis[i][8] = vis[0][i] = vis[8][i] = true;
    const int dr[4] = {1, -1, 0, 0}, dc[4] = {0, 0, -1, 1};
    const string dirs = "DULR";
    ll ways = 0;
    auto go = [&](auto&& self, int r, int c, int step) -> void {
        if (r == 7 && c == 1) {  // every path ends here, so stop here
            if (step == 48) ways++;
            return;
        }
        // blocked on two opposite sides, free on the other two: the free
        // cells split into two parts and one of them can never be visited
        bool up = vis[r - 1][c], down = vis[r + 1][c];
        bool left = vis[r][c - 1], right = vis[r][c + 1];
        if (up && down && !left && !right) return;
        if (left && right && !up && !down) return;
        vis[r][c] = true;
        rep(k, 4) {
            if (s[step] != '?' && s[step] != dirs[k]) continue;
            int nr = r + dr[k], nc = c + dc[k];
            if (!vis[nr][nc]) self(self, nr, nc, step + 1);
        }
        vis[r][c] = false;
    };
```

```
    go(go, 1, 1, 0);
    print(ways);
}
```

The border of blocked cells makes every neighbour access safe and turns "outside the grid" into "visited", which is exactly what both pruning rules need. The search never reads `s[48]`: a path that has made 48 moves has visited all 49 squares, including the end corner, and the search stops as soon as it reaches that corner.

Pruning that matters

- **Check constraints early.** Test a choice when it is made, not when the solution is complete; test the most restrictive constraints first.
- **Look for dead states**, as in Grid Paths: parity, connectivity of what remains, or a resource that can no longer suffice ("the remaining items cannot fill the remaining capacity").
- **Bound optimistically.** If the current cost plus a lower bound on the cost of the rest cannot beat the best solution so far, stop. This is **branch and bound**; a cheap but tight bound is worth more than a clever search order.
- **Order the choices** so that good solutions are found first: the better the first solution, the more the bound prunes. Greedy order (largest item first, nearest city first) is a good default.
- **Choose the most constrained variable next**: in Sudoku-like problems, fill the cell with the fewest remaining options, so that contradictions surface near the root.
- **Break symmetry.** Fix the first element, or require choices in increasing order when their order does not matter; this divides the search by the size of the symmetry group (up to $n!$).
- **Memoise.** If the same partial state can be reached along different paths, you are really doing dynamic programming (Part V), and caching the result of each state turns exponential time into polynomial.

6.6 Meet in the middle

GOLD

For n around 40, the $2^{40} \approx 10^{12}$ subsets are far too many, but $2^{20} \approx 10^6$ is nothing. **Meet in the middle** splits the items into two halves, enumerates the subsets of each half separately, and combines the two lists with sorting and two pointers, binary search or a hash map. Every subset of all the items is a pair (subset of the left half, subset of the right half), so questions about the whole set become questions about pairs from two lists of $2^{n/2}$ elements each, and those can be answered in about $2^{n/2}$ time instead of 2^n.

Example: Meet in the Middle (CSES)

Count the subsets of $n \le 40$ numbers (each up to 10^9) whose sum is exactly x.

Solution. Enumerate the subset sums L of the first half and R of the second half. The answer is the number of pairs $(a, b) \in L \times R$ with $a + b = x$. With both lists sorted, walk one pointer up L and another down R: if $a + b < x$, the current a is too small for every remaining b, so move

up in L; if $a + b > x$, move down in R; if they are equal, the runs of equal values on both sides contribute the product of their lengths (Figure 6.7).

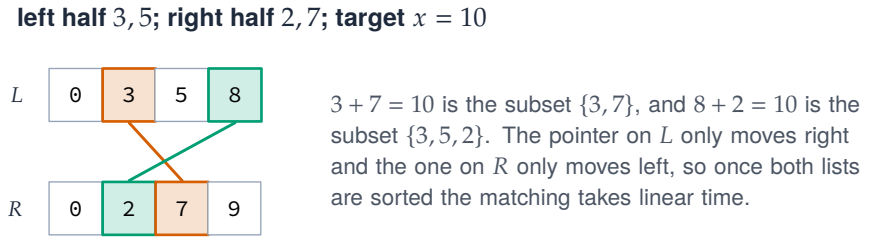


Figure 6.7: Meet in the middle for subset sums. L and R hold the $2^{n/2}$ subset sums of each half, sorted. A subset of all items is one sum from each list, so the pairs adding up to x are exactly the subsets with sum x.

```
void solve() {
    int n;
    ll x;
    cin >> n >> x;
    vi t(n);
    cin >> t;
    auto sums = [&](int lo, int hi) {  // all subset sums of t[lo..hi), sorted
        vector<ll> s{0};
        rep(i, lo, hi) {  // merge s with s + t[i]: stays sorted, linear time
            vector<ll> shifted(sz(s)), merged(2 * sz(s));
            rep(j, sz(s)) shifted[j] = s[j] + t[i];
            merge(all(s), all(shifted), merged.begin());
            s = merged;
        }
        return s;
    };
    vector<ll> L = sums(0, n / 2), R = sums(n / 2, n);
    ll ways = 0;
    int i = 0, j = sz(R) - 1;  // two pointers: L ascending, R descending
    while (i < sz(L) && j >= 0) {
        if (L[i] + R[j] < x) i++;
        else if (L[i] + R[j] > x) j--;
        else {  // equal: multiply the sizes of the two runs of equal values
            ll a = L[i], b = R[j], ca = 0, cb = 0;
            while (i < sz(L) && L[i] == a) i++, ca++;
            while (j >= 0 && R[j] == b) j--, cb++;
            ways += ca * cb;
        }
    }
    print(ways);
}
```

Generate the sums already sorted Sorting 2^{20} numbers costs a factor of 20. Instead, build the sums one item at a time: if s is the sorted list of sums of the first i items, the list for $i + 1$ items is the *merge* of s and $s + t_i$, two sorted lists, in linear time. The sizes double at each step, so the whole half costs $O(2^{n/2})$, and the program above runs in 0.1 s for $n = 40$.

</> Template: Meet in the middle (subset sums) Ctrl+Alt+T, Shift+U prefix `cpmitm`

Variations. The combining step depends on the question. To find the subset sum closest to x, binary search each $a \in L$ for $x - a$ in R. For subsets with a given *XOR*, count matching values with a hash map. To count subsets with sum *at most* x, count for each a the elements of R that are at most $x - a$, again with two pointers. Beyond subsets, the idea applies to any search of depth d whose two halves can be combined: a search of b^d states becomes two of $b^{d/2}$ each. This is how bidirectional BFS finds shortest paths in huge implicit graphs (Chapter 11), and how 4-SUM ($a + b + c + d = S$) becomes pairs (a, b) against pairs (c, d) in $O(n^2 \log n)$.

> **How to recognize it** $n \le 40$ (or 36, or 45) together with a task that is exponential in n: subset sums, subset XORs, "split into two groups", knapsack with huge weights. More generally, any search where $b^{d/2}$ states are affordable but b^d are not, and where a solution can be cut into two independent halves that meet in a simple condition.

6.7 Fracturing search: the k best solutions

IOI / ADVANCED

Sometimes you need not only the best solution but the k best: the k smallest subset sums, the k cheapest spanning trees, the k shortest paths. **Fracturing search** finds them by arranging *all* solutions into a tree in which every child is no better than its parent and every solution appears exactly once. A priority queue then produces solutions in order: pop the best node, report it, push its children. The first pop is the root, the best solution overall; each later pop is the best node whose parent has already been reported, and because children are never better than their parents, that is the best solution not yet reported. After k pops you have the k best solutions, and the queue never holds more than $k \times$ (number of children) $+ 1$ nodes.

Example: The k smallest subset sums

Given n positive numbers, list the k smallest sums over all non-empty subsets.

Solution. Sort the numbers and describe a subset by its sum and the index i of its largest element. Every subset other than $\{a_0\}$ has exactly one *parent*, from which it arises by one of two moves: *add* the element a_{i+1}, or *replace* the largest element a_i by a_{i+1}. (To find the parent of a subset with largest index j: if index $j-1$ is also in the subset, remove j; otherwise replace j by $j-1$.) Neither move decreases the sum, because the numbers are sorted and positive, so the tree rooted at $\{a_0\}$ is heap-ordered, and popping the queue k times lists the k smallest sums (Figure 6.8).

```
void solve() {
    int n, k;
    cin >> n >> k;
    vi a(n);
    cin >> a;
    sort(all(a));
    minpq<pll> pq;  // (sum, i): a subset whose largest element is a[i]
    pq.push({a[0], 0});
    vi out;
    while (sz(out) < k) {
        auto [s, i] = pq.top();
        pq.pop();
```

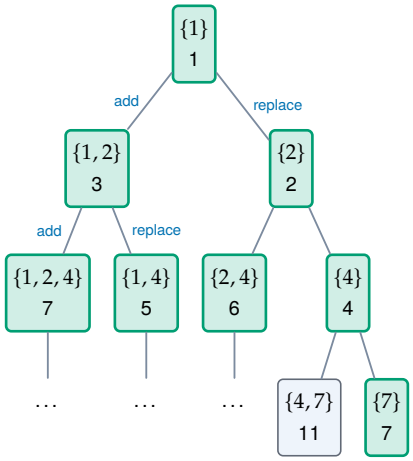


Figure 6.8: The successor tree for $a = 1, 2, 4, 7$; each node shows a subset and its sum. Left children add the next element, right children replace the largest element by the next one. Sums never decrease downwards, so a priority queue pops them in order: $1, 2, 3, 4, 5, 6, 7, 7, \ldots$ (green: the first eight pops).

```
        out.push_back(s);
        if (i + 1 < n) {  // two children; together they reach every subset once
            pq.push({s + a[i + 1], i + 1});         // also take a[i + 1]
            pq.push({s - a[i] + a[i + 1], i + 1});  // swap a[i] for a[i + 1]
        }
    }
    print(out);
}
```

The general recipe: find a way to *fracture* the solution space into disjoint parts, each described by a few constraints and each with an easily computed best element, such that removing the best element of a part leaves a few smaller parts of the same form. Then let the priority queue explore the parts in order of their best elements. For the k shortest paths, a part is "paths that share this prefix and then avoid these edges"; for the k smallest spanning trees it is "trees that contain these edges and avoid those". The total cost is $O(k \cdot (c \cdot B + \log k))$, where c is the number of children per node and B the cost of finding the best element of a part.

6.8 Casework and ad hoc problems

BRONZE

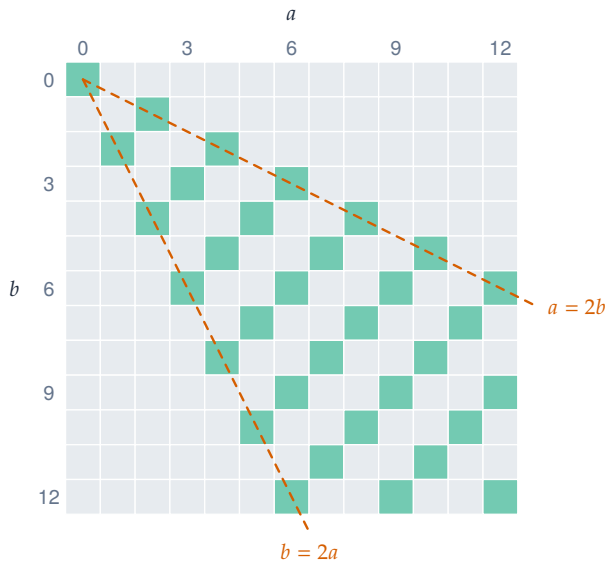
Many Bronze problems and most Codeforces Division 2 A and B problems have no named algorithm. The solution is a single observation, or a careful division into cases, and such problems reward one specific habit: *compute small cases with a brute force and look at the table*. The brute force is a complete search, which you can write in minutes and trust; the table shows you the rule; and the rule is usually much easier to prove once you know what it is.

Example: Coin Piles (CSES)

Two piles hold a and b coins. One move removes one coin from one pile and two coins from the other. Can both piles be emptied? Answer up to 10^5 such questions with $a, b \le 10^9$.

Solution. Tabulate the answers for small a and b with a brute force (Figure 6.9). Two facts jump out, and both are easy to prove. Every move removes three coins, so $a + b$ must be divisible by 3. And every move removes at most two coins from one pile while removing at least one from

the other, so if one pile holds more than twice the other, the larger pile can never be emptied. The table suggests that these two conditions are also sufficient, and indeed they are: if both hold, then $x = (2a - b)/3$ moves of the first kind and $y = (2b - a)/3$ of the second kind are non-negative integers that empty both piles.



Green: both piles can be emptied. The green cells lie on every third anti-diagonal ($a + b \equiv 0 \pmod 3$) and inside the wedge between the lines $b = 2a$ and $a = 2b$.

Figure 6.9: Coin Piles for $0 \le a, b \le 12$, computed by brute force. Looking at such a table is often the fastest way to find the rule; proving it afterwards takes two lines.

```
void solve() {
    int t;
    cin >> t;
    while (t--) {
        ll a, b;
        cin >> a >> b;
        // each move removes 3 coins; no pile may exceed twice the other
        yn((a + b) % 3 == 0 && 2 * min(a, b) >= max(a, b));
    }
}
```

Getting casework right List the cases on paper before coding, and check that they cover every input and do not overlap. For each case, construct a small example and check your formula on it by hand. The special values (0, 1, all elements equal, $n = 1$, $n = 2$, the maximum value) are where casework breaks, so test each of them. When the analysis grows beyond five cases, look for a simpler characterisation, or brute force a small parameter instead of reasoning about it.

Ad hoc habits Look for invariants (a sum, a parity or a remainder that no operation changes), extremal elements (what must happen to the maximum?), and better representations (differences $a_{i+1} - a_i$ instead of values, prefix sums instead of elements, positions instead of values). Above all, write the brute force, print a table of answers for all small inputs, and look at it from every angle: by rows, by columns, by diagonals, modulo small numbers. The

pattern is often the whole solution, and the brute force then doubles as the checker for your stress test (Chapter 2).

Chapter summary

- Simulate with an explicit state and one step function; when T is huge, find the cycle and skip it. The state must contain everything that influences the future.
- Count the search space first (Table 6.1), search the smaller of inputs and answers, and look for critical values.
- Subsets are bitmasks ($n \leq 20$); incremental tables and Gray codes remove the factor n; submask loops cost 3^n in total; Gosper's hack visits the k-subsets.
- Permutations come from `next_permutation` ($n \leq 10$), started from the sorted order; the factorial number system jumps to the k-th one.
- Backtracking modifies one shared state and undoes each change. Dead-state detection, bounds, good ordering and symmetry breaking can make it faster by factors of hundreds.
- Meet in the middle turns 2^n into $2^{n/2}$; build the half sums already sorted by merging.
- Fracturing search lists the k best solutions with a priority queue over a heap-ordered tree of all solutions.
- For ad hoc problems, tabulate small cases with a brute force and find the rule in the table.

Practice problems (CSES Problem Set)

SIMULATION AND ENUMERATION

- Number Spiral
- Two Knights
- Apple Division
- Creating Strings
- Gray Code
- Tower of Hanoi

BACKTRACKING, MEET IN THE MIDDLE AND AD HOC

- Chessboard and Queens
- Grid Path Description
- Meet in the Middle
- Coin Piles
- Two Sets
- Palindrome Reorder

CHAPTER 7

Sorting and Greedy Algorithms

Sorting is the most useful preprocessing step in competitive programming. Once data is ordered, equal values sit next to each other, extremes sit at the ends, and a whole family of questions collapses into a single left-to-right scan. Greedy algorithms, which build a solution by repeatedly making the choice that looks best right now, are sorting's natural partner: almost every greedy algorithm begins by sorting the input into the order in which its choices will be made. This chapter treats both, with an emphasis on the part that contests actually test: *finding* the right order, and *proving* that the greedy choice is safe.

IN THIS CHAPTER BRONZE SILVER GOLD

- use sorting to turn questions about pairs and groups into scans, and pick the right library routine;
- sort integers in linear time with counting sort and radix sort;
- design comparators with exchange arguments, and know when a comparator is legal;
- compress coordinates;
- prove greedy algorithms with "greedy stays ahead" and exchange arguments, and break wrong ones with counterexamples;
- combine greedy choices with heaps and ordered sets, including greedy algorithms that can undo their choices.

Before you start: Chapters 1 to 5, in particular the exchange argument of Section 1.5.

7.1 Sorting as a tool

BRONZE

`sort` runs in $O(n \log n)$: about 40 ms for 10^6 numbers (Table 7.1), so for typical limits it is almost free. What makes it so useful is what the order gives you afterwards.

- **Equal values are adjacent.** Counting duplicates, grouping items by key, and counting distinct values all become one scan: after sorting, a new group starts exactly where `a[i] != a[i - 1]`. The idiom `a.erase(unique(all(a)), a.end())` removes duplicates.
- **Order statistics are positions.** The k-th smallest value is `a[k - 1]`, the median is in the middle, and "how many values are below x" is a binary search (Chapter 8).
- **Closest values are neighbours.** For any $i < k < j$ in sorted order, $a_j - a_i \geq a_k - a_i$, so the minimum difference between two values is attained by two adjacent values. The same holds for "the value nearest to x": it is next to where x would be inserted.
- **Scans with two pointers become possible.** Once both of two lists are sorted, matching them (pairs with a given sum, assigning applicants to apartments) takes one linear pass (Chapter 9).
- **Sorting fixes the order of decisions.** A greedy algorithm needs to know which item to consider first; sorting by the right key is usually the whole difficulty.

A beautiful example of the last point, and of an invariant that holds throughout a scan of sorted data, is the following problem.

Example: Missing Coin Sum (CSES)

Given $n \le 2 \cdot 10^5$ coins with positive values, find the smallest sum that cannot be formed by choosing a subset of the coins.

Solution. Sort the coins and maintain the invariant *every sum* $0, 1, \ldots, r$ *can be formed with the coins seen so far*; initially $r = 0$. Take the next coin c. If $c \le r + 1$, then adding c to the sums $0, \ldots, r$ gives all sums $c, \ldots, r + c$, which together with $0, \ldots, r$ cover $0, \ldots, r + c$ without a gap, so r becomes $r + c$. If $c > r + 1$, then $r + 1$ can never be formed: the coins seen so far sum to exactly r (by induction, r is the sum of all coins so far), and every remaining coin is at least $c > r + 1$ by the sorted order. So the answer is $r + 1$ (Figure 7.1).

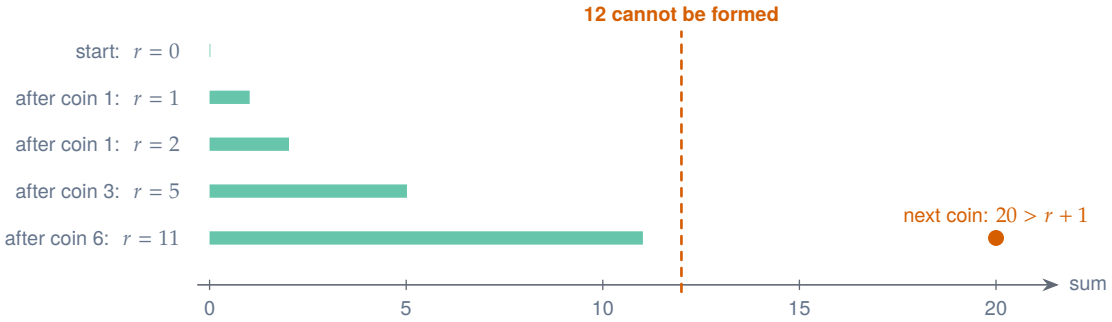


Figure 7.1: Missing Coin Sum with the sorted coins $1, 1, 3, 6, 20$. Each green bar is the range $0, \ldots, r$ of sums that can be formed after taking the coins so far. Each coin at most $r + 1$ extends the range without a gap; the coin 20 would leave the gap $12, \ldots, 19$, so the answer is 12.

```
void solve() {
    int n;
    cin >> n;
    vi x(n);
    cin >> x;
    sort(all(x));
    ll reach = 0;  // invariant: every sum 0, 1, ..., reach can be formed
    for (ll c : x) {
        if (c > reach + 1) break;  // then reach + 1 can never be formed
        reach += c;
    }
    print(reach + 1);
}
```

7.1.1 The sorting toolbox

The standard library offers more than `sort`, and choosing the right routine matters when the input is large or only part of the order is needed (Figure 7.2).

- `sort(all(a))` sorts in $O(n \log n)$ (introsort: quicksort that falls back to heapsort, so the worst case is guaranteed). Equal elements may be reordered.

- `stable_sort` keeps equal elements in their original order, at $O(n \log n)$ with extra memory. It is what you need when sorting by one key after another ("by age, and among equal ages by the original order").
- `partial_sort(a.begin(), a.begin() + k, a.end())` puts the k smallest elements, sorted, at the front, in $O(n \log k)$. The rest is left in unspecified order.
- `nth_element(a.begin(), a.begin() + k, a.end())` puts the element that belongs at position k there, with smaller or equal elements before it and larger or equal ones after it, in $O(n)$ on average: the fastest way to find a median or a threshold.
- Pairs, tuples and `arrays` compare lexicographically, so `sort` on a vector of `tuple<int, int, int>` sorts by the first component, then the second, then the third. To sort by the second component first, store it first.
- To sort indices by a key without moving the data (an **argsort**), sort `iota(all(ord), 0)` with a comparator that compares `key[i]` and `key[j]`. Keep the original indices whenever the output must refer to them.

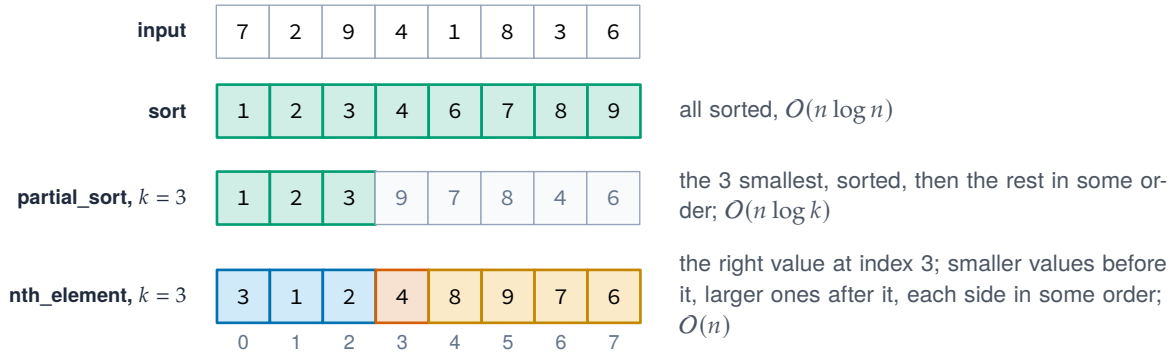


Figure 7.2: What `sort`, `partial_sort` and `nth_element` guarantee. Grey cells are in an unspecified order; with `nth_element`, blue cells are at most the value at index k and orange cells at least it.

</> Template: Argsort + custom comparator Ctrl+Alt+T, Shift+J prefix `cpargsort`

7.2 Counting sort and radix sort

SILVER

Comparison sorts cannot beat $O(n \log n)$: there are $n!$ possible orders and each comparison has two outcomes, so some input needs $\log_2 n! \approx n \log_2 n$ comparisons. Sorting *integers* escapes this bound by not comparing at all.

Counting sort. If the keys are small integers in $[0, K)$, count how many items have each key, turn the counts into starting positions with a prefix sum, and place every item at the next free position of its key (Figure 7.3). This takes $O(n + K)$ and is *stable*: items with equal keys keep their input order, because they are placed in input order. Stability is what makes the next algorithm work.

Radix sort. To sort d-digit numbers, counting-sort them by the *last* digit, then by the second-to-last, and so on up to the first digit (Figure 7.4). This is **radix sort**, least significant digit first. It is correct by induction: after the pass on digit j, the numbers are sorted by their last j digits, because the pass orders them by digit j, and numbers that tie on digit j keep their previous order, which by induction was sorted by the last $j - 1$ digits. Each pass costs $O(n + b)$ for base b.

For 64-bit integers the template uses base 2^{16}, four passes of 16 bits each, and flips the sign bit first so that negative numbers sort correctly as unsigned values. Table 7.1 shows that it is more than

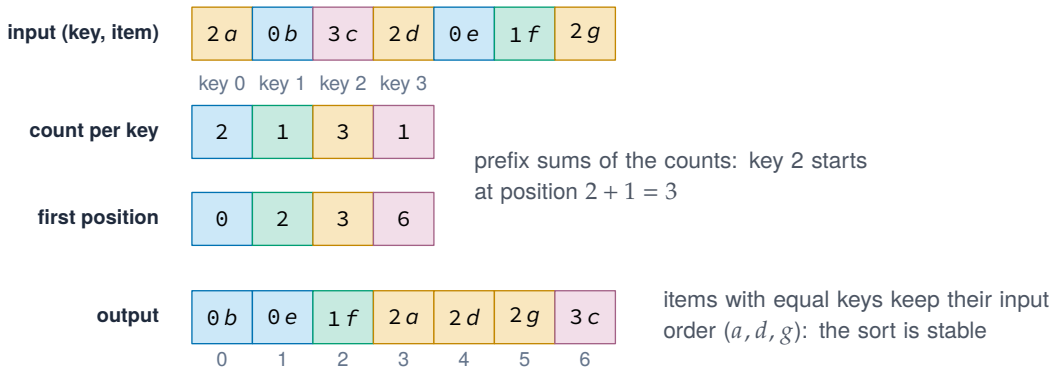


Figure 7.3: Counting sort with keys in $[0, 4)$. Count the keys, turn the counts into first positions with a prefix sum, then place the items in input order, each at the next free position of its key.

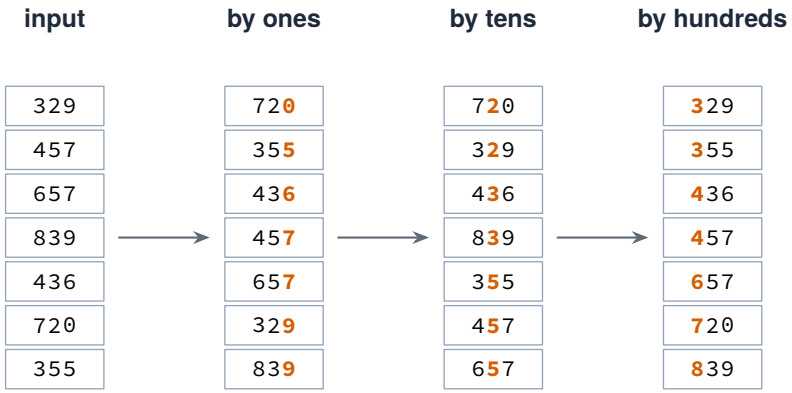


Figure 7.4: Radix sort of three-digit numbers: three stable counting sorts, by the ones digit, the tens digit and the hundreds digit (the digit used in each pass is red). After the pass on the tens digit, the numbers are sorted by their last two digits; after the last pass, completely.

twice as fast as `sort` on large inputs. That rarely decides a contest problem, but it does when n is 10^7, when the same data is sorted many times, or inside a suffix array construction (Chapter 27), which is a radix sort of pairs of ranks.

Routine	$n = 10^6$	$n = 10^7$	Guarantee
`sort`	43 ms	515 ms	$O(n \log n)$
`stable_sort`	51 ms	615 ms	$O(n \log n)$, stable
radix sort (template)	17 ms	238 ms	$O(n)$, 4 passes, stable
`nth_element` (median)	6 ms	48 ms	$O(n)$ on average
`partial_sort`, $k = 100$	< 1 ms	2 ms	$O(n \log k)$

Table 7.1: Measured costs of sorting random 64-bit integers (`g++ -O2`, best of three runs).

</> Template: Radix sort and counting sort Ctrl+Alt+U, Shift+R prefix `cpradix`

7.3 Comparators and exchange arguments

SILVER

When the order you need is not the natural order of the values, you write it as a comparator: a function `cmp(a, b)` that returns `true` when a must come before b. Finding the comparator is the creative part, and the tool for it is the *adjacent exchange argument* of Section 1.5: take an optimal

order, look at two neighbours x and y, and ask when swapping them helps. Because only the two neighbours are affected, the condition usually becomes a simple inequality in the parameters of x and y, and that inequality *is* the comparator.

Example: Weighted completion time

Task i takes t_i time units and has weight w_i (both positive). The tasks run one after another, and the cost is $\sum_i w_i C_i$, where C_i is the time at which task i finishes. Find the minimum cost.

Solution. Consider two adjacent tasks x and y that start at time T (Figure 7.5). Swapping them does not change when any other task finishes, so only their own two terms change. With x first, the two terms are $w_x(T + t_x) + w_y(T + t_x + t_y)$; with y first, $w_y(T + t_y) + w_x(T + t_x + t_y)$. The difference is $w_y t_x - w_x t_y$, so x should come first exactly when $t_x w_y < t_y w_x$, that is, when $t_x/w_x < t_y/w_y$. Sort by this ratio (**Smith's rule**); any order that is not sorted has an adjacent pair that can be swapped without increasing the cost, so the sorted order is optimal.

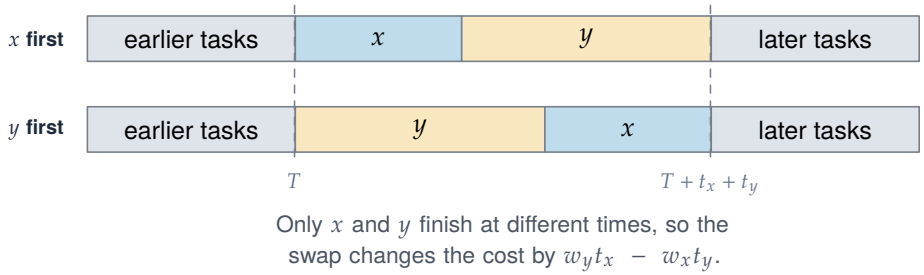


Figure 7.5: The exchange argument behind Smith's rule. Swapping two adjacent tasks changes only their own finishing times; the tasks before and after them are unaffected.

```
void solve() {
    int n;
    cin >> n;
    vector<pll> job(n);  // (t, w): duration and weight, both positive
    for (auto& [t, w] : job) cin >> t >> w;
    // x goes first exactly when t_x * w_y < t_y * w_x (Smith's rule)
    sort(all(job), [](const pll& x, const pll& y) {
        return x.first * y.second < y.first * x.second;
    });
    ll time = 0, cost = 0;
    for (auto [t, w] : job) {
        time += t;  // this job finishes at the current time
        cost += w * time;
    }
    print(cost);
}
```

The comparator cross-multiplies instead of comparing the ratios t/w as doubles: integer arithmetic is exact, and with $t, w \le 10^6$ the products fit comfortably in 64 bits. With all weights equal, Smith's rule becomes *shortest job first*, which minimises the sum (and so the average) of completion times.

7.3.1 When is a comparator legal?

`sort` assumes that the comparator is a **strict weak ordering**: $\text{cmp}(a, a)$ is false; if $\text{cmp}(a, b)$ and $\text{cmp}(b, c)$ then $\text{cmp}(a, c)$; and "neither comes before the other" is transitive too, so that equivalent elements form groups. If any of these fails, the behaviour is undefined, and in practice `sort` can produce an unsorted result, loop forever or read outside the array. An exchange argument only tells you how to order *two* elements; before trusting it as a comparator, check that it is transitive, usually by rewriting it as a comparison of a single key computed from each element, like t/w above.

Example: The largest concatenation

Arrange n non-negative integers in a row so that their concatenation, read as one number, is as large as possible (for $5, 50, 56$ the answer is 56550).

Solution. Adjacent exchange: a should precede b when the string $a + b$ is larger than $b + a$. This is transitive, because $a + b > b + a$ as numbers means $a \cdot 10^{|b|} + b > b \cdot 10^{|a|} + a$, which rearranges to $a/(10^{|a|} - 1) > b/(10^{|b|} - 1)$: a comparison of one key per number. So sorting with it is legal and, by the exchange argument, optimal. If the largest element is 0, all are, and the answer is the single digit 0.

```
void solve() {
    int n;
    cin >> n;
    vector<string> s(n);
    cin >> s;
    // a goes first when the concatenation a + b beats b + a
    sort(all(s), [](const string& a, const string& b) {
        return a + b > b + a;
    });
    if (s[0] == "0") s = {"0"};  // every number is zero
    string out;
    for (auto& x : s) out += x;
    print(out);
}
```

Comparators that break `sort` Never use `<=` in a comparator: $\text{cmp}(a, a)$ becomes true, and GCC's `sort` can then run past the end of the array. Never compare floating-point keys that may be NaN. And be careful with comparators that look symmetric but are not transitive, such as "a before b if a beats b" in a rock-paper-scissors tournament: no order is consistent with them. When in doubt, compile with `-D_GLIBCXX_DEBUG`, which checks the comparator on every call.

7.4 Coordinate compression

SILVER

Many data structures index arrays by value: counting arrays, Fenwick trees, segment trees. When values are huge (up to 10^9) but there are few of them ($2 \cdot 10^5$), **coordinate compression** replaces every value by its *rank* among the distinct values (Figure 7.6). The order of values is preserved, equal values stay equal, and the new values are small enough to index arrays.

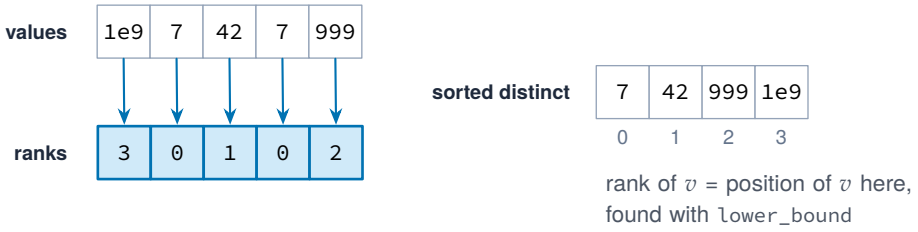


Figure 7.6: Coordinate compression: every value is replaced by its index in the sorted list of distinct values.

In code: copy the values, sort, remove duplicates, and map each value v to `lower_bound(all(vals), v) - vals.begin()`. Building costs $O(n \log n)$ and each lookup $O(\log n)$; if lookups are frequent, store the ranks once in an array.

</> Template: Coordinate compression Ctrl+Alt+T, Shift+C prefix `cpcompress`

> **⚠ Compress everything that will be looked up** Offline problems often have values in the data *and* in the queries. Collect all of them before compressing, or a query value that does not occur in the data has no rank. When intervals are compressed and the gaps between endpoints matter ("is the point between two endpoints covered?"), also insert $v + 1$ for each endpoint v, or keep the original values next to the ranks to measure lengths.

7.5 Greedy algorithms and their proofs SILVER

A **greedy algorithm** builds a solution step by step, always taking the choice that looks best now and never reconsidering it. Greedy algorithms are short and fast, and they are wrong more often than they are right: many natural greedy rules fail on small inputs. So a greedy algorithm needs either a proof or, at the very least, a stress test against a brute force (Chapter 1). Two proof patterns cover almost every contest greedy.

- **Greedy stays ahead.** Show that after each step, the greedy partial solution is at least as good as the corresponding part of *any* solution, by some measure; then it is also at least as good at the end.
- **Exchange argument.** Take an optimal solution that differs from the greedy one, and transform it step by step into the greedy solution, never making it worse (Section 1.5). Then the greedy solution is optimal too.

7.5.1 Interval scheduling

Example: Most disjoint intervals

Given n intervals, choose as many as possible so that no two of them overlap.

Solution. Sort the intervals by their end. Scan them, taking every interval that starts no earlier than the end of the last interval taken (Figure 7.7). This takes $O(n \log n)$; the CSES problem Movie Festival is exactly this (Chapter 31).

Why it works (greedy stays ahead). Let $g_1, g_2, \ldots, g_k$ be the intervals chosen by the greedy algorithm and $o_1, o_2, \ldots, o_m$ those of any valid solution, both in order of time. We claim that g_i ends no later than o_i for every $i \le k$. For $i = 1$ this holds because g_1 has the earliest end of all intervals. If g_i

ends no later than o_i, then o_{i+1}, which starts after o_i ends, also starts after g_i ends, so o_{i+1} was still available when the greedy algorithm chose g_{i+1}, and the greedy algorithm chose the available interval with the earliest end: g_{i+1} ends no later than o_{i+1}. Finally, if $m > k$, the interval o_{k+1} starts after o_k ends, hence after g_k ends, so the greedy algorithm would have taken another interval: a contradiction. So $k \geq m$.

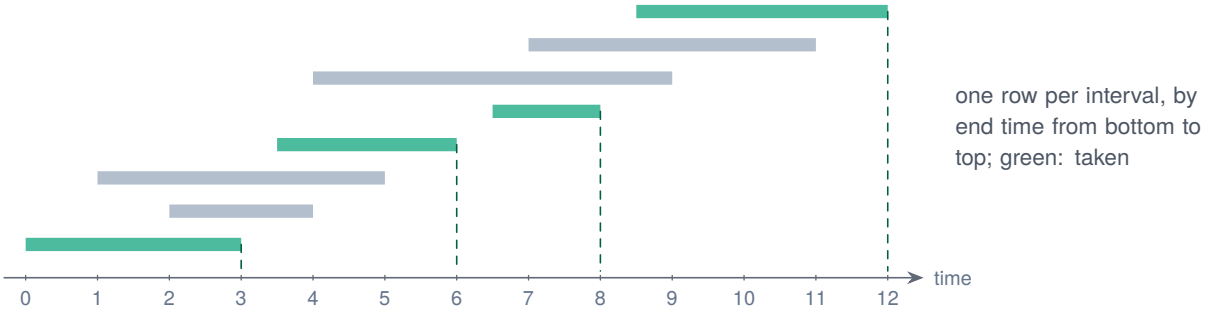


Figure 7.7: Earliest end first. Each interval is on its own row, in order of end time. The algorithm takes an interval whenever it starts at or after the end of the last one taken (dashed lines): four intervals, the maximum.

</> Template: Intervals: merge + max disjoint (greedy) Ctrl+Alt+T, Shift+N prefix `cpintervals`

The same greedy, with the same proof, solves the *stabbing* problem: the fewest points such that every interval contains at least one of them. Sort by end, and whenever an interval does not contain the last point placed, place a new point at its end. The number of points equals the maximum number of disjoint intervals, an instance of a min-max duality that reappears with matchings and flows (Chapter 15).

7.5.2 Breaking a wrong greedy

Plausible greedy rules fail all the time, and the fastest way to discover it is to look for a small counterexample, by hand or with a stress test. Figure 7.8 shows three classics. The lesson is not that greedy is bad, but that the *specific* rule matters, and that a rule is only as good as its proof.



Figure 7.8: Three plausible greedy rules and their counterexamples. Interval scheduling needs the earliest *end*; coin change needs dynamic programming unless the coin system is special (such as $1, 2, 5, 10, \ldots$).

7.6 Classic greedy problems SILVER

A small number of greedy ideas account for a large share of contest problems. This section works through two of them in detail, each with its proof, because the proofs transfer: the slope argument for medians reappears in slope trick (Chapter 20), and the exchange argument behind Huffman's algorithm is the template for every "merge the two smallest" problem. A catalogue of the other classic rules closes the section.

7.6.1 Medians minimise distances

Example: Stick Lengths (CSES)

Given n sticks with lengths $p_1, \ldots, p_n$, make all sticks equally long; changing a length by one costs one. Find the minimum total cost.

Solution. The cost of the target length x is $f(x) = \sum_i |p_i - x|$. Moving x one unit to the right increases the distance to every stick at or left of x and decreases it for every stick right of x, so the slope of f is (number of sticks to the left) − (number to the right). This slope is negative while fewer than half of the sticks are to the left and positive afterwards, so f is minimised at a *median* (Figure 7.9). Any median works, and `nth_element` finds one in linear time.

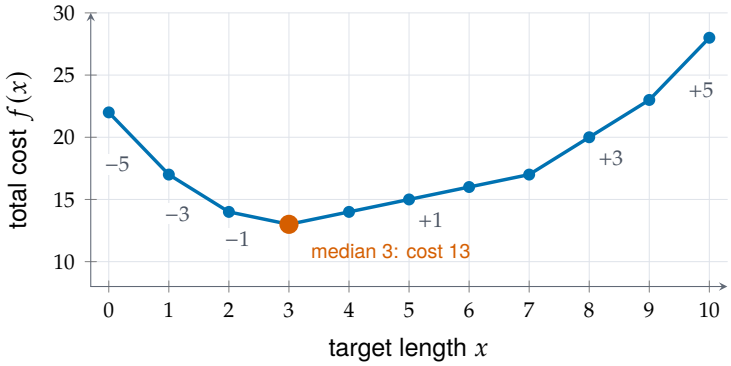


Figure 7.9: The cost $f(x) = \sum |p_i - x|$ for sticks $1, 2, 3, 7, 9$. It is piecewise linear and convex; its slope (numbers on the segments) goes up by 2 at each stick, and changes sign at the median.

```
void solve() {
    int n;
    cin >> n;
    vi p(n);
    cin >> p;
    // a median minimises the sum of distances
    nth_element(p.begin(), p.begin() + n / 2, p.end());
    ll med = p[n / 2], cost = 0;
    for (ll x : p) cost += abs(x - med);
    print(cost);
}
```

The same argument, on each coordinate separately, shows that the point minimising the sum of Manhattan distances to a set of points in the plane is the coordinate-wise median. With squared distances instead, the optimum is the mean.

7.6.2 Merging with the smallest first: Huffman's algorithm

Example: Stick Divisions (CSES)

A stick of length x must be cut into n pieces of given lengths $d_1, \ldots, d_n$ (which sum to x). Cutting a stick of length ℓ into two parts costs ℓ. Find the minimum total cost.

Solution. Run the process backwards: start from the pieces and *join* two sticks at a time, paying the length of the result. The sequence of cuts is a binary tree whose leaves are the pieces and whose internal nodes are the cut sticks, and the total cost is the sum of the internal nodes, which equals $\sum_i d_i \cdot \text{depth}_i$ (Figure 7.10). This is exactly the cost of a prefix code with frequencies d_i, and **Huffman's algorithm** minimises it: repeatedly join the two shortest sticks.

Why joining the two shortest is safe (exchange argument). In an optimal tree, take a deepest internal node; its two children are leaves at the maximum depth. Swapping these leaves with the two shortest pieces cannot increase $\sum d_i \,\text{depth}_i$, since the shortest pieces move deeper and the longer ones move up. So some optimal tree has the two shortest pieces as siblings; joining them first and treating the result as a single piece of length $d_a + d_b$ leaves a smaller instance of the same problem.

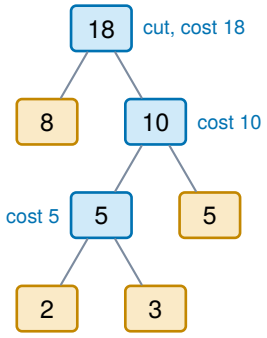


Figure 7.10: Cutting a stick of length 18 into pieces $2, 3, 5, 8$ (orange). Huffman's algorithm joins $2 + 3 = 5$, then $5 + 5 = 10$, then $10 + 8 = 18$; the cuts (blue) cost $5 + 10 + 18 = 33 = 8 \cdot 1 + 5 \cdot 2 + 2 \cdot 3 + 3 \cdot 3$.

```
void solve() {
    int x, n;
    cin >> x >> n;
    minpq<ll> pq;
    rep(i, n) {
        ll d;
        cin >> d;
        pq.push(d);
    }
    ll cost = 0;
    while (sz(pq) > 1) {  // undo the cuts: always join the two shortest
        ll a = pq.top();
        pq.pop();
        ll b = pq.top();
        pq.pop();
        cost += a + b;  // cutting a + b into a and b costs a + b
        pq.push(a + b);
    }
    print(cost);
}
```

7.6.3 A catalogue of classic greedy rules

Table 7.2 lists the greedy rules that every contestant should know by heart, together with the proof technique. Each has been used in countless problems, often in disguise.

Problem	Greedy rule	Proof
most disjoint intervals	earliest end first	stays ahead
fewest points stabbing all intervals	sort by end; a point at the end of each unstabbed interval	stays ahead
fewest rooms for all meetings	by start time; reuse the room that frees earliest (heap)	depth bound
minimise the sum of completion times	shortest job first (Smith's rule with weights)	adjacent exchange
minimise the maximum lateness	earliest deadline first	adjacent exchange
merge piles, cost = size of result	join the two smallest (Huffman)	exchange
equalise values, cost = distance	move everything to a median	slope argument
fractional knapsack	by value per unit of weight	exchange
coin change, canonical systems	largest coin that fits	special structure
unit jobs with deadlines and profits	by deadline; keep the best in a heap	exchange (matroid)

Table 7.2: Classic greedy rules. The 0/1 knapsack and coin change with arbitrary coins are *not* solved by greedy rules; they need dynamic programming (Part V).

7.7 Greedy with heaps and ordered sets

GOLD

Many greedy algorithms process items in a fixed order (by time, by deadline) and at each step choose among a changing set of candidates: the cheapest item so far, the member who became free last. A priority queue or an ordered multiset holds the candidates, and the greedy choice becomes a heap operation or a `lower_bound`.

7.7.1 Deadlines and profits

Example: Scheduling unit jobs

There are n jobs, each taking one day, with deadline d_i and profit p_i; a job earns its profit only if it is finished by its deadline (on day d_i or earlier), and one job can be done per day. Maximise the total profit.

Solution. A set of jobs can be scheduled if and only if, for every d, at most d of them have deadline at most d (then doing them in order of deadline works). Scan the jobs in order of deadline and keep the chosen profits in a min-heap. Add each job; if the heap now holds more jobs than the current deadline allows, remove the smallest profit (Figure 7.11). The removed job is the one that is least useful, and every job still in the heap can be scheduled.

```
void solve() {
    int n;
    cin >> n;
    vector<pll> job(n);  // (deadline, profit); each job takes one day
    for (auto& [d, p] : job) cin >> d >> p;
    sort(all(job));
    minpq<ll> kept;  // profits of the jobs chosen so far
    for (auto [d, p] : job) {
```

Job (d, p)	Limit d	Heap after adding	Removed	Chosen profits	
$(1,5)$	1	$\{5\}$	–	5	
$(2,3)$	2	$\{3,5\}$	–	$3,5$	
$(2,8)$	2	$\{3,5,8\}$	3	$5,8$	
$(3,1)$	3	$\{1,5,8\}$	–	$1,5,8$	
$(3,6)$	3	$\{1,5,6,8\}$	1	$5,6,8$	
$(3,4)$	3	$\{4,5,6,8\}$	4	$5,6,8$	(total 19)

Figure 7.11: Scheduling unit jobs with a min-heap. Whenever the heap holds more jobs than days available, the cheapest job is dropped. The final jobs are done on days 1, 2, 3 in order of deadline: profits 5, 8, 6.

```
        kept.push(p);
        if (sz(kept) > d) kept.pop();  // only d jobs fit by day d
    }
    ll total = 0;
    for (; !kept.empty(); kept.pop()) total += kept.top();
    print(total);
}
```

This problem is the standard example of a *matroid*: the sets of jobs that can be scheduled satisfy the exchange property (a smaller schedulable set can always be extended by some job of a larger one), and on every matroid, taking elements in order of decreasing weight whenever they keep the set independent is optimal. Kruskal's algorithm for minimum spanning trees (Chapter 13) is the other famous instance.

7.7.2 Best fit with an ordered multiset

Example: Movie Festival II (CSES)

A club of k members wants to watch as many of n movies as possible (counting each watched movie once); each member can watch any number of movies, one at a time, and a movie that ends at time t does not overlap one that starts at time t.

Solution. Process the movies by end time, as for a single person. For each movie, some member must be free at its start time a; among the members who are, give the movie to the one who *became free last* (the largest free time $\le a$). This *best fit* leaves the members with earlier free times available for movies that start earlier, and an exchange argument shows that it never does worse than any other choice. An ordered multiset of free times finds that member with `upper_bound` in $O(\log k)$.

```
void solve() {
    int n, k;
    cin >> n >> k;
    vector<pll> mv(n);  // (end, start), so that sorting orders by end
    for (auto& [b, a] : mv) cin >> a >> b;
    sort(all(mv));
    multiset<ll> freeAt;  // when each club member becomes free
    rep(i, k) freeAt.insert(0);
```

```
    int watched = 0;
    for (auto [b, a] : mv) {
        auto it = freeAt.upper_bound(a);  // first member free after a
        if (it == freeAt.begin()) continue;  // nobody is free at time a
        freeAt.erase(prev(it));  // the member who became free last
        freeAt.insert(b);
        watched++;
    }
    print(watched);
}
```

7.7.3 Greedy that can change its mind

Some problems defeat every fixed greedy rule because an early choice only turns out to be wrong much later. **Regret greedy** (also called greedy with undo) keeps that later option open: whenever it makes a choice, it also pushes a special candidate into the heap that represents *undoing* the choice, priced so that taking it later has exactly the effect of having chosen differently.

Example: Buy Low Sell High (Codeforces 865D)

The price of a share is known for each of n days. Each day you may buy one share, sell one share, or do nothing; you start with no shares. Maximise the profit.

Solution. Keep a min-heap of prices. On each day with price p, if the cheapest price h in the heap is below p, sell a share bought at h for a profit of $p - h$, remove h, and push p *twice*. One copy is today as an ordinary buying opportunity. The other copy is the regret option: if a later day has price $q > p$, "buying at p and selling at q" earns $q - p$, and added to the $p - h$ already earned it gives $q - h$, exactly the profit of selling the share bought at h on the later day instead. If $h \geq p$, just push p (Figure 7.12).

Price	Action	Heap afterwards	Profit
3	heap empty: push 3	$\{3\}$	0
1	$3 \geq 1$: push 1	$\{1,3\}$	0
4	sell the share bought at 1 (+3); push 4 twice	$\{3,4,4\}$	3
5	sell the share bought at 3 (+2); push 5 twice	$\{4,4,5,5\}$	5
2	$4 \geq 2$: push 2	$\{2,4,4,5,5\}$	5
6	sell the share bought at 2 (+4); push 6 twice	$\{4,4,5,5,6,6\}$	9

Figure 7.12: Regret greedy on the prices $3, 1, 4, 5, 2, 6$. The optimal plan buys on days 1, 2 and 5 and sells on days 3, 4 and 6, for a profit of 9. The duplicated prices are the options to move a sale to a later, better day.

```
void solve() {
    int n;
    cin >> n;
    minpq<ll> pq;  // prices at which we can still buy (or re-sell)
    ll profit = 0;
    rep(i, n) {
        ll p;
        cin >> p;
```

```
        if (!pq.empty() && pq.top() < p) {
            profit += p - pq.top();  // sell today what was cheapest
            pq.pop();
            pq.push(p);  // allows this sale to be moved to a later day
        }
        pq.push(p);  // today as an ordinary buying opportunity
    }
    print(profit);
}
```

Regret greedy is a lightweight form of the augmenting paths of min-cost flow (Chapter 15): the "undo" candidates are the reverse edges of the residual graph. When a problem looks like a flow problem but n is 10^5 or more, try to find the greedy with undo that simulates the flow.

How to recognize it Greedy is likely when the statement asks to "maximise the number of" or "minimise the maximum", the limits are large ($n = 2 \cdot 10^5$) with no obvious DP state, and brute-force answers on small inputs always agree with some simple rule. The first step is almost always to sort by the right key; find the key with an adjacent exchange argument. Greedy is unlikely when choices interact in complicated ways (then think of dynamic programming) or when the problem is a known hard problem such as 0/1 knapsack or set cover, unless the limits reveal special structure.

Chapter summary

- After sorting, equal values are adjacent, order statistics are positions, and closest values are neighbours; `nth_element` finds a median in linear time and `partial_sort` the k smallest.
- Counting sort and radix sort sort integers in linear time and are stable; the template sorts 64-bit integers more than twice as fast as `sort`.
- Find comparators with adjacent exchange arguments, and make sure they are strict weak orderings, ideally by comparing one key per element.
- Coordinate compression maps huge values to ranks; compress the query values too.
- Prove greedy algorithms by "greedy stays ahead" or by exchange arguments, and break wrong ones with small counterexamples.
- Heaps and ordered multisets implement greedy choices among changing candidates; regret greedy pushes undo options into the heap.

Practice problems (CSES Problem Set)

SORTING

- Distinct Numbers
- Apartments
- Ferris Wheel
- Concert Tickets
- Missing Coin Sum
- Collecting Numbers

GREEDY

- Movie Festival
- Stick Lengths
- Tasks and Deadlines
- Reading Books
- Towers
- Stick Divisions

GREEDY WITH DATA STRUCTURES

- Room Allocation
- Movie Festival II
- Factory Machines

CHAPTER 8

Binary Search and Unimodal Optimisation

Binary search finds a boundary in a monotone world in logarithmically many steps. Everyone knows it as the way to look up a value in a sorted array, but its real power in contests lies elsewhere: in *binary search on the answer*, which turns "find the best value" into the far easier question "is this value achievable?", and in its relatives for real numbers, unimodal functions, ratios and batches of queries. This chapter develops binary search from a single loop invariant, so that the off-by-one errors that plague it disappear, and then applies it in all of these forms.

IN THIS CHAPTER BRONZE SILVER GOLD IOI / ADVANCED

- write binary search from an invariant, and use `lower_bound` and `upper_bound` correctly;
- recognise monotone predicates and binary search on the answer;
- find the k-th smallest element of a set that is never built;
- binary search over real numbers, and maximise ratios with fractional programming;
- optimise unimodal and convex functions with ternary search;
- answer many binary-search queries at once with parallel binary search.

Before you start: Chapters 3 and 7.

8.1 Searching a sorted array BRONZE

To find where a value x belongs in a sorted array, compare it with the middle element and discard the half that cannot contain the answer. Each comparison halves the range, so after $\lceil \log_2(n+1) \rceil$ comparisons nothing is left: 20 comparisons suffice for 10^6 elements and 60 for 10^{18}.

The cleanest way to write it, and to never get it wrong, is to decide exactly what the answer is and to maintain an *invariant*. We look for the first index i with $a_i \geq x$ (or n if there is none), and keep a half-open range $[lo, hi)$ with the invariant

$$a_i < x \text{ for all } i < lo, \qquad a_i \geq x \text{ for all } i \geq hi.$$

Initially $lo = 0$ and $hi = n$, and the invariant holds vacuously. Look at $mid = \lfloor (lo + hi)/2 \rfloor$, which lies in $[lo, hi)$: if $a_{mid} < x$, the invariant still holds with $lo = mid + 1$; otherwise it holds with $hi = mid$. Either way the range shrinks, and when $lo = hi$ the invariant says that lo is exactly the first index with $a_i \geq x$ (Figure 8.1).

8.1.1 The standard library

The library functions implement exactly this search, and it is worth knowing precisely what each returns (Figure 8.2):

- `lower_bound(all(a), x)`: the first position with value $\geq x$, the insertion point of x before any equal values;

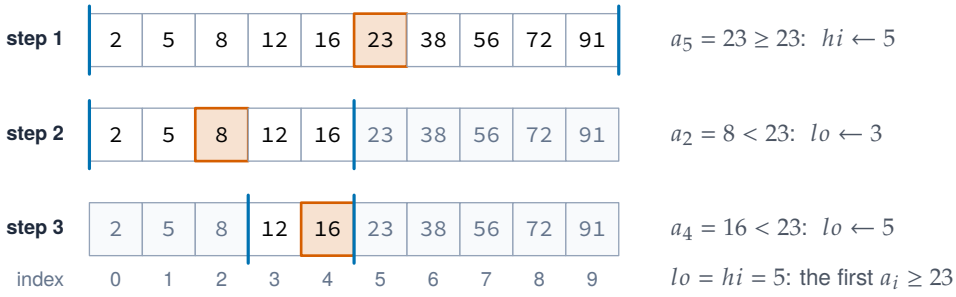


Figure 8.1: Binary search for the first element ≥ 23. The blue bars delimit the range $[lo, hi)$ that may still contain the boundary; grey cells are already decided, and red marks the middle element compared in each step.

- `upper_bound(all(a), x)`: the first position with value $> x$, the insertion point after equal values;
- `equal_range(all(a), x)`: both at once; the difference of the two positions is the number of copies of x;
- the last value $\leq x$ (the *predecessor*) is just before `upper_bound`, if that is not the beginning, and the first value $\geq x$ (the *successor*) is at `lower_bound`, if that is not the end.

All of them return iterators; subtract `a.begin()` to get an index. A reversed array is searched with `greater<>()` as the comparator, and `binary_search` only answers yes or no, which is rarely enough.

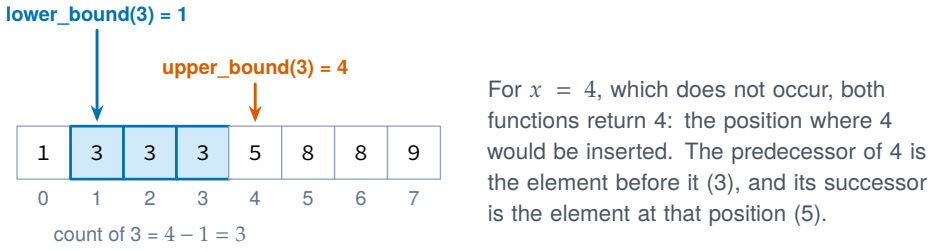


Figure 8.2: `lower_bound` and `upper_bound` on a sorted array with repeated values. The copies of x occupy exactly the positions from `lower_bound` up to (not including) `upper_bound`.

Example: Counting values in ranges

Given n numbers and q queries $[l, r]$, report how many of the numbers lie in each range. Limits: $n, q \leq 2 \cdot 10^5$.

Solution. Sort once. The values in $[l, r]$ occupy the positions from the first value $\geq l$ up to (not including) the first value $> r$, so each query is two binary searches: $O((n + q) \log n)$ in total.

```
void solve() {
    int n, q;
    cin >> n >> q;
    vi a(n);
    cin >> a;
    sort(all(a));
    while (q--) {
        ll l, r;
        cin >> l >> r;
        // [first value >= l, first value > r) holds the values in [l, r]
        auto from = lower_bound(all(a), l), to = upper_bound(all(a), r);
```

```
        print(to - from);
    }
}
```

Searching a `set` with the free function `lower_bound(all(s), x)` compiles for a `set` or `map`, but set iterators cannot jump to the middle, so the free function walks through the elements one by one and takes *linear* time. Always use the member function `s.lower_bound(x)`, which descends the tree in $O(\log n)$. This single mistake turns many correct solutions into time limit verdicts.

8.2 Binary search on a monotone predicate

SILVER

Nothing in the argument above used the array itself, only the fact that the condition $a_i \geq x$ is false for a prefix of the indices and true for the rest. So binary search works for any **monotone predicate**: a yes/no function $ok(v)$ on integers such that once it is true, it stays true for all larger v (Figure 8.3). The search finds the boundary, the first v with $ok(v)$ true, with $O(\log(\text{range}))$ evaluations of ok, and the predicate can be anything you can compute: a greedy simulation, a flow, a DP.

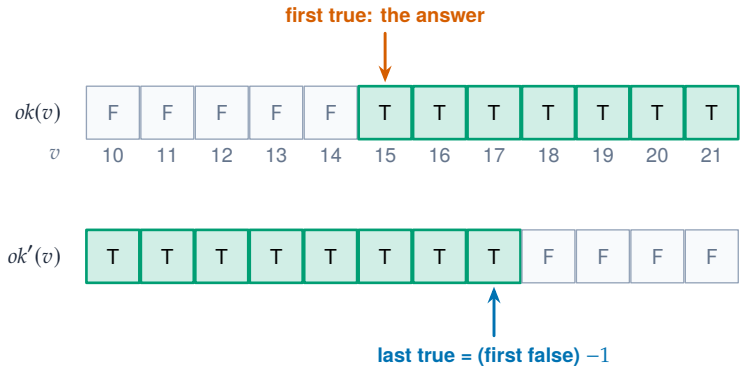


Figure 8.3: Monotone predicates. Top: false up to some point, true afterwards; binary search finds the first true value. Bottom: the "maximise" form, true up to some point; its last true value is one less than its first false value.

Algorithm: First true of a monotone predicate

Require: ok monotone on $[lo, hi]$, and $ok(hi)$ true (or hi means "no answer")

```
while lo < hi do                          ▷ invariant: the answer lies in [lo, hi]
    mid ← lo + ⌊(hi − lo)/2⌋              ▷ lo ≤ mid < hi
    if ok(mid) then
        hi ← mid                          ▷ mid may be the answer
    else
        lo ← mid + 1                      ▷ everything up to mid is false
return lo
```

Three details make this loop robust. First, mid is computed as $lo + (hi - lo)/2$, never as $(lo + hi)/2$, which overflows when both are close to 2^{63}. Second, because $mid < hi$, both branches strictly shrink the range, so the loop terminates; the tempting variant `lo = mid` (without +1) can leave lo unchanged when $hi = lo + 1$ and loop forever. Third, the initial hi must be a value where ok is certainly true, or a sentinel one past the range that you check for after the loop; then the answer is always inside $[lo, hi]$.

For the "maximise" form (the last v with $ok(v)$ true, as in Figure 8.3, bottom), either search for the first false value and subtract one, or mirror the loop: round mid *up*, $mid = lo + \lceil (hi - lo)/2 \rceil$, and set $lo \leftarrow mid$ on true and $hi \leftarrow mid - 1$ on false. The next section uses the second form.

</> Template: Binary search on the answer Ctrl+Alt+T, A prefix `cpbsearch`

Binary search by jumps An equivalent formulation, often shorter and immune to off-by-one errors, builds the answer bit by bit: start at a position where ok is false and take jumps of decreasing powers of two, moving whenever the target of the jump is still false.

```
ll x = lo - 1;  // ok(lo - 1) is false (or lo - 1 is outside the domain)
for (ll step = 1LL << 62; step > 0; step >>= 1)
    if (x + step < hi && !ok(x + step)) x += step;
// x is the last false value, so x + 1 is the first true one
```

The same pattern descends a Fenwick tree to find a prefix sum in $O(\log n)$ (Chapter 22) and climbs a tree with binary lifting (Chapter 16).

Galloping when the range is unknown If the answer can be arbitrarily large and ok gets expensive for large values, find an upper bound first by doubling: try $1, 2, 4, 8, \ldots$ until ok becomes true, then binary search between the last two tries. This *exponential search* costs $O(\log v)$ evaluations, where v is the answer itself rather than the largest possible answer, and it never evaluates ok far beyond v, which also avoids overflow inside ok.

8.2.1 Beyond monotonicity: keep a half that surely contains an answer

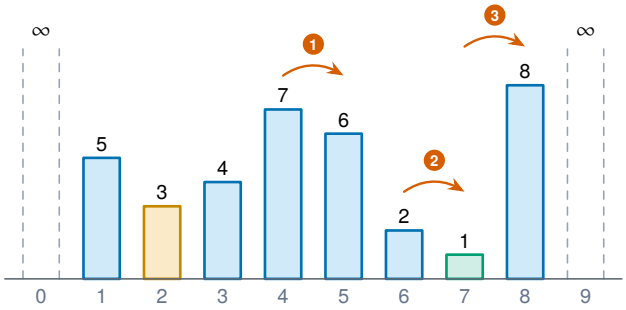
Binary search needs less than a monotone predicate. All it needs is that, after each question, *one half is guaranteed to contain some answer*. Bisection for roots is the classic example: if a continuous function has $f(lo) < 0 < f(hi)$, then whichever half has a sign change at its ends contains a root, even though f need not be monotone. The discrete analogue is just as useful.

Example: Searching Local Minimum (Codeforces 1479A)

An unknown permutation $a_1, \ldots, a_n$ ($n \le 10^5$) can be read one element per query. Find a *local minimum*, an index i with $a_{i-1} > a_i < a_{i+1}$ (where $a_0 = a_{n+1} = \infty$), using at most 100 queries.

Solution. Keep a range $[lo, hi]$ with $a_{lo-1} > a_{lo}$ and $a_{hi} < a_{hi+1}$: walking downhill from lo into the range can never leave it, so it contains a local minimum. Initially $[1, n]$ qualifies. Read a_{mid} and a_{mid+1}: if $a_{mid} < a_{mid+1}$, the range $[lo, mid]$ still qualifies; otherwise $[mid + 1, hi]$ does. When $lo = hi$, both conditions together say that lo is a local minimum (Figure 8.4). This takes $2\lceil \log_2 n \rceil \le 34$ queries.

```
map<int, int> memo;
auto ask = [&](int i) -> int {  // a[i], with a[0] = a[n + 1] = infinity
    if (i < 1 || i > n) return 1e9;
    if (!memo.count(i)) {
        cout << "? " << i << endl;  // endl flushes: the judge waits for it
        cin >> memo[i];
    }
```



Step 1: $7 > 6$, go right to $[5, 8]$.
Step 2: $2 > 1$, go right to $[7, 8]$.
Step 3: $1 < 8$, keep $[7, 7]$: index 7 (green) is a local minimum. Index 2 (orange) is another one, in the half that was dropped.

Figure 8.4: Finding a local minimum by binary search. Each step compares a_{mid} with a_{mid+1} and keeps the half on the downhill side; that half is guaranteed to contain a local minimum, although not necessarily the global one.

```
    return memo[i];
};
int lo = 1, hi = n;  // a local minimum lies in [lo, hi]
while (lo < hi) {
    int mid = (lo + hi) / 2;
    if (ask(mid) < ask(mid + 1)) hi = mid;
    else lo = mid + 1;
}
cout << "! " << lo << endl;
```

Interactive problems In an interactive problem the judge answers your queries while your program runs, so every query must be *flushed* (`endl` or `cout.flush()`), and the query limit is part of the problem: $\lceil \log_2 n \rceil$ queries allow a plain binary search, $2\lceil \log_2 n \rceil$ allow comparing neighbours, and a limit like $n/2$ hints at pairing. Cache answers so that no position is asked twice, and test locally with a small program that plays the judge. The template `cpinteractive` (Ctrl+Alt+T, Shift+I) has the query helpers.

8.3 Binary search on the answer

SILVER

Many optimisation problems ask for the smallest (or largest) value with some property, where *checking* whether a given value works is much easier than *finding* the best value directly. If the property is monotone, binary search on the answer reduces the optimisation problem to $O(\log(\text{range}))$ checks. This is the idea behind Array Division (Section 1.2) and Factory Machines (Chapter 31); here is the classic "maximise the minimum" version.

Example: Aggressive cows

There are n stalls at given positions on a line. Place $k \geq 2$ cows in distinct stalls so that the minimum distance between two cows is as large as possible.

Solution. Ask instead: *can the cows be placed so that every two are at least d apart?* This is monotone (if a placement works for d, the same placement works for every smaller d), and it is easy to check greedily: sort the stalls, put a cow in the first stall, and put each next cow in the first stall at least d beyond the previous cow. Placing each cow as early as possible never hurts, by a "greedy stays ahead" argument like the one for interval scheduling (Section 7.5). The answer is the largest d for which the check succeeds (Figure 8.5), found in $O(n \log n + n \log(\text{range}))$.



Figure 8.5: The greedy check for stalls $1, 2, 4, 8, 9$ and $k = 3$ cows. With $d = 3$ the cows go to 1, 4 and 8; with $d = 4$ only two fit (1 and 8). The answer is 3, the largest d that passes.

```
void solve() {
    int n, k;
    cin >> n >> k;
    vi x(n);
    cin >> x;
    sort(all(x));
    // can k cows be placed with every two at least d apart? (greedy)
    auto ok = [&](ll d) {
        int cows = 1;
        ll last = x[0];
        rep(i, 1, n) if (x[i] - last >= d) cows++, last = x[i];
        return cows >= k;
    };
    ll lo = 0, hi = x[n - 1] - x[0];  // ok(lo) holds; find the last d with ok
    while (lo < hi) {
        ll mid = lo + (hi - lo + 1) / 2;  // round up, or lo = mid never moves
        if (ok(mid)) lo = mid;
        else hi = mid - 1;
    }
    print(lo);
}
```

How to recognize it "Minimise the maximum", "maximise the minimum", "the smallest time, size or capacity such that. . . ", or any answer where checking a candidate is easier than constructing the best one. The check is often a greedy simulation, and the monotonicity argument is usually one sentence: "a solution for the stricter requirement is also a solution for the looser one".

8.3.1 The k-th smallest element of an implicit set

A second family of problems asks for the k-th smallest element of a set that is far too large to build: all pairwise sums, all products in a table, all subarray sums. Binary search the *value*: the k-th smallest is the smallest x such that at least k elements are $\le x$. Counting the elements $\le x$ is usually much easier than listing them.

Example: Multiplication Table (CSES)

Find the median of the $n \times n$ multiplication table (the entries $i \cdot j$ for $1 \le i, j \le n$), where $n \le 10^6$ is odd.

Solution. The median is the k-th smallest entry for $k = (n^2 + 1)/2$. Row i contains the multiples $i, 2i, \ldots, ni$, and exactly $\min(n, \lfloor x/i \rfloor)$ of them are at most x, so counting the entries $\le x$ takes

$O(n)$ (Figure 8.6). Binary search the smallest x whose count reaches k: $O(n \log n^2)$ in total, 0.07 s for $n = 10^6$. The answer is always an entry of the table, because the count only increases at entries.

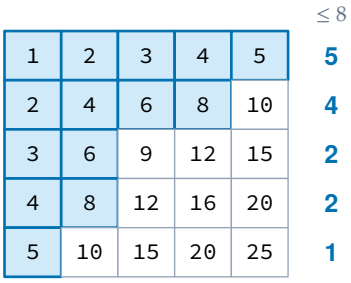


Figure 8.6: The 5×5 multiplication table. The entries ≤ 8 (blue) form a staircase; row i contributes $\min(n, \lfloor 8/i \rfloor)$ of them (right).

```
void solve() {
    ll n;
    cin >> n;
    ll k = (n * n + 1) / 2;  // the median is the k-th smallest entry
    // how many entries i * j (1 <= i, j <= n) are at most x?
    auto count = [&](ll x) {
        ll c = 0;
        for (ll i = 1; i <= n; i++) c += min(n, x / i);
        return c;
    };
    ll lo = 1, hi = n * n;  // the first x with count(x) >= k
    while (lo < hi) {
        ll mid = lo + (hi - lo) / 2;
        if (count(mid) >= k) hi = mid;
        else lo = mid + 1;
    }
    print(lo);
}
```

The same pattern finds the k-th smallest pairwise sum (count pairs with sum $\leq x$ by two pointers), the k-th smallest subarray sum of a positive array (two pointers again), and the k-th smallest distance between points on a line. When the count itself needs a data structure, the whole search is sometimes replaced by a walk down that structure, as in the order-statistics tree of Chapter 23.

8.4 Real numbers and fractional programming

GOLD

When the answer is a real number, the loop cannot run until $lo = hi$. The robust approach is to run a *fixed number of iterations*: each halves the interval, so 100 iterations shrink any interval of `double`s far below their precision. Comparing `hi - lo` with an epsilon is fragile instead: with values around 10^9 and `eps = 1e-9` the loop never ends, because consecutive doubles near 10^9 are about 10^{-7} apart and the interval cannot shrink further.

</> Template: Binary search on a real answer Ctrl+Alt+T, Shift+F prefix `cpbsreal`

8.4.1 Maximising a ratio

A particularly important use of real-valued binary search is **fractional programming**: maximising a ratio $\sum a_i / \sum b_i$ (with all $b_i > 0$) over some family of choices, such as subsets, paths, cycles or subarrays. Ratios do not decompose, so DP and greedy algorithms cannot optimise them directly. But for any λ,

$$\frac{\sum a_i}{\sum b_i} \geq \lambda \iff \sum (a_i - \lambda b_i) \geq 0,$$

so "can the ratio reach λ?" is the question "is the maximum of $\sum(a_i - \lambda b_i)$ non-negative?", an ordinary optimisation problem with weights $a_i - \lambda b_i$. The answer is monotone in λ, so binary search on λ finds the best ratio.

Example: Maximum average subarray

Given $n \leq 10^5$ numbers and k, find the maximum average of a subarray of length at least k.

Solution. The average is the ratio $\sum a_i / \sum 1$. For a candidate λ, subtract λ from every element and ask whether some subarray of length $\geq k$ has a non-negative sum. With prefix sums p of the shifted values, that is whether $p_r - \min_{l \leq r-k} p_l \geq 0$ for some r, one linear scan. A hundred iterations of binary search then take $O(100\, n)$.

```
void solve() {
    int n, k;
    cin >> n >> k;
    vector<double> a(n);
    cin >> a;
    // is there a subarray of length >= k with average >= lam? Exactly when
    // some such subarray has sum of (a[i] - lam) >= 0
    auto ok = [&](double lam) {
        vector<double> p(n + 1, 0);
        rep(i, n) p[i + 1] = p[i] + a[i] - lam;
        double lowest = 1e18;  // the smallest p[l] with l <= r - k
        rep(r, k, n + 1) {
            chmin(lowest, p[r - k]);
            if (p[r] - lowest >= 0) return true;
        }
        return false;
    };
    double lo = *min_element(all(a)), hi = *max_element(all(a));
    rep(it, 100) {
        double mid = (lo + hi) / 2;
        if (ok(mid)) lo = mid;
        else hi = mid;
    }
    cout << fixed << setprecision(6) << lo << '\n';
}
```

Dinkelbach's method. Write $g(\lambda) = \max \sum(a_i - \lambda b_i)$ over all choices. Each choice contributes the line $A - \lambda B$ (with $A = \sum a_i$, $B = \sum b_i$), so g is the upper envelope of lines with negative slopes: decreasing and convex, with its root exactly at the best ratio λ^* (Figure 8.7). Instead of bisecting, **Dinkelbach's method** jumps: solve the inner problem at the current λ, take the optimal choice's own ratio A/B as the next λ, and repeat until $g(\lambda) = 0$. This is Newton's method on g, and it

typically converges in a handful of iterations, and exactly (with no precision issues) when the inner problem can be solved with fractions or integers.

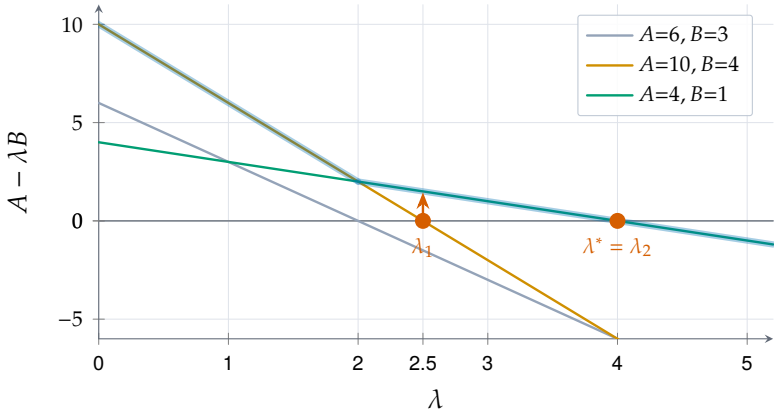


Figure 8.7: Fractional programming with three choices. The best ratio $\lambda^* = 4$ is the root of the upper envelope g (thick blue). Dinkelbach's method starts at $\lambda_0 = 0$, where the best line is $10 - 4\lambda$; its root gives $\lambda_1 = 10/4$. At λ_1 the best line is $4 - \lambda$, whose root $\lambda_2 = 4$ is the answer.

Fractional programming appears in disguise surprisingly often: the minimum mean cycle (weights $w_e - \lambda$, check for a negative cycle), the optimal ratio spanning tree (MST with weights $a_e - \lambda b_e$), the densest subgraph (maximum closure with weights shifted by λ), and "best average" problems of every kind.

8.5 Ternary search on unimodal functions

GOLD

A function is **unimodal** if it strictly increases up to its maximum and strictly decreases afterwards (or the reverse, for a minimum). To find the peak, evaluate f at two interior points $m_1 < m_2$. If $f(m_1) < f(m_2)$, the peak cannot be to the left of m_1: there the function is still increasing, and a peak to the left would make f decrease from m_1 to m_2. Otherwise the peak cannot be to the right of m_2. Either way a third of the interval is discarded (Figure 8.8), and after k steps the interval has shrunk by $(2/3)^k$: 100 steps are plenty for doubles.

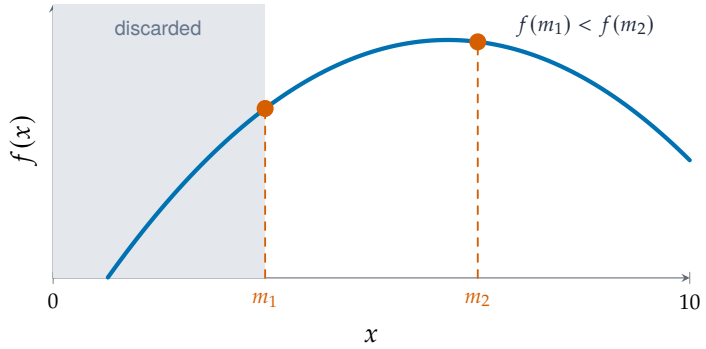


Figure 8.8: One step of ternary search for a maximum. Since $f(m_1) < f(m_2)$, the maximum cannot lie in $[0, m_1]$ (grey), which is discarded.

Integers: binary search on the slope. For a function on integers, ternary search needs care near the end. A cleaner method uses the fact that a unimodal function's *difference* $f(x+1) - f(x)$ changes sign once: binary search the first x with $f(x) \geq f(x+1)$ (for a maximum). This is an ordinary monotone predicate and costs $2\log(\text{range})$ evaluations. The template contains both versions.

</> Template: Ternary search (unimodal max) Ctrl+Alt+T, Q prefix `cpternary`

Convexity is the real condition. In practice the functions you ternary search are usually *convex*: maxima of linear functions, sums of absolute values, sums of convex costs. A convex function is unimodal (or constant on its minimum), and sums and maxima of convex functions are convex, so convexity is easy to establish from the structure of the cost even when the function has no closed form.

Example: Weakness and Poorness (Codeforces 578C)

Given $n \le 2 \cdot 10^5$ integers a_i with $|a_i| \le 10^4$, find a real x minimising the *weakness* of the sequence $a_1 - x, \dots, a_n - x$: the largest absolute value of the sum of a segment.

Solution. For a fixed segment, its sum $S - Lx$ is linear in x, so its absolute value is convex; the weakness is the maximum of these convex functions, hence convex in x. Ternary search on x; for a given x, the largest segment sum and the smallest segment sum come from two runs of Kadane's algorithm (Chapter 31), and the weakness is the larger of the maximum and minus the minimum.

```
void solve() {
    int n;
    cin >> n;
    vector<double> a(n);
    cin >> a;
    // the largest |sum| of a segment of a[i] - x: the larger of the best
    // sum and minus the worst sum (Kadane); a convex function of x
    auto weakness = [&](double x) {
        double best = 0, worst = 0, hi = 0, lo = 0;
        for (double v : a) {
            hi = max(0.0, hi) + v - x;
            lo = min(0.0, lo) + v - x;
            best = max(best, hi);
            worst = min(worst, lo);
        }
        return max(best, -worst);
    };
    double l = -1e4, r = 1e4;
    rep(it, 100) {  // ternary search: each step keeps 2/3 of the range
        double m1 = l + (r - l) / 3, m2 = r - (r - l) / 3;
        if (weakness(m1) < weakness(m2)) r = m2;
        else l = m1;
    }
    cout << fixed << setprecision(9) << weakness((l + r) / 2) << '\n';
}
```

Plateaus break ternary search If f is constant on some interval other than its optimum, $f(m_1) = f(m_2)$ says nothing about where the optimum is, and ternary search may discard it. Ternary search is safe for strictly unimodal functions and for convex functions (whose only possible plateau is the set of minima). For functions of two variables, nested ternary search works when the function is convex in both jointly, but is only a heuristic otherwise.

Golden-section search Ternary search evaluates f twice per step. If the two interior points divide the interval in the golden ratio, $m_1 = r - (r-l)/\varphi$ and $m_2 = l + (r-l)/\varphi$ with $\varphi = (1+\sqrt{5})/2$, then after discarding one side, the surviving interior point is exactly one of the two points needed in the next step. So each step costs *one* evaluation and shrinks the interval by $1/\varphi \approx 0.618$: about 1.4 times fewer evaluations than ternary search for the same precision, which matters when each evaluation is a full $O(n)$ pass.

8.6 Parallel binary search

IOI / ADVANCED

Suppose there are q queries, each of which could be answered by its own binary search over the same sequence of m events: "after how many events does query i become true?". Each check replays events from the beginning, so separate searches cost $O(q \cdot m \log m)$ event operations. **Parallel binary search** runs all q searches together, in rounds. In each round, every unfinished query has a current range and a middle; sort the queries by their middles, replay the events *once* from the start, and check each query at the moment the replay reaches its middle. After $\lceil \log_2(m+2) \rceil$ rounds every range has shrunk to one value, for a total of $O((m+q)\log m)$ event operations plus sorting.

Example: First moment of connectivity

m edges are added one by one to a graph with n vertices. For each of q queries (a, b), find the first time after which a and b are connected (0 if $a = b$, -1 if never). Limits: $n, m, q \le 2 \cdot 10^5$.

Solution. Connectivity only grows as edges are added, so each query's answer is a monotone boundary. In each round, sort the queries by their middles, rebuild a DSU from scratch and add the edges in order, checking each query when its middle is reached (Figure 8.9). Eighteen rounds of m unions and q checks answer everything in 0.4 s.

Query (answer)	Round 1	Round 2	Round 3	Round 4
q_1 (3)	$[0,8]$, mid 4: yes	$[0,4]$, mid 2: no	$[3,4]$, mid 3: yes	$[3,3]$ done
q_2 (6)	$[0,8]$, mid 4: no	$[5,8]$, mid 6: yes	$[5,6]$, mid 5: no	$[6,6]$ done
q_3 (1)	$[0,8]$, mid 4: yes	$[0,4]$, mid 2: yes	$[0,2]$, mid 1: yes	$[0,1]$, mid 0: no
q_4 (never)	$[0,8]$, mid 4: no	$[5,8]$, mid 6: no	$[7,8]$, mid 7: no	$[8,8]$ done
Replays	edges 1–4	edges 1–6	edges 1–7	none

Figure 8.9: Parallel binary search with $m = 7$ events, so every range starts as $[0,8]$ (8 means "never"). In each round the queries are checked in order of their middles during a single replay of the events; "yes" means connected after *mid* events. After round 4 every range is a single value; for q_3 it is $[1,1]$.

```
void solve() {
    int n, m, q;
    cin >> n >> m >> q;
    vector<pii> e(m), qs(q);
    for (auto& [u, v] : e) cin >> u >> v;
    for (auto& [a, b] : qs) cin >> a >> b;
    vi lo(q, 0), hi(q, m + 1);  // answer in [lo, hi]; m + 1 means never
    while (true) {
        vector<pii> byMid;  // (mid, query) for the unfinished queries
        rep(i, q) if (lo[i] < hi[i]) byMid.push_back({(lo[i] + hi[i]) / 2, i});
```

```
        if (byMid.empty()) break;
        sort(all(byMid));
        DSU d(n + 1);  // one replay of the edges serves every query
        int added = 0;
        for (auto [mid, i] : byMid) {
            while (added < mid) {  // add edges 1..mid
                d.unite(e[added].first, e[added].second);
                added++;
            }
            if (d.same(qs[i].first, qs[i].second)) hi[i] = mid;
            else lo[i] = mid + 1;
        }
    }
    rep(i, q) print(lo[i] > m ? -1 : lo[i]);
}
```

The technique needs only two things: each query's answer is monotone in the number of events, and the events can be replayed from scratch cheaply. It shines when the events are hard to *undo* (DSU unions, which cannot be rolled back without extra work) or when the per-query check needs a data structure that all queries can share. The classic example is Meteors (POI 2011): each of k meteor showers adds rain to a range of sectors, and each country wants to know after which shower its sectors have collected enough; each round replays the showers with a Fenwick tree for range additions, for $O((n+k)\log k \log n)$ in total.

Chapter summary

- Write binary search from an invariant: the answer stays in $[lo, hi]$, $mid = lo + (hi - lo)/2$, and one branch moves lo past mid.
- `lower_bound` is the first $\geq x$, `upper_bound` the first $> x$; on a `set` use the member functions.
- Binary search on the answer turns optimisation into checking; the k-th smallest of an implicit set is the first value with at least k elements below it.
- For real answers run a fixed number of iterations; fractional programming turns ratios into sums with weights $a_i - \lambda b_i$, and Dinkelbach's method converges in a few steps.
- Ternary search optimises unimodal (in practice, convex) functions; on integers, binary search the slope instead.
- Parallel binary search answers many monotone queries with one replay of the events per round.

Practice problems (CSES Problem Set)

BINARY SEARCH

- Concert Tickets
- Sum of Two Values
- Factory Machines
- Array Division
- Multiplication Table
- Increasing Subsequence

CHAPTER 9

Prefix Sums, Two Pointers and Sliding Windows

A surprising number of problems ask about *subarrays*: their sums, their distinct values, their medians, the longest or shortest one with some property. An array of n elements has about $n^2/2$ subarrays, so looking at each is quadratic. This chapter collects the techniques that bring such questions down to linear or near-linear time: prefix sums, which answer any range sum in constant time; difference arrays, their inverse; the hash-map trick for counting subarrays with a given sum; two pointers, which slide a window across the array while touching every element only twice; and the stacks and multisets that maintain more complex information about a sliding window. Together they solve a large fraction of all Silver problems and appear as building blocks inside countless harder ones.

IN THIS CHAPTER BRONZE SILVER GOLD

- answer range-sum queries in one and two dimensions in constant time;
- apply many range updates offline with difference arrays, including rectangles and arithmetic progressions;
- count subarrays with a given sum, remainder or balance using prefix sums and a hash map;
- recognise when two pointers apply, and prove that they run in linear time;
- maintain the median of a sliding window, and find largest rectangles with monotonic stacks.

Before you start: Chapters 3, 5 and 7.

9.1 Prefix sums

BRONZE

Define $P_0 = 0$ and $P_{i+1} = P_i + a_i$, so that P_i is the sum of the first i elements $a_0, \ldots, a_{i-1}$. Then the sum of any range is the difference of two prefix sums:

$$a_l + a_{l+1} + \cdots + a_{r-1} = P_r - P_l,$$

because P_r contains exactly the elements $a_0, \ldots, a_{r-1}$ and subtracting P_l removes $a_0, \ldots, a_{l-1}$ (Figure 9.1). The array P takes $O(n)$ to build and then answers every range sum in $O(1)$. The *half-open* convention, ranges $[l, r)$ that include l and exclude r, is what makes the formula clean: there is no ± 1 anywhere, the empty range $l = r$ has sum $P_l - P_l = 0$, and P has one more entry than a.

</> Template: Prefix sums (1D) Ctrl+Alt+T, 1 prefix `cpprefix`

The same idea works for any operation that can be *undone*. Prefix XORs give range XORs, since $x \oplus x = 0$. Prefix counts give "how many times does the letter c occur in $[l, r)$?" for every letter (26 prefix arrays, or one array of 26 counters per position). Prefix sums modulo a prime give range

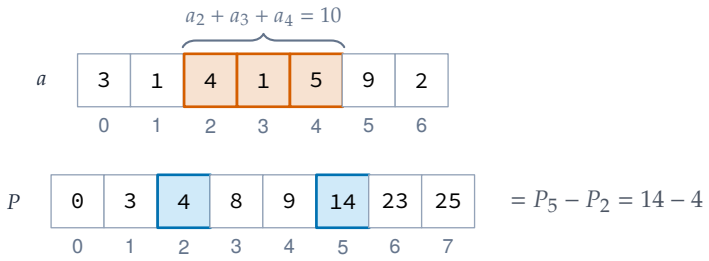


Figure 9.1: Prefix sums sit *between* the elements: P_i is the sum of everything to the left of boundary i. A range sum is the difference of the prefix sums at its two boundaries.

sums modulo that prime, and prefix *products* give range products if every element is invertible. Operations that cannot be undone, such as minimum, maximum and gcd, have no prefix trick: a range minimum is not a difference of two prefix minima. Those need the sparse table or segment tree of Part VI.

Prefix sums without special cases Use the half-open convention everywhere and size P as $n + 1$. When the input is 1-indexed and inclusive, convert once at reading time (`l--`, keep `r`), not inside every formula. For circular arrays, append the array to itself and use ranges of length at most n. And when values or counts may exceed 2^{31}, the prefix sums certainly will: make them `long long` even if the elements fit in `int`.

9.1.1 Two dimensions

Let $P_{i,j}$ be the sum of the rectangle of rows $[0, i)$ and columns $[0, j)$. The sum of a rectangle with rows $[r_1, r_2)$ and columns $[c_1, c_2)$ follows by inclusion–exclusion: take everything above and to the left of its lower-right corner, remove the strip above it and the strip to its left, and add back the corner region, which was removed twice (Figure 9.2):

$$\text{sum} = P_{r_2,c_2} - P_{r_1,c_2} - P_{r_2,c_1} + P_{r_1,c_1}.$$

The table itself is built with the same identity in reverse, $P_{i+1,j+1} = P_{i,j+1} + P_{i+1,j} - P_{i,j} + a_{i,j}$, in $O(nm)$.

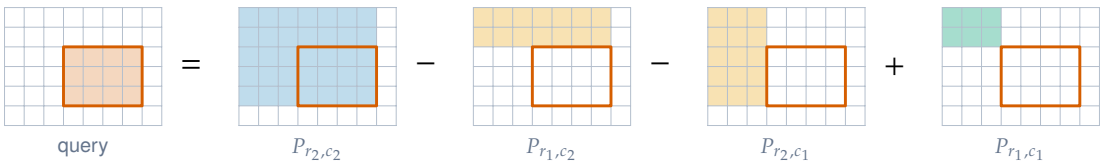


Figure 9.2: Two-dimensional prefix sums as a picture equation. The query rectangle (red frame) equals the region above and left of its lower-right corner, minus the strip above it, minus the strip to its left, plus the corner region that both strips removed.

Example: Forest Queries (CSES)

An $n \times n$ grid ($n \le 1000$) contains trees. Answer $2 \cdot 10^5$ queries: how many trees are in a given rectangle?

Solution. Build P once, answer each query with the four-term formula. Converting the 1-indexed, inclusive rectangle to the half-open form is the only place where care is needed.

```cpp
void solve() {
    int n, q;
    cin >> n >> q;
    // P[i][j] = number of trees in rows < i and columns < j
    vvi P(n + 1, vi(n + 1, 0));
    rep(i, n) {
        string row;
        cin >> row;
        rep(j, n) {
            int tree = row[j] == '*';
            P[i + 1][j + 1] = P[i][j + 1] + P[i + 1][j] - P[i][j] + tree;
        }
    }
    while (q--) {
        int y1, x1, y2, x2;  // rows y1..y2, columns x1..x2, 1-indexed
        cin >> y1 >> x1 >> y2 >> x2;
        y1--, x1--;  // to the half-open rectangle [y1, y2) x [x1, x2)
        print(P[y2][x2] - P[y1][x2] - P[y2][x1] + P[y1][x1]);
    }
}
```

</> Template: 2D prefix sums Ctrl+Alt+T, Shift+A prefix `cpprefix2d`

In d dimensions the formula has 2^d terms with alternating signs, one per corner of the box. A different way to see it, which generalises better, is to take prefix sums along one dimension at a time: first along the rows, then along the columns of the result. After both passes, each entry is the sum of its whole upper-left rectangle. The same "one dimension at a time" idea computes sums over all submasks of every bitmask (Chapter 19), which is prefix sums in n dimensions of size 2.

9.2 Difference arrays

SILVER

Prefix sums answer many queries about a fixed array. The inverse problem, applying many *updates* to an array and reading it only at the end, is solved by the inverse operation. The **difference array** of a is $D_i = a_i - a_{i-1}$ (with $a_{-1} = 0$), so a is the prefix sum of D. Adding v to every element of $[l, r)$ changes only two differences: D_l grows by v and D_r shrinks by v. So each range update costs $O(1)$, and one prefix-sum pass at the end rebuilds the array (Figure 9.3).

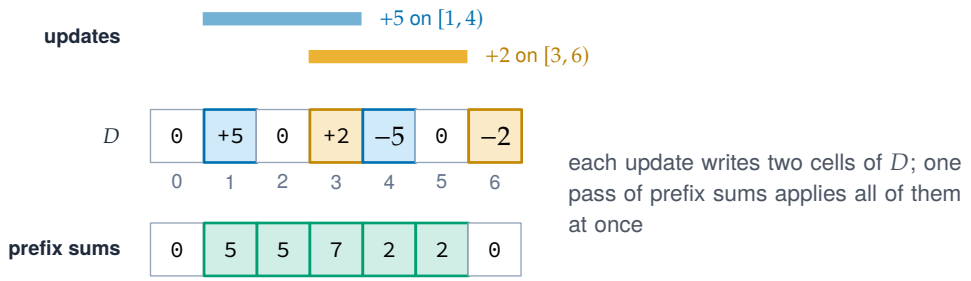


Figure 9.3: A difference array. The update $+5$ on $[1, 4)$ becomes $+5$ at index 1 and -5 at index 4; after all updates, the prefix sums of D are the final values.

</> Template: Difference array (offline range add, 1D) Ctrl+Alt+T, 3 prefix `cpdiff`

9.2.1 Rectangles

In two dimensions, adding v to the rectangle $[x_1, x_2) \times [y_1, y_2)$ writes four corners of the difference array: $+v$ at (x_1, y_1) and (x_2, y_2), and $-v$ at (x_1, y_2) and (x_2, y_1). After 2D prefix sums, the $+v$ at (x_1, y_1) spreads to the whole quadrant below and to the right of it; the two $-v$ remove the parts beyond x_2 and beyond y_2; and the second $+v$ restores the part beyond both, which was removed twice (Figure 9.4). This is inclusion–exclusion again, run backwards.

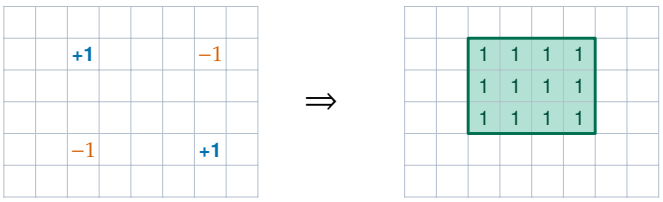


Figure 9.4: Adding 1 to the rectangle of columns 2–5 and rows 1–3. The difference array gets $+1$ at the rectangle's top-left cell and at the cell diagonally beyond its bottom-right corner, and -1 at the cells just beyond its other two corners; 2D prefix sums turn these four values into exactly the rectangle.

Example: Painting the Barn (USACO Silver)

Farmer John paints $n \le 10^5$ axis-parallel rectangles on a barn wall with coordinates in $[0, 1000]$; each rectangle adds one coat of paint to the unit squares it covers. How many unit squares end up with exactly k coats?

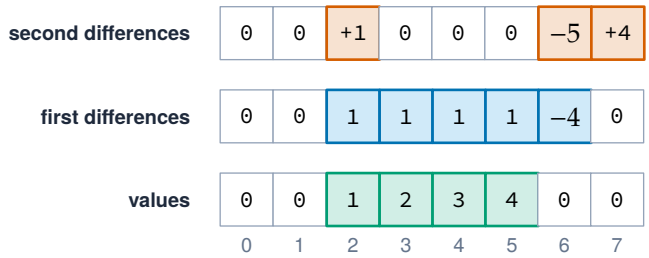
Solution. Four updates per rectangle into a 1001×1001 difference array, one 2D prefix-sum pass, and a count: about 10^6 operations, regardless of how large the rectangles are.

```
void solve() {
    int n, k;
    cin >> n >> k;
    const int N = 1001;
    vvi D(N + 1, vi(N + 1, 0));  // 2D difference array
    rep(i, n) {
        int x1, y1, x2, y2;  // the unit squares [x1, x2) x [y1, y2)
        cin >> x1 >> y1 >> x2 >> y2;
        D[x1][y1]++, D[x1][y2]--, D[x2][y1]--, D[x2][y2]++;
    }
    int cells = 0;
    rep(x, N) rep(y, N) {  // 2D prefix sums in place: D[x][y] = coats on (x, y)
        if (x > 0) D[x][y] += D[x - 1][y];
        if (y > 0) D[x][y] += D[x][y - 1];
        if (x > 0 && y > 0) D[x][y] -= D[x - 1][y - 1];
        if (D[x][y] == k) cells++;
    }
    print(cells);
}
```

Template: Difference array (offline rectangle add, 2D) Ctrl+Alt+T, 4 prefix `cpdiff2d`

9.2.2 Arithmetic progressions: differences of differences

What if an update adds $1, 2, 3, \ldots$ to the elements of a range? The first differences of such an update are not two spikes but a whole block of ones, so one level of differences is not enough. Take differences *twice*: the second differences of the progression are $+1$ at its start, $-(len + 1)$ just after its end and $+len$ one step later, three spikes again (Figure 9.5). Two prefix-sum passes rebuild the array. In general, an update by a polynomial of degree d becomes $O(d)$ spikes after $d + 1$ levels of differences, and $d + 1$ prefix-sum passes undo them.



adding $1, 2, 3, 4$ to positions $2..5$ ($len = 4$): three spikes in the second differences; two passes of prefix sums rebuild the values

Figure 9.5: An arithmetic progression added to a range is three spikes in the second differences.

```
void solve() {
    int n, q;
    cin >> n >> q;
    vi d2(n + 2, 0);  // second differences
    while (q--) {
        int l, r;  // add 1, 2, ..., r - l + 1 to positions l..r (0-indexed)
        cin >> l >> r;
        int len = r - l + 1;
        d2[l] += 1;  // the first differences become 1 on [l, r]
        d2[r + 1] -= len + 1;  // ... then drop by len right after r
        d2[r + 2] += len;  // ... and are back to 0 after that
    }
    vi a(n);
    ll d1 = 0, v = 0;
    rep(i, n) {
        d1 += d2[i];  // first difference a[i] - a[i - 1]
        v += d1;
        a[i] = v;
    }
    print(a);
}
```

How to recognize it Many updates, few (or one) reads, and the updates add something simple to a contiguous range: an interval of days, a rectangle of cells, a path in a tree (Chapter 16). When reads and updates are interleaved, the same two-point update goes into a Fenwick tree instead, which supports prefix sums online (Chapter 22).

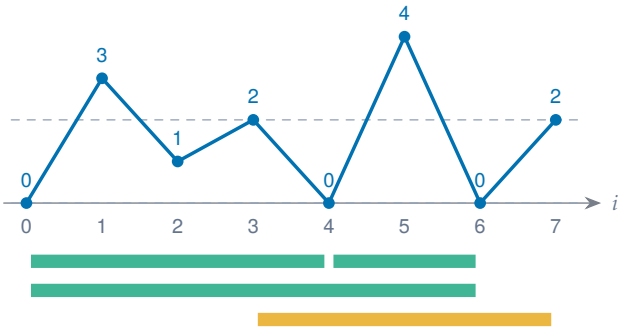
9.3 Counting subarrays with a hash map

SILVER

"How many subarrays have sum exactly x?" With prefix sums, the subarray $[l, r)$ has sum x exactly when $P_r - P_l = x$, that is, when $P_l = P_r - x$. So scan r from left to right, keep a map from each

prefix sum seen so far to the number of times it has occurred, and add the count of $P_r - x$ at each step: every earlier prefix with that value is the left end of a subarray ending at r with sum x. This counts all subarrays in $O(n \log n)$ with a `map` (or expected $O(n)$ with a hash map), even when the values are negative, which rules out two pointers. The complete program is the worked problem Subarray Sums II (Chapter 31).

Picturing the prefix sums as a walk makes these counting problems visual: the subarrays with sum 0 are exactly the pairs of moments at which the walk is at the same height (Figure 9.6), and the subarrays with sum x are the pairs of moments whose heights differ by x.



$a = 3, -2, 1, -2, 4, -4, 2$
prefix sums $P_0, \ldots, P_7$ (blue);
equal heights are zero-sum
subarrays: $[0, 4)$, $[4, 6)$, $[0, 6)$ at
height 0 and $[3, 7)$ at height 2

Figure 9.6: Prefix sums as a walk. Every pair of equal heights is a subarray with sum 0; the bars below the axis show the four such subarrays (green at height 0, orange at height 2). Counting them is counting pairs of equal values in P.

The same scan answers a family of questions; only the map and its lookup change.

- **Sum divisible by m:** two prefixes with equal P mod m (Section 2.8); normalise negative remainders with `((P % m) + m) % m`.
- **XOR equal to x:** prefix XORs, and $X_r \oplus X_l = x \iff X_l = X_r \oplus x$.
- **Equal numbers of two kinds of elements:** count one kind as +1 and the other as −1, and look for zero sums.
- **The longest such subarray:** store the *first* index of each prefix value instead of a count; the longest subarray ending at r starts at the first occurrence of the value it needs.

Example: Longest balanced substring

Find the length of the longest substring of a binary string that contains equally many zeros and ones.

Solution. With +1 for each one and −1 for each zero, balanced substrings are exactly the zero-sum subarrays. The longest one ending at position i starts right after the *first* prefix with the same sum, so a map from prefix sum to the first index where it occurred gives the answer in one pass.

```
void solve() {
    int n;
    cin >> n;
    string s;
    cin >> s;
    // count 1 as +1 and 0 as -1: balanced means a zero sum
    map<int, int> first;  // the first index at which each prefix sum occurs
    first[0] = 0;
```

```
    int pre = 0, best = 0;
    rep(i, n) {
        pre += s[i] == '1' ? 1 : -1;
        if (first.count(pre)) chmax(best, i + 1 - first[pre]);
        else first[pre] = i + 1;
    }
    print(best);
}
```

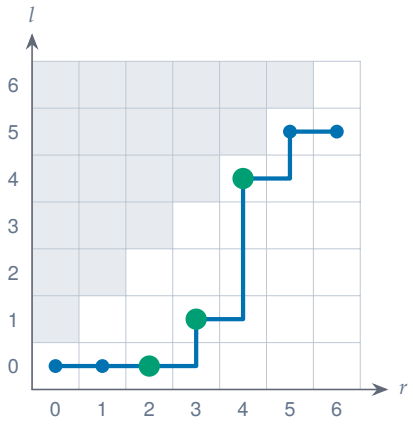
⚠ Hash maps and adversarial tests An `unordered_map` of prefix sums is expected $O(1)$ per operation, but on Codeforces, tests are often built to collide with the default hash, which makes every operation linear. Use a `map`, a custom hash with a random seed (Chapter 4), or, when the prefix values lie in a small range (as in the balanced-string problem, where they lie in $[-n, n]$), a plain array with an offset.

9.4 Two pointers

SILVER

Suppose we want, for every right end r, the smallest left end l such that the window $[l, r]$ satisfies some condition, and the condition is *monotone*: shrinking a valid window keeps it valid (for example "the sum is at most x" with non-negative values, or "all values are distinct"). Then the best left end $l(r)$ never moves backwards when r grows, because a window that was invalid stays invalid when it is extended to the right. So the two ends can be moved with two pointers, both only forwards: for each new r, advance l while the window is invalid.

Why it is linear. Each step of the algorithm moves one pointer one position forwards, and each pointer moves at most n times, so there are at most $2n$ steps in total, however the work is distributed among the iterations. Plotting the pair (r, l) as a point, the algorithm walks a monotone staircase path from the bottom-left to the top-right corner of an $n \times n$ grid (Figure 9.7). This is the simplest instance of amortised analysis (Section 3.5).



Subarray Sums I with $a = 2, 4, 1, 2, 7, 1, 3$ and $x = 7$. For each r the dot marks the left end $l(r)$ after shrinking; green dots are windows with sum exactly 7: $[0, 2]$, $[1, 3]$ and $[4, 4]$. The path only goes right or up, so it has at most $2n$ steps.

Figure 9.7: Two pointers as a staircase. Moving r is a step to the right, moving l a step up; the shaded region ($l > r$) is never entered here. The total number of steps is at most $2n$, although a single value of r can move l many times.

Example: Subarray Sums I (CSES)

Count the subarrays of an array of $n \leq 2 \cdot 10^5$ *positive* integers whose sum is exactly x.

Solution. With positive values, the sum of a window only grows when it is extended and only shrinks when it is cut, so for each right end there is at most one left end with sum x, and it can be found with two pointers: add a_r, then remove elements from the left while the sum exceeds x.

```
void solve() {
    int n;
    ll x;
    cin >> n >> x;
    vi a(n);
    cin >> a;
    ll sum = 0, ways = 0;  // all values are positive
    int l = 0;
    rep(r, n) {  // the window is a[l..r]
        sum += a[r];
        while (sum > x) sum -= a[l++];  // too large: shrink from the left
        if (sum == x) ways++;
    }
    print(ways);
}
```

Template: Two pointers (sliding window) Ctrl+Alt+T, 5 prefix `cptwoptr`

Example: Playlist (CSES)

Find the longest contiguous part of a playlist in which no song repeats.

Solution. "All songs distinct" is monotone under shrinking. For each right end, the left end must move past the previous occurrence of the new song if that occurrence is inside the window. Storing the last position of every song lets the left pointer jump there directly.

```
void solve() {
    int n;
    cin >> n;
    vi k(n);
    cin >> k;
    map<ll, int> last;  // the last position of each song so far
    int l = 0, best = 0;
    rep(r, n) {  // the longest window of distinct songs that ends at r
        auto it = last.find(k[r]);
        if (it != last.end() && it->second >= l) l = it->second + 1;
        last[k[r]] = r;
        chmax(best, r - l + 1);
    }
    print(best);
}
```

9.4.1 Counting windows: at most, and exactly

When the condition is monotone, all windows ending at r with left end at least $l(r)$ are valid, so the number of valid windows is $\sum_r (r - l(r) + 1)$, again in linear time.

Example: Subarray Distinct Values (CSES)

Count the subarrays that contain at most k distinct values.

Solution. Keep the count of each value in the window in a map; the number of distinct values is the size of the map. For each r, add a_r and shrink from the left while there are more than k distinct values; then add $r - l + 1$.

```
void solve() {
    int n, k;
    cin >> n >> k;
    vi a(n);
    cin >> a;
    map<ll, int> cnt;  // occurrences of each value in the window a[l..r]
    ll ways = 0;
    int l = 0;
    rep(r, n) {
        cnt[a[r]]++;
        while (sz(cnt) > k) {  // too many distinct values: shrink
            if (--cnt[a[l]] == 0) cnt.erase(a[l]);
            l++;
        }
        ways += r - l + 1;  // every window a[l'..r] with l' >= l is fine
    }
    print(ways);
}
```

Exactly k = at most k − at most $k-1$ "Exactly k distinct values" is not monotone: shrinking a window can drop it to $k-1$. But

$$\#\{\text{exactly } k\} = \#\{\text{at most } k\} - \#\{\text{at most } k-1\},$$

and both terms on the right are monotone conditions that two pointers count directly. The same subtraction handles "sum exactly x" with non-negative values (at most x minus at most $x-1$), "exactly k odd numbers" and many more.

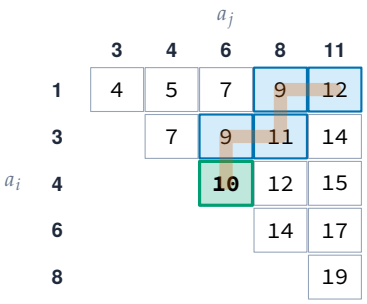
9.4.2 Pointers from both ends

The other classic form of two pointers starts one pointer at each end of a sorted array and moves them towards each other.

Example: Sum of Two Values (CSES)

Find two positions (distinct) whose values add up to x, or report that there are none.

Solution. Sort the values (remembering their original positions) and start with i at the smallest and j at the largest. If $a_i + a_j < x$, then a_i is too small even for the largest remaining partner, so it can be discarded: i moves right. If $a_i + a_j > x$, then a_j is too large even for the smallest remaining partner: j moves left. Each step discards one value for good, which is why the scan is linear and misses nothing (Figure 9.8).



Sums $a_i + a_j$ for $a = 1, 3, 4, 6, 8, 11$ and target 10. They increase downwards and to the right. From the top-right corner, a sum that is too large rules out its whole column (move left), and one that is too small rules out its whole row (move down).

Figure 9.8: Two pointers from both ends as a walk through the table of pair sums: $12 > 10$ (left), $9 < 10$ (down), $11 > 10$ (left), $9 < 10$ (down), 10 found. Each step discards a whole row or column, so at most n steps are needed.

```
void solve() {
    int n;
    ll x;
    cin >> n >> x;
    vector<pll> a(n);  // (value, original position)
    rep(i, n) cin >> a[i].first, a[i].second = i + 1;
    sort(all(a));
    int i = 0, j = n - 1;
    while (i < j) {
        ll s = a[i].first + a[j].first;
        if (s == x) {
            print(a[i].second, a[j].second);
            return;
        }
        if (s < x) i++;  // a[i] is too small for every remaining partner
        else j--;  // a[j] is too large for every remaining partner
    }
    print("IMPOSSIBLE");
}
```

Fixing one element and running this scan on the rest solves 3-SUM (Sum of Three Values) in $O(n^2)$; for 4-SUM, fix two elements, or use meet in the middle over pairs (Section 6.6).

How to recognize it "The longest or shortest subarray such that...", "count the subarrays with at most...", pairs in a sorted array with a given sum or difference. The key test is monotonicity: does shrinking a valid window keep it valid? With non-negative values sums are monotone; with negative values they are not, and prefix sums with a map or a data structure take over.

9.5 Sliding windows

GOLD

When every window has the same length k, the question is usually an aggregate of the window: its minimum, maximum, sum, median or number of distinct values. The sum and the number of distinct values are updated in $O(1)$ as one element enters and one leaves. The minimum and maximum need the monotonic deque of Section 5.3, which is amortised $O(1)$ per step; for any other associative operation, the two-stack queue of Section 5.7 works. The median is harder, because it needs order statistics of a multiset that changes at both ends.

Example: Sliding Window Median (CSES)

For each window of k consecutive elements, print its median (the lower one when k is even).

Solution. Split the window into two multisets: *low* holds the $\lceil k/2 \rceil$ smallest values and *high* the rest, so the median is the largest element of *low* (Figure 9.9). A new element goes into the set it belongs to by comparison with that median; the leaving element is erased from whichever set contains it; then at most one element moves between the sets to restore the sizes. Every step is $O(\log k)$.



Figure 9.9: Sliding window median with $k = 5$. *low* keeps the three smallest values of the window, *high* the other two; the median is the largest value in *low*. When the window slides, 5 is erased and 2 inserted, and the median becomes 3.

```
void solve() {
    int n, k;
    cin >> n >> k;
    vi a(n);
    cin >> a;
    // low: the (k + 1) / 2 smallest values of the window, high: the rest
    multiset<ll> low, high;
    auto balance = [&]() {
        while (sz(low) > (sz(low) + sz(high) + 1) / 2) {
            high.insert(*prev(low.end()));
            low.erase(prev(low.end()));
        }
        while (sz(low) < (sz(low) + sz(high) + 1) / 2) {
            low.insert(*high.begin());
            high.erase(high.begin());
        }
    };
    vi med;
    rep(i, n) {
        if (low.empty() || a[i] <= *prev(low.end())) low.insert(a[i]);
        else high.insert(a[i]);
        if (i >= k) {  // a[i - k] leaves the window
            auto it = low.find(a[i - k]);
            if (it != low.end()) low.erase(it);
```

```
            else high.erase(high.find(a[i - k]));
        }
        balance();
        if (i >= k - 1) med.push_back(*prev(low.end()));  // largest of low
    }
    print(med);
}
```

Two details deserve attention. Erasing a value with `low.erase(x)` would remove *all* copies of it; `erase` of an iterator removes exactly one. And the leaving element must be erased from the set that actually contains it, which the code decides by searching *low* first. Keeping the sums of both sets as well gives Sliding Window Cost, the minimum total distance from the window's values to a single point: by Section 7.6 that point is the median, and the cost is $(\text{med} \cdot |low| - \Sigma_{low}) + (\Sigma_{high} - \text{med} \cdot |high|)$.

9.6 Monotonic stacks in action

GOLD

The monotonic stack of Section 5.2 computes, for every element, the nearest smaller (or larger) element on each side in $O(n)$. Those two numbers describe the largest window in which an element is the minimum, and that is exactly what many window problems need: the contribution technique of Section 2.1 counts the subarrays in which each element is the minimum, and the following classic finds the largest rectangle under a histogram.

Example: Advertisement (CSES)

A fence consists of n boards of width 1 and heights h_i. Find the largest rectangular advertisement that can be placed on the fence, aligned with the boards.

Solution. The largest rectangle has the height of its lowest board i (otherwise it could be made taller), and extends left and right until a lower board blocks it. With L_i and R_i, the nearest lower boards on each side from two monotonic stack passes, the rectangle for board i has width $R_i - L_i - 1$, and the answer is $\max_i h_i(R_i - L_i - 1)$ (Figure 9.10).

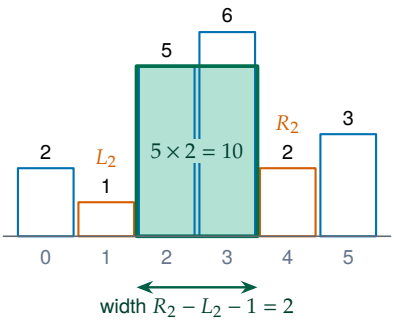


Figure 9.10: The largest rectangle in the histogram $2, 1, 5, 6, 2, 3$ has height 5, set by board 2. It extends from just after the nearest lower board on the left ($L_2 = 1$) to just before the nearest lower board on the right ($R_2 = 4$); both blockers are drawn in red.

```
void solve() {
    int n;
    cin >> n;
    vi h(n);
```

```
    cin >> h;
    // L[i], R[i]: nearest bars to the left and right lower than h[i]
    vi L(n), R(n), st;
    rep(i, n) {
        while (!st.empty() && h[st.back()] >= h[i]) st.pop_back();
        L[i] = st.empty() ? -1 : st.back();
        st.push_back(i);
    }
    st.clear();
    per(i, n) {
        while (!st.empty() && h[st.back()] >= h[i]) st.pop_back();
        R[i] = st.empty() ? n : st.back();
        st.push_back(i);
    }
    ll best = 0;
    rep(i, n) chmax(best, h[i] * (R[i] - L[i] - 1));  // as tall as bar i
    print(best);
}
```

</> Template: Monotonic stack (next greater / previous smaller) Ctrl+Alt+T, 7 prefix `cpmonostack`

From one dimension to two. The largest rectangle of free cells in an $n \times m$ grid (Maximum Building I) reduces to n histograms: for each row, let h_j be the number of consecutive free cells ending at this row in column j (zero if the cell is blocked, otherwise one more than in the previous row). The best rectangle whose bottom edge lies on this row is the largest rectangle of that histogram, so the whole grid takes $O(nm)$. Counting all free rectangles instead of finding the largest one (Maximum Building II) uses the same heights with the contribution technique.

Chapter summary

- Prefix sums answer range sums in $O(1)$ after $O(n)$ work; use half-open ranges and $n + 1$ entries. In two dimensions, inclusion–exclusion over four corners.
- Difference arrays apply range updates in $O(1)$ each and rebuild with one prefix-sum pass; rectangles need four corners, arithmetic progressions a second level of differences.
- Counting subarrays with sum x is counting pairs of prefix sums with difference x: scan with a map. Variations handle remainders, XOR, balance, and the longest such subarray.
- Two pointers work when the window condition is monotone; both pointers move forwards, so the scan is linear. Count "exactly" as "at most k" minus "at most $k - 1$".
- Two multisets maintain a sliding median; monotonic stacks give the largest window in which each element is the minimum, which solves the largest rectangle problems.

Practice problems (CSES Problem Set)

PREFIX SUMS AND DIFFERENCES

- Static Range Sum Queries
- Forest Queries
- Subarray Sums II
- Subarray Divisibility

TWO POINTERS

- Subarray Sums I
- Sum of Two Values
- Sum of Three Values
- Sum of Four Values
- Playlist
- Subarray Distinct Values

SLIDING WINDOWS AND STACKS

- Sliding Window Median
- Sliding Window Cost
- Nearest Smaller Values
- Advertisement
- Maximum Building I

CHAPTER 10

Bits and Rectangles

Computers store integers in binary, and a single machine instruction applies a logical operation to all 64 bits of a word at once. Competitive programmers exploit this in three ways: to treat a number as a small *set* (a bitmask), to reason about numbers *one bit at a time* (the key to almost every XOR problem), and to process 64 booleans per instruction (bitsets). This chapter develops all three, together with two beautiful bit-level objects, Gray codes and de Bruijn sequences. It closes with axis-parallel rectangles, the geometry of USACO Bronze, whose intersection formula is the two-dimensional version of the interval arithmetic used throughout the book.

IN THIS CHAPTER BRONZE SILVER GOLD

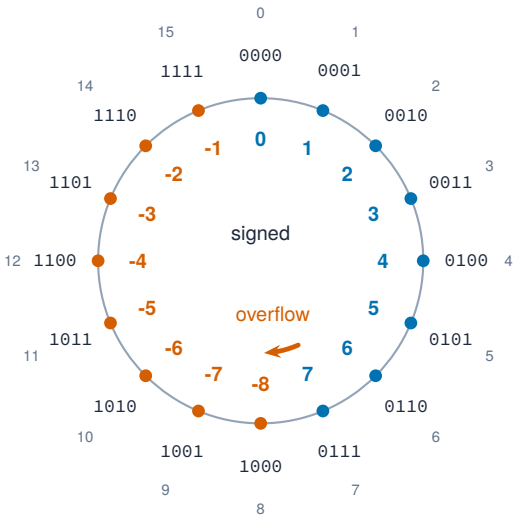
- understand two's complement, and avoid the classic overflow and precedence bugs of bit operations;
- use the everyday bit tricks and compiler built-ins, and count bits in closed form;
- solve XOR problems bit by bit, and find maximum XOR pairs with a binary trie;
- speed up boolean DPs and set operations by a factor of 64 with bitsets;
- generate Gray codes and de Bruijn sequences, and see why they matter;
- compute intersections and unions of rectangles.

Before you start: Chapters 4 and 6.

10.1 Binary numbers and two's complement BRONZE

A w-bit unsigned integer x stands for $\sum_i x_i 2^i$, where $x_i \in \{0, 1\}$ is *bit* i; bit 0 is the least significant bit and bit $w-1$ the most significant. Signed integers use **two's complement**: the top bit has weight -2^{w-1} instead of $+2^{w-1}$. The effect is easiest to see on a wheel (Figure 10.1): the 2^w bit patterns are arranged in a circle, adding one moves one step clockwise, and the signed and unsigned readings differ only in where the circle is cut. Three consequences are worth remembering.

- $-x$ is `~x + 1`: flip every bit and add one. In particular -1 has all bits set, and x and $-x$ agree in their lowest set bit and differ in every bit above it (Figure 10.2).
- Overflow wraps around the circle: one step past the largest signed value $2^{w-1}-1$ lands on the smallest, -2^{w-1}. For unsigned types this wrap-around is defined behaviour; for signed types it is undefined behaviour, which the optimiser is allowed to exploit in surprising ways (Section 4.9).
- The range of a w-bit signed type is $[-2^{w-1}, 2^{w-1}-1]$: about $\pm 2.1 \cdot 10^9$ for `int` and $\pm 9.2 \cdot 10^{18}$ for `long long`.



4-bit two's complement. Outside: the unsigned value; on the ring: the bit pattern; inside: the signed value. Adding 1 moves clockwise. The only difference between the two readings is where the circle is cut: between 15 and 0 for unsigned values, between 7 and -8 for signed ones.

Figure 10.1: Two's complement on a wheel. Patterns with top bit 0 (blue) mean the same in both readings; patterns with top bit 1 (red) mean $v - 16$ as signed numbers. Signed overflow is the step from 7 to -8.

Operator	Meaning	Example	Result
`a & b`	AND: 1 where both are 1	`1100 & 1010`	`1000`
`a | b`	OR: 1 where either is 1	`1100 | 1010`	`1110`
`a ^ b`	XOR: 1 where they differ	`1100 ^ 1010`	`0110`
`~a`	NOT: flip every bit	`~0`	`-1` (all ones)
`a << k`	shift left: multiply by 2^k	`0011 << 2`	`1100`
`a >> k`	shift right: divide by 2^k, rounding down	`1100 >> 2`	`0011`

Table 10.1: The bitwise operators. All of them cost the same as an addition.

The three classic bit bugs **Width:** `1 << 40` shifts an `int` and is undefined; write `1LL << 40`. Shifting by the width of the type or more, or by a negative amount, is undefined too. **Precedence:** the comparison operators bind *tighter* than `&`, `|` and `^`, so `x & 1 == 0` means `x & (1 == 0)`, which is always 0; write `(x & 1) == 0`. Shifts bind tighter than `&`, so the common `x >> i & 1` is correct as written. **Signedness:** right-shifting a negative number copies the sign bit, so `-8 >> 1` is -4, and `-1 >> 1` stays -1; for pure bit manipulation of arbitrary patterns, use `unsigned long long`.

10.2 Bit tricks

SILVER

Table 10.2 collects the expressions that appear in almost every bitmask solution. Most of them follow from two observations. First, subtracting one from x flips its lowest set bit and every zero below it, and nothing else. Second, $-x = \sim x + 1$ agrees with x at the lowest set bit and below, and is the complement of x above it (Figure 10.2). So `x & (x - 1)` removes the lowest set bit, and `x & -x` isolates it.

</> Template: Bit helpers Ctrl+Alt+T, Y prefix `cpbits`

To iterate over the set bits of x, peel them off one at a time: `for (ll m = x; m; m &= m - 1)` visits each set bit once (its index is `__builtin_ctzll(m)`), so the loop costs the popcount of x rather than

Expression	Meaning
`x >> i & 1`	bit i of x
`x | (1LL << i)`	set bit i
`x & ~(1LL << i)`	clear bit i
`x ^ (1LL << i)`	toggle bit i
`x & -x`	the lowest set bit, as a number ($12 \to 4$); the step size of a Fenwick tree
`x & (x - 1)`	x without its lowest set bit; it is 0 exactly when x is 0 or a power of two
`(1LL << n) - 1`	the full set $\{0, 1, \ldots, n-1\}$
`(a & b) == a`	$a \subseteq b$ as sets
`__builtin_popcountll(x)`	the number of set bits
`__builtin_ctzll(x)`, `__builtin_clzll(x)`	trailing, leading zeros (undefined for $x = 0$)
`63 - __builtin_clzll(x)`	the index of the highest set bit, $\lfloor \log_2 x \rfloor$ (also `__lg(x)`)
`__builtin_parityll(x)`	the parity of the number of set bits

Table 10.2: Everyday bit tricks. The built-ins compile to single instructions on modern processors; the versions without `ll` take 32-bit arguments and silently truncate 64-bit ones.

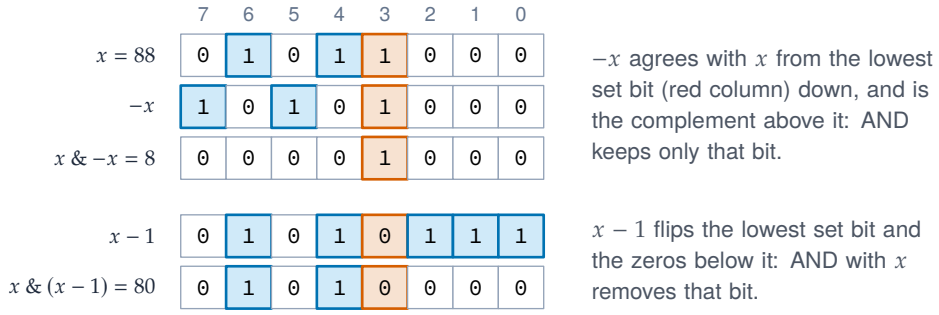


Figure 10.2: The two fundamental tricks on $x = 88 = 01011000_2$ (bit numbers on top). Its lowest set bit is bit 3, worth 8.

the width of the word. This is how Kernighan counted bits, and it is the right loop whenever the sets are sparse.

10.2.1 Counting bits in closed form

Many problems ask for a sum of bit-related quantities over a huge range of numbers. The key is that bit b of the numbers $0, 1, 2, \ldots$ follows a fixed periodic pattern: 2^b zeros, then 2^b ones, repeating with period 2^{b+1} (Figure 10.3). So each bit can be counted in $O(1)$, and a question about 10^{15} numbers takes 50 steps.

Example: Counting Bits (CSES)

Count the total number of one bits in the binary representations of $1, 2, \ldots, n$, for $n \le 10^{15}$.

Solution. For each bit b, the $n+1$ numbers $0, \ldots, n$ contain $\lfloor (n+1)/2^{b+1} \rfloor$ complete periods, each with 2^b ones, and a partial period of $r = (n+1) \bmod 2^{b+1}$ numbers, which contains $\max(0, r - 2^b)$ ones.

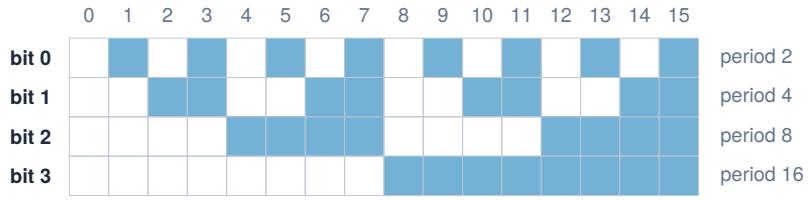


Figure 10.3: The bits of $0, \ldots, 15$ (one column per number, blue = 1). Bit b alternates between runs of 2^b zeros and 2^b ones, so the number of ones among $0, \ldots, n$ is the number of complete periods times 2^b, plus the ones in the last, partial period.

```
void solve() {
    ll n;
    cin >> n;
    // count the numbers 0..n with bit b set: bit b repeats the pattern
    // 2^b zeros, 2^b ones, so count whole periods and the partial one
    ll total = 0;
    for (int b = 0; (1LL << b) <= n; b++) {
        ll period = 1LL << (b + 1), half = 1LL << b;
        total += (n + 1) / period * half + max(0LL, (n + 1) % period - half);
    }
    print(total);
}
```

10.3 XOR

GOLD

XOR is addition modulo 2 in every bit separately, without carries. That makes it the most algebraically pleasant of the bit operations, and entire problem genres are built on the following properties.

- **It cancels:** $x \oplus x = 0$ and $x \oplus 0 = x$, and XOR is commutative and associative. So XOR-ing a value twice removes it, and the XOR of a list in which every value appears twice except one is that value.
- **Prefix XORs** work exactly like prefix sums: $a_l \oplus \cdots \oplus a_{r-1} = X_r \oplus X_l$ (Section 9.1).
- **Bits are independent:** bit b of $x \oplus y$ depends only on bit b of x and y. So XOR problems can be solved one bit at a time, and "maximise the XOR" problems greedily from the highest bit.
- **Addition is XOR plus carries:** $a + b = (a \oplus b) + 2(a \,\&\, b)$, because the bits where both are one are exactly the carries (Figure 10.4). Consequently $a \oplus b \le a + b$, and $a + b = a \oplus b$ exactly when $a \,\&\, b = 0$.

The XOR of $0, 1, \ldots, n$ The prefix XOR $0 \oplus 1 \oplus \cdots \oplus n$ has a period-4 closed form: it equals n, 1, $n + 1$ or 0 when n mod 4 is 0, 1, 2 or 3. (Pairs $2k, 2k + 1$ XOR to 1, and two such pairs cancel.) So the XOR of any range of consecutive integers $l..r$ is $f(r) \oplus f(l - 1)$ in $O(1)$.

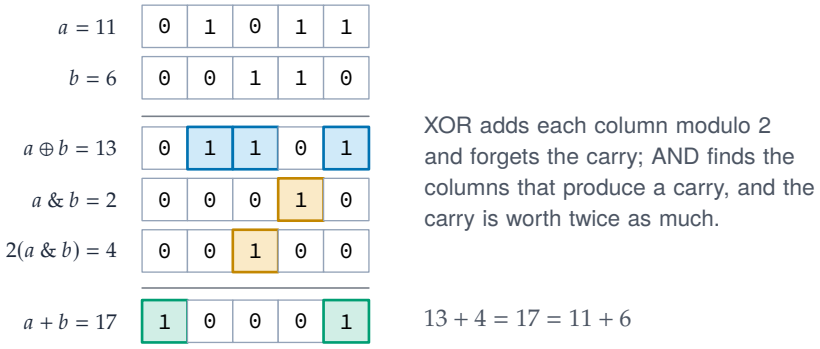


Figure 10.4: Addition is XOR plus twice the AND: $a + b = (a \oplus b) + 2(a \,\&\, b)$.

10.3.1 Counting one bit at a time

Sums of XORs over many pairs decompose by bits. A pair contributes 2^b to the sum of XORs exactly when its two numbers differ in bit b; if c_b numbers have bit b set, $c_b(n - c_b)$ pairs differ there. So

$$\sum_{i<j} a_i \oplus a_j = \sum_b 2^b \, c_b \, (n - c_b),$$

computed in $O(n \log(\max a))$ instead of $O(n^2)$. The same per-bit decomposition gives sums of ANDs (pairs with both bits set: $\binom{c_b}{2}$) and ORs (all pairs minus pairs with both bits clear), and it extends from pairs to subarrays (count prefix XORs with each bit value) and to paths in trees (Chapter 16).

```
void solve() {
    int n;
    cin >> n;
    vi a(n);
    cin >> a;
    // a pair contributes 2^b exactly when its two numbers differ in bit b
    ll total = 0;
    rep(b, 21) {
        ll ones = 0;
        for (ll x : a) ones += x >> b & 1;
        total += (1LL << b) * ones * (n - ones);
    }
    print(total);
}
```

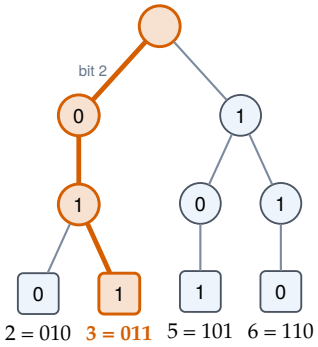
10.3.2 Maximum XOR with a binary trie

Example: Maximum XOR pair

Given $n \le 2 \cdot 10^5$ numbers below 2^{30}, find the maximum of $a_i \oplus a_j$ over all pairs.

Solution. A higher bit outweighs all lower bits together ($2^b > 2^b - 1 = \sum_{c<b} 2^c$), so the best partner for x is found greedily from the top bit down: at each bit, prefer a partner whose bit *differs* from x's, if any candidate with the prefix chosen so far has it. A **binary trie** stores the numbers as root-to-leaf paths of bits, most significant first, so that "does a number with this

prefix exist?" is "does this child exist?" (Figure 10.5). Insert the numbers one by one and query each against those before it: $O(30)$ per operation.



Query $x = 4 = 100_2$ against the set $\{2, 3, 5, 6\}$:

bit 2: x has 1, take 0 +4
bit 1: x has 0, take 1 +2
bit 0: x has 0, take 1 +1

The walk ends at 3, and $4 \oplus 3 = 7$ is the maximum.

Figure 10.5: A binary trie of the 3-bit numbers 2, 3, 5 and 6. To maximise the XOR with x, walk down from the root and take the child with the bit opposite to x's whenever it exists (red path).

```
void solve() {
    int n;
    cin >> n;
    vi a(n);
    cin >> a;
    const int B = 30;  // values below 2^30
    vector<array<int, 2>> child(1, {0, 0});  // binary trie, node 0 = root
    auto insert = [&](ll x) {
        int v = 0;
        per(b, B) {
            int c = x >> b & 1;
            if (!child[v][c]) {
                child[v][c] = sz(child);
                child.push_back({0, 0});
            }
            v = child[v][c];
        }
    };
    auto best_with = [&](ll x) {  // max of x ^ y over the inserted y
        int v = 0;
        ll res = 0;
        per(b, B) {
            int want = (x >> b & 1) ^ 1;  // the opposite bit gives a 1
            if (child[v][want]) res |= 1LL << b, v = child[v][want];
            else v = child[v][want ^ 1];
        }
        return res;
    };
    ll best = 0;
    insert(a[0]);
    rep(i, 1, n) {
        chmax(best, best_with(a[i]));
        insert(a[i]);
    }
    print(best);
}
```

</> Template: Binary trie (maximum XOR) Ctrl+Alt+T, ` prefix `cpxortrie`

Combined with prefix XORs, the same trie finds the maximum XOR of a *subarray* (Maximum Xor Subarray): it is the maximum XOR of two prefix values. Storing counts in the trie nodes answers "how many y have $x \oplus y < k$?" by the same walk, and deletions are just decrements. When the question is about XORs of arbitrary *subsets* rather than pairs, the right tool is a linear basis over GF(2), Gaussian elimination on bits (Chapter 26).

</> Template: XOR basis (linear basis over GF(2)) Ctrl+Alt+Y, Shift+X prefix `cpxorbasis`

10.4 Bitsets: 64 booleans per instruction

GOLD

`bitset<N>` stores N booleans in $N/64$ machine words and implements `&`, `|`, `^`, `~` and the shifts `<<`, `>>` one word at a time. Any algorithm whose inner loop applies the same boolean operation to a whole row of values therefore runs about 64 times faster when the row is a bitset, turning $O(n^2)$ into $O(n^2/64)$ and $O(n^3)$ into $O(n^3/64)$. That factor routinely decides between a time limit exceeded and an accepted solution.

The canonical example is subset sums. If bit s of `can` records whether the sum s can be formed, then adding a coin of value x creates exactly the sums $s + x$: the new set is `can | (can << x)` (Figure 10.6), one shifted OR instead of a loop over all sums.

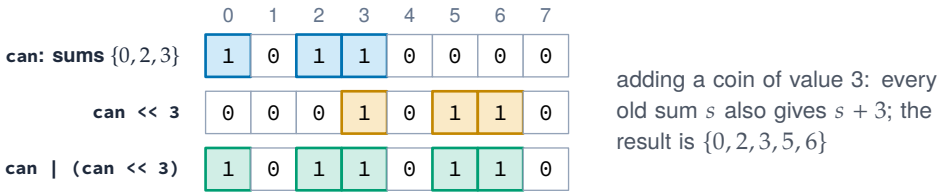


Figure 10.6: The bitset step of subset-sum DP. Bit s (numbered from the left here) says whether the sum s is reachable; a shift by x moves every sum up by x, and the OR merges old and new sums.

Example: Money Sums (CSES)

Given $n \le 100$ coins with values up to 1000, list all sums that can be formed with some non-empty subset of the coins.

Solution. The sums are below 10^5, so a bitset of $10^5 + 1$ bits holds the reachable sums; each coin is one shifted OR. The total work is $100 \cdot 10^5/64 \approx 1.6 \cdot 10^5$ word operations.

```
void solve() {
    int n;
    cin >> n;
    const int MAXS = 100000;
    bitset<MAXS + 1> can;  // can[s]: the coins seen so far can make s
    can[0] = 1;
    rep(i, n) {
        int x;
        cin >> x;
        can |= can << x;  // every old sum, and every old sum plus x
    }
    vi sums;
    for (int s = 1; s <= MAXS; s++) if (can[s]) sums.push_back(s);
```

```
    print(sz(sums));
    print(sums);
}
```

</> Template: Subset sums with bitset Ctrl+Alt+T, Shift+1 prefix `cpbitsetsum`

On a larger instance, 1000 coins with sums up to 10^6, the same DP with a byte array takes 0.26 s (and that already benefits from the compiler's vectorisation), while the bitset version takes 0.019 s, 14 times faster. Other classic uses:

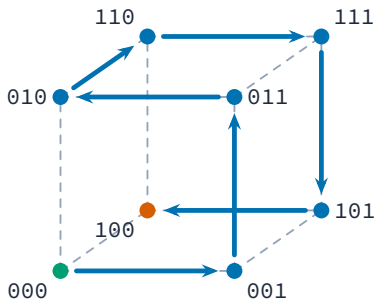
- **Transitive closure and reachability.** Store the rows of an adjacency matrix as bitsets; "everything reachable from v" is the OR of the rows of v's successors, processed in reverse topological order: $O(nm/64)$ for a DAG.
- **Counting triangles.** For each edge (u, v), the common neighbours are `(adj[u] & adj[v]).count()`: $O(mn/64)$.
- **Iterating over set bits.** GCC's `b._Find_first()` and `b._Find_next(i)` jump to the next set bit a word at a time, which makes sparse bitsets fast to traverse.

⚠ Bitset sizes are compile-time constants The size N must be known at compile time, so choose the largest size the limits allow. When the actual size varies a lot between test cases, the extra work is proportional to N, not to the actual size; a common trick is a template function `template <int N> void run()` instantiated for a few sizes ($2^{10}, 2^{12}, \ldots$) and dispatched on the input. Also remember the memory: 10^4 bitsets of 10^5 bits already take 125 MB.

10.5 Gray codes and de Bruijn sequences GOLD

10.5.1 Gray codes

A **Gray code** lists all n-bit strings so that consecutive strings differ in exactly one bit. Section 6.3 used it to update a structure with one insertion or deletion per subset. Geometrically, the n-bit strings are the corners of an n-dimensional cube, two corners are adjacent when they differ in one bit, and a Gray code is a path that visits every corner exactly once, a *Hamiltonian path* of the cube (Figure 10.7). The *reflected* Gray code $g_k = k \oplus \lfloor k/2 \rfloor$ is built recursively: list the $(n-1)$-bit code with a 0 in front, then the same list in reverse order with a 1 in front; the reflection guarantees that the two halves meet with a one-bit change.



000 → 001 → 011 → 010 → 110 → 111 → 101 → 100: every step walks along one edge of the cube (flips one bit). The start is green and the end red.

Figure 10.7: The 3-bit reflected Gray code as a Hamiltonian path on the cube, whose corners are the 3-bit strings and whose edges join strings that differ in one bit. Bit 0 runs left to right, bit 1 bottom to top, bit 2 front to back.

Example: Gray Code (CSES)

List all 2^n bit strings of length n so that consecutive strings differ in exactly one bit.

```
void solve() {
    int n;
    cin >> n;
    rep(k, 1 << n) {
        int g = k ^ (k >> 1);  // the k-th Gray code
        string s(n, '0');
        rep(b, n) if (g >> b & 1) s[n - 1 - b] = '1';
        print(s);
    }
}
```

Template: Gray code Ctrl+Alt+U, 0 prefix `cpgray`

10.5.2 De Bruijn sequences

A **de Bruijn sequence** of order n is a cyclic bit string of length 2^n that contains every n-bit string exactly once as a window; written out linearly, it needs $2^n + n - 1$ characters, which is optimal since there are 2^n windows. For $n = 3$, the string `0001011100` contains the eight windows `000`, `001`, `010`, `101`, `011`, `111`, `110`, `100`, each once.

The construction is a graph problem in disguise. Make a vertex for each $(n-1)$-bit string, and for each n-bit string $w = b_1 b_2 \dots b_n$ an edge from $b_1 \dots b_{n-1}$ to $b_2 \dots b_n$ (Figure 10.8). Consecutive windows of a string overlap in $n-1$ bits, so a string that contains every window once is exactly a walk that uses every edge once: an *Eulerian circuit*. Every vertex has two outgoing edges (append 0 or 1) and two incoming ones, so the graph is Eulerian and the circuit exists (Chapter 14 explains Eulerian circuits and Hierholzer's algorithm, which the code below uses).

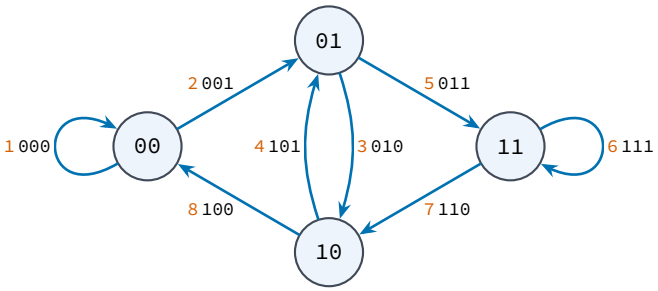


Figure 10.8: The de Bruijn graph of order 3: vertices are 2-bit strings, and each 3-bit string is an edge from its first two bits to its last two. The red numbers give an Eulerian circuit; reading the last bit of each edge after the start vertex `00` spells the de Bruijn sequence `00 01011100`.

Example: De Bruijn Sequence (CSES)

Construct a shortest bit string that contains every bit string of length $n \le 15$ as a substring.

Solution. Find an Eulerian circuit in the de Bruijn graph with Hierholzer's algorithm (iteratively, since the graph has 2^{15} edges), write down the start vertex, then the last bit of every edge in circuit order.

```
void solve() {
    int n;
    cin >> n;
    // Eulerian circuit in the graph whose vertices are the (n-1)-bit
    // strings and whose edges are the n-bit strings
    if (n == 1) {
        print("01");
        return;
    }
    int V = 1 << (n - 1);
    vector<int> used(V, 0);  // how many of the two out-edges were taken
    string out;
    vector<int> st{0};  // Hierholzer's algorithm, iterative
    vector<int> bits{0};
    while (!st.empty()) {
        int v = st.back();
        if (used[v] < 2) {
            int b = used[v]++;
            st.push_back(((v << 1) | b) & (V - 1));
            bits.push_back(b);
        } else {
            out += char('0' + bits.back());
            st.pop_back();
            bits.pop_back();
        }
    }
    out.pop_back();  // the dummy bit of the start vertex
    reverse(all(out));
    print(string(n - 1, '0') + out);
}
```

</> Template: De Bruijn sequence Ctrl+Alt+U, 9 prefix `cpdebruijn`

The template builds the sequence for any alphabet size k without a graph, by concatenating the Lyndon words whose length divides n in lexicographic order (the FKM algorithm; Lyndon words appear in Chapter 27). De Bruijn sequences answer "the shortest string containing all words of length n" (the classic example is opening a keypad lock with the fewest key presses), and they power a famous bit trick.

Finding a bit position with a de Bruijn multiplication The 32-bit constant $D =$ `0x077CB531` is a de Bruijn sequence of order 5: its 32 cyclic 5-bit windows are all different. Multiplying a power of two 2^i by D shifts D left by i, so the top five bits of the product are the i-th window of D, a unique fingerprint of i. A 32-entry table maps fingerprints back to positions:

```
int tab[32];
for (int i = 0; i < 32; i++) tab[(0x077CB531u << i) >> 27] = i;
// the index of the lowest set bit of a non-zero 32-bit x
auto ctz = [&](uint32_t x) { return tab[((x & -x) * 0x077CB531u) >> 27]; };
```

> Built-ins make this unnecessary in C++ contests, but the trick shows up in bitboard chess engines and on judges without them, and it is the nicest proof that de Bruijn sequences are more than a curiosity.

10.6 Rectangle geometry

BRONZE

Many USACO Bronze problems involve axis-parallel rectangles given by two corners, the lower-left (x_1, y_1) and the upper-right (x_2, y_2). Everything rests on one observation: two rectangles intersect in a rectangle whose projection on each axis is the intersection of their projections. On one axis, $[a_1, a_2) \cap [b_1, b_2) = [\max(a_1, b_1), \min(a_2, b_2))$, which is empty when the lower end is not below the upper end. So the intersection's width is $\max(0, \min(x_2) - \max(x_1))$ and its height $\max(0, \min(y_2) - \max(y_1))$ (Figure 10.9).

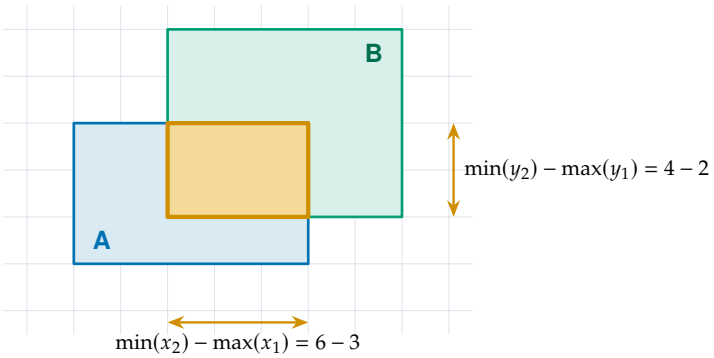


Figure 10.9: The intersection of two rectangles (orange) is a rectangle whose width and height are the overlaps of the projections; if either overlap is negative, the rectangles do not intersect and the area is 0.

Example: Blocked Billboard (USACO 2017 December Bronze)

Two billboards (which do not overlap each other) are partly hidden by a truck. Given the three rectangles, find the total visible area of the billboards.

Solution. The visible part of a billboard is its area minus its intersection with the truck, and because the billboards do not overlap, the two visible areas simply add up.

```
void solve() {
    array<ll, 4> a, b, t;  // x1 y1 x2 y2: lower-left and upper-right
    for (auto* r : {&a, &b, &t})
        for (ll& v : *r) cin >> v;
    auto area = [](const array<ll, 4>& r) {
        return (r[2] - r[0]) * (r[3] - r[1]);
    };
    auto overlap = [](const array<ll, 4>& p, const array<ll, 4>& q) {
        ll w = max(0LL, min(p[2], q[2]) - max(p[0], q[0]));
        ll h = max(0LL, min(p[3], q[3]) - max(p[1], q[1]));
        return w * h;
    };
    // the billboards do not overlap each other, so subtract separately
    print(area(a) - overlap(a, t) + area(b) - overlap(b, t));
}
```

Template: Rectangles (intersection, union of two) Ctrl+Alt+T, Shift+4 prefix `cprect`

Unions and coverings. The area of the union of two rectangles is $|A| + |B| - |A \cap B|$, and of three it is the inclusion–exclusion sum over all seven non-empty combinations, where the intersection of several rectangles is again the overlap of all their projections. Covering questions reduce to corners: in Blocked Billboard II, a tarp covering the visible part of a billboard must cover the whole billboard unless the truck covers at least two of its corners, in which case the visible part is itself a rectangle. For the union of many rectangles, compress the coordinates and use a 2D difference array (Section 9.2) when they are small, or sweep a vertical line across the x-coordinates with a segment tree that maintains the covered length of the y-axis (Chapter 22), in $O(n \log n)$.

Chapter summary

- Two's complement: $-x = \sim x + 1$, and overflow wraps (undefined for signed types). Write `1LL << k`, parenthesise comparisons, and use unsigned types for raw bit patterns.
- `x & -x` isolates the lowest set bit, `x & (x - 1)` removes it; the built-ins count and locate bits in one instruction.
- Bit b of consecutive numbers is periodic with period 2^{b+1}, which counts bits over huge ranges in closed form.
- XOR problems decompose by bits; maximum XOR queries use a binary trie, subset XOR problems a linear basis.
- Bitsets run boolean row operations 64 at a time: subset sums, reachability, triangle counting.
- Gray codes are Hamiltonian paths of the cube; de Bruijn sequences are Eulerian circuits of the de Bruijn graph.
- Rectangles intersect in the overlap of their projections; unions follow by inclusion–exclusion or sweeping.

Practice problems (CSES and USACO)

BITS

- Bit Strings
- Counting Bits
- Gray Code
- Maximum Xor Subarray
- Money Sums
- De Bruijn Sequence

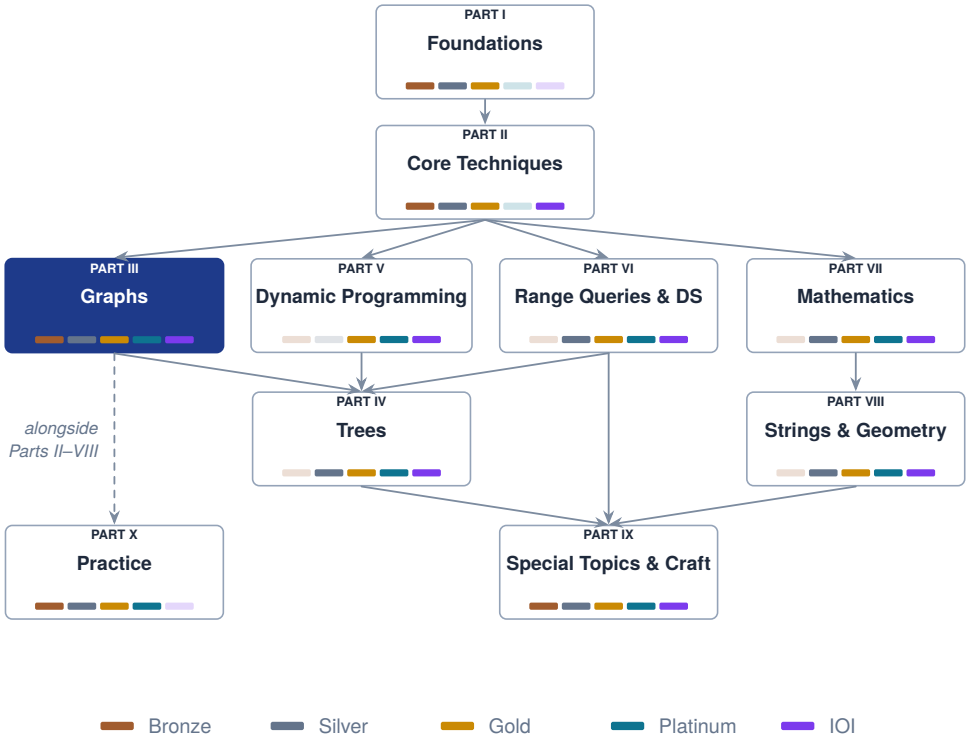
RECTANGLES (USACO BRONZE)

- Blocked Billboard (2017 December)
- Blocked Billboard II (2018 January)

PART III

Graphs

Traversals, shortest paths, spanning trees, connectivity and flows: modelling a problem as a graph and choosing the right algorithm for it.



CHAPTER 11

Graph Fundamentals

A **graph** is a set of **vertices** (also called nodes) joined by **edges**. Road maps, friendships, dependencies between tasks, the cells of a maze, the states of a puzzle and the moves between them: an astonishing number of problems become graph problems once you see them that way, and then a small toolbox of traversal algorithms answers them. This chapter builds that toolbox. It introduces the vocabulary and the ways to store a graph, then the two fundamental searches, depth-first and breadth-first, together with their direct applications: connected components, shortest paths in unweighted graphs and grids, bipartiteness, cycles, and the special structure of graphs in which every vertex has exactly one outgoing edge. It closes with two techniques for searching huge implicit graphs: bidirectional search and A*.

IN THIS CHAPTER BRONZE SILVER GOLD IOI / ADVANCED

- speak the language of graphs, and choose between adjacency lists, matrices and edge lists;
- run DFS and BFS, and know exactly what each one guarantees;
- find components, shortest paths in grids, escape routes and 2-colourings;
- detect and output cycles in undirected and directed graphs;
- analyse functional graphs, and find cycles with $O(1)$ memory;
- search huge implicit graphs with bidirectional BFS and A*.

Before you start: Chapters 3 and 5.

11.1 Vocabulary BRONZE

We write n for the number of vertices and m for the number of edges. An edge of an *undirected* graph joins two vertices symmetrically; an edge of a *directed* graph (a **arc**) goes from a tail to a head. A graph is *weighted* when every edge carries a number, its length or cost (Figure 11.1). Unless a problem says otherwise, assume a *simple* graph: no *self-loops* (edges from a vertex to itself) and no *parallel edges* (two edges joining the same pair). Contest inputs, however, often contain both, and the algorithms of this chapter handle them with at most a line of care.

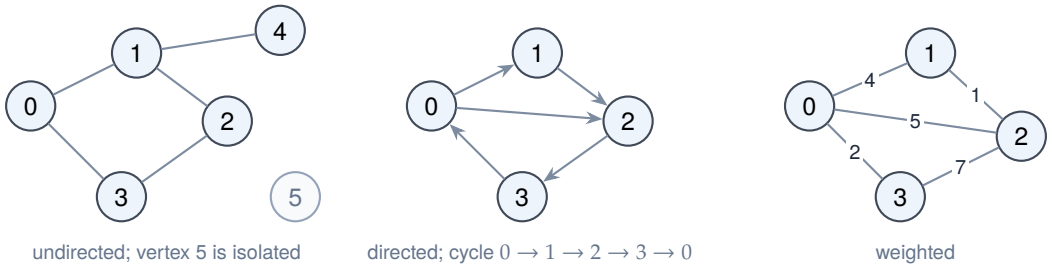


Figure 11.1: An undirected graph, a directed graph and a weighted graph.

- A **path** is a sequence of vertices in which consecutive vertices are joined by edges; it is *simple* if no vertex repeats. A **cycle** is a path of at least one edge that returns to its start (in an undirected simple graph, of length at least 3).
- An undirected graph is **connected** if every two vertices are joined by a path. Its maximal connected pieces are its **connected components**; every vertex belongs to exactly one.
- The **degree** of a vertex is the number of edges at it. Every edge adds one to the degrees of both its ends, so the degrees sum to $2m$ (the *handshake lemma*); in particular, the number of vertices of odd degree is even. In a directed graph, each vertex has an in-degree and an out-degree, and both sum to m.
- A **tree** is a connected graph without cycles. The following are equivalent for a graph with n vertices: it is a tree; it is connected with $n-1$ edges; it is acyclic with $n-1$ edges; any two vertices are joined by exactly one simple path. A **forest** is a graph whose components are trees.
- A **DAG** is a directed acyclic graph. DAGs are exactly the graphs that have a topological order (Chapter 13), and they are the natural shape of every dynamic programming (Part V).
- A graph is **bipartite** if its vertices split into two sides with every edge crossing between them (Section 11.6).

Many graphs in contests are *implicit*: the vertices are the cells of a grid, the arrangements of a puzzle, the numbers up to 10^6, and the edges are the legal moves between them. Such graphs are never stored; the search generates the neighbours of a vertex when it needs them.

11.2 Representing a graph

BRONZE

Three representations cover almost every need (Figure 11.2).

- The **adjacency list** stores, for every vertex, the list of its neighbours. It takes $O(n+m)$ memory, and visiting the edges of a vertex costs its degree, so a whole traversal costs $O(n+m)$. It is the default.
- The **adjacency matrix** is an $n \times n$ table with a 1 (or the edge weight) where an edge exists. It answers "is there an edge from u to v?" in $O(1)$ but needs $O(n^2)$ memory and $O(n)$ to list a vertex's neighbours, so it suits only small or dense graphs: Floyd–Warshall, bitmask DP, n up to a few thousand.
- The **edge list** is just the array of edges. It is what Kruskal's algorithm sorts and what Bellman–Ford relaxes, and it is the natural format when edges are processed in some order rather than explored from vertices.

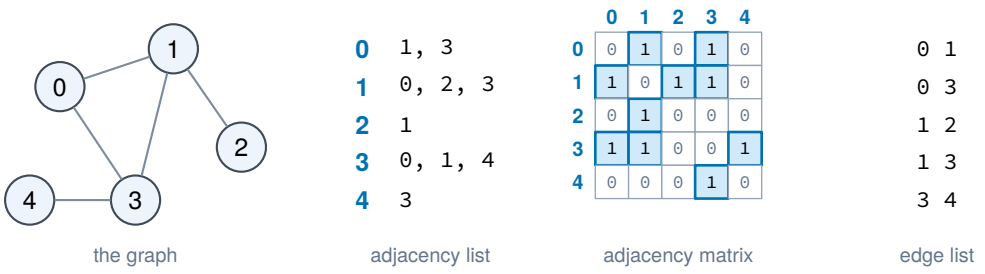


Figure 11.2: One undirected graph in the three standard representations. The adjacency list has one entry per edge end ($2m$ in total), the matrix has n^2 entries whatever the number of edges, and the edge list has m entries.

Reading a graph into adjacency lists is the first few lines of countless solutions. Most inputs number the vertices from 1; either convert to 0-based indices while reading or allocate $n + 1$ lists and ignore list 0, but do it the same way everywhere.

```
int n, m;
cin >> n >> m;
vvi adj(n + 1);  // vertices 1..n, as in the input
rep(i, m) {
    int u, v;
    cin >> u >> v;
    adj[u].push_back(v);
    adj[v].push_back(u);  // leave this line out for a directed graph
}
```

</> Template: Read graph (n, m, m edges) Ctrl+Alt+T, Shift+G prefix `cpgraph`

</> Template: Read weighted graph (n, m, edges u v w) Ctrl+Alt+T, Shift+E prefix `cpgraphw`

Flat adjacency arrays A `vector<vector<int>>` makes n separate allocations and scatters the lists across memory. For graphs with millions of edges, the *compressed sparse row* format is faster and smaller: count the degrees, turn them into starting offsets with a prefix sum, and fill one array of $2m$ neighbours, so that the neighbours of u are `nb[start[u]]` up to `nb[start[u + 1] - 1]`. It is counting sort (Section 7.2) applied to the edge list. On a random graph with 10^6 vertices and $4 \cdot 10^6$ edges, building nested vectors took 0.97 s against 0.08 s for the flat array, and a BFS took 0.10 s against 0.07 s: the many small allocations, not the traversal, are what hurts (Section 3.7).

11.3 Depth-first search

SILVER

Depth-first search (DFS) explores a graph the way you would explore a maze with a ball of string: always go deeper along an unexplored edge, and when there is none, back up to the previous vertex and try its next edge. Implemented recursively it is a few lines, visits every vertex reachable from the start exactly once, and looks at every edge at most twice, so it runs in $O(n + m)$.

Algorithm: Depth-first search

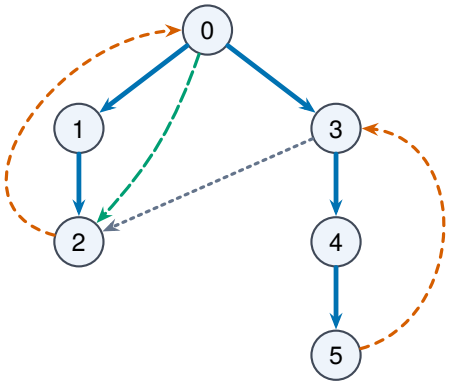
```
1 procedure DFS(u)
2     mark u as visited;  tin[u] ← timer++                ▷ discovery time
3     for all edges u → v do
4         if v is not visited then
5             parent[v] ← u;  DFS(v)                      ▷ a tree edge
6     tout[u] ← timer++                                   ▷ finishing time
```

The edges along which DFS discovers new vertices form the **DFS tree** (a forest if the graph is not connected). The *discovery* and *finishing* times nest like parentheses: the interval $[tin[v], tout[v]]$ of a descendant lies inside that of its ancestor, and the intervals of unrelated vertices are disjoint. Relative to the tree, every edge of a directed graph falls into one of four classes (Figure 11.3):

- **tree edges** discover a new vertex;
- **back edges** go to an ancestor that is still open (on the recursion stack): each one closes a cycle;

- **forward edges** go to a descendant that is already finished;
- **cross edges** go to a vertex in another, already finished branch.

In an *undirected* graph there are only tree edges and back edges: an edge to an already finished vertex in another branch would have been explored from that vertex while it was open, as a tree edge. This single fact is the basis of bridges, articulation points and cycle detection in undirected graphs (Chapter 14), while the four classes drive cycle detection, topological sorting and strongly connected components in directed graphs.



tree edge
back edge (to an open ancestor)
forward edge (to a finished descendant)
cross edge (to a finished other branch)

vertex	0	1	2	3	4	5
discovered	1	2	3	6	7	8
finished	12	5	4	11	10	9

adjacency lists 0: 1, 2, 3; 1: 2; 2: 0; 3: 2, 4; 4: 5; 5: 3

Figure 11.3: DFS from vertex 0 on a directed graph and the classification of its edges. The back edges $2 \to 0$ and $5 \to 3$ each close a cycle. The intervals from discovery to finishing time (table) nest exactly as the tree does: $[2, 5]$ and $[6, 11]$ lie inside $[1, 12]$ and are disjoint from each other.

Example: Building Roads (CSES)

A country has n cities and m roads. Add the minimum number of new roads so that every city can reach every other, and print them.

Solution. Each connected component needs to be linked to the others, and k components need exactly $k-1$ new roads (each road merges at most two components). Run a DFS from every city not yet visited: each start is a new component, and the start vertices serve as representatives. Linking consecutive representatives gives the $k-1$ roads.

```
void solve() {
    int n, m;
    cin >> n >> m;
    vvi adj(n + 1);
    rep(i, m) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    vector<bool> seen(n + 1, false);
    auto dfs = [&](auto&& self, int u) -> void {
        seen[u] = true;
        for (int v : adj[u])
            if (!seen[v]) self(self, v);
    };
    vi leader;  // one vertex from each connected component
```

```
    rep(v, 1, n + 1) if (!seen[v]) leader.push_back(v), dfs(dfs, v);
    print(sz(leader) - 1);  // a road between consecutive leaders
    rep(i, 1, sz(leader)) print(leader[i - 1], leader[i]);
}
```

</> Template: DFS (recursive, all components) Ctrl+Alt+T, D prefix `cpdfs`

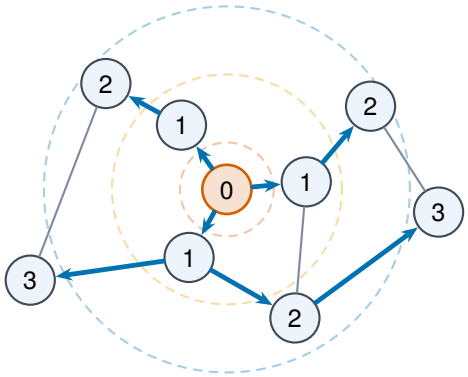
> **⚠ Recursion depth** On a path of 10^6 vertices the recursion is 10^6 calls deep, which needs tens of megabytes of stack. Codeforces and your local builds (which use a 256 MB stack) are fine, and CSES is fine too, but judges with the default 8 MB or 1 MB stack crash with a segmentation fault that looks like a wrong answer. When in doubt, use an explicit stack (as in the flood fill of Section 11.5) or process vertices in BFS order, which suffices for most tree DPs.

11.4 Breadth-first search

SILVER

Breadth-first search (BFS) explores a graph in order of distance from the start: first the start, then all its neighbours, then all their unvisited neighbours, and so on, layer by layer (Figure 11.4). A FIFO queue produces this order. BFS runs in $O(n + m)$ and, in an unweighted graph, finds the *shortest* path from the start to every vertex.

Why the distances are correct. At every moment the queue holds vertices of at most two consecutive distances, d and $d + 1$, with all the d's in front. (Initially it holds the start alone. Popping a vertex at distance d appends unvisited neighbours at distance $d + 1$ at the back, which keeps the property.) So vertices are popped in non-decreasing order of their assigned distances, and a vertex is first discovered from a vertex of the smallest possible distance: by induction, the distance assigned to each vertex is the length of a shortest path to it. Recording, for each vertex, the vertex it was discovered from gives a *BFS tree*, whose paths back to the start are shortest paths.



Vertices are labelled with their distance from the start. Blue arrows form a BFS tree: every vertex is reached from a vertex one layer closer. Grey edges join vertices of equal or adjacent layers and are never used for discovery.

Figure 11.4: BFS explores layer by layer. The queue holds at most two consecutive layers at a time, so the first discovery of a vertex is along a shortest path.

</> Template: BFS (shortest path, unweighted) Ctrl+Alt+T, B prefix `cpbfs`

Example: Labyrinth (CSES)

In an $n \times m$ labyrinth ($n, m \le 1000$) with walls, find a shortest path from `A` to `B` and print it as a string of moves `L`, `R`, `U`, `D`, or report that none exists.

Solution. BFS from `A` over the free cells. Instead of a parent cell, store the direction of the move that first reached each cell; walking these directions backwards from `B` (undoing each move) leads to `A`, and reversing the collected letters gives the path. The grid is never turned into an adjacency list: the neighbours of a cell are generated from direction arrays.

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<string> g(n);
    cin >> g;
    const int dr[4] = {-1, 1, 0, 0}, dc[4] = {0, 0, -1, 1};
    const string name = "UDLR";
    vvi from(n, vi(m, -1));  // the direction that first reached each cell
    queue<pii> q;
    rep(r, n) rep(c, m) if (g[r][c] == 'A') q.push({r, c}), from[r][c] = 4;
    while (!q.empty()) {
        auto [r, c] = q.front();
        q.pop();
        rep(d, 4) {
            int nr = r + dr[d], nc = c + dc[d];
            if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue;
            if (g[nr][nc] == '#' || from[nr][nc] != -1) continue;
            from[nr][nc] = d;
            q.push({nr, nc});
        }
    }
    int r = 0, c = 0;
    rep(i, n) rep(j, m) if (g[i][j] == 'B') r = i, c = j;
    if (from[r][c] == -1) {
        print("NO");
        return;
    }
    string path;  // walk back from B to A along the recorded directions
    while (from[r][c] != 4) {
        int d = from[r][c];
        path += name[d];
        r -= dr[d], c -= dc[d];
    }
    reverse(all(path));
    print("YES");
    print(sz(path));
    print(path);
}
```

</> Template: Grid BFS (shortest path on a grid) Ctrl+Alt+T, G prefix `cpgridbfs`

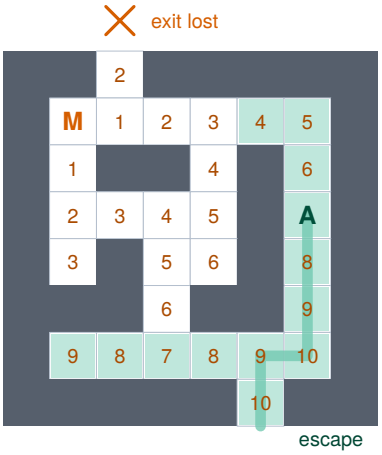
11.4.1 Several sources at once

To find, for every vertex, the distance to the *nearest* of several sources (the nearest hospital, the first moment fire reaches a cell), there is no need for one BFS per source. Put all the sources into the queue with distance 0 before the search starts: the result is the same as a BFS from an imaginary super-source joined to all of them, which is exactly the distance to the nearest source. This **multi-source BFS** costs one ordinary BFS.

Example: Monsters (CSES)

You and several monsters are in a labyrinth. Each step, you move to an adjacent free cell, and so does every monster, in the worst possible way for you. Escape through any border cell, never entering a cell that a monster can reach at the same time or earlier.

Solution. One multi-source BFS from all monsters computes the earliest time a monster can occupy each cell. A second BFS from your start then enters a cell only if you arrive strictly earlier than every monster (Figure 11.5). Any border cell that the second BFS reaches is an escape; its path is recovered from stored directions as in Labyrinth. The key insight is that the monsters need not be simulated step by step: if you can reach a cell strictly before any monster, no monster strategy can catch you there.



Numbers: the earliest time a monster can be in each cell (multi-source BFS from M). Green: the cells you reach strictly earlier. The top exit is reached by a monster at time 2, long before you could get there; the bottom exit is safe, reached at time 5 against the monster's 10.

Figure 11.5: Monsters as two BFS runs. You may enter a cell only if you get there strictly before any monster; the escape path (thick green line) goes through such cells to the bottom border.

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<string> g(n);
    cin >> g;
    const int dr[4] = {-1, 1, 0, 0}, dc[4] = {0, 0, -1, 1};
    const string name = "UDLR";
    const int UNSEEN = 1e9;  // larger than any distance
    // multi-source BFS from all monsters: when can a monster reach a cell?
    vvi mon(n, vi(m, UNSEEN));
    queue<pii> q;
    rep(r, n) rep(c, m) if (g[r][c] == 'M') mon[r][c] = 0, q.push({r, c});
    auto bfs = [&](vvi& dist, vvi* from) {
        while (!q.empty()) {
```

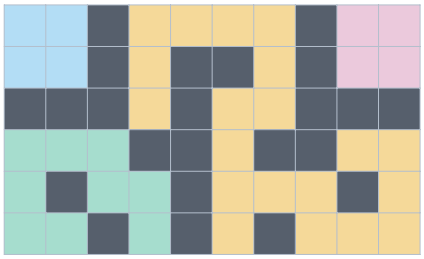
```
            auto [r, c] = q.front();
            q.pop();
            rep(d, 4) {
                int nr = r + dr[d], nc = c + dc[d];
                if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue;
                if (g[nr][nc] == '#' || dist[nr][nc] != UNSEEN) continue;
                int t = dist[r][c] + 1;
                if (from && t >= mon[nr][nc]) continue;  // monster is first
                dist[nr][nc] = t;
                if (from) (*from)[nr][nc] = d;
                q.push({nr, nc});
            }
        }
    };
    bfs(mon, nullptr);
    vvi me(n, vi(m, UNSEEN)), from(n, vi(m, -1));
    rep(r, n) rep(c, m) if (g[r][c] == 'A') me[r][c] = 0, q.push({r, c});
    bfs(me, &from);
    rep(r, n) rep(c, m) {
        bool border = r == 0 || r == n - 1 || c == 0 || c == m - 1;
        if (!border || me[r][c] == UNSEEN) continue;
        string path;  // reachable border cell: walk back to the start
        for (int y = r, x = c; me[y][x] > 0; ) {
            int d = from[y][x];
            path += name[d];
            y -= dr[d], x -= dc[d];
        }
        reverse(all(path));
        print("YES");
        print(sz(path));
        print(path);
        return;
    }
    print("NO");
}
```

BFS with 0–1 weights, and BFS on states If every edge has weight 0 or 1, a *deque* replaces the queue: push the endpoint of a 0-edge to the front and of a 1-edge to the back. This **0–1 BFS** keeps the two-layers invariant and runs in $O(n + m)$ (Chapter 12). And when the rules of movement depend on more than the position (keys collected, direction faced, remaining jumps), BFS runs unchanged on the graph whose vertices are *states* (position, extra information); the only question is whether the number of states fits in time and memory.

11.5 Grids and flood fill

SILVER

A grid is a graph whose vertices are cells and whose edges join neighbouring cells, usually the four orthogonal neighbours. **Flood fill** labels each connected region: start a DFS or BFS from any unlabelled free cell and label everything it reaches (Figure 11.6). Every cell is visited once, so labelling the whole grid takes $O(nm)$.



Four rooms, each found by one search started at its first free cell in reading order. The big orange room winds through gaps in the walls; the search follows it regardless of its shape.

Figure 11.6: Flood fill on a grid: dark cells are walls, and each colour is one connected region of free cells.

Example: Counting Rooms (CSES)

Count the rooms (connected regions of floor cells) in an $n \times m$ map, $n, m \le 1000$.

Solution. Scan the cells; every unvisited floor cell starts a new room, which is then flooded. The code floods with an explicit stack, marking each cell when it is pushed, so that no cell enters the stack twice and the recursion depth is no concern, even for a 1000×1000 spiral.

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<string> g(n);
    cin >> g;
    const int dr[4] = {-1, 1, 0, 0}, dc[4] = {0, 0, -1, 1};
    int rooms = 0;
    rep(sr, n) rep(sc, m) {
        if (g[sr][sc] != '.') continue;
        rooms++;  // a new room: flood it with an explicit stack
        vector<pii> st{{sr, sc}};
        g[sr][sc] = '#';  // mark cells when they are pushed
        while (!st.empty()) {
            auto [r, c] = st.back();
            st.pop_back();
            rep(d, 4) {
                int nr = r + dr[d], nc = c + dc[d];
                if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue;
                if (g[nr][nc] != '.') continue;
                g[nr][nc] = '#';
                st.push_back({nr, nc});
            }
        }
    }
    print(rooms);
}
```

Template: Flood fill (grid regions) Ctrl+Alt+T, F prefix `cpflood`

Grid habits Write the bounds check once, as the first line of the neighbour loop. For eight directions, use

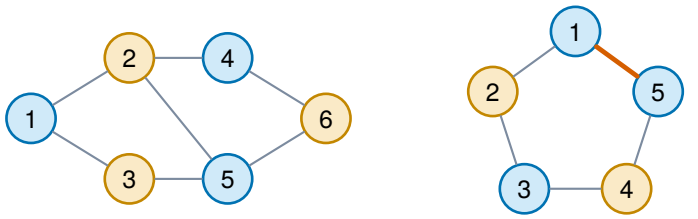
```
const int dr[8] = {-1, -1, -1, 0, 0, 1, 1, 1};
const int dc[8] = {-1, 0, 1, -1, 1, -1, 0, 1};
```

Encode a cell as the single number $r \cdot m + c$ when a data structure (a DSU, a visited array, a hash set) wants one index. And remember that the *perimeter* of a region is the number of (cell, direction) pairs whose neighbour is outside the region, which a flood fill can count as it goes: that is the whole of Icy Perimeter (USACO 2019 January Silver).

11.6 Bipartite graphs and 2-colouring

SILVER

A graph is **bipartite** if its vertices can be coloured with two colours so that every edge joins two different colours. The characterisation is clean: *a graph is bipartite if and only if it has no cycle of odd length*. One direction is immediate: going around a cycle, the colours must alternate, so the cycle has even length. For the other, run BFS in every component and colour each vertex by the parity of its distance. An edge between two vertices of the same colour joins two vertices whose distances have the same parity (the distances of adjacent vertices differ by at most one, so they are equal); together with the two BFS paths back to their lowest common ancestor in the BFS tree, that edge closes a cycle of odd length (Figure 11.7). So either the parity colouring works, or the graph has an odd cycle.



(a) BFS layers alternate colours; every edge crosses: bipartite

(b) an odd cycle: the red edge joins two vertices of the same colour

Figure 11.7: Two-colouring by BFS parity. In (a) the colouring succeeds. In (b), a 5-cycle, some edge must join equal colours: graphs with an odd cycle are exactly the graphs that are not bipartite.

Example: Building Teams (CSES)

Divide n pupils into two teams so that no two friends are in the same team, or report that it is impossible.

Solution. BFS from every uncoloured pupil, giving each newly discovered pupil the colour opposite to the one it was discovered from. If an edge ever joins two pupils of the same colour, there is an odd cycle and no division exists.

```
void solve() {
    int n, m;
    cin >> n >> m;
    vvi adj(n + 1);
    rep(i, m) {
        int a, b;
        cin >> a >> b;
```

```
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    vi team(n + 1, 0);  // 0 = not coloured yet, otherwise 1 or 2
    rep(s, 1, n + 1) {
        if (team[s]) continue;
        team[s] = 1;
        queue<int> q;
        q.push(s);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int v : adj[u]) {
                if (!team[v]) team[v] = 3 - team[u], q.push(v);
                else if (team[v] == team[u]) {  // an odd cycle
                    print("IMPOSSIBLE");
                    return;
                }
            }
        }
    }
    print(vi(team.begin() + 1, team.end()));
}
```

</> Template: Bipartite check (2-coloring) Ctrl+Alt+T, 2 prefix `cpbipartite`

How to recognize it "Split into two groups such that each pair of rivals is separated", "two teams", "can the map be 2-coloured", and checkerboard arguments on grids (a grid graph is always bipartite: colour by $(r + c)$ mod 2). Constraints of the form "x and y in the *same* group" mixed with "different groups" are handled by a DSU with parity (Chapter 13), and arbitrary boolean constraints on pairs by 2-SAT (Chapter 14). Bipartite graphs are also where matching theory lives (Chapter 15).

11.7 Finding cycles

GOLD

Undirected graphs. In a DFS of an undirected graph, every non-tree edge goes to an ancestor (Section 11.3), so a cycle exists exactly when some vertex u has an edge to an already visited vertex v other than its parent. The cycle is v, the tree path down to u, and the edge back. Two details matter in practice. Parallel edges to the parent must not count (they would make a 2-cycle), which is why the code skips the parent vertex rather than the parent edge. And an edge from u to a *child* that was already explored can reappear through a parallel edge; comparing depths distinguishes it from a real back edge, which goes *up*.

Example: Round Trip (CSES)

Find a round trip, a route that visits at least three distinct cities and returns to its start without repeating a city, or report that none exists.

```
void solve() {
    int n, m;
    cin >> n >> m;
    vvi adj(n + 1);
    rep(i, m) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    vi par(n + 1, 0), depth(n + 1, -1);  // depth -1: not visited yet
    vi cycle;
    // returns true once a cycle has been found and stored in cycle
    auto dfs = [&](auto&& self, int u) -> bool {
        for (int v : adj[u]) {
            if (v == par[u]) continue;  // back along the tree edge
            if (depth[v] == -1) {
                par[v] = u, depth[v] = depth[u] + 1;
                if (self(self, v)) return true;
            } else if (depth[v] < depth[u]) {  // an edge up to an ancestor
                for (int x = u; x != v; x = par[x]) cycle.push_back(x);
                cycle.push_back(v);
                cycle.push_back(u);
                return true;
            }  // otherwise a child reached again through a parallel edge
        }
        return false;
    };
    rep(s, 1, n + 1) {
        if (depth[s] != -1) continue;
        depth[s] = 0;
        if (!dfs(dfs, s)) continue;
        print(sz(cycle));
        print(cycle);
        return;
    }
    print("IMPOSSIBLE");
}
```

When only the *existence* of a cycle matters, a DSU is even simpler: add the edges one by one, and the first edge whose endpoints are already connected closes a cycle (Chapter 13).

Directed graphs. In a directed graph, an edge to a visited vertex does not imply a cycle: cross and forward edges reach finished vertices without closing anything. What closes a cycle is an edge to a vertex that is still *open*, on the current recursion path. So DFS keeps three states: white (not visited), grey (open, on the stack) and black (finished). An edge into a grey vertex is a back edge, and the cycle is the stack segment from that vertex down to the current one (Figure 11.8). Conversely, if DFS finds no back edge, the finishing order is a topological order, so the graph is acyclic: a directed graph has a cycle if and only if DFS finds a back edge.

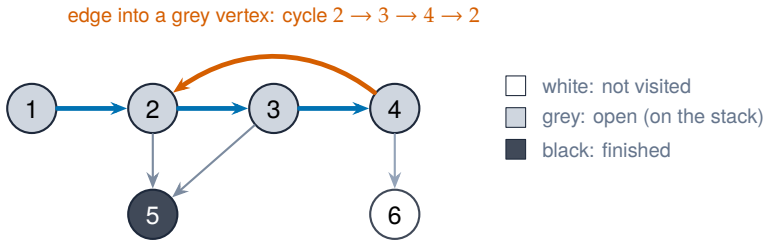


Figure 11.8: Three colours in a directed DFS. The search is at vertex 4 with the path $1 \to 2 \to 3 \to 4$ open. The edge $3 \to 5$ led to a finished vertex (no cycle there), but the edge $4 \to 2$ enters an open vertex and closes a cycle.

Example: Round Trip II (CSES)

Find a directed cycle in a graph of $n \le 10^5$ cities and $m \le 2 \cdot 10^5$ one-way flights, and print it.

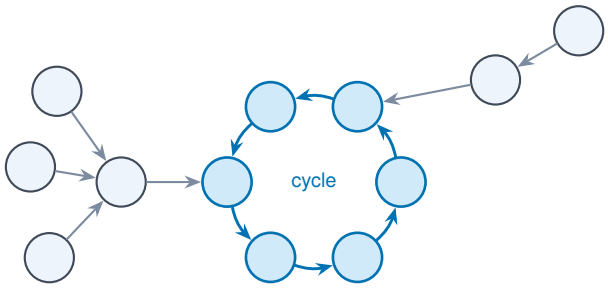
```
void solve() {
    int n, m;
    cin >> n >> m;
    vvi adj(n + 1);
    rep(i, m) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
    }
    vi color(n + 1, 0), par(n + 1, -1);  // 0 white, 1 grey (on stack), 2 black
    vi cycle;
    auto dfs = [&](auto&& self, int u) -> bool {
        color[u] = 1;
        for (int v : adj[u]) {
            if (color[v] == 1) {  // an edge back into the current path
                cycle.push_back(v);
                for (int x = u; x != v; x = par[x]) cycle.push_back(x);
                cycle.push_back(v);
                reverse(all(cycle));
                return true;
            }
            if (color[v] == 0) {
                par[v] = u;
                if (self(self, v)) return true;
            }
        }
        color[u] = 2;  // finished: no cycle goes through u any more
        return false;
    };
    rep(s, 1, n + 1) {
        if (color[s] || !dfs(dfs, s)) continue;
        print(sz(cycle));
        print(cycle);
        return;
    }
    print("IMPOSSIBLE");
}
```

Template: Find a directed cycle Ctrl+Alt+T, / prefix `cpcycle`

11.8 Functional graphs

GOLD

In a **functional graph** (or successor graph) every vertex has exactly one outgoing edge, $v \to f(v)$. They arise from permutations, from "each cow points to its best friend" and from any deterministic process, where f maps a state to the next one (Section 6.1). Their structure is rigid. Follow the edges from any vertex: since there are finitely many vertices, the walk eventually repeats a vertex and from then on goes around a cycle forever. So every component consists of exactly one cycle with trees hanging into it, their edges pointing towards the cycle (Figure 11.9).



Blue: the cycle of the component. Grey: trees hanging into it, all edges pointing towards the cycle. From any vertex, the walk reaches the cycle after some number of steps (the tail) and then repeats with the cycle's length.

Figure 11.9: A component of a functional graph: exactly one cycle, with in-trees attached to its vertices.

For every vertex, two numbers describe its future completely: the length of its tail (steps until it reaches a cycle) and the length of that cycle. Both can be computed for all vertices in $O(n)$ by walking from each unvisited vertex until the walk hits a vertex that is either on the current walk (a new cycle has been found) or already solved (then the tail lengths follow by counting back); Planets Cycles in Chapter 31 does exactly this. "Where is v after k steps?" for many v and huge k is answered by binary lifting in $O(\log k)$ each.

Template: Functional graph (successor graph) cycles Ctrl+Alt+T, , prefix `cpfuncgraph`

Template: K-th successor (binary lifting / jumping) Ctrl+Alt+T, . prefix `cpjump`

11.8.1 Cycle finding with constant memory

Sometimes f is given by a formula on a huge domain ($f(x) = x^2 + c \bmod N$ in Pollard's rho, the next state of a large simulation) and the visited vertices cannot be stored. **Floyd's cycle-finding algorithm** runs two walkers from x_0, the tortoise one step at a time and the hare two steps at a time. Once both are on the cycle, the hare gains one step per round on the tortoise, so they meet within $\mu + \lambda$ rounds, where μ is the tail length and λ the cycle length. **Brent's algorithm** is the practical refinement: the hare walks alone while the tortoise waits at the positions $1, 2, 4, 8, \ldots$; when the hare meets the waiting tortoise, the number of steps since the last teleport is exactly λ. Then two walkers started λ steps apart from x_0 meet precisely at the cycle's entry, after μ steps. Brent uses about a third fewer evaluations of f than Floyd, and both use $O(1)$ memory.

```
template <class T, class F>
pair<ll, ll> find_rho(T x0, F f) {  // returns (mu, lambda)
    ll power = 1, lam = 1;
    T tortoise = x0, hare = f(x0);
    while (!(tortoise == hare)) {  // the tortoise waits at 1, 2, 4, ...
        if (power == lam) tortoise = hare, power *= 2, lam = 0;
```

```
        hare = f(hare), lam++;
    }
    T a = x0, b = x0;  // b starts lambda steps ahead: they meet at the entry
    for (ll i = 0; i < lam; i++) b = f(b);
    ll mu = 0;
    while (!(a == b)) a = f(a), b = f(b), mu++;
    return {mu, lam};
}
```

Template: Cycle finding in O(1) memory (Floyd / Brent) Ctrl+Alt+U, F prefix `cpcyclefind`

11.9 Bidirectional search and A*

IOI / ADVANCED

In an implicit graph with branching factor b, a BFS to depth d explores about b^d states. Two ideas cut this down, one by halving the exponent and one by steering the search.

Bidirectional BFS. When the target is known, search from both ends at once, expanding one whole layer at a time from whichever side has the smaller frontier, and stop as soon as the two searches touch. Each side then only needs depth about $d/2$, so the work drops from about b^d to about $2b^{d/2}$ (Figure 11.10). This is meet in the middle (Section 6.6) for shortest paths. The first contact found while expanding a layer gives a shortest path: before that layer, no contact means the distance exceeds the sum of the two explored depths, and every contact found during the layer has exactly that length plus one.

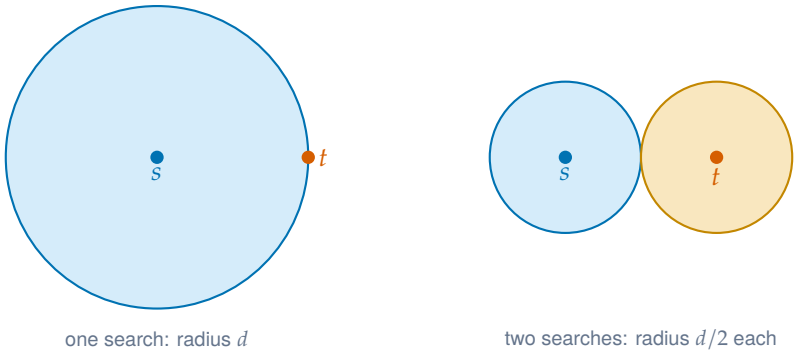


Figure 11.10: Why bidirectional search wins: in a graph where the number of states within distance r grows like b^r, two balls of radius $d/2$ contain far fewer states than one ball of radius d.

Example: Swap Game (CSES)

A 3×3 grid holds the numbers $1, \ldots, 9$. One move swaps two adjacent numbers. Find the minimum number of moves that sorts the grid.

Solution. The states are the $9! = 362\,880$ permutations and each has 12 neighbours, so even a plain BFS fits. Bidirectional BFS, from the start and from the sorted grid, explores far fewer states whenever the answer is not close to the maximum (Figure 11.11). Each state is encoded as the nine digits read row by row, a number below 10^9.

```cpp
void solve() {
    // a state is the 3 x 3 grid read row by row as a 9-digit number
    auto encode = [](const array<int, 9>& a) {
        int x = 0;
        for (int v : a) x = x * 10 + v;
        return x;
    };
    auto decode = [](int x) {
        array<int, 9> a;
        per(i, 9) a[i] = x % 10, x /= 10;
        return a;
    };
    array<int, 9> start;
    for (int& v : start) cin >> v;
    int s = encode(start), t = 123456789;
    vector<pii> swaps;  // the 12 pairs of adjacent cells
    rep(i, 9) {
        if (i % 3 < 2) swaps.push_back({i, i + 1});
        if (i < 6) swaps.push_back({i, i + 3});
    }
    // bidirectional BFS: grow the smaller frontier until they touch
    unordered_map<int, int> dist[2];
    vi frontier[2] = {{s}, {t}};
    dist[0][s] = 0, dist[1][t] = 0;
    if (s == t) {
        print(0);
        return;
    }
    while (true) {
        int side = sz(frontier[0]) <= sz(frontier[1]) ? 0 : 1;
        vi next;
        for (int x : frontier[side]) {
            array<int, 9> a = decode(x);
            int d = dist[side][x];
            for (auto [i, j] : swaps) {
                swap(a[i], a[j]);
                int y = encode(a);
                swap(a[i], a[j]);
                if (dist[side].count(y)) continue;
                auto it = dist[side ^ 1].find(y);
                if (it != dist[side ^ 1].end()) {  // the two searches meet
                    print(d + 1 + it->second);
                    return;
                }
                dist[side][y] = d + 1;
                next.push_back(y);
            }
        }
        frontier[side] = next;
    }
}
```

</> Template: Bidirectional BFS (meet in the middle on shortest paths) Ctrl+Alt+U, 2 prefix `cpbibfs`

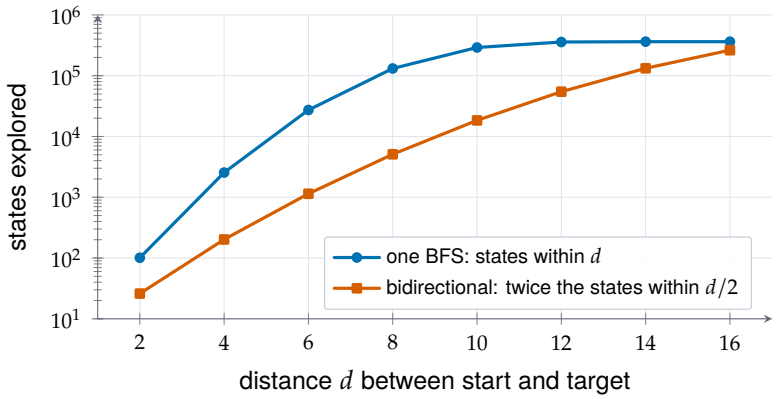


Figure 11.11: Swap Game: the number of states that each search explores in the worst case, computed from the exact distance distribution of all 9! states. At distance 10 bidirectional search explores 16 times fewer states; near the maximum distance 16 the gain shrinks, because a plain BFS cannot explore more than the 9! states that exist.

A*. When a good estimate of the remaining distance is available, a search can be *steered* towards the target. A* is Dijkstra's algorithm (Chapter 12) with priority $g(v) + h(v)$, where $g(v)$ is the distance found so far and $h(v)$ is a lower bound on the remaining distance to the target, such as the Manhattan distance on a grid. If h never overestimates (it is *admissible*), the first time the target is taken from the priority queue its distance is optimal. If moreover $h(u) \le w(u,v) + h(v)$ for every edge (it is *consistent*, as the Manhattan distance is on a grid), no vertex is ever expanded twice, and A* is exactly Dijkstra on the edge weights $w(u,v) - h(u) + h(v) \ge 0$. With $h = 0$ it is plain Dijkstra; the better the bound, the fewer vertices are expanded (Figure 11.12).

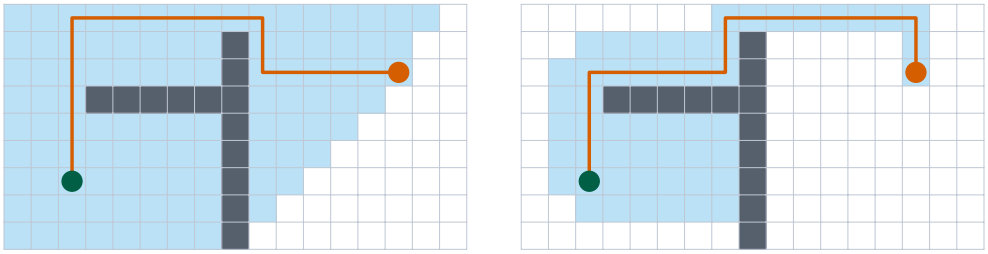


Figure 11.12: BFS versus A* from the green start to the red target around a wall (dark). Blue cells were expanded before the target; both searches find a shortest path of length 20 (red), but A* with the Manhattan distance as its bound expands half as many cells.

A* is the method of choice for puzzles with a good lower bound (the sum of Manhattan distances of the tiles in the 15-puzzle), and its memory-light variant IDA*, iterative deepening on the bound $g + h$, solves the 15-puzzle within the limits of an ordinary contest problem. In typical contest graphs without such a bound, bidirectional BFS or a smarter state space are usually the better investment.

Chapter summary

- Store graphs as adjacency lists by default; matrices for small dense graphs, edge lists for edge-by-edge algorithms.
- DFS runs in $O(n + m)$; its tree and discovery times classify edges into tree, back, forward and cross edges. In undirected graphs only tree and back edges exist.

- BFS finds shortest paths in unweighted graphs; several sources go into the queue at once; directions or parents recover the path.
- A graph is bipartite if and only if it has no odd cycle; BFS parity finds the colouring or the cycle.
- Undirected cycles are back edges to non-parent ancestors; directed cycles are edges into grey vertices.
- Every component of a functional graph is one cycle with in-trees; Floyd and Brent find tails and cycles in $O(1)$ memory.
- Bidirectional BFS turns b^d into $2b^{d/2}$; A* steers Dijkstra with an admissible lower bound.

Practice problems (CSES Problem Set)

TRAVERSALS

- Counting Rooms
- Labyrinth
- Building Roads
- Message Route
- Building Teams
- Monsters

CYCLES, FUNCTIONAL GRAPHS AND SEARCHES

- Round Trip
- Round Trip II
- Planets Cycles
- Planets Queries I
- Swap Game
- Planets Queries II

CHAPTER 12

Shortest Paths

Finding the cheapest way from one place to another is the most common graph question in contests, and it appears in many disguises: the fewest moves in a puzzle, the cheapest schedule, the latest start time consistent with a set of deadlines. There is no single best algorithm. The right one depends on the edge weights (all equal, zero or one, non-negative, arbitrary), on whether the graph has cycles, and on whether distances are needed from one source or between all pairs. This chapter develops each algorithm from the single operation they share, relaxation, proves why it works, and shows the modelling tricks that make it apply: states that remember extra information, reversed graphs, negated weights and systems of inequalities.

IN THIS CHAPTER SILVER GOLD PLATINUM

- choose the right shortest-path algorithm from the weights and the question;
- run Dijkstra's algorithm, prove it, and extend it to states, counts and path information;
- handle 0–1 weights with a deque, and negative weights with Bellman–Ford;
- find and print negative cycles, and detect unbounded longest paths;
- solve systems of difference constraints as shortest-path problems;
- compute all-pairs distances with Floyd–Warshall, and shortest and longest paths in DAGs.

Before you start: Chapter 11 (BFS and DFS) and Section 5.4 (heaps).

12.1 Choosing the algorithm SILVER

Figure 12.1 is the decision procedure, and Table 12.1 the reference. Two questions settle almost every case: what the weights look like, and whether the graph is a DAG.

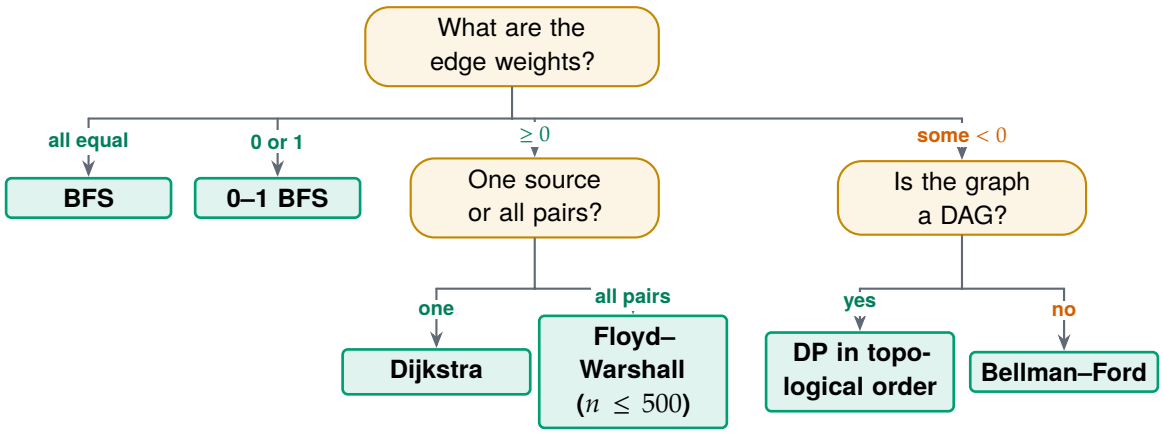


Figure 12.1: Which shortest-path algorithm? A DAG allows any weights (and even longest paths) with a simple DP; negative weights in a graph with cycles need Bellman–Ford, or Floyd–Warshall for all pairs.

Weights and question	Algorithm	Time	Remarks
all equal	BFS	$O(n+m)$	Chapter 11
0 or 1	0–1 BFS	$O(n+m)$	deque instead of queue
non-negative, one source	Dijkstra	$O((n+m)\log n)$	the workhorse
any, one source	Bellman–Ford	$O(nm)$	detects negative cycles
any, all pairs	Floyd–Warshall	$O(n^3)$	three lines; $n \le 500$
any, DAG	DP in topological order	$O(n+m)$	longest paths too
non-negative, all pairs, sparse	$n\times$ Dijkstra	$O(n(n+m)\log n)$	Johnson if some are negative

Table 12.1: Shortest-path algorithms at a glance.

12.2 Relaxation and shortest-path trees

SILVER

Every algorithm in this chapter maintains tentative distances $d(v)$, upper bounds on the true distances $\delta(v)$, with $d(s) = 0$ and $d(v) = \infty$ elsewhere at the start. The only way they ever change is by **relaxation**: for an edge $u \to v$ of weight w,

$$\text{if } d(u) + w < d(v) \text{ then } d(v) \leftarrow d(u) + w, \quad par(v) \leftarrow u.$$

Relaxation keeps every $d(v)$ an upper bound, because $d(u) + w$ is the length of a real walk to v. At the end, when no edge can be relaxed, every edge satisfies the *triangle inequality* $d(v) \le d(u) + w$, and the distances are exact: along any shortest path $s = v_0, v_1, \ldots, v_k$, induction gives $d(v_i) \le \delta(v_i)$. The algorithms differ only in the *order* in which they relax edges, and that order is what they need to be both correct and fast.

The parent pointers form a **shortest-path tree**: following *par* from any vertex back to s spells out a shortest path, in reverse. So storing one parent per vertex is all it takes to print paths, and it works because shortest paths have *optimal substructure*: every prefix of a shortest path is itself a shortest path.

12.3 Dijkstra's algorithm

GOLD

Dijkstra's algorithm handles non-negative weights. It repeatedly takes the vertex with the smallest tentative distance that is not yet *final*, declares it final, and relaxes its outgoing edges (Figure 12.2). A priority queue of (distance, vertex) pairs finds that vertex; instead of decreasing keys, the implementation pushes a new pair whenever a distance improves and skips popped pairs whose distance is out of date (*lazy deletion*).

Algorithm: Dijkstra

```
d(s) ← 0, d(v) ← ∞ for v ≠ s; push (0, s)
while the priority queue is not empty do
    pop the pair (x, u) with the smallest x
    if x ≠ d(u) then continue                    ▷ an outdated pair
    for all edges u → v of weight w do
        if d(u) + w < d(v) then
            d(v) ← d(u) + w; push (d(v), v)
```

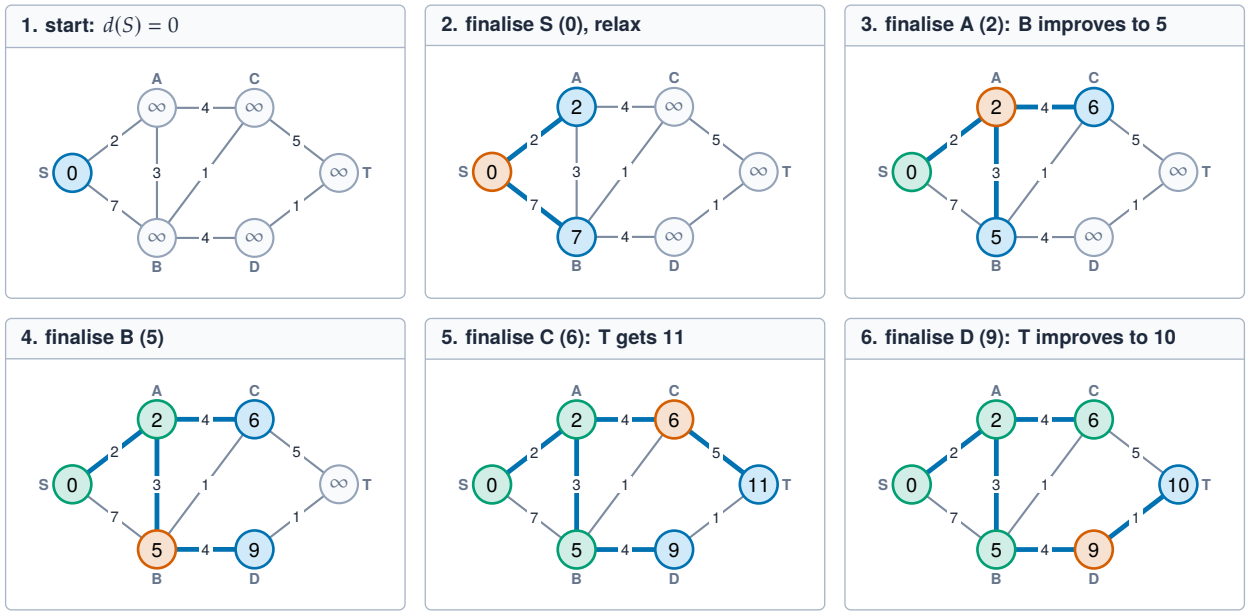


Figure 12.2: Dijkstra's algorithm from S. Green vertices are final, red is the vertex being finalised, blue ones are in the queue with a tentative distance, and thick blue edges are the current parent pointers. The last step finalises T with distance 10, and the parent pointers form the shortest-path tree.

Why it is correct. Claim: when u is popped with $x = d(u)$, $d(u) = \delta(u)$. Suppose not, and let u be the first vertex finalised with $d(u) > \delta(u)$. Take a shortest path from s to u, and let y be its first vertex that is not final, with predecessor x' on the path (which is final, hence correct). When x' was finalised, the edge $x' \to y$ was relaxed, so $d(y) = \delta(y)$. Since weights are non-negative and y lies on the path to u, $\delta(y) \le \delta(u) < d(u)$, so $d(y) < d(u)$, and the queue would have popped y before u: a contradiction. This is exactly where non-negativity is used, and why a single negative edge can break the algorithm.

Complexity. Every edge pushes at most one pair, so the queue sees at most $m + 1$ pushes and pops: $O((n + m) \log m) = O((n + m) \log n)$. On dense graphs ($m \approx n^2$) the $O(n^2)$ version without a heap, which scans all vertices for the minimum, is faster.

```
vector<ll> dist(n, LLONG_MAX);
minpq<pll> pq;  // (distance, vertex)
dist[src] = 0;
pq.push({0, src});
while (!pq.empty()) {
    auto [d, u] = pq.top();
    pq.pop();
    if (d != dist[u]) continue;  // an outdated entry ("lazy deletion")
    for (auto [v, w] : adj[u])
        if (d + w < dist[v]) dist[v] = d + w, pq.push({dist[v], v});
}
```

</> Template: Dijkstra Ctrl+Alt+T, J prefix `cpdijkstra`

The complete program for the basic problem is the worked problem Shortest Routes I (Chapter 31). The two problems below show the two most important extensions: richer states, and information gathered along the shortest paths.

12.3.1 Dijkstra on states

Example: Flight Discount (CSES)

Find the cheapest route from city 1 to city n ($n \le 10^5$, $m \le 2 \cdot 10^5$ one-way flights) when exactly one flight may be taken at half price (rounded down).

Solution. The cost of the rest of the trip depends on the city *and* on whether the coupon is still available. So the search runs on the graph of states (v, used), which has $2n$ vertices: a flight $u \to v$ of price w becomes the edges $(u,0) \to (v,0)$ and $(u,1) \to (v,1)$ of weight w, plus the coupon edge $(u,0) \to (v,1)$ of weight $\lfloor w/2 \rfloor$. One Dijkstra on this layered graph answers the question.

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<vector<pll>> adj(n + 1);
    rep(i, m) {
        ll a, b, c;
        cin >> a >> b >> c;
        adj[a].push_back({b, c});
    }
    // state (v, used): at city v, the coupon already used (1) or not (0)
    vector<array<ll, 2>> dist(n + 1, {INF, INF});
    minpq<tuple<ll, int, int>> pq;  // (distance, city, used)
    dist[1][0] = 0;
    pq.push({0, 1, 0});
    while (!pq.empty()) {
        auto [d, u, used] = pq.top();
        pq.pop();
        if (d != dist[u][used]) continue;  // an outdated entry
        for (auto [v, w] : adj[u]) {
            if (d + w < dist[v][used]) {  // pay the full price
                dist[v][used] = d + w;
                pq.push({dist[v][used], v, used});
            }
            if (!used && d + w / 2 < dist[v][1]) {  // use the coupon now
                dist[v][1] = d + w / 2;
                pq.push({dist[v][1], v, 1});
            }
        }
    }
    print(min(dist[n][0], dist[n][1]));
}
```

Layered graphs "At most k free edges", "at most k stops", "the parity of the number of steps", "which key you carry": whenever the future depends on a small amount of extra information, make it part of the vertex. The graph grows by that factor ($k+1$ layers, 2 parities, 2^{keys} key sets), and the algorithm stays the same. The alternative for Flight Discount, which avoids the layered graph, is also worth knowing: run Dijkstra from 1 and, on the reversed graph, from n; the answer is the minimum over all flights $u \to v$ of $d_1(u) + \lfloor w/2 \rfloor + d_n(v)$.

12.3.2 Counting and comparing shortest paths

Example: Investigation (CSES)

For the cheapest routes from city 1 to city n, find their price, how many there are (modulo $10^9 + 7$), and the fewest and the most flights among them. All prices are positive.

Solution. Dijkstra finalises vertices in order of distance, and with positive weights every predecessor of v on a cheapest route is finalised before v. So when a vertex u is finalised its counts are complete and can be pushed along its edges: a strictly cheaper route to v overwrites v's information, an equally cheap one merges into it.

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<vector<pll>> adj(n + 1);
    rep(i, m) {
        ll a, b, c;
        cin >> a >> b >> c;
        adj[a].push_back({b, c});
    }
    vector<ll> dist(n + 1, LLONG_MAX), ways(n + 1, 0);  // ways mod MOD
    vi fewest(n + 1, 0), most(n + 1, 0);  // flights on the cheapest routes
    minpq<pll> pq;
    dist[1] = 0, ways[1] = 1;
    pq.push({0, 1});
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (d != dist[u]) continue;
        // u is final: its counts are complete, so push them forward
        for (auto [v, w] : adj[u]) {
            if (d + w < dist[v]) {  // a strictly cheaper route: start over
                dist[v] = d + w;
                ways[v] = ways[u];
                fewest[v] = fewest[u] + 1, most[v] = most[u] + 1;
                pq.push({dist[v], v});
            } else if (d + w == dist[v]) {  // another cheapest route
                ways[v] = (ways[v] + ways[u]) % MOD;
                fewest[v] = min(fewest[v], fewest[u] + 1);
                most[v] = max(most[v], most[u] + 1);
            }
        }
    }
    print(dist[n], ways[n], fewest[n], most[n]);
}
```

More Dijkstra variations

- **Which edges lie on some shortest path?** Run Dijkstra from s and, on the reversed graph, from t; the edge $u \to v$ of weight w lies on a shortest path exactly when $d_s(u) + w + d_t(v) = d_s(t)$.

- **Minimax paths** (minimise the largest edge on the path): replace $d(u)+w$ by $\max(d(u), w)$; the same proof works. Kruskal gives the same answers for all pairs at once (Chapter 13).
- **The k shortest walks**: let every vertex be popped up to k times, keeping a counter per vertex; the k-th pop of t is the k-th shortest walk (CSES Flight Routes).
- **Multiple sources**: push them all with distance 0, exactly as in multi-source BFS.
- **Distances to one target**: run Dijkstra from the target on the reversed graph.

The classic Dijkstra bugs Forgetting the `if (d != dist[u]) continue;` line keeps the answer correct but can make the algorithm quadratic. Declaring `priority_queue<pll>` without `greater` (or `minpq`) turns it into a max-heap, which silently explores in the wrong order. Using `int` distances overflows with 10^5 edges of weight 10^9. And with a negative edge, the algorithm is simply wrong, even when there is no negative cycle.

12.4 0–1 BFS

GOLD

When every weight is 0 or 1, Dijkstra's priority queue can be replaced by a deque. Relaxing along a 0-edge produces a vertex with the *same* distance as the current one, so it goes to the front; a 1-edge produces the next distance, so it goes to the back. The deque then always holds tentative distances d at the front and $d+1$ at the back (Figure 12.3), which is exactly the order Dijkstra would use, at $O(1)$ per operation: $O(n+m)$ in total.

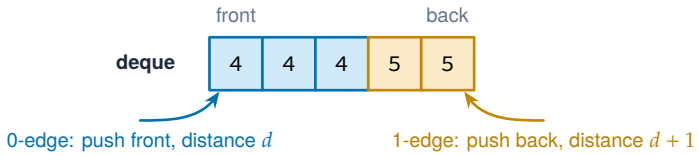


Figure 12.3: 0–1 BFS keeps the deque sorted by distance: vertices at distance d (blue) in front, $d+1$ (orange) behind.

Example: Fewest walls to break

In an $n \times m$ grid of free cells and walls, moving into a free cell costs nothing and moving into a wall cell (breaking it) costs 1. Find the minimum cost of a path from the top-left to the bottom-right cell (counting the start cell if it is a wall).

Solution. A 0–1 BFS over cells, where the weight of a move is 1 exactly when the target cell is a wall.

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<string> g(n);
    cin >> g;
    const int dr[4] = {-1, 1, 0, 0}, dc[4] = {0, 0, -1, 1};
    // entering a wall cell costs 1 (break it), a free cell costs 0
    vvi dist(n, vi(m, INF));
    deque<pii> dq;
    dist[0][0] = g[0][0] == '#';
```

```
    dq.push_back({0, 0});
    while (!dq.empty()) {
        auto [r, c] = dq.front();
        dq.pop_front();
        rep(d, 4) {
            int nr = r + dr[d], nc = c + dc[d];
            if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue;
            int w = g[nr][nc] == '#';
            if (dist[r][c] + w >= dist[nr][nc]) continue;
            dist[nr][nc] = dist[r][c] + w;
            if (w == 0) dq.push_front({nr, nc});  // same distance: front
            else dq.push_back({nr, nc});  // one more: back
        }
    }
    print(dist[n - 1][m - 1]);
}
```

</> Template: 0-1 BFS Ctrl+Alt+T, 0 prefix `cp01bfs`

The same idea extends to small integer weights: with weights in $\{0, 1, \ldots, k\}$, keep $k+1$ buckets (a circular array of queues indexed by distance modulo $k+1$), known as *Dial's algorithm*, for $O(nk+m)$.

12.5 Negative weights: Bellman–Ford

PLATINUM

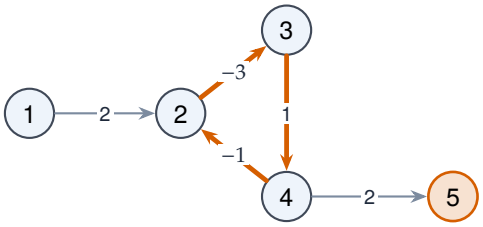
With negative edges, the order of relaxations can no longer be chosen greedily. **Bellman–Ford** does not choose at all: it relaxes *every* edge, in any order, $n-1$ times. After round k, every vertex whose shortest path uses at most k edges has its correct distance (induction on k: the last edge of such a path is relaxed in round k, after its start became correct). If there is no negative cycle reachable from the source, shortest paths are simple and have at most $n-1$ edges, so $n-1$ rounds suffice. If an edge can still be relaxed in round n, there is a negative cycle: distances would keep falling forever.

Example: Cycle Finding (CSES)

Determine whether a directed graph ($n \le 2500$, $m \le 5000$) contains a negative cycle, and if so, print one.

Solution. Start every distance at 0, as if a virtual source had 0-edges to every vertex; then every negative cycle is reachable. Run n rounds and remember a vertex relaxed in the last one. That vertex may hang off the cycle rather than lie on it (Figure 12.4), but its parent pointers lead back into the cycle: after n steps back along them we are certainly on it, and one more walk around recovers the cycle.

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<array<ll, 3>> e(m);
    for (auto& [a, b, c] : e) cin >> a >> b >> c;
    // all distances start at 0: a virtual source reaches every vertex
    vector<ll> dist(n + 1, 0);
    vi par(n + 1, 0);
```



The cycle $2 \to 3 \to 4 \to 2$ has weight -3. A vertex relaxed in round n can be off the cycle, like 5; walking back along parent pointers ($5 \to 4 \to 3 \to 2 \to 4 \to \cdots$) enters the cycle within n steps.

Figure 12.4: A negative cycle (red) and a vertex that depends on it. Everything reachable from a negative cycle has distance $-\infty$.

```
    int last = 0;  // a vertex relaxed in round n, if any
    rep(round, n) {
        last = 0;
        for (auto [a, b, c] : e)
            if (dist[a] + c < dist[b])
                dist[b] = dist[a] + c, par[b] = a, last = b;
    }
    if (!last) {
        print("NO");
        return;
    }
    // last may hang off the cycle: n steps back along par land on it
    int x = last;
    rep(i, n) x = par[x];
    vi cycle{x};
    for (int y = par[x]; y != x; y = par[y]) cycle.push_back(y);
    cycle.push_back(x);
    reverse(all(cycle));
    print("YES");
    print(cycle);
}
```

</> Template: Bellman-Ford (negative edges) Ctrl+Alt+T, Shift+W prefix `cpbellman`

12.5.1 Longest paths and unbounded answers

Negating all weights turns a longest-path problem into a shortest-path problem, and a positive cycle into a negative one. The subtle part is deciding when the answer is unbounded: only if a cycle that can be improved forever is both reachable from the start *and* able to reach the target.

Example: High Score (CSES)

Each tunnel $a \to b$ changes your score by x (possibly negative). Find the maximum score of a route from room 1 to room n, or -1 if it can be arbitrarily large.

Solution. Run Bellman–Ford for $n-1$ rounds on the negated scores. Any vertex that can still be improved afterwards is on or after a positive-score cycle reachable from 1. Then run n more rounds that mark every such vertex and spread the mark along edges; if room n gets marked, the answer is unbounded.

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<array<ll, 3>> e(m);
    for (auto& [a, b, c] : e) cin >> a >> b >> c;
    // longest path = shortest path with negated scores
    const ll UNREACHED = LLONG_MAX;
    vector<ll> dist(n + 1, UNREACHED);
    dist[1] = 0;
    rep(round, n - 1)
        for (auto [a, b, c] : e)
            if (dist[a] != UNREACHED && dist[a] - c < dist[b])
                dist[b] = dist[a] - c;
    // anything still improvable lies on or after a positive-score cycle;
    // n more rounds spread that mark to everything reachable from it
    vector<bool> inf(n + 1, false);
    rep(round, n)
        for (auto [a, b, c] : e) {
            if (dist[a] == UNREACHED) continue;
            if (inf[a] || dist[a] - c < dist[b])
                inf[b] = true, dist[b] = dist[a] - c;
        }
    print(inf[n] ? -1 : -dist[n]);
}
```

SPFA The "Shortest Path Faster Algorithm" is Bellman–Ford with a queue of vertices whose distance changed. It is often much faster in practice, but its worst case is still $O(nm)$, and problem setters build that case on purpose. Use it where Bellman–Ford is intended anyway, such as inside min-cost flow with negative costs, never as a replacement for Dijkstra.

Johnson's reweighting With negative edges but no negative cycle, all-pairs distances in a sparse graph are best computed with *potentials*: run Bellman–Ford once from a virtual source to get $h(v)$, then run Dijkstra from every vertex with the weights $w'(u,v) = w(u,v) + h(u) - h(v)$, which are non-negative by the triangle inequality. Every path from s to t changes by the same amount $h(s) - h(t)$, so shortest paths are preserved and $\delta(s,t) = \delta'(s,t) - h(s) + h(t)$. Min-cost flow uses exactly these potentials to keep running Dijkstra after negative residual edges appear (Chapter 15).

12.6 Systems of difference constraints

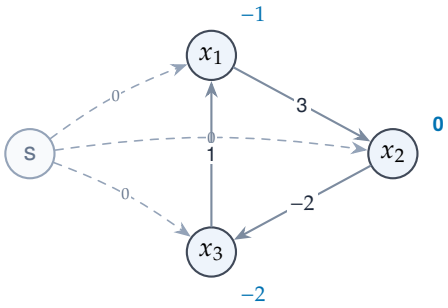
PLATINUM

A surprising number of scheduling problems reduce to a system of inequalities of the form $x_b - x_a \le c$: "task b starts at most c after task a", "the prefix sum up to j exceeds the one up to i by at least 3". Such a **system of difference constraints** is exactly the triangle inequality of shortest paths. Make a vertex per variable and an edge $a \to b$ of weight c per constraint, and add a virtual source with 0-edges to all vertices. Then:

- if the graph has a negative cycle, the system has no solution: adding up the constraints around the cycle gives $0 \le$ (negative number);

- otherwise the shortest distances $x_v = d(v)$ satisfy every constraint, because $d(b) \le d(a) + c$ for every edge.

So Bellman–Ford decides feasibility and constructs a solution (Figure 12.5). Among all solutions with every $x_v \le 0$, it even finds the componentwise largest one. Equalities are two inequalities, and $x_b - x_a \ge c$ is $x_a - x_b \le -c$.



Constraints $x_2 - x_1 \le 3$, $x_3 - x_2 \le -2$, $x_1 - x_3 \le 1$. The cycle $x_1 \to x_2 \to x_3 \to x_1$ has weight $2 \ge 0$, so the system is feasible, and the distances from the virtual source s (blue) are a solution: $x = (-1, 0, -2)$.

Figure 12.5: A system of difference constraints as a graph. Each constraint $x_b - x_a \le c$ is an edge $a \to b$ of weight c; shortest distances from a virtual source satisfy all constraints at once.

Example: Difference constraints

Given n variables and m constraints $x_b - x_a \le c$, print integer values satisfying all of them, or `NO`.

```
void solve() {
    int n, m;
    cin >> n >> m;
    // constraint x_b - x_a <= c becomes the edge a -> b of weight c
    vector<array<ll, 3>> e(m);
    for (auto& [a, b, c] : e) cin >> a >> b >> c;
    vector<ll> x(n + 1, 0);  // a virtual source with 0-edges to everyone
    rep(round, n + 1) {
        bool changed = false;
        for (auto [a, b, c] : e)
            if (x[a] + c < x[b]) x[b] = x[a] + c, changed = true;
        if (!changed) {  // shortest distances satisfy every constraint
            print(vi(x.begin() + 1, x.end()));
            return;
        }
    }
    print("NO");  // still changing after n + 1 rounds: a negative cycle
}
```

</> Template: System of difference constraints Ctrl+Alt+U, Shift+I prefix `cpdiffcons`

How to recognize it Variables that are only compared by differences: start times with precedence and deadline constraints, positions on a line with minimum and maximum gaps, prefix sums with bounds on segment sums ("every window of length k contains at least a chosen elements"). When the constraints involve a parameter, binary search the parameter and test feasibility with Bellman–Ford.

12.7 All pairs: Floyd–Warshall

GOLD

Floyd–Warshall computes all n^2 distances in $O(n^3)$ with three nested loops. It is a dynamic programme over the set of vertices allowed as *intermediate* stops: after the iteration for k, `d[i][j]` is the length of the shortest path from i to j whose intermediate vertices all lie in $\{1, \dots, k\}$. Allowing k as a new stop either does not help, or the best path passes through k exactly once, splitting into a path $i \rightsquigarrow k$ and a path $k \rightsquigarrow j$ that use only earlier stops (Figure 12.6):

$$d_k[i][j] = \min\bigl(d_{k-1}[i][j],\ d_{k-1}[i][k] + d_{k-1}[k][j]\bigr).$$

The update can be done in place, because row k and column k do not change during iteration k. It handles negative edges, and afterwards $d[i][i] < 0$ exactly when i lies on a negative cycle.

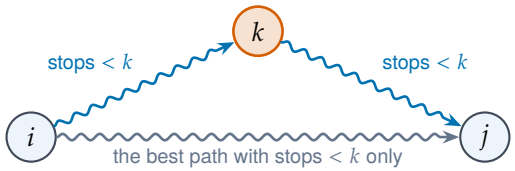


Figure 12.6: The Floyd–Warshall recurrence. With stops up to k allowed, the best path from i to j either avoids k (grey) or visits it once, and then both halves use only stops below k (blue).

Example: Shortest Routes II (CSES)

Answer 10^5 queries for the shortest distance between two of $n \le 500$ cities connected by two-way roads.

```
void solve() {
    int n, m, q;
    cin >> n >> m >> q;
    const ll NONE = LLONG_MAX / 4;  // large, but NONE + NONE does not overflow
    vector<vector<ll>> d(n + 1, vector<ll>(n + 1, NONE));
    rep(v, 1, n + 1) d[v][v] = 0;
    rep(i, m) {
        ll a, b, c;
        cin >> a >> b >> c;
        chmin(d[a][b], c);  // keep the cheapest of parallel roads
        chmin(d[b][a], c);
    }
    rep(k, 1, n + 1) rep(i, 1, n + 1) rep(j, 1, n + 1)  // k outermost!
        chmin(d[i][j], d[i][k] + d[k][j]);
    while (q--) {
        int a, b;
        cin >> a >> b;
        print(d[a][b] == NONE ? -1 : d[a][b]);
    }
}
```

Template: Floyd-Warshall (all pairs) Ctrl+Alt+T, W prefix `cpfloyd`

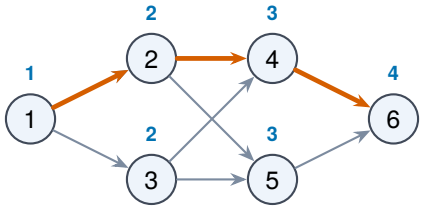
Floyd–Warshall is a template for more Replacing (min, +) by (*or*, *and*) computes the transitive closure, which with bitsets runs in $O(n^3/64)$ (Section 10.4); (max, min) computes widest paths. Because the outer loop adds vertices one at a time in any order you like, processing them sorted by some value answers "the shortest path using only vertices of value at most x" for every x in one run. And adding a single new edge (a, b) to a finished distance matrix costs only $O(n^2)$: $d[i][j] \leftarrow \min(d[i][j], d[i][a] + w + d[b][j])$.

Loop order and infinity The loop over k must be the outermost; with any other order the result is wrong on some inputs. And "infinity" must survive additions: with `LLONG_MAX`, `d[i][k] + d[k][j]` overflows, so use a quarter of it (or skip pairs where either term is infinite, as the template does).

12.8 Shortest and longest paths in DAGs

GOLD

In a DAG, process the vertices in topological order (Chapter 13): when a vertex is reached, every edge into it has already been relaxed, so its distance is final, and relaxing its outgoing edges once is enough. This takes $O(n + m)$ and allows any weights. Most importantly, it solves *longest* paths, which are hopeless in general graphs (they generalise Hamiltonian paths) but easy in DAGs: just maximise instead of minimise (Figure 12.7). Every DP is, at heart, a shortest or longest path in the DAG of its states (Part V).



Vertices in topological order, with the most cities on a route from 1 (blue). Each value is one more than the best predecessor; the red route $1 \to 2 \to 4 \to 6$ visits 4 cities.

Figure 12.7: Longest path in a DAG by DP in topological order. Parent pointers recover the route.

Example: Longest Flight Route (CSES)

Find a route from city 1 to city n that visits as many cities as possible, in a network of one-way flights that contains no cycles.

```
void solve() {
    int n, m;
    cin >> n >> m;
    vvi adj(n + 1);
    vi indeg(n + 1, 0);
    rep(i, m) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        indeg[b]++;
    }
    // Kahn's algorithm gives a topological order of the DAG
    vi order, q;
    rep(v, 1, n + 1) if (!indeg[v]) q.push_back(v);
```

```
    while (!q.empty()) {
        int u = q.back();
        q.pop_back();
        order.push_back(u);
        for (int v : adj[u])
            if (--indeg[v] == 0) q.push_back(v);
    }
    // best[v]: most cities on a route 1 -> v (0 = unreachable)
    vi best(n + 1, 0), par(n + 1, 0);
    best[1] = 1;
    for (int u : order) {
        if (!best[u]) continue;
        for (int v : adj[u])
            if (best[u] + 1 > best[v]) best[v] = best[u] + 1, par[v] = u;
    }
    if (!best[n]) {
        print("IMPOSSIBLE");
        return;
    }
    vi route;
    for (int v = n; v; v = par[v]) route.push_back(v);
    reverse(all(route));
    print(sz(route));
    print(route);
}
```

Chapter summary

- All algorithms relax edges; they differ in the order. Parent pointers give the shortest-path tree.
- Dijkstra finalises vertices in order of distance and needs non-negative weights; skip outdated queue entries.
- Put extra information into the vertex (layered graphs); gather counts in finalisation order.
- 0–1 weights: a deque. Negative weights: Bellman–Ford, $n - 1$ rounds, plus one to detect negative cycles.
- Longest paths negate weights; they are unbounded only if an improvable cycle is reachable and reaches the target.
- Difference constraints are shortest paths from a virtual source; a negative cycle means no solution.
- Floyd–Warshall adds allowed intermediate vertices one by one: $O(n^3)$, any weights.
- In a DAG, relax in topological order: shortest and longest paths in linear time.

Practice problems (CSES Problem Set)

SINGLE SOURCE

- Shortest Routes I
- Flight Discount
- Investigation
- Flight Routes
- High Score
- Cycle Finding

ALL PAIRS AND DAGS

- Shortest Routes II
- Longest Flight Route
- Game Routes

CHAPTER 13

Union–Find, Spanning Trees and Topological Order

This chapter is about three ideas that organise graphs globally rather than by searching them. The disjoint set union structure maintains a partition of the vertices under merging, in almost constant time per operation; it answers "are these two connected?" as edges arrive, and with small extensions it maintains equations and parities along the way. Minimum spanning trees connect everything as cheaply as possible, and their theory (the cut and cycle properties) explains a whole family of greedy algorithms. Prüfer codes turn labelled trees into sequences and back, which makes counting trees a matter of counting sequences. And topological order arranges a directed acyclic graph so that every edge points forward, the precondition for every dynamic programme on a DAG.

IN THIS CHAPTER GOLD PLATINUM IOI / ADVANCED

- implement a DSU with union by size and path compression, and extend it with sizes, offsets and parities;
- prove the cut and cycle properties, and use them to justify Kruskal and Prim;
- answer offline "connected using edges of weight at most x" queries, and build Kruskal reconstruction trees;
- encode and decode Prüfer sequences, and count labelled trees with given degrees;
- compute topological orders, detect cycles with them, and run DP along them.

Before you start: Chapter 11 (graph basics) and Section 7.5 (exchange arguments).

13.1 Disjoint set union

GOLD

A **disjoint set union** (DSU, also *union–find*) maintains a partition of $\{0, \dots, n-1\}$ into disjoint sets under two operations: *unite* the sets containing a and b, and *find* a representative of the set containing x, so that "are a and b in the same set?" becomes "do they have the same representative?". Each set is stored as a rooted tree of parent pointers whose root is the representative; uniting two sets links one root under the other (Figure 13.1).

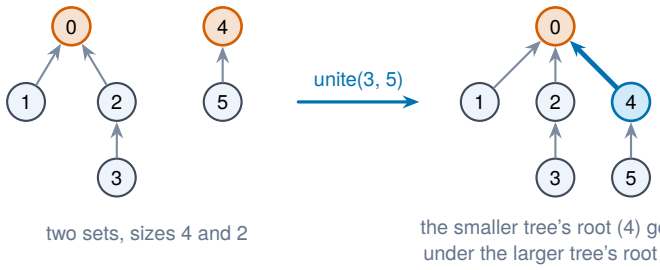


Figure 13.1: A DSU stores each set as a tree of parent pointers (arrows point to the parent); roots (red) are the representatives. Union by size links the root of the smaller tree under the root of the larger one.

Two optimisations make the operations fast.

- **Union by size**: when uniting, link the root of the smaller tree under the root of the larger one. A vertex then gets deeper only when its tree at least doubles in size, which can happen at most $\log_2 n$ times, so every tree has depth $O(\log n)$.
- **Path compression**: during *find*, make the vertices on the path point directly to the root, or at least closer to it (Figure 13.2). Later finds on the same path become $O(1)$.

With both, a sequence of operations costs $O(\alpha(n))$ amortised per operation, where α is the inverse Ackermann function, which is at most 4 for any n that fits in the universe. For contest purposes, DSU operations are constant time. The template uses *path halving* (`x = par[x] = par[par[x]]`), which gets the same bound with a loop instead of recursion.

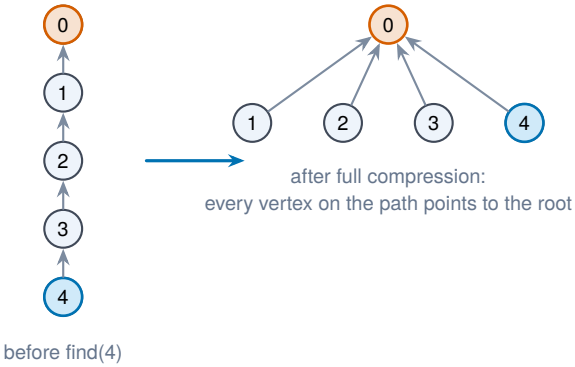


Figure 13.2: Path compression. A *find* on the deepest vertex walks the whole path once, and afterwards every vertex on it is one step from the root.

</> Template: DSU (union-find) Ctrl+Alt+T, U prefix `cpdsu`

The complete use of a DSU for incremental connectivity, printing the number of components and the largest component after every new road, is the worked problem Road Construction (Chapter 31). When the edges are *removed* instead, reverse time so that they are added (Section 2.4).

> **How to recognize it** Merging groups and asking whether two things are in the same group; connectivity while edges are only added; Kruskal; "the first moment two things become connected" (add edges by time or weight); equivalence classes ("a and b have the same colour"); offline queries sorted by a threshold.

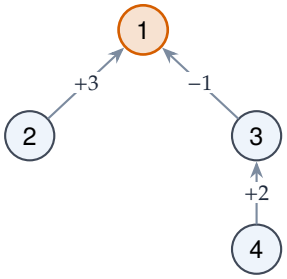
13.1.1 Beyond connectivity: sizes, offsets and parities

The root of a set is a natural place to store information about the whole set: its size, its sum, its minimum, a list of members. Uniting merges the information of the two roots. More interesting is information stored on the *edges* of the DSU trees: each vertex stores a value relative to its parent, and a find accumulates these values along the path to the root.

Example: Consistent differences

There are n unknowns. Process q statements "$x_b - x_a = c$", reporting whether each one is consistent with the accepted statements so far (and accepting it if so), and queries "what is $x_b - x_a$?", answering `?` if it is not determined.

Solution. A **weighted DSU**. Every vertex v stores $off(v) = x_v - x_{par(v)}$; a find with path compression sums the offsets on the way, so afterwards $off(v) = x_v - x_{root}$ (Figure 13.3). Two vertices in the same set have a determined difference $off(b) - off(a)$. Uniting two sets for a new statement sets the offset of one root relative to the other so that the statement holds.



Edge labels are $off(v) = x_v - x_{par(v)}$. Relative to the root: $x_2 - x_1 = 3$, $x_3 - x_1 = -1$, $x_4 - x_1 = -1 + 2 = 1$. So the query $x_4 - x_2$ answers $1 - 3 = -2$, and the statement $x_4 - x_2 = 5$ is a conflict.

Figure 13.3: A weighted DSU: offsets along the path to the root add up to the difference between a vertex and its root.

```
void solve() {
    int n, q;
    cin >> n >> q;
    // par[v] and off[v] = x_v - x_par[v]; the root r has off[r] = 0
    vi par(n + 1), off(n + 1, 0), siz(n + 1, 1);
    iota(all(par), 0);
    // find also makes off[v] = x_v - x_root (path compression)
    auto find = [&](auto&& self, int v) -> int {
        if (par[v] == v) return v;
        int r = self(self, par[v]);
        off[v] += off[par[v]];  // par[v] now points at r directly
        par[v] = r;
        return r;
    };
    while (q--) {
        int type, a, b;
        cin >> type >> a >> b;
        int ra = find(find, a), rb = find(find, b);
        if (type == 1) {  // the fact x_b - x_a = c
            ll c;
            cin >> c;
            if (ra == rb) {
                print(off[b] - off[a] == c ? "OK" : "CONFLICT");
                continue;
            }
            // link the roots so that x_b - x_a = c holds
            ll d = off[a] + c - off[b];  // the required x_rb - x_ra
            if (siz[ra] < siz[rb]) swap(ra, rb), d = -d;
            par[rb] = ra, off[rb] = d, siz[ra] += siz[rb];
            print("OK");
        } else {  // query x_b - x_a
```

```
            if (ra != rb) print("?");
            else print(off[b] - off[a]);
        }
    }
}
```

The same structure with offsets modulo 2 is the *parity DSU*: it maintains constraints "a and b are in different groups" and "in the same group", detects the first contradiction (an odd cycle), and so checks bipartiteness online as edges arrive. With offsets in any group (addition modulo k, XOR, permutations composed along the path) the idea is the same.

What a DSU cannot do, and the workarounds A DSU cannot split a set. If edges are only deleted, reverse time so that they are inserted. If insertions and deletions mix, answer the queries offline with a *DSU with rollback* (union by size without path compression, keeping a stack of changes to undo) over a segment tree on the time axis, in $O(m \log m \log n)$ (Chapter 14). And when sets carry lists that must really be merged, merge the smaller list into the larger one: every element moves $O(\log n)$ times (small to large, Chapter 16).

</> Template: DSU with rollback Ctrl+Alt+Y, R prefix `cpdsurb`

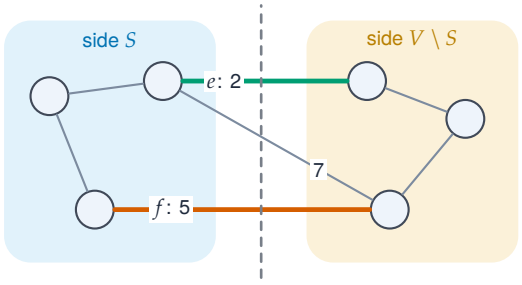
13.2 Minimum spanning trees

GOLD

A **spanning tree** of a connected graph is a subset of $n-1$ edges that connects all vertices, and a **minimum spanning tree** (MST) is one of minimum total weight. Every MST algorithm is justified by two properties. A *cut* is a split of the vertices into two non-empty sides; an edge *crosses* the cut if its ends are on different sides.

- **Cut property.** For any cut, a lightest edge crossing it belongs to some MST.
- **Cycle property.** For any cycle, an edge that is strictly heavier than all the others on the cycle belongs to no MST.

Proof of the cut property (an exchange argument). Let e be a lightest edge crossing a cut, and T an MST without e. Adding e to T creates a cycle, which crosses the cut at e and therefore at least once more, at some edge f of T (Figure 13.4). Replacing f by e gives a spanning tree of weight $w(T) - w(f) + w(e) \le w(T)$, because e is a lightest crossing edge: it is also an MST, and it contains e. The cycle property follows the same way: removing the heaviest edge of a cycle from any spanning tree and reconnecting with another edge of the cycle never increases the weight.



Three edges cross the cut (dashed line). If a spanning tree uses f but not the lightest crossing edge e, swapping f for e keeps it spanning (the cycle through e crosses the cut twice) and does not make it heavier.

Figure 13.4: The cut property: a lightest edge crossing any cut can always be part of a minimum spanning tree.

13.2.1 Kruskal's algorithm

Kruskal's algorithm sorts the edges by weight and adds each edge whose endpoints are not yet connected, which a DSU checks in $O(\alpha(n))$ (Figure 13.5). Each accepted edge is a lightest edge crossing the cut between its endpoint's component and the rest, so it is safe by the cut property; each rejected edge is the heaviest edge of the cycle it would close. The running time is dominated by sorting: $O(m \log m)$.

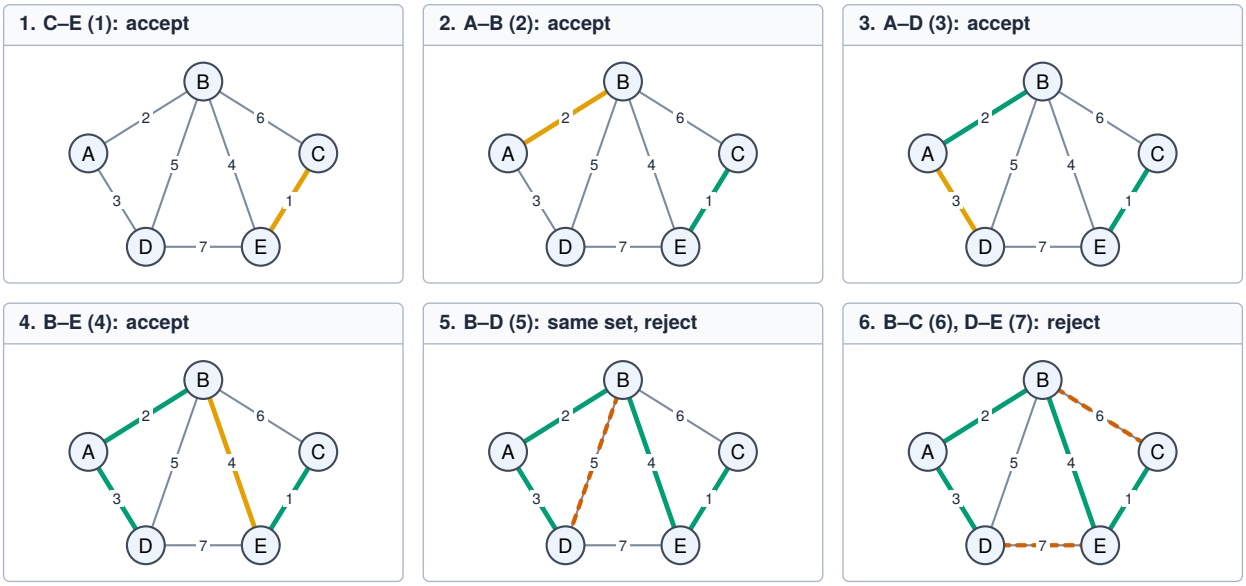


Figure 13.5: Kruskal's algorithm. Edges are taken in order of weight; an edge is accepted (orange, then green) when it joins two different components and rejected (red, dashed) when its endpoints are already connected. The MST has weight $1 + 2 + 3 + 4 = 10$.

Example: Road Reparation (CSES)

Repairing each road has a cost. Find the minimum total cost of repairing roads so that all cities are connected, or report that it is impossible.

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<array<ll, 3>> e(m);  // (cost, a, b): sorting orders by cost
    for (auto& [c, a, b] : e) cin >> a >> b >> c;
    sort(all(e));
    DSU d(n + 1);
    ll total = 0;
    int used = 0;
    for (auto [c, a, b] : e)
        if (d.unite(a, b)) total += c, used++;  // skip edges inside a component
    if (used == n - 1) print(total);
    else print("IMPOSSIBLE");
}
```

</> Template: Kruskal MST Ctrl+Alt+T, K prefix `cpkruskal`

13.2.2 Prim's algorithm

Prim's algorithm grows a single tree from any start vertex, always adding the cheapest edge that leaves the tree; each such edge is a lightest edge crossing the cut between the tree and the rest. With a priority queue it runs in $O(m \log n)$, essentially Dijkstra with the key $w(u, v)$ instead of $d(u) + w(u, v)$. Its real strength is the dense case: when every pair of n points is an edge ($m \approx n^2/2$), the array version below runs in $O(n^2)$ without ever building the edge list, while Kruskal would have to sort $n^2/2$ edges.

Example: Connecting points

Connect $n \le 2000$ points in the plane by a network of segments of minimum total cost, where a segment costs the squared distance between its endpoints.

```
void solve() {
    int n;
    cin >> n;
    vector<pll> p(n);
    for (auto& [x, y] : p) cin >> x >> y;
    auto cost = [&](int i, int j) {  // squared Euclidean distance
        ll dx = p[i].first - p[j].first, dy = p[i].second - p[j].second;
        return dx * dx + dy * dy;
    };
    // O(n^2) Prim: best[v] = cheapest edge from the tree to v
    vector<ll> best(n, LLONG_MAX);
    vector<bool> in(n, false);
    best[0] = 0;
    ll total = 0;
    rep(it, n) {
        int u = -1;  // the cheapest vertex outside the tree
        rep(v, n) if (!in[v] && (u == -1 || best[v] < best[u])) u = v;
        in[u] = true;
        total += best[u];
        rep(v, n) if (!in[v]) chmin(best[v], cost(u, v));
    }
    print(total);
}
```

</> Template: Prim MST (dense graphs) Ctrl+Alt+T, Shift+5 prefix `cpprim`

13.2.3 Thresholds and the Kruskal reconstruction tree

The MST also solves *bottleneck* questions. For every pair of vertices, the path between them in an MST minimises the largest edge weight among all paths (a *minimax* path). So "the smallest range D such that every cow can talk to every other, relaying messages" is the largest edge of the MST (USACO Moocast), and "can u reach v using only edges of weight at most x?" depends only on the MST.

For many such queries, the offline recipe is: sort the edges by weight and the queries by threshold, and add edges with a DSU while sweeping the threshold upwards. At each query, the DSU contains exactly the edges of weight at most the threshold.

Example: Reachability under a weight limit

For each of q queries (v, x), count the vertices reachable from v using only edges of weight at most x.

```
void solve() {
    int n, m, q;
    cin >> n >> m >> q;
    vector<array<ll, 3>> e(m);  // (weight, a, b)
    for (auto& [w, a, b] : e) cin >> a >> b >> w;
    sort(all(e));
    vector<array<ll, 3>> qs(q);  // (limit, vertex, index)
    rep(i, q) cin >> qs[i][1] >> qs[i][0], qs[i][2] = i;
    sort(all(qs));
    DSU d(n + 1);
    vi ans(q);
    int j = 0;
    for (auto [x, v, i] : qs) {  // queries by increasing limit
        while (j < m && e[j][0] <= x) d.unite(e[j][1], e[j][2]), j++;
        ans[i] = d.size(v);  // everything connected with edges <= x
    }
    for (ll a : ans) print(a);
}
```

When the queries must be answered *online*, record Kruskal's merges as a tree instead: every accepted edge of weight w becomes a new node, labelled w, whose two children are the roots of the components it joins (Figure 13.6). This **Kruskal reconstruction tree** has the original vertices as leaves, and labels increase towards the root. The vertices reachable from v with edges of weight at most x are exactly the leaves below the highest ancestor of v with label at most x, which binary lifting finds in $O(\log n)$ (Chapter 16); counts, sums and ranks over those leaves become subtree queries.

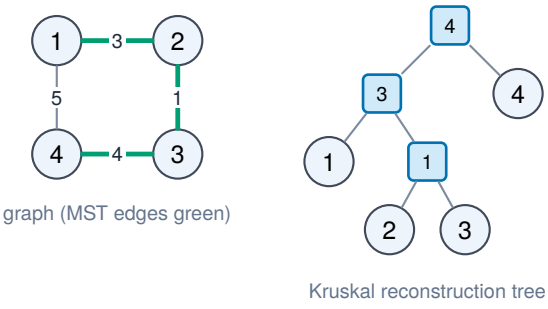


Figure 13.6: A graph and its Kruskal reconstruction tree. Internal nodes (blue squares) are the accepted edges, labelled with their weights; labels grow towards the root.

Template: Kruskal reconstruction tree Ctrl+Alt+Y, K prefix `cpkrt`

More MST facts worth knowing

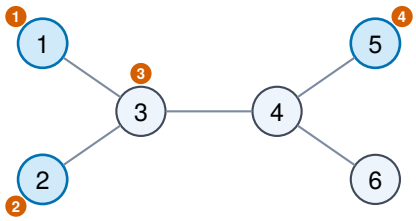
- **Borůvka's algorithm**: in each round, every component adds its cheapest outgoing edge; the number of components at least halves, so $O(\log n)$ rounds suffice. It shines when the graph is implicit (weights like $a_i \oplus a_j$) and "the cheapest edge leaving a component" can be found with a data structure such as a binary trie (Section 10.3).

- **Uniqueness**: if all weights are distinct, the MST is unique.
- **Second-best MST**: add one non-tree edge (u, v) and remove the heaviest edge on the tree path between u and v; the best such swap gives the second-best spanning tree (maximum on paths by binary lifting).
- **Manhattan MST**: for points with the L_1 distance, only $O(n)$ candidate edges (the nearest point in each of eight octants) are needed, which brings the MST of 10^5 points within reach.

13.3 Prüfer codes and counting trees

IOI / ADVANCED

Labelled trees on the vertices $1, \dots, n$ can be encoded as sequences, which turns questions about trees into questions about sequences. The **Prüfer code** of a tree is built by repeatedly removing the *smallest* leaf and writing down its neighbour, until two vertices remain (Figure 13.7). The result is a sequence of length $n - 2$ over $\{1, \dots, n\}$, and every such sequence arises from exactly one tree. Decoding reverses the process: each vertex v that appears k times in the sequence has degree $k + 1$, so the leaves at every step are known; at step i, the smallest current leaf is attached to the i-th entry, which loses one from its remaining degree.



Remove the smallest leaf, write its neighbour:
① leaf 1, write 3 ② leaf 2, write 3
③ leaf 3 (a leaf now), write 4 ④ leaf 5, write 4
Two vertices (4, 6) remain: the Prüfer code is $3, 3, 4, 4$. Vertices 3 and 4 have degree 3 and appear twice; leaves never appear.

Figure 13.7: Computing a Prüfer code. The red numbers give the order in which leaves are removed.

Example: Tree from a Prüfer code

Reconstruct the tree with a given Prüfer code.

Solution. Keep the current leaves in an ordered set (a vertex is a leaf when its remaining degree is 1). For each entry v of the code, the smallest leaf is attached to v; remove it, and decrease the remaining degree of v, which becomes a leaf when it reaches 1. The two vertices left at the end form the last edge. With a set this is $O(n \log n)$; the template does it in $O(n)$ with a moving pointer.

```
void solve() {
    int n;
    cin >> n;
    vi seq(n - 2);
    cin >> seq;
    vi deg(n + 1, 1);  // a vertex appears (degree - 1) times
    for (int v : seq) deg[v]++;
    set<int> leaves;
    rep(v, 1, n + 1) if (deg[v] == 1) leaves.insert(v);
    for (int v : seq) {  // the smallest leaf was removed, attached to v
        int leaf = *leaves.begin();
        leaves.erase(leaves.begin());
        print(leaf, v);
```

```
        if (--deg[v] == 1) leaves.insert(v);
    }
    print(*leaves.begin(), *next(leaves.begin()));  // the last edge
}
```

</> Template: Prufer code (tree <-> sequence) Ctrl+Alt+U, U prefix cpprufer

The bijection gives two famous counts at once. There are n^{n-2} labelled trees on n vertices (*Cayley's formula*), because there are n^{n-2} sequences. And since vertex i appears exactly $d_i - 1$ times, the number of labelled trees in which vertex i has degree d_i is the number of arrangements of that multiset,

$$\binom{n-2}{d_1-1,\ d_2-1,\ \ldots,\ d_n-1} = \frac{(n-2)!}{\prod_i (d_i-1)!},$$

provided all $d_i \geq 1$ and $\sum_i (d_i - 1) = n - 2$ (otherwise there are none).

```
void solve() {
    int n;
    cin >> n;
    vi d(n);
    cin >> d;
    if (n == 1) {  // a single vertex: the only tree has degree 0
        print(d[0] == 0 ? 1 : 0);
        return;
    }
    // Prufer: vertex i appears d_i - 1 times in a sequence of length n - 2
    ll total = 0;
    for (ll x : d) {
        if (x < 1) {
            print(0);
            return;
        }
        total += x - 1;
    }
    if (total != n - 2) {
        print(0);
        return;
    }
    vi fact(n, 1);
    rep(i, 1, n) fact[i] = fact[i - 1] * i % MOD;
    auto power = [](ll b, ll e) {
        ll r = 1;
        for (b %= MOD; e; e >>= 1, b = b * b % MOD)
            if (e & 1) r = r * b % MOD;
        return r;
    };
    ll ans = fact[n - 2];  // (n - 2)! / prod (d_i - 1)!
    for (ll x : d) ans = ans * power(fact[x - 1], MOD - 2) % MOD;
    print(ans);
}
```

Counting spanning structures The generalisation of Cayley's formula is the number of spanning forests of the complete graph in which given vertices lie in different trees: $k\,n^{n-k-1}$ forests of n labelled vertices with k trees, where k specified vertices are the roots. For arbitrary graphs, the matrix-tree theorem counts spanning trees as a determinant (Chapter 25).

13.4 Topological order

GOLD

A **topological order** of a directed graph lists its vertices so that every edge points forward, from an earlier vertex to a later one (Figure 13.8). It exists exactly when the graph has no directed cycle: a cycle can never point forward all the way round, and conversely every DAG has a vertex with no incoming edges (follow edges backwards from any vertex; without cycles the walk must stop), which can go first.

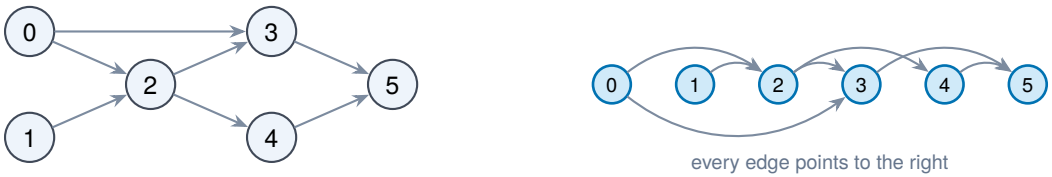


Figure 13.8: A DAG and one of its topological orders. Kahn's algorithm produces it by repeatedly removing a vertex with no incoming edges: first 0 and 1, then 2, then 3 and 4, then 5.

Kahn's algorithm does exactly that: keep the in-degrees and a queue of vertices whose in-degree is 0; pop a vertex, append it to the order, and decrease the in-degrees of its successors, pushing those that reach 0. If fewer than n vertices come out, the remaining ones all have an incoming edge from another remaining vertex, so they contain a cycle. The alternative is DFS: a vertex finishes only after everything reachable from it, so the vertices in *decreasing* finishing time form a topological order.

Example: Course Schedule (CSES)

Find an order in which to take n courses respecting m prerequisites "course a before course b", or report that none exists.

```
void solve() {
    int n, m;
    cin >> n >> m;
    vvi adj(n + 1);
    vi indeg(n + 1, 0);
    rep(i, m) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        indeg[b]++;
    }
    queue<int> q;  // courses whose prerequisites are all taken
    rep(v, 1, n + 1) if (!indeg[v]) q.push(v);
    vi order;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
```

```
        order.push_back(u);
        for (int v : adj[u])
            if (--indeg[v] == 0) q.push(v);
    }
    if (sz(order) < n) print("IMPOSSIBLE");  // the rest lie on cycles
    else print(order);
}
```

</> Template: Topological sort (Kahn) Ctrl+Alt+T, O prefix `cptopo`

How to recognize it Dependencies ("task a before task b"), prerequisites, "is there an order consistent with these comparisons?", build systems, and every DP over a DAG. The lexicographically smallest topological order uses a min-heap instead of the queue; the order that places a particular vertex as early as possible is found by running Kahn on the *reversed* graph and taking the largest available vertex each time.

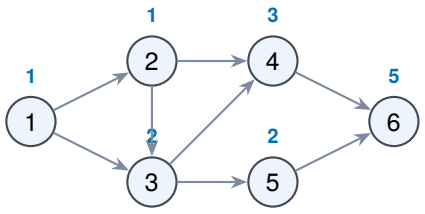
13.4.1 Dynamic programming along a topological order

In topological order, every vertex is processed after all its predecessors, so any quantity defined through incoming edges is computed in one pass: longest and shortest paths (Section 12.8), the number of paths, reachability with extra conditions (Figure 13.9).

Example: Game Routes (CSES)

Count the routes from level 1 to level n in a game whose teleporters form a DAG, modulo $10^9 + 7$.

Solution. The number of routes to v is the sum of the numbers of routes to its predecessors: $ways(v) = \sum_{u \to v} ways(u)$, with $ways(1) = 1$, evaluated in topological order.



Blue: the number of routes from 1. Each value is the sum over the incoming edges: $ways(4) = ways(2) + ways(3) = 1 + 2$, and $ways(6) = 3 + 2 = 5$.

Figure 13.9: Counting paths in a DAG by summing over incoming edges in topological order.

```
void solve() {
    int n, m;
    cin >> n >> m;
    vvi adj(n + 1);
    vi indeg(n + 1, 0);
    rep(i, m) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        indeg[b]++;
    }
    vi order, st;  // a topological order by Kahn's algorithm
```

```
    rep(v, 1, n + 1) if (!indeg[v]) st.push_back(v);
    while (!st.empty()) {
        int u = st.back();
        st.pop_back();
        order.push_back(u);
        for (int v : adj[u])
            if (--indeg[v] == 0) st.push_back(v);
    }
    vi ways(n + 1, 0);  // ways[v]: routes from 1 to v, modulo MOD
    ways[1] = 1;
    for (int u : order)
        for (int v : adj[u]) ways[v] = (ways[v] + ways[u]) % MOD;
    print(ways[n]);
}
```

Cycles? Condense first If the graph has cycles, contract every strongly connected component to a single vertex (Chapter 14). The result is a DAG, and many questions about walks that may revisit vertices (the most coins collectable, the largest set of mutually reachable rooms) become DP on that DAG (CSES Coin Collector).

Chapter summary

- A DSU with union by size and path compression costs $O(\alpha(n))$ per operation; offsets along the tree maintain differences, parities and other group-valued relations.
- The cut property (a lightest crossing edge is safe) justifies Kruskal ($O(m \log m)$, with a DSU) and Prim ($O(n^2)$ on dense graphs, $O(m \log n)$ with a heap).
- MST paths are minimax paths; threshold queries are answered offline by sorting, or online with a Kruskal reconstruction tree.
- Prüfer codes are a bijection between labelled trees and sequences of length $n-2$: n^{n-2} trees, and multinomial counts for given degrees.
- A graph has a topological order if and only if it is acyclic; Kahn's algorithm finds one or detects the cycle, and DP along it counts paths and optimises over them.

Practice problems (CSES Problem Set)

UNION–FIND AND SPANNING TREES

- Road Reparation
- Road Construction
- Network Breakdown
- New Roads Queries

TOPOLOGICAL ORDER AND DAGS

- Course Schedule
- Longest Flight Route
- Game Routes
- Course Schedule II
- Coin Collector

CHAPTER 14

Connectivity

Chapter 11 asked whether two vertices are connected. This chapter asks how *robustly* they are connected, and what the connections look like at a finer scale. In a directed graph, which vertices can reach each other in both directions? In an undirected graph, which single edge or vertex is so important that its failure disconnects the network? Can a route use every road exactly once? And what if the graph changes, with roads opening and closing over time? All of these questions are answered by variations of one idea, the *low-link* values of a depth-first search, plus a handful of beautiful reductions: boolean formulas to strongly connected components (2-SAT), tours to degrees (Euler), and deletions to rollbacks (offline dynamic connectivity).

IN THIS CHAPTER PLATINUM IOI / ADVANCED

- find strongly connected components with Tarjan's algorithm, and do DP on the condensation;
- solve 2-SAT through the implication graph;
- find bridges and articulation points with low-link values, and build bridge trees and block-cut trees;
- decide whether Eulerian paths and circuits exist, and construct them with Hierholzer's algorithm;
- handle edge deletions offline with a segment tree on time and a DSU with rollback;
- use Euler's formula for planar graphs, and reason about critical vertices and dominators.

Before you start: Chapters 11 and 13, in particular DFS trees and edge classification (Section 11.3).

14.1 Strongly connected components PLATINUM

In a directed graph, two vertices are **strongly connected** if each can reach the other. This is an equivalence relation, and its classes are the **strongly connected components** (SCCs). Contracting every SCC to a single vertex gives the **condensation**, which is always a DAG: a cycle through several components would merge them into one (Figure 14.1). So every directed graph is a DAG of strongly connected pieces, and many questions split into a question inside each piece (where everything is mutually reachable) and a DP over the DAG.

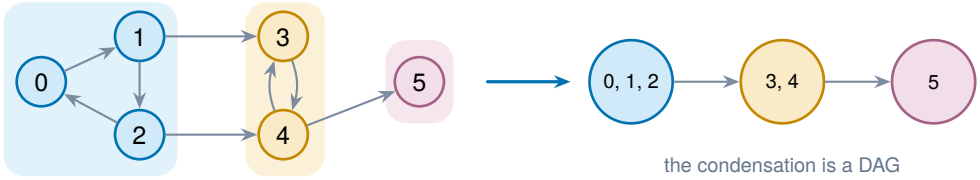


Figure 14.1: A directed graph with three strongly connected components (shaded), and its condensation.

Tarjan's algorithm. One DFS finds all SCCs. Every vertex gets a discovery time $tin(v)$ and a **low-link** value $low(v)$: the smallest discovery time reachable from v's DFS subtree by following

tree edges and then *one* edge to a vertex that is still on a stack of "open" vertices. Vertices are pushed when they are discovered. When DFS finishes a vertex u with $low(u) = tin(u)$, nothing in its subtree can reach an earlier open vertex, so u is the first vertex of its component that DFS discovered, and the component is exactly the vertices above u on the stack: pop them (Figure 14.2). The whole algorithm is one DFS, $O(n + m)$.

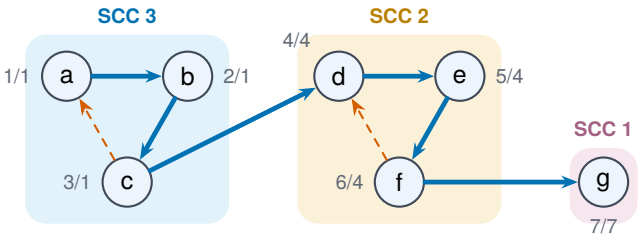


Figure 14.2: Tarjan's algorithm on a graph with three SCCs; labels are tin/low, blue edges are tree edges and red dashed edges go back to open vertices. Vertex g finishes first with $low = tin$ and forms SCC 1; then vertex d (popping f, e, d), then vertex a. Components come out in reverse topological order of the condensation.

Example: Planets and Kingdoms (CSES)

Divide the planets of a directed teleporter network into kingdoms, so that two planets are in the same kingdom exactly when each can reach the other.

```
void solve() {
    int n, m;
    cin >> n >> m;
    vvi adj(n + 1);
    rep(i, m) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
    }
    // Tarjan: tin = discovery time, low = smallest tin reachable from the
    // subtree through one edge to a vertex that is still on the stack
    vi tin(n + 1, 0), low(n + 1, 0), comp(n + 1, 0), st;
    vector<bool> on(n + 1, false);
    int timer = 0, k = 0;
    auto dfs = [&](auto&& self, int u) -> void {
        tin[u] = low[u] = ++timer;
        st.push_back(u), on[u] = true;
        for (int v : adj[u]) {
            if (!tin[v]) self(self, v), low[u] = min(low[u], low[v]);
            else if (on[v]) low[u] = min(low[u], tin[v]);
        }
        if (low[u] == tin[u]) {  // u is the first vertex of its SCC: pop it
            k++;
            int v;
            do {
                v = st.back();
                st.pop_back();
                on[v] = false, comp[v] = k;
            } while (v != u);
        }
```

```
    };
    rep(v, 1, n + 1) if (!tin[v]) dfs(dfs, v);
    print(k);
    print(vi(comp.begin() + 1, comp.end()));
}
```

</> Template: SCC (Tarjan) Ctrl+Alt+T, C prefix cpscc

Why components come out in reverse topological order. If there is an edge from component A to component B, then B is completed before A: either DFS entered B first (and B, unable to reach A, finished before A was even discovered) or it entered A first and reached B inside A's subtree. So Tarjan's numbers decrease along edges of the condensation, and processing components in *increasing* number visits every component after all its successors. **Kosaraju's algorithm** is the alternative: one DFS for finishing times, then DFS on the reversed graph in decreasing finishing time, each tree being an SCC. It is easier to prove but does twice the work.

Example: Flight Routes Check (CSES)

Decide whether every city can reach every other by one-way flights; if not, print two cities a, b with no route from a to b.

Solution. No SCC algorithm is needed: the graph is strongly connected exactly when vertex 1 reaches every vertex and every vertex reaches 1 (search the reversed graph from 1). A vertex that fails either test gives the pair.

```
void solve() {
    int n, m;
    cin >> n >> m;
    vvi adj(n + 1), radj(n + 1);
    rep(i, m) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        radj[b].push_back(a);
    }
    // strongly connected iff 1 reaches everyone and everyone reaches 1
    auto reach = [&](const vvi& g) {
        vector<bool> seen(n + 1, false);
        vi st{1};
        seen[1] = true;
        while (!st.empty()) {
            int u = st.back();
            st.pop_back();
            for (int v : g[u])
                if (!seen[v]) seen[v] = true, st.push_back(v);
        }
        return seen;
    };
    vector<bool> fwd = reach(adj), bwd = reach(radj);
    rep(v, 1, n + 1) {
        if (!fwd[v]) {  // no route 1 -> v
            print("NO");
```

```
            print(1, v);
            return;
        }
        if (!bwd[v]) {  // no route v -> 1
            print("NO");
            print(v, 1);
            return;
        }
    }
    print("YES");
}
```

Example: Coin Collector (CSES)

Each room contains coins, and tunnels are one-way. Starting anywhere and ending anywhere, collect as many coins as possible (a room's coins are collected once).

Solution. Inside an SCC, a walk can visit every room and return, so entering a component means collecting all of it. Condense, give each component the sum of its coins, and find the heaviest path in the condensation. Tarjan's numbering is already a reverse topological order, so one pass in increasing component number computes the best path starting at each component.

```
void solve() {
    int n, m;
    cin >> n >> m;
    vi coins(n + 1);
    rep(v, 1, n + 1) cin >> coins[v];
    vvi adj(n + 1);
    rep(i, m) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
    }
    vi tin(n + 1, 0), low(n + 1, 0), comp(n + 1, 0), st;
    vector<bool> on(n + 1, false);
    int timer = 0, k = 0;
    auto dfs = [&](auto&& self, int u) -> void {
        tin[u] = low[u] = ++timer;
        st.push_back(u), on[u] = true;
        for (int v : adj[u]) {
            if (!tin[v]) self(self, v), low[u] = min(low[u], low[v]);
            else if (on[v]) low[u] = min(low[u], tin[v]);
        }
        if (low[u] == tin[u]) {
            k++;
            int v;
            do {
                v = st.back();
                st.pop_back();
                on[v] = false, comp[v] = k;
            } while (v != u);
```

```
        }
    };
    rep(v, 1, n + 1) if (!tin[v]) dfs(dfs, v);
    // Tarjan numbers SCCs in reverse topological order: an edge between
    // components goes from a larger number to a smaller one
    vi total(k + 1, 0), best(k + 1, 0);
    rep(v, 1, n + 1) total[comp[v]] += coins[v];
    vvi out(k + 1);
    rep(u, 1, n + 1) for (int v : adj[u])
        if (comp[u] != comp[v]) out[comp[u]].push_back(comp[v]);
    ll ans = 0;
    rep(c, 1, k + 1) {  // successors (smaller numbers) are already done
        for (int d : out[c]) chmax(best[c], best[d]);
        best[c] += total[c];
        chmax(ans, best[c]);
    }
    print(ans);
}
```

How to recognize it "Mutually reachable", "can every vertex reach every other?", cyclic dependencies, and "maximise what you collect when you may revisit vertices" (condense, then DP on the DAG). Also the minimum number of edges to add to make a graph strongly connected: max(#sources, #sinks) of the condensation, unless it is already a single component.

14.2 2-SAT

PLATINUM

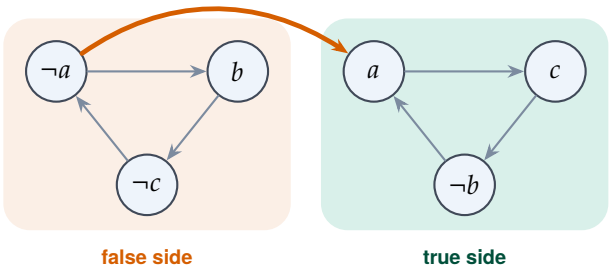
A **2-SAT** instance has boolean variables and clauses of the form $(a \vee b)$, where a and b are variables or negated variables (*literals*). General satisfiability is NP-hard, but with two literals per clause it is solvable in linear time. The clause $(a \vee b)$ is equivalent to the two implications $\neg a \Rightarrow b$ and $\neg b \Rightarrow a$. Build the **implication graph** on the $2n$ literals with these edges (Figure 14.3). Then:

- if some variable x and its negation $\neg x$ lie in the same SCC, then $x \Rightarrow \neg x$ and $\neg x \Rightarrow x$, and no assignment works;
- otherwise, set x true exactly when the SCC of x comes *after* the SCC of $\neg x$ in topological order (with Tarjan's numbering: when $comp(x) < comp(\neg x)$).

The second rule works because the implication graph is symmetric (an edge $u \to v$ comes with $\neg v \to \neg u$): choosing, for each variable, the literal later in topological order never creates an implication from a true literal to a false one.

Example: Giant Pizza (CSES)

Each of n family members has two wishes of the form "topping x on" or "topping x off", and wants at least one of them fulfilled. Choose the m toppings to satisfy everyone, or report that it is impossible.



Clauses $(a \vee b)$, $(\neg a \vee c)$, $(\neg b \vee \neg c)$, $(a \vee a)$. Each clause gives two edges; $(a \vee a)$ gives $\neg a \to a$ (red). The SCC of a comes later in topological order than that of $\neg a$, so a, c and $\neg b$ are true.

Figure 14.3: An implication graph with two SCCs that are mirror images of each other. The edge between them fixes the order, and the later component is set true: a = true, b = false, c = true.

```
void solve() {
    int n, m;  // n wishes, m toppings
    cin >> n >> m;
    // literal 2x: topping x is on, 2x + 1: it is off
    int N = 2 * m;
    vvi adj(N);
    auto lit = [](char sign, int x) { return 2 * (x - 1) + (sign == '-'); };
    rep(i, n) {
        char s1, s2;
        int x1, x2;
        cin >> s1 >> x1 >> s2 >> x2;
        int a = lit(s1, x1), b = lit(s2, x2);
        adj[a ^ 1].push_back(b);  // (a or b): not a implies b,
        adj[b ^ 1].push_back(a);  // and not b implies a
    }
    vi tin(N, 0), low(N, 0), comp(N, 0), st;
    vector<bool> on(N, false);
    int timer = 0, k = 0;
    auto dfs = [&](auto&& self, int u) -> void {
        tin[u] = low[u] = ++timer;
        st.push_back(u), on[u] = true;
        for (int v : adj[u]) {
            if (!tin[v]) self(self, v), low[u] = min(low[u], low[v]);
            else if (on[v]) low[u] = min(low[u], tin[v]);
        }
        if (low[u] == tin[u]) {
            k++;
            int v;
            do {
                v = st.back();
                st.pop_back();
                on[v] = false, comp[v] = k;
            } while (v != u);
        }
    };
    rep(v, N) if (!tin[v]) dfs(dfs, v);
    string ans(m, '?');
    rep(x, m) {
        if (comp[2 * x] == comp[2 * x + 1]) {  // x on implies x off and back
            print("IMPOSSIBLE");
            return;
```

```
        }
        // choose the literal whose SCC is later in topological order,
        // which is the smaller Tarjan number
        ans[x] = comp[2 * x] < comp[2 * x + 1] ? '+' : '-';
    }
    rep(x, m) cout << ans[x] << (x + 1 < m ? ' ' : '\n');
}
```

</> Template: 2-SAT Ctrl+Alt+Y, 2 prefix `cp2sat`

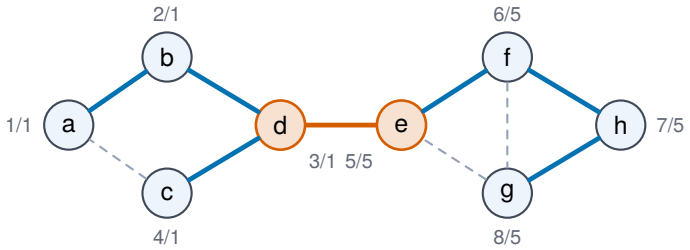
How to recognize it Every object has exactly two options (on/off, left/right, team A/B, top/bottom), and every constraint talks about at most two objects: "not both" is $(\neg a \vee \neg b)$, "at least one" is $(a \vee b)$, "exactly one" is both, "if a then b" is $(\neg a \vee b)$, and "a must be true" is $(a \vee a)$. Constraints on three objects at once make the problem NP-hard, so look for a way to express every constraint pairwise. "At most one of these k" is expressible with $O(k)$ auxiliary prefix variables instead of $O(k^2)$ clauses.

14.3 Bridges and articulation points

PLATINUM

In an undirected graph, a **bridge** is an edge whose removal increases the number of connected components, and an **articulation point** (cut vertex) is a vertex whose removal does. Both are found with one DFS and low-link values: here $low(v)$ is the smallest discovery time reachable from v's subtree using tree edges downwards and then at most one back edge (Figure 14.4). Recall that an undirected DFS has only tree and back edges, and back edges go to ancestors.

- A tree edge u–v (v the child) is a **bridge** if and only if $low(v) > tin(u)$: nothing in v's subtree has a back edge to u or above, so removing the edge cuts the subtree off. Back edges are never bridges; they lie on a cycle.
- A non-root vertex u is an **articulation point** if and only if some child v has $low(v) \geq tin(u)$: v's subtree cannot get above u without u. The root is one exactly when it has at least two DFS children.



Labels tin/low from a DFS started at a (blue tree edges, grey dashed back edges). $low(e) = 5 > tin(d) = 3$, so d–e is a bridge; d and e are articulation points.

Figure 14.4: Bridges and articulation points from low-link values: the red edge is the only bridge, and the red vertices are the articulation points.

Example: Necessary Roads (CSES)

List every road whose removal would disconnect some pair of cities.

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<vector<pii>> adj(n + 1);  // (neighbour, edge id)
    rep(i, m) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back({b, i});
        adj[b].push_back({a, i});
    }
    vi tin(n + 1, 0), low(n + 1, 0);
    vector<pii> bridges;
    int timer = 0;
    auto dfs = [&](auto&& self, int u, int pe) -> void {  // pe: parent edge
        tin[u] = low[u] = ++timer;
        for (auto [v, id] : adj[u]) {
            if (id == pe) continue;  // skip only the edge we came along
            if (tin[v]) low[u] = min(low[u], tin[v]);  // a back edge
            else {
                self(self, v, id);
                low[u] = min(low[u], low[v]);
                // nothing below v climbs to u or above: u-v is a bridge
                if (low[v] > tin[u]) bridges.push_back({u, v});
            }
        }
    };
    rep(v, 1, n + 1) if (!tin[v]) dfs(dfs, v, -1);
    print(sz(bridges));
    for (auto [a, b] : bridges) print(a, b);
}
```

Parallel edges For bridges, skip the edge you arrived along by its *id*, not by the parent vertex. Otherwise a doubled road u–v is reported as a bridge, although removing one copy leaves the other. For articulation points, skipping the parent vertex is fine, because parallel edges never change whether a vertex separates the graph.

Example: Necessary Cities (CSES)

List every city whose removal would disconnect some pair of the remaining cities.

```
void solve() {
    int n, m;
    cin >> n >> m;
    vvi adj(n + 1);
    rep(i, m) {
        int a, b;
```

```
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    vi tin(n + 1, 0), low(n + 1, 0);
    vector<bool> cut(n + 1, false);
    int timer = 0;
    auto dfs = [&](auto&& self, int u, int p) -> void {
        tin[u] = low[u] = ++timer;
        int children = 0;
        for (int v : adj[u]) {
            if (v == p) continue;
            if (tin[v]) low[u] = min(low[u], tin[v]);
            else {
                self(self, v, u);
                low[u] = min(low[u], low[v]);
                children++;
                // v's subtree cannot climb above u: removing u cuts it off
                if (p != 0 && low[v] >= tin[u]) cut[u] = true;
            }
        }
        if (p == 0 && children >= 2) cut[u] = true;  // the root
    };
    rep(v, 1, n + 1) if (!tin[v]) dfs(dfs, v, 0);
    vi cities;
    rep(v, 1, n + 1) if (cut[v]) cities.push_back(v);
    print(sz(cities));
    print(cities);
}
```

</> Template: Bridges + articulation points Ctrl+Alt+T, Shift+B prefix `cpbridges`

14.3.1 Bridge trees and block-cut trees

Removing all bridges splits a graph into **2-edge-connected components**: inside one, every two vertices are joined by two edge-disjoint paths. Contracting each of them gives the **bridge tree**, whose edges are exactly the bridges. Questions like "which edges lie on every path from a to b?" then become "which edges lie on the tree path between the components of a and b?", answered with LCA (Chapter 16).

Biconnected components (blocks) are the vertex version: maximal subgraphs without articulation points. Every edge belongs to exactly one block, while an articulation point belongs to several. The **block-cut tree** has a node for every block and one for every articulation point, joined when the articulation point belongs to the block (Figure 14.5). "Which vertices lie on every path from a to b?" becomes "which articulation-point nodes lie on the tree path?".

</> Template: 2-edge-connected components Ctrl+Alt+Y, B prefix `cp2ecc`

</> Template: Biconnected components (blocks) Ctrl+Alt+Y, Shift+B prefix `cpbcc`

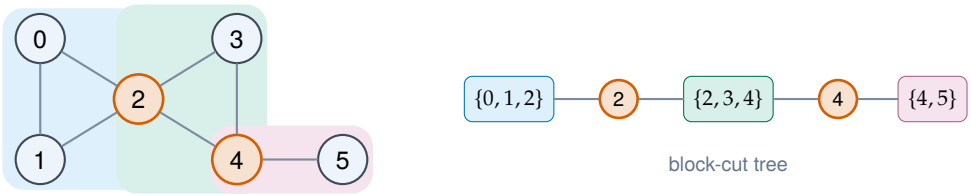


Figure 14.5: Three blocks (shaded) that share the articulation points 2 and 4, and the block-cut tree, which alternates between block nodes and articulation-point nodes. The single edge 4–5 is a block of its own (and a bridge).

14.4 Eulerian paths and circuits

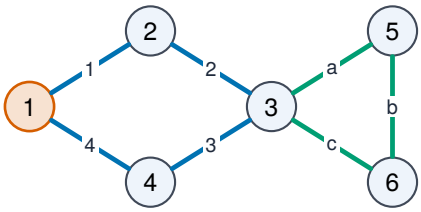
PLATINUM

An **Eulerian path** uses every edge exactly once; an **Eulerian circuit** also returns to its start. Unlike Hamiltonian paths, which visit every *vertex* once and are NP-hard to find, Eulerian paths are decided by degrees alone (plus the condition that all edges lie in one connected component), as Table 14.1 shows. The reason is local: every time a walk passes through a vertex it uses one edge in and one edge out, so all vertices except the ends must have balanced degrees.

	Circuit exists if and only if	Path exists if and only if
undirected	every degree is even	exactly 0 or 2 vertices have odd degree (start at one of them)
directed	in-degree = out-degree everywhere	one vertex has out − in = 1 (the start), one has in − out = 1 (the end), all others are balanced

Table 14.1: Existence of Eulerian circuits and paths, assuming all edges are in one (weakly) connected component.

Hierholzer's algorithm constructs the tour in $O(n+m)$. Walk along unused edges from the start until you get stuck; by the degree conditions, you can only get stuck at the end vertex. Then back up along the walk: whenever a vertex on it still has unused edges, a closed sub-tour starting there can be spliced into the walk at that point (Figure 14.6). With an explicit stack this is a few lines: the top of the stack advances along an unused edge if it has one, and otherwise is popped and appended to the output, which is the tour in reverse order.



The first walk from 1 (blue, edges 1–4) closes without using the triangle at 3. Backing up to 3, which still has unused edges, starts a sub-tour a, b, c (green) that is spliced in: $1, 2, 3, 5, 6, 3, 4, 1$.

Figure 14.6: Hierholzer's algorithm splices closed sub-tours into the current walk wherever unused edges remain.

Example: Mail Delivery (CSES)

Find a route that starts and ends at post office 1 and uses every street exactly once, or report that none exists.

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<vector<pii>> adj(n + 1);  // (neighbour, edge id)
    vi deg(n + 1, 0);
    rep(i, m) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back({b, i});
        adj[b].push_back({a, i});
        deg[a]++, deg[b]++;
    }
    rep(v, 1, n + 1) if (deg[v] % 2) {  // a circuit needs even degrees
        print("IMPOSSIBLE");
        return;
    }
    // Hierholzer: walk unused edges; a vertex with none left is final
    vector<bool> used(m, false);
    vi ptr(n + 1, 0), st{1}, tour;
    while (!st.empty()) {
        int u = st.back();
        while (ptr[u] < sz(adj[u]) && used[adj[u][ptr[u]].second]) ptr[u]++;
        if (ptr[u] == sz(adj[u])) {
            tour.push_back(u);
            st.pop_back();
        } else {
            auto [v, id] = adj[u][ptr[u]];
            used[id] = true;
            st.push_back(v);
        }
    }
    if (sz(tour) != m + 1) {  // some edges are not connected to city 1
        print("IMPOSSIBLE");
        return;
    }
    print(tour);
}
```

Example: Teleporters Path (CSES)

Find a route from level 1 to level n that uses every one-way teleporter exactly once.

```
void solve() {
    int n, m;
    cin >> n >> m;
    vvi adj(n + 1);
    vi out(n + 1, 0), in(n + 1, 0);
    rep(i, m) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        out[a]++, in[b]++;
```

```
    }
    // a path 1 -> n: out - in is +1 at 1, -1 at n, 0 elsewhere
    bool ok = out[1] - in[1] == 1 && in[n] - out[n] == 1;
    rep(v, 2, n) if (in[v] != out[v]) ok = false;
    if (!ok) {
        print("IMPOSSIBLE");
        return;
    }
    vi ptr(n + 1, 0), st{1}, path;  // Hierholzer on a directed graph
    while (!st.empty()) {
        int u = st.back();
        if (ptr[u] < sz(adj[u])) st.push_back(adj[u][ptr[u]++]);
        else path.push_back(u), st.pop_back();
    }
    if (sz(path) != m + 1) {  // some edges are unreachable from 1
        print("IMPOSSIBLE");
        return;
    }
    reverse(all(path));
    print(path);
}
```

</> Template: Eulerian path / circuit (Hierholzer) Ctrl+Alt+Y, E prefix `cpeuler`

Euler tours as a modelling tool "Use every transition exactly once" is an Eulerian path on the graph whose edges are the transitions: de Bruijn sequences (Section 10.5), arranging dominoes in a chain, or reconstructing a string from all its length-k substrings. And "orient the edges so that every vertex has in-degree equal to out-degree" is solved by following an Euler circuit after pairing up odd vertices with dummy edges.

14.5 Offline dynamic connectivity

IOI / ADVANCED

Suppose edges are added *and* removed, and the number of components is wanted after every operation. A DSU cannot delete, but offline there is an elegant way around it. Every edge is alive during an interval of time. Build a segment tree over the time axis and store each edge in the $O(\log q)$ nodes that exactly cover its interval (Figure 14.7). Then walk the segment tree depth first with a **DSU with rollback** (union by size, no path compression, and a stack of the changes made): entering a node unites its edges, reaching a leaf reads off the answer for that moment, and leaving a node undoes its unions. At each leaf, exactly the edges alive at that time have been united. The total cost is $O(q \log q \log n)$.

Example: Dynamic Connectivity (CSES)

A graph starts with m edges; each of k operations adds or removes an edge. Print the number of components initially and after every operation.

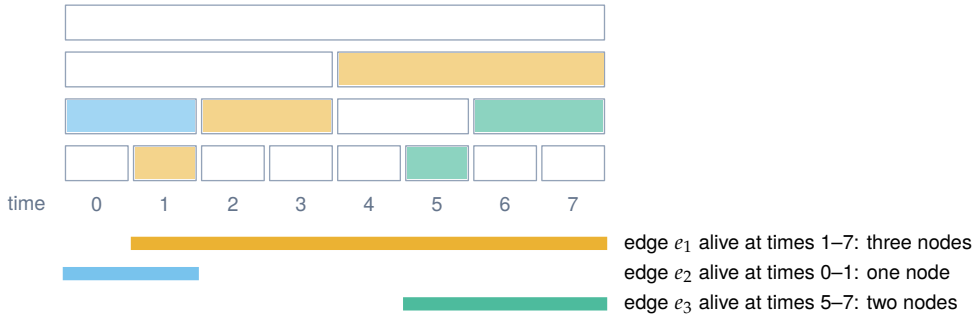


Figure 14.7: A segment tree on time. Each edge is stored in the $O(\log q)$ nodes that tile its lifetime; a leaf at time t sees exactly the edges stored on its root path, which are the edges alive at t.

```
void solve() {
    int n, m, k;
    cin >> n >> m >> k;
    // every edge is alive during an interval of times 0..k; time 0 is the
    // initial graph, time t is right after operation t
    map<pii, int> since;  // alive edge -> the time it appeared
    vector<array<int, 4>> live;  // (from, to, a, b): alive on [from, to]
    auto norm = [](int a, int b) { return pii(min(a, b), max(a, b)); };
    rep(i, m) {
        int a, b;
        cin >> a >> b;
        since[norm(a, b)] = 0;
    }
    rep(t, 1, k + 1) {
        int type, a, b;
        cin >> type >> a >> b;
        if (type == 1) since[norm(a, b)] = t;
        else {
            auto it = since.find(norm(a, b));
            live.push_back({it->second, t - 1, a, b});
            since.erase(it);
        }
    }
    for (auto [e, t] : since) live.push_back({t, k, e.first, e.second});
    // each interval is stored in O(log k) nodes of a segment tree on time
    int T = k + 1;
    vector<vector<pii>> node(4 * T);
    auto add = [&](auto&& self, int v, int l, int r, int ql, int qr, pii e) {
        if (qr < l || r < ql) return;
        if (ql <= l && r <= qr) {
            node[v].push_back(e);
            return;
        }
        int mid = (l + r) / 2;
        self(self, 2 * v, l, mid, ql, qr, e);
        self(self, 2 * v + 1, mid + 1, r, ql, qr, e);
    };
    for (auto [from, to, a, b] : live) add(add, 1, 0, T - 1, from, to, {a, b});
    // DSU with rollback: union by size, no path compression
    vi par(n + 1), siz(n + 1, 1);
```

```
    iota(all(par), 0);
    int comps = n;
    vector<pii> history;  // (attached root, new root) for undo
    auto find = [&](int x) {
        while (par[x] != x) x = par[x];
        return x;
    };
    vi ans(T);
    auto walk = [&](auto&& self, int v, int l, int r) -> void {
        int saved = sz(history);
        for (auto [a, b] : node[v]) {
            int ra = find(a), rb = find(b);
            if (ra == rb) continue;
            if (siz[ra] < siz[rb]) swap(ra, rb);
            par[rb] = ra, siz[ra] += siz[rb], comps--;
            history.push_back({rb, ra});
        }
        if (l == r) ans[l] = comps;
        else {
            int mid = (l + r) / 2;
            self(self, 2 * v, l, mid);
            self(self, 2 * v + 1, mid + 1, r);
        }
        while (sz(history) > saved) {  // undo this node's unions
            auto [rb, ra] = history.back();
            history.pop_back();
            par[rb] = rb, siz[ra] -= siz[rb], comps++;
        }
    };
    walk(walk, 1, 0, T - 1);
    print(ans);
}
```

</> Template: Offline dynamic connectivity (edge deletions) Ctrl+Alt+Y, Shift+R prefix `cpofflinecc`

The segment tree on time The technique needs only a structure that can *undo* its last operation. A DSU with rollback, a knapsack array that is copied before each insertion, a linear basis with snapshots, a stack of any kind: each item lives in $O(\log q)$ nodes, and every node's operations are undone on the way back up. It turns "insertions and deletions, answer queries" into "insertions only" for any such structure, at the cost of one logarithmic factor.

14.6 Euler's formula and planar graphs

IOI / ADVANCED

A graph is *planar* if it can be drawn in the plane without crossing edges; such a drawing divides the plane into *faces*, including the unbounded outer face. **Euler's formula** says that for a connected planar drawing,

$$V - E + F = 2,$$

and with C connected components, $V - E + F = 1 + C$ (Figure 14.8). Two consequences are useful. A simple planar graph with $V \geq 3$ has $E \leq 3V - 6$, so planar graphs are sparse. And counting faces, which is often what a problem asks ("how many regions does this drawing create?"), reduces to

counting vertices, edges and components, which is frequently easier than flood filling the regions: a DSU over the vertices, applied while the edges are drawn, keeps C up to date.

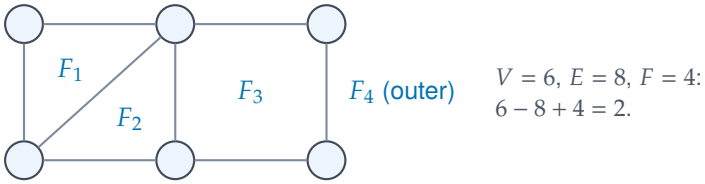


Figure 14.8: Euler's formula on a small planar graph: three bounded faces and the outer face.

14.7 Critical vertices, edges and dominators

IOI / ADVANCED

A vertex or edge is *critical* for getting from s to t if every route of the relevant kind uses it. Three versions come up, and they have completely different answers.

- **Every path, undirected graph.** The critical vertices are the articulation points on the path between s and t in the block-cut tree, and the critical edges are the bridges on the path in the bridge tree.
- **Every shortest path.** With Dijkstra from s and from t (on the reversed graph), an edge (u, v, w) lies on some shortest path if and only if $d_s(u) + w + d_t(v) = D$. It lies on *all* of them if moreover it carries all of them: $ways_s(u) \cdot ways_t(v) = ways_s(t)$. The counts are huge, so compare them modulo one or two random large primes. A vertex v is critical if and only if $d_s(v) + d_t(v) = D$ and $ways_s(v) \cdot ways_t(v) = ways_s(t)$.
- **Every path, directed graph.** A vertex v **dominates** w if every path from s to w passes through v. The immediate dominators form the **dominator tree** (Figure 14.9), and the vertices critical for reaching t are exactly the ancestors of t in it. In a DAG, build it in topological order: the immediate dominator of w is the LCA, in the tree built so far, of all predecessors of w. General graphs need the Lengauer–Tarjan algorithm, $O(m\,\alpha(n))$.

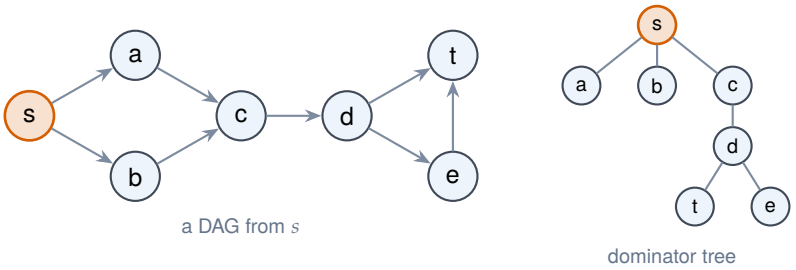


Figure 14.9: A DAG and its dominator tree. Every path from s to t passes through c and d (the ancestors of t), but neither a nor b is unavoidable, so c's immediate dominator is s.

> **How to recognize it** "Which roads, if closed, make the trip impossible (or longer)?", "checkpoints every route must pass", "which server failures cut off a client". First decide whether the question is about all paths or all *shortest* paths, and whether the graph is directed: the three cases above have different answers.

Link-cut trees. Fully dynamic connectivity in a *forest* (link, cut, path aggregates, online) is solved by the **link-cut tree** in amortised $O(\log n)$ per operation. It represents each tree as a set of preferred paths stored in splay trees keyed by depth, and an *access* operation makes the path from the root

to a vertex preferred. It is powerful but long and error-prone; in contests, first look for an offline solution (a segment tree on time, Kruskal order, an Euler tour with a treap), which almost always exists.

Chapter summary

- SCCs: Tarjan's single DFS with low-link values; components come out in reverse topological order of the condensation, ready for DP.
- 2-SAT: clauses become implications; unsatisfiable exactly when x and $\neg x$ share an SCC; otherwise take the literal whose SCC is later in topological order.
- Bridges: $low(v) > tin(u)$; articulation points: $low(v) \geq tin(u)$ (the root needs two children). Skip the parent edge by id.
- Eulerian paths are decided by degrees and built by Hierholzer's algorithm in linear time.
- Deletions offline: a segment tree on time plus any structure with undo.
- $V - E + F = 1 + C$ for planar drawings; dominator trees answer "unavoidable vertex" questions in directed graphs.

Practice problems (CSES Problem Set)

STRONG CONNECTIVITY AND 2-SAT

- Flight Routes Check
- Planets and Kingdoms
- Coin Collector
- Giant Pizza

BRIDGES, CUT VERTICES AND EULER TOURS

- Necessary Roads
- Necessary Cities
- Mail Delivery
- Teleporters Path
- De Bruijn Sequence
- Dynamic Connectivity

CHAPTER 15

Flows and Matchings

Network flow models anything that moves through a network with capacities: water through pipes, data through links, workers to jobs. Its power in contests comes from *modelling*. Assignment problems, disjoint paths, choosing a profitable set of projects with prerequisites, separating two groups at minimum cost, covering a DAG with few chains: all of these reduce to a maximum flow, a minimum cut or a matching, and then a single library routine solves them. This chapter develops the theory that makes those reductions trustworthy (residual graphs, the max-flow min-cut theorem, König's and Hall's theorems, linear programming duality in the form of potentials) and the algorithms behind the templates, from Dinic's blocking flows to the Hungarian algorithm, Edmonds' blossoms, Stoer–Wagner and Gomory–Hu trees.

IN THIS CHAPTER IOI / ADVANCED

- understand residual graphs and augmenting paths, and run Dinic's algorithm;
- use the max-flow min-cut theorem to model separation and selection problems;
- find disjoint paths, maximum bipartite matchings, minimum vertex covers and path covers;
- solve assignment problems with the Hungarian algorithm and min-cost flow;
- handle lower bounds, general (non-bipartite) matching, global minimum cuts and all-pairs minimum cuts.

Before you start: Chapters 11 and 12 (BFS, Dijkstra, Bellman–Ford).

15.1 Maximum flow

IOI / ADVANCED

A **flow network** is a directed graph with a *capacity* $c(e) \geq 0$ on every edge, a source s and a sink t. A **flow** assigns a value $0 \leq f(e) \leq c(e)$ to every edge such that, at every vertex other than s and t, the flow coming in equals the flow going out (*conservation*). The value of the flow is the net amount leaving s, and the *maximum flow* problem asks for the largest possible value.

15.1.1 Residual graphs and augmenting paths

The key to every flow algorithm is the **residual graph** of a flow. For every edge $u \to v$ with capacity c and flow f, it contains a *forward* edge $u \to v$ with the unused capacity $c - f$, and a *backward* edge $v \to u$ with capacity f: pushing flow backwards along it cancels flow that was sent earlier. A path from s to t in the residual graph is an **augmenting path**; pushing its bottleneck (smallest residual capacity) along it increases the flow value and keeps all constraints. Figure 15.1 shows why backward edges are indispensable: a greedy first path can block the optimum, and only cancelling part of it recovers.

A flow is maximum exactly when its residual graph has no augmenting path; the next section proves this. The algorithms differ in *which* augmenting paths they use.

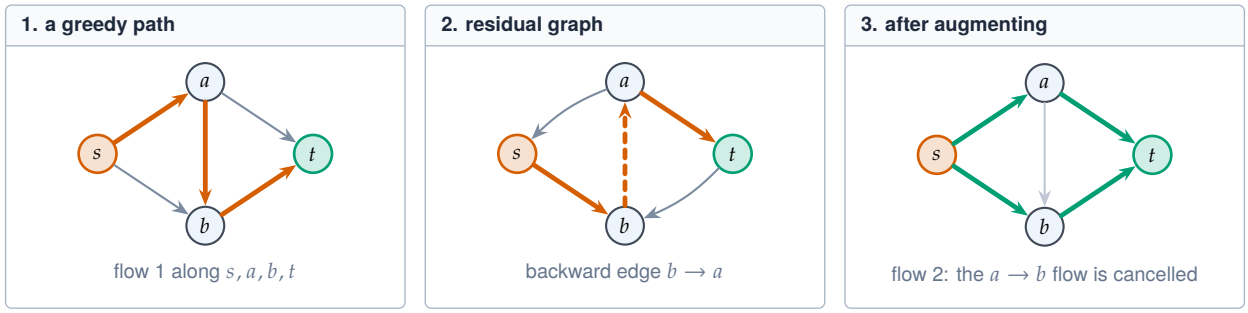
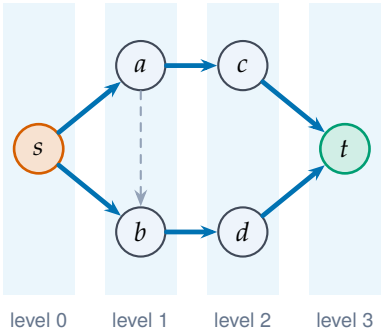


Figure 15.1: Why residual graphs have backward edges. All capacities are 1. The first augmenting path uses $a \to b$ and blocks both remaining paths; in the residual graph, the backward edge $b \to a$ (dashed) lets the second path s, b, a, t reroute the first unit, giving the maximum flow 2.

- **Ford–Fulkerson**: any augmenting path, found by DFS. With integer capacities each augmentation adds at least 1, so it runs in $O(mF)$ for a maximum flow of value F: fine when F is small, as in matching problems.
- **Edmonds–Karp**: always a *shortest* augmenting path (BFS), which bounds the number of augmentations by $O(nm)$ regardless of the capacities: $O(nm^2)$.
- **Dinic**: in each phase, BFS computes the distance *level* of every vertex, and a DFS that only follows edges from one level to the next pushes a *blocking flow*, saturating all shortest augmenting paths at once (Figure 15.2). The distance from s to t grows with every phase, so there are at most n phases. Dinic runs in $O(n^2m)$ in general, $O(m\sqrt{n})$ on unit-capacity bipartite graphs, and is extremely fast in practice; it is the default.



One phase of Dinic: BFS assigns levels, and only edges from level i to level $i+1$ (blue) belong to the level graph. The edge $a \to b$ stays inside a level and is ignored in this phase; blocking-flow DFS never looks at it.

Figure 15.2: The level graph of one phase of Dinic's algorithm.

Example: Download Speed (CSES)

Computers are joined by one-way connections with given speeds. Find the maximum speed at which data can be downloaded from computer 1 to computer n.

Solution. The maximum flow from 1 to n, with speeds as capacities.

```
void solve() {  // Dinic (cpdinic) is pasted above solve()
    int n, m;
    cin >> n >> m;
    Dinic g(n + 1);
    rep(i, m) {
        ll a, b, c;
        cin >> a >> b >> c;
```

```
        g.add_edge(a, b, c);  // a one-way connection of capacity c
    }
    print(g.max_flow(1, n));
}
```

</> Template: Max flow (Dinic) + min cut Ctrl+Alt+Y, Y prefix `cpdinic`

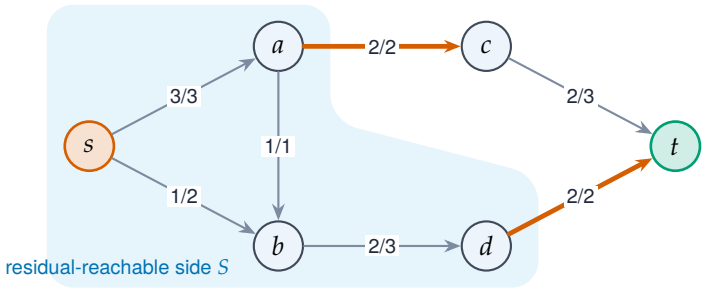
15.2 Minimum cut

IOI / ADVANCED

An s–t **cut** splits the vertices into a side S containing s and a side T containing t; its capacity is the total capacity of the edges from S to T. Every unit of flow must cross from S to T at least once, so the value of any flow is at most the capacity of any cut.

Max-flow min-cut theorem The maximum flow value equals the minimum cut capacity. Moreover, after a max-flow algorithm finishes, the vertices reachable from s in the residual graph form the S side of a minimum cut.

Proof. Let f be a flow with no augmenting path, and let S be the set of vertices reachable from s in its residual graph; $t \notin S$. Every edge from S to T is saturated (otherwise its head would be reachable), and every edge from T to S carries no flow (otherwise its backward residual edge would make its tail reachable). So the net flow across the cut is exactly its capacity, and since the flow value equals the net flow across any cut, the flow value equals this cut's capacity. As no flow exceeds any cut, both are optimal (Figure 15.3). The same argument shows that "no augmenting path" implies "maximum", which is why the augmenting-path algorithms are correct.



Labels are flow/capacity; the flow has value 4. The red edges $a \to c$ and $d \to t$ leave S, both saturated, with total capacity $2 + 2 = 4$: no flow can do better.

Figure 15.3: A maximum flow and a minimum cut of the same value. The shaded side $S = \{s, a, b, d\}$ is what is reachable from s in the residual graph.

Example: Police Chase (CSES)

Close as few two-way streets as possible so that there is no route from crossing 1 to crossing n, and print them.

Solution. With every street a capacity-1 edge in both directions, the minimum number of streets to close is the minimum cut. After the max flow, the streets with one end on each side of the residual-reachable set form a minimum cut.

```
void solve() {  // Dinic (cpdinic) is pasted above solve()
    int n, m;
    cin >> n >> m;
    Dinic g(n + 1);
```

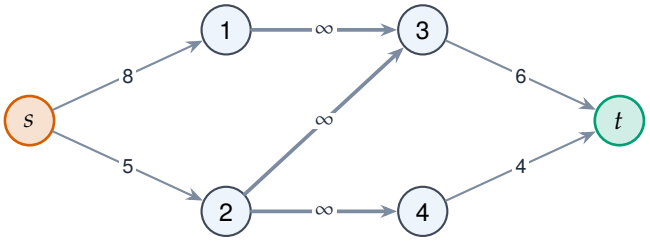
```
    vector<pii> street(m);
    for (auto& [a, b] : street) {
        cin >> a >> b;
        g.add_edge(a, b, 1, 1);  // a two-way street: capacity 1 each way
    }
    print(g.max_flow(1, n));  // the size of a minimum cut
    // the cut: streets between the source side and the sink side
    for (auto [a, b] : street)
        if (g.min_cut_side(a) != g.min_cut_side(b)) print(a, b);
}
```

15.2.1 Project selection

Min cuts shine when a problem asks to *choose* a subset under dependencies. There are n projects with profits p_i (possibly negative), and requirements "choosing a forces choosing b". Build a network with an edge $s \to i$ of capacity p_i for every profitable project, an edge $i \to t$ of capacity $-p_i$ for every costly one, and an edge $a \to b$ of infinite capacity for every requirement (Figure 15.4). A finite cut with source side S never cuts a requirement edge, so S is a valid selection, and its capacity is exactly the profit given up plus the cost paid:

$$\text{best profit} = \sum_{p_i>0} p_i - \text{min cut}.$$

This is the *maximum weight closure* problem, and a surprising number of problems are it in disguise: choosing experiments that need equipment, rooms that need keys, vertices of a graph that must include all successors.



Projects 1, 2 earn 8 and 5; projects 3, 4 cost 6 and 4. Project 1 needs 3; project 2 needs 3 and 4. Taking everything earns $13 - 10 = 3$; taking only 1 and 3 earns $8 - 6 = 2$. The minimum cut, 10, gives $13 - 10 = 3$.

Figure 15.4: Project selection as a minimum cut. Infinite edges can never be cut, so the source side is always a closed set of projects.

```
void solve() {  // Dinic (cpdinic) is pasted above solve()
    int n, m;  // n projects, m requirements "a needs b"
    cin >> n >> m;
    int s = 0, t = n + 1;
    Dinic g(n + 2);
    ll positive = 0;
    rep(i, 1, n + 1) {
        ll p;
        cin >> p;  // profit, possibly negative
        if (p > 0) g.add_edge(s, i, p), positive += p;
        else g.add_edge(i, t, -p);
    }
    rep(j, m) {
        int a, b;
        cin >> a >> b;
```

```
        g.add_edge(a, b, INF);  // choosing a forces choosing b
    }
    // best profit = all positive profits - the cheapest way to give up
    // some of them or pay for the needed costs: a minimum cut
    print(positive - g.max_flow(s, t));
}
```

How to recognize it "Remove the cheapest set of edges or vertices so that s and t are separated", "split into two groups where each item has a cost for each side and some pairs cost extra if separated", "choose a subset with profits and prerequisites". Whenever the objective is the cost of *separating* things, think of a minimum cut. Costs on vertices become edges by splitting each vertex into an in-copy and an out-copy joined by an edge of that cost.

15.3 Disjoint paths

IOI / ADVANCED

The maximum number of *edge-disjoint* paths from s to t is the maximum flow with all capacities 1: each unit of flow is one path, and conversely disjoint paths give a flow. Dually, by **Menger's theorem**, it equals the minimum number of edges whose removal separates s from t. For *vertex-disjoint* paths, split every vertex v other than s and t into v_{in} and v_{out} joined by an edge of capacity 1, redirect incoming edges to v_{in} and outgoing ones from v_{out} (Figure 15.5); now a vertex can carry at most one unit of flow.

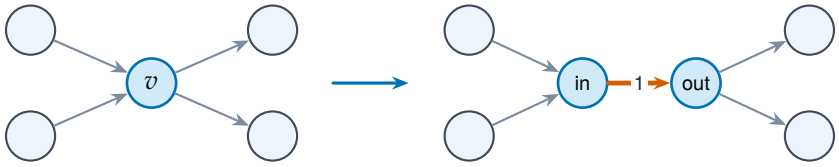


Figure 15.5: Vertex splitting. The edge $v_{\text{in}} \to v_{\text{out}}$ of capacity 1 (or of the vertex's cost) makes a vertex capacity into an edge capacity.

Example: Distinct Routes (CSES)

Find as many routes from room 1 to room n as possible, such that no one-way teleporter is used by two routes, and print them.

Solution. The number of routes is the maximum flow with unit capacities. To print them, repeatedly walk from 1 along edges that carry flow and have not been used yet: by conservation, the walk can always continue until it reaches n, and each walk uses up one unit of flow.

```
void solve() {  // Dinic (cpdinic) is pasted above solve()
    int n, m;
    cin >> n >> m;
    Dinic g(n + 1);
    vvi out(n + 1);  // edge ids leaving each room
    vi to(2 * m);
    rep(i, m) {
        int a, b;
        cin >> a >> b;
        int id = g.add_edge(a, b, 1);
```

```
        out[a].push_back(id), to[id] = b;
    }
    ll k = g.max_flow(1, n);
    print(k);
    // each unit of flow is one route: follow edges that carry flow,
    // using every edge once (a used edge is marked by clearing its flow)
    vector<bool> used(2 * m, false);
    rep(r, k) {
        vi route{1};
        int u = 1;
        while (u != n) {
            for (int id : out[u])
                if (!used[id] && g.flow_on(id) > 0) {
                    used[id] = true, u = to[id];
                    break;
                }
            route.push_back(u);
        }
        print(sz(route));
        print(route);
    }
}
```

15.4 Bipartite matching

IOI / ADVANCED

A **matching** is a set of edges no two of which share a vertex. In a bipartite graph with sides L and R, a maximum matching is a maximum flow in the network $s \to L \to R \to t$ with all capacities 1. The flow view translates into a direct algorithm: an augmenting path is now an **alternating path** that starts at an unmatched left vertex, alternates between non-matching and matching edges, and ends at an unmatched right vertex; flipping every edge on it increases the matching by one (Figure 15.6). **Berge's lemma** says that a matching is maximum exactly when no augmenting path exists.

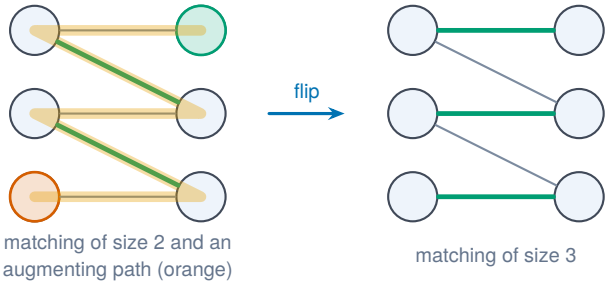


Figure 15.6: An augmenting path runs from an unmatched left vertex (red) to an unmatched right vertex (green), alternating between edges outside and inside the matching (green). Flipping it grows the matching by one.

Kuhn's algorithm looks for an augmenting path from every left vertex by DFS, in $O(nm)$ total and ten lines of code. **Hopcroft–Karp** finds a maximal set of shortest augmenting paths per phase, like Dinic, in $O(m\sqrt{n})$; use it for n, m around 10^5.

Example: School Dance (CSES)

Given which boys and girls are willing to dance together, form as many dance pairs as possible and print them.

```
void solve() {
    int n, m, k;  // n boys, m girls, k possible pairs
    cin >> n >> m >> k;
    vvi adj(n + 1);
    rep(i, k) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
    }
    // Kuhn: for every boy, look for an augmenting path by DFS
    vi girl_of(m + 1, 0), seen(m + 1, 0);
    int stamp = 0;
    auto augment = [&](auto&& self, int boy) -> bool {
        for (int g : adj[boy]) {
            if (seen[g] == stamp) continue;
            seen[g] = stamp;
            // g is free, or her partner can move to another girl
            if (!girl_of[g] || self(self, girl_of[g])) {
                girl_of[g] = boy;
                return true;
            }
        }
        return false;
    };
    int pairs = 0;
    rep(boy, 1, n + 1) {
        stamp++;
        if (augment(augment, boy)) pairs++;
    }
    print(pairs);
    rep(g, 1, m + 1) if (girl_of[g]) print(girl_of[g], g);
}
```

</> Template: Bipartite matching (Hopcroft-Karp) Ctrl+Alt+Y, Shift+H prefix `cphopcroft`

15.5 König, Hall and path covers

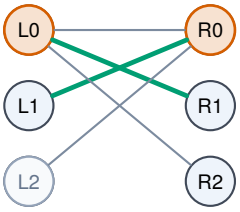
IOI / ADVANCED

Matchings are more useful for the theorems they come with than for themselves; each of these turns a hard-looking optimisation into a matching size.

- **König's theorem**: in a bipartite graph, the size of a minimum *vertex cover* (a set of vertices touching every edge) equals the size of a maximum matching.
- **Independent sets**: the complement of a vertex cover is an independent set, so a maximum independent set of a bipartite graph has $|L| + |R|$ − matching vertices.
- **Hall's theorem**: L can be matched completely if and only if every subset $X \subseteq L$ has at least $|X|$ neighbours. The obstruction is always a small neighbourhood.

- **Edge covers**: with no isolated vertices, a minimum edge cover has n − matching edges.

Constructing the cover (König). After a maximum matching, let Z be the vertices reachable from unmatched left vertices by alternating paths (non-matching edges from left to right, matching edges from right to left). Then $(L \setminus Z) \cup (R \cap Z)$ is a vertex cover of exactly the matching's size (Figure 15.7): every matching edge contributes exactly one endpoint, and no edge can go from $L \cap Z$ to $R \setminus Z$, or its right end would be reachable.



Maximum matching L0–R1, L1–R0 (green); L2 is unmatched. Alternating paths from L2 reach R0 and, through R0's partner, L1: $Z = \{L2, R0, L1\}$. The cover $(L \setminus Z) \cup (R \cap Z) = \{L0, R0\}$ (red) touches every edge and has size 2, the matching size.

Figure 15.7: König's construction of a minimum vertex cover from a maximum matching.

</> Template: Bipartite minimum vertex cover / maximum independent set (Konig) Ctrl+Alt+U, V prefix `cpkonig`

15.5.1 Path covers and Dilworth's theorem

A **path cover** of a DAG is a set of vertex-disjoint paths that together contain every vertex. Make a bipartite graph with a left copy and a right copy of every vertex and an edge from u's left copy to v's right copy for every DAG edge $u \to v$. A matching chooses, for some vertices, a successor on their path, with no two vertices choosing the same successor; every matched edge glues two paths into one. So the minimum number of paths is n − maximum matching (Figure 15.8).

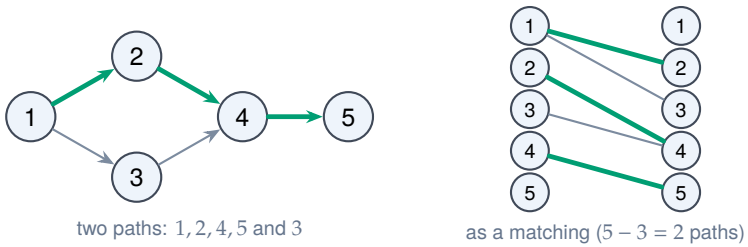


Figure 15.8: A minimum path cover of a DAG (green) and the corresponding matching between "left" (predecessor) and "right" (successor) copies.

```
void solve() {  // HopcroftKarp (cphopcroft) is pasted above solve()
    int n, m;
    cin >> n >> m;
    // left copy: v as a predecessor on a path, right copy: v as a successor
    HopcroftKarp hk(n, n);
    rep(i, m) {
        int a, b;
        cin >> a >> b;
        hk.add_edge(a - 1, b - 1);  // the edge a -> b may join two paths
    }
    // every matched edge glues two paths together: n - matching paths
    print(n - hk.max_matching());
}
```

If paths may share vertices (a *chain* cover of a partial order), first take the transitive closure, then apply the same formula. **Dilworth's theorem** states that the minimum number of chains covering a partial order equals the size of its largest *antichain* (a set of pairwise incomparable elements); for example, the minimum number of non-increasing subsequences that cover a sequence equals the length of its longest strictly increasing subsequence.

15.6 The assignment problem: the Hungarian algorithm

IOI / ADVANCED

In the **assignment problem**, n workers must be matched to n jobs, worker i doing job j costs a_{ij}, and the total cost is to be minimised. It is a minimum-cost perfect matching in a complete bipartite graph, and the **Hungarian algorithm** solves it in $O(n^3)$ with *potentials*: numbers u_i for rows and v_j for columns with $a_{ij} - u_i - v_j \geq 0$ for all cells. For any such potentials, every assignment costs at least $\sum u_i + \sum v_j$ (sum the inequalities over the assigned cells). The algorithm adjusts the potentials, one row at a time, until the assigned cells are all *tight* ($a_{ij} = u_i + v_j$); then the assignment's cost equals the lower bound, so it is optimal (Figure 15.9). This is linear programming duality in its simplest form.

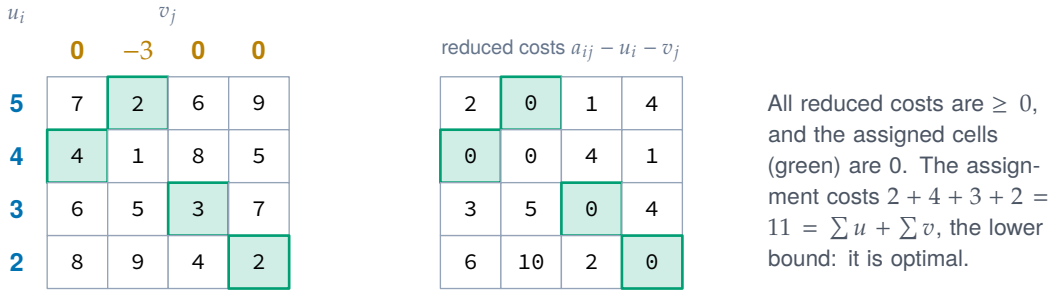


Figure 15.9: The Hungarian algorithm's certificate of optimality. Left: costs with row potentials u (blue) and column potentials v (orange); right: the reduced costs.

Example: Task Assignment (CSES)

Assign $n \leq 200$ employees to n tasks, one each, minimising the total cost, and print the assignment.

```
void solve() {  // hungarian (cphungarian) is pasted above solve()
    int n;
    cin >> n;
    vector<vector<ll>> cost(n, vector<ll>(n));
    for (auto& row : cost) cin >> row;
    auto [total, task] = hungarian(cost);  // row i gets column task[i]
    print(total);
    rep(i, n) print(i + 1, task[i] + 1);
}
```

</> Template: Hungarian algorithm (min-cost assignment) Ctrl+Alt+U, Shift+H prefix `cphungarian`

For maximum-weight assignments, negate the costs; for rectangular problems (n rows, $m \geq n$ columns), the template assigns every row a distinct column. The $O(n^3)$ bound beats a general min-cost flow on dense assignment problems by a wide margin.

15.7 Minimum-cost flow

IOI / ADVANCED

When every edge also has a cost per unit of flow, the **minimum-cost flow** problem asks for the cheapest flow of a given value (or the cheapest among maximum flows). The standard method is *successive shortest paths*: repeatedly push flow along the cheapest s–t path of the residual graph, where backward edges have the negated cost, until the required value is reached. Each augmentation keeps the flow optimal among flows of its value, because a flow is cost-optimal exactly when its residual graph has no negative cycle. Backward edges have negative costs, so the shortest paths need Bellman–Ford (the template uses SPFA), or Dijkstra with *potentials*: after each round, add the distances to the potentials, which makes every residual edge non-negative again (Johnson's reweighting, Section 12.5).

Example: Parcel Delivery (CSES)

Send k parcels from city 1 to city n; route i carries at most r_i parcels at cost c_i each. Find the minimum total cost, or report that it is impossible.

```
void solve() {  // MCMF (cpmcmf) is pasted above solve()
    int n, m, k;
    cin >> n >> m >> k;
    MCMF g(n + 1);
    rep(i, m) {
        ll a, b, r, c;
        cin >> a >> b >> r >> c;  // at most r parcels, c per parcel
        g.add_edge(a, b, r, c);
    }
    auto [flow, cost] = g.run(1, n, k);  // the cheapest way to send k
    print(flow < k ? -1 : cost);
}
```

</> Template: Min-cost max-flow Ctrl+Alt+Y, Shift+Y prefix `cpmcmf`

Min-cost flow as a modelling language Assignment with capacities (a worker can take up to c jobs), k disjoint paths of minimum total length (CSES Distinct Routes II), transporting goods between warehouses, and "choose k non-overlapping intervals with maximum weight" (a chain of time points with capacity-k edges and interval edges of negative cost) are all min-cost flows. When costs are convex in the amount (the second unit costs more than the first), model each unit as a separate parallel edge with its own cost.

15.8 Flows with lower bounds

IOI / ADVANCED

Sometimes an edge must carry *at least* $l(e)$ units. Force the lower bounds first: pretend $l(e)$ units flow along every edge, keep only the remaining capacity $c(e) - l(e)$, and note that each vertex now has an imbalance (more forced flow in than out, or the reverse). Add a new source S' with edges to every vertex with excess and a new sink T' with edges from every vertex with deficit, and an edge $t \to s$ of infinite capacity. A feasible flow exists if and only if the maximum flow from S' to T' saturates all edges out of S'; then the flow on $t \to s$ is a feasible s–t flow, and one more max flow from s to t (with the helper edges removed) maximises it. Circulations with demands, and minimum flows (the smallest feasible flow), use the same construction.

</> Template: Max flow with lower bounds Ctrl+Alt+Y, Shift+G prefix `cpflowlb`

15.9 Matching in general graphs: blossoms

IOI / ADVANCED

In a non-bipartite graph, alternating paths still characterise maximum matchings (Berge's lemma holds for every graph), but finding them is harder: an odd cycle lets an alternating path reach the same vertex with both parities, which confuses a plain search. **Edmonds' blossom algorithm** handles this. When the search finds an odd alternating cycle, a *blossom*, it contracts the whole cycle into a single vertex and continues (Figure 15.10); an augmenting path in the contracted graph lifts to one in the original graph, because an odd cycle can be entered at any vertex and traversed on the correct side. The template runs in $O(n^3)$, comfortable for $n \le 500$.

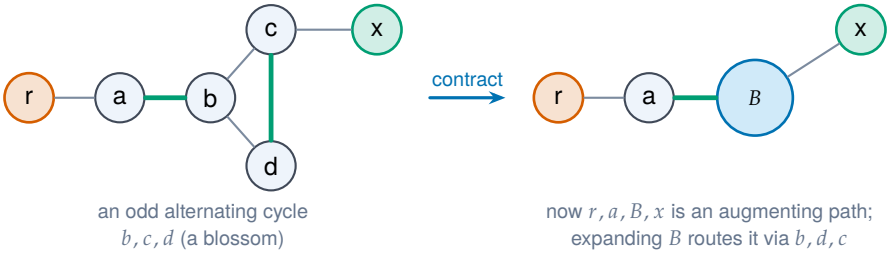


Figure 15.10: A blossom. The search from the unmatched vertex r reaches the odd cycle b, c, d; contracting it into B reveals the augmenting path to the unmatched vertex x, which in the original graph goes r, a, b, d, c, x.

Example: Maximum matching in a general graph

Given an undirected graph with $n \le 500$ vertices, find the size of a maximum matching.

```
void solve() {  // blossom_matching (cpblossom) is pasted above solve()
    int n, m;
    cin >> n >> m;
    vvi adj(n);
    rep(i, m) {
        int a, b;
        cin >> a >> b;
        adj[a - 1].push_back(b - 1);
        adj[b - 1].push_back(a - 1);
    }
    vi mate = blossom_matching(adj);  // mate[v] = partner or -1
    int pairs = 0;
    rep(v, n) if (mate[v] > v) pairs++;
    print(pairs);
}
```

</> Template: Maximum matching in a general graph (Edmonds' blossom) Ctrl+Alt+U, Shift+G prefix `cpblossom`

15.10 Global and all-pairs minimum cuts

IOI / ADVANCED

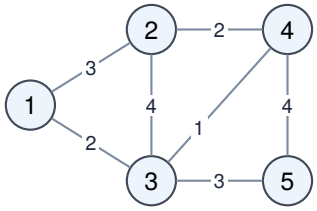
Global minimum cut. The smallest set of edges (by total weight) whose removal disconnects an undirected graph, with no fixed source or sink, could be found with $n - 1$ max flows from a fixed

vertex. The **Stoer–Wagner algorithm** does better with no flow at all. Each phase grows a set A from any vertex by repeatedly adding the vertex most tightly connected to A (the largest total edge weight into A, a *maximum adjacency order*). If s and t are the last two vertices added, the weight of t's connection to everything else is a minimum s–t cut. Either that cut is the global minimum, or s and t are on the same side of it, so they can be merged into one vertex. After $n-1$ phases the answer is the best cut seen, in $O(n^3)$ with a matrix.

```
void solve() {  // stoer_wagner (cpstoerwagner) is pasted above solve()
    int n, m;
    cin >> n >> m;
    vector<vector<ll>> w(n, vector<ll>(n, 0));
    rep(i, m) {
        ll a, b, c;
        cin >> a >> b >> c;
        w[a - 1][b - 1] += c, w[b - 1][a - 1] += c;  // parallel edges add up
    }
    print(stoer_wagner(w));
}
```

</> Template: Global minimum cut (Stoer-Wagner) Ctrl+Alt+U, Shift+A prefix `cpstoerwagner`

All-pairs minimum cuts: the Gomory–Hu tree. Remarkably, the $\binom{n}{2}$ minimum cuts of an undirected graph take only $n-1$ different values, and they are organised by a tree. A **Gomory–Hu tree** is a weighted tree on the same vertices such that the minimum cut between any u and v in the graph equals the smallest edge weight on the tree path between them (Figure 15.11). Gusfield's algorithm builds it with $n-1$ max flows: for $s = 1, \dots, n-1$, compute the minimum cut between s and its current tree parent, attach s with that weight, and re-hang every later vertex that lies on s's side of the cut under s. Afterwards every min-cut query is a path-minimum query in a tree.



an undirected capacitated graph



its Gomory–Hu tree

min cut$(2,5) = \min(8,6,7) = 6$, min cut$(1,x) = 5$ for every x: vertex 1 has total incident weight 5.

Figure 15.11: A graph and its Gomory–Hu tree, computed with four max flows. Every one of the ten pairwise minimum cuts is the smallest weight on a tree path.

```
void solve() {  // gomory_hu (cpgomoryhu) is pasted above solve()
    int n, m, q;
    cin >> n >> m >> q;
    vector<array<ll, 3>> e(m);
    for (auto& [a, b, c] : e) cin >> a >> b >> c, a--, b--;
    auto [par, w] = gomory_hu(n, e);  // a tree on the vertices
    // min cut(u, v) = the lightest edge on the tree path from u to v
    vi depth(n, 0);
    rep(v, 1, n) {  // depth by walking up; n is small in this problem
        for (int x = v; x != 0; x = par[x]) depth[v]++;
```

```
    }
    while (q--) {
        int u, v;
        cin >> u >> v;
        u--, v--;
        ll best = LLONG_MAX;
        while (u != v) {  // climb from the deeper end, taking the minimum
            if (depth[u] < depth[v]) swap(u, v);
            chmin(best, w[u]);
            u = par[u];
        }
        print(best);
    }
}
```

Template: Gomory-Hu tree (all-pairs minimum cut) Ctrl+Alt+U, A prefix `cpgomoryhu`

Chapter summary

- Residual graphs with backward edges make augmenting paths correct; Dinic's level graphs and blocking flows make them fast.
- Max flow = min cut; the residual-reachable set is a minimum cut. Project selection is a minimum cut with infinite edges.
- Edge-disjoint paths are unit flows; vertex capacities become edges by splitting vertices.
- Bipartite matching: augmenting alternating paths (Kuhn, Hopcroft–Karp); König: vertex cover = matching; DAG path cover = $n-$ matching; Dilworth for chains and antichains.
- Assignment: the Hungarian algorithm with potentials, certified by duality. Min-cost flow: successive shortest paths.
- Lower bounds reduce to a feasibility flow; general matching needs blossoms; Stoer–Wagner and Gomory–Hu trees answer global and all-pairs minimum cuts.

Practice problems (CSES Problem Set)

FLOWS AND CUTS

- Download Speed
- Police Chase
- Distinct Routes
- Distinct Routes II

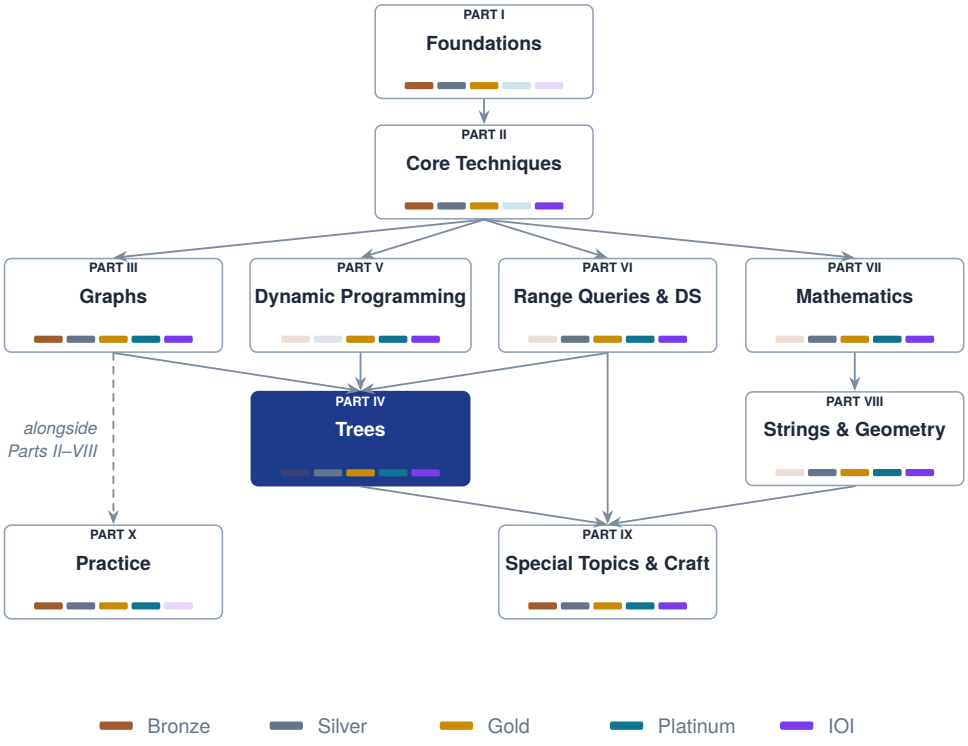
MATCHINGS AND ASSIGNMENT

- School Dance
- Task Assignment
- Parcel Delivery

PART IV

Trees

Rooted trees, the Euler tour, binary lifting, DP on trees, and the decompositions (heavy-light, centroid, virtual trees) that turn tree queries into array queries.



CHAPTER 16

Tree Techniques

A tree is a connected graph with n vertices and $n-1$ edges; equivalently, a graph in which every two vertices are joined by exactly one simple path. That uniqueness makes trees the friendliest of graphs. Recursion over subtrees is natural, every path decomposes at a lowest common ancestor, and dynamic programming works cleanly because subtrees do not overlap. This chapter collects the techniques that turn those facts into algorithms: traversal orders, diameters and farthest vertices, flattening a tree with an Euler tour, binary lifting and the many ways to compute lowest common ancestors, DP on trees for one root and for all roots at once, merging DP tables and sets efficiently, and deciding whether two trees have the same shape.

IN THIS CHAPTER SILVER GOLD PLATINUM

- root a tree and compute parents, depths, subtree sizes and orders without deep recursion;
- find diameters and the farthest vertex from every vertex;
- answer subtree and path queries by flattening the tree with an Euler tour;
- compute lowest common ancestors by binary lifting, and offline with Tarjan's algorithm;
- design tree DPs, reroot them to get every root's answer, and merge child tables in $O(n^2)$ total;
- merge sets small to large, and test trees for isomorphism with canonical labels.

Before you start: Chapter 11 (BFS and DFS) and Chapter 13 (DSU).

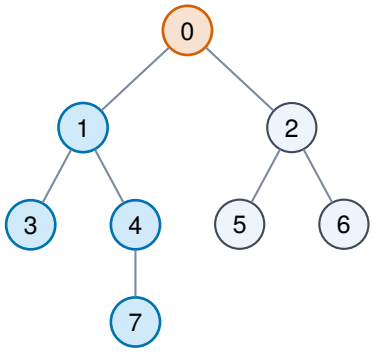
16.1 Rooted trees and traversal orders SILVER

A few counting facts are used constantly. A tree on n vertices has exactly $n-1$ edges; removing any edge splits it into two trees, and adding any edge creates exactly one cycle. The degrees sum to $2n-2$, so the average degree is below 2 and every tree with at least two vertices has at least two leaves. Most tree problems give the tree as $n-1$ undirected edges; some give a parent array instead.

Choosing a root gives every vertex a **parent** (except the root), **children**, a **depth** (the number of edges to the root) and a **subtree** (itself and all its descendants), as in Figure 16.1. Every vertex other than the root has exactly one parent, so the parent array alone describes the rooted tree, and one traversal computes all of these quantities.

16.1.1 The BFS-order idiom

Two orders of the vertices are especially useful. In a *BFS order* every parent comes before its children, so reading the order backwards processes all children before their parent: exactly what a bottom-up computation needs. Building the order needs no queue beyond the order itself, and no recursion:



root 0 has depth 0; $parent(4) = 1$;
$children(1) = \{3, 4\}$;
$subtree(1) = \{1, 3, 4, 7\}$, size 4 (blue);
$depth(7) = 3$; the leaves are 3, 7, 5, 6.

BFS order: $0, 1, 2, 3, 4, 5, 6, 7$
DFS order: $0, 1, 3, 4, 7, 2, 5, 6$

Figure 16.1: A rooted tree and its basic quantities. In the DFS order, the subtree of 1 occupies the consecutive positions 1–4.

```
vi order{1}, par(n + 1, 0), depth(n + 1, 0);
rep(i, n) for (int v : adj[order[i]])  // order grows while it is scanned
    if (v != par[order[i]]) {
        par[v] = order[i], depth[v] = depth[order[i]] + 1;
        order.push_back(v);
    }
vi sub(n + 1, 1);  // subtree sizes
per(i, 1, n) sub[par[order[i]]] += sub[order[i]];  // children first
```

Every vertex is appended once, when its parent is scanned, so this is $O(n)$. Skipping the parent is enough to avoid visiting a vertex twice, because in a tree no other edge leads back to an already visited vertex. The last loop is the model for bottom-up tree computations in this chapter: walk the order backwards and let every vertex hand its value to its parent. The complete program for subtree sizes is the worked problem Subordinates (Chapter 31).

Recursion depth and parent arrays A tree with $2 \cdot 10^5$ vertices may be a single path, and a recursive DFS on it is $2 \cdot 10^5$ calls deep. Most judges configure a large stack, but a default 1 MB stack, as on Windows, overflows somewhere between 10^4 and 10^5 frames, depending on the frame size. The BFS order and the explicit-stack DFS of Section 16.3 avoid the problem entirely. When the input is a parent array, check whether the statement guarantees $p_i < i$: if it does, the order $1, 2, \ldots, n$ already has parents before children; if it does not (as in Company Queries II below), build the children lists and a BFS order first.

16.1.2 DFS orders

A DFS visits a vertex, then each child's whole subtree in turn, so it produces three useful sequences (Figure 16.2). The **preorder** lists the vertices when they are entered; every subtree is a contiguous block that starts with its root, which is the basis of the Euler tour technique (Section 16.3). The **postorder** lists them when they are left; subtrees are again contiguous, now ending with their root, and children always precede parents. The full **Euler tour** writes a vertex down every time the walk passes through it, $2n - 1$ entries in all; between the first visits of u and v the walk climbs no higher than their lowest common ancestor and does pass through it, so the LCA is the shallowest vertex of that stretch, a range minimum query (Section 16.4).

</> Template: Tree DFS (parent, depth, subtree size, Euler tour) Ctrl+Alt+T, E prefix `cptree`

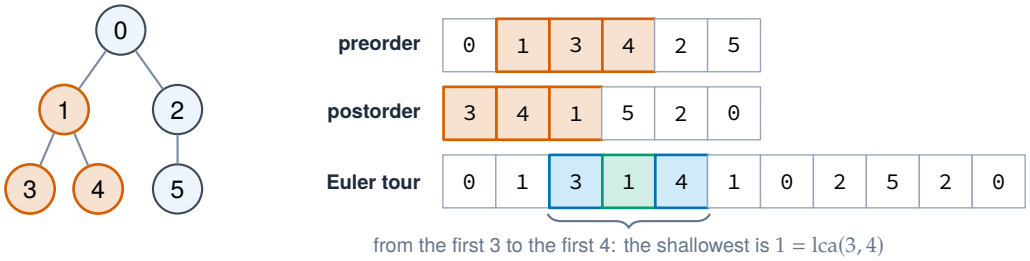


Figure 16.2: The three DFS orders of one tree. In preorder and postorder the subtree of 1 (red) is a contiguous block. In the Euler tour, the shallowest vertex between the first visits of 3 and 4 is their LCA (green).

16.2 Diameter and farthest vertices

GOLD

The **diameter** of a tree is its longest path. Two BFS passes find it: from any vertex, the farthest vertex a is an endpoint of some diameter, and the farthest vertex from a is the other endpoint b (the worked problem Tree Diameter).

Why the first BFS lands on a diameter endpoint. Let x–y be a diameter, let a be the farthest vertex from the start s, and let c be the vertex of the path x–y closest to s; every path from s into x–y enters it at c. Let w be the last vertex that the path $s \to a$ shares with the path $s \to c$.

- If $w \neq c$, the branch to a leaves before reaching c. Since a is at least as far from s as x is, $d(w,a) \geq d(w,c) + d(c,x)$, so the path $a \to w \to c \to y$ has length at least $d(c,x) + d(c,y) = d(x,y)$.
- If $w = c$, then $d(c,a) \geq d(c,x)$ and $d(c,a) \geq d(c,y)$ for the same reason. The branch to a leaves c towards at most one of the two ends; walking from a to c and on to the other end gives a path of length at least $d(x,y)$.

Either way a longest path ends at a (Figure 16.3). The argument uses only that the graph is a tree and that the edge weights are non-negative; it fails on graphs with cycles and with negative weights.

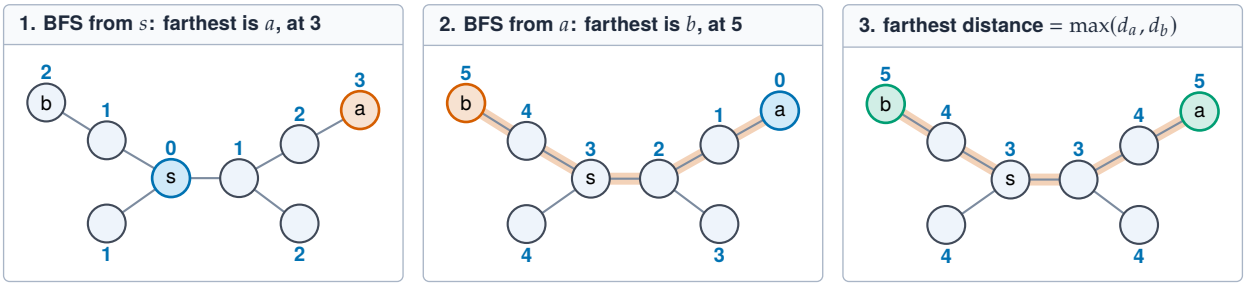


Figure 16.3: The two-BFS diameter algorithm; the numbers are BFS distances. The farthest vertex from s is a, the farthest from a is b, and a–b (red) is a diameter. Panel 3 is the corollary: the farthest vertex from any vertex is a or b.

The same exchange argument proves a stronger fact: for *every* vertex v, a farthest vertex from v is one of the two ends a, b of a fixed diameter. So after computing distances from a and from b, the farthest distance from v is $\max(d_a(v), d_b(v))$.

Example: Tree Distances I (CSES)

For every vertex of a tree, find the largest distance to another vertex.

```
void solve() {
    int n;
    cin >> n;
    vvi adj(n + 1);
    rep(i, n - 1) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    auto bfs = [&](int s) {  // distances from s
        vi d(n + 1, -1);
        vi q{s};
        d[s] = 0;
        rep(i, sz(q)) for (int v : adj[q[i]])
            if (d[v] < 0) d[v] = d[q[i]] + 1, q.push_back(v);
        return d;
    };
    // the farthest vertex from anywhere is an end of a diameter (a or b)
    vi d1 = bfs(1);
    int a = max_element(all(d1)) - d1.begin();
    vi da = bfs(a);
    int b = max_element(all(da)) - da.begin();
    vi db = bfs(b);
    vi ans(n);
    rep(v, 1, n + 1) ans[v - 1] = max(da[v], db[v]);
    print(ans);
}
```

</> Template: Tree diameter Ctrl+Alt+T, Shift+D prefix `cpdiameter`

The **center** of a tree (the middle vertex or middle edge of any diameter) minimises the largest distance to all vertices, and it is found by peeling leaves layer by layer until one or two vertices remain (used in Section 16.10). The **centroid**, a vertex whose removal leaves components of size at most $n/2$, is a different notion; it drives the decompositions of Chapter 17.

16.3 The Euler tour technique

GOLD

Number the vertices in the order a DFS enters them: $tin(v)$ is the entry time, and $tout(v)$ the time when the DFS leaves v, which is one more than the largest entry time in its subtree. Then the subtree of v is exactly the contiguous range of positions $[tin(v), tout(v))$ (Figure 16.4). This **Euler tour technique** flattens the tree into an array, and every subtree operation becomes a range operation that a Fenwick or segment tree supports. As a bonus, u is an ancestor of v exactly when $tin(u) \le tin(v) < tout(u)$, an $O(1)$ ancestor test.



Figure 16.4: The Euler tour technique. Each vertex shows its range $[tin, tout)$; listing the vertices by entry time makes every subtree a contiguous block, here positions $[1, 4)$ for the subtree of 1.

Example: Subtree Queries (CSES)

Every vertex of a rooted tree has a value. Process updates "set the value of s to x" and queries "the sum of the values in the subtree of s".

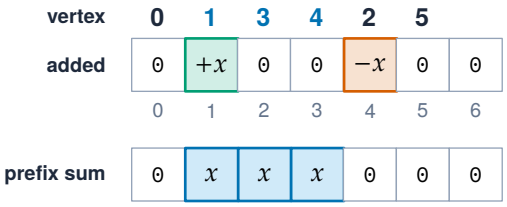
Solution. Flatten with an iterative DFS and keep the values in a Fenwick tree at position $tin(v)$. A subtree sum is the sum over positions $[tin(s), tout(s))$, and an update adds the difference at one position.

```
void solve() {  // BIT (cpbit) is pasted above solve()
    int n, q;
    cin >> n >> q;
    vi val(n + 1);
    rep(v, 1, n + 1) cin >> val[v];
    vvi adj(n + 1);
    rep(i, n - 1) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    // Euler tour: the subtree of v is the position range [tin[v], tout[v])
    vi tin(n + 1), tout(n + 1), st{1}, par(n + 1, 0), it(n + 1, 0);
    int timer = 0;
    tin[1] = timer++;
    while (!st.empty()) {  // iterative DFS
        int u = st.back();
        if (it[u] < sz(adj[u])) {
            int v = adj[u][it[u]++];
            if (v == par[u]) continue;
            par[v] = u, tin[v] = timer++;
            st.push_back(v);
        } else {
            tout[u] = timer;
            st.pop_back();
        }
    }
    BIT bit(n);
    rep(v, 1, n + 1) bit.add(tin[v], val[v]);
    while (q--) {
        int type, s;
        cin >> type >> s;
```

```
        if (type == 1) {
            ll x;
            cin >> x;
            bit.add(tin[s], x - val[s]), val[s] = x;
        } else print(bit.sum(tin[s], tout[s]));  // the positions of s's subtree
    }
}
```

16.3.1 Paths to the root

The same flattening handles sums along the path from the root to a vertex, with one twist. The value of v contributes to the path sum of every vertex in v's subtree. So add it at position $tin(v)$ and subtract it at $tout(v)$: the prefix sum up to $tin(u)$ then contains v's value exactly when u lies in v's subtree, that is, when v is on the path from the root to u (Figure 16.5). A path between any u and w follows by inclusion–exclusion: $path(u) + path(w) - 2\,path(\ell) + val(\ell)$ with $\ell = \mathrm{lca}(u, w)$.



Vertex 1 has value x. It is added at $tin(1) = 1$ and removed at $tout(1) = 4$, so the prefix sums at positions 1, 2 and 3 (the vertices 1, 3, 4 of its subtree) include it, and no other prefix sum does.

Figure 16.5: Root-path sums with an Euler tour, for the value x of vertex 1 alone in the tree of Figure 16.4. A value is added where its subtree's range starts and removed where it ends.

Example: Path Queries (CSES)

Every vertex has a value. Process updates of single values and queries for the sum of the values on the path from the root to a vertex.

```
void solve() {  // BIT (cpbit) is pasted above solve()
    int n, q;
    cin >> n >> q;
    vi val(n + 1);
    rep(v, 1, n + 1) cin >> val[v];
    vvi adj(n + 1);
    rep(i, n - 1) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    vi tin(n + 1), tout(n + 1), st{1}, par(n + 1, 0), it(n + 1, 0);
    int timer = 0;
    tin[1] = timer++;
    while (!st.empty()) {
        int u = st.back();
        if (it[u] < sz(adj[u])) {
            int v = adj[u][it[u]++];
            if (v == par[u]) continue;
```

```
            par[v] = u, tin[v] = timer++;
            st.push_back(v);
        } else {
            tout[u] = timer;
            st.pop_back();
        }
    }
    // a value at v counts for every vertex in v's subtree: add it at tin[v]
    // and remove it at tout[v]; a prefix sum then gives the root path sum
    BIT bit(n + 1);
    auto change = [&](int v, ll d) {
        bit.add(tin[v], d), bit.add(tout[v], -d);
    };
    rep(v, 1, n + 1) change(v, val[v]);
    while (q--) {
        int type, s;
        cin >> type >> s;
        if (type == 1) {
            ll x;
            cin >> x;
            change(s, x - val[s]), val[s] = x;
        } else print(bit.sum(0, tin[s] + 1));  // positions 0..tin[s]
    }
}
```

How to recognize it "Update a vertex, query a subtree", "update a subtree, query a vertex" (a range add and a point query), "path from the root" sums, and ancestor tests. When both endpoints of a path vary and values change, heavy-light decomposition takes over (Chapter 17).

16.4 Binary lifting and the lowest common ancestor

GOLD

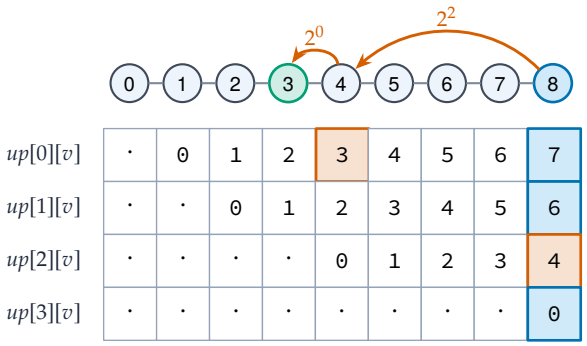
The **lowest common ancestor** $\text{lca}(u, v)$ is the deepest vertex that is an ancestor of both. The path between u and v climbs from u to the LCA and descends to v, so $\text{dist}(u, v) = \mathit{depth}(u) + \mathit{depth}(v) - 2\,\mathit{depth}(\text{lca})$, and many path questions reduce to it.

Binary lifting stores $up[j][v]$, the ancestor 2^j levels above v, computed level by level with $up[j][v] = up[j-1][\,up[j-1][v]\,]$ in $O(n \log n)$. A jump of k levels is then a sum of powers of two: one lookup per set bit of k (Figure 16.6).

Example: Company Queries I (CSES)

Given the boss of every employee, answer queries: who is the k-th boss above employee x (or -1 if there is none)?

```
void solve() {
    int n, q;
    cin >> n >> q;
    const int LOG = 18;  // 2^18 > 2 * 10^5
    // up[j][v] = the boss 2^j levels above v, 0 if there is none
```



Row j holds the ancestor 2^j levels up (a dot: none), and is built from row $j-1$, e.g. $up[2][8] = up[1][up[1][8]] = up[1][6] = 4$. Column 8 (blue) is everything vertex 8 stores. Climbing $k = 5 = 4 + 1$ levels takes one lookup per set bit (red): $up[2][8] = 4$, then $up[0][4] = 3$.

Figure 16.6: Binary lifting on a path of 9 vertices rooted at 0. With $\lceil \log_2 n \rceil$ rows in the table, any ancestor is at most that many lookups away.

```
    vector<vi> up(LOG, vi(n + 1, 0));
    rep(v, 2, n + 1) cin >> up[0][v];
    rep(j, 1, LOG) rep(v, 1, n + 1) up[j][v] = up[j - 1][up[j - 1][v]];
    while (q--) {
        int x, k;
        cin >> x >> k;
        // k is a sum of powers of two: one jump per set bit
        for (int j = 0; j < LOG && x; j++)
            if (k >> j & 1) x = up[j][x];
        print(x ? x : -1);
    }
}
```

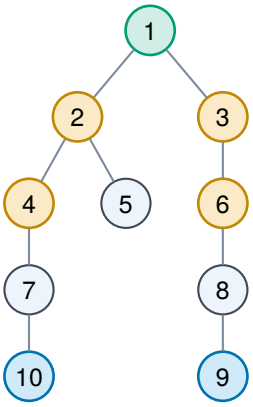
The table has $\lceil \log_2 n \rceil$ rows of $n+1$ entries, $18 \times 2 \cdot 10^5$ numbers here; with `#define int long long` that is about 29 MB, fine for a 256 MB limit but worth a thought at $n = 10^6$, where an `int32_t` table halves it. Keeping the level j as the *outer* index makes the construction loop scan memory sequentially. Nothing in the construction needs a tree: the same table answers "where am I after k steps" in any *functional graph*, where every vertex has one successor (Planets Queries I, Chapter 31).

</> Template: K-th successor (binary lifting) Ctrl+Alt+T, . prefix `cpjump`

For the LCA, first lift the deeper vertex to the depth of the other. If they now coincide, that vertex is the LCA. Otherwise, for j from the largest down, jump both vertices by 2^j whenever that keeps them *different*; this is binary search by jumps (Section 8.2) on the predicate "the ancestors at this height differ". At the end they are the two children of the LCA just below it, and the LCA is their common parent (Figure 16.7).

Algorithm: LCA by binary lifting

1 **if** $depth(a) < depth(b)$ **then**
2 swap a and b
3 **for** $j \leftarrow 0$ **to** $L-1$ **do** ▷ *climb* $depth(a) - depth(b)$ *levels*
4 **if** bit j of $depth(a) - depth(b)$ is set **then**
5 $a \leftarrow up[j][a]$
6 **if** $a = b$ **then**
7 **return** a
8 **for** $j \leftarrow L-1$ **down to** 0 **do**
9 **if** $up[j][a] \neq up[j][b]$ **then**



lca(10, 9), both at depth 4, so no levelling is needed (blue).

$j = 2$: jumping 4 levels reaches 1 from both: equal, skip.
$j = 1$: jumping 2 levels gives 4 and 6: different, take it.
$j = 0$: jumping 1 level gives 2 and 3: different, take it.

The two vertices (orange) are now children of the LCA, so $\text{lca}(10, 9) = \textit{parent}(2) = 1$ (green).

Figure 16.7: Computing an LCA by binary lifting: bring both vertices to the same depth, then take the largest jumps that keep them apart.

10 $a \leftarrow up[j][a]$; $b \leftarrow up[j][b]$
11 **return** $up[0][a]$

Why it is correct. After levelling, let h be the number of levels between a (or b) and their LCA. The ancestors of a and b that are t levels up are different for $t < h$ and equal for $t \geq h$, a monotone predicate in t. The downward loop builds the largest t with different ancestors, $t = h - 1$, bit by bit from the top, exactly like binary search by jumps; one more step up reaches the LCA. For the root, $up[j]$ points at the root itself, so an overshooting jump lands on equal vertices and is simply not taken. Each query costs $2\lceil \log_2 n \rceil$ lookups.

Example: Company Queries II (CSES)

Answer queries: the lowest common boss of employees a and b.

```
void solve() {
    int n, q;
    cin >> n >> q;
    const int LOG = 18;
    vector<vi> up(LOG, vi(n + 1, 1));  // above the root: the root itself
    vvi kids(n + 1);
    rep(v, 2, n + 1) cin >> up[0][v], kids[up[0][v]].push_back(v);
    vi depth(n + 1, 0), order{1};  // depths in BFS order from the root
    rep(i, n) for (int c : kids[order[i]]) {
        depth[c] = depth[order[i]] + 1;
        order.push_back(c);
    }
    rep(j, 1, LOG) rep(v, 1, n + 1) up[j][v] = up[j - 1][up[j - 1][v]];
    auto lca = [&](int a, int b) {
        if (depth[a] < depth[b]) swap(a, b);
        int diff = depth[a] - depth[b];
        rep(j, LOG) if (diff >> j & 1) a = up[j][a];  // same depth now
        if (a == b) return a;
        per(j, LOG) if (up[j][a] != up[j][b]) a = up[j][a], b = up[j][b];
        return up[0][a];  // largest jumps that keep them apart, then one more
    };
    while (q--) {
```

```
        int a, b;
        cin >> a >> b;
        print(lca(a, b));
    }
}
```

Template: LCA (binary lifting) Ctrl+Alt+T, L prefix `cplca`

Other ways to get an LCA

- **Euler tour + sparse table**: write down a vertex every time the DFS passes through it (length $2n-1$); the LCA of u and v is the shallowest vertex between their first occurrences, a range minimum: $O(n \log n)$ preprocessing and $O(1)$ per query (Chapter 21).
- **Heavy-light decomposition**: $O(\log n)$ per query with $O(n)$ memory and a tiny constant (Chapter 17).
- **Offline**: Tarjan's algorithm, next.
- **Path aggregates**: store the maximum (or sum, or minimum) of the 2^j edges above v next to $up[j][v]$, and combine them during the jumps: maximum edge on a path in $O(\log n)$.

Jump pointers with one pointer per vertex When vertices arrive one at a time as new leaves, and memory is tight, a single extra pointer per vertex still gives $O(\log n)$ ancestor queries. Each vertex stores its parent and one *jump*, set when the vertex is added:

```
void add_leaf(int v, int p) {  // the root has par = jump = itself
    par[v] = p, depth[v] = depth[p] + 1;
    int j = jump[p];
    jump[v] = depth[p] - depth[j] == depth[j] - depth[jump[j]] ? jump[j] : p;
}
int ancestor_at_depth(int v, int d) {  // requires d <= depth[v]
    while (depth[v] > d) v = depth[jump[v]] >= d ? jump[v] : par[v];
    return v;
}
```

The jump lengths follow the skew-binary number system, so any climb takes at most about $2\log_2 n$ steps (41 on a path of 10^6 vertices, measured). For the LCA, level the two vertices, then move both by their jumps when the jumps differ and by their parents otherwise; at equal depths the jumps have equal lengths.

Template: Jump pointers (one pointer per vertex) Ctrl+Alt+U, J prefix `cpjumpptr`

16.5 Offline LCA: Tarjan's algorithm PLATINUM

When all queries are known in advance, one DFS with a DSU answers them in nearly linear time. When the DFS *finishes* a vertex, it merges that vertex's set into its parent's set, and the parent becomes the *anchor* of the merged set. At the moment the DFS finishes u, the finished vertices are grouped into sets, one for every open ancestor of u, each containing that ancestor's finished part of the tree (Figure 16.8). So for a query (u, w) in which w is already finished, the anchor of w's set is

exactly the deepest open ancestor of w, and that is $\mathrm{lca}(u, w)$. Every query is answered when its later endpoint finishes.

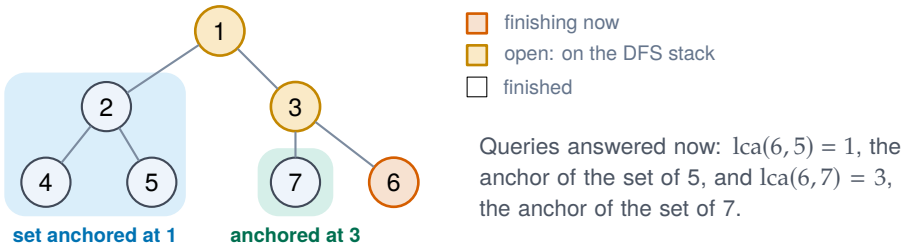


Figure 16.8: Tarjan's offline LCA in the middle of the DFS. Every finished vertex belongs to a set (shaded) that hangs from an open ancestor, its anchor; for a finished w, the anchor of w's set is its LCA with the vertex being finished.

Example: Distance Queries (CSES)

Answer queries for the distance between two vertices of a tree.

```
void solve() {  // DSU (cpdsu) is pasted above solve()
    int n, q;
    cin >> n >> q;
    vvi adj(n + 1);
    rep(i, n - 1) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    vector<vector<pii>> ask(n + 1);  // (other vertex, query index)
    rep(i, q) {
        int a, b;
        cin >> a >> b;
        ask[a].push_back({b, i});
        ask[b].push_back({a, i});
    }
    // Tarjan's offline LCA: a finished vertex joins its parent's set, whose
    // anchor is the parent. For a query (u, w) answered when u finishes and
    // w is already finished, the anchor of w's set is lca(u, w).
    DSU d(n + 1);
    vi anchor(n + 1), depth(n + 1, 0), par(n + 1, 0), it(n + 1, 0), ans(q);
    iota(all(anchor), 0);
    vector<bool> done(n + 1, false);
    vi st{1};
    while (!st.empty()) {  // iterative DFS
        int u = st.back();
        if (it[u] < sz(adj[u])) {
            int v = adj[u][it[u]++];
            if (v == par[u]) continue;
            par[v] = u, depth[v] = depth[u] + 1;
            st.push_back(v);
            continue;
        }
```

```
        done[u] = true;
        for (auto [w, i] : ask[u]) {
            if (!done[w]) continue;
            int l = anchor[d.find(w)];  // lca(u, w)
            ans[i] = depth[u] + depth[w] - 2 * depth[l];
        }
        st.pop_back();
        if (par[u]) d.unite(u, par[u]), anchor[d.find(u)] = par[u];
    }
    for (ll a : ans) print(a);
}
```

</> Template: Offline LCA (Tarjan) Ctrl+Alt+U, Shift+L prefix `cpofflinelca`

Complexity and when to use it. Each vertex is pushed and popped once, each query is looked at twice (once from each endpoint; only the second look answers it), and the DSU operations cost $O(\alpha(n))$ each: $O((n+q)\,\alpha(n))$ in total, with arrays only. That makes it the fastest LCA method in practice when all queries are known in advance, and it needs $O(n+q)$ memory instead of $O(n \log n)$. A query (u, u) is answered with lca $= u$, since u's own set is anchored at u when it finishes. When queries arrive online, or depend on earlier answers, use binary lifting or the Euler tour with a sparse table instead.

16.6 DP on trees

GOLD

Most optimisation and counting problems on trees are solved by computing a small table for every subtree from its children's tables, children first. The state is almost always "the vertex, plus a little information about the vertex itself" (chosen or not, matched or not, its colour), and the transition combines the children independently, because their subtrees do not interact except through the parent.

A first example: maximum-weight independent set. Choose vertices of maximum total weight so that no two chosen vertices are adjacent. For each vertex keep two numbers: $in(v)$, the best total in v's subtree when v is chosen, and $out(v)$, the best when it is not. A chosen vertex forces its children out; an unchosen vertex leaves each child free:

$$in(v) = w(v) + \sum_c out(c), \qquad out(v) = \sum_c \max\big(in(c), out(c)\big),$$

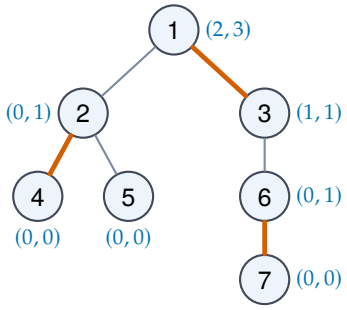
summing over the children c of v. The answer is $\max(in(r), out(r))$ at the root r. Every vertex and every edge is handled once, so the DP is $O(n)$. Counting versions have the same shape with products in place of sums: the number of independent sets satisfies $in(v) = \prod_c out(c)$ and $out(v) = \prod_c (in(c) + out(c))$, because the children's choices are independent.

The recipe. Root the tree anywhere. Ask what a parent needs to know about a child's subtree to extend a solution (is the child chosen, matched, coloured red?) and make that the state. Write the step that folds one child into its parent; it must not depend on the order of the children. Process the vertices children first, in reverse BFS order, and read the answer at the root.

Example: Tree Matching (CSES)

Find a maximum matching in a tree: as many edges as possible, no two sharing a vertex.

Solution. For every vertex v keep two values: $free(v)$, the best matching in v's subtree when v itself stays unmatched, and $best(v)$, the best when v may be matched. Then $free(v) = \sum_c best(c)$ over the children, and matching v with a child c changes the total by $free(c) + 1 - best(c)$; the best such gain gives $best(v)$ (Figure 16.9). A greedy algorithm, bottom-up, that matches each vertex with its parent whenever both are still free, is also optimal, by an exchange argument.



Each vertex shows $(free, best)$. At 3: $free(3) = best(6) = 1$, and matching 3 with 6 would gain $free(6) + 1 - best(6) = 0$, so $best(3) = 1$. At the root: $free(1) = 1 + 1 = 2$, and matching 1 with 3 gains $1 + 1 - 1 = 1$, so the maximum matching has 3 edges (red).

Figure 16.9: The tree-matching DP on a small tree, and a maximum matching read back from it.

```
void solve() {
    int n;
    cin >> n;
    vvi adj(n + 1);
    rep(i, n - 1) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    vi order{1}, par(n + 1, 0);
    rep(i, n) for (int v : adj[order[i]])
        if (v != par[order[i]]) par[v] = order[i], order.push_back(v);
    // free[v]: best matching in v's subtree with v left unmatched;
    // best[v]: best matching in v's subtree with v allowed to be matched
    vi freeV(n + 1, 0), best(n + 1, 0);
    per(i, n) {  // children before parents
        int v = order[i];
        ll gain = 0;  // the most we gain by matching v with one child
        for (int c : adj[v]) {
            if (c == par[v]) continue;
            freeV[v] += best[c];
            chmax(gain, freeV[c] + 1 - best[c]);
        }
        best[v] = freeV[v] + gain;
    }
    print(best[1]);
}
```

</> Template: Tree DP (subtree DP, iterative) Ctrl+Alt+T, ; prefix `cptreedp`

How to recognize it An optimum or a count over choices of vertices or edges of a tree: matchings, independent sets, dominating sets, colourings, "choose k vertices", "count subtrees with a property". Write the state for a vertex, decide what a child must tell its parent, and process vertices in reverse BFS order to avoid deep recursion.

16.7 DP for every root: rerooting

GOLD

Some problems want the answer for *every* choice of root, such as the sum of distances from each vertex to all others. Running the DP from every root costs $O(n^2)$. **Rerooting** computes all answers in two passes. The first pass roots the tree at one vertex and computes subtree values. The second pass moves the root across each edge, from a parent p to its child v, and updates the answer in $O(1)$ from p's answer: only the edge p–v changes sides (Figure 16.10).

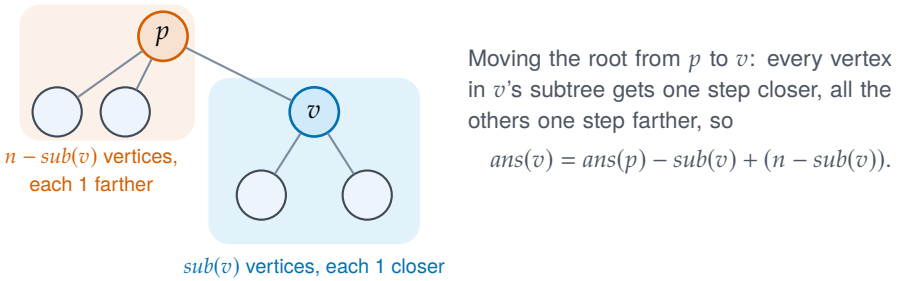


Figure 16.10: Rerooting the sum of distances across one edge.

Example: Tree Distances II (CSES)

For every vertex, find the sum of its distances to all other vertices.

```
void solve() {
    int n;
    cin >> n;
    vvi adj(n + 1);
    rep(i, n - 1) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    // BFS order from vertex 1: parents always come before children
    vi order{1}, par(n + 1, 0), depth(n + 1, 0);
    rep(i, n) for (int v : adj[order[i]]) {
        if (v == par[order[i]]) continue;
        par[v] = order[i], depth[v] = depth[order[i]] + 1;
        order.push_back(v);
    }
    vi sub(n + 1, 1);  // subtree sizes, children first
    per(i, 1, n) sub[par[order[i]]] += sub[order[i]];
    vi ans(n + 1, 0);
    rep(v, 1, n + 1) ans[1] += depth[v];  // the sum of distances from 1
    // moving the root from p to its child v: sub[v] vertices get 1 closer,
```

```
    // the other n - sub[v] get 1 farther
    rep(i, 1, n) {
        int v = order[i];
        ans[v] = ans[par[v]] - sub[v] + (n - sub[v]);
    }
    print(vi(ans.begin() + 1, ans.end()));
}
```

</> Template: Rerooting (DP for every root) Ctrl+Alt+T, ' prefix `cpreroot`

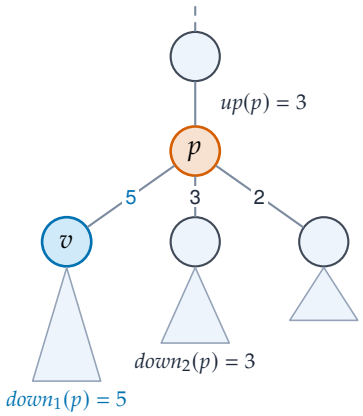
16.7.1 When the combine cannot be undone

The update above subtracts v's contribution from p's answer. That works for sums but not for maxima: knowing that the longest path from p has length 5 says nothing about the longest one once the branch into v is removed. The standard fix is to keep the *two* best values over the children, so that the best value avoiding any single child is always at hand (Figure 16.11).

For the farthest distance from every vertex (Tree Distances I again, now without the diameter argument), let $down_1(v) \geq down_2(v)$ be the two longest downward paths from v that start with different children, and let $up(v)$ be the longest path from v whose first step goes to its parent. For a child v of p,

$$up(v) = 1 + \max\big(up(p),\ \text{the best downward path from } p \text{ that does not start with } v\big),$$

where the second term is $down_2(p)$ if $down_1(p)$ goes through v, and $down_1(p)$ otherwise. The farthest distance from v is $\max(down_1(v), up(v))$: one pass children first for the downward values, one pass parents first for the upward values.



The numbers on the edges are the longest downward paths from p through each child. The best one, 5, goes through v, so for v the best path from p that avoids v is $\max(up(p), down_2(p)) = 3$ and $up(v) = 4$. For every other child it is $\max(up(p), down_1(p)) = 5$, giving $up = 6$.

Figure 16.11: Rerooting with a maximum: keeping the two best child values answers "the best value without this child" in $O(1)$.

```
void solve() {
    int n;
    cin >> n;
    vvi adj(n + 1);
    rep(i, n - 1) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
```

```
        adj[b].push_back(a);
    }
    vi order{1}, par(n + 1, 0);
    rep(i, n) for (int v : adj[order[i]])
        if (v != par[order[i]]) par[v] = order[i], order.push_back(v);
    // down1[v] >= down2[v]: the two longest downward paths from v that start
    // with different children; up[v]: the longest path from v whose first
    // step goes to v's parent
    vi down1(n + 1, 0), down2(n + 1, 0), up(n + 1, 0);
    per(i, 1, n) {  // children first
        int v = order[i], p = par[v], len = down1[v] + 1;
        if (len > down1[p]) down2[p] = down1[p], down1[p] = len;
        else chmax(down2[p], len);
    }
    rep(i, 1, n) {  // parents first
        int v = order[i], p = par[v];
        // the longest path from p that does not go down into v
        int other = down1[p] == down1[v] + 1 ? down2[p] : down1[p];
        up[v] = 1 + max(up[p], other);
    }
    vi ans(n);
    rep(v, 1, n + 1) ans[v - 1] = max(down1[v], up[v]);
    print(ans);
}
```

When two children tie for the best value, $down_2(p) = down_1(p)$, so the test `down1[p] == down1[v] + 1` picks the right value either way.

Sum over all pairs: count what each edge separates The sum of the distances over all pairs of vertices needs no rerooting at all. The edge from v to its parent lies on the path between two vertices exactly when one of them is in v's subtree and the other is not, so it contributes $sub(v)\,(n - sub(v))$, and the total is $\sum_{v \neq r} sub(v)\,(n - sub(v))$, computed in $O(n)$. The same "count the pairs an edge separates" argument gives expected distances between random vertices, and, with the edges sorted by weight and a DSU, the sum over all pairs of the heaviest edge on their path: an edge that joins components of sizes a and b is the heaviest edge for exactly $a \cdot b$ pairs (Section 2.1).

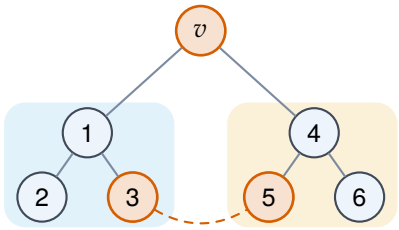
Designing a rerooting DP Write the answer for a fixed root as "merge, over the children, of *lift*(child's value)". If *merge* is associative with an identity, the generic template computes it for every root: the value of the "rest of the tree" seen from a child is the merge of the parent's upward value and the siblings' values, which prefix and suffix merges provide without any inverse. For the farthest vertex, *merge* = max and $lift(x) = x + 1$; for sums of distances, carry (sum, count) pairs. Only invertible merges, like sums, allow the "subtract the child" shortcut used above.

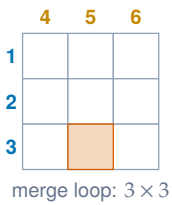
16.8 Merging child tables: the tree knapsack

PLATINUM

When the DP state contains a *count* ("choose exactly j vertices in the subtree"), merging a child into its parent is a knapsack-style convolution, $new[a + b] = \max(old[a] + child[b])$. Done over full-size arrays this looks like $O(n^3)$. But if each table is only as long as its subtree, the total work is $O(n^2)$.

Why. Merging two tables of sizes s_1 and s_2 costs $s_1 s_2$, one unit for every pair of vertices with one vertex in each part. Every pair of vertices is merged in this way exactly once, at their LCA, when their two branches are combined (Figure 16.12). There are $\binom{n}{2}$ pairs, hence $O(n^2)$. If the tables are capped at k entries, the same counting gives $O(nk)$.





Merging the blue table into the orange one is a double loop over their sizes: one cell per pair of vertices, one from each side. The pair $(3, 5)$ is counted in this merge only, at its LCA v; over the whole run every pair is counted once.

Figure 16.12: Why merging subtree tables costs $O(n^2)$ in total: the work of all merges together is at most one unit per pair of vertices.

Example: Choosing k connected vertices

Every vertex of a tree rooted at 1 has a weight (possibly negative). Choose exactly k vertices such that the parent of every chosen vertex other than the root is also chosen, with maximum total weight.

Solution. $f_v[j]$ is the best weight of j chosen vertices in v's subtree forming such a closed set (so v is chosen when $j \geq 1$). Merge the children one by one with the knapsack convolution; a child may contribute only if v itself is chosen.

```
void solve() {
    int n, k;
    cin >> n >> k;
    vi w(n + 1), par(n + 1, 0);
    rep(v, 1, n + 1) cin >> w[v];
    vvi child(n + 1);
    rep(v, 2, n + 1) {
        cin >> par[v];
        child[par[v]].push_back(v);
    }
    // f[v][j]: best weight choosing j vertices of v's subtree, closed under
    // parents (so v itself is chosen whenever j >= 1); arrays grow with size
    const ll NEG = LLONG_MIN / 4;
    vector<vi> f(n + 1);
    auto dfs = [&](auto&& self, int v) -> void {
        f[v] = {0, w[v]};
        for (int c : child[v]) {
            self(self, c);
            vi merged(min<int>(k, sz(f[v]) + sz(f[c]) - 2) + 1, NEG);
            rep(a, sz(f[v])) rep(b, sz(f[c])) {  // every pair of sizes once
                if (a + b >= sz(merged) || (a == 0 && b > 0)) continue;
                if (f[v][a] == NEG || f[c][b] == NEG) continue;
                chmax(merged[a + b], f[v][a] + f[c][b]);
            }
            f[v] = merged;
            f[c].clear();
```

```
        }
    };
    dfs(dfs, 1);
    print(k < sz(f[1]) ? f[1][k] : NEG);
}
```

</> Template: Tree knapsack (merging child DPs) Ctrl+Alt+Y, Shift+K prefix `cptreeknap`

Two details of the code matter. The merged table is only as long as the two tables together (capped at $k+1$ entries), which is what the counting argument needs; allocating $k+1$ entries at every vertex would make every merge cost $\Theta(k^2)$, and $\Theta(nk^2)$ in total. And a child's part may be non-empty only if v itself is chosen ($a \geq 1$), which is how "closed under parents" enters the merge.

Why the cap gives $O(nk)$. Call a part *small* if it has fewer than k vertices and *big* otherwise. A merge of a small part into a big one costs at most k times the small part's size, and each vertex is on the small side of such a merge at most once, since afterwards it belongs to a big part: $O(nk)$ in total. Merges of two big parts cost $O(k^2)$ each, and there are fewer than n/k of them, because each one reduces the number of disjoint big parts: $O(nk)$ again. Merges of two small parts are paid one unit per pair, and the pairs involved lie inside disjoint regions of fewer than $2k$ vertices each, so they total at most $\sum 2k \cdot s_i = O(nk)$.

16.9 Small-to-large merging

PLATINUM

To compute, for every vertex, something about the *set* of values in its subtree (the number of distinct colours, the most frequent value), give each vertex a set and merge the children's sets into the parent, always inserting the elements of the *smaller* set into the *larger* one. An element is moved only when the set it belongs to at least doubles in size, so each element moves $O(\log n)$ times: $O(n \log n)$ insertions in total, and $O(n \log^2 n)$ time with `std::set`. Swapping two `sets` is $O(1)$, which is what makes "keep the larger set" free (Figure 16.13).

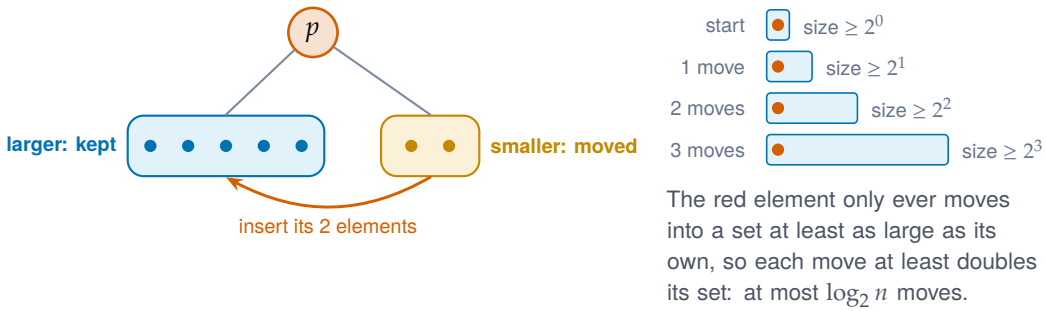


Figure 16.13: Small-to-large merging. Left: the parent keeps the larger child set (an $O(1)$ swap) and inserts the elements of the smaller one. Right: why each element moves $O(\log n)$ times.

Example: Distinct Colors (CSES)

For every vertex of a tree, count the distinct colours in its subtree.

```
void solve() {
    int n;
    cin >> n;
    vi color(n + 1);
    rep(v, 1, n + 1) cin >> color[v];
    vvi adj(n + 1);
    rep(i, n - 1) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    vi order{1}, par(n + 1, 0);
    rep(i, n) for (int v : adj[order[i]])
        if (v != par[order[i]]) par[v] = order[i], order.push_back(v);
    vector<set<ll>> s(n + 1);
    vi ans(n + 1);
    per(i, n) {  // children first; then give the parent the bigger set
        int v = order[i];
        s[v].insert(color[v]);
        ans[v] = sz(s[v]);
        int p = par[v];
        if (!p) continue;
        if (sz(s[p]) < sz(s[v])) swap(s[p], s[v]);  // O(1) swap of sets
        for (ll x : s[v]) s[p].insert(x);  // insert the smaller into the larger
        s[v].clear();
    }
    print(vi(ans.begin() + 1, ans.end()));
}
```

</> Template: Small-to-large merging Ctrl+Alt+Y, M prefix `cpsmall2large`

The same pattern handles richer questions. For "the most frequent colour in every subtree", merge `map<colour, count>` tables small to large and keep, next to each table, its largest count and the sum of the colours that reach it; an insertion updates both in $O(1)$ after the map lookup. For queries "how many vertices of v's subtree lie at depth d", merge `map<depth, count>`. The one rule is that the parent must *adopt* the larger container, by `swap` or by keeping an index to it, and never copy it: copying the larger side costs its full size and destroys the bound.

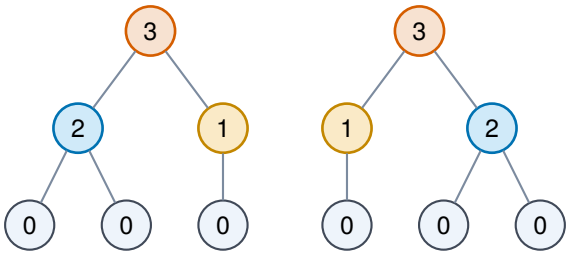
> **DSU on tree (Sack)** The same $O(n \log n)$ bound holds with plain arrays instead of sets: process the heavy child (the one with the largest subtree) last and keep its counts; then add the vertices of the light subtrees one by one, answer, and, if the current vertex is itself a light child, erase everything on the way back. Each vertex is re-added once for every light edge above it, which is $O(\log n)$ times (Chapter 17). It is faster and lighter on memory than merging sets, at the cost of more code.

16.10 Tree isomorphism and hashing

PLATINUM

Two rooted trees are *isomorphic* if one can be relabelled into the other while keeping the root, that is, if they have the same shape up to reordering children. The **AHU algorithm** decides it with canonical labels: process the vertices children first, and give every vertex the label of the *sorted*

list of its children's labels, using a dictionary that assigns a new small integer to every new list (Figure 16.14). Leaves get the label of the empty list. Two subtrees get the same label exactly when they are isomorphic, by induction on height, so the trees are isomorphic exactly when the roots' labels are equal. With a shared dictionary, all trees of a test file can be compared by their root labels, in $O(n \log n)$ total.



Labels: $() \to 0$, $(0) \to 1$, $(0,0) \to 2$, $(1,2) \to 3$. Both roots get the label of the sorted list $(1,2)$, so the trees are isomorphic although they are drawn differently.

Figure 16.14: AHU canonical labels. Each vertex shows the label of the sorted multiset of its children's labels.

Example: Tree Isomorphism I (CSES)

Decide whether two trees rooted at vertex 1 are isomorphic.

```
void solve() {
    int t;
    cin >> t;
    map<vi, int> id;  // canonical id of each multiset of child ids (AHU)
    auto read_ids = [&](int n) {
        vvi adj(n + 1);
        rep(i, n - 1) {
            int a, b;
            cin >> a >> b;
            adj[a].push_back(b);
            adj[b].push_back(a);
        }
        vi order{1}, par(n + 1, 0), code(n + 1);
        rep(i, n) for (int v : adj[order[i]])
            if (v != par[order[i]]) par[v] = order[i], order.push_back(v);
        vvi kids(n + 1);
        per(i, n) {  // children first
            int v = order[i];
            sort(all(kids[v]));
            auto [it, fresh] = id.insert({kids[v], sz(id)});
            code[v] = it->second;
            if (par[v]) kids[par[v]].push_back(code[v]);
        }
        return code[1];  // equal ids <=> isomorphic rooted trees
    };
    while (t--) {
        int n;
        cin >> n;
        int a = read_ids(n), b = read_ids(n);
        yn(a == b);
    }
}
```

Complexity. A vertex with d children sorts d labels and looks up a list of length d in the dictionary, which costs $O(d \log n)$ with `map<vi, int>`. The d values sum to $n-1$, so one tree costs $O(n \log n)$. The labels are small integers (at most the number of distinct subtree shapes seen so far), so they can index arrays directly afterwards.

Unrooted trees. An isomorphism of unrooted trees maps centers to centers, and a tree has one center or two adjacent ones. So root the first tree at one of its centers and compare its label with the second tree rooted at each of its (at most two) centers. Centers are found by peeling leaves layer by layer: every round removes all current leaves, and the one or two vertices that remain last are the centers, because peeling shortens every longest path by exactly 2.

Example: Tree Isomorphism II (CSES)

Decide whether two unrooted trees are isomorphic.

```
void solve() {
    int t;
    cin >> t;
    map<vi, int> id;
    auto rooted_id = [&](const vvi& adj, int root) {
        int n = sz(adj) - 1;
        vi order{root}, par(n + 1, 0), code(n + 1);
        rep(i, n) for (int v : adj[order[i]])
            if (v != par[order[i]]) par[v] = order[i], order.push_back(v);
        vvi kids(n + 1);
        per(i, n) {
            int v = order[i];
            sort(all(kids[v]));
            auto [it, fresh] = id.insert({kids[v], sz(id)});
            code[v] = it->second;
            if (par[v]) kids[par[v]].push_back(code[v]);
        }
        return code[root];
    };
    auto centers = [](const vvi& adj) {  // peel leaves layer by layer
        int n = sz(adj) - 1, left = n;
        vi deg(n + 1), layer;
        rep(v, 1, n + 1) {
            deg[v] = sz(adj[v]);
            if (deg[v] <= 1) layer.push_back(v);
        }
        while (left > 2) {
            left -= sz(layer);
            vi next;
            for (int v : layer)
                for (int u : adj[v])
                    if (--deg[u] == 1) next.push_back(u);
            layer = next;
        }
        return layer;  // one or two centers
    };
    while (t--) {
        int n;
```

```
        cin >> n;
        vvi g[2];
        rep(k, 2) {
            g[k].assign(n + 1, {});
            rep(i, n - 1) {
                int a, b;
                cin >> a >> b;
                g[k][a].push_back(b);
                g[k][b].push_back(a);
            }
        }
        // root the first tree at a center, compare with every center of the
        // second: an isomorphism maps centers to centers
        int a = rooted_id(g[0], centers(g[0])[0]);
        bool same = false;
        for (int c : centers(g[1])) same = same || rooted_id(g[1], c) == a;
        yn(same);
    }
}
```

</> Template: Tree isomorphism (canonical IDs, AHU) Ctrl+Alt+U, H prefix `cptreehash`

⚠ Hashing trees A popular shortcut hashes a vertex as, for example, $1 + \sum_c h(c)^2$ modulo a prime or with random polynomial evaluation. Such hashes are fast and work well for counting distinct subtree shapes, but fixed, well-known hash functions have known collisions that anti-hash tests exploit. Use the exact AHU labels when a wrong answer is fatal, or a hash with random parameters chosen at run time (Section 4.8).

Chapter summary

- BFS order read backwards processes children before parents without recursion; preorder and postorder make subtrees contiguous; the full Euler tour turns LCA into a range minimum.
- Two BFS passes find a diameter, and the farthest vertex from anywhere is one of its ends.
- Euler tours turn subtree and root-path queries into range queries on a Fenwick tree.
- Binary lifting jumps k levels and finds LCAs in $O(\log n)$; jump pointers do it with one pointer per vertex; Tarjan's DSU algorithm answers LCAs offline in $O((n+q)\,\alpha(n))$.
- Tree DP combines children bottom-up; rerooting gives every root's answer in two passes, subtracting a child for sums and keeping the two best children for maxima.
- Sums over all pairs of vertices are often sums over edges: each edge separates $sub(v)$ $(n - sub(v))$ pairs.
- Merging tables sized by subtrees costs $O(n^2)$ in total (pairs meet once at their LCA); merging sets small to large costs $O(n \log n)$ moves.
- AHU canonical labels decide rooted isomorphism; for unrooted trees, root at the centers.

Practice problems (CSES Problem Set)

TRAVERSALS, DISTANCES AND QUERIES

- Subordinates
- Tree Diameter
- Tree Distances I
- Tree Distances II
- Subtree Queries
- Path Queries

ANCESTORS, DP AND STRUCTURE

- Company Queries I
- Company Queries II
- Distance Queries
- Tree Matching
- Distinct Colors
- Tree Isomorphism I
- Tree Isomorphism II

CHAPTER 17

Tree Decompositions

The techniques of the previous chapter answer questions about one root path, one subtree or one fixed root. Harder questions need the tree cut into pieces with a logarithmic guarantee. Paths between arbitrary vertices under updates call for heavy-light decomposition; problems about *all* paths of a tree at once call for centroid decomposition; statistics of every subtree can be gathered in one shared table with DSU on tree, or in linear time with long-path decomposition; many small queries shrink to virtual trees; and trees that change shape are kept as Euler tour trees or link-cut trees. Most figures use the same running example, so the decompositions can be compared directly.

IN THIS CHAPTER PLATINUM IOI / ADVANCED

- split a tree into heavy paths and answer path queries with updates in $O(\log^2 n)$;
- decompose a tree by centroids and solve problems about all of its paths, offline and online;
- gather subtree statistics in one shared table (DSU on tree), and depth-indexed DP in linear time (long-path decomposition);
- compress a tree to a few key vertices (virtual trees);
- keep a forest under link and cut (Euler tour trees), and know what link-cut trees and top trees add.

Before you start: Chapter 16 (Euler tours, LCA, small-to-large) and Chapter 22 (segment trees).

17.1 Heavy-light decomposition PLATINUM

Consider Path Queries II: vertex values change, and a query asks for the maximum on the path between two arbitrary vertices. The Euler tour of Section 16.3 handles subtrees and sums along root paths, but a maximum cannot be "subtracted", and a path between two arbitrary vertices is not a prefix of anything. **Heavy-light decomposition** (HLD) cuts the tree into vertical paths such that every path of the tree uses only a few of them, and lays each vertical path out contiguously in an array. A segment tree over that array then answers each piece with one range query.

Heavy and light edges. For every vertex with children, the **heavy child** is a child with the largest subtree (ties broken arbitrarily); the edge to it is *heavy*, and the edges to all other children are *light*. Every vertex has at most one heavy child edge, so the heavy edges form disjoint vertical paths, the **heavy paths** or *chains*. Every vertex lies on exactly one chain, and the top vertex of its chain is its *head*; a vertex whose parent edge is light starts a new chain (Figure 17.1).

> **Fact** On the path from any vertex to the root there are at most $\log_2 n$ light edges. If $p \to v$ is a light edge, the heavy child h of p has $sub(h) \geq sub(v)$, so $sub(p) \geq 1 + sub(v) + sub(h) > 2\,sub(v)$: every light edge on the way up more than doubles the subtree size, which never exceeds n (Figure 17.2).

So a root path touches at most $\log_2 n + 1$ chains, and the path between u and v, which lies inside the union of the two root paths, is cut into at most $2\log_2 n + 2$ *chain pieces*.

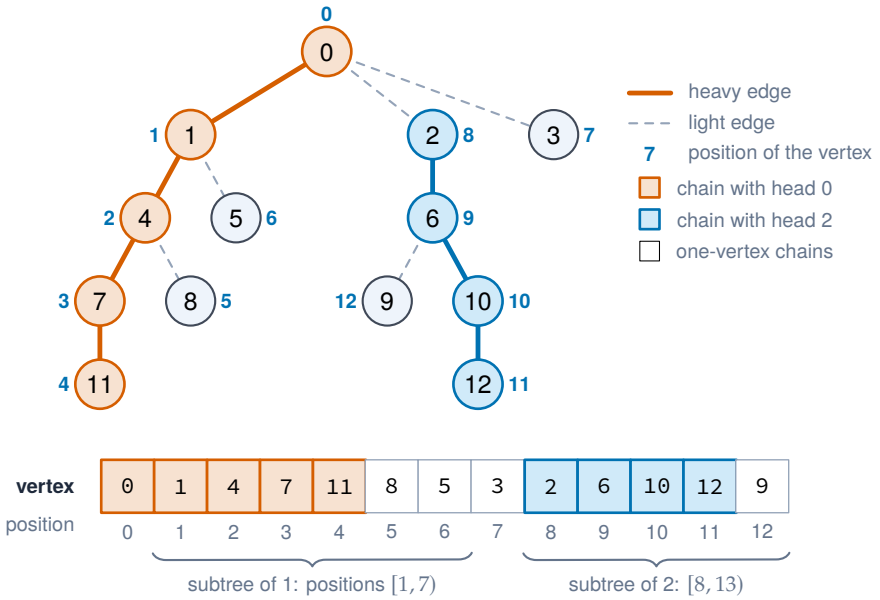


Figure 17.1: Heavy-light decomposition of the running example. Heavy edges form the chains 0–1–4–7–11 and 2–6–10–12; every other vertex is a chain by itself. Numbering each chain from its head, and only then the light subtrees hanging from it, deepest first, makes every chain *and* every subtree a contiguous block of positions.

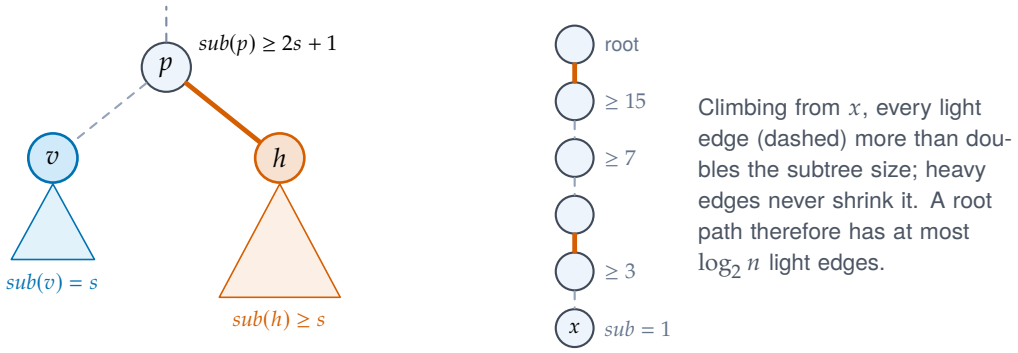
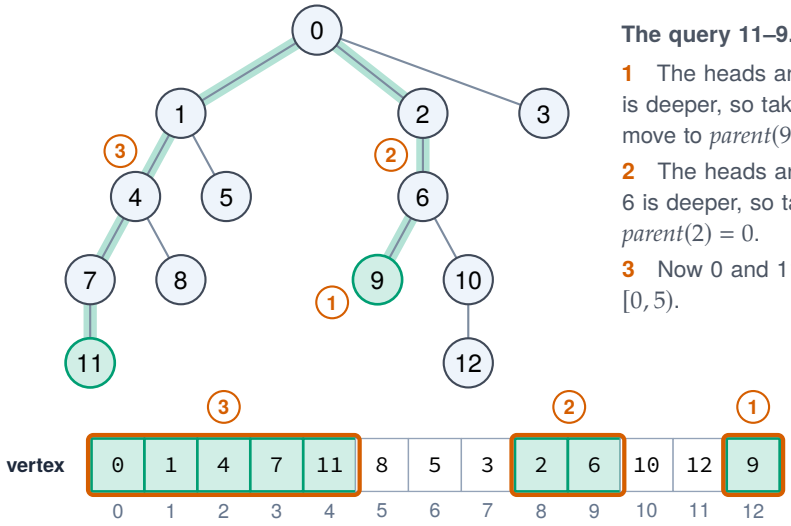


Figure 17.2: The light-edge lemma behind every $\log n$ bound of this section.

Positions. Number the vertices so that every chain is a contiguous block that starts at its head: walk a chain from its head, numbering as you go, and push the light children met on the way onto a stack, to be numbered later. Because the light subtrees are taken from the stack deepest first, the numbering is also a DFS preorder, so the subtree of v is the block $[pos(v), pos(v) + sub(v))$ exactly as with the Euler tour (Figure 17.1). One array, one segment tree, serves both path queries and subtree queries.

Answering a path query. While a and b lie on different chains, look at the one whose chain head is *deeper*, say b: its chain piece from $head(b)$ down to b is the position range $[pos(head(b)), pos(b)]$, so query it and move b to the parent of its head. Once both are on one chain, the piece between them is a single range (Figure 17.3). Moving the vertex with the deeper head never passes the LCA: if the LCA were on b's chain at or below $head(b)$, then a, a descendant of it on another chain, would have a head strictly deeper than $head(b)$. Every move crosses a light edge, so there are $O(\log n)$ pieces, and with a segment tree a query costs $O(\log^2 n)$ and an update $O(\log n)$.



The query 11–9.

1 The heads are 0 and 9; the head of 9 is deeper, so take the range $[12, 13)$ and move to $parent(9) = 6$.

2 The heads are 0 and 2; the head of 6 is deeper, so take $[8, 10)$ and move to $parent(2) = 0$.

3 Now 0 and 11 share a chain: take $[0, 5)$.

Figure 17.3: A path query on the running example. The path from 11 to 9 (green) is cut into three chain pieces, which are three ranges of positions (framed); each costs one segment-tree query.

Example: Path Queries II (CSES)

A tree has a value at every vertex. Process q operations: set the value of a vertex, or report the maximum value on the path between two vertices ($n, q \le 2 \cdot 10^5$).

Solution. HLD positions, and a bottom-up max segment tree over them. The program below runs in 0.25 s on a path and 0.21 s on a random tree with $n = q = 2 \cdot 10^5$ (`g++ -O2`).

```
void solve() {  // HLD (cphld) is pasted above solve()
    int n, q;
    cin >> n >> q;
    vi val(n);
    for (ll& x : val) cin >> x;
    vvi adj(n);
    rep(i, n - 1) {
        int a, b;
        cin >> a >> b;
        a--, b--;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    HLD h(adj);
    // a bottom-up max segment tree over the HLD positions
    vi t(2 * n);
    rep(v, n) t[n + h.pos[v]] = val[v];
    per(i, 1, n) t[i] = max(t[2 * i], t[2 * i + 1]);
    auto query = [&](int l, int r) {  // max over positions [l, r)
        ll res = LLONG_MIN;
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l & 1) chmax(res, t[l++]);
            if (r & 1) chmax(res, t[--r]);
        }
        return res;
    };
```

```
    while (q--) {
        int type, a, b;
        cin >> type >> a >> b;
        if (type == 1) {  // the value of vertex a becomes b
            int i = n + h.pos[a - 1];
            for (t[i] = b; i > 1; i >>= 1) t[i >> 1] = max(t[i], t[i ^ 1]);
        } else {  // O(log n) chain pieces, one range query each
            ll best = LLONG_MIN;
            h.for_path(a - 1, b - 1,
                       [&](int l, int r) { chmax(best, query(l, r)); });
            print(best);
        }
    }
}
```

</> Template: Heavy-light decomposition Ctrl+Alt+Y, H prefix `cphld`

The template's `for_path(a, b, f)` calls `f(l, r)` once for every piece, as a half-open range of positions, so any segment tree plugs in: sums, minima, a lazy tree with range assignment. It also offers `lca(a, b)`, which runs the same loop without the queries and is one of the fastest LCA methods in practice: $O(\log n)$ per query, $O(n)$ memory, and on most trees only a few iterations, one per chain switch.

Values on edges When the values live on edges, store each edge's value at its deeper endpoint. A path query must then skip the LCA, whose stored value belongs to the edge above it: in the last piece, use $[pos(\text{lca}) + 1, pos(v)]$. The template marks the line to change. Forgetting it gives answers that are wrong only when the edge above the LCA is the extreme one, which random tests rarely catch.

Subtrees and paths in one structure Because HLD positions are also a preorder, "add x to every value in the subtree of v" is a range update on $[pos(v), pos(v) + sub(v))$, while "sum on the path u–v" is the usual $O(\log n)$ ranges. One lazy segment tree handles both kinds of operations, which neither the Euler tour alone nor binary lifting can do.

How to recognize it "Update a vertex or edge; query a sum, minimum or maximum on the path between two vertices", or range updates on paths. For path queries without updates, binary lifting or prefix sums along the Euler tour are simpler; HLD earns its place where paths *and* updates meet. With $n = q = 2 \cdot 10^5$ the $O(\log^2 n)$ bound is far from tight: most paths cross only a few light edges.

17.2 Centroid decomposition

PLATINUM

Some problems ask about *all* paths of a tree: how many have exactly k edges, how many have weight at most W, which has the largest XOR. There are $\Theta(n^2)$ paths, but a divide-and-conquer handles them in bulk: pick a vertex c, deal with all paths through c, delete c, and recurse into the pieces that remain. Every path is handled exactly once, at the first of its vertices to be picked. Picking a *centroid* each time keeps the recursion $O(\log n)$ deep.

Centroids. A **centroid** of a tree with n vertices is a vertex whose removal leaves components of at most $n/2$ vertices each. One always exists, and a walk finds it: root the tree anywhere and, while some child of the current vertex has a subtree of more than $n/2$ vertices, step into that child. After a step into c, the component above c has $n - sub(c) < n/2$ vertices, so only children can be too big; subtree sizes decrease, so the walk stops, at a centroid. A tree has one centroid or two adjacent ones.

The decomposition. Find a centroid c, process the paths through c, delete c, and recurse into every remaining component. Each component has at most half the vertices of the one it came from, so the recursion is at most $\lfloor \log_2 n \rfloor + 1$ levels deep, and each vertex belongs to one component per level. If the work at each component is linear in its size, the total is $O(n \log n)$. Linking every centroid below the centroid whose deletion created its component gives the **centroid tree** (Figure 17.4).

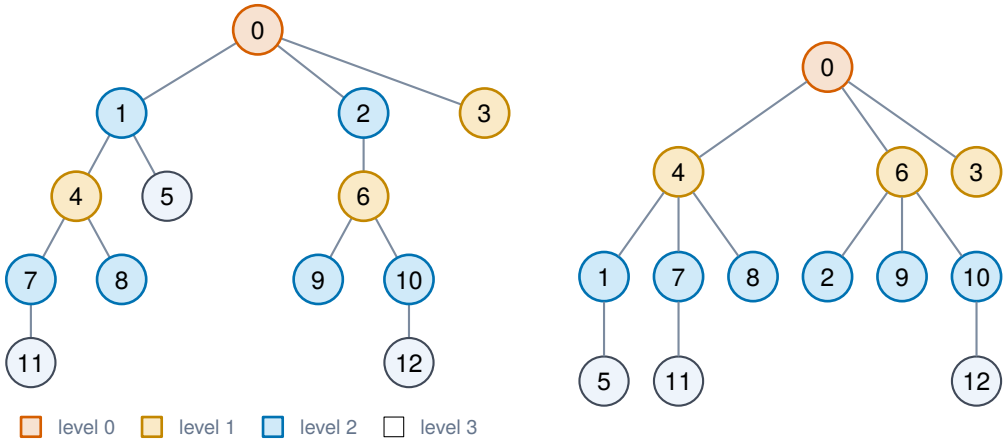


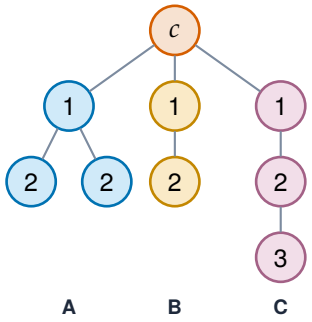
Figure 17.4: Centroid decomposition of the running example. Removing the centroid 0 leaves parts of 6, 5 and 1 vertices, at most $13/2$ each; their centroids 4, 6 and 3 form level 1, and so on. Right: the centroid tree, of depth $3 \le \log_2 13$. The path from 11 to 9 contains exactly one vertex of the lowest level, 0, which is the LCA of 11 and 9 in the centroid tree.

Key idea For any two vertices u and v, the path between them contains exactly one vertex of minimum level: their LCA c in the centroid tree. Until c is deleted, u and v stay in one component, and c is the first vertex of the path to be deleted. Hence the "paths through c inside c's component" partition all paths, and $\text{dist}(u, v) = \text{dist}(u, c) + \text{dist}(c, v)$.

Example: Fixed-Length Paths I (CSES)

Count the paths with exactly k edges in a tree of $n \le 2 \cdot 10^5$ vertices.

Solution. At a centroid c, a path through c joins two vertices in different branches of c (or c itself) whose depths below c add up to k. Keep $cnt[d]$, the number of vertices at depth d found so far, starting with c itself at depth 0. Visit the branches one by one: first add $cnt[k - d]$ for every depth d of the branch, which pairs it only with earlier branches, then add the branch's own depths to cnt (Figure 17.5).



	$cnt[0]$	$cnt[1]$	$cnt[2]$	$cnt[3]$	new paths
start: c alone	1	0	0	0	
after branch A	1	1	2	0	0
after branch B	1	2	3	0	$2+1=3$
after branch C	1	3	4	1	$3+2+1=6$

Figure 17.5: Counting the paths of length $k = 3$ through a centroid c; vertices show their depth below c. Branch B finds $cnt[2] + cnt[1] = 3$ partners for its depths 1 and 2, branch C finds 6, so $0+3+6=9$ paths pass through c. Paths inside one branch are counted later, at a deeper centroid.

```
void solve() {
    int n, k;
    cin >> n >> k;
    vvi adj(n + 1);
    rep(i, n - 1) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    vector<bool> gone(n + 1, false);  // centroids already removed
    vi par(n + 1), sub(n + 1), dep(n + 1), cnt(n + 1, 0);
    auto bfs = [&](int s, int p) {  // the part still attached to s, BFS order
        vi order{s};
        par[s] = p;
        rep(i, sz(order)) for (int v : adj[order[i]]) {
            if (v == par[order[i]] || gone[v]) continue;
            par[v] = order[i];
            order.push_back(v);
        }
        return order;
    };
    ll ans = 0;
    vi roots{1};  // one vertex of every component still to decompose
    while (!roots.empty()) {
        int r = roots.back();
        roots.pop_back();
        vi order = bfs(r, 0);
        per(i, sz(order)) {
            int v = order[i];
            sub[v] = 1;
            for (int u : adj[v])
                if (u != par[v] && !gone[u]) sub[v] += sub[u];
        }
        int total = sz(order), c = r;
        for (bool moved = true; moved;) {  // step into a child with > half
            moved = false;
            for (int u : adj[c])
                if (u != par[c] && !gone[u] && 2 * sub[u] > total) {
```

```
                    c = u, moved = true;
                    break;
                }
            }
            // paths through c: pair each branch with c and the earlier branches
            cnt[0] = 1, dep[c] = 0;
            int maxd = 0;
            for (int u : adj[c]) {
                if (gone[u]) continue;
                vi part = bfs(u, c);
                for (int v : part) dep[v] = dep[par[v]] + 1;
                for (int v : part)
                    if (dep[v] <= k) ans += cnt[k - dep[v]];
                for (int v : part)
                    if (dep[v] <= k) cnt[dep[v]]++, chmax(maxd, dep[v]);
            }
            fill(cnt.begin(), cnt.begin() + maxd + 1, 0);
            gone[c] = true;
            for (int u : adj[c])
                if (!gone[u]) roots.push_back(u);
        }
        print(ans);
    }
```

The decomposition uses an explicit stack of component roots and BFS orders, so it never recurses: on a path of $2 \cdot 10^5$ vertices, the worst case for depth, it runs in 0.20 s, and in 0.17 s on a random tree. The counting array is cleared only up to the largest depth used, so each component costs time linear in its size, never $O(n)$.

Fixed-Length Paths II: a range of lengths. When the length may be anywhere in $[k_1, k_2]$, the lookup $cnt[k - d]$ becomes a range sum over $cnt[k_1 - d..k_2 - d]$, so the counts need prefix sums. Rebuilding the prefix sums after every branch costs the current largest depth, which could be large while the branch is tiny; the fix is to visit the branches in increasing order of height. Then the largest depth so far never exceeds the height of the branch being added, so each rebuild costs no more than that branch's size, and the whole decomposition stays $O(n \log n)$ (the sort by height costs $O(\deg \log \deg)$ per centroid).

> **Inclusion–exclusion instead of branch order** For conditions that a sort handles, such as "depth sum at most K", count *all* pairs in the component, ignoring branches, with a sort and two pointers; the pairs inside one branch were counted wrongly (their real path does not pass through c), so subtract the same count run on each branch's list of depths. This avoids any order between branches, at the cost of an extra log for sorting.

17.2.1 The centroid tree as a data structure

The centroid tree also supports *online* queries. Every vertex has $O(\log n)$ centroid ancestors, and by the key idea above, the path from v to any vertex w passes through one of them. Store at each centroid c a summary of its component (for example, the distance from c to the nearest marked vertex), store at each vertex the distances to its centroid ancestors, and a query combines $O(\log n)$ summaries.

Example: Nearest red vertex

In a tree of n vertices, vertex 1 is red and the others are blue. Process q operations: paint vertex v red, or report the distance from v to the nearest red vertex ($n, q \le 2 \cdot 10^5$). This is Codeforces 342E, *Xenia and Tree*, with larger limits.

Solution. For every centroid c keep $best(c)$, the distance from c to the nearest red vertex of its component. Painting v updates $best$ at the $O(\log n)$ centroid ancestors of v; a query returns $\min_c \operatorname{dist}(v,c) + best(c)$ over them. Every candidate is the length of a real walk, so it is never too small; and the path to the nearest red vertex w passes through the centroid LCA c of v and w, where $\operatorname{dist}(v,c) + best(c) \le \operatorname{dist}(v,w)$.

```
void solve() {
    int n, q;
    cin >> n >> q;
    vvi adj(n + 1);
    rep(i, n - 1) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    vector<bool> gone(n + 1, false);
    vi par(n + 1), sub(n + 1), dist(n + 1);
    vector<vector<pii>> anc(n + 1);  // (centroid ancestor, distance to it)
    auto bfs = [&](int s, int p) {
        vi order{s};
        par[s] = p;
        rep(i, sz(order)) for (int v : adj[order[i]]) {
            if (v == par[order[i]] || gone[v]) continue;
            par[v] = order[i];
            order.push_back(v);
        }
        return order;
    };
    vi roots{1};
    while (!roots.empty()) {
        int r = roots.back();
        roots.pop_back();
        vi order = bfs(r, 0);
        per(i, sz(order)) {
            int v = order[i];
            sub[v] = 1;
            for (int u : adj[v])
                if (u != par[v] && !gone[u]) sub[v] += sub[u];
        }
        int total = sz(order), c = r;
        for (bool moved = true; moved;) {
            moved = false;
            for (int u : adj[c])
                if (u != par[c] && !gone[u] && 2 * sub[u] > total) {
                    c = u, moved = true;
                    break;
```

```
                }
        }
        vi comp = bfs(c, 0);  // c's component, now in BFS order from c
        for (int v : comp) {
            dist[v] = v == c ? 0 : dist[par[v]] + 1;
            anc[v].push_back({c, dist[v]});
        }
        gone[c] = true;
        for (int u : adj[c])
            if (!gone[u]) roots.push_back(u);
    }
    // best[c]: distance from centroid c to the nearest red vertex of its part
    vi best(n + 1, INF);
    auto paint = [&](int v) {
        for (auto [c, d] : anc[v]) chmin(best[c], d);
    };
    paint(1);
    while (q--) {
        int type, v;
        cin >> type >> v;
        if (type == 1) {
            paint(v);
            continue;
        }
        ll res = INF;  // the path to the nearest red vertex passes some c
        for (auto [c, d] : anc[v]) chmin(res, d + best[c]);
        print(res);
    }
}
```

In the running example, vertex 11 stores its centroid ancestors 0, 4, 7 and 11 at distances 4, 2, 1 and 0. If 9 is red, then $best(0) = 3$, and the query from 11 finds $4 + 3 = 7$, the true distance, through the centroid 0 where the two vertices were separated. The lists hold $O(n \log n)$ pairs in total; with $n = q = 2 \cdot 10^5$ the program runs in 0.44 s on a path and 0.42 s on a random tree.

</> Template: Centroid decomposition Ctrl+Alt+Y, C prefix `cpcentroid`

> **How to recognize it** "Count or optimise over all paths" with a condition on length, weight, XOR or colour; "for each k, how many paths of length k"; or online queries about distances to a changing set of vertices. If the answer for a path depends only on the two endpoints' data relative to a common vertex, centroid decomposition applies.

17.3 DSU on tree (Sack) PLATINUM

Small-to-large merging (Section 16.9) keeps one set per vertex and moves the smaller sets into the larger. When the information is a global counting array, such as "how many vertices of each colour", moving it is impossible and copying it is too slow. **DSU on tree**, also called *Sack*, keeps a single global array and reuses the heavy child's contents instead of moving anything. To process a vertex v, and either keep or erase its counts afterwards:

1. process each light child, erasing its counts afterwards;

2. process the heavy child, *keeping* its counts;
3. add v and every vertex of the light subtrees, walking their Euler-tour ranges;
4. the array now describes exactly the subtree of v: answer v's question;
5. if v itself is a light child, remove its whole subtree again.

Why $O(n \log n)$. A vertex x is added in step 3 once for every ancestor that reaches it through a light child, that is, once per light edge on its root path, and it is removed in step 5 at most as often. By the light-edge lemma of Section 17.1 that is at most $\log_2 n$ times, plus once for itself (Figure 17.6).

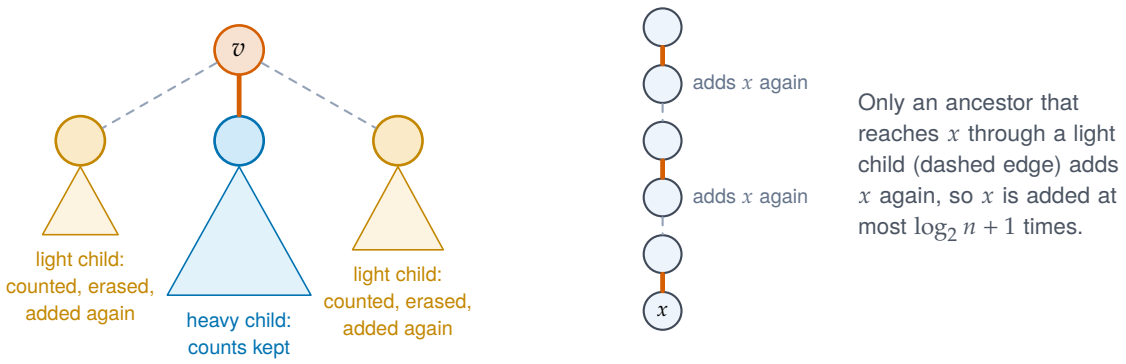


Figure 17.6: DSU on tree. Left: when v is processed, the heavy child's counts are still in the array, and only v and the light subtrees are added. Right: why every vertex is added $O(\log n)$ times.

Example: Dominating colours

Every vertex of a tree rooted at 1 has a colour in $1..n$. For every vertex v, report the sum of all colours that occur in v's subtree the maximum number of times ($n \le 10^5$; Codeforces 600E, *Lomsat gelral*).

Solution. DSU on tree with a counting array *cnt*, the current largest count, and the sum of the colours reaching it. Adding a vertex updates all three in $O(1)$; erasing resets the two numbers and decrements the counts. The recursion is replaced by an explicit stack of (vertex, keep, children done) triples, since a path of 10^5 vertices would otherwise recurse 10^5 deep.

```
void solve() {
    int n;
    cin >> n;
    vi col(n + 1);
    rep(v, 1, n + 1) cin >> col[v];
    vvi adj(n + 1);
    rep(i, n - 1) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    vi order{1}, par(n + 1, 0), sub(n + 1, 1), heavy(n + 1, 0);
    rep(i, n) for (int v : adj[order[i]])
        if (v != par[order[i]]) par[v] = order[i], order.push_back(v);
    per(i, 1, n) sub[par[order[i]]] += sub[order[i]];
    for (int v : order)
        for (int c : adj[v])
```

```
            if (c != par[v] && (!heavy[v] || sub[c] > sub[heavy[v]]))
                heavy[v] = c;
    vi tin(n + 1), ver, st{1};  // preorder: ver[tin[v] .. + sub[v]) = subtree
    while (!st.empty()) {
        int v = st.back();
        st.pop_back();
        tin[v] = sz(ver), ver.push_back(v);
        for (int c : adj[v])
            if (c != par[v]) st.push_back(c);
    }
    vi cnt(n + 1, 0), ans(n + 1);
    ll top = 0, sum = 0;  // the largest count, the sum of colours reaching it
    auto add = [&](int v) {
        int c = col[v];
        if (++cnt[c] > top) top = cnt[c], sum = c;
        else if (cnt[c] == top) sum += c;
    };
    // an explicit stack of (vertex, keep its counts?, children done?)
    vector<array<ll, 3>> todo{{1, 1, 0}};
    while (!todo.empty()) {
        auto [v, keep, done] = todo.back();
        todo.pop_back();
        if (!done) {  // light children run first (popped first), heavy last
            todo.push_back({v, keep, 1});
            if (heavy[v]) todo.push_back({heavy[v], 1, 0});
            for (int c : adj[v])
                if (c != par[v] && c != heavy[v]) todo.push_back({c, 0, 0});
            continue;
        }
        // the heavy child's counts are still there; add everything else
        for (int c : adj[v])
            if (c != par[v] && c != heavy[v])
                rep(t, tin[c], tin[c] + sub[c]) add(ver[t]);
        add(v);
        ans[v] = sum;
        if (!keep) {  // a light child: leave the counts empty again
            rep(t, tin[v], tin[v] + sub[v]) cnt[col[ver[t]]]--;
            top = sum = 0;
        }
    }
    print(vi(ans.begin() + 1, ans.end()));
}
```

</> Template: DSU on tree (Sack) Ctrl+Alt+U, Shift+T prefix `cpsack`

The order of the stack matters: the heavy child is pushed *before* the light children, so it is popped, and processed, after all of them, and its counts are the ones still present when v is finished. With $n = 2 \cdot 10^5$ the program runs in 0.17 s on a path and 0.12 s on a random tree.

Tip Small-to-large merging, DSU on tree and the offline methods of later chapters overlap. Use small-to-large when each vertex needs a set or map of its own afterwards; DSU on tree when the statistic lives in one array (counts by colour, by depth, by value) and must support "the most frequent" or "how many with count at least k"; and the Euler tour with an offline

range technique when the question is a plain range question on the flattened tree, such as the number of distinct values (Mo's algorithm, Chapter 23).

17.4 Long-path decomposition

PLATINUM

Many tree DPs are indexed by depth: $f_v[d]$ is the number (or best value) of vertices at distance d below v. Merging the children's arrays naively costs the sum of all heights over all vertices, which is $\Theta(n^2)$ on a path. Small to large by height gives $O(n \log n)$; a sharper idea gives $O(n)$.

Long paths. The **height** $h(v)$ is the number of vertices on the longest downward path from v. Call the child with the largest height the *long child* and decompose the tree into **long paths** exactly as HLD does with sizes. Now $f_v[d] = f_{long}[d-1]$ for every $d \geq 1$, plus the contributions of the short children. So give each long path one block of a shared buffer, as long as the path, and let the array of every vertex on the path start one cell after its parent's (Figure 17.7): a vertex inherits its long child's array without copying, sets $f_v[0] = 1$, and merges only its short children by hand.

Why $O(n)$. Merging a short child c costs $h(c)$, the length of the long path that starts at c. Every long path starts at a short child (or at the root) and is merged once, into its parent, so the total cost is the total length of all long paths, which is n.

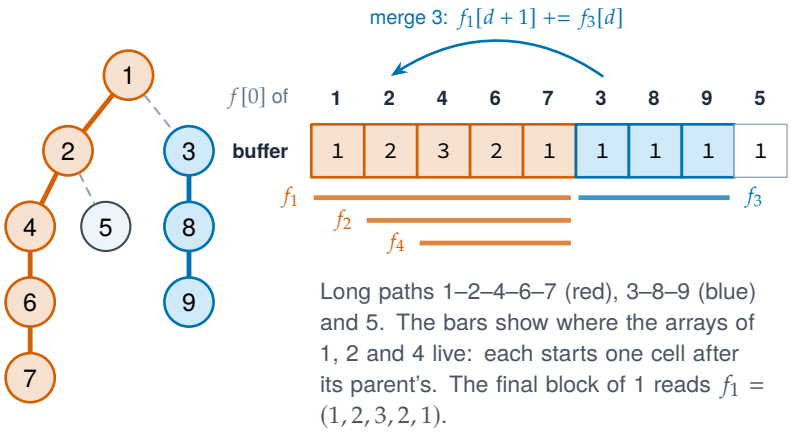


Figure 17.7: Long-path decomposition. Each long path owns one block of the buffer, so a vertex inherits its long child's counts in $O(1)$; only the short child 3 is merged by hand. Three vertices lie at distance 2 below 1, the most at any distance, so the dominant depth of 1 is 2.

Example: Dominant depths

In a tree rooted at 1 with $n \leq 10^6$ vertices, for every vertex v find the smallest d that maximises the number of vertices of v's subtree at distance exactly d from v (Codeforces 1009F, *Dominant Indices*).

Solution. Long-path decomposition with counts. Vertex v inherits the best depth of its long child plus one, compares it with $f_v[0] = 1$, and re-checks only the cells that its short children change; values only grow, so the running best stays correct.

```
void solve() {
    int n;
    cin >> n;
    vvi adj(n + 1);
    rep(i, n - 1) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    vi order{1}, par(n + 1, 0);
    rep(i, n) for (int v : adj[order[i]])
        if (v != par[order[i]]) par[v] = order[i], order.push_back(v);
    // h[v]: vertices on the longest downward path from v; lc[v]: its child
    vi h(n + 1, 1), lc(n + 1, 0);
    per(i, 1, n) {
        int v = order[i], p = par[v];
        if (h[v] + 1 > h[p]) h[p] = h[v] + 1, lc[p] = v;
    }
    // each long path owns one block of the buffer, and along the path the
    // arrays are shifted by one: a vertex inherits its long child's array
    vi pos(n + 1), cnt(n, 0), ans(n + 1);
    int next = 0;
    for (int v : order) {
        if (v == 1 || lc[par[v]] != v) pos[v] = next, next += h[v];
        else pos[v] = pos[par[v]] + 1;
    }
    per(i, n) {  // children first
        int v = order[i];
        auto f = cnt.begin() + pos[v];  // f[d]: vertices at distance d below v
        f[0] = 1;
        ans[v] = lc[v] ? ans[lc[v]] + 1 : 0;
        if (f[ans[v]] <= 1) ans[v] = 0;
        for (int c : adj[v]) {  // merge the short children by hand
            if (c == par[v] || c == lc[v]) continue;
            rep(d, h[c]) {
                f[d + 1] += cnt[pos[c] + d];
                ll a = f[d + 1], b = f[ans[v]];
                if (a > b || (a == b && d + 1 < ans[v])) ans[v] = d + 1;
            }
        }
    }
    print(vi(ans.begin() + 1, ans.end()));
}
```

</> Template: Long-path decomposition Ctrl+Alt+U, Shift+N prefix `cplongpath`

The buffer has exactly n cells, because the blocks are as long as the long paths and those partition the vertices. With $n = 10^6$ the program runs in 0.42 s on a path, 0.39 s on a random tree and 0.36 s on a caterpillar, most of it spent reading the input.

Level ancestors in $O(1)$: the ladder algorithm Extend every long path upwards by its own length to get a *ladder* of at most twice the length; the ladders have $O(n)$ cells in total. To find the k-th ancestor of v, first jump 2^j levels with binary lifting, where 2^j is the highest power of two not above k, landing at u. Since u has a descendant 2^j levels below it, the long path through u continues for at least 2^j more vertices below u, so its ladder extends at least $2^j > k - 2^j$ levels above its head, and the remaining climb is a single array lookup in that ladder. Preprocessing is $O(n \log n)$, and every query is two lookups.

17.5 Virtual (auxiliary) trees

PLATINUM

Some problems ask many queries, each about a small set of k *key vertices*, with the sum of all k bounded by about $2 \cdot 10^5$. A DP over the whole tree costs $O(n)$ per query, far too much, but most of the tree is irrelevant to a query: only the keys and the places where their paths branch matter. The **virtual tree** of the keys contains the keys and the LCAs of pairs of keys; its edges stand for the compressed paths between them, with the path length as weight (Figure 17.8).

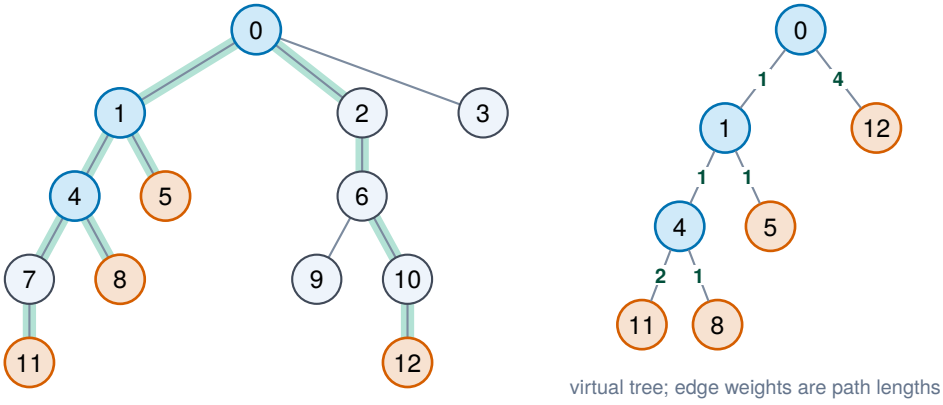


Figure 17.8: The virtual tree of the keys 11, 8, 5 and 12 (red). In DFS order the keys are 11, 8, 5, 12, and the LCAs of neighbours in that order, 4, 1 and 0 (blue), complete the vertex set. The smallest subtree containing the keys (green) has 10 edges, the total weight of the virtual tree.

Construction. Sort the keys by DFS entry time, $v_1, \ldots, v_k$. Every LCA of two keys is already the LCA of two *neighbours* in this order: for $i < j$, $\mathrm{lca}(v_i, v_j)$ is the shallowest of $\mathrm{lca}(v_t, v_{t+1})$ for $i \le t < j$. So adding the $k-1$ neighbour LCAs closes the set under LCA, and the virtual tree has at most $2k-1$ vertices. Sort this set by entry time again, $w_1, \ldots, w_m$; then the virtual parent of w_i is $\mathrm{lca}(w_{i-1}, w_i)$.

Why the parent rule works. The parent of w_i must be the deepest vertex of the set that is a proper ancestor of w_i. The vertex $\mathrm{lca}(w_{i-1}, w_i)$ belongs to the set and is an ancestor of w_i; it is not w_i itself, since w_i comes after w_{i-1} in preorder and so cannot be its ancestor. Any other ancestor a of w_i in the set is entered before w_i, hence no later than w_{i-1}, and its subtree range covers everything up to w_i, so a is an ancestor of w_{i-1} too, and therefore of their LCA. With $O(\log n)$ LCAs, one query costs $O(k \log n)$ after the sorting.

Example: Key vertex queries

A tree has n vertices. Each of q queries gives k distinct key vertices ($\sum k \le 2 \cdot 10^5$); report the number of edges of the smallest subtree containing all keys, and the sum of the distances over all pairs of keys.

Solution. Build the virtual tree; the HLD template provides both a DFS order (`pos`) and LCAs. Walk the virtual vertices in reverse DFS order, children first, and count the keys below each one. A virtual edge of length ℓ above a part with b keys belongs to the smallest subtree, and every one of its ℓ original edges separates b keys from the other $k - b$, so it adds ℓ to the first answer and $\ell\, b\, (k - b)$ to the second (the "count what each edge separates" trick of Section 16.7).

```
void solve() {  // cphld and cpvirtual are pasted above solve()
    int n, q;
    cin >> n >> q;
    vvi adj(n);
    rep(i, n - 1) {
        int a, b;
        cin >> a >> b;
        a--, b--;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    HLD h(adj);  // h.pos is a DFS order, h.lca an LCA
    auto lca = [&](int a, int b) { return h.lca(a, b); };
    vi idx(n, -1), below;
    while (q--) {
        int k;
        cin >> k;
        vi keys(k);
        for (ll& v : keys) cin >> v, v--;
        auto [vs, vp] = virtual_tree(keys, h.pos, lca);
        int m = sz(vs);
        rep(i, m) idx[vs[i]] = i;
        below.assign(m, 0);  // key vertices in each virtual subtree
        for (int v : keys) below[idx[v]] = 1;
        ll steiner = 0, pairs = 0;
        per(i, 1, m) {  // vs is in DFS order: children before parents
            int v = vs[i], p = vp[i];
            ll len = h.depth[v] - h.depth[p];  // edges of the compressed path
            steiner += len;
            pairs += len * below[i] * (k - below[i]);
            below[idx[p]] += below[i];
        }
        print(steiner, pairs);
        for (int v : vs) idx[v] = -1;
    }
}
```

Template: Virtual tree Ctrl+Alt+Y, V prefix `cpvirtual`

The array `idx` maps a vertex to its index in the current virtual tree and is reset after each query, touching only the $O(k)$ vertices used; clearing an n-sized array per query would cost $O(nq)$, the classic mistake with virtual trees. With $\sum k = 2 \cdot 10^5$ the program runs in 0.15 s.

The smallest subtree from the DFS order alone For keys sorted by entry time, $v_1, \ldots, v_k$, the number of edges of the smallest subtree containing them is

$$\tfrac{1}{2}\big(\operatorname{dist}(v_1, v_2) + \operatorname{dist}(v_2, v_3) + \cdots + \operatorname{dist}(v_{k-1}, v_k) + \operatorname{dist}(v_k, v_1)\big),$$

because walking from key to key in DFS order and back to the start traverses every edge of that subtree exactly twice, like the DFS itself. In Figure 17.8: $(3 + 3 + 6 + 8)/2 = 10$. With a set ordered by entry time, the formula also maintains the subtree size under insertions and deletions of keys in $O(\log n)$ each.

How to recognize it Many queries, each naming a few vertices, with "the sum of k over all queries is at most . . . " in the constraints, and a question that would be an easy tree DP if it were asked once.

17.6 Euler tour trees: link and cut

IOI / ADVANCED

So far the tree never changed. In a *dynamic forest*, edges are added (link) and removed (cut) online, and queries ask whether two vertices are connected, or for the sum of the values in a vertex's tree. When all operations are known in advance, divide and conquer over time with a rollback DSU is usually simpler (Section 14.5); online, keep every tree as its Euler tour inside a balanced binary search tree.

Representation. An **Euler tour tree** stores the tour of each tree as a sequence that contains every vertex once and every edge twice, once in each direction (Figure 17.9). The sequence is read cyclically, so rerooting the tree at v is a rotation that starts the sequence at v: one split and one merge. With a treap (Chapter 23) every operation is $O(\log n)$ expected:

- **connected**(u, v): are u and v in the same treap? Compare the roots.
- **link**(u, v): reroot both trees at u and v, then concatenate tour(u), the node (u, v), tour(v) and the node (v, u).
- **cut**(u, v): reroot at u; the tour is now $A\,(u, v)\,B\,(v, u)\,C$, where B is v's side. Split out B and the two edge nodes, and merge A with C.
- **aggregates**: every treap node stores the sum (or minimum, or count of vertices) of its subtree, so the treap root holds the aggregate of the whole tree; changing a vertex value updates one root path.

Example: Forest operations

A forest on n vertices starts without edges; every vertex has a value. Process online: link two vertices of different trees, cut an existing edge, ask whether two vertices are connected, change a value, and report the sum of the values in a vertex's tree.

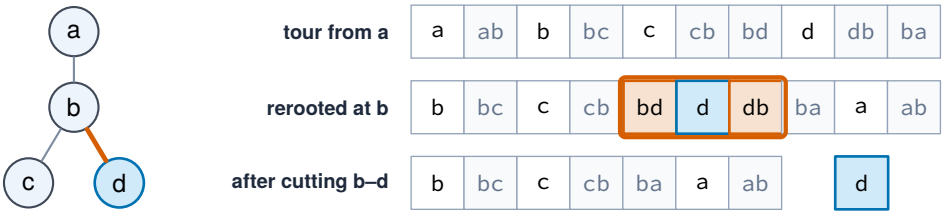


Figure 17.9: An Euler tour tree. The tour lists every vertex once (white) and every edge once in each direction (grey). Rerooting at b rotates the tour; cutting b–d removes the two edge nodes (red) together with d's side between them (blue), which leaves the tour of the remaining tree and the tour of d alone.

```
void solve() {  // Euler tour tree (cpett) is pasted above solve()
    int n, q;
    cin >> n >> q;
    ETT f(n);
    rep(v, n) {
        ll x;
        cin >> x;
        f.set(v, x);
    }
    while (q--) {
        int type, u;
        cin >> type >> u;
        u--;
        if (type == 5) {  // the sum of the values in u's tree
            print(f.comp_sum(u));
            continue;
        }
        ll x;
        cin >> x;
        if (type == 1) f.link(u, x - 1);
        else if (type == 2) f.cut(u, x - 1);
        else if (type == 3) yn(f.connected(u, x - 1));
        else f.set(u, x);
    }
}
```

</> Template: Euler tour tree Ctrl+Alt+U, E prefix `cpett`

The template finds the treap node of an edge through a `map` from vertex pairs, which costs another $O(\log n)$ per link or cut. Subtree aggregates are available too: to get the subtree of v when the tree is rooted at r, reroot at r, and the subtree is the part of the tour between the nodes (p, v) and (v, p), where p is the parent of v.

17.7 Link-cut trees and top trees

IOI / ADVANCED

Euler tour trees answer questions about whole trees and subtrees, but not about *paths*: the tour mixes the vertices of a path with everything hanging from it. A **link-cut tree** (Sleator and Tarjan) keeps a dynamic forest with path aggregates, such as the maximum edge weight between two vertices, in $O(\log n)$ amortised time per operation. It splits each tree into *preferred paths*, much like the chains of HLD but changing over time, and stores each preferred path in a splay tree ordered

by depth. Its central operation, *access*(v), makes the path from the root to v preferred, after which that path is a single splay tree and any path aggregate is read at its root. Rerooting reverses a whole preferred path, with a lazy "reverse" flag.

The classic use is a *dynamic minimum spanning forest* under edge insertions: when an edge u–v of weight w arrives and u, v are already connected, find the heaviest edge on the path between them; if it is heavier than w, cut it and link the new edge, which by the cycle property of Section 13.2 keeps the forest minimum. **Top trees** combine path and subtree aggregates in one structure, with $O(\log n)$ per operation, but they are long to implement and rarely appear outside the hardest olympiad problems. Table 17.1 summarises the structures of this chapter.

structure	link, cut	path aggregates	subtree aggregates	time per operation	code
HLD + segment tree	no	yes	yes	$O(\log^2 n)$	short
centroid tree	no	distances	no	$O(\log n)$	short
Euler tour tree	yes	no	yes	$O(\log n)$ expected	medium
link-cut tree	yes	yes	extra work	$O(\log n)$ amortised	medium
top tree	yes	yes	yes	$O(\log n)$	long

Table 17.1: What the tree structures of this chapter support. "Distances" means queries that combine distances to centroid ancestors, as in the nearest-red-vertex problem.

Tip Before reaching for a dynamic tree, check whether the problem is offline. Operations known in advance usually yield to a sweep, to divide and conquer over time with a rollback DSU, or to a static HLD over the final tree, all of them shorter and safer to write in a contest.

Chapter summary

- A root path crosses at most $\log_2 n$ light edges, so HLD cuts every path into $O(\log n)$ chain pieces, each a range of positions; positions also keep subtrees contiguous.
- Centroid decomposition has $O(\log n)$ levels; every path is handled at exactly one centroid, its LCA in the centroid tree, which gives $O(n \log n)$ algorithms for all-paths problems and $O(\log n)$ online distance queries.
- DSU on tree keeps the heavy child's counts and re-adds light subtrees: $O(n \log n)$ with a single global array.
- Long-path decomposition shares one buffer along each long path and runs depth-indexed DP in $O(n)$.
- A virtual tree on k keys has at most $2k - 1$ vertices: sort by entry time, add neighbour LCAs, and the parent of each vertex is its LCA with its predecessor.
- Euler tour trees keep a forest under link and cut with component aggregates; link-cut trees add path aggregates.

Practice problems (CSES Problem Set)

HEAVY-LIGHT AND CENTROID DECOMPOSITION

- Path Queries II
- Counting Paths
- Finding a Centroid
- Fixed-Length Paths I
- Fixed-Length Paths II

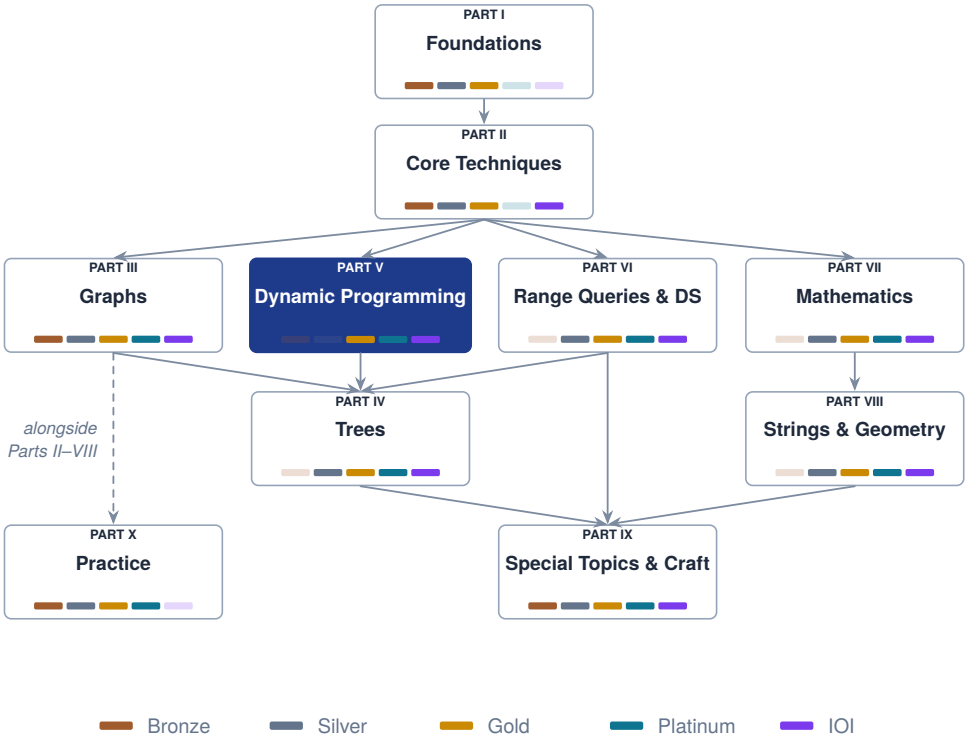
SUBTREE STATISTICS

- Distinct Colors
- Subtree Queries
- Path Queries

PART V

Dynamic Programming

From the first recurrence to the optimizations that decide olympiad medals: knapsack, subsequences, bitmask, range, digit and profile DP, convex hull trick, divide and conquer, Knuth, Aliens and slope trick.



CHAPTER 18

Dynamic Programming Fundamentals

Dynamic programming (DP) solves a problem by combining the answers to smaller versions of the same problem, computing each smaller answer only once. After complexity analysis it is the most important idea in competitive programming, and the one that takes longest to master, because the hard part is not the code but the choice of *what* to compute. The good news is that almost every DP is designed by answering the same five questions, and that a handful of classic families (sums of coins, paths in grids, knapsacks, subsequences, pairs of strings, shapes) cover most of what appears in practice. This chapter develops the method on those families; Chapter 19 adds the patterns built on top of it (bitmasks, intervals, digits, profiles, trees of states), and Chapter 20 the optimizations that make a correct but slow DP fast.

IN THIS CHAPTER GOLD

- recognise overlapping subproblems, and see every DP as a computation on a DAG of states;
- design a DP by answering five questions: state, transition, base cases, order and answer;
- write it top-down or bottom-up, pulling values into a state or pushing them out of it;
- solve the classic families: coins and sums, grid paths, knapsacks, increasing subsequences, edit distance and shapes;
- reconstruct an optimal solution, shrink the memory to one or two rows, and debug a DP systematically.

Before you start: Chapter 6 (recursion and complete search) and Chapter 3.

18.1 What dynamic programming is GOLD

The Fibonacci numbers satisfy $F_0 = 0$, $F_1 = 1$ and $F_n = F_{n-1} + F_{n-2}$. Computed by plain recursion, F_n takes exponential time, because the recursion solves the same subproblems over and over (Figure 18.1): the call for F_n makes $2F_{n+1} - 1$ calls in total, about $4 \cdot 10^{10}$ for F_{50} (Section 3.6). Yet there are only 51 *different* subproblems, F_0 to F_{50}. Remembering each answer the first time it is computed brings the cost down to 51 additions.

DP applies when a problem has two properties. **Overlapping subproblems**: the same smaller problem is needed many times, so remembering its answer pays off. **Optimal substructure**: the answer for a problem can be assembled from the answers of its subproblems; for an optimisation problem, an optimal solution is made of optimal solutions of the parts, and for a counting problem, the solutions split into disjoint classes that are counted separately and added. When the subproblems do not overlap at all, the same recursion is plain divide and conquer.

DP is a computation on a DAG. Draw one vertex per *state* (subproblem) and an arrow from each state to the states its answer depends on. The arrows never form a cycle, because each state depends only on smaller ones, so the states form a DAG, and DP evaluates a function on it in topological order. A minimisation DP is a shortest path in this DAG, a maximisation a longest path, and a

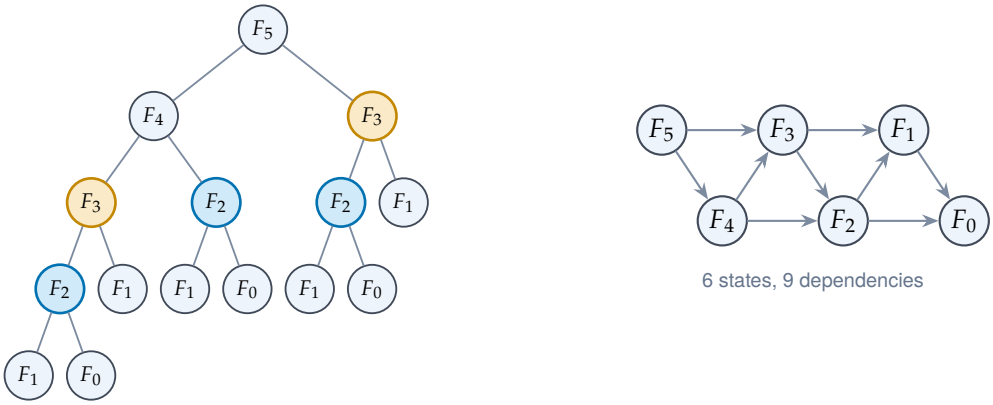


Figure 18.1: Left: the recursion tree of F_5 has 15 calls, because F_3 (orange) is solved twice and F_2 (blue) three times. Right: the same computation as a DAG of distinct states, where an arrow points to a value that is needed; DP evaluates each state once, in an order that respects the arrows.

counting DP counts paths (Section 12.8). This view explains the cost, $O(\text{states} + \text{transitions})$, and it tells you what to do when the dependencies *do* form cycles: then the problem is a shortest-path problem on a general graph, solved with Dijkstra or BFS rather than DP.

18.2 The recipe: five questions

GOLD

1. **State.** What does $dp[\dots]$ mean, in one precise sentence? ("$dp[s]$ is the number of ordered ways to roll a total of s.") Most wrong DPs have a vague state.
2. **Transition.** How is a state computed from smaller ones? Look at the *last* decision: the last roll, the last coin, the last item taken or skipped.
3. **Base cases.** Which smallest states are known directly? ($dp[0] = 1$: the empty sequence.)
4. **Order.** In which order must the states be computed so that every state's dependencies are ready? (Increasing s.)
5. **Answer.** Which state, or combination of states, answers the question? ($dp[n]$.)

The running time is the number of states times the cost of one transition; if that is too slow, shrink one of the two (Chapter 20). The memory is the number of states that must be stored at the same time, which is often far fewer than all of them (Section 18.9).

Example: Dice Combinations (CSES)

Count the ways to get the sum n by throwing a die one or more times, where the order of the throws matters, modulo $10^9 + 7$ ($n \le 10^6$).

Solution. The five answers above: $dp[s] = dp[s-1] + dp[s-2] + \cdots + dp[s-6]$, the last throw being $1, 2, \dots$ or 6 (Figure 18.2). There are $n + 1$ states and six terms per state; the program runs in 0.15 s for $n = 10^6$.

```
void solve() {
    int n;
    cin >> n;
    vi dp(n + 1, 0);  // dp[s]: the number of ordered ways to roll the sum s
    dp[0] = 1;
```

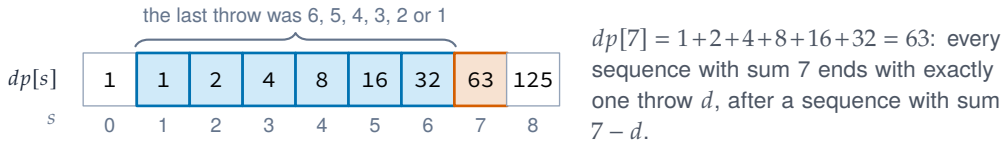


Figure 18.2: The recipe on Dice Combinations. Each state adds up the six states before it; $dp[0] = 1$ counts the empty sequence.

```
    rep(s, 1, n + 1) rep(d, 1, 7)  // the last roll is d
        if (d <= s) dp[s] = (dp[s] + dp[s - d]) % MOD;
    print(dp[n]);
}
```

18.2.1 Top-down or bottom-up

There are two ways to evaluate the same recurrence. **Memoization** (top-down) keeps the recursive definition and adds a cache: the first call for a state computes and stores its answer, later calls return it. **Tabulation** (bottom-up) fills a table in an order that respects the dependencies.

```
vi memo(n + 1, -1);  // -1: not computed yet
auto fib = [&](auto&& self, int i) -> ll {  // top-down
    if (i <= 1) return i;
    ll& res = memo[i];
    if (res != -1) return res;
    return res = self(self, i - 1) + self(self, i - 2);
};
vi f(n + 1);  // bottom-up
f[1] = 1;
rep(i, 2, n + 1) f[i] = f[i - 1] + f[i - 2];
```

Memoization is the quickest to write from a recurrence, needs no thought about the order, and computes only the states that are actually reached, which matters when few of them are. Tabulation has no recursion overhead (often two to five times faster), no risk of a deep recursion overflowing the stack, and allows the memory tricks of Section 18.9. A reasonable habit: think top-down, write bottom-up, and fall back to memoization when the reachable states are sparse or the order is awkward.

> **⚠ The "not computed" marker** A cache initialised with -1 fails silently if -1 is a legitimate answer, and one initialised with 0 recomputes every state whose answer is 0, which can make a memoized DP exponential again. Use a separate `vector<bool> done`, or a value outside the range of answers.

18.2.2 Pulling and pushing

A transition can be written in two directions (Figure 18.3). In *pull* form, each state reads the states it depends on: $dp[s] = \sum_d dp[s-d]$. In *push* form, each finished state adds its value into the states that depend on it: for every d, $dp[s+d]$ += $dp[s]$. Both visit the same arrows of the DAG and give the same result. Pull is natural when the predecessors of a state are easy to list; push when the

successors are, as in "from this position you may jump to any of these", or when a state has many predecessors but only a few successors.

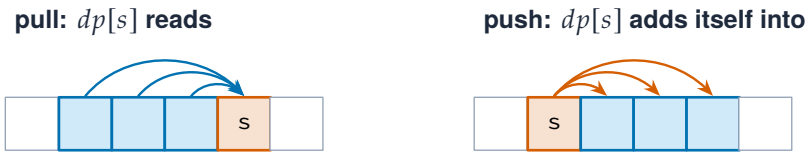


Figure 18.3: The same transitions in pull form (left) and push form (right). Pull computes one state from finished ones; push distributes one finished state forwards.

18.3 Coins and sums

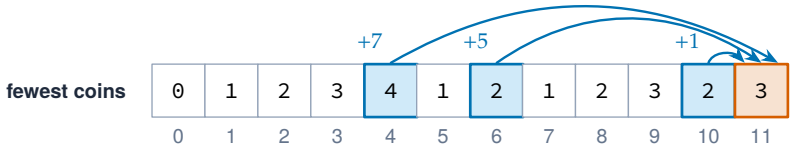
GOLD

Coin problems are the cleanest place to see what a state must and must not remember. Given coin values $c_1, \ldots, c_k$ (each usable any number of times) and a target sum x, three questions have three different DPs.

Example: Minimizing Coins (CSES)

Find the fewest coins with total x, or report that it is impossible ($k \le 100$, $x \le 10^6$).

Solution. $dp[s] = 1 + \min_j dp[s - c_j]$ over the coins with $c_j \le s$, with $dp[0] = 0$ and impossible sums marked by a value larger than any answer (Figure 18.4). Here 10^8 transitions take 0.21 s.



Coins $\{1, 5, 7\}$: $dp[11] = 1 + \min(dp[10], dp[6], dp[4]) = 1 + \min(2, 2, 4) = 3$, for example $5 + 5 + 1$. Taking the largest coin first gives $7 + 1 + 1 + 1 + 1$, five coins.

Figure 18.4: Minimizing Coins for every sum up to 11. The last coin is 1, 5 or 7, so $dp[11]$ looks back at three cells.

```
void solve() {
    int n, x;
    cin >> n >> x;
    vi c(n);
    for (ll& v : c) cin >> v;
    vi dp(x + 1, INF);  // dp[s]: the fewest coins with sum s (INF: impossible)
    dp[0] = 0;
    rep(s, 1, x + 1) for (ll v : c)  // the last coin is v
        if (v <= s && dp[s - v] != INF) chmin(dp[s], dp[s - v] + 1);
    print(dp[x] == INF ? -1 : dp[x]);
}
```

Greedy coins Taking the largest coin that fits is optimal for "canonical" systems such as $\{1, 2, 5, 10, 20, 50\}$, but not in general: Figure 18.4 shows coins $\{1, 5, 7\}$ where greedy uses five coins for 11 and the DP three. Unless the statement guarantees a canonical system, or you can prove the greedy choice, use the DP.

18.3.1 Counting: the order of the loops is the order of the coins

Counting the ways to form x comes in two flavours. If the order of the coins matters ($2 + 1$ and $1 + 2$ are different), sum over the *last* coin, exactly as in Dice Combinations. If it does not (Coin Combinations II), the same recurrence counts every multiset once for each of its orderings, and the fix is a different state: $dp_j[s]$ is the number of ways to form s using only the first j coin types. Then $dp_j[s] = dp_{j-1}[s] + dp_j[s - c_j]$ (no coin of type j, or at least one), every multiset is built in one canonical order, types in increasing order, and the two loops are nested the other way round (Figure 18.5).

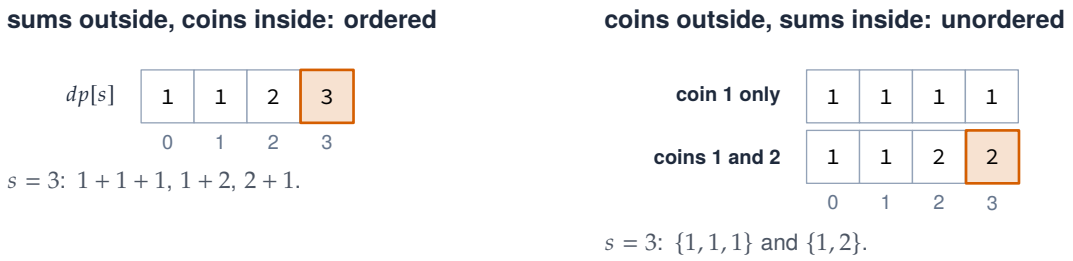


Figure 18.5: Coins $\{1, 2\}$ and sum 3. With the sums in the outer loop, the inner loop chooses the last coin, so every order is counted. With the coins in the outer loop, coin 2 is added only after all coins 1 are placed, so every multiset is counted once.

Example: Coin Combinations I (CSES)

Count the *ordered* ways to form x from the coins, modulo $10^9 + 7$ ($k \le 100$, $x \le 10^6$).

Solution. Sums outside, coins inside. Coin Combinations II, the unordered version, is the worked problem of Section 31.11; its code differs only in the order of the two loops.

```
void solve() {
    int n, x;
    cin >> n >> x;
    vi c(n);
    for (ll& v : c) cin >> v;
    vi dp(x + 1, 0);  // dp[s]: ordered sequences of coins with sum s
    dp[0] = 1;
    rep(s, 1, x + 1) for (ll v : c)  // sums outside: the last coin is any v
        if (v <= s && (dp[s] += dp[s - v]) >= MOD) dp[s] -= MOD;
    print(dp[x]);
}
```

Both values being added are below $10^9 + 7$, so a single subtraction replaces the slower `%` operator in the inner loop: the program runs in 0.21 s for 10^8 transitions.

Tip When a counting DP gives too large an answer, the usual cause is that the same object is reached along several paths of the state DAG. Pin down a *canonical* way to build each object (a fixed order of the parts, the smallest element first, the last change at a fixed place) and make the state follow it.

18.4 Paths in grids

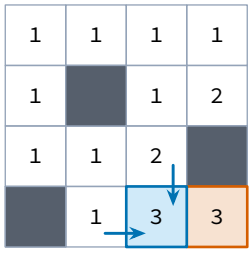
GOLD

Monotone paths in a grid, moving only right or down, are the simplest two-dimensional DP: the last step into a cell came from the cell above or the cell to the left. Counting paths adds the two; a maximum-sum path takes the larger and adds the cell's value; a minimum-cost path takes the smaller.

Example: Grid Paths (CSES)

An $n \times n$ grid has traps. Count the paths from the top-left to the bottom-right corner that move only right or down and avoid the traps, modulo $10^9 + 7$ ($n \le 1000$).

Solution. $dp[r][c] = dp[r-1][c] + dp[r][c-1]$ for a free cell, 0 for a trap (Figure 18.6); 0.11 s for $n = 1000$.



Dark cells are traps and hold 0. Each free cell adds the cell above and the cell to its left: $3 = 2 + 1$ (blue). The corner holds 3, the number of trap-free monotone paths.

Figure 18.6: Grid Paths on the sample grid of the problem.

```
void solve() {
    int n;
    cin >> n;
    vector<string> g(n);
    for (auto& row : g) cin >> row;
    vvi dp(n, vi(n, 0));  // dp[r][c]: paths from (0, 0) to (r, c)
    rep(r, n) rep(c, n) {
        if (g[r][c] == '*') continue;  // a trap: no path ends here
        if (r == 0 && c == 0) dp[r][c] = 1;
        if (r > 0) dp[r][c] += dp[r - 1][c];  // arrived from above
        if (c > 0) dp[r][c] += dp[r][c - 1];  // arrived from the left
        dp[r][c] %= MOD;
    }
    print(dp[n - 1][n - 1]);
}
```

Two walkers at once "Walk from the top-left to the bottom-right and back, collecting values; a cell visited twice counts once" is not two independent paths, because the second path depends on what the first collected. Move two walkers forward *simultaneously*, one step each: after t steps both lie on the anti-diagonal $r + c = t$, so the state (t, r_1, r_2) determines both

positions. The walkers are on the same cell exactly when $r_1 = r_2$, which is where the double count is removed. That is $O(n^3)$ states with four transitions each.

18.5 Knapsack

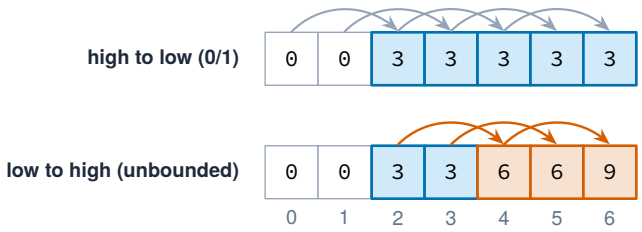
GOLD

Items have weights w_i and values v_i; choose a set of total weight at most W with the largest total value. The last decision is whether item i is in the set: with $dp[i][c]$ the best value using the first i items and capacity c,

$$dp[i][c] = \max\big(dp[i-1][c],\ dp[i-1][c-w_i] + v_i\big) \qquad \text{(the second option only if } w_i \le c\text{)},$$

in $O(nW)$ time. The worked problem Book Shop (Section 31.12) is exactly this, with prices as weights and pages as values.

One row is enough. Row i depends only on row $i-1$, so a single array can be updated in place, with one catch: $dp[c]$ needs the *old* $dp[c-w_i]$. Updating the capacities from high to low keeps that value old; updating them from low to high would use a value that already includes item i, so the item could be taken again and again (Figure 18.7). That is not a bug but a different problem: the low-to-high loop solves the *unbounded* knapsack, where every item has unlimited copies.



One item of weight 2 and value 3 is added to an empty table. Going down, every cell reads a cell that does not contain the item yet, so it is used at most once. Going up, cell 4 reads cell 2, which already contains it (red): the item is taken twice, then three times.

Figure 18.7: The direction of the capacity loop is the whole difference between the 0/1 and the unbounded knapsack.

18.5.1 Counting subsets

Replacing max by + counts the subsets instead of optimising over them, and the same direction rule applies.

Example: Two Sets II (CSES)

Count the ways to split $\{1, 2, \ldots, n\}$ into two sets with equal sums, modulo $10^9 + 7$ ($n \le 500$).

Solution. The total $n(n+1)/2$ must be even; then count the subsets with half the total. Every split is two such subsets (one side and the other), so the count must be halved, which modulo a prime means multiplying by the inverse of 2. Simpler: leave the number n out of the DP. Exactly one side of every split lacks n, so the subsets of $\{1, \ldots, n-1\}$ with half the total are the splits, each counted once.

```
void solve() {
    int n;
    cin >> n;
    int total = n * (n + 1) / 2;
    if (total % 2) {
```

```
        print(0);
        return;
    }
    int half = total / 2;
    vi dp(half + 1, 0);  // dp[s]: subsets of the numbers seen so far with sum s
    dp[0] = 1;
    // leave n out: each split is counted once, by its side without n
    rep(i, 1, n) per(s, i, half + 1) dp[s] = (dp[s] + dp[s - i]) % MOD;
    print(dp[half]);
}
```

18.5.2 Bounded copies

When item i has k_i copies, treating every copy as a separate item costs $O(W \sum k_i)$. *Binary splitting* replaces the k copies by bundles of $1, 2, 4, \ldots, 2^{t-1}$ copies and a remainder $r = k - (2^t - 1) < 2^t$, each bundle a 0/1 item. Every count from 0 to $2^t - 1$ is a sum of distinct powers of two, and every count from $2^t - 1$ to k is r plus one of those, so the bundles reach exactly the counts $0, \ldots, k$. The cost drops to $O(W \sum \log k_i)$.

Example: Bounded knapsack

There are n kinds of items; kind i has weight w_i, value v_i and k_i copies. Maximise the value within capacity W ($n \le 100$, $W \le 10^5$, $k_i \le 1000$).

```
void solve() {
    int n, cap;
    cin >> n >> cap;
    vi dp(cap + 1, 0);  // dp[c]: the best value with total weight at most c
    rep(i, n) {
        ll w, v, k;
        cin >> w >> v >> k;
        // bundles of 1, 2, 4, ... copies and a remainder: every count from
        // 0 to k is a sum of distinct bundles, so 0/1 choices suffice
        for (ll b = 1; k > 0; b *= 2) {
            ll take = min(b, k);
            k -= take;
            per(c, take * w, cap + 1)
                chmax(dp[c], dp[c - take * w] + take * v);
        }
    }
    print(dp[cap]);
}
```

With $n = 100$ kinds of 1000 copies each and $W = 10^5$ the program makes about 10^8 updates in 0.15 s. An exact $O(nW)$ algorithm, which splits the capacities by their remainder modulo w_i and slides a monotone queue along each class, is in Section 20.1.

Template: Knapsack (0/1, unbounded) Ctrl+Alt+T, Shift+Q prefix cpknap

When the capacity is huge but the values are small With W up to 10^9 but values summing to at most 10^5, swap the roles of the two numbers: $dp[v]$ is the smallest weight that reaches value exactly v, and the answer is the largest v with $dp[v] \le W$. Always ask which quantity is small enough to index the table. For plain subset-sum feasibility, a bitset makes the $0/1$ knapsack 64 times faster (Section 10.4).

18.6 Increasing subsequences

GOLD

The **longest increasing subsequence** (LIS) of $a_0, \ldots, a_{n-1}$ is the longest sequence of indices $i_1 < i_2 < \ldots$ with $a_{i_1} < a_{i_2} < \ldots$. The textbook DP takes the last element as the state: $dp[i]$, the length of the longest increasing subsequence that ends at a_i, is $1 + \max\{dp[j] : j < i,\ a_j < a_i\}$, in $O(n^2)$.

The $O(n \log n)$ method. Keep instead, for every length k, the *smallest possible last value* of an increasing subsequence of length $k+1$ seen so far; call it $tail[k]$. A smaller last value is never worse, because anything that extends a subsequence also extends one that ends lower. The tails are strictly increasing (a subsequence of length $k+2$ contains one of length $k+1$ that ends lower), so a new element x either extends the longest subsequence, when it is larger than every tail, or improves the first tail that is $\ge x$, found by binary search (Figure 18.8). The number of tails at the end is the LIS length. The worked problem of Section 31.13 uses exactly this.

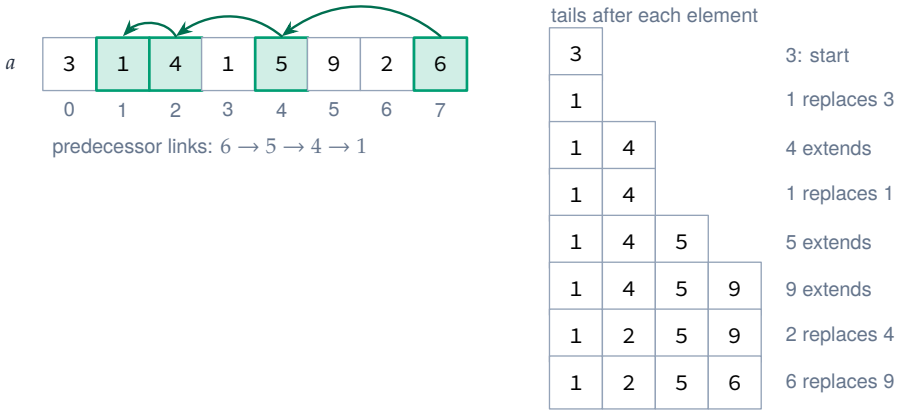


Figure 18.8: The tails for $3, 1, 4, 1, 5, 9, 2, 6$. The final list has length 4, the LIS length. The tails themselves $(1, 2, 5, 6)$ are not a subsequence; following each element's predecessor, the tail before it at the moment it was placed, gives one: $1, 4, 5, 6$.

Reconstruction. When a_i is placed at position k of the tails, the element at position $k-1$ at that moment is the end of a subsequence of length k that a_i extends; record it as $prv[i]$. Storing indices rather than values in the tails makes this possible, and following prv from the last tail spells out an LIS backwards.

Example: Print a longest increasing subsequence

Given $n \le 2 \cdot 10^5$ numbers, print the length of a longest strictly increasing subsequence and one such subsequence.

```cpp
void solve() {
    int n;
    cin >> n;
    vi a(n);
    for (ll& x : a) cin >> x;
    // tail[k]: index of the smallest possible last element of an increasing
    // subsequence of length k + 1; prv[i]: the element before a[i] in it
    vi tail, prv(n, -1);
    rep(i, n) {
        auto it = lower_bound(all(tail), a[i],
                              [&](ll j, ll x) { return a[j] < x; });
        int k = it - tail.begin();
        if (k > 0) prv[i] = tail[k - 1];
        if (it == tail.end()) tail.push_back(i);
        else *it = i;
    }
    vi lis;  // walk the predecessors back from the end of the longest one
    for (int i = tail.back(); i != -1; i = prv[i]) lis.push_back(a[i]);
    reverse(all(lis));
    print(sz(lis));
    print(lis);
}
```

</> Template: Longest increasing subsequence Ctrl+Alt+T, Shift+L prefix `cplis`

The LIS in disguise

- **Non-decreasing** subsequences: search with `upper_bound` instead of `lower_bound`, so that equal values extend.
- **Fewest decreasing subsequences** covering an array equals the LIS length (a case of Dilworth's theorem); greedily placing each element on the leftmost pile whose top is larger, as in Towers (CSES), builds such a cover.
- **Nested boxes or envelopes**: sort by width increasing and, for equal widths, by height *decreasing*, then take the LIS of the heights; the decreasing tie-break stops two boxes of equal width from nesting.
- **Fewest changes** that make an integer array strictly increasing: n minus the longest non-decreasing subsequence of $a_i - i$.

18.7 Two sequences: LCS and edit distance

GOLD

With two sequences, the state is a pair of prefixes: $dp[i][j]$ is the answer for the first i elements of a and the first j of b, and the transitions look at the last elements, a_{i-1} and b_{j-1}.

- **Longest common subsequence**: if the last elements are equal, match them; otherwise at least one of the two is not in the common subsequence:

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1 & \text{if } a_{i-1} = b_{j-1}, \\ \max\big(dp[i-1][j],\ dp[i][j-1]\big) & \text{otherwise.} \end{cases}$$

- **Edit distance** (the fewest insertions, deletions and replacements turning a into b): the last operation deletes a_{i-1}, inserts b_{j-1}, or aligns the two, for free if they are equal:

$$dp[i][j] = \min\big(dp[i-1][j]+1,\ dp[i][j-1]+1,\ dp[i-1][j-1]+[a_{i-1}\neq b_{j-1}]\big),$$

with $dp[i][0] = i$ and $dp[0][j] = j$.

		M	O	V	I	E
	0	1	2	3	4	5
L	1	1	2	3	4	5
O	2	2	1	2	3	4
V	3	3	2	1	2	3
E	4	4	3	2	2	2

Edit distance from LOVE to MOVIE is 2. The highlighted path, read from the top-left: replace L by M (diagonal, +1), keep O and V (diagonal, +0), insert I (right, +1), keep E. A diagonal step is a match or a replacement, a step right an insertion, a step down a deletion.

Figure 18.9: The edit-distance table of LOVE (rows) and MOVIE (columns), and an optimal alignment read from it.

Example: Edit Distance (CSES)

Find the edit distance of two strings of length at most 5000.

Solution. The table has $2.5 \cdot 10^7$ cells; each row depends only on the previous one, so two rows of length $m+1$ suffice. That keeps the memory at $O(m)$ instead of 200 MB of 64-bit numbers, and the program runs in 0.19 s.

```
void solve() {
    string a, b;
    cin >> a >> b;
    int n = sz(a), m = sz(b);
    // prev[j]: edits turning a[0, i) into b[0, j); only two rows are kept
    vi prev(m + 1), cur(m + 1);
    iota(all(prev), 0);  // from the empty prefix: insert j characters
    rep(i, 1, n + 1) {
        cur[0] = i;  // to the empty prefix: delete i characters
        rep(j, 1, m + 1)  // delete a[i - 1], insert b[j - 1], replace or keep
            cur[j] = min({prev[j] + 1, cur[j - 1] + 1,
                          prev[j - 1] + (a[i - 1] != b[j - 1])});
        swap(prev, cur);
    }
    print(prev[m]);
}
```

</> Template: LCS + edit distance Ctrl+Alt+T, [prefix cplcs

Reconstruction in linear memory: Hirschberg Two rows give the distance but not the alignment, which needs the whole table to walk back. *Hirschberg's algorithm* recovers it in $O(nm)$ time and $O(n+m)$ memory: run the two-row DP forwards on the first half of a and backwards on the second half, find the column j where the optimal path crosses the middle row (the one minimising the sum of the two half costs), and recurse on the two halves. The total work is $nm + nm/2 + nm/4 + \cdots < 2nm$.

18.8 Choosing the state

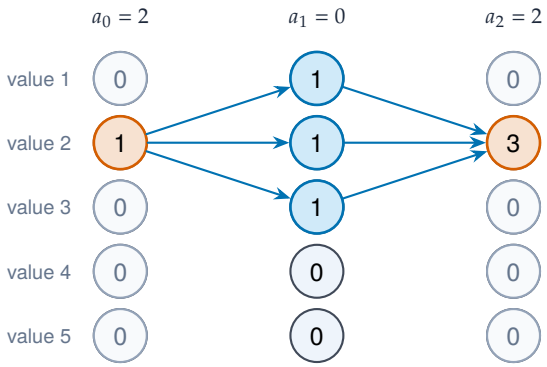
GOLD

The state must contain exactly what the future depends on. A useful test: if two different histories reach the same state but could continue in different ways, the state is missing information; if the state distinguishes histories that always continue alike, it is larger than necessary.

Example: Array Description (CSES)

An array of n values in $1..m$ has adjacent values differing by at most 1. Some values are known, the others (shown as 0) are lost. Count the arrays that match, modulo 10^9+7 ($n \le 10^5$, $m \le 100$).

Solution. The index alone is not a valid state, since which values may come next depends on the last value; so the state is the pair (position, last value). Each layer of states depends only on the previous layer (Figure 18.10): $O(nm)$ time, 0.11 s at the limits.



Each vertex is a state (position, last value) and shows its number of ways. A known value allows a single vertex in its layer (red). The three arrays $2,1,2$, $2,2,2$ and $2,3,2$ are the three paths into the last layer.

Figure 18.10: Array Description with $n = 3$, $m = 5$ and the array $2, 0, 2$ as a layered DAG of states.

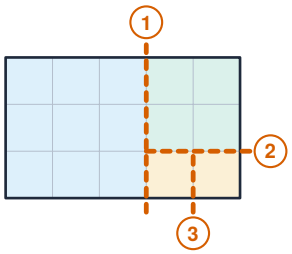
```
void solve() {
    int n, m;
    cin >> n >> m;
    vi a(n);
    for (ll& v : a) cin >> v;
    // dp[v]: ways to fill the prefix so far, ending with the value v;
    // dp[0] and dp[m + 1] stay 0 as sentinels
    vi dp(m + 2, 0);
    rep(v, 1, m + 1) dp[v] = a[0] == 0 || a[0] == v;
    rep(i, 1, n) {
        vi nd(m + 2, 0);
        rep(v, 1, m + 1) {
            if (a[i] != 0 && a[i] != v) continue;
            nd[v] = (dp[v - 1] + dp[v] + dp[v + 1]) % MOD;
        }
        dp = nd;
    }
    ll ans = 0;
    rep(v, 1, m + 1) ans = (ans + dp[v]) % MOD;
    print(ans);
}
```

States that are shapes. Sometimes the natural subproblem is a smaller copy of the input's *shape*, as when a rectangle is cut into rectangles.

Example: Rectangle Cutting (CSES)

A rectangle of $a \times b$ unit squares is cut, one straight cut through a whole piece at a time, until every piece is a square. Find the fewest cuts ($a, b \le 500$).

Solution. A first cut splits $w \times h$ into $i \times h$ and $(w-i) \times h$, or $w \times i$ and $w \times (h-i)$, and the two pieces are then solved independently: $dp[w][h]$ is the minimum over all first cuts of 1+ the two pieces' values, with $dp[w][w] = 0$ (Figure 18.11). Cuts at i and at $w-i$ give the same two pieces, so only cuts up to the middle are tried, which halves the $O(ab(a+b))$ work; 0.24 s for 500×499.



3×5: cut off a 3×3 square (cut 1); the remaining 3×2 piece loses a 2×2 square (cut 2); the last 1×2 strip is cut into two unit squares (cut 3). No two cuts suffice, so $dp[3][5] = 3$.

Figure 18.11: An optimal way to cut a 3×5 rectangle into squares.

```
void solve() {
    int a, b;
    cin >> a >> b;
    // dp[w][h]: the fewest cuts that turn a w x h rectangle into squares
    vvi dp(a + 1, vi(b + 1, INF));
    rep(w, 1, a + 1) rep(h, 1, b + 1) {
        if (w == h) {
            dp[w][h] = 0;
            continue;
        }
        // the first cut, vertical or horizontal; by symmetry, up to the middle
        rep(i, 1, w / 2 + 1) chmin(dp[w][h], dp[i][h] + dp[w - i][h] + 1);
        rep(i, 1, h / 2 + 1) chmin(dp[w][h], dp[w][i] + dp[w][h - i] + 1);
    }
    print(dp[a][b]);
}
```

18.9 Reconstruction, memory and debugging

GOLD

Reconstructing a solution. To output the choices and not only the value, either store, for every state, which transition achieved the optimum (a parent pointer, as *prv* in the LIS), or keep the table and re-derive the choice afterwards: walk from the answer state and, at each step, find a transition whose value plus its cost equals the stored value, exactly as the highlighted path of Figure 18.9 was found. The second way costs no extra memory during the DP but needs the full table during the walk.

Shrinking the memory. If row i of the table depends only on row $i-1$, keep two rows and swap them (edit distance), or one row updated in the right direction (knapsack). If a state depends on the last k states only, a ring buffer of k cells is enough. Each saving gives up reconstruction, unless the choices are stored separately in a compact form, such as one bit per state.

A checklist for wrong DP answers

- Unreachable states initialised to 0 instead of $-\infty$ or $+\infty$, so that impossible choices look free.
- An in-place update in the wrong direction, so that one item is used twice (or a value is read after it changed).
- Overflow in a counting DP: reduce modulo after *every* addition and multiplication, and multiply in 64 bits.
- A state that is missing information: two histories reach it but need different futures.
- Double counting: the same object built along two paths of the state DAG; fix a canonical construction order.
- Off-by-one base cases: is $dp[0]$ the empty prefix (usually 1 way, or cost 0), and is the answer $dp[n]$ or $dp[n-1]$?

The fastest way to find such bugs is a brute force on tiny inputs and a random test generator (Chapter 30).

How to recognize it Counting ways, a minimum or maximum cost, or "is it possible", over a sequence of decisions where the future depends on the past only through a small summary. Limits such as $n \le 5000$ (for $O(n^2)$), $n \cdot W \le 10^8$, or a grid of 1000×1000 are typical. A brute-force recursion that keeps being called with the same arguments is a DP waiting to be memoized.

Chapter summary

- A DP evaluates a function on a DAG of states: the cost is states times transitions, and minimum, maximum and counting DPs are shortest paths, longest paths and path counts in that DAG.
- Design with five questions: state, transition (look at the last decision), base cases, order, answer.
- Memoization is quick to write; tabulation is faster, stack-safe, and allows rolling arrays. Pull and push are the same arrows read in two directions.
- The loop order of a coin DP decides between ordered and unordered counts; the capacity direction decides between the 0/1 and the unbounded knapsack; binary splitting handles bounded copies.
- The LIS needs only the smallest tail of every length: $O(n \log n)$, and predecessor links reconstruct it.
- Pairs of prefixes solve LCS and edit distance; two rows suffice for the value, Hirschberg recovers the alignment.
- The state must hold exactly what the future depends on: the last value, the shape that remains.

Practice problems (CSES Problem Set)

COUNTING AND OPTIMISING SUMS

- Dice Combinations
- Minimizing Coins
- Coin Combinations I
- Coin Combinations II
- Removing Digits
- Money Sums
- Two Sets II

GRIDS, KNAPSACKS AND SEQUENCES

- Grid Paths
- Book Shop
- Array Description
- Edit Distance
- Rectangle Cutting
- Increasing Subsequence
- Towers

CHAPTER 19

DP Patterns

Most dynamic programs belong to a small number of families, and recognising the family gives the state almost for free. A set of up to 20 items becomes a bitmask; a contiguous segment becomes a pair of endpoints; a huge number becomes its digits plus a "still equal to the bound" flag; a grid that is too wide to enumerate becomes the boundary between its filled and empty parts; a recurrence applied 10^{18} times becomes a matrix power. This chapter goes through these families, each with a problem solved end to end, together with the two topics that most often hide inside them: expectations and games.

IN THIS CHAPTER GOLD PLATINUM

- use bitmask DP over subsets, including pair-valued states and submask enumeration in $O(3^n)$;
- write interval DP, and turn two-player games into one difference;
- count numbers up to 10^{18} with digit DP, and tile grids with profile DP;
- sum over subsets in $O(n2^n)$, and convolve over disjoint subsets;
- schedule weighted intervals, compute probabilities and expectations, and solve games by DP;
- jump 10^{18} steps of a linear recurrence with a matrix power.

Before you start: Chapter 18 (the five-question recipe) and Chapter 10 (bitmasks).

19.1 Bitmask DP

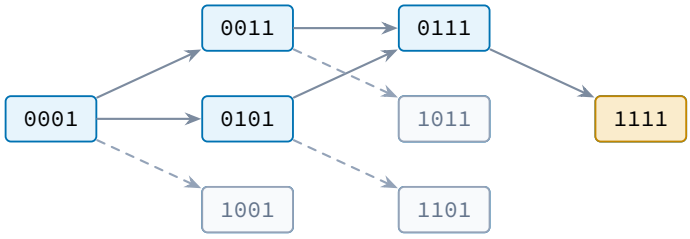
GOLD

When the future depends on *which* of n items have been used, and n is at most about 20, store that set as an n-bit mask (Section 10.2). There are 2^n sets, about 10^6 for $n = 20$, so the state may add one more index of size n, and each transition may try every item: $O(2^n n^2)$ is about $4 \cdot 10^8$ simple operations at $n = 20$, which is the usual budget.

Example: Hamiltonian Flights (CSES)

There are $n \le 20$ cities and m one-way flights. Count the routes from city 1 to city n that visit every city exactly once, modulo $10^9 + 7$.

Solution. $dp[mask][v]$ is the number of routes that start at city 1, visit exactly the cities of *mask* and end at v. A route grows by one flight $v \to w$ to a city $w \notin mask$. Masks only grow, so increasing numeric order is a valid order (Figure 19.1). Two observations halve the work and more: every useful mask contains city 1, and city n must be last, so a mask that contains city n but not every city is never extended.



Masks for $n = 4$, city 1 being the lowest bit. Every route starts at city 1, so only masks with that bit appear; masks with city 4 (the highest bit) that are not complete (grey) receive values but are never extended.

Figure 19.1: The DP of Hamiltonian Flights moves through subsets of cities, one city per step.

```
void solve() {
    int n, m;
    cin >> n >> m;
    vvi cnt(n, vi(n, 0));  // the number of flights from city a to city b
    rep(i, m) {
        int a, b;
        cin >> a >> b;
        cnt[a - 1][b - 1]++;
    }
    // dp[mask * n + v]: routes from city 0 through exactly the cities of
    // mask, ending at v; 32-bit entries, since there are 2^20 * 20 of them
    vector<uint32_t> dp((1 << n) * n, 0);
    dp[1 * n + 0] = 1;  // the route that has only visited city 0
    int full = (1 << n) - 1;
    rep(mask, 1, 1 << n) {
        if (!(mask & 1)) continue;  // every route starts at city 0
        if (mask >> (n - 1) & 1 && mask != full) continue;  // n - 1 is last
        rep(u, n) {
            ll ways = dp[mask * n + u];
            if (!ways) continue;
            rep(v, n) if (!(mask >> v & 1) && cnt[u][v]) {  // push to v
                uint32_t& t = dp[(mask | 1 << v) * n + v];
                t = (t + ways * cnt[u][v]) % MOD;
            }
        }
    }
    print(dp[full * n + n - 1]);
}
```

The table has $2^{20} \cdot 20 \approx 2 \cdot 10^7$ entries: 84 MB with 32-bit entries, twice that with 64-bit ones, which is why the type is chosen explicitly. With all 380 possible flights the program runs in 0.18 s. A flat array indexed by $mask \cdot n + v$ is also much faster than a `vector` of 2^{20} small vectors.

</> Template: Bitmask DP (TSP) Ctrl+Alt+T,] prefix `cptsp`

19.1.1 Pair-valued states

The state does not have to hold a single number. When a set of items is packed into bins one item at a time, the only thing the future needs is the number of bins used and how full the current bin is.

Example: Elevator Rides (CSES)

There are $n \le 20$ people with weights w_i and an elevator with capacity x. What is the fewest rides that carry everyone?

Solution. For every set of people, keep the best pair (rides, weight of the last ride) that carries exactly that set, compared lexicographically. Adding person p either fits into the last ride or opens a new one. Fewer rides is always better, and with as many rides a lighter last ride is better, because anything that can board after the heavier can board after the lighter: so the lexicographic minimum dominates every other way of carrying the set (Table 19.1).

people already carried	their best pair	boards last	new pair
$\{4,8,6\}$	$(2,8)$	1	$(2,9)$
$\{4,8,1\}$	$(2,4)$	6	$(2,10)$
$\{4,6,1\}$	$(2,1)$	8	$(2,9)$
$\{8,6,1\}$	$(2,6)$	4	$(2,10)$
all four people, the minimum:			$(\mathbf{2},\mathbf{9})$

Table 19.1: Elevator Rides with weights 4, 8, 6, 1 and capacity 10: the state of the full set, from its four predecessors. Two rides suffice, the last one carrying 9.

```
void solve() {
    int n, x;
    cin >> n >> x;
    vi w(n);
    for (ll& v : w) cin >> v;
    // best[mask]: the smallest (rides, weight of the last ride) for carrying
    // exactly the people in mask; the last ride may still take more people
    vector<pll> best(1 << n, {INF, 0});
    best[0] = {1, 0};
    rep(mask, 1, 1 << n) rep(p, n) {
        if (!(mask >> p & 1)) continue;
        auto [rides, last] = best[mask ^ (1 << p)];  // p is the last to board
        if (last + w[p] <= x) last += w[p];
        else rides++, last = w[p];
        chmin(best[mask], pll{rides, last});
    }
    print(best[(1 << n) - 1].first);
}
```

The obvious alternative, "$dp[mask]$ is the fewest rides, built from a submask that fits into one ride", is correct but costs $O(3^n)$; the pair-valued state brings it to $O(2^n n)$, 0.08 s for $n = 20$.

19.1.2 Enumerating submasks

Some transitions really do range over all subsets of the current mask, for example "split the set into groups, each with a cost". The loop

```
for (int sub = mask; sub > 0; sub = (sub - 1) & mask) { /* use sub */ }
```

visits every non-empty submask of `mask` in decreasing order: subtracting 1 clears the lowest set bit and sets all bits below it, and the `&` keeps only those inside the mask. Over all masks the total is 3^n, not 4^n, because each element is either outside the mask, in the mask but not the submask, or in both: three choices per element. That is $3.5 \cdot 10^9$ for $n = 20$, too many, but fine for $n \le 16$ ($4.3 \cdot 10^7$).

How to recognize it $n \le 20$ (or $n \le 16$ for $O(3^n)$) and an answer that depends on a *set* of chosen or visited things: Hamiltonian paths, assigning n tasks to n people ($dp[mask]$ is the best cost of giving the first $|mask|$ people the tasks in $mask$), covering, packing into bins, partitioning into groups.

19.2 Interval DP

GOLD

When a problem removes, merges or splits *contiguous* parts of a sequence, the state is a segment: $dp[l][r]$ is the answer for $a_l, \ldots, a_r$. The transition either removes one end ($dp[l+1][r]$ and $dp[l][r-1]$) or tries every split point k,

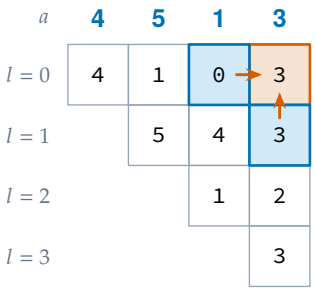
$$dp[l][r] = \min_{l \le k < r} \big(dp[l][k] + dp[k+1][r]\big) + cost(l, r),$$

and the states are computed by increasing length, so that every shorter segment is ready. Splits cost $O(n^3)$ in total, and Knuth's optimization (Section 20.4) often brings them to $O(n^2)$.

Example: Removal Game (CSES)

Two players take turns removing the first or the last number of a list ($n \le 5000$); each wants to maximise the sum of their own numbers. What is the first player's total when both play optimally?

Solution. Store the *difference* $d[l][r]$, the best (mover's total minus the other's total) on $a_l, \ldots, a_r$. The mover takes an end and then becomes the other player of the rest: $d[l][r] = \max(a_l - d[l+1][r],\ a_r - d[l][r-1])$ (Figure 19.2). The first player's total is $(\text{sum} + d[0][n-1])/2$.



$d[l][r]$ for $a = 4, 5, 1, 3$; row l, column r. For the whole list: taking 4 leaves $d[1][3] = 3$ to the opponent, $4 - 3 = 1$; taking 3 leaves $d[0][2] = 0$, $3 - 0 = 3$. So $d[0][3] = 3$, and the first player collects $(13 + 3)/2 = 8$.

Figure 19.2: The difference table of Removal Game; each cell looks at the cell below it and the cell to its left.

```
void solve() {
    int n;
    cin >> n;
    vi a(n);
    ll total = 0;
```

```
    for (ll& x : a) cin >> x, total += x;
    // d[l]: the best (mover's total - other's total) on the segment of the
    // current length that starts at l; lengths grow, so d[l + 1] and d[l]
    // still hold the values for the segments one shorter
    vi d(a);
    rep(len, 2, n + 1) rep(l, n - len + 1)
        d[l] = max(a[l] - d[l + 1], a[l + len - 1] - d[l]);
    print((total + d[0]) / 2);
}
```

The full table would need $n^2 = 2.5 \cdot 10^7$ numbers; going by length, one array suffices, because the segment of length ℓ at l needs only the two segments of length $\ell - 1$ at l and $l + 1$, and $l + 1$ has not been overwritten yet when l is computed. The program runs in 0.07 s.

</> Template: Range DP Ctrl+Alt+T, - prefix `cprangedp`

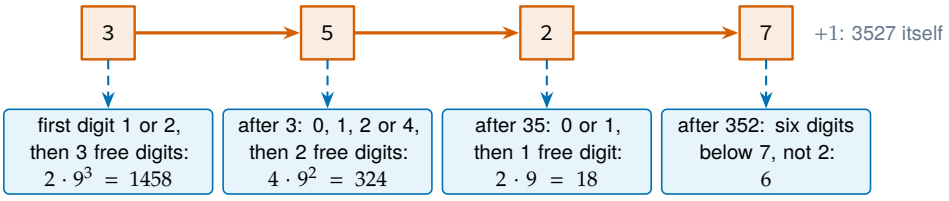
One number for two players In a two-player zero-sum game, store the *difference* between the player to move and the other player. Since the roles swap after every move, the recurrence is always "my gain now minus the opponent's best difference afterwards", and a single table replaces two. The same trick turns any "both play optimally" DP into a maximisation.

How to recognize it Merging adjacent things (piles of stones, matrix chain multiplication), removing elements whose neighbours then change, palindromes ("fewest insertions to make a palindrome"), parenthesisations, and games that take from either end. $n \le 500$ is typical for $O(n^3)$, $n \le 5000$ for $O(n^2)$.

19.3 Digit DP

PLATINUM

"How many integers in $[a, b]$ have property P?" with b up to 10^{18} is answered as $f(b) - f(a-1)$, where $f(x)$ counts the good integers in $[0, x]$. To count up to x, build the integer digit by digit from the most significant one. As long as every digit so far equals x's digit, the integer is *tight*: the next digit may not exceed x's next digit. The first time a smaller digit is chosen, the integer is already below x, every later digit is free, and the count of completions no longer depends on x at all (Figure 19.3). So the state is (position, tight, and whatever the property needs, such as the previous digit), only a few hundred states for 18 digits.



Shorter numbers add $9 + 81 + 729 = 819$ and the number 0 adds 1, so $f(3527) = 1 + 819 + 1458 + 324 + 18 + 6 + 1 = 2627$ numbers in $[0, 3527]$ have no two equal adjacent digits.

Figure 19.3: Digit DP for Counting Numbers with $x = 3527$. The tight path (red) follows x; each dashed branch leaves it with a smaller digit, after which all later digits are free, so its count is a closed formula.

Example: Counting Numbers (CSES)

Count the integers in $[a, b]$ ($0 \le a \le b \le 10^{18}$) in which no two adjacent digits are equal.

Solution. For this property the free completions have a closed form: after a fixed digit, each later digit avoids only its left neighbour, so k free digits give 9^k completions. The count is then a walk along the tight path with one branch per position, exactly the sum of Figure 19.3. For properties without a closed form, the same walk with a memoized DP over (position, tight, extra state) is the general method, and the template implements it.

```
void solve() {
    ll a, b;
    cin >> a >> b;
    // the numbers in [0, x] with no two equal adjacent digits
    auto count = [](ll x) -> ll {
        if (x < 10) return x + 1;  // also right for x = -1: none
        string s = to_string(x);
        int L = sz(s);
        vi pw(L + 1, 1);  // 9^k: each later digit avoids only its neighbour
        rep(i, 1, L + 1) pw[i] = pw[i - 1] * 9;
        ll res = 1;  // the number 0
        rep(len, 1, L) res += pw[len];  // every shorter length
        rep(i, L) {  // length L, first smaller than x at digit i
            int lo = i == 0 ? 1 : 0;
            rep(d, lo, s[i] - '0')
                if (i == 0 || d != s[i - 1] - '0') res += pw[L - 1 - i];
            if (i > 0 && s[i] == s[i - 1]) return res;  // x's prefix is bad
        }
        return res + 1;  // x itself
    };
    print(count(b) - count(a - 1));
}
```

</> Template: Digit DP Ctrl+Alt+T, = prefix `cpdigit`

Leading zeros and the empty prefix When the property depends on the digits actually written ("no two equal adjacent digits", "the digit sum is prime"), leading zeros must not count as digits: 0012 is the number 12, whose first digit is 1. Either count each shorter length separately, as above, or add a `started` flag to the state. Also check $f(a-1)$ for $a = 0$: it must be 0, not an error.

19.4 Boundaries and profiles

PLATINUM

When a grid is filled piece by piece, the part already filled matters to the future only through its *boundary*. If the grid is narrow, the boundary is small enough to be the state.

A two-state boundary. In Counting Towers (CSES), a tower of width 2 and height n is cut into rectangular blocks; count the ways. Build it row by row from the bottom. What the next row may do depends only on whether the top row is one block of width 2 (state a) or two separate blocks of width 1 (state b), as in Figure 19.4. From a, the next row either continues the wide block,

starts a new wide block, or starts two narrow ones; from b, each column continues its block or starts a new one ($2 \cdot 2 = 4$ ways), or both blocks end and a wide one starts. So $a_{h+1} = 2a_h + b_h$ and $b_{h+1} = a_h + 4b_h$, with $a_1 = b_1 = 1$.

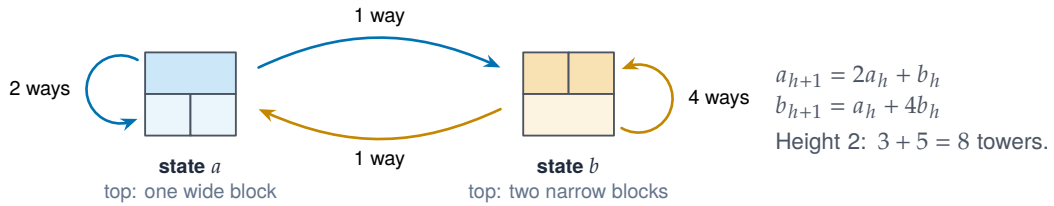
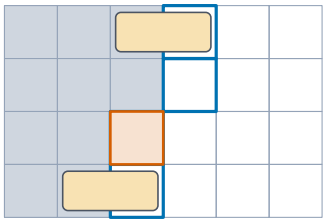


Figure 19.4: Counting Towers as a two-state machine: the future of a partial tower depends only on the shape of its top row.

```
void solve() {
    int t;
    cin >> t;
    const int N = 1000000;
    // a[h]: towers of height h whose top row is one block of width 2;
    // b[h]: towers whose top row is two different blocks of width 1.
    // From a, the next row grows the wide block or starts a new one (to a),
    // or starts two narrow blocks (to b); from b, each column grows its block
    // or starts a new one (4 ways, to b), or a new wide block starts (to a).
    vi a(N + 1), b(N + 1);
    a[1] = b[1] = 1;
    rep(h, 1, N) {
        a[h + 1] = (2 * a[h] + b[h]) % MOD;
        b[h + 1] = (a[h] + 4 * b[h]) % MOD;
    }
    while (t--) {
        int n;
        cin >> n;
        print((a[n] + b[n]) % MOD);
    }
}
```

Broken profiles. For a wider grid the boundary is a bitmask. In Counting Tilings (CSES), an $n \times m$ grid with $n \le 10$ is tiled with dominoes. Decide the cells one at a time, column by column and top to bottom within a column. The *profile* is the set of n cells just beyond the decided ones: in the current column below the current cell, and in the next column above it (Figure 19.5). Bit r of the state says whether that cell is already covered by a domino sticking out of a decided cell. At the current cell there are three cases: it is already covered (clear the bit and move on), a horizontal domino covers it and the cell to its right (set the bit), or a vertical domino covers it and the cell below (set the next bit, if that cell is free). That is $nm\,2^n$ steps of constant work, 10^7 at the limits.

```
void solve() {
    int n, m;
    cin >> n >> m;
    // cells are decided column by column, top to bottom; bit r of the mask
    // tells whether cell r of the frontier is already covered (rows above
    // the current cell: in the next column, the others: in this column)
    vi dp(1 << n, 0), nd(1 << n);
```



Grey: decided. Blue frames: the profile, here the cells $(0,3)$, $(1,3)$, $(2,2)$, $(3,2)$, one per row. Two dominoes (orange) stick into it, so the state is the mask 1001 (rows 3 and 0 covered). The current cell (red) is free: place a horizontal domino into column 3, or a vertical one over row 3, which is not possible here because that cell is covered.

Figure 19.5: The broken profile of Counting Tilings, column 2, row 2 of a 4×6 grid.

```
    dp[0] = 1;
    rep(c, m) rep(r, n) {
        fill(all(nd), 0);
        rep(mask, 1 << n) {
            ll val = dp[mask];
            if (!val) continue;
            if (mask >> r & 1) {  // covered from the left: move on
                int to = mask ^ 1 << r;
                nd[to] = (nd[to] + val) % MOD;
                continue;
            }
            int to = mask | 1 << r;  // a horizontal domino into column c + 1
            nd[to] = (nd[to] + val) % MOD;
            if (r + 1 < n && !(mask >> (r + 1) & 1)) {  // a vertical domino
                to = mask | 1 << (r + 1);
                nd[to] = (nd[to] + val) % MOD;
            }
        }
        swap(dp, nd);
    }
    print(dp[0]);
}
```

</> Template: Broken profile DP (domino tilings) Ctrl+Alt+Y, Shift+P prefix `cpprofile`

Deciding one *cell* at a time, rather than a whole column at once, is what makes the profile "broken": a whole column would need $2^n \times 2^n$ transitions per column, cell by cell only three per state. The program needs 0.06 s for a 10×1000 grid. The same idea counts colourings with local constraints, placements of non-attacking pieces, and paths in narrow grids.

19.5 Sums over subsets

PLATINUM

Given a value $f(T)$ for every subset T of B bits, the *zeta transform* is $F(S) = \sum_{T \subseteq S} f(T)$ for every S. Enumerating submasks costs $O(3^B)$; the **sum over subsets** (SOS) DP does it in $O(B \cdot 2^B)$ by adding one bit dimension at a time:

```
rep(i, B) rep(m, 1 << B)
    if (m >> i & 1) f[m] += f[m ^ 1 << i];
```

Why it works. After the passes for bits $0, \ldots, i$, $f[m]$ holds the sum of the original values of the masks T that agree with m on bits $i+1, \ldots, B-1$ and are submasks of m on bits $0, \ldots, i$. The pass for bit i adds, to every m with bit i set, the value of m without bit i, which covers exactly

the submasks that have bit i cleared (Figure 19.6). After the last pass, $f[m] = F(m)$. Running the passes with − instead of + undoes the transform (the Möbius transform), and flipping the condition (`!(m >> i & 1)`, adding $f[m \mid 2^i]$) gives sums over *supersets*.

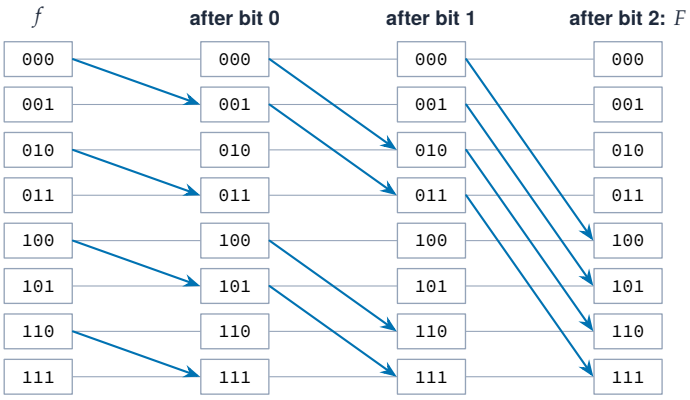


Figure 19.6: The SOS DP for three bits as a butterfly network. In the pass for bit i, every mask with bit i set also receives the value of the mask without it (blue). After three passes each mask holds the sum over all its submasks; for example 111 has collected all eight values, along exactly one path each.

Example: Bit Problem (CSES)

Given $n \le 2 \cdot 10^5$ numbers $x_i \le 10^6$, report for every x_i the number of x_j with $x_i \mid x_j = x_i$, the number with $x_i \,\&\, x_j = x_i$, and the number with $x_i \,\&\, x_j \neq 0$.

Solution. The first condition says x_j is a submask of x_i: subset sums of the count array. The second says x_j is a supermask: superset sums. The third is the complement of "x_j is a submask of $\neg x_i$". One pass over 20 bits computes both tables; 0.34 s in all.

```
void solve() {
    int n;
    cin >> n;
    const int B = 20;  // x <= 10^6 < 2^20
    vi x(n), sub(1 << B, 0), sup(1 << B, 0);
    for (ll& v : x) cin >> v, sub[v]++, sup[v]++;
    // sub[m]: how many x_j are submasks of m; sup[m]: how many are supermasks
    rep(i, B) rep(m, 1 << B) {
        if (m >> i & 1) sub[m] += sub[m ^ 1 << i];
        else sup[m] += sup[m ^ 1 << i];
    }
    int full = (1 << B) - 1;
    // x_j | x = x, x_j & x = x, and x_j & x != 0 (all but the submasks of ~x)
    for (ll v : x) print(sub[v], sup[v], n - sub[full ^ v]);
}
```

</> Template: SOS DP Ctrl+Alt+Y, U prefix `cpsos`

19.5.1 Subset convolution

The *subset convolution* of a and b is $c(S) = \sum_{T \subseteq S} a(T)\, b(S \setminus T)$: the sum over all ways to split S into two disjoint parts. Its brute force is $O(3^n)$. The zeta transform alone cannot do it, because the pointwise product of two zeta transforms counts pairs (T, U) with $T \cup U = S$ that may *overlap*.

The fix is to remember sizes: split a by the number of elements, $a_k(T) = a(T)$ if $|T| = k$ and 0 otherwise, transform every a_k and b_k, multiply them as polynomials in the size, $h_k = \sum_{i+j=k} \hat{a}_i \hat{b}_j$, and transform back. A pair (T, U) with $T \cup U = S$ and $|T| + |U| = |S|$ is necessarily disjoint, so $c(S)$ is the inverse transform of $h_{|S|}$ at S. The cost is $O(n^2 2^n)$: about 1 s for $n = 20$ with the template, which also keeps the memory to 176 MB by storing 32-bit values.

Example: Disjoint splits

Given $a(T)$ and $b(U)$ for every subset of $n \le 20$ elements, compute for every set S the sum of $a(T)\,b(S \setminus T)$ over all splits of S, modulo $10^9 + 7$. (With $a = b =$ the indicator of the "valid" groups, $c(S)$ counts the ordered ways to split S into two valid groups.)

```
void solve() {  // cpsubsetconv is pasted above solve()
    int n;
    cin >> n;
    vi a(1 << n), b(1 << n);
    for (ll& x : a) cin >> x;
    for (ll& x : b) cin >> x;
    print(subset_convolution(a, b));
}
```

</> Template: Subset convolution Ctrl+Alt+U, Shift+S prefix `cpsubsetconv`

How to recognize it "For every mask, sum (or count, or take the maximum) over its submasks or supermasks"; pairs with conditions like $x \,\&\, y = 0$ or $x \mid y = x$; values up to 2^{20}. Splitting a set into two or more disjoint parts with independent costs is a subset convolution, and repeating it k times gives the ways to split into k parts (Chapter 26 also covers the XOR and AND convolutions).

19.6 Scheduling with sorted events

GOLD

When the items are intervals in time, sorting them by *end* time gives a natural order: a solution that ends with interval i consists of i and a solution among the intervals that end before i starts, which form a prefix of the sorted list. Binary search finds that prefix.

Example: Projects (CSES)

There are $n \le 2 \cdot 10^5$ projects, each with a start day, an end day and a reward. You can work on one project per day, and a project must be done entirely. Maximise the total reward.

Solution. Sort by end day. $dp[i]$ is the best reward using only the first i projects: either project i is skipped, $dp[i-1]$, or it is taken together with the best of the projects that end before its start day, $dp[j] + p_i$ (Figure 19.7). $O(n \log n)$, 0.26 s at the limits.

```
void solve() {
    int n;
    cin >> n;
    vector<array<ll, 3>> pr(n);  // (end day, start day, reward)
    for (auto& p : pr) cin >> p[1] >> p[0] >> p[2];
```

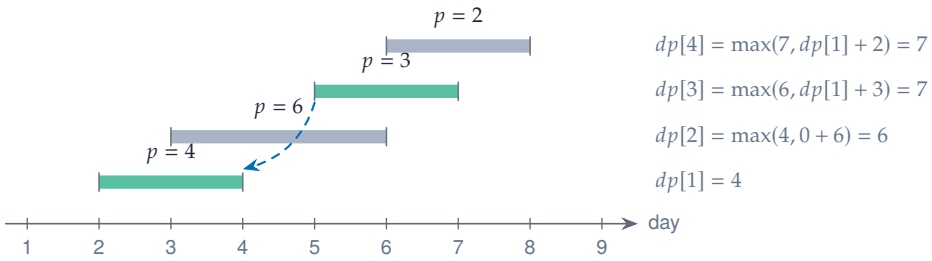


Figure 19.7: Projects sorted by end day. The project $[5, 7]$ is compatible with the projects that end before day 5, here only the first one (dashed), so taking it gives $dp[1] + 3 = 7$. The best schedule takes the two green projects.

```
    sort(all(pr));
    vi ends(n);
    rep(i, n) ends[i] = pr[i][0];
    // dp[i]: the best total reward using only the first i projects by end
    vi dp(n + 1, 0);
    rep(i, n) {
        // projects ending before this one's start day are compatible with it
        int j = lower_bound(all(ends), pr[i][1]) - ends.begin();
        dp[i + 1] = max(dp[i], dp[j] + pr[i][2]);  // skip it, or take it
    }
    print(dp[n]);
}
```

19.7 Probability and expectation

PLATINUM

A probability DP replaces "number of ways" with "probability", and the transitions multiply by the probability of each choice. Expected values add one more idea.

Linearity of expectation $E[X + Y] = E[X] + E[Y]$ for *any* random variables, independent or not. So the expected number of events that happen is the sum of their probabilities: the expected number of fixed points of a random permutation is $n \cdot \frac{1}{n} = 1$, whatever n is.

Example: Dice Probability (CSES)

Throw $n \leq 100$ dice. What is the probability that the sum is between a and b?

Solution. The distribution of the sum after i dice is a DP over the sum: each value spreads $\frac{1}{6}$ of its probability to the six next sums (Figure 19.8).

```
void solve() {
    int n, a, b;
    cin >> n >> a >> b;
    vector<double> p(6 * n + 1, 0.0);  // the distribution of the sum so far
    p[0] = 1;
    rep(i, n) {
        vector<double> q(6 * n + 1, 0.0);
        rep(s, 6 * i + 1) if (p[s] > 0)
            rep(d, 1, 7) q[s + d] += p[s] / 6;
```

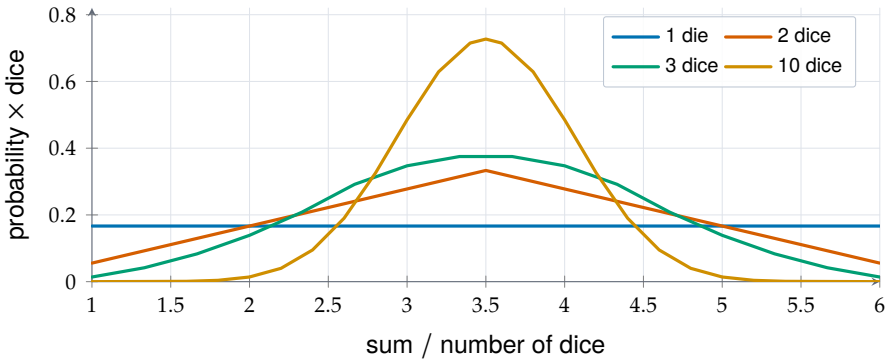


Figure 19.8: The distribution of the sum of n dice, rescaled to the average per die. One die is flat, two give a triangle, and ten are already close to a bell curve centred at 3.5.

```
        swap(p, q);
    }
    double ans = 0;
    rep(s, a, b + 1) ans += p[s];
    cout << fixed << setprecision(6) << ans << '\n';
}
```

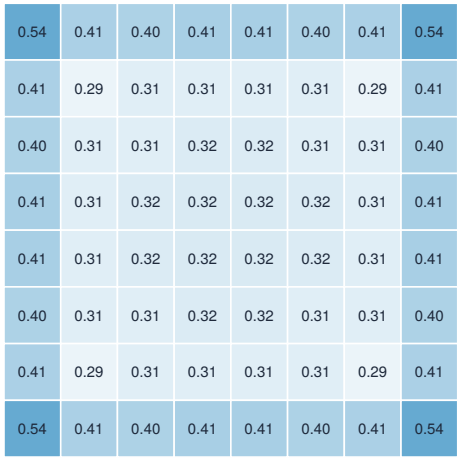
Counting with $P(\max \le x)$. The maximum of independent values is awkward directly but easy through its cumulative distribution: $\max \le x$ exactly when every value is $\le x$. In Candy Lottery (CSES), each of n children gets a uniform number of candies in $1..k$, and $P(\max \le x) = (x/k)^n$, so $P(\max = x) = (x/k)^n - ((x-1)/k)^n$ and the expectation is a single sum.

```
void solve() {
    int n, k;
    cin >> n >> k;
    // P(max <= x) = (x / k)^n, so P(max = x) is a difference of two powers
    double ans = 0;
    rep(x, 1, k + 1) {
        double le = pow((double)x / k, n), lt = pow((double)(x - 1) / k, n);
        ans += x * (le - lt);
    }
    cout << fixed << setprecision(6) << ans << '\n';
}
```

Example: Moving Robots (CSES)

Each square of a chessboard has a robot. In each of $k \le 100$ steps, every robot moves to a uniformly random neighbouring square. What is the expected number of empty squares at the end?

Solution. By linearity, the expected number of empty squares is the sum over squares of the probability that the square is empty. The robots move independently, so that probability is the product, over all robots, of the probability that the robot is not there; and where one robot ends is a 64-state probability DP over k steps (Figure 19.9).



The probability that each square is empty after $k = 10$ steps. Corner squares have only two neighbours, so robots leave them more often than they arrive. The expected number of empty squares is the sum of the 64 values, 23.12.

Figure 19.9: Moving Robots: a heat map of the per-square probabilities whose sum is the answer.

```
void solve() {
    int k;
    cin >> k;
    // the robots move independently, so a cell is empty with probability
    // prod over robots (1 - P(this robot ends there)); add over the cells
    vector<double> empty(64, 1.0);
    rep(start, 64) {
        vector<double> p(64, 0.0);  // where this robot is after each step
        p[start] = 1;
        rep(step, k) {
            vector<double> q(64, 0.0);
            rep(c, 64) {
                if (p[c] == 0) continue;
                int r = c / 8, col = c % 8;
                vi nb;
                if (r > 0) nb.push_back(c - 8);
                if (r < 7) nb.push_back(c + 8);
                if (col > 0) nb.push_back(c - 1);
                if (col < 7) nb.push_back(c + 1);
                for (int d : nb) q[d] += p[c] / sz(nb);
            }
            swap(p, q);
        }
        rep(c, 64) empty[c] *= 1 - p[c];
    }
    double ans = 0;
    for (double e : empty) ans += e;
    cout << fixed << setprecision(6) << ans << '\n';
}
```

Expected values with loops. An expected-value DP usually runs backwards from the end: $E[s]$, the expected remaining cost from state s, is the cost of one step plus $\sum_t P(s \to t)\,E[t]$. When a state can return to itself with probability p, the equation contains $E[s]$ on both sides and is solved for it: rolling a die until a 6 appears takes $E = 1 + \frac{5}{6}E$ throws, so $E = 6$, and in general an experiment that succeeds with probability p needs $1/p$ tries on average. When states form longer cycles, the equations form a linear system (Gaussian elimination, Chapter 26).

Expectations modulo a prime When an expected value is requested "as $P \cdot Q^{-1} \bmod 998244353$", run the whole DP in modular arithmetic: every division by a probability's denominator becomes a multiplication by a modular inverse (Chapter 24), and the result is exact. Floating point is only for problems that print decimals, and then `double` with error $\le 10^{-6}$ is enough far more often than `long double` is needed.

19.8 Games

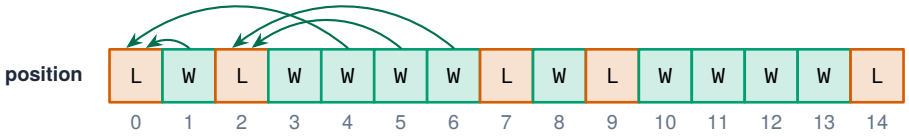
GOLD

In a two-player game with perfect information, no chance and no draws, where the player who cannot move loses, every position is either **winning** for the player to move or **losing**. A position is winning exactly when *some* move leads to a losing position, and losing when *every* move (possibly none) leads to a winning one. Positions that come later in play are decided first, so this is a DP over the DAG of positions, in reverse order of play.

Example: Stick Game (CSES)

A heap has $n \le 10^6$ sticks, and a move removes $p_1, p_2, \ldots$ or p_k of them ($k \le 100$). For every heap size from 1 to n, does the first player win?

Solution. $win[0]$ is false (no move), and $win[i]$ is true when some $p_j \le i$ has $win[i - p_j]$ false (Figure 19.10): $O(nk)$, 0.11 s.



Moves remove 1, 3 or 4 sticks. Every winning position (W) has a move to a losing one (arrows: from 1, 4, 5 and 6 sticks); from 0, 2 and 7 sticks every move reaches a W. The pattern repeats with period 7, which is typical: game DPs over a single number often become periodic.

Figure 19.10: Winning and losing positions of the Stick Game with moves $\{1, 3, 4\}$.

```
void solve() {
    int n, k;
    cin >> n >> k;
    vi p(k);
    for (ll& x : p) cin >> x;
    // win[i]: the player to move with i sticks left wins; with 0 sticks
    // there is no move, so the player to move has lost
    vector<bool> win(n + 1, false);
    rep(i, 1, n + 1) for (ll x : p)
        if (x <= i && !win[i - x]) {  // a move to a losing position
            win[i] = true;
            break;
        }
    string s;
    rep(i, 1, n + 1) s += win[i] ? 'W' : 'L';
    print(s);
}
```

When a position is a *sum* of independent games (several heaps, the player moves in one of them), the win/lose labels of the parts do not determine the label of the sum; their Grundy numbers do, and Nim is the model (Chapter 25).

19.9 Linear recurrences and matrix powers

GOLD

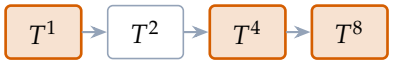
When the state is a small vector and every step transforms it by the same linear map, n steps are the n-th power of a matrix, and fast exponentiation computes it with $O(\log n)$ matrix products. For a state of size k that is $O(k^3 \log n)$: for $k = 6$ and $n = 10^{18}$, about 60 products of 6×6 matrices.

Example: Throwing Dice (CSES)

Count the ordered ways to reach the sum n with dice, modulo $10^9 + 7$, for $n \le 10^{18}$.

Solution. Dice Combinations (Section 18.2) with a huge n. The recurrence $f(s+1) = f(s) + f(s-1) + \cdots + f(s-5)$ moves the vector of the last six values by a fixed matrix (Figure 19.11); since $f(0) = 1$ and earlier values are 0, $f(n)$ is the top-left entry of T^n. The program takes 0.08 s.

$$\begin{pmatrix} f(s+1) \\ f(s) \\ f(s-1) \\ f(s-2) \\ f(s-3) \\ f(s-4) \end{pmatrix} = \underbrace{\begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}}_{T} \begin{pmatrix} f(s) \\ f(s-1) \\ f(s-2) \\ f(s-3) \\ f(s-4) \\ f(s-5) \end{pmatrix}$$



Each power is the square of the one before it. $n = 13 = 1101_2$, so $T^{13} = T^8 \cdot T^4 \cdot T^1$ (red).

Figure 19.11: Top: one step of the dice recurrence as a matrix product. Bottom: fast exponentiation squares T repeatedly and multiplies in the powers that match the set bits of n.

```
void solve() {
    ll n;
    cin >> n;
    using Mat = array<array<ll, 6>, 6>;
    auto mul = [](const Mat& A, const Mat& B) {
        Mat C{};
        rep(i, 6) rep(k, 6) if (A[i][k]) rep(j, 6)
            C[i][j] = (C[i][j] + A[i][k] * B[k][j]) % MOD;
        return C;
    };
    // the vector (f(s), f(s - 1), ..., f(s - 5)) moves one step by T
    Mat T{}, R{};
    rep(j, 6) T[0][j] = 1;  // f(s + 1) = f(s) + ... + f(s - 5)
    rep(i, 1, 6) T[i][i - 1] = 1;  // the other entries shift down
    rep(i, 6) R[i][i] = 1;
    for (ll e = n; e > 0; e >>= 1, T = mul(T, T))
        if (e & 1) R = mul(R, T);
```

```
    print(R[0][0]);  // T^n applied to (f(0), 0, ..., 0) = (1, 0, ..., 0)
}
```

Template: Matrix exponentiation Ctrl+Alt+T, Shift+X prefix `cpmatrix`

Beyond addition and multiplication A matrix power works over any *semiring*. With (min, +) in place of (+, ×), the n-th power of the weighted adjacency matrix gives the shortest walks with exactly n edges (Graph Paths II); with ordinary arithmetic it counts walks of length n (Graph Paths I); with (OR, AND) it gives reachability in exactly n steps. For a recurrence of order k in the thousands, the $O(k^3 \log n)$ matrix power is too slow, and the polynomial methods of Chapter 26 (Kitamasa, Berlekamp–Massey) take $O(k^2 \log n)$ or $O(k \log k \log n)$.

How to recognize it n up to 10^{18}, a DP whose state is small ($k \le 100$) and whose transition does not depend on the step: linear recurrences, counting walks of length n in a small graph, DP over positions with a small alphabet, tilings of a $k \times n$ strip.

Chapter summary

- Bitmask DP handles sets of up to about 20 items in $O(2^n n^2)$; pair-valued states and dominance can replace $O(3^n)$ submask enumeration, which is itself 3^n and not 4^n.
- Interval DP fills $dp[l][r]$ by increasing length; for games, storing the difference of the two players' scores turns the game into a single maximisation.
- Digit DP counts up to 10^{18} with a tight flag: after the first smaller digit, the count of completions is independent of the bound.
- A narrow grid is filled cell by cell with its boundary (profile) as a bitmask: $O(nm\,2^n)$.
- SOS DP sums over subsets in $O(B\,2^B)$; ranking by size turns it into subset convolution in $O(n^2 2^n)$.
- Linearity of expectation reduces an expected count to a sum of probabilities; maxima go through $P(\max \le x)$.
- Win/lose positions are a DP in reverse order of play; fixed linear transitions jump 10^{18} steps with matrix powers.

Practice problems (CSES Problem Set)

SETS, INTERVALS, DIGITS AND PROFILES

- Hamiltonian Flights
- Elevator Rides
- Removal Game
- Counting Numbers
- Counting Towers
- Counting Tilings
- Bit Problem
- Projects

PROBABILITY, GAMES AND MATRICES

- Dice Probability
- Candy Lottery
- Moving Robots
- Stick Game
- Throwing Dice
- Fibonacci Numbers
- Graph Paths I
- Graph Paths II

CHAPTER 20

DP Optimizations

A correct DP that is too slow can usually be accelerated without changing its states, because the transitions that make it slow ("the minimum over a range of earlier states", "the best split point", "the best earlier line") have structure that a data structure or a monotonicity argument can exploit. This chapter goes through the standard accelerations, roughly in increasing order of difficulty, each with the shape of transition it recognises, the reason it is correct, and a problem solved with it. Table 20.1 is the index; the rest of the chapter explains every row.

IN THIS CHAPTER GOLD PLATINUM IOI / ADVANCED

- replace range transitions by prefix sums and monotone queues, including an exact $O(nW)$ bounded knapsack;
- use the monotonicity of optimal split points: divide and conquer, monotone minima and SMAWK, and Knuth's optimization;
- turn "the best earlier state" into "the lowest line" with the convex hull trick, in its three forms;
- keep convex value functions by their breakpoints (slope trick), and remove a "use exactly k" dimension (Aliens trick);
- convolve convex sequences in linear time, and know when bitsets or swapping dimensions are the right tool.

Before you start: Chapters 18 and 19, and the monotonic deque of Section 9.5.

transition	technique	speed-up
$dp_i = \sum_{j \in \text{window}} dp_j$, or the minimum over a sliding window	prefix sums, monotone queue	$O(nk) \to O(n)$
$dp[g][i] = \min_j dp[g-1][j] + cost(j, i)$, best j monotone in i	divide and conquer	$O(n^2) \to O(n \log n)$ per layer
row minima of a totally monotone matrix	SMAWK	$O(n \log n) \to O(n)$
$dp[l][r] = \min_m dp[l][m] + dp[m+1][r] + cost(l, r)$	Knuth	$O(n^3) \to O(n^2)$
$dp_i = \min_j(k_j x_i + m_j)$	convex hull trick	$O(n^2) \to O(n)$ or $O(n \log n)$
a convex piecewise-linear value function	slope trick	$O(nV) \to O(n \log n)$
"exactly k" with an answer concave in k	Aliens trick	removes the factor k
$c_k = \min_{i+j=k} a_i + b_j$, a and b convex	merging slopes	$O(nm) \to O(n+m)$
boolean rows shifted and OR-ed	bitset	divides by 64

Table 20.1: Which optimization fits which shape of transition.

20.1 Windows and monotone queues

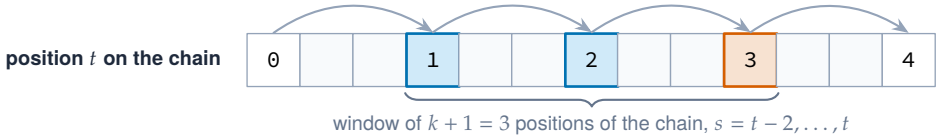
GOLD

The simplest accelerations replace a loop over a range of earlier states. If $dp_i = \sum_{j=i-k}^{i-1} dp_j$, keep the prefix sums of dp and each state costs $O(1)$. If $dp_i = c_i + \min_{i-k \le j < i} dp_j$, keep the monotonic deque of Section 9.5: indices with increasing values, the front being the minimum of the window. Both turn $O(nk)$ into $O(n)$.

The exact bounded knapsack. Section 18.5 handled an item with k copies by binary splitting, in $O(W \log k)$. A monotone queue does it in $O(W)$. Fix the remainder r of the capacity modulo the item's weight w: the capacities $c = r, r+w, r+2w, \ldots$ form a chain, and with t copies the new value at $c = r + tw$ is

$$new[r+tw] = \max_{t-k \le s \le t} \big(dp[r+sw] + (t-s)v\big) = tv + \max_{t-k \le s \le t} \big(dp[r+sw] - sv\big),$$

the maximum of a sliding window of length $k+1$ along the chain (Figure 20.1). Every capacity enters and leaves the queue once per item, so each item costs $O(W)$.



The cells are capacities $0, 1, 2, \ldots$; for an item of weight $w = 3$ only capacities with the same remainder modulo w interact, and the numbered cells are the chain $c = r + tw$. Along it, the new value at $t = 3$ is the best of the three window cells $s = 1, 2, 3$, each corrected by $-sv$; the deque gives that maximum in amortised $O(1)$.

Figure 20.1: The bounded knapsack along one remainder class, with $k = 2$ copies.

```
void solve() {
    int n, cap;
    cin >> n >> cap;
    vi dp(cap + 1, 0), nd(cap + 1), qs(cap + 1), qv(cap + 1);
    rep(i, n) {
        ll w, v, k;
        cin >> w >> v >> k;
        // capacities with the same remainder mod w form a chain c = r + t*w:
        // nd[c] = t*v + max over t - k <= s <= t of (dp[r + s*w] - s*v),
        // a sliding-window maximum (a deque kept in the arrays qs, qv)
        rep(r, min<ll>(w, cap + 1)) {
            int head = 0, tail = 0;  // the window, values decreasing
            for (ll t = 0, c = r; c <= cap; t++, c += w) {
                ll val = dp[c] - t * v;
                while (tail > head && qv[tail - 1] <= val) tail--;
                qs[tail] = t, qv[tail++] = val;
                if (qs[head] < t - k) head++;  // at most k copies of item i
                nd[c] = qv[head] + t * v;
            }
        }
        swap(dp, nd);
    }
    print(dp[cap]);
}
```

The deque lives in two arrays reused for every chain, because creating a `std::deque` per chain would cost more than the DP itself when the weights are large. With 100 kinds of 1000 copies each and $W = 10^5$, binary splitting takes 0.09 s and the queue 0.06 s; the gap grows with k, and the queue's bound does not depend on k at all.

20.2 Divide and conquer optimization

PLATINUM

A *layered* DP splits the first i elements into g groups:

$$dp_g(i) = \min_{j<i}\big(dp_{g-1}(j) + cost(j, i)\big),$$

where $cost(j, i)$ is the cost of the group $a_j, \ldots, a_{i-1}$. Computed directly, each layer costs $O(n^2)$. Let $opt(i)$ be the best j (the leftmost one, if several tie). In many problems $opt(i) \le opt(i+1)$: the last group never starts earlier when one more element arrives. Then compute the middle row i first by scanning all candidates; its optimum bounds both halves, rows left of the middle only scan $j \le opt(i)$ and rows right of it only $j \ge opt(i)$ (Figure 20.2). Every level of the recursion scans about n candidates in total, so a layer costs $O(n \log n)$.

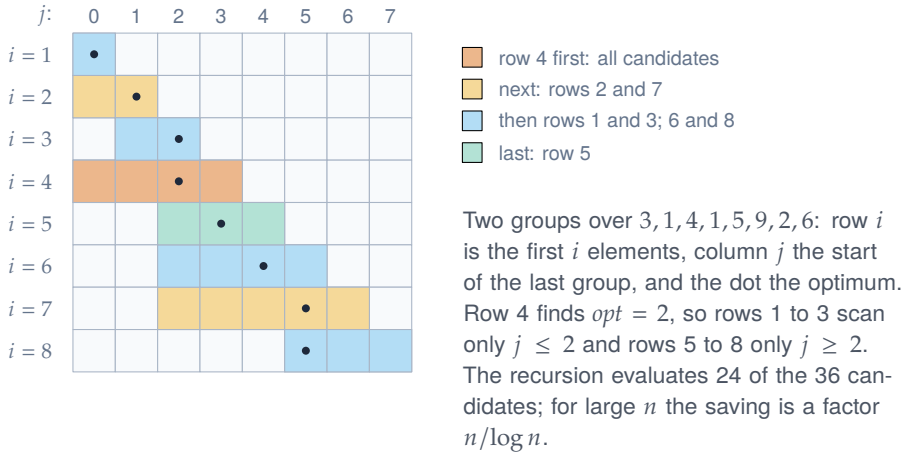


Figure 20.2: Divide and conquer optimization: each row's optimum bounds the rows below and above it in the recursion.

Example: Subarray Squares (CSES)

Split an array of $n \le 3000$ positive numbers into k contiguous parts so that the sum of the squares of the parts' sums is as small as possible.

Solution. k layers of the DP above with $cost(j, i) = (pre_i - pre_j)^2$, each by divide and conquer: $O(kn \log n)$, 0.12 s for $n = k = 3000$.

```
void solve() {
    int n, k;
    cin >> n >> k;
    vi pre(n + 1, 0);
    rep(i, n) {
        ll x;
        cin >> x;
        pre[i + 1] = pre[i] + x;
```

```
    }
    auto cost = [&](int j, int i) {  // the group a[j, i)
        ll s = pre[i] - pre[j];
        return s * s;
    };
    // prev[i]: the best split of the first i elements into g - 1 groups;
    // cur[i]: into g groups
    vi prev(n + 1, INF), cur(n + 1);
    prev[0] = 0;
    // fill cur[lo, hi) knowing that their best last split lies in [olo, ohi]
    auto go = [&](auto&& self, int lo, int hi, int olo, int ohi) -> void {
        if (lo >= hi) return;
        int mid = (lo + hi) / 2, best = olo;
        cur[mid] = INF;
        rep(j, olo, min(mid - 1, ohi) + 1) {  // the last group is a[j, mid)
            if (prev[j] == INF) continue;
            ll c = prev[j] + cost(j, mid);
            if (c < cur[mid]) cur[mid] = c, best = j;
        }
        self(self, lo, mid, olo, best);
        self(self, mid + 1, hi, best, ohi);
    };
    rep(g, 1, k + 1) {
        go(go, 0, n + 1, 0, n);
        swap(prev, cur);
    }
    print(prev[n]);
}
```

</> Template: Divide & conquer optimization Ctrl+Alt+Y, Q prefix cpdnc

When is the optimum monotone? A sufficient condition is the *quadrangle inequality*: $cost(a,c)+cost(b,d) \le cost(a,d)+cost(b,c)$ for all $a \le b \le c \le d$ ("crossing is cheaper than nesting"). It holds for (sum of the group)^2 with non-negative values, for the number of pairs inside a group, for the variance of a group and for the distance of the group's elements to their median. Its consequence is that the matrix $M[i][j] = dp_{g-1}(j) + cost(j,i)$ is *totally monotone*, which is exactly what the next section exploits further.

Tip In a contest, checking monotonicity with a brute force on random small inputs (compute every $opt(i)$ in $O(n^2)$ and assert that it never decreases) is faster and safer than proving the quadrangle inequality.

20.3 Monotone minima and SMAWK

IOI / ADVANCED

The divide and conquer of the previous section is an instance of a general problem. A matrix M (usually implicit: its entries are computed on demand) is *monotone* if the column of the leftmost minimum of each row never decreases from one row to the next. Finding all row minima of an $n \times m$ monotone matrix takes $O((n+m)\log n)$ evaluations with the same recursion: middle row first, then the halves with restricted columns. The template exposes exactly that, with the matrix given as a function.

</> Template: Monotone minima Ctrl+Alt+U, M prefix `cpmonominima`

If the matrix is *totally* monotone (every 2×2 submatrix is monotone, as for a Monge matrix), the **SMAWK** algorithm finds all row minima with only $O(n + m)$ evaluations. It alternates two steps. *Reduce*: when there are more columns than rows, compare entries along a stack of candidate columns and discard columns that can contain no row minimum, until at most n remain. *Interpolate*: solve the problem for the odd-numbered rows recursively, then fill in each even row by scanning only the columns between the minima of its two neighbouring rows. Both steps are linear, and the recursion halves the rows. SMAWK matters when $\log n$ is the difference between passing and failing, for example in some layered DPs with millions of states, or in the (min, +) convolution of Section 20.8; in most contests the $O((n + m) \log n)$ recursion is enough and much simpler to get right.

20.4 Knuth's optimization

PLATINUM

For interval DPs of the form

$$dp[l][r] = \min_{l \le m < r} \big(dp[l][m] + dp[m+1][r]\big) + cost(l, r),$$

let $opt(l, r)$ be the best split point. When *cost* satisfies the quadrangle inequality and is monotone under inclusion, the optimum is sandwiched between its neighbours (Figure 20.3):

$$opt(l, r-1) \le opt(l, r) \le opt(l+1, r).$$

Restricting the loop over m to that range makes the whole DP $O(n^2)$: for a fixed length, the ranges of consecutive l overlap only at their ends, so their total length telescopes to $opt(n - len, n - 1) - opt(0, len - 2) + n = O(n)$, and there are n lengths.

a	2	3	1	4	2	3
$l = 0$	0	0	0	1	2	2
$l = 1$		1	1	2	2	3
$l = 2$			2	2	3	3
$l = 3$				3	3	3
$l = 4$					4	4
$l = 5$						5

$opt(l, r)$ for Knuth Division on $2, 3, 1, 4, 2, 3$ (row l, column r). The whole array only tries the cuts $2 \le m \le 3$, the range given by its left neighbour $opt(0, 4) = 2$ and the cell below, $opt(1, 5) = 3$. The values never decrease along a row or down a column.

Figure 20.3: The table of optimal split points of an interval DP is monotone in both directions.

Example: Knuth Division (CSES)

An array of $n \le 5000$ numbers is cut into single elements; each step cuts one piece into two, at a cost equal to the sum of the piece. Minimise the total cost.

Solution. The first cut of $a_l, \ldots, a_r$ splits it after some m, and both parts are then solved independently: $cost(l, r)$ is the sum of the segment, which satisfies Knuth's conditions. The table of $2.5 \cdot 10^7$ cells is stored as a triangle, rows of decreasing length, and the optimum table in 32 bits, which keeps the memory near 150 MB; the program runs in 0.29 s.

```
void solve() {
    int n;
    cin >> n;
    vi pre(n + 1, 0);
    rep(i, n) {
        ll x;
        cin >> x;
        pre[i + 1] = pre[i] + x;
    }
    // row l stores the segments [l, r] at index r - l (a triangle):
    // D[l][r - l] is the cheapest way to cut a[l..r] into single elements,
    // O[l][r - l] the best first cut (it splits off [l, O] and [O + 1, r])
    vector<vi> D(n);
    vector<vector<int32_t>> O(n);
    rep(l, n) D[l].assign(n - l, 0), O[l].assign(n - l, l);
    rep(len, 2, n + 1) rep(l, n - len + 1) {
        int r = l + len - 1;
        // Knuth: opt(l, r - 1) <= opt(l, r) <= opt(l + 1, r)
        int lo = O[l][r - 1 - l], hi = min<int>(O[l + 1][r - l - 1], r - 1);
        ll best = INF;
        rep(m, lo, hi + 1) {
            ll c = D[l][m - l] + D[m + 1][r - m - 1];
            if (c < best) best = c, O[l][r - l] = m;
        }
        D[l][r - l] = best + pre[r + 1] - pre[l];
    }
    print(D[0][n - 1]);
}
```

</> Template: Knuth optimization Ctrl+Alt+Y, Shift+Q prefix `cpknuth`

How to recognize it Optimal merging or splitting of a sequence where the cost of an operation is the total of the piece (merging piles of stones, cutting a stick or an array, Huffman-like costs with a fixed order), and optimal binary search trees. n up to a few thousand with an $O(n^3)$ interval DP is the signal.

20.5 The convex hull trick

PLATINUM

When the transition has the form $dp_i = \min_{j<i}(k_j x_i + m_j)$ (usually after expanding a product such as $s_i f_j$ or a square $(pre_i - pre_j)^2$), every earlier state j is a *line* $y = k_j x + m_j$, and state i asks which line is lowest at $x = x_i$. The minimum of a set of lines is a concave piecewise-linear function, the *lower envelope*; lines that never appear on it can be discarded forever (Figure 20.4).

Monotone slopes: a deque. If the lines arrive with decreasing slopes, each new line enters the envelope at the right end, and the lines it makes useless are at that end too: a line b between a and c is useless when c overtakes b no later than b overtakes a. So the envelope is a stack, each line pushed and popped once. If the queries also come in increasing order, a pointer moving right answers them in amortised $O(1)$; otherwise binary search on the envelope answers each in $O(\log n)$. The template compares the intersection points as exact fractions, so it never overflows or loses precision.

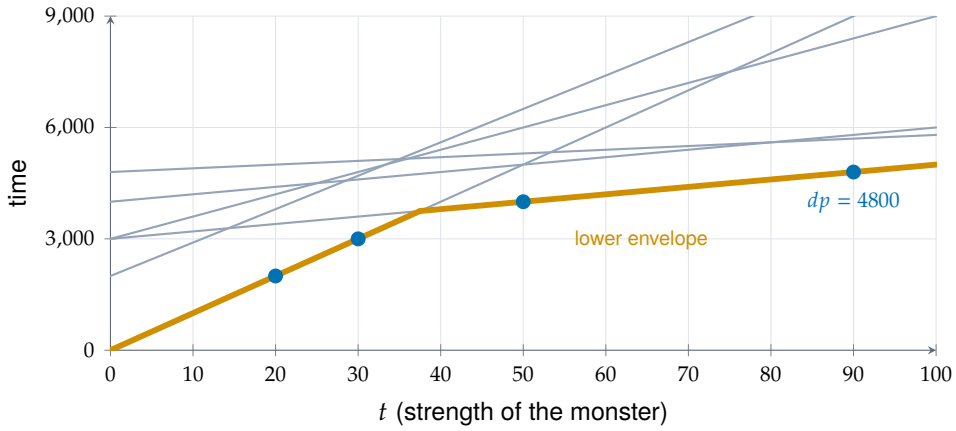


Figure 20.4: Monster Game I on the sample with $x = 100$: one line per earlier kill, $y = f_j t + dp_j$. Each query takes the lowest line at $t = s_i$ (blue points, using only the lines that existed at that time). Of the six lines, only $100t$ and $20t + 3000$ end up on the final lower envelope (orange).

Example: Monster Game I (CSES)

A game has $n \le 2 \cdot 10^5$ levels. On each level you may kill or escape the monster, except on the last level, where it must be killed. Killing a monster of strength s takes $s \cdot f$ time, where f is your current skill factor (initially x), and killing monster i sets f to f_i. The strengths never decrease and the factors f_i never increase. Minimise the total time.

Solution. $dp_i = \min_j(dp_j + s_i f_j)$ over the previous kill j (or the start, with $f = x$ and time 0): line j has slope f_j and intercept dp_j, and is queried at s_i. Slopes arrive non-increasing and queries non-decreasing, so the monotone hull with its pointer runs in $O(n)$; 0.08 s.

```
void solve() {  // MonoCHT (cpmonocht) is pasted above solve()
    int n;
    ll x;
    cin >> n >> x;
    vi s(n), f(n);
    for (ll& v : s) cin >> v;
    for (ll& v : f) cin >> v;
    // dp[i]: the least time to kill monster i = min over the previous kill j
    // of dp[j] + s[i] * f[j]; the start is a kill with skill x at time 0.
    // Each kill j is the line y = f[j] * t + dp[j], queried at t = s[i].
    MonoCHT h;
    h.add(x, 0);
    ll dp = 0;
    rep(i, n) {
        dp = h.query_inc(s[i]);  // s is non-decreasing
        h.add(f[i], dp);  // the slopes f are non-increasing
    }
    print(dp);  // the last monster must be killed
}
```

</> Template: Monotone convex hull trick Ctrl+Alt+Y, Z prefix `cpmonocht`

Any order: LineContainer and Li Chao trees. Without the monotonicity, the envelope must accept lines anywhere. A *LineContainer* keeps the lines of the envelope in a balanced tree ordered

by slope, each storing the x where the next line takes over; inserting a line removes its useless neighbours, and a query is a binary search over those breakpoints, both $O(\log n)$. A *Li Chao tree* is a segment tree over the x-domain in which every node keeps the line that is best at its midpoint; inserting a line walks down one root path, swapping lines where the newcomer wins at the midpoint, and a query takes the minimum along the path to x. It is $O(\log C)$ per operation for a domain of size C, simpler to get right than the LineContainer, and it also accepts line *segments* (in $O(\log^2 C)$).

Example: Monster Game II (CSES)

The same game without the monotonicity of the strengths and the factors.

Solution. The same lines, inserted into a LineContainer; it keeps an upper envelope, so every line is stored negated: 0.09 s.

```
void solve() {  // LineContainer (cplinecontainer) is pasted above solve()
    int n;
    ll x;
    cin >> n >> x;
    vi s(n), f(n);
    for (ll& v : s) cin >> v;
    for (ll& v : f) cin >> v;
    // the lines of Monster Game I, now in any order; the container keeps an
    // upper envelope, so each line is stored negated to get a minimum
    LineContainer h;
    h.add(-x, 0);
    ll dp = 0;
    rep(i, n) {
        dp = -h.query(s[i]);
        h.add(-f[i], -dp);
    }
    print(dp);
}
```

Template: LineContainer / Li Chao tree Ctrl+Alt+Y, L / Shift+L prefix `cplinecontainer, cplichao`

How to recognize it After expanding the cost, the transition reads $dp_i = \min_j(A_j \cdot B_i + C_j) + D_i$: one product of a term of j and a term of i, plus terms of only one of them. Squares of prefix sums, "rate times amount" and "distance times weight" are the usual sources. Which variant to use: monotone slopes and queries, the deque with a pointer; monotone slopes only, the deque with binary search; neither, the LineContainer, or a Li Chao tree when the queries are integers in a known range or the lines are segments.

20.6 Slope trick

IOI / ADVANCED

Some DPs keep a whole function of a value: $f_i(v)$, the least cost of the first i elements if the i-th one ends at value v. When every operation keeps f_i *convex and piecewise linear with integer slopes*, store it by its breakpoints: the multiset of points where the slope increases by 1, plus the minimum value. "Add $|v - a|$" inserts the breakpoint a twice (the slope rises by 2 there), "take the prefix minimum"

($\min_{u \le v} f(u)$) deletes every breakpoint to the right of the minimum, and shifts are lazy offsets. Each becomes a few heap operations (Figure 20.5).

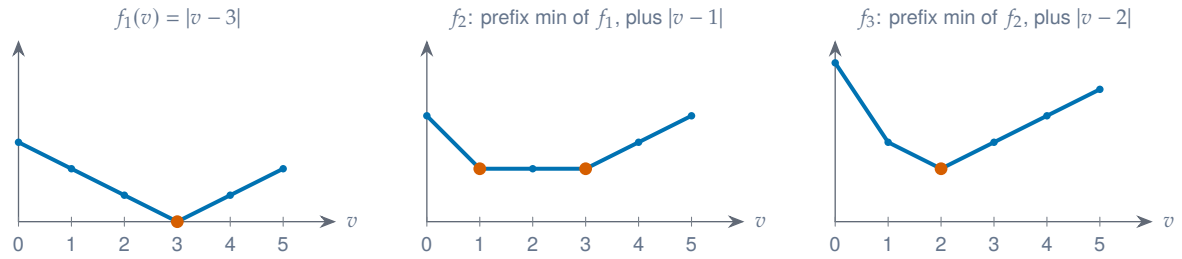


Figure 20.5: Slope trick on $a = 3, 1, 2$ (make it non-decreasing at the least total change). Each panel is the cost as a function of the value v of the last element; red dots mark the breakpoints that matter. The answer is the minimum of f_3, which is 2 (for example $1, 1, 2$).

Example: Increasing Array II (CSES)

Change elements of an array ($n \le 2 \cdot 10^5$) by ± 1 per move until it is non-decreasing. Minimise the number of moves.

Solution. Only the decreasing part of f matters for the prefix minimum, so keep a max-heap of its breakpoints. For each a, push a (adding $|v - a|$); if the largest breakpoint is now above a, the cheapest final value for this prefix lies above a, so pay the difference, pop it and push a once more. $O(n \log n)$, 0.07 s. For a *strictly* increasing array, apply the same to $a_i - i$.

```
void solve() {
    int n;
    cin >> n;
    // f(v): the least cost so far if the last element ends at value v. f is
    // convex and piecewise linear; the max-heap holds the points where its
    // slope rises by 1, so its top is where f stops decreasing
    priority_queue<ll> pq;
    ll cost = 0;
    rep(i, n) {
        ll a;
        cin >> a;
        pq.push(a);  // add |v - a|
        if (pq.top() > a) {  // the cheapest end value lies above a: lower it
            cost += pq.top() - a;
            pq.pop();
            pq.push(a);
        }
    }
    print(cost);
}
```

</> Template: Slope trick Ctrl+Alt+Y, Shift+S prefix `cpslope`

How to recognize it A DP over a sequence whose state is a *value* with a huge range, and whose cost is a sum of absolute values or other convex functions: make an array monotone at minimum total change, move tokens so that no two share a position, keep consecutive values

within a distance. If a brute-force DP over small values shows a convex, piecewise-linear row with integer slopes, the slope trick applies.

20.7 Aliens trick: Lagrangian relaxation

IOI / ADVANCED

"Choose *exactly* k things to maximise the total" often has a natural DP with an extra dimension for the number chosen, which multiplies the cost by k. If the best total $g(k)$ is *concave* in k (each extra item gains no more than the previous one), the dimension can be removed. Charge a penalty λ for every chosen thing and solve the unconstrained problem "maximise $g(c) - \lambda c$ over all counts c", which is the original DP without the count. The optimal c is where a line of slope λ touches the graph of g from above (Figure 20.6), so it decreases as λ grows; binary search λ until the optimal count is k, and the answer is (the penalised optimum) $+ \lambda k$. The cost is the DP's cost times the number of search steps, about log of the slope range.

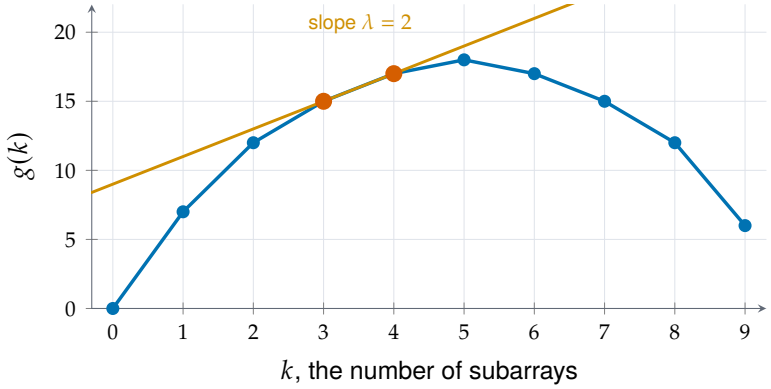


Figure 20.6: The best total $g(k)$ of exactly k disjoint subarrays of $5, -2, 4, -6, 3, -1, 2, -3, 4$ is concave. With a penalty of 2 per subarray the penalised optimum is $g(k) - 2k = 9$, attained at $k = 3$ *and* $k = 4$ (red): a tie, because the slope from 3 to 4 is exactly 2.

Example: Exactly k subarrays

Given $n \le 2 \cdot 10^5$ integers in $[-10^9, 10^9]$ and $k \le n$, choose exactly k non-empty, pairwise disjoint subarrays with the largest total sum.

Solution. The total is concave in k (it is a min-cost flow in disguise). For a penalty λ, a two-state DP (the current element is or is not the end of a chosen subarray) finds the best penalised score in $O(n)$, breaking ties towards *fewer* subarrays. Binary search the smallest λ whose optimum uses at most k subarrays; at that λ, k is among the optimal counts, and the answer is the penalised score plus λk. About 50 steps of $O(n)$: 0.09 s.

```
void solve() {
    int n, k;
    cin >> n >> k;
    vi a(n);
    for (ll& x : a) cin >> x;
    // the best (score, -count) when any number of subarrays may be taken and
    // each one costs lam; among equal scores, fewer subarrays win
    auto run = [&](ll lam) {
        // out: the last element is in no subarray; in: it ends a subarray
```

```
        pll out{0, 0}, in{-INF, 0};
        for (ll x : a) {
            pll done = max(out, in);  // the best with everything so far closed
            pll ext{in.first + x, in.second};  // extend the open subarray
            pll start{done.first + x - lam, done.second - 1};  // or open one
            in = max(ext, start);
            out = done;
        }
        return max(out, in);
    };
    // the fewest-subarray optimum shrinks as lam grows: find the smallest lam
    // where it uses at most k subarrays, then add the k penalties back. Every
    // slope of the answer in k lies in [-1e9, 2e14], so this range is enough
    // and keeps the scores inside run far from overflow
    ll lo = -3e9, hi = 3e14;
    while (lo < hi) {
        ll mid = lo + (hi - lo) / 2;
        if (-run(mid).second <= k) hi = mid;
        else lo = mid + 1;
    }
    print(run(lo).first + lo * k);
}
```

⚠ Ties, and the range of the penalty Several counts can be optimal for the same λ, as in Figure 20.6, so the DP may never report exactly k. Break ties consistently (here: fewest subarrays), search for the smallest λ with count $\leq k$, and add back λk with the *requested* k, not the count the DP returned. Second, bound λ by the possible slopes of g, not by something huge: with $\lambda = -3 \cdot 10^{14}$ the DP above would reward $2 \cdot 10^5$ subarrays with $6 \cdot 10^{19}$, past the range of 64-bit integers. The first version of this solution made exactly that mistake; small random tests passed, and only a full-size test showed the overflow.

20.8 Convex $(\min, +)$ convolution

IOI / ADVANCED

The $(\min, +)$ convolution $c_k = \min_{i+j=k}(a_i + b_j)$ appears whenever two independent parts share a budget: a_i is the best cost of spending i units on the first part, b_j on the second. In general it costs $O(nm)$, but two special cases are fast.

- **Both convex**: the result is convex, and its sequence of slopes is the two slope sequences merged in sorted order, like merging two sorted lists (the Minkowski sum of two convex polygonal lines, Figure 20.7): $O(n + m)$. The reason is that, with convex costs, the cheapest next unit is always the cheaper of the two parts' next units.
- **One convex, one arbitrary**: the matrix $M[k][j] = b_j + a_{k-j}$ is totally monotone when a is convex, so its row minima come from monotone minima in $O((n + m) \log)$, or from SMAWK in $O(n + m)$.

</> Template: $(\min, +)$ convolution (convex) Ctrl+Alt+U, Shift+C prefix `cpminplus`

A typical use is a knapsack with *small weights* but many items: group the items by weight w; within a group, taking the c most valuable items is best, and the values of the first c items form a concave sequence. Merging a group into the table is then, for every remainder modulo w, a

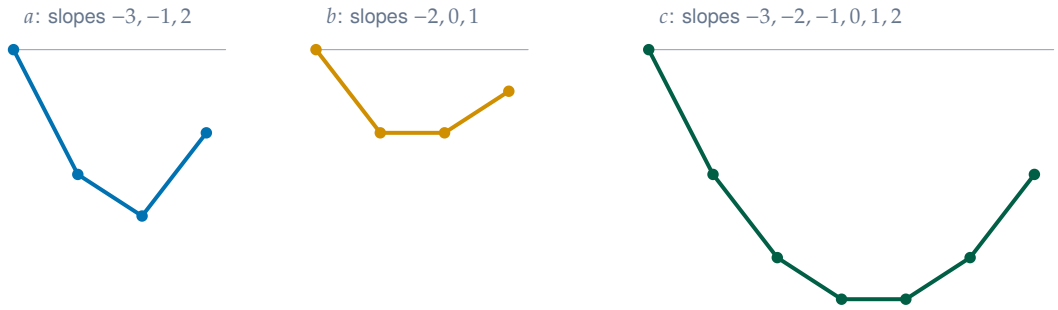


Figure 20.7: Merging the slopes of two convex sequences gives their $(\min, +)$ convolution. Here the result is $c = (0, -3, -5, -6, -6, -5, -3)$, exactly what the brute force $\min_{i+j=k}(a_i + b_j)$ computes.

$(\max, +)$ convolution of the table (arbitrary) with that concave sequence: monotone minima per remainder class instead of a loop over all counts.

20.9 Bitsets and swapped dimensions

GOLD

Bitsets. A boolean DP whose transition shifts a whole row, like subset sum ($can[s]$ |= $can[s - a_i]$), runs 64 times faster on a `bitset`: `b |= b << a` (Section 10.4 measures a 14× speed-up on Money Sums). When the items sum to S, there are at most $O(\sqrt{S})$ distinct values among them, since $1+2+\cdots+t \le S$ forces $t \le \sqrt{2S}$. Group equal values and split each group into bundles of $1, 2, 4, \ldots$ copies (Section 18.5): the bitset subset sum then costs $O(S\sqrt{S}\log/64)$ in the worst case, and much less in practice, the standard solution to "can these be split into two equal halves" with S up to 10^6.

</> Template: Subset sums with bitset Ctrl+Alt+T, Shift+1 prefix `cpbitsetsum`

Swapping the value and a dimension. When a state would include a huge quantity (a weight up to 10^9) but another quantity is small (at most 100 items, values summing to 10^5), make the small quantity the index and the huge one the value that is minimised: $dp[value]$ is the least weight that reaches it. The LIS tail array (Section 18.6) is the same idea: instead of "the longest subsequence ending at each value", "the smallest last value for each length". Asking "which quantity is small enough to be an index?" is often the whole optimization.

Chapter summary

- Range transitions become $O(1)$ with prefix sums or a monotone queue; along each remainder class, a queue gives the exact $O(nW)$ bounded knapsack.
- When the optimal split point is monotone (quadrangle inequality), divide and conquer computes a layer in $O(n \log n)$, SMAWK in $O(n)$, and Knuth's sandwich $opt(l, r-1) \le opt(l, r) \le opt(l+1, r)$ makes an interval DP $O(n^2)$.
- Products of a j-term and an i-term make each earlier state a line: a deque for monotone slopes, a LineContainer or a Li Chao tree otherwise.
- Convex piecewise-linear value functions are stored by their breakpoints in heaps (slope trick).
- "Exactly k" with a concave answer becomes a penalty λ found by binary search; break ties consistently and bound λ by the slopes.
- Convex sequences convolve by merging slopes; bitsets divide boolean DPs by 64.

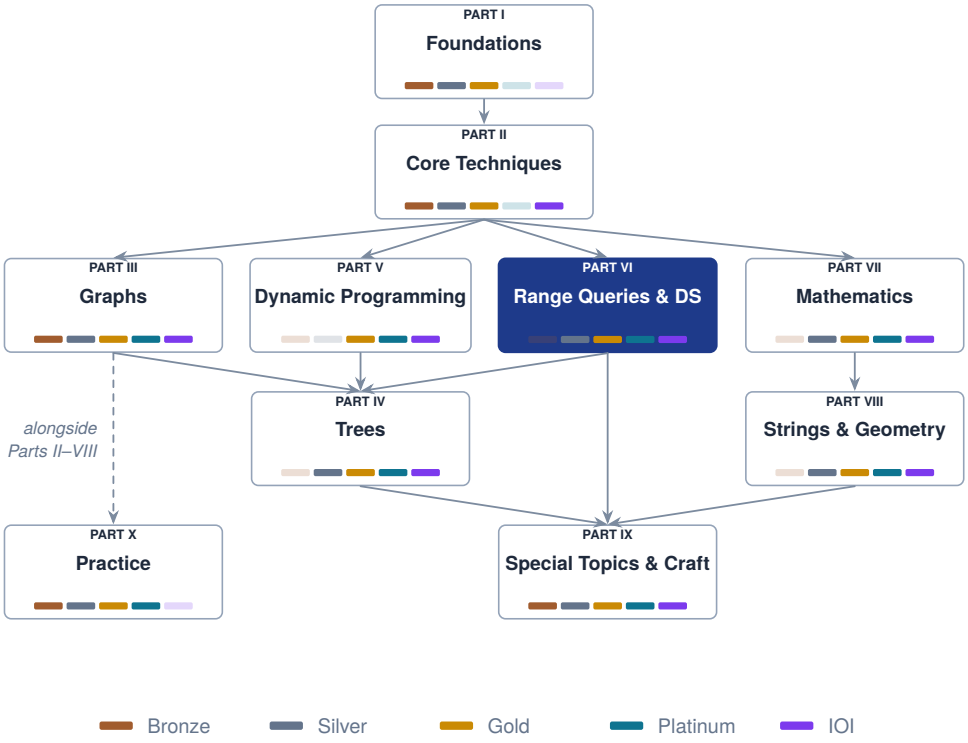
Practice problems (CSES Problem Set)

- Subarray Squares
- Houses and Schools
- Knuth Division
- Monster Game I
- Monster Game II
- Increasing Array II
- Money Sums
- Book Shop
- Elevator Rides

PART VI

Range Queries and Data Structures

Answering questions about ranges of an array, with and without updates: sparse tables, Fenwick and segment trees in all their forms, square-root techniques, treaps and offline methods.



CHAPTER 21

Static Range Queries

A **range query** asks for an aggregate (sum, minimum, gcd, ...) over a contiguous part $[l, r)$ of an array. When the array never changes, preprocessing buys very fast queries. This chapter covers the static case; Chapter 22 adds updates.

Structure	Preprocessing	Query	Works for
prefix sums	$O(n)$	$O(1)$	invertible operations: $+, \oplus, \times$ mod p
sparse table	$O(n \log n)$	$O(1)$	idempotent: min, max, gcd, AND, OR
sparse table (disjoint)	$O(n \log n)$	$O(1)$	any associative operation
sqrt decomposition	$O(n)$	$O(\sqrt{n})$	anything, also with updates
Mo's algorithm (offline)	–	$O(\sqrt{n})$ amortized	"add/remove one element" statistics

21.1 Sparse table

GOLD

Precompute $\text{st}[k][i] = \min(a_i, \ldots, a_{i+2^k-1})$ for every power of two: $\text{st}[k][i] = \min(\text{st}[k-1][i], \text{st}[k-1][i+2^{k-1}])$. A query $[l, r)$ takes the largest k with $2^k \le r - l$ and combines two overlapping blocks that together cover the range.

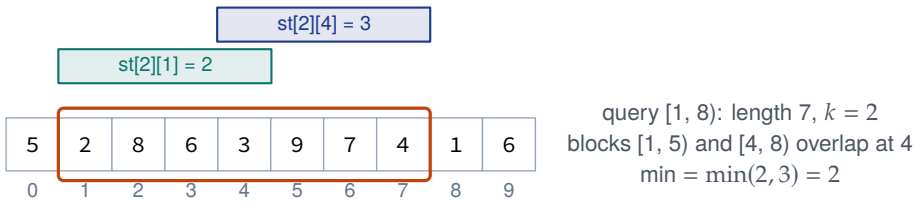


Figure 21.1: Any range is covered by two blocks of the same power-of-two length. Overlap is harmless for min.

</> Template: Sparse table Ctrl+Alt+T, R prefix `cpsparse`

> **Only idempotent operations** Overlapping blocks count some elements twice. For min, max, gcd, AND, OR that changes nothing; for sums it is wrong. Use prefix sums for sums, or the "disjoint sparse table" variant for general associative operations.

21.2 Divide and conquer for static range queries

PLATINUM

For an associative operation that is *not* idempotent and not invertible (products of matrices, "max subarray sum" nodes, knapsack-like combinations), answer queries offline: split the array at the

middle m; for every query crossing m, combine a precomputed suffix aggregate of $[l, m)$ with a prefix aggregate of $[m, r)$; recurse on queries entirely inside a half. Each query is answered at one level, each level costs $O(n)$ aggregates: $O((n + q) \log n)$ combinations.

Knapsack range queries "Can a subset of $a_l, \ldots, a_{r-1}$ sum to s?" for many queries: with this divide and conquer, precompute bitset knapsacks of all suffixes of the left half and prefixes of the right half at each level; a query combines two bitsets. The same skeleton answers "best DP over a range" whenever DP states of two halves can be merged.

21.3 Square root decomposition

PLATINUM

Split the array into blocks of size $B \approx \sqrt{n}$ and keep an aggregate per block. A query uses the whole blocks it covers ($\leq n/B$) and the individual elements at its ends ($\leq 2B$): $O(\sqrt{n})$ per operation. Updates change one element and its block aggregate. It is slower than a segment tree but handles operations segment trees find awkward ("count elements $\leq x$ in a range with point updates": keep each block sorted).

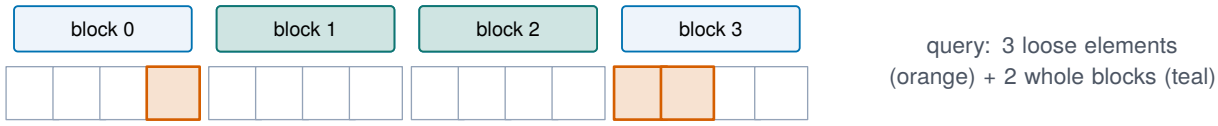


Figure 21.2: A query touches at most two partial blocks and $O(\sqrt{n})$ whole blocks.

Heavy / light thinking Square-root ideas go far beyond arrays: in a graph, vertices of degree $> \sqrt{m}$ are "heavy" and there are at most $\sqrt{m}$ of them, so "for every heavy vertex precompute, for every light vertex iterate over neighbours" balances the work. Triangle counting, "sum over neighbours" queries and many string problems (few distinct long lengths) use it.

21.4 Mo's algorithm

PLATINUM

Offline queries on ranges where you can *add* or *remove* one element at the end of the current window cheaply (number of distinct values, most frequent value, number of pairs of equal values). Sort queries by (block of l, r) and move a window between consecutive queries. r moves $O(n)$ per block of l, l moves $O(\sqrt{n})$ per query: in total $O((n + q)\sqrt{n})$ add/remove calls.

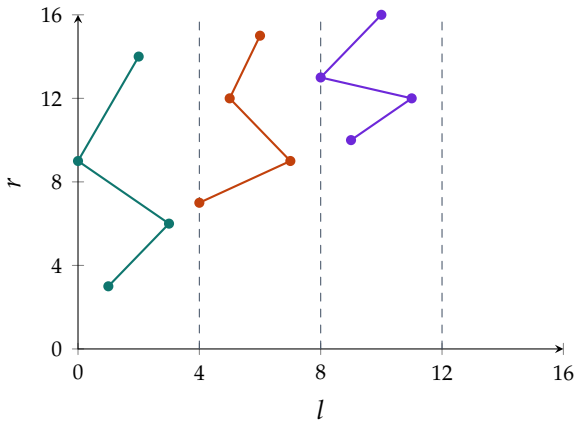


Figure 21.3: Queries grouped by the block of l; inside a block r is monotone (alternating direction, the zig-zag trick).

</> Template: Mo's algorithm Ctrl+Alt+Y, O prefix `cpmo`

Mo's variants

- **With updates**: add time as a third coordinate; blocks of size $n^{2/3}$ give $O(n^{5/3})$.
- **On trees**: flatten with an Euler tour where each vertex appears twice; a path becomes a range (with the LCA handled separately), toggling a vertex on its second appearance.
- **Hilbert order**: sorting queries along a Hilbert curve instead of by block reduces the constant noticeably.
- **Rollback Mo**: when removing is hard but adding is easy (maximum), keep l at the block end and roll back.

How to recognize it Many range queries, no updates (or few), a statistic that is easy to maintain under adding/removing a single element but not decomposable (distinct counts, mode, "number of pairs with equal values"), and $n, q \le 2 \cdot 10^5$. If queries are online, Mo's does not apply.

CHAPTER 22

Fenwick Trees and Segment Trees

When the array changes between queries, prefix sums break: one update would invalidate $O(n)$ prefix values. Fenwick trees and segment trees store *partial* aggregates so that both updates and queries touch only $O(\log n)$ of them.

22.1 Fenwick tree (binary indexed tree)

GOLD

A Fenwick tree stores, at 1-based position i, the sum of the lowbit$(i) = i \,\&\, -i$ elements ending at i. A prefix sum walks down $i \leftarrow i - \text{lowbit}(i)$ (removing the last set bit); an update walks up $i \leftarrow i + \text{lowbit}(i)$. Both take $O(\log n)$ steps.

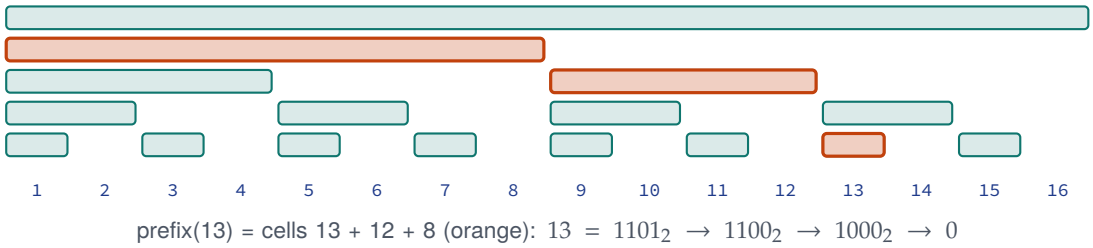


Figure 22.1: Each Fenwick cell covers a block ending at its index, of length equal to its lowest set bit.

</> Template: Fenwick tree Ctrl+Alt+T, I prefix `cpbit`

Fenwick tricks

- **Range update, point query**: store differences; add v at l and $-v$ at r; a point value is a prefix sum.
- **Range update, range query**: two Fenwick trees (`cpbitrange`, Ctrl+Alt+Y, I).
- **k-th element**: with counts, descend from the highest power of two (the template's `lower_bound`) in $O(\log n)$ instead of binary search plus prefix sums in $O(\log^2 n)$. An order-statistics set over small values.
- **Inversions**: scan left to right; for each a_i, count earlier elements greater than it with a Fenwick tree over (compressed) values. $O(n \log n)$.
- **2D**: nest the loops (`cpbit2d`); memory $O(nm)$, so only for moderate grids.

22.2 Segment tree

GOLD

A segment tree is a binary tree over the array: each node stores the aggregate of a range, the root the whole array, the leaves single elements. Any range is the disjoint union of $O(\log n)$ nodes; an update changes the $O(\log n)$ nodes on one leaf-to-root path.

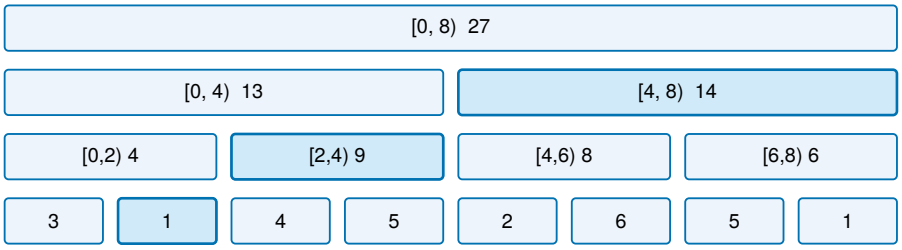


Figure 22.2: A segment tree of sums. A query uses at most two nodes per level.

The library has two implementations:

- **Bottom-up** (`cpseg`): an array of $2n$ nodes, loops instead of recursion; the fastest choice for point updates and range queries. Change `T`, `ID` and `op` for min, max, gcd, ...
- **Custom node** (`cpsegnode`): any associative node with a `+` operator (the example stores the maximum subarray sum), plus `max_right`, which walks down the tree to answer "first position where the prefix aggregate fails a condition".

</> Template: Segment tree / custom-node segment tree Ctrl+Alt+T, S / Ctrl+Alt+Y, N prefix `cpseg`, `cpsegnode`

Designing a node A node must summarise its range well enough that two neighbouring summaries can be merged. For "maximum subarray sum" that needs four numbers: total, best prefix, best suffix, best inside. For "number of distinct adjacent pairs" it needs the first and last element. Ask: *what does the merge of two halves need to know about each half?*

22.3 Lazy propagation

PLATINUM

Range *updates* (add v to every element of $[l, r)$) would touch every leaf. Instead, apply the update to the $O(\log n)$ covering nodes and leave a *tag* there: "my children still owe this update". When a later operation needs to go below a tagged node, it first *pushes* the tag to the children.

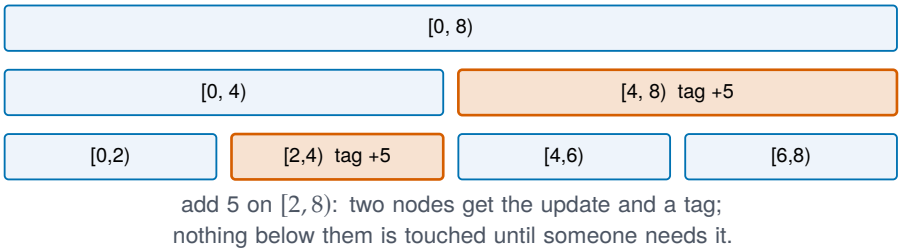


Figure 22.3: Lazy tags record pending updates.

</> Template: Lazy segment tree (add; sum, min) / (assign + add) Ctrl+Alt+T, Shift+S / Ctrl+Alt+Y, A prefix `cplazy`, `cplazyassign`

What can be lazy You need (1) a tag that can be applied to a whole node's aggregate without looking at its children (add v to a sum: sum += $v \cdot$ length), and (2) a way to compose two tags into one ("assign x then add y" = "assign $x + y$"). If both exist, the update is lazy-able.

> Affine maps $x \mapsto ax + b$ compose, so "multiply a range and add to a range" works; "set each element to its square root" does not (but see segment tree beats).

22.4 Counting minimums

PLATINUM

A node that stores the minimum *and how many elements attain it* (or their total length) supports range add and answers "how many zeros are there?" whenever values never drop below zero. The classic application: the area of a union of rectangles by sweeping a vertical line, adding +1 over a rectangle's y-interval when its left edge is crossed and -1 at its right edge; the covered length is the total length minus the length whose count is 0.

</> Template: Min + count segment tree Ctrl+Alt+Y, Shift+C prefix `cpmincount`

22.5 Sparse (dynamic) segment trees

PLATINUM

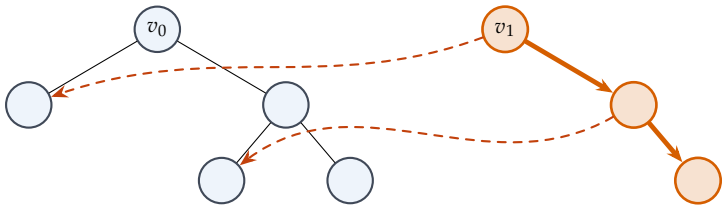
When positions go up to 10^9 or 10^{18} and cannot be compressed offline (they arrive online), create nodes only when an update reaches them. Each update creates $O(\log(\text{range}))$ nodes; unvisited parts of the range simply do not exist and count as the identity.

</> Template: Dynamic segment tree Ctrl+Alt+Y, D prefix `cpdynseg`

22.6 Persistent segment trees

PLATINUM

A persistent structure keeps every old version. Because an update changes only one root-to-leaf path, copy just that path and point the new nodes at the old, unchanged subtrees: $O(\log n)$ new nodes per update, every version still queryable.



Updating a leaf in version v_0 creates version v_1: only the three orange nodes are new; dashed edges reuse the old subtrees.

Figure 22.4: Path copying.

Example: k-th smallest in a range

Answer q queries "the k-th smallest value in $a[l, r)$".

Solution. Build a persistent segment tree over (compressed) values; version i contains the counts of $a_0, \ldots, a_{i-1}$. The difference of versions r and l holds the counts of $a[l, r)$; walk down both trees together: if the left child's count difference is at least k, go left, else subtract and go right. $O(\log n)$ per query.

</> Template: Persistent segment tree Ctrl+Alt+Y, P prefix `cppersist`

22.7 Segment tree beats

IOI / ADVANCED

Operations like "$a_i \leftarrow \min(a_i, x)$ on a range" are not lazy-able in general, because the effect on a sum depends on how many elements exceed x. Segment tree beats stores in each node the maximum, the strict second maximum and the count of the maximum. If $x \geq$ max, nothing happens; if second max $< x <$ max, only the maximum elements change and the node is updated in $O(1)$; otherwise recurse. A potential argument shows the total work is $O((n+q)\log^2 n)$ (Ji Ruyi's analysis).

</> Template: Segment tree beats Ctrl+Alt+Y, S prefix `cpbeats`

22.8 Range queries with sweep lines

PLATINUM

Many offline range-query problems become easy when queries are sorted by their right end and the array is "swept". The canonical example: the number of distinct values in $a[l, r)$. Sweep r from left to right; keep a Fenwick tree with a 1 at the *last* occurrence of each value seen so far (moving the 1 when a value reappears). Then the answer for $[l, r)$ is the number of 1s in positions $[l, r)$.

</> Template: Distinct values in ranges Ctrl+Alt+Y, Shift+D prefix `cpdistinct`

> **How to recognize it** Offline queries whose answer depends on "the last/first occurrence", "the nearest greater element", or on pairs (i, j) with $i < j$: sort the queries by r, and when the sweep reaches j, activate what j creates. This transforms 2D conditions ($l \leq i$ and $j < r$) into one dimension handled by the sweep and one by a Fenwick tree.

22.9 2D range queries

PLATINUM

Counting points in axis-aligned rectangles, offline: sweep x, keep a Fenwick tree over (compressed) y, and split each rectangle query into two prefix queries at x_1 and x_2. $O((n+q)\log n)$ time and $O(n)$ memory, far better than a 2D Fenwick tree when coordinates are large. For online queries: a Fenwick tree of sorted vectors (static points), a wavelet tree (Chapter 23), or a 2D segment tree / merge sort tree.

22.10 Max suffix query with insertions only

IOI / ADVANCED

If you only insert pairs (k, v) and ask "the maximum v among keys $\geq x$", keep a `map` in which values strictly decrease as keys increase: a new pair that is dominated (some larger key already has a larger value) is not inserted; otherwise it removes the pairs it dominates. The answer to a query is the first entry with key $\geq x$. Each pair is inserted and removed once: $O(\log n)$ amortized. This "staircase" structure is how 2D dominance DPs ("the longest chain of pairs increasing in both coordinates") stay simple.

CHAPTER 23

Ordered Sets, Treaps and Other Structures

23.1 More operations on sorted sets

GOLD

Beyond insert and find, a `set`/`multiset`/`map` answers ordered questions in $O(\log n)$:

Expression	Answer
`*s.lower_bound(x)`	smallest element $\geq x$ (check `!= s.end()` first)
`*prev(s.lower_bound(x))`	largest element $< x$ (check `!= s.begin()` first)
`*prev(s.upper_bound(x))`	largest element $\leq x$
`*s.begin(), *s.rbegin()`	minimum, maximum
`s.erase(s.begin())`	remove the minimum
`ms.erase(ms.find(x))`	remove one copy from a multiset
`m.lower_bound(k)->second`	value of the first key $\geq k$

Example: Sliding window median

Report the median of every window of length k.

Solution. Two multisets: `low` holds the smaller half, `high` the larger half, balanced so that $|\text{low}| = \lceil k/2 \rceil$. The median is `*low.rbegin()`. Adding and removing an element and rebalancing is $O(\log k)$.

23.2 Sorted sets with custom comparators

GOLD

A set orders its elements by a comparator, which can be any strict weak ordering. Two elements for which neither is "less" than the other are considered *equal* and only one is kept, even if they differ in fields the comparator ignores.

```
struct Seg { int l, r, id; };
struct ByRight {
    bool operator()(const Seg& a, const Seg& b) const {
        return make_pair(a.r, a.id) < make_pair(b.r, b.id);   // id breaks ties: no
    accidental merges
    }
};
set<Seg, ByRight> active;
```

⚠ Comparators that change Never modify a field that the comparator uses while the element is in the set, and never use a comparator whose result depends on external state that changes (a global "current x"), unless you can prove the relative order of the stored elements never changes. Sweep-line algorithms that sort segments by their y at the current x rely on exactly that proof.

23.3 Order statistics

GOLD

"What is the k-th smallest?" and "how many are smaller than x?" in a changing set: use the policy-based `ordered_set` (Chapter 4), or a Fenwick tree over compressed values when all values are known in advance (faster and simpler to reason about).

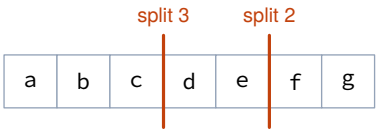
</> Template: Ordered set (PBDS) Ctrl+Alt+T, Shift+O prefix `cposet`

23.4 Treaps

PLATINUM

A **treap** is a binary search tree in which every node also has a random priority, and the tree is a heap by priority. Random priorities keep the expected depth $O(\log n)$. Everything is built from two operations:

- **split**(t, k): cut t into the first k elements and the rest;
- **merge**(a, b): join two treaps where every element of a comes before every element of b.



An *implicit* treap is keyed by position. To reverse $[3, 5)$: split off the first 3, split the next 2, flip a "reversed" flag on the middle piece, merge the three back. Every operation on a range works this way.

Figure 23.1: Range operations on an implicit treap: split, operate on the middle, merge.

With an *implicit key* (position instead of value), a treap is a sequence that supports, in expected $O(\log n)$: insert and delete at any position, reverse a range, cut and paste a block, and range aggregates with lazy tags.

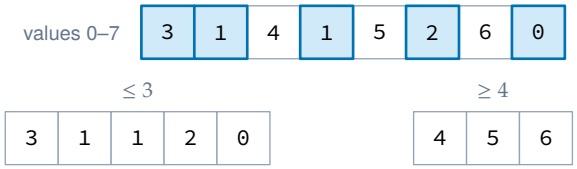
</> Template: Implicit treap Ctrl+Alt+Y, T prefix `cptreap`

🔍 How to recognize it The array itself changes shape: "move the segment $[l, r)$ to the front", "reverse a range", "insert an element at position p", or rope-like string editing. If the array only changes values, a segment tree is simpler and faster.

23.5 Wavelet trees

IOI / ADVANCED

A wavelet tree answers "k-th smallest in $a[l, r)$" and "how many elements of $a[l, r)$ are $\le x$" in $O(\log V)$, online, without persistence. The root splits values into a lower and an upper half and records, for every prefix of the array, how many elements went to the lower half; each child does the same for its elements.



Teal elements go left. The node stores the running count of "went left", so any range $[l, r)$ maps to a range in each child in $O(1)$.

Figure 23.2: One level of a wavelet tree; stable partitioning preserves the order within each half.

</> Template: Wavelet tree Ctrl+Alt+Y, W prefix `cpwavelet`

23.6 Hash maps

SILVER

`unordered_map` gives $O(1)$ expected operations but can be attacked (Chapter 5): use the randomized `custom_hash`. For heavy use, `__gnu_pbds::gp_hash_table` with the same hash is often 2–3 times faster. When keys are small integers, a plain array beats any hash map; when all keys are known in advance, sorting and binary search (or compression) is predictable and hack-proof.

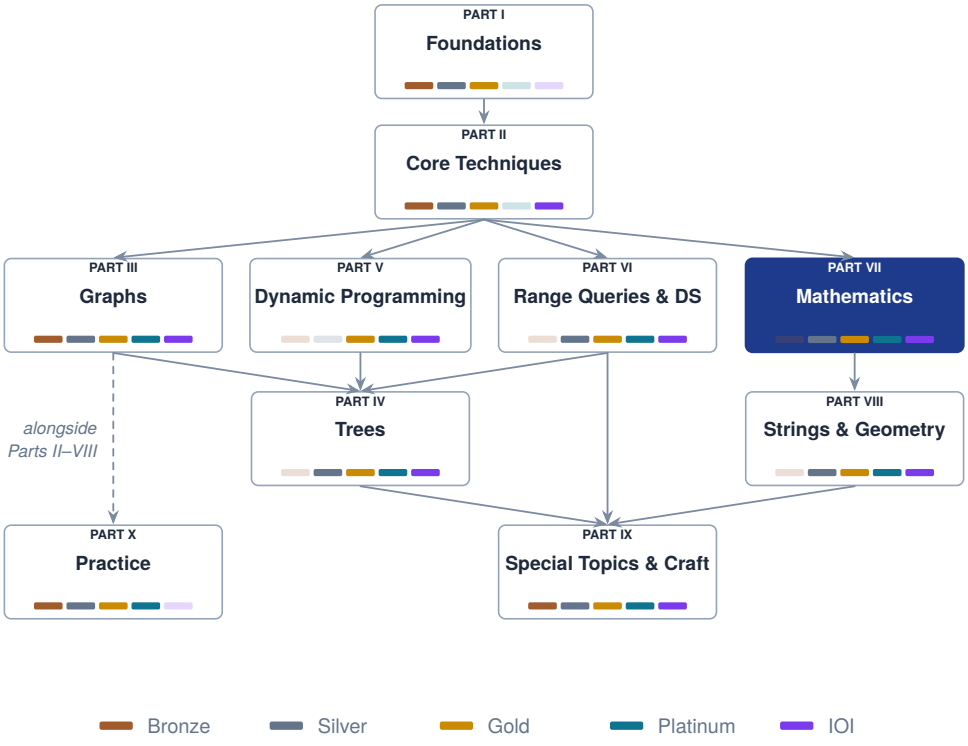
</> Template: Anti-hack hash Ctrl+Alt+T, Shift+H prefix `cphash`

Hashing sets and multisets To compare sets quickly, assign each element a random 64-bit number and represent a set by the XOR (or sum, for multisets) of its elements' numbers. Adding or removing an element is $O(1)$, and "are these two subarrays permutations of each other?" becomes a prefix-sum comparison. The probability of a false match is about 2^{-64} per comparison.

PART VII

Mathematics

Number theory, modular arithmetic, combinatorics, linear algebra and polynomials: the mathematics that appears in contests, with the algorithms that compute it.



CHAPTER 24

Number Theory

24.1 Divisibility

SILVER

d divides n ($d \mid n$) if $n = kd$ for an integer k. Divisors come in pairs $(d, n/d)$ with one of them at most $\sqrt{n}$, so all divisors are found by trying $d = 1, \ldots, \lfloor\sqrt{n}\rfloor$ in $O(\sqrt{n})$.

Divisor counts If $n = p_1^{e_1} \cdots p_k^{e_k}$, the number of divisors is $(e_1 + 1) \cdots (e_k + 1)$ and their sum is $\prod_i \frac{p_i^{e_i+1}-1}{p_i-1}$. Divisor counts are small: at most 240 for $n \le 10^6$, 1 344 for $n \le 10^9$, 103 680 for $n \le 10^{18}$. So "loop over all divisors of every value" is often affordable.

Template: Factorize + divisors (trial division) Ctrl+Alt+T, Shift+P prefix `cpfactor`

24.1.1 GCD and LCM

Euclid's algorithm: $\gcd(a, b) = \gcd(b, a \bmod b)$, $\gcd(a, 0) = a$. It takes $O(\log \min(a, b))$ steps (the numbers at least halve every two steps). $\operatorname{lcm}(a, b) = a/\gcd(a, b) \cdot b$ (divide first to avoid overflow). C++17 has `gcd` and `lcm`.

GCD identities $\gcd(a, b) = \gcd(a - b, b)$, so the gcd of an array does not change when you subtract one element from another; the gcd of all differences $|a_i - a_j|$ equals $\gcd(a_2 - a_1, a_3 - a_1, \ldots)$. And $\gcd(F_m, F_n) = F_{\gcd(m,n)}$ for Fibonacci numbers. Prefix gcds of an array take at most log(max) distinct values, which makes "all subarray gcds" tractable.

24.2 Primes and sieves

SILVER

A prime has exactly two divisors. To list all primes up to N, the sieve of Eratosthenes crosses out the multiples of each prime, starting from its square:

	2	**3**	~~4~~	**5**	~~6~~	**7**	~~8~~	~~9~~	~~10~~
11	~~12~~	**13**	~~14~~	~~15~~	~~16~~	**17**	~~18~~	**19**	~~20~~
~~21~~	~~22~~	**23**	~~24~~	~~25~~	~~26~~	~~27~~	~~28~~	**29**	~~30~~
31	~~32~~	~~33~~	~~34~~	~~35~~	~~36~~	**37**	~~38~~	~~39~~	~~40~~
41	~~42~~	**43**	~~44~~	~~45~~	~~46~~	**47**	~~48~~	~~49~~	

Crossed: a multiple of a smaller prime. Teal: primes. The sieve runs in $O(N \log \log N)$; a linear sieve records the smallest prime factor of every number in $O(N)$.

Figure 24.1: The sieve of Eratosthenes up to 49.

Template: Sieve (primes + smallest prime factor) Ctrl+Alt+T, P prefix `cpsieve`

Factorising many numbers With the smallest prime factor `spf[x]` precomputed up to N, any $x \le N$ factorises in $O(\log x)$ by repeatedly dividing by `spf[x]`. For many numbers up to 10^{12}, sieve primes up to 10^6 and trial-divide by primes only (78 498 of them).

Prime facts There are 25 primes below 100, 168 below 1000, 78 498 below 10^6, 664 579 below 10^7 and 50 847 534 below 10^9 (about $N/\ln N$). Gaps between consecutive primes below 10^{18} are at most about 1 500, so "the next prime after x" is found quickly by testing candidates. Useful primes: $10^9 + 7$, $998\,244\,353 = 119 \cdot 2^{23} + 1$ (NTT-friendly), $2^{61} - 1$ (hashing).

24.3 Large numbers: Miller–Rabin and Pollard rho

IOI / ADVANCED

For n up to 10^{18}, trial division is too slow. **Miller–Rabin** tests primality: write $n - 1 = 2^s d$; for a base a, n is "probably prime" if $a^d \equiv 1$ or $a^{2^r d} \equiv -1 \pmod n$ for some $r < s$. A fixed set of seven bases makes it deterministic for all 64-bit n. **Pollard's rho** finds a factor of a composite n by iterating $x \mapsto x^2 + c \bmod n$ and looking for $\gcd(|x - y|, n) > 1$; by the birthday paradox a factor p appears after about $\sqrt{p} \le n^{1/4}$ steps.

Template: Miller–Rabin + Pollard rho Ctrl+Alt+Y, X prefix `cppollard`

24.4 Modular arithmetic

GOLD

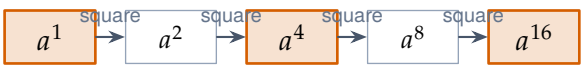
"Print the answer modulo $10^9 + 7$" means the true answer is huge and you should compute everything modulo M. Addition, subtraction and multiplication can be reduced at every step:

$$(a + b) \bmod M = \big((a \bmod M) + (b \bmod M)\big) \bmod M,$$
$$(a \cdot b) \bmod M = \big((a \bmod M) \cdot (b \bmod M)\big) \bmod M.$$

Negative numbers and overflow In C++ `(-7) % 3 == -1`. After a subtraction write `((a - b) % M + M) % M`. Two values below $10^9 + 7$ multiply to about 10^{18}, which fits in `long long` but not `int`; for moduli up to 10^{18} use `__int128` or a long-double `mulmod`.

24.4.1 Fast exponentiation

a^b by repeated squaring: $a^b = (a^{b/2})^2$ (times a if b is odd). $O(\log b)$ multiplications.



$21 = 10101_2$, so $a^{21} = a^{16} \cdot a^4 \cdot a^1$: 4 squarings and 2 extra multiplications.

Figure 24.2: Binary exponentiation multiplies the powers that correspond to set bits of the exponent.

Template: Modular power + inverse / Mint Ctrl+Alt+T, M / Shift+M prefix `cpmodpow, cpmint`

24.4.2 Division: modular inverses

Division is multiplication by an inverse: a^{-1} is the number with $a \cdot a^{-1} \equiv 1 \pmod M$. It exists iff $\gcd(a, M) = 1$.

- M prime: **Fermat's little theorem** $a^{M-1} \equiv 1$, so $a^{-1} = a^{M-2}$ (fast exponentiation).
- Any M: the extended Euclidean algorithm finds x with $ax + My = 1$; then $a^{-1} = x \bmod M$.
- All inverses of $1..n$ in $O(n)$: $\text{inv}[i] = -(M/i) \cdot \text{inv}[M \bmod i] \bmod M$.

Use Mint The library's `mint` type overloads `+ - * /` so modular code reads like ordinary arithmetic and cannot forget a reduction. It costs nothing at run time compared with manual `%`. For problems with the other common modulus change `MOD` in the template.

24.5 Extended Euclid and linear equations

GOLD

The extended Euclidean algorithm returns x, y with $ax+by = \gcd(a,b)$. Consequently the equation $ax + by = c$ has integer solutions iff $\gcd(a,b) \mid c$, and all solutions are $x = x_0 + k\frac{b}{g}$, $y = y_0 - k\frac{a}{g}$.

Template: Extended GCD Ctrl+Alt+T, X prefix `cpextgcd`

24.6 Chinese remainder theorem

PLATINUM

A system $x \equiv r_1 \pmod{m_1}$, $x \equiv r_2 \pmod{m_2}$ has a solution iff $r_1 \equiv r_2 \pmod{\gcd(m_1, m_2)}$, and then it is unique modulo $\text{lcm}(m_1, m_2)$. Combining two congruences at a time solves any number of them.

Template: CRT (any moduli) Ctrl+Alt+Y, 1 prefix `cpcrt`

CRT as a tool When a modulus is composite and awkward ($M = p_1 p_2$), compute the answer modulo each prime separately (where inverses and Lucas's theorem work) and glue the results with CRT. The same trick lets NTT multiply polynomials with huge coefficients: convolve modulo three NTT primes and reconstruct.

24.7 Euler's totient function

PLATINUM

$\varphi(n)$ counts $1 \le k \le n$ with $\gcd(k,n) = 1$: $\varphi(n) = n \prod_{p \mid n}(1 - \frac{1}{p})$. **Euler's theorem**: $a^{\varphi(n)} \equiv 1 \pmod{n}$ when $\gcd(a,n) = 1$. It reduces enormous exponents: $a^b \bmod n = a^{b \bmod \varphi(n)}$ for coprime a; for any a, $a^b \equiv a^{(b \bmod \varphi(n)) + \varphi(n)}$ whenever $b \ge \varphi(n)$ (the "generalised Euler theorem", the standard way to evaluate towers $a^{b^c} \bmod n$).

Template: Euler's totient Ctrl+Alt+Y, Shift+E prefix `cpphi`

Totient identities $\sum_{d \mid n} \varphi(d) = n$. The number of pairs $1 \le a, b \le n$ with $\gcd(a,b) = 1$ is $2\sum_{k=1}^{n} \varphi(k) - 1$. Fractions $\frac{a}{b}$ in lowest terms with $b \le n$ are counted by $\sum \varphi(b)$ (Farey sequences).

24.8 Möbius function and inversion

IOI / ADVANCED

$\mu(n) = 0$ if n has a squared prime factor, otherwise $(-1)^{\text{number of prime factors}}$. Its key property: $\sum_{d|n} \mu(d) = [n = 1]$. That turns "count pairs with gcd = 1" into sums over divisors:

$$\#\{(a,b) \le N : \gcd(a,b) = 1\} = \sum_{a,b} \sum_{d|\gcd(a,b)} \mu(d) = \sum_{d=1}^{N} \mu(d) \left\lfloor \frac{N}{d} \right\rfloor^2.$$

More generally, **Möbius inversion**: if $g(n) = \sum_{d|n} f(d)$ then $f(n) = \sum_{d|n} \mu(d)\, g(n/d)$. Counting objects "with gcd exactly k" is usually easiest as "gcd divisible by k" (a simple count) followed by inversion.

</> Template: Möbius sieve Ctrl+Alt+Y, Shift+M prefix `cpmobius`

24.9 Floor-division blocks

PLATINUM

$\lfloor n/i \rfloor$ takes at most $2\sqrt{n}$ distinct values as i runs from 1 to n, and each value is taken on a contiguous block of i. Iterating over blocks turns sums like $\sum_{i=1}^{n} f(i)\lfloor n/i \rfloor$ into $O(\sqrt{n})$ terms (given prefix sums of f).

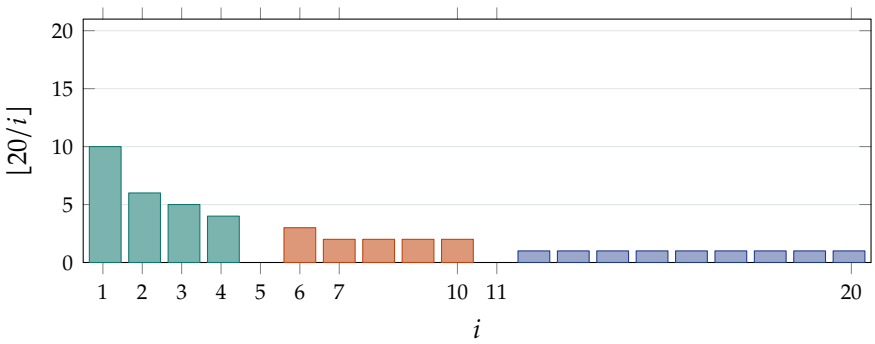


Figure 24.3: $\lfloor 20/i \rfloor$: the value 1 covers $i = 11..20$, the value 2 covers $7..10$. A block starting at l ends at $r = \lfloor n/\lfloor n/l \rfloor \rfloor$.

</> Template: Floor-division blocks Ctrl+Alt+Y, Shift+F prefix `cpfloorblocks`

Example: Sum of divisors (CSES)

Compute $\sum_{k=1}^{n} \sigma(k) \bmod 10^9 + 7$ for $n \le 10^{12}$, where σ is the sum of divisors.

Solution. Swap the order: every d divides $\lfloor n/d \rfloor$ numbers up to n, so the sum is $\sum_d d\lfloor n/d \rfloor$. Over a block $[l, r]$ with constant q, the contribution is $q \cdot \frac{(l+r)(r-l+1)}{2}$. $O(\sqrt{n})$.

Prefix sums of multiplicative functions To sum φ or μ up to $n = 10^{10}$ you cannot sieve that far. Use a Dirichlet identity instead, e.g. $\sum_{d=1}^{n} \Phi(\lfloor n/d \rfloor) = \frac{n(n+1)}{2}$ where Φ is the prefix sum of φ. Sieve Φ up to $n^{2/3}$ and compute the larger values recursively over floor blocks with memoisation: $O(n^{2/3})$ (Du's sieve). Min_25 and similar sieves extend this to general multiplicative functions; they are the "Part 2" of this topic and rarely needed below the highest levels.

</> Template: Divisor sieve (count + sum) Ctrl+Alt+T, Shift+2 prefix `cpdivsieve`

CHAPTER 25

Combinatorics, Probability and Games

25.1 Counting principles

SILVER

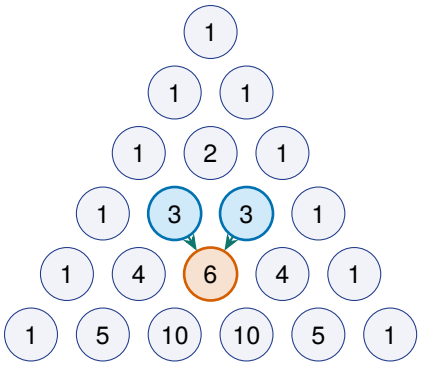
- **Product rule**: k independent choices with $c_1, \ldots, c_k$ options give $c_1 \cdots c_k$ outcomes.
- **Sum rule**: if outcomes split into disjoint cases, add the counts of the cases.
- **Complement**: count everything, subtract the bad outcomes.
- **Bijection**: count something easier that is in one-to-one correspondence.
- **Double counting / contribution**: sum over elements how many structures each belongs to.

Choose k of n	order matters	order does not matter
without repetition	$n!/(n-k)!$	$\binom{n}{k} = \frac{n!}{k!(n-k)!}$
with repetition	n^k	$\binom{n+k-1}{k}$ (stars and bars)

25.2 Binomial coefficients

GOLD

$\binom{n}{k}$ is the number of k-element subsets of an n-element set. Modulo a prime $p > n$, precompute factorials and inverse factorials once in $O(N)$; then each $\binom{n}{k} = n!\,(k!)^{-1}((n-k)!)^{-1}$ is $O(1)$.



Pascal's rule: $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$ (choose element n or not). Works modulo *any* number, in $O(n^2)$.

Figure 25.1: Pascal's triangle.

</> Template: nCr / factorials, Pascal Ctrl+Alt+T, N / Ctrl+Alt+Y, Shift+N prefix `cpncr`, `cppascal`

Identities worth knowing $\sum_k \binom{n}{k} = 2^n$; $\binom{n}{k} = \binom{n}{n-k}$; $\sum_{i=k}^{n} \binom{i}{k} = \binom{n+1}{k+1}$ (hockey stick); $\sum_k \binom{m}{k}\binom{n}{r-k} = \binom{m+n}{r}$ (Vandermonde); $k\binom{n}{k} = n\binom{n-1}{k-1}$. Paths from $(0,0)$ to (a,b) with unit right/up steps: $\binom{a+b}{a}$.

Lucas's theorem For a small prime p (so $n \geq p$ and factorials vanish mod p), write n and k in base p: $\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod{p}$. In particular $\binom{n}{k}$ is odd iff $k \,\&\, n = k$ (every bit of k is set in n).

25.3 Stars and bars

GOLD

The number of ways to split n identical items into k distinguishable boxes (possibly empty) is $\binom{n+k-1}{k-1}$: arrange n stars and $k-1$ bars in a row. With every box non-empty, $\binom{n-1}{k-1}$.

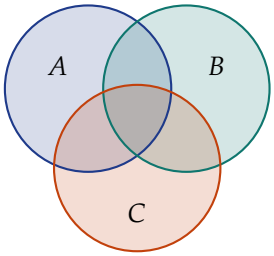


Figure 25.2: A distribution is an arrangement of stars and bars.

25.4 Inclusion–exclusion

PLATINUM

$|A_1 \cup \cdots \cup A_k| = \sum |A_i| - \sum |A_i \cap A_j| + \sum |A_i \cap A_j \cap A_l| - \ldots$: add single sets, subtract pairs, add triples, and so on. Over subsets S: $\sum_{S \neq \emptyset} (-1)^{|S|+1} \left| \bigcap_{i \in S} A_i \right|$.



$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$. The centre is added 3 times, subtracted 3 times, added once: counted exactly once.

Figure 25.3: Inclusion–exclusion for three sets.

</> Template: Inclusion–exclusion (divisible by any) Ctrl+Alt+Y, 3 prefix `cpincexc`

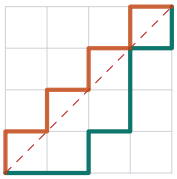
How to recognize it "At least one", "none of the forbidden", "all conditions hold" with few conditions ($k \leq 20$), "coprime to m" (the conditions are the distinct primes of m), surjections, and counting with restrictions that are easy to force but hard to forbid. Derangements are the classic example.

Derangements Permutations with no fixed point: $D_n = n! \sum_{k=0}^{n} \frac{(-1)^k}{k!}$ (inclusion–exclusion over fixed points), or $D_n = (n-1)(D_{n-1} + D_{n-2})$ with $D_0 = 1, D_1 = 0$: $1, 0, 1, 2, 9, 44, 265$. $D_n/n! \to 1/e$.

25.5 Catalan numbers

PLATINUM

$C_n = \frac{1}{n+1}\binom{2n}{n}$: $1, 1, 2, 5, 14, 42, 132, 429, \ldots$ They count balanced bracket sequences with n pairs, binary trees with n nodes, triangulations of a convex $(n+2)$-gon, monotone lattice paths that never cross the diagonal, and many more. Recurrence: $C_{n+1} = \sum_{i=0}^{n} C_i C_{n-i}$ (split at the bracket that closes the first one).



Teal: a path that stays below the diagonal, a valid bracket sequence `(()())()` reading right = "(" and up = ")". Orange crosses above it: invalid. The reflection trick shows the invalid paths number $\binom{2n}{n+1}$, so the valid ones are $\binom{2n}{n} - \binom{2n}{n+1} = C_n$.

Figure 25.4: Catalan numbers count lattice paths that never go above the diagonal.

</> Template: nCr / factorials (includes catalan) Ctrl+Alt+T, N prefix `cpncr`

25.6 Burnside's lemma

IOI / ADVANCED

To count objects *up to symmetry* (necklaces up to rotation, colourings of a cube up to rotation), average the number of objects fixed by each symmetry: #classes $= \frac{1}{|G|}\sum_{g\in G} \mathrm{fix}(g)$. Necklaces of length n with k colours up to rotation: $\frac{1}{n}\sum_{d\mid n} \varphi(d)\, k^{n/d}$.

25.7 Probability

GOLD

For a finite set of equally likely outcomes, $P(E) = |E|/|\Omega|$. Expected values are weighted averages; by **linearity**, the expected sum of anything is the sum of expectations, even for dependent quantities (Chapter 19). The expected number of distinct values among n random draws from k values is $k\,(1-(1-1/k)^n)$; the expected maximum of dice uses $P(\max \le x) = (x/6)^n$.

> **Expected value of a maximum or minimum** $E[\max] = \sum_{x\ge 1} P(\max \ge x)$ for non-negative integers, and $P(\max \le x)$ is a product of independent probabilities. Converting "expectation of a max" into a sum of "probability that all are $\le x$" removes the max.

25.8 Game theory

PLATINUM

25.8.1 Winning and losing positions

In an impartial game (both players have the same moves) where the player who cannot move loses, every position is winning (W) or losing (L): L if all moves lead to W positions, W if some move leads to an L position. Compute this backwards from the terminal positions.

25.8.2 Nim

Several piles; a move removes any positive number of stones from one pile. The first player wins iff the XOR of the pile sizes is non-zero. From a non-zero XOR there is always a move to XOR 0 (reduce a pile that has the XOR's highest bit), and from XOR 0 every move breaks it.

25.8.3 Sprague–Grundy

Every impartial game position has a **Grundy number**: $g(x) = \mathrm{mex}\{g(y) : y \text{ reachable from } x\}$ (mex = the smallest non-negative integer not in the set). A position is losing iff $g = 0$, and a *sum* of independent games (a move is made in exactly one of them) has Grundy number equal to the

XOR of the parts. So every "several independent piles" game is Nim in disguise once the Grundy numbers of single piles are known.

x	0	1	2	3	4	5	6	7	8	9	10
$g(x)$, moves remove 1, 3 or 4	0	1	0	1	2	3	2	0	1	0	1

</> Template: Grundy numbers Ctrl+Alt+Y, G prefix `cpgrundy`

Find the pattern Grundy sequences of single-pile games are usually periodic after a short prefix. Compute g up to a few thousand with the template, print it, find the period, and answer queries for x up to 10^{18} by periodicity.

How to recognize it Two players alternate, perfect information, no chance, the player unable to move loses: think W/L DP for one game, Grundy numbers and XOR for several independent games. "The last player to move *loses*" (misère) changes the analysis; for Nim it only matters when all piles have size ≤ 1.

CHAPTER 26

Linear Algebra and Polynomials

26.1 Matrices and matrix exponentiation

GOLD

An $n \times m$ matrix times an $m \times p$ matrix costs $O(nmp)$; for square matrices $O(n^3)$. Matrix power A^k by repeated squaring takes $O(n^3 \log k)$ (Chapter 19 shows how linear recurrences become matrix powers).

Walks in a graph If A is the adjacency matrix of a graph, $(A^k)_{uv}$ is the number of walks of length exactly k from u to v. With $(\min, +)$ instead of $(+, \times)$ it is the length of the shortest walk with exactly k edges.

</> Template: Matrix exponentiation Ctrl+Alt+T, Shift+X prefix `cpmatrix`

26.2 Gaussian elimination

PLATINUM

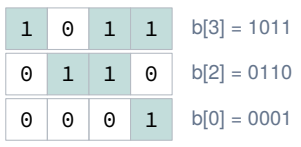
To solve n linear equations in n unknowns, repeatedly pick a pivot row with a non-zero coefficient in the current column, scale it, and subtract multiples of it from the other rows until the matrix is diagonal. $O(n^3)$. Modulo a prime use modular inverses for scaling; over the reals choose the pivot with the largest absolute value (partial pivoting) to limit rounding error. The same procedure computes the **rank** (number of pivots), the **determinant** (product of pivots, with a sign flip per row swap) and the **inverse** matrix.

How to recognize it "Each of these switches toggles some lamps; can all lamps be turned on?" (a system over GF(2), where each row is a bitset and elimination runs 64 times faster), expected values on a graph with cycles (one equation per state: $E_v = 1 + \sum_w p_{vw} E_w$), counting spanning trees (Kirchhoff's theorem: a determinant of the Laplacian), and anything phrased as "linear combinations".

26.3 XOR basis (linear algebra over GF(2))

IOI / ADVANCED

Numbers with XOR form a vector space over the two-element field: a number is a bit vector, XOR is vector addition. A **linear basis** keeps at most one vector per leading bit. Inserting x XORs it with basis vectors from the top bit down; if it reaches 0 it was already representable, otherwise it becomes a new basis vector.



Each basis vector has a distinct highest bit; the three vectors span $2^3 = 8$ values. Maximum subset XOR, greedily from the top: $0 \oplus 1011 = 1011$, then $1011 \oplus 0110 = 1101$ (larger, keep), then $1101 \oplus 0001 = 1100$ would be smaller, so the maximum is 1101.

Figure 26.1: A linear basis over GF(2).

</> Template: XOR basis Ctrl+Alt+Y, Shift+X prefix `cpxorbasis`

How to recognize it "Maximum XOR of a subset", "number of distinct subset XORs" (2^{rank}), "can x be written as a XOR of some of these", "k-th smallest subset XOR", XOR of paths in a graph (any cycle's XOR can be added freely: put all cycle XORs into a basis). Values up to 2^{60} with n up to 10^5 are typical.

Prefix bases for range queries "Maximum subset XOR of $a[l, r)$" online: keep, for every prefix, a basis where each vector remembers the *latest* index it came from, preferring newer vectors on insertion. For a query, use only basis vectors of prefix r with index $\geq l$. $O((n + q) \cdot \text{bits})$.

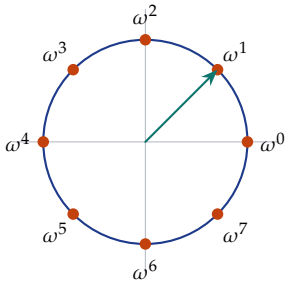
26.4 Polynomial multiplication and FFT

IOI / ADVANCED

Multiplying polynomials $A(x) = \sum a_i x^i$ and $B(x) = \sum b_j x^j$ gives coefficients $c_k = \sum_{i+j=k} a_i b_j$: the **convolution** of the coefficient arrays. Directly it is $O(nm)$. The Fast Fourier Transform does it in $O(n \log n)$:

1. **Evaluate** A and B at N special points, the complex N-th roots of unity $\omega^k = e^{2\pi ik/N}$ ($N \geq n + m - 1$, a power of two);
2. **multiply** the values pointwise: $C(\omega^k) = A(\omega^k)B(\omega^k)$;
3. **interpolate** C back from its values (an inverse FFT, which is the same algorithm with ω^{-1}).

Evaluation at all roots of unity is fast because of symmetry: splitting A into even and odd coefficients, $A(x) = A_{\text{even}}(x^2) + x\,A_{\text{odd}}(x^2)$, and the squares of the N-th roots are the $N/2$-th roots. So two half-size problems plus $O(N)$ work: $T(N) = 2T(N/2) + O(N) = O(N \log N)$.



The 8th roots of unity. Squaring maps ω^k to ω^{2k}: the 8 points collapse onto the 4 points $\omega^0, \omega^2, \omega^4, \omega^6$, the 4th roots of unity. This halving is why the recursion works.

Figure 26.2: Roots of unity: the evaluation points of the FFT.

</> Template: FFT (exact integer convolution) / NTT Ctrl+Alt+Y, F / Shift+T prefix `cpfft, cpntt`

26.4.1 NTT

The **number theoretic transform** is the same algorithm over integers modulo a prime $p = c \cdot 2^k + 1$ (for example $998\,244\,353 = 119 \cdot 2^{23} + 1$), which has 2^k-th roots of unity. It is exact (no rounding) and gives coefficients modulo p, which is what counting problems ask for.

Precision of the complex FFT Doubles hold about 15 significant digits. The library's FFT uses one complex transform with long-double roots and stays exact while $\sum a_i^2 + \sum b_i^2$ times $\log N$ is below roughly 10^{15}. For larger values modulo an arbitrary number, split each coefficient into two 15-bit halves (three or four FFTs) or use three NTT primes with CRT.

26.5 Applications of convolution

IOI / ADVANCED

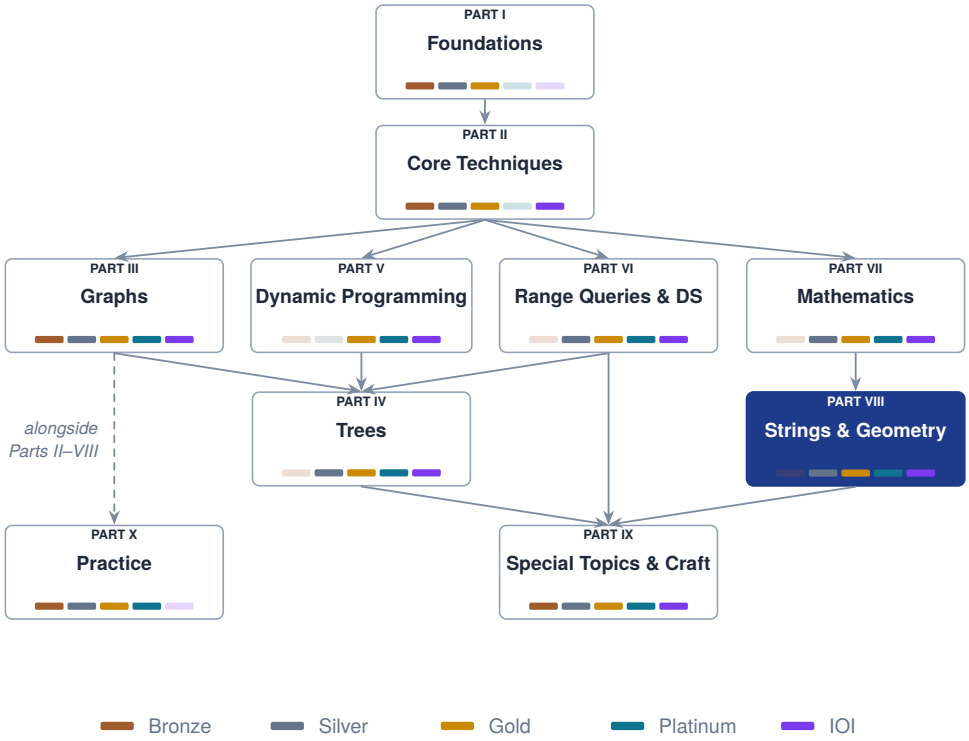
Problem	Convolution
number of pairs (i, j) with $a_i + b_j = s$, for all s	histograms of a and b, convolved
all pairwise sums / differences of two sets	histogram of a with reversed histogram of b
string matching with wildcards	$\sum(p_j - t_{i+j})^2 p_j t_{i+j}$ expands into three convolutions
Hamming distance of a pattern at every shift	one convolution per letter of the alphabet
counting ways to pick one item from each of k groups	multiply the groups' generating functions
multiplying big integers	digits as coefficients, then carry

Generating functions Encode a counting problem as a polynomial: the number of ways to reach total s with coins is the coefficient of x^s in $\prod_i \frac{1}{1-x^{c_i}}$. Products are convolutions, and with polynomial inverse, logarithm and exponential (Newton's method on power series, each $O(n \log n)$) many "count the ways" problems with n up to 10^5 become a few lines once a polynomial library is available. Divide-and-conquer FFT (multiplying many small polynomials in a balanced tree, or "online" convolution for $f_n = \sum_{i<n} f_i g_{n-i}$) covers most of the rest.

PART VIII

Strings and Geometry

Searching, hashing and suffix structures for strings; vectors, hulls and sweeps for geometry.



CHAPTER 27

Strings

String problems look diverse, but most reduce to a few questions: does a pattern occur (and where), are two substrings equal, what is the longest common prefix of two suffixes, which substrings are palindromes. This chapter builds the standard tools in order of power.

27.1 String hashing

GOLD

Map a string to a number: $h(s) = s_0 B^{n-1} + s_1 B^{n-2} + \cdots + s_{n-1} \bmod M$ for a random base B. Prefix hashes H_i (hash of the first i characters) let you hash any substring in $O(1)$:

$$\text{hash}(s[l, r)) = H_r - H_l \cdot B^{r-l} \pmod{M}.$$

Equal strings have equal hashes; different strings collide with probability about n/M per comparison.

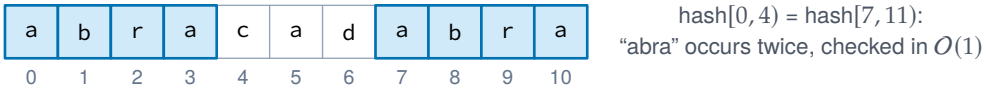


Figure 27.1: Comparing substrings by their hashes.

</> Template: String hashing Ctrl+Alt+T, H prefix `cpstrhash`

Weak hashes get hacked With $M = 2^{64}$ (natural overflow) there is a known string (Thue–Morse) that collides for every base. With a fixed small prime and fixed base, others can build collisions. The library uses $M = 2^{61} - 1$ and a base chosen at random at run time: collision probability about 10^{-18} per comparison, and nobody can precompute a bad input.

Hash + binary search The longest common prefix of two suffixes: binary search the length, comparing hashes: $O(\log n)$. That gives $O(\log n)$ lexicographic comparison of any two substrings, the longest palindrome centred anywhere, the longest repeated substring (binary search the length, put all substring hashes of that length in a hash set), and suffix sorting in $O(n \log^2 n)$ without a suffix array.

27.2 Prefix function and KMP

PLATINUM

The **prefix function** $\pi[i]$ is the length of the longest proper prefix of $s[0..i]$ that is also a suffix of it (a *border*). It is computed in $O(n)$ by trying the borders of the previous position in decreasing order.

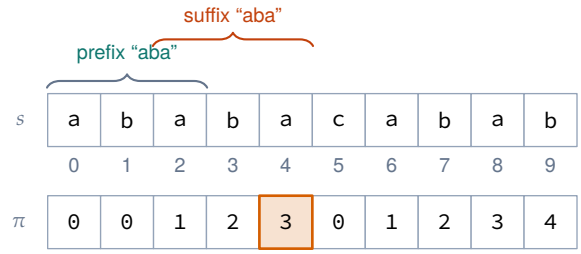


Figure 27.2: $\pi[4] = 3$: “aba” is both a prefix and a suffix of “ababa”.

To find a pattern p in a text t, compute π of $p + \# + t$ (with a separator not in either string): every position where $\pi = |p|$ ends an occurrence. This is the KMP algorithm, $O(|p| + |t|)$.

</> Template: KMP (prefix function + pattern search) Ctrl+Alt+T, Shift+K prefix `cpkmp`

> **Periods** The smallest period of s is $n - \pi[n-1]$; s is a repetition of a shorter string iff that period divides n. Borders of borders are borders, so following π from the end enumerates all of them.

27.3 Z-function

PLATINUM

$z[i]$ is the length of the longest common prefix of s and its suffix starting at i. The algorithm keeps the rightmost “Z-box” $[l, r)$ that matches a prefix and reuses its information: $O(n)$.



$z[4] = 4$: “aabx” matches the prefix; the Z-box is $[4, 8)$

Figure 27.3: The Z-function.

</> Template: Z-function Ctrl+Alt+T, Z prefix `cpz`

27.4 Tries

GOLD

A **trie** stores a set of strings as a tree whose edges are letters; each string is a root-to-node path. It answers “is s in the set”, “how many strings start with p” and lexicographic walks in $O(|s|)$.

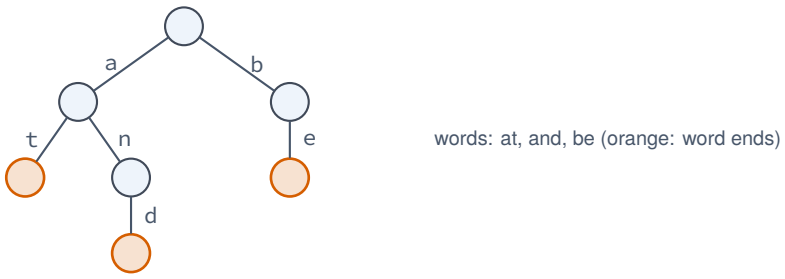


Figure 27.4: A trie of three words.

</> Template: Trie / binary trie Ctrl+Alt+T, Shift+T / ` prefix `cptrie, cpxortrie`

27.5 Aho–Corasick

IOI / ADVANCED

To find occurrences of *many* patterns in a text at once, build their trie and add **suffix links**: the link of a node points to the longest proper suffix of its string that is also a node of the trie (KMP's prefix function, generalised to a trie). With the full transition table the text is scanned once: $O(\text{total pattern length} \cdot \Sigma + |t|)$. Summing matches along suffix links counts every occurrence of every pattern.

</> Template: Aho–Corasick Ctrl+Alt+Y, Shift+A prefix `cpaho`

> **How to recognize it** Many patterns and one text ("count occurrences of each dictionary word"), and DP over strings that must avoid (or must contain) a set of forbidden words: the automaton's states become the DP states ("number of strings of length n that contain no forbidden word" = walks of length n in the automaton avoiding terminal states, possibly with matrix power).

27.6 Manacher's algorithm

PLATINUM

Manacher computes, for every centre, the radius of the longest palindrome centred there, in $O(n)$, by reusing the mirror image inside the rightmost palindrome found so far (the same idea as the Z-box). Odd and even lengths are handled by two arrays (or by inserting separators). It gives the longest palindromic substring and the total number of palindromic substrings.

</> Template: Manacher Ctrl+Alt+Y, 6 prefix `cpmanacher`

27.7 Suffix arrays

IOI / ADVANCED

The **suffix array** is the list of starting positions of all suffixes in sorted order; the **LCP array** stores the longest common prefix of each pair of neighbours in that order.

rank	sa	lcp	suffix
0	5	1	a
1	3	3	ana
2	1	0	anana
3	0	0	banana
4	4	2	na
5	2	–	nana

Figure 27.5: Suffix array and LCP array of “banana” (lcp with the next row).

Built in $O(n \log n)$ by sorting cyclic shifts by their first 2^k characters (the rank of each half is already known from the previous round); LCP in $O(n)$ by Kasai’s algorithm.

</> Template: Suffix array + LCP Ctrl+Alt+Y, 4 prefix `cpsa`

What suffix arrays answer

- **Number of distinct substrings**: $\frac{n(n+1)}{2} - \sum \text{lcp}$.
- **LCP of any two suffixes**: the minimum of the LCP array between their ranks (sparse table: $O(1)$).
- **Longest repeated substring**: the maximum LCP value.
- **Longest common substring of s and t**: build on $s + \# + t$; the maximum LCP between neighbours from different strings.
- **Pattern search**: binary search the sorted suffixes, $O(|p| \log n)$.
- **k-th lexicographic substring**: walk the ranks, each suffix contributing $n - \text{sa}[i] - \text{lcp}[i-1]$ new ones.

27.8 Suffix automaton

IOI / ADVANCED

The **suffix automaton** of s is the smallest deterministic automaton accepting exactly the substrings of s. It has at most $2n - 1$ states and $3n - 4$ transitions and is built online in $O(n\Sigma)$ (one character at a time). Each state corresponds to a set of substrings that end at the same set of positions; `len` is the longest of them and the suffix link points to the state of the next shorter group.

</> Template: Suffix automaton Ctrl+Alt+Y, 5 prefix `cpsam`

Suffix automaton answers Distinct substrings = $\sum_{v \neq 0} (\text{len}(v) - \text{len}(\text{link}(v)))$. Occurrence counts: propagate end counts along suffix links in decreasing `len`. Longest common substring of s and t: walk t through the automaton of s keeping the current match length. It is the most powerful single string structure, and the least intuitive; reach for hashing or a suffix array first when they suffice.

27.9 Choosing a string tool

GOLD

Task	Simplest tool that works
one pattern in one text	KMP or Z-function (or `string::find` for small inputs)
compare arbitrary substrings	hashing
many patterns in one text	Aho–Corasick
borders, periods, repetitions	prefix function
palindromes	Manacher, or hashing of s and its reverse
distinct substrings, LCPs, lexicographic order	suffix array
online substring queries, occurrence counts	suffix automaton
set of strings, prefix counts, max XOR	trie / binary trie

CHAPTER 28

Computational Geometry

Geometry problems are mostly about a few robust primitives used carefully. The golden rule: **stay in integers whenever the input is integral**. Almost everything below needs only additions, multiplications and comparisons.

28.1 Vectors, dot and cross products

GOLD

A point is a vector (x, y). For vectors a, b:

- $a \cdot b = a_x b_x + a_y b_y = |a||b| \cos\theta$: positive if the angle is acute, zero if perpendicular.
- $a \times b = a_x b_y - a_y b_x = |a||b| \sin\theta$: positive if b is counter-clockwise from a, negative if clockwise, zero if collinear. $|a \times b|$ is the area of the parallelogram they span.

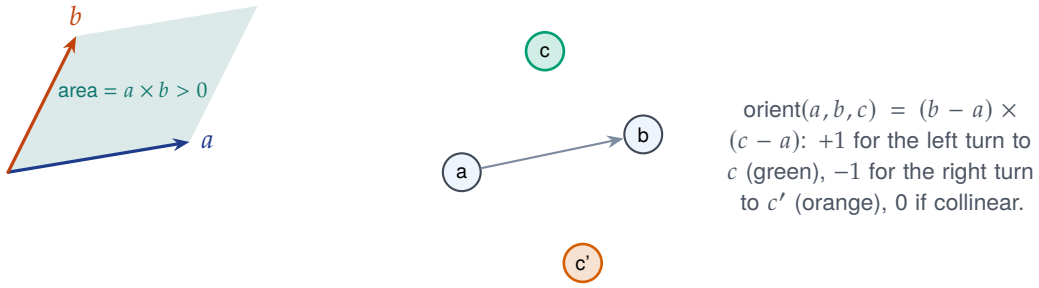


Figure 28.1: The cross product gives signed area and orientation.

</> Template: Geometry primitives Ctrl+Alt+T, Shift+3 prefix `cppoint`

> **⚠ Precision** Coordinates up to 10^9 give cross products up to about $4 \cdot 10^{18}$: fits in `long long`, barely. Products of three coordinates do not. When floating point is unavoidable (circles, intersections), compare with a tolerance (`fabs(a - b) < 1e-9`), prefer `long double`, and avoid `atan2` and square roots where a comparison of squared quantities works.

28.2 Segment intersection

GOLD

Segments ab and cd properly cross iff c and d are on opposite sides of line ab *and* a and b are on opposite sides of line cd (all orientation signs strictly opposite). Otherwise they touch only if an endpoint of one lies on the other.

</> Template: Segment intersection Ctrl+Alt+Y, 7 prefix `cpseginter`

28.3 Polygon area, lattice points and point location

GOLD

Shoelace formula: for a polygon with vertices $p_0, \ldots, p_{n-1}$ in order, $2 \cdot \text{Area} = \left|\sum_i p_i \times p_{i+1}\right|$ (the sign tells the orientation: positive is counter-clockwise).

> **Pick's theorem** For a simple polygon with integer vertices, $\text{Area} = I + \frac{B}{2} - 1$, where I counts lattice points strictly inside and B lattice points on the boundary. An edge from p to q contains $\gcd(|q_x - p_x|, |q_y - p_y|)$ boundary points (not counting p). So $I = \frac{2A-B+2}{2}$ is computable without enumerating points.

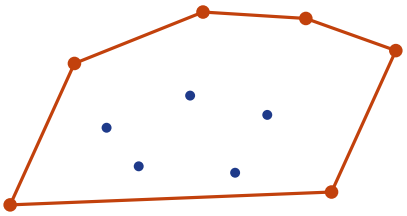
Point in polygon: cast a ray to the right from the point and count the edges it crosses; odd means inside. Handle "on the boundary" first, and count an edge only when the point's y is in $[\min y, \max y)$ of the edge, which treats vertices consistently.

</> Template: Polygon: area, point inside, lattice points Ctrl+Alt+Y, 8 prefix `cppolygon`

28.4 Convex hull

PLATINUM

The convex hull is the smallest convex polygon containing all the points: a rubber band stretched around them. Andrew's **monotone chain** sorts the points by x, builds the lower hull left to right keeping only left turns, then the upper hull right to left. $O(n \log n)$.



Orange: the hull. When a new point makes a right turn with the last two hull points, the middle one is popped: every point is pushed and popped at most once after sorting.

Figure 28.2: The convex hull of a point set.

</> Template: Convex hull Ctrl+Alt+T, Shift+V prefix `cphull`

> **What to do with a hull**
>
> - **Farthest pair** (diameter): rotating calipers walk two pointers around the hull in $O(h)$.
> - **Point in convex polygon** in $O(\log n)$: binary search the wedge around vertex 0.
> - **Minimum-area enclosing rectangle**, width, and "maximise $a \cdot x + b \cdot y$" queries (a linear function is maximised at a hull vertex, found by binary search on edge directions).
> - The convex hull trick of Chapter 20 is the same object in the dual plane.

28.5 Closest pair of points

PLATINUM

Sort by x and sweep. Keep the points whose x is within the best distance d found so far in a set ordered by y; for a new point, only the points in that set with y in $[y - d, y + d]$ can beat d, and there are $O(1)$ of them (they are pairwise at least d apart). $O(n \log n)$.

</> Template: Closest pair Ctrl+Alt+Y, 9 prefix `cpclosest`

28.6 Sweep lines

PLATINUM

A sweep line moves across the plane and processes *events* (segment starts and ends, points, queries) in order, keeping the objects crossing the line in a data structure. It turns 2D problems into a sequence of 1D problems.

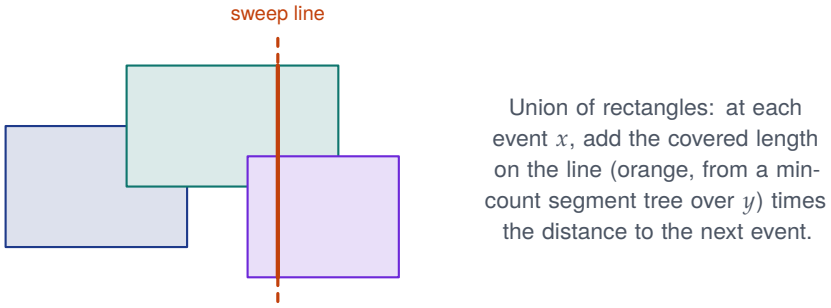


Figure 28.3: A vertical sweep over rectangles.

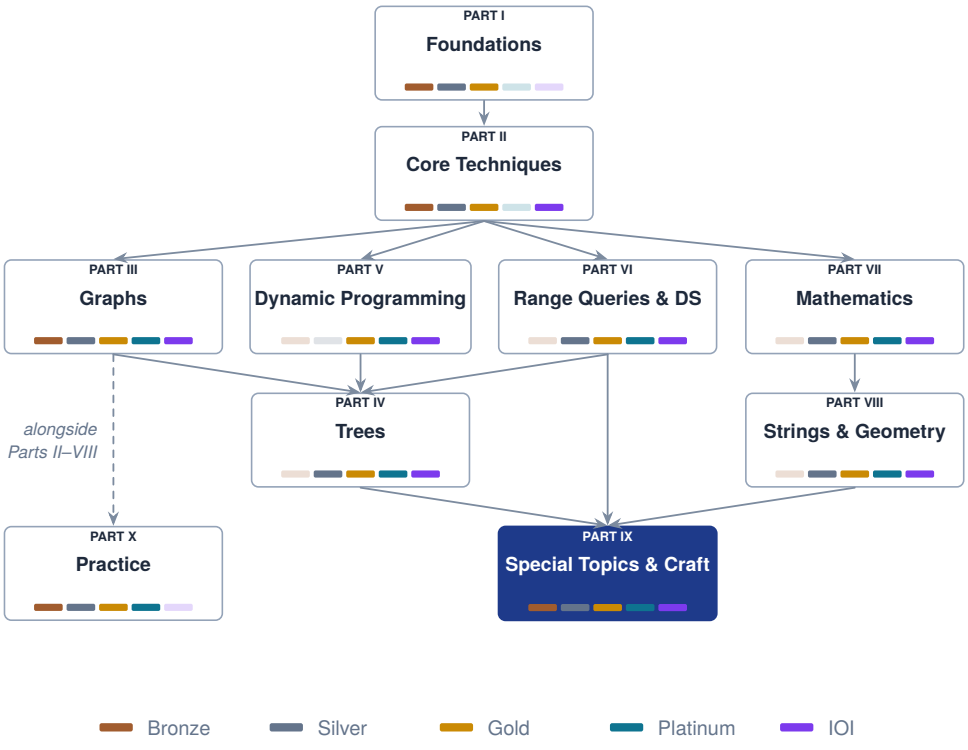
How to recognize it Many intervals, rectangles or segments and a question about their overlap, union or intersections; "for each point, how many rectangles contain it"; "the maximum number of overlapping intervals" (a 1D sweep over $+1/-1$ events). Sort the events, decide what the "active set" must support, pick the data structure.

Sorting by angle without atan2 To sort vectors by polar angle exactly, split them into two half-planes (upper: $y > 0$ or $y = 0, x > 0$; lower: the rest), compare halves first, and within a half use the cross product: a comes before b iff $a \times b > 0$. Integer-exact, and the basis of angular sweeps ("maximum number of points visible in a 90° window").

PART IX

Special Topics and Contest Craft

Randomization, interactive problems, bitsets, matroids and vectorization; the craft of contests; and a catalogue of the tricks experienced competitors reach for.



CHAPTER 29

Special Topics

29.1 Randomized algorithms

PLATINUM

Randomness gives simple algorithms with guarantees that hold with overwhelming probability, and it defeats inputs built to break deterministic choices.

- **Random pivots and shuffles**: shuffling the input before inserting into a naive BST, before a quickselect, or before an incremental algorithm (smallest enclosing circle, expected $O(n)$) turns worst cases into expected cases.
- **Randomized hashing**: random bases and seeds (string hashing, `custom_hash`, set hashing by random 64-bit values) make collisions both rare and impossible to construct in advance.
- **Sampling**: if an answer is shared by at least half of the elements (a majority, a gcd that divides at least half of the numbers, a line through at least a fraction of the points), pick a random element and test it: each try succeeds with probability $\geq \frac{1}{2}$, so 30 tries fail with probability 2^{-30}.
- **Random priorities**: treaps, random weights in skip lists.

Example: Ghd (Codeforces 364D)

Find the largest g that divides at least half of $n \leq 10^6$ numbers up to 10^{12}.

Solution. The answer divides some element of the "good" half, so a random element is a candidate source with probability $\geq \frac{1}{2}$. For a random a_i, the answer is a divisor of a_i; count, for each divisor of a_i, how many elements share it (via gcds with a_i, grouped). A dozen random picks suffice.

Seeds Seed with the clock (`chrono::steady_clock::now().time_since_epoch().count()`), not a constant. On Codeforces, a fixed seed lets anyone read your code and craft an anti-random input during the hacking phase.

Template: Random generator Ctrl+Alt+T, Shift+R prefix `cprng`

29.2 Interactive problems

GOLD

The judge answers your questions during the run. Rules that always apply:

- **Flush after every query**: `cout << ... << endl` (`endl` flushes) or `cout.flush()`. Forgetting it gives "Idleness limit exceeded".
- **Count your queries** against the limit before coding; $\log_2(10^9) \approx 30$, $\lceil \log_3 n \rceil$ for ternary splits.
- **Stop immediately** if the judge answers -1 (wrong query or limit exceeded), or you get a misleading verdict.

- Some judges are **adaptive**: the hidden answer is only fixed to be consistent with your questions, so randomised strategies that rely on a fixed hidden value can fail. Design strategies that are worst-case optimal.

</> Template: Interactive helpers Ctrl+Alt+T, Shift+I prefix `cpinteractive`

Testing interactive solutions locally CPH cannot play the judge. Write a small interactor that reads your queries and answers them, or temporarily replace `ask` with a function that computes the answer from a hidden array you generate; the rest of the solution stays unchanged.

29.3 Communication problems

IOI / ADVANCED

Your program runs twice: first to *encode* some information into a limited message, then (without memory of the first run) to *decode* it. Typical tools: fixing a canonical order so both runs agree, encoding numbers in the "shape" of the output (which elements are chosen, their order: $n!$ permutations carry $\log_2 n!$ bits), and error-correcting parity bits. Think of the two runs as two parties who agreed on a protocol in advance and design the protocol first.

29.4 Bitsets

PLATINUM

`bitset<N>` packs booleans 64 per machine word, and its operations work word by word. Any algorithm dominated by OR/AND of boolean rows gets a 64× speed-up:

- subset sums and knapsack feasibility: `b |= b << w;`
- transitive closure / reachability in a DAG: `reach[u] |= reach[v]` in reverse topological order, $O(nm/64)$;
- counting triangles or common neighbours: `(adj[u] & adj[v]).count();`
- Gaussian elimination over GF(2) with rows as bitsets.

Bitset sizes are compile-time `N` must be a constant. If the needed size varies between tests, use the maximum; or write a template function instantiated for a few sizes and dispatch at run time.

29.5 Matroids

IOI / ADVANCED

A **matroid** is a ground set with a family of "independent" subsets such that (1) subsets of independent sets are independent and (2) if A and B are independent with $|A| < |B|$, some element of $B \setminus A$ can be added to A keeping it independent (the *exchange property*). The theorem that makes them useful: **on a matroid, the greedy algorithm (add elements in order of decreasing weight whenever independence is kept) finds a maximum-weight independent set.**

Matroid	Independent sets, and the greedy algorithm it justifies
graphic	forests of a graph: Kruskal's MST
linear (over GF(2))	linearly independent vectors: XOR basis with maximum total weight
uniform	sets of size $\leq k$: take the k heaviest
partition	at most c_i elements from group i
scheduling	sets of unit jobs that can all meet their deadlines: the deadline greedy of Chapter 7

How to recognize it A greedy "sort by weight, take if still feasible" that you cannot prove with an exchange argument: check whether feasibility defines a matroid; if it does, greedy is correct. If feasibility is the intersection of *two* matroids (for example "a spanning forest using at most one edge of each colour": graphic ∩ partition), greedy fails but **matroid intersection** finds the largest common independent set with repeated augmenting paths in an exchange graph, in $O(r \cdot n \cdot (\text{oracle cost}))$ for rank r. Bipartite matching is the simplest case (two partition matroids).

29.6 Vectorization and pragmas

IOI / ADVANCED

Modern CPUs process 4–8 integers per instruction (SIMD). GCC vectorizes simple loops over contiguous arrays automatically at `-O3`; some contestants add

```
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,popcnt,lzcnt")
```

at the top of the file. It can make an $O(n^2)$ loop over plain arrays several times faster, which occasionally turns an intended-to-fail n^2 into a pass. It is not a substitute for a better algorithm, it helps only branch-free loops over arrays (not `set`, `map` or recursion), and `target` can crash with "illegal instruction" on judges whose CPUs lack AVX2 (some older USACO and ICPC machines). Keep it out of your default template and add it only when a constant factor is the bottleneck.

29.7 Constructive problems

SILVER

"Output any array / permutation / graph with property P" problems reward patterns more than algorithms:

- Solve small n by brute force (write the brute force!), print all solutions, and look for a structure that generalises: alternating, sorted then reversed, pairs swapped, a spiral.
- Build inductively: from a solution for n, get one for $n+1$ or $n+2$ by a local change.
- Look for invariants that rule out some n (parity, sums modulo something); the impossible cases often show the construction for the rest.
- Extremal choices: put the largest element first, connect everything to one vertex, use a star or a path.

CHAPTER 30

Contest Craft

Knowing algorithms wins only if you can apply them under time pressure. This chapter collects the practical knowledge that turns knowledge into accepted solutions: how to spend contest time, how to debug, and a large table for recognising which technique a problem wants.

30.1 During a contest

BRONZE

- **Read all problems early** (or at least the first few). Problems are not always sorted by difficulty for you.
- **Solve on paper first.** Code only when you can state the algorithm and its complexity.
- **Test before submitting**: samples, the smallest input, the largest input (generate one and time it), and the edge cases you listed while reading.
- **Timebox.** If you have been stuck for 20–30 minutes without a new idea, switch problems and come back.
- **Penalty matters.** On Codeforces and ICPC wrong submissions cost time; a two-minute local stress test is cheaper than a wrong attempt.
- **After the contest, upsolve**: read the editorial for every problem you could not solve and implement it. This is where most improvement happens.

30.2 Debugging

BRONZE

1. **Re-read the statement.** Most wrong answers are misreadings: 1-indexed vs 0-indexed, "at most" vs "exactly", the output format, multiple test cases.
2. **Check the extremes**: $n = 1$, all elements equal, the maximum values (overflow), negative numbers, empty answers.
3. **Print intermediate values** with `dbg` and compare with a hand computation on the sample.
4. **Stress test** against a brute force (GUIDE.pdf, Section 4.4). This finds the bug in almost every case where the samples pass.
5. **Read your code as if someone else wrote it**, line by line, asking what each variable holds.

Verdict	Usual causes
Wrong answer	overflow (`int` products, `1 << 40`); uninitialised or unreset variables; off-by-one ranges; `-1 % m` is negative; comparing doubles with `==`; wrong output format (YES/Yes, missing newline between tests); a greedy that is not correct; reading input in the wrong order
Time limit	wrong complexity; `endl` or slow I/O; `map` where an array works; copying vectors into functions; clearing big arrays per test; a stray $O(n)$ operation inside a loop (`v.erase(v.begin())`, `s = s + c`, `count` on a vector); anti-hash tests on `unordered_map`; an infinite loop (a binary search that does not shrink)
Runtime error	out of bounds (the checked STL tells you where); stack overflow (deep recursion, big local arrays); division by zero; dereferencing `end()` or `begin()` of an empty set; an invalid comparator in `sort`
Memory limit	`long long` everywhere (your template); `vector<vector<...>` with millions of small rows; storing all states of a DP when only the last layer is needed; memoization in a `map`

30.3 Recognising the technique

SILVER

The table below maps signals in a statement to the techniques that usually solve them. It is a starting point for the "when you are stuck" checklist of Chapter 1, not a rule.

If you see…	Think of…	Ch.
$n \le 20$; choose a subset	bitmask brute force or bitmask DP	6, 19
$n \le 40$; subset sums	meet in the middle	6
$n \le 500$	$O(n^3)$: Floyd–Warshall, interval DP	12, 19
$n \le 5000$	$O(n^2)$ DP over pairs	18
$n \le 2 \cdot 10^5$	sort, binary search, heap, segment tree	7–22
values up to 10^{18}	math, fast exponentiation, digit DP, $O(\log)$ search	24, 19
"minimise the maximum", "maximise the minimum"	binary search on the answer	8
"the k-th smallest" of something implicit	binary search on the value, count $\le$ it	8
function rises then falls; convex cost	ternary search, convexity	8
sum or count over subarrays	prefix sums (+ hash map)	9
many range additions, read once at the end	difference array	9
longest/shortest subarray with a monotone condition	two pointers	9
window minimum/maximum	monotonic deque	9
"next greater", "nearest smaller", histograms	monotonic stack	9
sum over all subarrays of min/max	contribution technique + monotonic stack	9
XOR of pairs / subarrays	per-bit counting, prefix XOR, binary trie	10
maximum XOR of a subset	XOR basis	26
grid with walls, "fewest moves"	BFS on cells (0-1 BFS if some moves are free)	11, 12
connected regions in a grid	flood fill	11
weighted shortest path, non-negative	Dijkstra	12
negative weights, "arbitrarily small"	Bellman–Ford	12

If you see…	Think of…	Ch.
all pairs, $n \le 500$	Floyd–Warshall	12
merging groups, "are a and b connected"	DSU	13
connect everything at minimum cost	MST (Kruskal)	13
dependencies, "order consistent with"	topological sort, DP on DAG	13
every vertex has one outgoing edge	functional graph, cycle + tails, binary lifting	11
mutual reachability, cycles in a directed graph	SCC, condensation	14
two options per item with pairwise constraints	2-SAT	14
edges/vertices whose removal disconnects	bridges, articulation points	14
use every edge exactly once	Euler path (Hierholzer)	14
edge insertions and deletions, offline	segment tree on time + DSU rollback	14
pairing two groups; disjoint paths; minimum cut	matching, max flow	15
profits with prerequisites	project selection (min cut)	15
tree, "for every vertex" answer	rerooting DP	16
tree, path between u and v	LCA, binary lifting	16
subtree queries	Euler tour + Fenwick/segment tree	16
path queries with updates	HLD	17
count paths with a property in a tree	centroid decomposition	17
many queries on small vertex subsets	virtual tree	17
distinct values per subtree	small-to-large merging	16
range sums with point updates	Fenwick tree	22
range min / max with updates	segment tree	22
range updates and range queries	lazy segment tree	22
static range min, gcd	sparse table	21
offline range statistics (distinct, mode)	Mo's algorithm, sweep + Fenwick	21, 22
k-th smallest in a range	persistent segment tree, wavelet tree	22, 23
insert/delete/reverse in the middle of a sequence	implicit treap	23
counting ways modulo a prime	combinatorics, DP with `mint`	25, 18
"at least one", forbidden conditions	inclusion–exclusion	25
coprime pairs, gcd conditions	Möbius inversion, totient	24
$\sum f(i)\lfloor n/i \rfloor$, n up to 10^{12}	floor-division blocks	24
two players, perfect information	W/L DP, Grundy numbers, Nim	25
linear recurrence, n up to 10^{18}	matrix exponentiation	19
counting pairs with a given sum, many values	FFT / NTT convolution	26
equal substrings, palindromes	hashing, Manacher	27
pattern occurrences	KMP, Z-function; many patterns: Aho–Corasick	27
distinct substrings, suffix comparisons	suffix array, suffix automaton	27
points, polygons, turns	cross product, convex hull	28
overlapping intervals / rectangles	sweep line	28

If you see…	Think of…	Ch.
$dp_i = \min_j(a_j x_i + b_j)$	convex hull trick	20
split into k groups, cost satisfies quadrangle inequality	divide & conquer / Knuth optimization	20
"exactly k" with concave answer	Aliens trick	20
answer shared by at least half the input	randomized sampling	29

30.4 Numbers worth remembering

BRONZE

Quantity	Value
`int` max / `long long` max	$2\,147\,483\,647 \approx 2.1 \cdot 10^9$ / $9.22 \cdot 10^{18}$
$2^{10}, 2^{20}, 2^{30}, 2^{60}$	$\approx 10^3, 10^6, 10^9, 10^{18}$
10!, 12!, 13!, 20!	$3.6 \cdot 10^6, 4.8 \cdot 10^8, 6.2 \cdot 10^9, 2.4 \cdot 10^{18}$
$\binom{20}{10}, \binom{30}{15}, \binom{60}{30}$	$184\,756, 1.55 \cdot 10^8, 1.18 \cdot 10^{17}$
max divisors for $n \le 10^6/10^9/10^{18}$	240 / 1 344 / 103 680
primes below 10^6 / 10^7	78 498 / 664 579
$\sum_{i=1}^{n} \frac{n}{i}$	$\approx n \ln n$ ($\approx 1.4 \cdot 10^7$ for $n = 10^6$)
Catalan C_{10}, C_{15}, C_{19}	16 796, 9 694 845, 1 767 263 190
operations per second	$\approx 10^8$–$5 \cdot 10^8$ simple operations

30.5 How to practise

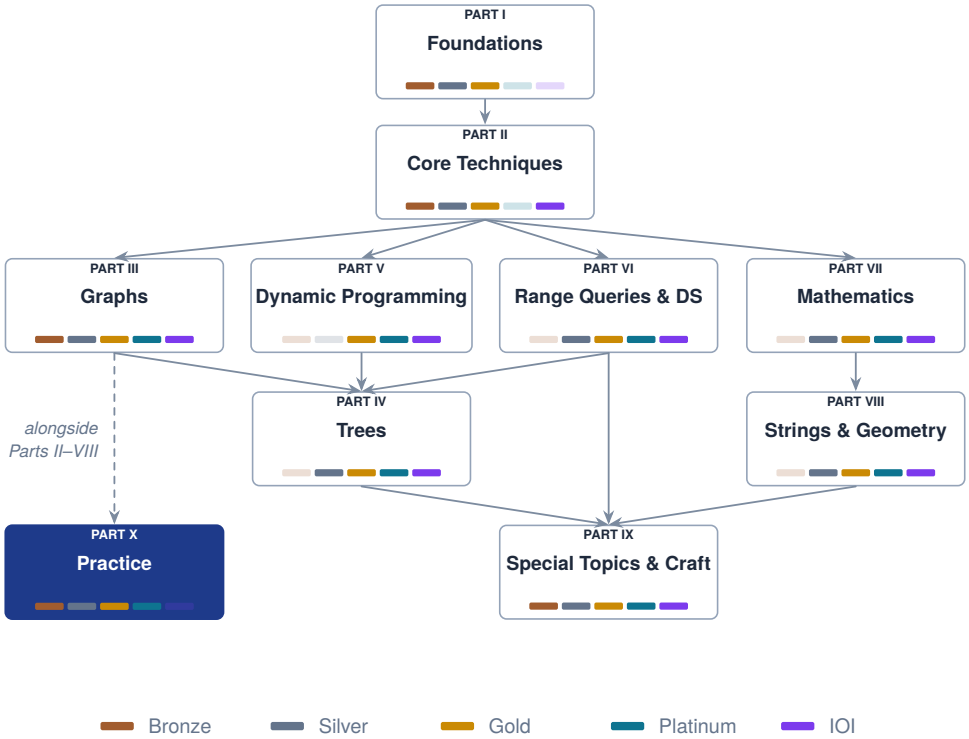
BRONZE

- **Train slightly above your level.** Problems you solve in 5 minutes teach little; problems you cannot solve after an hour, with the editorial, teach the most.
- **Go topic by topic, then mix.** Learn a technique with a focused set (the USACO Guide modules, CSES problem set sections), then do mixed contests so you learn to *recognise* it without being told.
- **Do virtual contests** with a timer; the pressure is part of what you are training.
- **Keep a log** of every bug that cost you a submission and every idea you learned from an editorial. Re-read it before contests: the same mistakes repeat.
- **Implement from memory.** After a template is understood, write it once without looking; knowing why it works is what lets you adapt it when a problem needs a variation.

PART X

Practice

Classic problems solved end to end, from reading the limits to tested code. Use them alongside Parts II–VIII.



CHAPTER 31

Worked Problems

This chapter puts the book to work. It solves seventeen classic problems from the CSES Problem Set (cses.fi/problemset) all the way from the statement to accepted code, and it shows the *thinking* that leads to each solution, not just the answer. The problems follow the order of the book: sorting and greedy, binary search, prefix sums, graphs, dynamic programming, trees, range queries, strings and binary lifting.

> **Get the most out of this chapter** Open the problem on CSES, send it to VS Code with Competitive Companion (CPH creates the file from your template) and try it for twenty minutes *before* reading on. If you are stuck, read only the "Find the idea" step and try again. Solving it yourself is what makes a technique stick; reading a solution only makes it look familiar.

31.1 The method

BRONZE

Every solution below is found with the same six steps. They are worth making a habit: most wrong answers in contests come from skipping step 2 (the solution was never fast enough) or step 6 (it was never tested on a hard case).

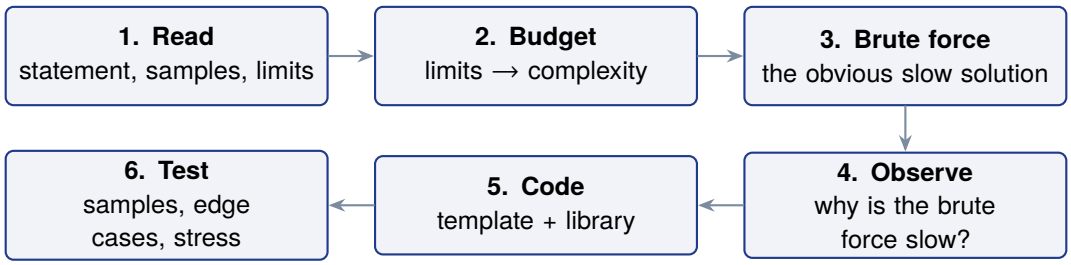


Figure 31.1: The six steps used for every problem in this chapter.

1. **Read** the statement twice. Note what is given, what is asked, and every limit.
2. **Budget.** A judge does roughly 10^8 simple operations per second. Table 3.1 turns the limits into the complexity you need: with $n \le 2 \cdot 10^5$, $O(n \log n)$ is fine and $O(n^2) = 4 \cdot 10^{10}$ is hopeless.
3. **Brute force.** Write down the simplest correct solution, even if it is far too slow. It fixes what you are computing, and later it is the reference for a stress test.
4. **Observe.** Ask what the brute force repeats or wastes. The answer is almost always one of the techniques in this book, and the "How to recognize it" boxes are there to help you spot which.
5. **Code** it with your template and, where one exists, a library template pasted with its shortcut.
6. **Test**: run the CPH samples, then the extremes (smallest input, largest input, all equal, negative numbers), then a stress test against the brute force if anything is uncertain.

About the code. Each solution is the body of `solve()` in your template: it replaces the empty `void solve() {}`. Remember what the template gives you: `int` is 64-bit (`#define int long long`), `rep(i, n)` loops $i = 0, \ldots, n-1$ and `rep(i, a, b)` loops $i = a, \ldots, b-1$, `per` loops backwards, `cin >> v` reads a whole vector, `print(a, b)` prints its arguments separated by spaces and ends the line (a vector prints as its elements), `chmax(a, b)` sets $a = \max(a, b)$, and `INF`, `MOD`, `vi`, `vvi`, `pii`, `pll`, `minpq` are ready to use. Where a library template is needed (DSU, Fenwick tree) it is pasted above `solve()` with its shortcut.

Every program here is tested Each solution in this chapter was compiled with your exact template and compiler flags and compared with a brute-force solution on 150 random inputs (the same kind of stress test as Section 4.4 of GUIDE.pdf). The largest cases were also timed with the judge's flags: every solution runs in well under 0.25 s. Statements are paraphrased and limits rounded; always read the original.

#	Problem	Technique	Chapter	Page
1	Maximum Subarray Sum	Kadane's algorithm (a one-variable DP)	18	336
2	Apartments	sorting + two pointers, greedy matching	7	338
3	Movie Festival	greedy by earliest end (exchange argument)	7	339
4	Factory Machines	binary search on the answer	8	340
5	Subarray Sums II	prefix sums + counting map	9	342
6	Counting Rooms	flood fill, connected components	11	343
7	Message Route	BFS + parent pointers	11	345
8	Shortest Routes I	Dijkstra	12	346
9	Road Construction	DSU (union–find)	13	348
10	Coin Combinations II	counting DP (unbounded knapsack)	18	349
11	Book Shop	0/1 knapsack	18	350
12	Increasing Subsequence	LIS in $O(n \log n)$	18	351
13	Subordinates	tree DP, children before parents	16	353
14	Tree Diameter	two BFS passes	16	354
15	Dynamic Range Sum Queries	Fenwick tree	22	356
16	String Matching	prefix function (KMP)	27	357
17	Planets Queries I	binary lifting on a functional graph	11	358

31.2 Maximum subarray sum

SILVER

Maximum Subarray Sum

CSES Problem Set

Given an array of n integers, find the largest sum of a contiguous, non-empty subarray. Limits: $n \le 2 \cdot 10^5$, $|a_i| \le 10^9$.

SAMPLE INPUT

```
8
-1 3 -2 5 3 -5 2 2
```

SAMPLE OUTPUT

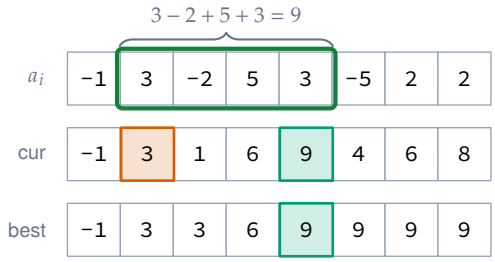
```
9
```

› Read the limits. $n \le 2 \cdot 10^5$ allows $O(n)$ or $O(n \log n)$. Trying every pair (l, r) is $O(n^2) = 4 \cdot 10^{10}$ even with prefix sums. The answer can reach $2 \cdot 10^5 \cdot 10^9 = 2 \cdot 10^{14}$, so it needs 64 bits (your template's `int` already is).

› Find the idea. Ask a smaller question: what is the best subarray that *ends* at position i? Call its sum cur_i. Such a subarray is either a_i alone, or a_i appended to the best subarray ending at $i-1$:

$$\text{cur}_i = \max(\text{cur}_{i-1} + a_i,\ a_i).$$

The answer is the largest cur_i. This is a dynamic programming whose whole state is one number, known as **Kadane's algorithm**. In words: keep a running sum, and start over whenever the sum so far is negative, because a negative prefix can only make what follows smaller.



cur is the best sum ending at i. Orange: the old sum was negative, so the subarray restarts at a_i. Green: the best value, reached at the end of the framed subarray.

Figure 31.2: Kadane's algorithm on the sample.

› Code.

```
void solve() {
    int n;
    cin >> n;
    ll best = LLONG_MIN, cur = 0;
    rep(i, n) {
        ll x;
        cin >> x;
        cur = max(cur + x, x);  // best sum of a subarray that ends here
        chmax(best, cur);
    }
    print(best);
}
```

› Complexity. $O(n)$ time and $O(1)$ extra memory: the array is never stored.

⚠ Starting from zero Initialising `best = 0` is wrong when every number is negative: for $[-3, -1]$ the answer is -1, because the subarray must be non-empty. Start from `LLONG_MIN` (or from a_0).

Variants you will meet

- **Length at least k**: with prefix sums, the answer is $\max_r \left(P_r - \min_{l \le r-k} P_l\right)$; keep the running minimum.
- **Circular array**: the best wrapping subarray is the total minus the *minimum* subarray. Take the larger of that and the ordinary answer, except when all numbers are negative (then the "wrap" would be empty).

- **Maximum-sum rectangle** in an $n \times m$ matrix: fix the top and bottom rows, add the columns between them into one array, and run Kadane on it: $O(n^2 m)$.

31.3 Apartments

SILVER

Apartments CSES Problem Set

There are n applicants and m apartments. Applicant i wants an apartment of size a_i and accepts any size in $[a_i - k, a_i + k]$. Each apartment can go to at most one applicant. How many applicants can get an apartment? Limits: $n, m \le 2 \cdot 10^5$; sizes and k up to 10^9.

SAMPLE INPUT	SAMPLE OUTPUT

`4 3 5`
`60 45 80 60`
`30 60 75`
	`2`

› Read the limits. Pairing every applicant with every apartment is $4 \cdot 10^{10}$ pairs; general matching is much worse. We need sorting and a single sweep: $O(n \log n + m \log m)$.

› Find the idea. Sort both lists and look at the smallest apartment b_j and the smallest applicant a_i:

- If $b_j < a_i - k$, the apartment is too small for applicant i, and for everyone after i too (they want even more). Nobody can use it: skip the apartment.
- If $b_j > a_i + k$, the apartment is too big for i, and every remaining apartment is even bigger. Applicant i can never be served: skip the applicant.
- Otherwise b_j fits a_i: match them. Giving the smallest usable apartment to the smallest applicant never hurts, because any later applicant who could use b_j could also use the larger apartments that a_i would otherwise take (an exchange argument, Section 1.5).

Every step discards an applicant or an apartment, so there are at most $n + m$ steps: this is the **two pointers** technique.

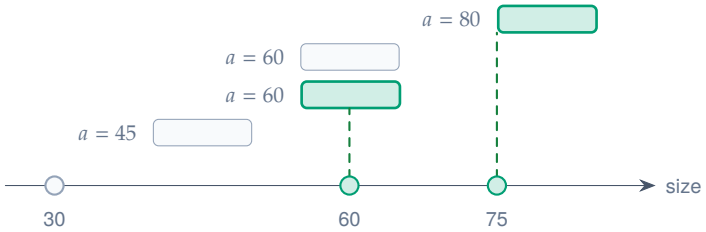


Figure 31.3: Sorted applicants (bars: acceptable sizes $[a_i - k, a_i + k]$ with $k = 5$) and apartments (dots). The sweep matches 60 with the first applicant who wants 60 and 75 with the applicant who wants 80; apartment 30 fits nobody.

› Code.

```
void solve() {
    int n, m, k;
    cin >> n >> m >> k;
    vi a(n), b(m);
    cin >> a >> b;
    sort(all(a)), sort(all(b));
    int ans = 0;
    for (int i = 0, j = 0; i < n && j < m;) {
        if (b[j] < a[i] - k) j++;       // too small for everyone still waiting
        else if (b[j] > a[i] + k) i++;  // no apartment left fits applicant i
        else ans++, i++, j++;
    }
    print(ans);
}
```

› **Complexity.** $O(n \log n + m \log m)$ for sorting; the sweep is $O(n + m)$.

> **How to recognize it** Two sets must be paired, and whether x and y can be paired depends on comparing their values (a threshold or a window). Sort both sides and sweep them together; the greedy match of "smallest with smallest" is usually optimal.

</> Template: Two pointers (sliding window) Ctrl+Alt+T, 5 prefix `cptwoptr`

31.4 Movie festival

SILVER

Movie Festival CSES Problem Set

There are n movies; movie i runs from time a_i to b_i ($a_i < b_i$). You can watch one movie at a time and only from start to end; a movie may start exactly when the previous one ends. What is the largest number of whole movies you can watch? Limits: $n \le 2 \cdot 10^5$, times up to 10^9.

SAMPLE INPUT	SAMPLE OUTPUT
3 3 5 4 9 5 8	2

› **Read the limits.** Trying all 2^n subsets works for $n \le 20$; here n is $2 \cdot 10^5$, so we need a greedy rule and $O(n \log n)$.

› **Find the idea.** Which movie should you watch first? The one that *ends* earliest, because it leaves the most time for everything else. Proof by exchange: take any optimal schedule and replace its first movie by the movie that ends earliest overall. That movie ends no later, so it does not clash with the rest of the schedule, and the count is the same. So some optimal schedule starts with the earliest-ending movie; apply the same argument to the movies that start after it ends. In code: sort by end time and take every movie that starts no earlier than the end of the last one taken.

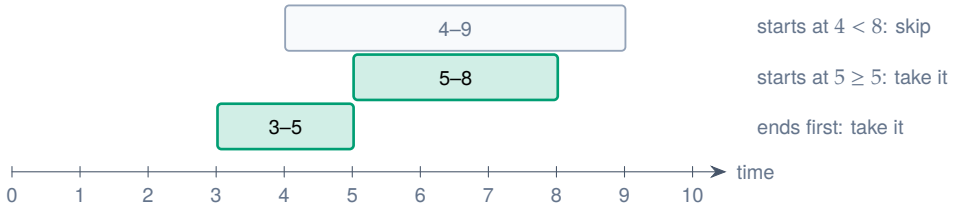


Figure 31.4: Movies sorted by end time; the greedy takes the green ones.

› Code.

```
void solve() {
    int n;
    cin >> n;
    vector<pii> mv(n);
    for (auto& [a, b] : mv) cin >> a >> b;
    sort(all(mv), [](const pii& x, const pii& y) { return x.second < y.second; });
    int cnt = 0, free_at = 0;  // the time the last chosen movie ends
    for (auto [a, b] : mv)
        if (a >= free_at) cnt++, free_at = b;
    print(cnt);
}
```

› Complexity. $O(n \log n)$ for the sort, then one pass.

⚠ Plausible rules that are wrong

- *Earliest start first*: for $[1, 10], [2, 3], [4, 5]$ it takes $[1, 10]$ and nothing else, but the answer is 2.
- *Shortest first*: for $[1, 5], [4, 7], [6, 10]$ it takes $[4, 7]$, which blocks both others; the answer is 2.

Whenever you invent a greedy rule, look for a small counterexample (or stress test it) before coding it.

</> Template: Intervals: merge + max disjoint Ctrl+Alt+T, Shift+N prefix `cpintervals`

31.5 Factory machines

SILVER

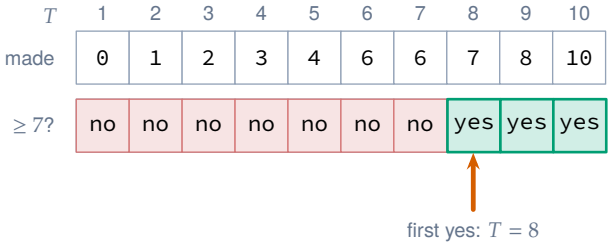
Factory Machines CSES Problem Set

A factory has n machines working at the same time; machine i makes one product every k_i seconds. What is the shortest time in which the machines can make t products together? Limits: $n \le 2 \cdot 10^5$; $t, k_i \le 10^9$.

SAMPLE INPUT	SAMPLE OUTPUT
3 7 3 2 5	8

› Read the limits. The answer can be as large as 10^{18} (one machine with $k = t = 10^9$), so trying times one second at a time is out of the question.

› Find the idea. Turn "find the minimum time" into a yes/no question: *can the machines make t products within time T?* That is easy: machine i makes $\lfloor T/k_i \rfloor$ products, so add these up and compare with t, in $O(n)$. The answer is also *monotone*: if time T is enough, so is $T+1$. A monotone yes/no question means **binary search on the answer** (Chapter 8): find the first T that says yes. The search range is $[1, \min_i k_i \cdot t]$, because the fastest machine alone makes t products in that time.



made$(T) = \sum_i \lfloor T/k_i \rfloor$ for $k = (3,2,5)$. The answers form a block of "no" followed by a block of "yes", so binary search finds the boundary.

Figure 31.5: Products made by time T in the sample, and the monotone yes/no question.

› Code.

```
void solve() {
    int n, t;
    cin >> n >> t;
    vi k(n);
    cin >> k;
    auto ok = [&](ll T) {  // can the machines make t products within time T?
        ll made = 0;
        for (ll x : k)
            if ((made += T / x) >= t) return true;
        return false;
    };
    ll lo = 1, hi = *min_element(all(k)) * t;  // the fastest machine alone is enough
    while (lo < hi) {
        ll mid = lo + (hi - lo) / 2;
        if (ok(mid)) hi = mid;
        else lo = mid + 1;
    }
    print(lo);
}
```

› Complexity. $O(n \log(\min k \cdot t))$: about 60 checks of $O(n)$ each, roughly $1.2 \cdot 10^7$ operations.

⚠ Overflow inside the check Summing $\lfloor T/k_i \rfloor$ over all machines can reach $2 \cdot 10^5 \cdot 10^{18}$, far beyond 64 bits. The check stops as soon as the count reaches t: before each addition the count is below $t \le 10^9$, so it never exceeds about $10^{18} + 10^9$, which fits. Also write the midpoint as `lo + (hi - lo) / 2`.

</> Template: Binary search on the answer Ctrl+Alt+T, A prefix `cpbsearch`

31.6 Subarray sums

SILVER

Subarray Sums II CSES Problem Set

Given n integers (which may be negative) and a target x, count the contiguous subarrays whose sum is exactly x. Limits: $n \le 2 \cdot 10^5$, $|a_i|, |x| \le 10^9$.

SAMPLE INPUT	SAMPLE OUTPUT
5 7 2 -1 3 5 -2	2

› Read the limits. All $O(n^2)$ pairs are too many. With only positive numbers a sliding window would work (the sum grows as the window grows), but negative numbers break that: this is exactly what separates "Subarray Sums II" from "Subarray Sums I".

› Find the idea. With prefix sums $P_0 = 0$ and $P_r = a_1 + \cdots + a_r$, the sum of $a_{l+1}, \ldots, a_r$ is $P_r - P_l$ (Chapter 9). So we count pairs $l < r$ with $P_l = P_r - x$. Sweep r from left to right and keep, in a map, how many earlier prefixes had each value: at each r, add the number of earlier prefixes equal to $P_r - x$, then record P_r.

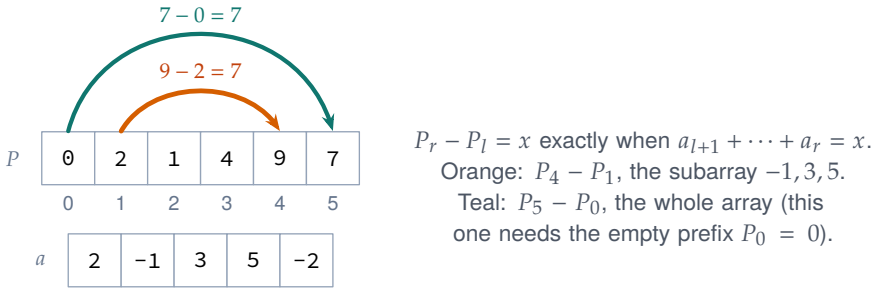


Figure 31.6: The two subarrays with sum 7 in the sample, seen as pairs of prefix sums that differ by 7.

› Code.

```
void solve() {
    int n;
    ll x;
    cin >> n >> x;
    map<ll, int> seen;  // how many prefixes had each sum
    seen[0] = 1;
    ll pre = 0, ans = 0;
    rep(i, n) {
        ll a;
        cin >> a;
        pre += a;
        auto it = seen.find(pre - x);  // earlier prefix P with pre - P = x
        if (it != seen.end()) ans += it->second;
        seen[pre]++;
    }
    print(ans);
}
```

› Complexity. $O(n \log n)$ with `map`. The answer can reach $n(n+1)/2 \approx 2 \cdot 10^{10}$, which needs 64 bits.

> **The empty prefix** `seen[0] = 1` stands for P_0, the sum of nothing. Without it every subarray that starts at the first element is missed; in the sample the answer would be 1.

> **Tip** `unordered_map` is faster on average, but on Codeforces it can be attacked with inputs that make every key collide; use `map`, or a hash map with a randomized hash (Chapter 5).

</> Template: Prefix sums (1D) Ctrl+Alt+T, 1 prefix `cpprefix`

31.7 Counting rooms

SILVER

Counting Rooms CSES Problem Set

A building map has n rows and m columns; each cell is floor (`.`) or wall (`#`). You can walk left, right, up and down through floor cells. Count the rooms. Limits: $n, m \le 1000$.

SAMPLE INPUT

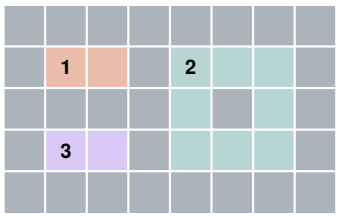
```
5 8
########
#..#...#
####.#.#
#..#...#
########
```

SAMPLE OUTPUT

```
3
```

› Read the limits. Up to 10^6 cells, so every cell may be touched only a constant number of times: $O(nm)$.

› Find the idea. A grid is a graph in disguise: every floor cell is a vertex, and neighbouring floor cells are joined by an edge. A room is a **connected component**. Scan the cells row by row; the first time you meet an unvisited floor cell you have found a new room, so count it and **flood fill** it (a BFS that marks every cell it reaches), so that none of its cells starts another count. Overwriting visited cells with `#` saves a separate visited array.



Walls are grey. Scanning row by row, the numbered cell is the first free cell met in each room. Each one starts a flood fill that paints its whole room, so later cells of that room are skipped and each room is counted exactly once.

Figure 31.7: The three rooms of the sample.

› Code.

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<string> g(n);
    cin >> g;
    const int dr[4] = {-1, 1, 0, 0}, dc[4] = {0, 0, -1, 1};
    int rooms = 0;
    rep(sr, n) rep(sc, m) {
        if (g[sr][sc] != '.') continue;
        rooms++;  // a new room: flood it so it is not counted again
        queue<pii> q;
        q.push({sr, sc});
        g[sr][sc] = '#';
        while (!q.empty()) {
            auto [r, c] = q.front();
            q.pop();
            rep(d, 4) {
                int nr = r + dr[d], nc = c + dc[d];
                bool inside = 0 <= nr && nr < n && 0 <= nc && nc < m;
                if (!inside || g[nr][nc] != '.') continue;
                g[nr][nc] = '#';
                q.push({nr, nc});
            }
        }
    }
    print(rooms);
}
```

› **Complexity.** $O(nm)$: every cell enters the queue at most once and looks at four neighbours.

> ⚠ **Recursive flood fill** A recursive DFS can go 10^6 calls deep on a 1000×1000 grid. Your local build has a 256 MB stack, so it passes on your machine, but on judges with the usual 1–8 MB stack it crashes. The queue-based BFS above has no such limit.

</> **Template:** Flood fill (grid regions) Ctrl+Alt+T, F prefix `cpflood`

31.8 Message route

SILVER

Message Route CSES Problem Set

A network has n computers and m two-way connections. Find a route from computer 1 to computer n that passes through as few computers as possible: print their number and the route, or `IMPOSSIBLE`. Limits: $n \le 10^5$, $m \le 2 \cdot 10^5$.

SAMPLE INPUT

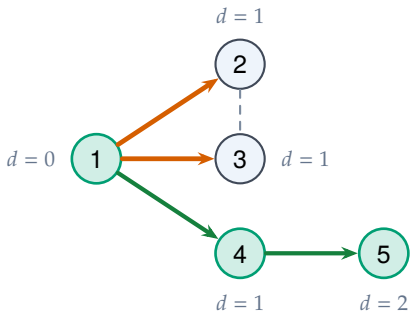
```
5 5
1 2
1 3
1 4
2 3
5 4
```

SAMPLE OUTPUT

```
3
1 4 5
```

› Read the limits. $O(n+m)$ or close to it.

› Find the idea. All connections cost the same, so the fewest computers means the fewest edges: an unweighted shortest path, which **BFS** finds (Chapter 11). BFS discovers vertices in order of distance. To print the route, remember for every vertex the vertex it was discovered from, `par[v]`. Following `par` back from n walks the shortest path backwards; reverse it.



Arrows point from `par[v]` to v: the edge through which v was discovered. Walking back from 5 gives $5, 4, 1$; reversed, the route is 1 4 5. The dashed edge joins two vertices of the same layer and is never used.

Figure 31.8: BFS from computer 1 on the sample, with parent pointers.

› Code.

```
void solve() {
    int n, m;
    cin >> n >> m;
    vvi adj(n);
    rep(i, m) {
        int u, v;
        cin >> u >> v;
        u--, v--;
        adj[u].push_back(v), adj[v].push_back(u);
    }
    vi dist(n, -1), par(n, -1);
    queue<int> q;
    q.push(0), dist[0] = 0;
    while (!q.empty()) {
```

```
        int u = q.front();
        q.pop();
        for (int v : adj[u])
            if (dist[v] == -1) dist[v] = dist[u] + 1, par[v] = u, q.push(v);
    }
    if (dist[n - 1] == -1) return print("IMPOSSIBLE");
    vi path;
    for (int v = n - 1; v != -1; v = par[v]) path.push_back(v + 1);
    reverse(all(path));
    print(sz(path));
    print(path);
}
```

› **Complexity.** $O(n + m)$.

Pitfall

- DFS does *not* find shortest paths; only BFS (or Dijkstra with weights) does.
- Mark a vertex when it is *pushed*, not when it is popped. Otherwise a vertex can enter the queue once per neighbour, and the queue can grow to $O(m)$ entries with repeated work.

Tip `return print("IMPOSSIBLE");` prints and leaves `solve()` in one line. It is legal because both functions return `void`; it saves an `else` and a level of indentation.

</> Template: BFS (shortest path, unweighted) Ctrl+Alt+T, B prefix `cpbfs`

31.9 Shortest routes

GOLD

Shortest Routes I CSES Problem Set

There are n cities and m one-way flights; each flight has a price. Print the cheapest total price from city 1 to every city (all cities can be reached). Limits: $n \le 10^5$, $m \le 2 \cdot 10^5$, prices up to 10^9.

SAMPLE INPUT

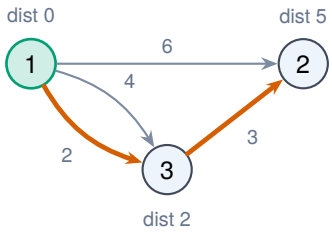
```
3 4
1 2 6
1 3 2
3 2 3
1 3 4
```

SAMPLE OUTPUT

```
0 5 2
```

› **Read the limits.** Bellman–Ford's $O(nm) = 2 \cdot 10^{10}$ is too slow. Weights are non-negative, so Dijkstra in $O((n + m) \log m)$ applies (Chapter 12).

› **Find the idea.** Always settle the unsettled city with the smallest known distance: with non-negative weights no later path can make it cheaper. A min-priority queue of (distance, city) pairs gives that city in $O(\log m)$. When a flight improves a distance, push the new pair and leave the old one in the queue; when an outdated pair is popped (`d != dist[u]`), skip it. This *lazy deletion* is simpler than updating entries inside the heap.



Step	Pop	What happens
1	$(0, 1)$	dist[2] = 6, dist[3] = 2; the second flight $1 \to 3$ (price 4) is no better
2	$(2, 3)$	$2+3 = 5 < 6$: dist[2] = 5, push $(5, 2)$
3	$(5, 2)$	city 2 has no flights out
4	$(6, 2)$	outdated: $6 \neq$ dist[2] = 5, skipped

Figure 31.9: Dijkstra on the sample. Orange: the cheapest routes.

› Code.

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<vector<pll>> adj(n);
    rep(i, m) {
        int a, b;
        ll c;
        cin >> a >> b >> c;
        adj[a - 1].push_back({b - 1, c});  // one-way flight
    }
    vector<ll> dist(n, INF);
    minpq<pll> pq;
    dist[0] = 0, pq.push({0, 0});
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (d != dist[u]) continue;
        for (auto [v, w] : adj[u])
            if (d + w < dist[v]) dist[v] = d + w, pq.push({dist[v], v});
    }
    print(dist);
}
```

› Complexity.

$O((n + m) \log m)$; 0.14 s on a random graph of the maximum size.

Pitfall

- Distances reach $(n - 1) \cdot 10^9 \approx 10^{14}$: 64-bit integers are required.
- Without the `d != dist[u]` check the answer is still right, but a city can be expanded once per outdated entry, which is quadratic on some graphs (a star with many improving edges).
- With *negative* weights Dijkstra is wrong; use Bellman–Ford or SPFA with care.

</> Template: Dijkstra Ctrl+Alt+T, J prefix `cpdijkstra`

31.10 Road construction

GOLD

Road Construction CSES Problem Set

There are n cities and no roads. Then m two-way roads are built one by one. After each road, print the number of connected components and the size of the largest one. Limits: $n \le 10^5$, $m \le 2 \cdot 10^5$.

SAMPLE INPUT	SAMPLE OUTPUT
5 3 1 2 1 3 4 5	4 2 3 3 2 3

› Read the limits. A BFS after every road costs $O(m(n+m)) \approx 6 \cdot 10^{10}$. Each road must be handled in (almost) constant time.

› Find the idea. Components only ever *merge*, never split: exactly what a **disjoint set union** (DSU) is made for (Chapter 13). `unite(a, b)` returns **`true`** if it merged two different components; the DSU also keeps the number of components and each component's size. The largest component can only grow, so a running maximum updated after each successful merge is enough.

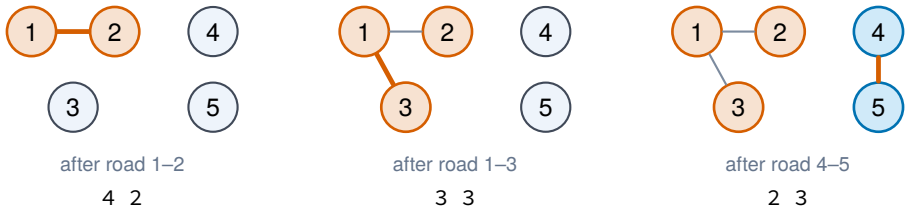


Figure 31.10: Components after each road (one colour per component of size at least 2) and the printed line.

› Code. Paste the DSU template above `solve()` with Ctrl+Alt+T, U, then:

```
void solve() {  // DSU (cpdsu) is pasted above solve()
    int n, m;
    cin >> n >> m;
    DSU d(n);
    int biggest = 1;
    rep(i, m) {
        int a, b;
        cin >> a >> b;
        if (d.unite(a - 1, b - 1)) chmax(biggest, d.size(a - 1));
        print(d.comps, biggest);
    }
}
```

› Complexity. $O(m\,\alpha(n))$, effectively $O(m)$.

Deleting edges: go backwards A DSU cannot split a component. If a problem *removes* roads and asks the same questions, read everything first and process the removals in reverse order as additions, then print the answers in the original order. If additions and removals are mixed, see offline dynamic connectivity (Chapter 13).

Template: DSU (union-find) Ctrl+Alt+T, U prefix cpdsu

31.11 Coin combinations

GOLD

Coin Combinations II CSES Problem Set

Given n distinct coin values and a target x, count the ways to make the sum x when the *order of coins does not matter* ($2+3$ and $3+2$ are the same way). Each coin value can be used any number of times. Print the count modulo 10^9+7. Limits: $n \le 100$, $x \le 10^6$.

SAMPLE INPUT	SAMPLE OUTPUT
3 9 2 3 5	3

› Read the limits. $n \cdot x = 10^8$ simple steps is fine. The number of ways is astronomically large, hence the modulus.

› Find the idea. Let `ways[s]` be the number of ways to make s with the coins processed so far; start with only `ways[0] = 1` (the empty sum). Add the coins one at a time: with coin c available, a way to make s either does not use c at all (the old `ways[s]`), or it is a way to make $s-c$ that may already contain c, plus one more c. Looping s *upwards* makes `ways[s - c]` already include uses of c, so the coin can be used many times. Putting the coins in the *outer* loop means every multiset is built in one fixed order (all 2s, then all 3s, then all 5s), so it is counted exactly once.

s	0	1	2	3	4	5	6	7	8	9
start	1	0	0	0	0	0	0	0	0	0
after coin 2	1	0	**1**	0	**1**	0	**1**	0	**1**	0
after coin 3	1	0	1	**1**	1	**1**	**2**	**1**	**2**	**2**
after coin 5	1	0	1	1	1	**2**	2	**2**	**3**	**3**

Figure 31.11: `ways[s]` after each coin; changed entries are highlighted. For example, after coin 5, `ways[8]` = 2 + `ways[3]` = 3: $\{2,2,2,2\}, \{2,3,3\}$ and $\{3,5\}$. The answer is `ways[9]` = 3: $\{2,2,5\}, \{3,3,3\}, \{2,2,2,3\}$.

› Code.

```
void solve() {
    int n, x;
    cin >> n >> x;
    vi c(n);
    cin >> c;
    vector<ll> ways(x + 1);
    ways[0] = 1;
    for (int coin : c)  // coins in the outer loop: each multiset is counted once
```

```
        for (int s = coin; s <= x; s++)
            if ((ways[s] += ways[s - coin]) >= MOD) ways[s] -= MOD;  // cheaper than %
    print(ways[x]);
}
```

› **Complexity.** $O(n \cdot x)$ time and $O(x)$ memory; 0.21 s at the maximum size.

The order of the loops is the problem Swapping the loops (sums outside, coins inside) counts *ordered* sequences instead: that is the other CSES problem, Coin Combinations I. On this sample it gives 8, not 3, because $\{2, 2, 5\}$ is counted as 2+2+5, 2+5+2 and 5+2+2, and $\{2, 2, 2, 3\}$ four times.

Tip `if ((w += v) >= MOD) w -= MOD;` replaces `w = (w + v) % MOD` whenever both values are already below `MOD`. A comparison is several times cheaper than a 64-bit division, which matters at 10^8 steps. Your local build uses the checked STL (`-D_GLIBCXX_DEBUG`), so this loop runs much slower on your machine than on the judge; GUIDE.pdf, Section 5.4 shows how to time a solution without it.

Template: Knapsack (0/1, unbounded, coin-change ways) Ctrl+Alt+T, Shift+Q prefix cpknap

31.12 Book shop

GOLD

Book Shop CSES Problem Set

A shop sells n books; book i costs h_i and has s_i pages. With a budget of x, buying each book at most once, what is the largest total number of pages? Limits: $n \le 1000$, $x \le 10^5$, prices and pages up to 1000.

SAMPLE INPUT	SAMPLE OUTPUT
4 10 4 8 5 3 5 12 8 1	13

› **Read the limits.** 2^{1000} subsets is impossible, but $n \cdot x = 10^8$ is fine: a DP over (book, budget).

› **Find the idea.** This is the **0/1 knapsack** (Chapter 18). Let `best[c]` be the most pages with total price at most c using the books seen so far. Adding book i either skips it or buys it:

$$\text{best}[c] \leftarrow \max\big(\text{best}[c],\ \text{best}[c - h_i] + s_i\big).$$

Iterate c *downwards*, so that `best[c - price[i]]` still describes the books *before* i and book i is bought at most once. (Upwards is the unbounded version, exactly as in the previous problem.)

budget c	0	1	2	3	4	5	6	7	8	9	10
start	0	0	0	0	0	0	0	0	0	0	0
book 1 (price 4, 5 pages)	0	0	0	0	**5**	**5**	**5**	**5**	**5**	**5**	**5**
book 2 (price 8, 12 pages)	0	0	0	0	5	5	5	5	**12**	**12**	**12**
book 3 (price 5, 8 pages)	0	0	0	0	5	**8**	**8**	**8**	12	**13**	**13**
book 4 (price 3, 1 page)	0	0	0	**1**	5	8	8	8	12	13	**13**

Figure 31.12: `best[c]` after each book. The answer 13 buys books 1 and 3 (price 9).

› Code.

```
void solve() {
    int n, x;
    cin >> n >> x;
    vi price(n), pages(n);
    cin >> price >> pages;
    vector<ll> best(x + 1, 0);  // best[c] = most pages with total price <= c
    rep(i, n)
        for (int c = x; c >= price[i]; c--)  // downwards: every book at most once
            chmax(best[c], best[c - price[i]] + pages[i]);
    print(best[x]);
}
```

› Complexity. $O(n \cdot x)$ time, $O(x)$ memory: one row instead of an $n \times x$ table.

⚠ The greedy by value per price Buying the books with the most pages per unit of price is wrong. With budget 10 and books (price 6, 7 pages), (5, 5), (5, 5): the best ratio takes the first book and then nothing fits (7 pages), but the two others give 10 pages.

</> Template: Knapsack (0/1, unbounded, coin-change ways) Ctrl+Alt+T, Shift+Q prefix `cpknap`

31.13 Longest increasing subsequence

GOLD

Increasing Subsequence CSES Problem Set

Given an array of n integers, find the length of its longest strictly increasing subsequence (the elements need not be adjacent). Limits: $n \le 2 \cdot 10^5$, values up to 10^9.

SAMPLE INPUT	SAMPLE OUTPUT
8 7 3 5 3 6 2 9 8	4

› Read the limits. The textbook DP, $\text{dp}[i] = 1 + \max\{\text{dp}[j] : j < i,\ a_j < a_i\}$, is $O(n^2) = 4 \cdot 10^{10}$. We need $O(n \log n)$.

› Find the idea. Keep `tail[k]`: the smallest possible last element of an increasing subsequence of length $k + 1$ among the elements seen so far. A small last element is always better, since more

future elements can extend it. The array `tail` is strictly increasing, so for each new x binary search finds the first entry $\geq x$ (`lower_bound`):

- if there is none, x extends the longest subsequence: append it;
- otherwise a subsequence of that length can now end with the smaller value x: overwrite the entry.

The length of `tail` at the end is the answer.

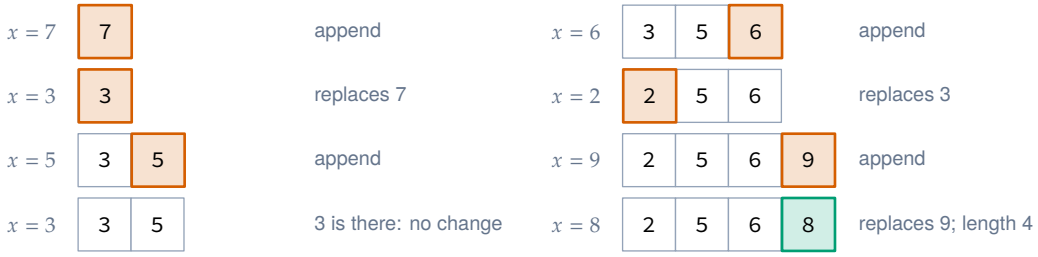


Figure 31.13: `tail` after each element of the sample (read the left column first).

› Code.

```
void solve() {
    int n;
    cin >> n;
    vi a(n);
    cin >> a;
    vi tail;  // tail[k] = smallest end of an increasing subsequence of length k + 1
    for (int x : a) {
        auto it = lower_bound(all(tail), x);
        if (it == tail.end()) tail.push_back(x);
        else *it = x;
    }
    print(sz(tail));
}
```

› Complexity. $O(n \log n)$.

> **⚠ `tail` is not a subsequence** The final `tail` is $2, 5, 6, 8$, but that is not a subsequence of the input (the 2 comes after the 5 and the 6). Only its *length* is meaningful. To output an actual subsequence, remember for each element its predecessor, as the library template does. For the longest *non-decreasing* subsequence use `upper_bound` instead.

</> Template: Longest increasing subsequence Ctrl+Alt+T, Shift+L prefix `cplis`

31.14 Subordinates

GOLD

Subordinates CSES Problem Set

A company has n employees. Employee 1 is the director and every other employee has one direct boss. For every employee, count their subordinates (everyone below them in the hierarchy). Limits: $n \le 2 \cdot 10^5$.

SAMPLE INPUT	SAMPLE OUTPUT
5 1 1 2 3	4 1 1 0 0

› Read the limits. Walking up from every employee to the director is $O(n^2)$ on a chain. We need $O(n)$.

› Find the idea. The hierarchy is a tree rooted at the director. The subordinates of u are its children and their subordinates:

$$\text{sub}[u] = \sum_{c \text{ child of } u} \big(\text{sub}[c] + 1\big).$$

This is a **tree DP** (Chapter 16): every child must be finished before its parent. A recursive DFS does that, but a chain of $2 \cdot 10^5$ employees makes it recurse $2 \cdot 10^5$ deep. The iterative version is just as short: list the vertices in BFS order from the root (parents before children), then process that list backwards.

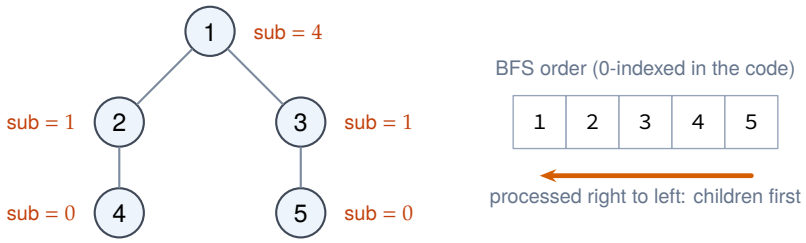


Figure 31.14: The sample hierarchy with its answers, and the order in which the DP visits it.

› Code.

```
void solve() {
    int n;
    cin >> n;
    vvi ch(n);
    rep(i, 1, n) {
        int b;
        cin >> b;
        ch[b - 1].push_back(i);  // employee i's boss is b
    }
    vi order{0};  // BFS order from the director: parents before children
    rep(i, n) for (int c : ch[order[i]]) order.push_back(c);
    vi sub(n, 0);
    per(i, n) {  // children before parents
        int u = order[i];
        for (int c : ch[u]) sub[u] += sub[c] + 1;
```

```
    }
    print(sub);
}
```

› **Complexity.** $O(n)$.

> **Tip** `order` grows while the loop reads it. That is safe with an index, as here, but *not* with a range-based `for` or an iterator: `push_back` may move the vector's storage and leave the iterator dangling.

</> Template: Tree DP (subtree DP, iterative) Ctrl+Alt+T, ; prefix `cptreedp`

31.15 Tree diameter

GOLD

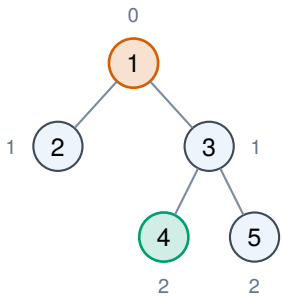
Tree Diameter CSES Problem Set

Given a tree with n nodes, find its diameter: the largest number of edges on a path between two nodes. Limits: $n \le 2 \cdot 10^5$.

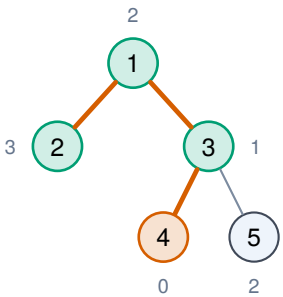
SAMPLE INPUT	SAMPLE OUTPUT
5 1 2 1 3 3 4 3 5	3

› **Read the limits.** A BFS from every node is $O(n^2)$. We need $O(n)$.

› **Find the idea.** Run a BFS from any node; the node `far` that is farthest from it is always one end of some diameter. Run a second BFS from `far`: the largest distance found is the diameter. (Sketch: if a longest path did not end at `far`, you could splice the path to `far` onto it and make it longer; Chapter 16 has the details and the DP alternative.)



BFS from 1: the farthest node is 4 (the first with distance 2)



BFS from 4: the largest distance is 3, along the diameter 4–3–1–2

Figure 31.15: The two BFS passes on the sample; numbers next to the nodes are distances.

› **Code.**

```
void solve() {
    int n;
    cin >> n;
    vvi adj(n);
    rep(i, n - 1) {
        int a, b;
        cin >> a >> b;
        a--, b--;
        adj[a].push_back(b), adj[b].push_back(a);
    }
    auto bfs = [&](int s) {  // distances from s
        vi d(n, -1);
        queue<int> q;
        q.push(s), d[s] = 0;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int v : adj[u])
                if (d[v] < 0) d[v] = d[u] + 1, q.push(v);
        }
        return d;
    };
    vi d0 = bfs(0);
    int far = max_element(all(d0)) - d0.begin();  // one end of a diameter
    vi df = bfs(far);
    print(*max_element(all(df)));
}
```

› **Complexity.** $O(n)$: two BFS passes. With $n = 1$ there are no edges and the answer is 0, which the code handles.

> ⚠ **Pitfall** The two-pass trick needs a *tree* with non-negative edge weights (with weights, use a DFS that adds them up instead of a BFS). On general graphs the longest simple path is NP-hard, and the trick gives wrong answers.

</> **Template:** Tree diameter Ctrl+Alt+T, Shift+D prefix `cpdiameter`

31.16 Range sum queries with updates

GOLD

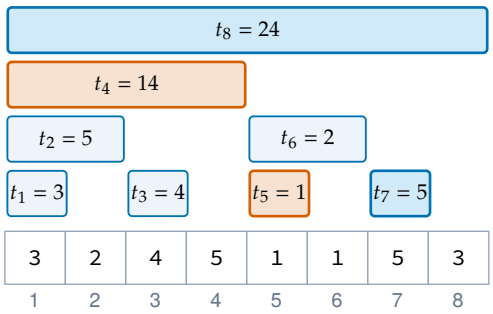
Dynamic Range Sum Queries CSES Problem Set

Given an array of n integers, process q queries: `1 k u` sets $a_k = u$; `2 a b` prints $a_a + \cdots + a_b$ (positions are 1-indexed). Limits: $n, q \le 2 \cdot 10^5$, values up to 10^9.

SAMPLE INPUT	SAMPLE OUTPUT
8 4 3 2 4 5 1 1 5 3 2 1 4 2 5 6 1 3 1 2 1 4	14 2 11

› Read the limits. Prefix sums answer a sum in $O(1)$ but an update changes $O(n)$ of them; a plain array updates in $O(1)$ but sums in $O(n)$. Either way the worst case is $O(nq) = 4 \cdot 10^{10}$. Both operations must be about $O(\log n)$.

› Find the idea. A Fenwick tree (binary indexed tree, Chapter 22) does both in $O(\log n)$. Entry t_i stores the sum of the lowbit(i) positions that end at i, where lowbit(i) is the lowest set bit of i. A prefix sum adds $O(\log n)$ entries, removing the lowest bit each time; an update changes $O(\log n)$ entries, adding the lowest bit each time. The tree supports "add v at i", so "set $a_k = u$" becomes "add $u - a_k$", and we keep the current array to know a_k.



Each bar is one entry t_i and the positions it sums. Orange: prefix(5) $= t_5 + t_4 = 1 + 14 = 15$ (index $5 \to 4 \to 0$). Teal: changing position 7 updates t_7 and t_8 (index $7 \to 8$).

Figure 31.16: A Fenwick tree over the sample array.

› Code. Paste the Fenwick template above `solve()` with Ctrl+Alt+T, I. It is 0-indexed with half-open ranges $[l, r)$, so positions $a..b$ of the statement are `sum(a - 1, b)`:

```
void solve() {  // BIT (cpbit) is pasted above solve()
    int n, q;
    cin >> n >> q;
    vi a(n);
    cin >> a;
    BIT bit(n);
    rep(i, n) bit.add(i, a[i]);
    while (q--) {
        int type, x, y;
        cin >> type >> x >> y;
        if (type == 1) x--, bit.add(x, y - a[x]), a[x] = y;  // set a[x] = y
```

```
        else print(bit.sum(x - 1, y));  // positions x..y, 1-indexed
    }
}
```

› Complexity. $O((n+q)\log n)$.

How to recognize it Point updates and range sums (or any operation that can be undone, like XOR): Fenwick tree. Range minimum or maximum with updates cannot be undone by subtraction: use a segment tree (Chapter 22). No updates at all: plain prefix sums.

</> Template: Fenwick tree / BIT Ctrl+Alt+T, I prefix `cpbit`

31.17 String matching

GOLD

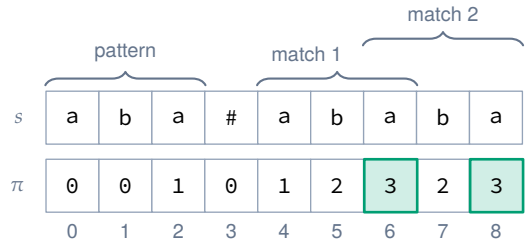
String Matching CSES Problem Set

Given a string and a pattern, count the positions where the pattern occurs in the string (occurrences may overlap). Limits: both lengths up to 10^6.

SAMPLE INPUT	SAMPLE OUTPUT
`saippuakauppias` `pp`	`2`

› Read the limits. Comparing the pattern at every position costs $O(|t| \cdot |p|)$, up to 10^{12} on inputs like `aaa...a` and `aa...ab`. We need $O(|t| + |p|)$.

› Find the idea. The **prefix function** $\pi[i]$ of a string is the length of the longest proper prefix of $s[0..i]$ that is also a suffix of it (Chapter 27). Compute it on $s = p + \# + t$. The separator appears in neither string, so no border can be longer than $|p|$, and $\pi[i] = |p|$ exactly when the pattern ends at position i. Computing π takes $O(|s|)$ in total: j grows by at most one per position and every step of the inner `while` loop shrinks it.



Pattern aba, text ababa. $\pi = 3 = |p|$ (green) marks the end of an occurrence; the two occurrences overlap.

Figure 31.17: The prefix function of pattern # text.

› Code.

```
void solve() {
    string text, pat;
    cin >> text >> pat;
    string s = pat + '#' + text;
    int m = sz(pat), cnt = 0;
    vi pi(sz(s));
    rep(i, 1, sz(s)) {  // prefix function of pat#text
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) j++;
        pi[i] = j;
        cnt += j == m;  // a border of length |pat| ends here: one occurrence
    }
    print(cnt);
}
```

› **Complexity.** $O(|t| + |p|)$ time and memory; 0.05 s at the maximum size.

Pitfall The separator must not occur in either string. CSES strings are lowercase letters, so `#` is safe; for arbitrary bytes, run the prefix function on the pattern alone and match the text against it (the library's `find_occurrences`).

</> Template: KMP (prefix function + pattern search) Ctrl+Alt+T, Shift+K prefix `cpkmp`

31.18 Teleporters: binary lifting

GOLD

Planets Queries I CSES Problem Set

There are n planets and each has one teleporter, which sends you to planet t_i. Answer q queries: starting from planet x and using a teleporter k times, where do you end up? Limits: $n, q \le 2 \cdot 10^5$, $k \le 10^9$.

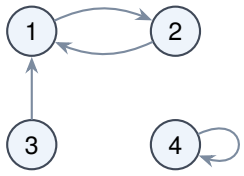
SAMPLE INPUT	SAMPLE OUTPUT
4 3 2 1 1 4 1 2 3 4 4 1	1 2 4

› **Read the limits.** Simulating $k \le 10^9$ steps per query is far too slow. Each query must take about $O(\log k)$.

› **Find the idea.** Every planet has exactly one outgoing edge: a **functional graph** (Chapter 11). Precompute `up[j][v]`, the planet reached from v after 2^j steps. Two jumps of 2^{j-1} make one jump of 2^j:

$$\text{up}[j][v] = \text{up}[j-1]\big[\text{up}[j-1][v]\big].$$

A query writes k in binary and makes one jump for every set bit, at most 30 jumps. This is **binary lifting**, the same table that finds ancestors and LCAs in trees (Chapter 16).



	$v=1$	$v=2$	$v=3$	$v=4$
`up[0][v]` (1 step)	2	1	1	4
`up[1][v]` (2 steps)	1	2	2	4
`up[2][v]` (4 steps)	1	2	**2**	4

Query $(x=3, k=4)$: $4 = 100_2$, so one jump: `up[2][3]` $= 2$.

Figure 31.18: The functional graph of the sample and the first rows of its jump table.

› Code.

```
void solve() {
    int n, q;
    cin >> n >> q;
    const int LOG = 30;  // k < 2^30
    vector<vi> up(LOG, vi(n));  // up[j][v] = planet reached from v after 2^j teleports
    rep(i, n) cin >> up[0][i], up[0][i]--;
    rep(j, 1, LOG) rep(i, n) up[j][i] = up[j - 1][up[j - 1][i]];
    while (q--) {
        int x, k;
        cin >> x >> k;
        x--;
        rep(j, LOG) if (k >> j & 1) x = up[j][x];
        print(x + 1);
    }
}
```

› Complexity. $O(n \log K)$ to build the table and $O(\log K)$ per query, with $K = 10^9$; 0.21 s at the maximum size.

> **Memory** The table holds $30n$ numbers: 48 MB with the template's 64-bit `int`. That is fine for CSES (512 MB) but on a 256 MB problem with bigger n, store it as `vector<vector<int32_t>>` to halve it.

</> Template: K-th successor (binary lifting / jumping) Ctrl+Alt+T, . prefix `cpjump`

31.19 Where to go next

BRONZE

What these problems teach

- **Ask a smaller question**: "best subarray *ending here*" (Kadane), "smallest tail for each length" (LIS), "can it be done in time T?" (binary search on the answer).
- **Change the representation**: subarray sums become pairs of prefix sums, a grid becomes a graph, a hierarchy becomes a tree.
- **Prove the greedy** with an exchange argument, or break it with a small counterexample.
- **Loop order is meaning**: coins outside or inside, budget upwards or downwards.
- **Check every bound**: 64-bit sums, overflow inside a check, recursion depth, memory of a table.

The CSES Problem Set has about 300 problems in the same style, grouped by topic, and it is the best next step after this chapter; the USACO Guide lists problems for every section of this book by level. Solve a few from each topic, and for every problem you could not solve, read the editorial and implement it yourself the same day.

CHAPTER A

Reference Tables

A.1 Complexity of the standard algorithms

Algorithm / structure	Complexity	Library prefix
sorting	$O(n \log n)$	–
binary search / on the answer	$O(\log n)$ / $O(\log R \cdot \text{check})$	`cpbsearch`
prefix sums (1D / 2D)	$O(n)$ build, $O(1)$ query	`cpprefix`, `cpprefix2d`
two pointers, sliding window	$O(n)$	`cptwoptr`, `cpwindow`
monotonic stack	$O(n)$	`cpmonostack`
BFS / DFS / flood fill	$O(n+m)$	`cpbfs`, `cpdfs`, `cpflood`
0-1 BFS	$O(n+m)$	`cp01bfs`
Dijkstra	$O((n+m)\log n)$	`cpdijkstra`
Bellman–Ford	$O(nm)$	`cpbellman`
Floyd–Warshall	$O(n^3)$	`cpfloyd`
topological sort	$O(n+m)$	`cptopo`
DSU	$O(\alpha(n))$ amortized	`cpdsu`
Kruskal / Prim (dense)	$O(m \log m)$ / $O(n^2)$	`cpkruskal`, `cpprim`
SCC, 2-SAT, bridges, BCC	$O(n+m)$	`cpscc`, `cp2sat`, `cpbridges`, `cpbcc`
Euler path	$O(n+m)$	`cpeuler`
Dinic / Hopcroft–Karp	$O(n^2 m)$ / $O(m\sqrt{n})$	`cpdinic`, `cphopcroft`
min-cost flow (SPFA)	$O(F \cdot nm)$ worst case	`cpmcmf`
LCA (binary lifting)	$O(n \log n) + O(\log n)$	`cplca`
HLD path query	$O(\log^2 n)$	`cphld`
centroid decomposition	$O(n \log n)$	`cpcentroid`
Fenwick tree	$O(\log n)$	`cpbit`, `cpbitrange`, `cpbit2d`
segment tree (+ lazy)	$O(\log n)$	`cpseg`, `cplazy`, `cpsegnode`
sparse table	$O(n \log n) + O(1)$	`cpsparse`
Mo's algorithm	$O((n+q)\sqrt{n})$	`cpmo`
persistent segment tree	$O(\log n)$, $O(\log n)$ memory per update	`cppersist`
treap	$O(\log n)$ expected	`cptreap`
LIS	$O(n \log n)$	`cplis`
knapsack	$O(nW)$	`cpknap`
bitmask DP (TSP)	$O(2^n n^2)$	`cptsp`

Algorithm / structure	Complexity	Library prefix
digit DP	$O(\text{digits} \cdot \text{states} \cdot 10)$	`cpdigit`
SOS DP	$O(B\,2^B)$	`cpsos`
D&C / Knuth optimization	$O(kn \log n)$ / $O(n^2)$	`cpdnc, cpknuth`
convex hull trick	$O(\log n)$ per operation	`cplinecontainer, cplichao`
sieve / linear sieve	$O(n \log\log n)$ / $O(n)$	`cpsieve`
trial division / Pollard rho	$O(\sqrt{n})$ / $O(n^{1/4})$	`cpfactor, cppollard`
modular power	$O(\log e)$	`cpmodpow`
matrix power	$O(k^3 \log n)$	`cpmatrix`
FFT / NTT	$O(n \log n)$	`cpfft, cpntt`
KMP / Z / Manacher	$O(n)$	`cpkmp, cpz, cpmanacher`
string hashing	$O(n) + O(1)$	`cpstrhash`
suffix array + LCP	$O(n \log n)$	`cpsa`
suffix automaton, Aho–Corasick	$O(n\Sigma)$	`cpsam, cpaho`
convex hull, closest pair	$O(n \log n)$	`cphull, cpclosest`

A.2 Formula sheet

Quantity	Formula
arithmetic series	$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$, $\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$
geometric series	$\sum_{i=0}^{n-1} r^i = \frac{r^n-1}{r-1}$
binomial	$\binom{n}{k} = \frac{n!}{k!(n-k)!}$, $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$
stars and bars	$\binom{n+k-1}{k-1}$ ways to put n items into k boxes
Catalan	$C_n = \frac{1}{n+1}\binom{2n}{n}$
derangements	$D_n = (n-1)(D_{n-1} + D_{n-2})$
divisors of $\prod p_i^{e_i}$	count $\prod(e_i+1)$, sum $\prod \frac{p_i^{e_i+1}-1}{p_i-1}$
totient	$\varphi(n) = n \prod_{p\mid n}(1 - 1/p)$
Fermat / Euler	$a^{p-1} \equiv 1 \pmod p$, $a^{\varphi(n)} \equiv 1 \pmod n$
Pick	$A = I + B/2 - 1$
shoelace	$2A = \lvert \sum_i (x_i y_{i+1} - x_{i+1} y_i) \rvert$
Euler (planar graphs)	$V - E + F = 2$
distance	Manhattan $\lvert dx\rvert + \lvert dy\rvert$; Chebyshev $\max(\lvert dx\rvert, \lvert dy\rvert)$; rotating by 45° ($u = x + y$, $v = x - y$) turns Manhattan into Chebyshev

A.3 Where the templates are

Every template named in this book is in `library/` and documented card by card in the User Guide (GUIDE.pdf, Chapter 8), with its keyboard shortcut. The one-page version is `CHEATSHEET.md`.

CHAPTER B

Template Index

Every template in the library, grouped as in `library/`, with its keyboard chord, its snippet prefix and the page of this book that explains it. Press the chord (Ctrl+Alt+T, Ctrl+Alt+Y or Ctrl+Alt+U, then the key) or type the prefix and Tab to paste it at the cursor; GUIDE.pdf has a card for each one with its usage. The table is generated from the library itself by `python tools/cp.py docs`, so it is always complete.

Basics: input, search, sorting, prefix sums, two pointers

Template	Keys	Prefix	Page
2D prefix sums	Ctrl+Alt+T, Shift+A	`cpprefix2d`	108
All permutations (next_permutation)	Ctrl+Alt+T, 9	`cpperm`	69
All subsets (bitmask loop)	Ctrl+Alt+T, 8	`cpsubsets`	67
Argsort + custom comparator	Ctrl+Alt+T, Shift+J	`cpargsort`	81
Binary search on a real answer	Ctrl+Alt+T, Shift+F	`cpbsreal`	100
Binary search on the answer	Ctrl+Alt+T, A	`cpbsearch`	97
Bit helpers	Ctrl+Alt+T, Y	`cpbits`	121
Coordinate compression	Ctrl+Alt+T, Shift+C	`cpcompress`	85
Difference array (offline range add, 1D)	Ctrl+Alt+T, 3	`cpdiff`	108
Difference array (offline rectangle add, 2D)	Ctrl+Alt+T, 4	`cpdiff2d`	109
Gray code	Ctrl+Alt+U, 0	`cpgray`	67
Intervals: merge + max disjoint (greedy)	Ctrl+Alt+T, Shift+N	`cpintervals`	9
Meet in the middle (subset sums)	Ctrl+Alt+T, Shift+U	`cpmitm`	75
Monotonic stack (next greater / previous smaller)	Ctrl+Alt+T, 7	`cpmonostack`	52
Prefix sums (1D)	Ctrl+Alt+T, 1	`cpprefix`	xv
Radix sort and counting sort	Ctrl+Alt+U, Shift+R	`cpradix`	82
Read graph (n, m, m edges)	Ctrl+Alt+T, Shift+G	`cpgraph`	135
Read vector	Ctrl+Alt+T, V	`cpread`	–
Read weighted graph (n, m, edges u v w)	Ctrl+Alt+T, Shift+E	`cpgraphw`	135
Recursive complete search (backtracking)	Ctrl+Alt+T, Shift+Y	`cpbacktrack`	71
Sliding window min / max (monotonic deque)	Ctrl+Alt+T, 6	`cpwindow`	54
Ternary search (unimodal max)	Ctrl+Alt+T, Q	`cpternary`	103
Two pointers (sliding window)	Ctrl+Alt+T, 5	`cptwoptr`	113

Graphs

Template	Keys	Prefix	Page
0-1 BFS	Ctrl+Alt+T, 0	`cp01bfs`	157
2-edge-connected components	Ctrl+Alt+Y, B	`cp2ecc`	185
2-SAT	Ctrl+Alt+Y, 2	`cp2sat`	183
Bellman-Ford (negative edges)	Ctrl+Alt+T, Shift+W	`cpbellman`	158
BFS (shortest path, unweighted)	Ctrl+Alt+T, B	`cpbfs`	137
Biconnected components (blocks)	Ctrl+Alt+Y, Shift+B	`cpbcc`	185
Bidirectional BFS (meet in the middle on shortest paths)	Ctrl+Alt+U, 2	`cpbibfs`	148
Bipartite check (2-coloring)	Ctrl+Alt+T, 2	`cpbipartite`	143
Bridges + articulation points	Ctrl+Alt+T, Shift+B	`cpbridges`	185
Cycle finding in O(1) memory (Floyd / Brent)	Ctrl+Alt+U, F	`cpcyclefind`	147
DFS (recursive, all components)	Ctrl+Alt+T, D	`cpdfs`	137
Dijkstra	Ctrl+Alt+T, J	`cpdijkstra`	153
DSU (union-find)	Ctrl+Alt+T, U	`cpdsu`	166
DSU with rollback	Ctrl+Alt+Y, R	`cpdsurb`	168
Eulerian path / circuit (Hierholzer)	Ctrl+Alt+Y, E	`cpeuler`	188
Find a directed cycle	Ctrl+Alt+T, /	`cpcycle`	146
Flood fill (grid regions)	Ctrl+Alt+T, F	`cpflood`	141
Floyd-Warshall (all pairs)	Ctrl+Alt+T, W	`cpfloyd`	161
Functional graph (successor graph) cycles	Ctrl+Alt+T, ,	`cpfuncgraph`	146
Grid BFS (shortest path on a grid)	Ctrl+Alt+T, G	`cpgridbfs`	138
K-th successor (binary lifting / jumping)	Ctrl+Alt+T, .	`cpjump`	146
Kruskal MST	Ctrl+Alt+T, K	`cpkruskal`	169
Offline dynamic connectivity (edge deletions)	Ctrl+Alt+Y, Shift+R	`cpofflinecc`	190
Prim MST (dense graphs)	Ctrl+Alt+T, Shift+5	`cpprim`	170
SCC (Tarjan)	Ctrl+Alt+T, C	`cpscc`	179
System of difference constraints	Ctrl+Alt+U, Shift+I	`cpdiffcons`	160
Topological sort (Kahn)	Ctrl+Alt+T, O	`cptopo`	175

Trees

Template	Keys	Prefix	Page
Centroid decomposition	Ctrl+Alt+Y, C	`cpcentroid`	238
DSU on tree (Sack): subtree statistics in one shared array	Ctrl+Alt+U, Shift+T	`cpsack`	240
Heavy-light decomposition	Ctrl+Alt+Y, H	`cphld`	233
Jump pointers (level ancestors with one extra pointer per vertex)	Ctrl+Alt+U, J	`cpjumpptr`	216
Kruskal reconstruction tree	Ctrl+Alt+Y, K	`cpkrt`	171
LCA (binary lifting)	Ctrl+Alt+T, L	`cplca`	216
Long-path decomposition: depth-indexed tree DP in O(n)	Ctrl+Alt+U, Shift+N	`cplongpath`	242
Offline LCA (Tarjan)	Ctrl+Alt+U, Shift+L	`cpofflinelca`	218
Prufer code (tree <-> sequence)	Ctrl+Alt+U, U	`cpprufer`	173

Template	Keys	Prefix	Page
Rerooting (DP for every root)	Ctrl+Alt+T, '	`cpreroot`	221
Small-to-large merging	Ctrl+Alt+Y, M	`cpsmall2large`	225
Tree DFS (parent, depth, subtree size, Euler tour)	Ctrl+Alt+T, E	`cptree`	208
Tree diameter	Ctrl+Alt+T, Shift+D	`cpdiameter`	210
Tree DP (subtree DP, iterative)	Ctrl+Alt+T, ;	`cptreedp`	219
Tree isomorphism (canonical IDs, AHU)	Ctrl+Alt+U, H	`cptreehash`	228
Tree knapsack (merging child DPs)	Ctrl+Alt+Y, Shift+K	`cptreeknap`	224
Virtual (auxiliary) tree	Ctrl+Alt+Y, V	`cpvirtual`	244

Data structures & range queries

Template	Keys	Prefix	Page
Anti-hack hash for unordered_map	Ctrl+Alt+T, Shift+H	`cphash`	304
Cartesian tree	Ctrl+Alt+U, C	`cpcartesian`	–
CDQ divide and conquer (3D dominance counting)	Ctrl+Alt+U, 3	`cpcdq`	–
Disjoint sparse table (O(1) queries for any associative operation)	Ctrl+Alt+U, T	`cpdisjointsparse`	–
Distinct values in ranges (offline sweep + BIT)	Ctrl+Alt+Y, Shift+D	`cpdistinct`	301
Dynamic (sparse) segment tree	Ctrl+Alt+Y, D	`cpdynseg`	300
Euler tour tree (dynamic forest: link, cut, connectivity, component sums)	Ctrl+Alt+U, E	`cpett`	246
Fenwick tree / BIT	Ctrl+Alt+T, I	`cpbit`	298
Fenwick tree 2D	Ctrl+Alt+Y, Shift+I	`cpbit2d`	–
Fenwick tree: range add + range sum	Ctrl+Alt+Y, I	`cpbitrange`	–
Implicit treap (sequence with split / merge)	Ctrl+Alt+Y, T	`cptreap`	303
Lazy segment tree (range add; range sum + min)	Ctrl+Alt+T, Shift+S	`cplazy`	299
Lazy segment tree (range assign + range add; sum, min)	Ctrl+Alt+Y, A	`cplazyassign`	299
Mo's algorithm (offline range queries)	Ctrl+Alt+Y, O	`cpmo`	297
Mo's algorithm on tree paths	Ctrl+Alt+U, O	`cpmotree`	–
Mo's algorithm with updates (time as a third dimension)	Ctrl+Alt+U, Shift+O	`cpmoupdates`	–
Ordered set (PBDS order statistics)	Ctrl+Alt+T, Shift+O	`cposet`	303
Persistent implicit treap (rope with versions)	Ctrl+Alt+U, R	`cpptreap`	–
Persistent segment tree (k-th smallest in range)	Ctrl+Alt+Y, P	`cppersist`	301
Segment tree (point update, range query)	Ctrl+Alt+T, S	`cpseg`	299
Segment tree beats (range chmin, sum, max)	Ctrl+Alt+Y, S	`cpbeats`	301
Segment tree with custom nodes + binary search	Ctrl+Alt+Y, N	`cpsegnode`	299
Segment tree: range add, min + count of min	Ctrl+Alt+Y, Shift+C	`cpmincount`	300
Sparse table (O(1) range min)	Ctrl+Alt+T, R	`cpsparse`	295
Wavelet tree (k-th smallest / count <= x in range)	Ctrl+Alt+Y, W	`cpwavelet`	304

Dynamic programming

Template	Keys	Prefix	Page
(min, +) convolution with convex sequences	Ctrl+Alt+U, Shift+C	`cpminplus`	291
Bitmask DP (travelling salesman)	Ctrl+Alt+T,]	`cptsp`	266
Broken profile DP (domino tilings)	Ctrl+Alt+Y, Shift+P	`cpprofile`	272
Digit DP	Ctrl+Alt+T, =	`cpdigit`	270
Divide & conquer DP optimization	Ctrl+Alt+Y, Q	`cpdnc`	284
Grid paths DP	Ctrl+Alt+T, Shift+Z	`cpgridpaths`	–
Knapsack (0/1, unbounded, coin-change ways)	Ctrl+Alt+T, Shift+Q	`cpknap`	257
Knuth optimization (interval DP)	Ctrl+Alt+Y, Shift+Q	`cpknuth`	286
LCS + edit distance	Ctrl+Alt+T, [	`cplcs`	260
Li Chao tree (min of lines)	Ctrl+Alt+Y, Shift+L	`cplichao`	288
LineContainer (convex hull trick, any order)	Ctrl+Alt+Y, L	`cplinecontainer`	288
Longest increasing subsequence	Ctrl+Alt+T, Shift+L	`cplis`	259
Monotone convex hull trick (sorted slopes)	Ctrl+Alt+Y, Z	`cpmonocht`	287
Monotone minima (row minima when the argmin moves right)	Ctrl+Alt+U, M	`cpmonominima`	285
Range (interval) DP	Ctrl+Alt+T, -	`cprangedp`	269
Slope trick (make an array non-decreasing)	Ctrl+Alt+Y, Shift+S	`cpslope`	289
Subset convolution	Ctrl+Alt+U, Shift+S	`cpsubsetconv`	274
Subset sums with bitset	Ctrl+Alt+T, Shift+1	`cpbitsetsum`	127
Sum over subsets (SOS DP)	Ctrl+Alt+Y, U	`cpsos`	273

Math

Template	Keys	Prefix	Page
Berlekamp-Massey (find a linear recurrence)	Ctrl+Alt+U, Shift+B	`cpbm`	–
Chinese remainder theorem (any moduli)	Ctrl+Alt+Y, 1	`cpcrt`	308
Determinant mod p, matrix-tree theorem	Ctrl+Alt+U, Shift+D	`cpdet`	–
Discrete logarithm (baby-step giant-step)	Ctrl+Alt+U, L	`cpdlog`	–
Divisor sieve (count + sum of divisors)	Ctrl+Alt+T, Shift+2	`cpdivsieve`	309
Du's sieve: prefix sums of phi and mu up to 1e10	Ctrl+Alt+U, D	`cpdusieve`	–
Euler's totient	Ctrl+Alt+Y, Shift+E	`cpphi`	308
Extended GCD + inverse for any modulus	Ctrl+Alt+T, X	`cpextgcd`	308
Factorize + divisors (trial division)	Ctrl+Alt+T, Shift+P	`cpfactor`	306
FFT (exact integer convolution)	Ctrl+Alt+Y, F	`cpfft`	315
Floor-division blocks	Ctrl+Alt+Y, Shift+F	`cpfloorblocks`	309
Grundy numbers (Sprague-Grundy)	Ctrl+Alt+Y, G	`cpgrundy`	313
Inclusion-exclusion (divisible by any)	Ctrl+Alt+Y, 3	`cpincexc`	311
Lagrange interpolation	Ctrl+Alt+U, I	`cplagrange`	–
Matrix exponentiation	Ctrl+Alt+T, Shift+X	`cpmatrix`	280
Miller-Rabin + Pollard rho (numbers up to 1e18)	Ctrl+Alt+Y, X	`cppollard`	307

Template	Keys	Prefix	Page
Mint (modular integer)	Ctrl+Alt+T, Shift+M	cpmint	307
Mobius function sieve	Ctrl+Alt+Y, Shift+M	cpmobius	309
Modular power + inverse	Ctrl+Alt+T, M	cpmodpow	307
N-th term of a linear recurrence (Kitamasa)	Ctrl+Alt+U, K	cpkitamasa	–
nCr / factorials mod p	Ctrl+Alt+T, N	cpncr	310
NTT (convolution mod 998244353)	Ctrl+Alt+Y, Shift+T	cpntt	315
Pascal triangle (nCr for any modulus)	Ctrl+Alt+Y, Shift+N	cppascal	310
Polynomial toolkit mod 998244353 (inverse, divmod, log, exp, sqrt, pow, multipoint)	Ctrl+Alt+U, Shift+P	cppoly	–
Prime counting and prime sums up to 1e11 (Lucy)	Ctrl+Alt+U, P	cpprimecount	–
Primitive root	Ctrl+Alt+U, G	cpprimroot	–
Sieve (primes + smallest prime factor)	Ctrl+Alt+T, P	cpsieve	306
Square root modulo a prime (Tonelli-Shanks)	Ctrl+Alt+U, Q	cpsqrtmod	–
Stern-Brocot: simplest fraction in an interval, best approximation	Ctrl+Alt+U, B	cpsternbrocot	–
Stirling numbers (both kinds) and Bell numbers	Ctrl+Alt+U, S	cpstirling	–
Walsh-Hadamard transform: XOR / AND / OR convolution	Ctrl+Alt+U, W	cpfwht	–
XOR basis (linear basis over GF(2))	Ctrl+Alt+Y, Shift+X	cpxorbasis	126

Strings

Template	Keys	Prefix	Page
Aho-Corasick (many patterns at once)	Ctrl+Alt+Y, Shift+A	cpaho	320
Binary trie (maximum XOR)	Ctrl+Alt+T, `	cpxortrie	126
KMP (prefix function + pattern search)	Ctrl+Alt+T, Shift+K	cpkmp	319
Lyndon factorization (Duval) and minimal rotation	Ctrl+Alt+U, Y	cplyndon	–
Manacher (all palindromes)	Ctrl+Alt+Y, 6	cpmanacher	320
Palindromic tree (eertree)	Ctrl+Alt+U, Shift+E	cpeertree	–
String hashing (mod 2^61-1)	Ctrl+Alt+T, H	cpstrhash	318
Suffix array + LCP	Ctrl+Alt+Y, 4	cpsa	321
Suffix automaton	Ctrl+Alt+Y, 5	cpsam	321
Trie	Ctrl+Alt+T, Shift+T	cptrie	320
Z-function	Ctrl+Alt+T, Z	cpz	319

Geometry

Template	Keys	Prefix	Page
Closest pair of points	Ctrl+Alt+Y, 9	cpclosest	324

Template	Keys	Prefix	Page
Convex hull (monotone chain)	Ctrl+Alt+T, Shift+V	`cphull`	324
Geometry primitives (integer points)	Ctrl+Alt+T, Shift+3	`cppoint`	323
Half-plane intersection	Ctrl+Alt+U, 4	`cphalfplane`	–
Minimum enclosing circle (Welzl, randomized)	Ctrl+Alt+U, Shift+M	`cpmincircle`	–
Polygon: area, point inside, lattice points	Ctrl+Alt+Y, 8	`cppolygon`	324
Rectangles (intersection, union of two)	Ctrl+Alt+T, Shift+4	`cprect`	131
Rotating calipers (diameter and width of a convex polygon)	Ctrl+Alt+U, 5	`cpcalipers`	–
Segment intersection	Ctrl+Alt+Y, 7	`cpseginter`	323

Flows & matching

Template	Keys	Prefix	Page
Bipartite matching (Hopcroft-Karp)	Ctrl+Alt+Y, Shift+H	`cphopcroft`	199
Bipartite minimum vertex cover / maximum independent set (Konig)	Ctrl+Alt+U, V	`cpkonig`	200
Global minimum cut (Stoer-Wagner)	Ctrl+Alt+U, Shift+A	`cpstoerwagner`	204
Gomory-Hu tree (all-pairs minimum cut)	Ctrl+Alt+U, A	`cpgomoryhu`	205
Hungarian algorithm (min-cost assignment)	Ctrl+Alt+U, Shift+H	`cphungarian`	201
Max flow (Dinic) + min cut	Ctrl+Alt+Y, Y	`cpdinic`	195
Max flow with lower bounds	Ctrl+Alt+Y, Shift+G	`cpflowlb`	203
Maximum matching in a general graph (Edmonds' blossom)	Ctrl+Alt+U, Shift+G	`cpblossom`	203
Min-cost max-flow	Ctrl+Alt+Y, Shift+Y	`cpmcmf`	202

Misc

Template	Keys	Prefix	Page
De Bruijn sequence	Ctrl+Alt+U, 9	`cpdebruijn`	129
Interactive problem helpers	Ctrl+Alt+T, Shift+I	`cpinteractive`	328
Random generator	Ctrl+Alt+T, Shift+R	`cprng`	327
Random hashing of sets and multisets (XOR / sum hashing)	Ctrl+Alt+U, X	`cpxorhash`	–
Simulated annealing skeleton	Ctrl+Alt+U, Z	`cpanneal`	–

Index

Symbols

$O(f(n))$ . . . 30
0–1 BFS . . . 140
0/1 knapsack . . . 350
2-SAT . . . 181
2-edge-connected components . . . 185

A

A* . . . 149
adjacency list . . . 134
adjacency matrix . . . 134
AHU algorithm . . . 225
alternating path . . . 198
arc . . . 133
argsort . . . 81
articulation point . . . 183
assignment problem . . . 201
augmentation . . . 57
augmenting path . . . 193

B

Backtracking . . . 69
Bellman–Ford . . . 157
Berge's lemma . . . 198
BFS . . . 345
Biconnected components . . . 185
binary heap . . . 44
Binary lifting . . . 213
binary lifting . . . 358
binary search on the answer . . . 341
binary trie . . . 124
bipartite . . . 134, 142
bitmask . . . 66
block-cut tree . . . 185
branch and bound . . . 73
Breadth-first search . . . 137
Brent's algorithm . . . 146
bridge . . . 183
bridge tree . . . 185

C

center . . . 210
centroid . . . 210, 234
centroid tree . . . 234
children . . . 207
complete search . . . 64
condensation . . . 177
connected . . . 134
connected component . . . 343
connected components . . . 134
contribution technique . . . 14
convolution . . . 315
coordinate compression . . . 84
cut . . . 195
cycle . . . 134
cycle detection . . . 63

D

DAG . . . 134
data structure . . . 50
de Bruijn sequence . . . 128
degree . . . 134
depth . . . 207
Depth-first search . . . 135
deque . . . 53
DFS tree . . . 135
diameter . . . 209
difference array . . . 108
Dijkstra . . . 346
Dijkstra's algorithm . . . 152
Dilworth's theorem . . . 201
Dinkelbach's method . . . 101
disjoint set union . . . 165, 348
dominates . . . 191
dominator tree . . . 191
DSU on tree . . . 238
DSU with rollback . . . 188

E

edge list . . . 134
edges . . . 133
Edmonds' blossom algorithm . . . 203
Euler tour . . . 208
Euler tour technique . . . 210
Euler tour tree . . . 245

Euler's formula 190
Eulerian circuit 186
Eulerian path 186

F

factorial number system 69
Fenwick tree 356
Flood fill 140
flood fill 343
flow 193
flow network 193
Floyd's cycle-finding algorithm 146
Floyd–Warshall 161
forest 134
fractional programming 101
Fracturing search 75
functional graph 146, 358

G

Gomory–Hu tree 204
Gosper's hack 68
graph 133
Gray code 67, 127
greedy algorithm 85
Grundy number 312

H

half-open range 43
heap 54
heavy child 230
heavy paths 230
Heavy-light decomposition 230
height 241
Hierholzer's algorithm 186
Hopcroft–Karp 198
Huffman's algorithm 88
Hungarian algorithm 201

I

implication graph 181
inclusion–exclusion 17
invariant 20

K

Kadane's algorithm 337
Kahn's algorithm 174
Kosaraju's algorithm 179
Kruskal reconstruction tree 171
Kruskal's algorithm 169
Kuhn's algorithm 198

L

LCP array 320
Lehmer code 69
linear basis 314
link-cut tree 191, 246
long paths 241
longest increasing subsequence 258
losing 278
low-link 177
lowest common ancestor 213

M

master theorem 34
matching 198
matroid 328
Meet in the middle 73
Memoization 252
Menger's theorem 197
minimum spanning tree 168
minimum-cost flow 202
monotone predicate 96
monovariant 21
multi-source BFS 139

N

number theoretic transform 315

O

Offline processing 19

P

Parallel binary search 104
parent 207
path 134
path cover 200
pigeonhole principle 22
postorder 208
potential 33
prefix function 318, 357
preorder 208
Prim's algorithm 170
pruning 69
Prüfer code 172

Q

queue 53

R

radix sort 81
range query 295
recursion tree 34
reflection principle 23
Regret greedy 91
relaxation 152
Rerooting 220
residual graph 193

S

shortest-path tree 152
SMAWK 285
Smith's rule 83
spanning tree 168
stack 51
STL 39
Stoer–Wagner algorithm 204
stress test 11
strict weak ordering 84
strongly connected 177
strongly connected components 177
submask 67
subtree 207
suffix array 320
suffix automaton 321
suffix links 320
sum over subsets 272
sweep 19
system of difference constraints 159

T

Tabulation 252
Top trees 247
topological order 174
treap 303
tree 134
tree DP 353
trie 319
two pointers 338
two's complement 120

U

undefined behaviour 48
unimodal 102

V

vertices 133
virtual tree 243

W

weighted DSU 167
winning 278